book 11

the icpc world finals

six finals, eight languages, first principles

Raeliosol

version 6.0, compiled 10 2026

Contents

1. the contest and the house rules 6
   1.1. what the contest is 6
   1.2. the shape of a finals 6
   1.3. the anatomy of a problem 7
   1.4. what the problems ask for 7
   1.5. the content policy 8
   1.6. the problem-chapter template 8
   1.7. the eight languages and their integers 9
   1.8. the toolbox layer 10
2. the c toolbox 12
   2.1. header-only, under the compiler's eye 12
   2.2. big integers, split in two headers 13
   2.3. the string map 16
   2.4. the min heap 18
   2.5. the ring deque 19
   2.6. the vector and the sort idiom 21
   2.7. grid index math 23
   2.8. reading input 24
   2.9. number theory with an honest product 26
   2.10. md5 from the RFC 28
   2.11. a json reader 30
   2.12. across the eight languages 33
3. the rust toolbox 35
   3.1. one file, one gate 35
   3.2. input over a locked stdin 36
   3.3. the token scanner 37
   3.4. the collections, borrowed whole 39
   3.5. index math 40
   3.6. the ordered map, not the hash map 41
   3.7. big integers, the same twelve limbs 42
   3.8. number theory with a native u128 45
   3.9. md5 and json leaves 46
   3.10. built here, cited elsewhere 48
4. the go toolbox 50
   4.1. a generic heap over container/heap 50
   4.2. reading input the scanner way 51
   4.3. coordinates on the grid 53
   4.4. the math/big wing 54
   4.5. modular arithmetic through bits 56
   4.6. digests and json leaves 58
   4.7. built here, cited elsewhere 60
5. the java toolbox 61
   5.1. fast input and output 61
   5.2. the token scanner discipline 63
   5.3. big integers, the 128-bit roads 65
   5.4. collections mapping 67
   5.5. the judge invocation and the jvm facts 69
   5.6. built here, cited elsewhere 71

6. the c# toolbox 72
6.1. input, the shapes puzzles ship in 72
6.2. the grid vector 74
6.3. the deque the bcl never shipped 76
6.4. modular arithmetic, long and biginteger 78
6.5. built here, cited elsewhere 81
7. the javascript toolbox 83
7.1. the min-heap, comparator included 83
7.2. the ring deque 85
7.3. the number kit lives on bigint 87
7.4. grid and input, plain data 89
7.5. md5 from node:crypto 91
7.6. built here, cited elsewhere 92
8. the python toolbox 94
8.1. queues, two import lines 94
8.2. memoization, three ways 96
8.3. the combinator iterators 97
8.4. numbers and line parsing 99
8.5. digests and json leaves 101
8.6. the lightest toolbox 103
9. the lua toolbox 105
9.1. modules, assertions, and the runner 105
9.2. big integers over limb tables 107
9.3. the heap with a comparator 110
9.4. the ring deque, rotating 111
9.5. the set, deterministic iteration 113
9.6. the grid 114
9.7. string utilities, concat not squares 116
9.8. number theory, wrap stated then avoided 117
9.9. md5 on 64-bit integers 119
9.10. json to plain tables 121
9.11. across the eight languages 124
10. icpc world finals 2017 126
10.1. the year-local helpers: chord geometry, matching, flow, and rank selection 126
10.2. problem A, airport construction 144
10.3. problem B, get a clue! 152
10.4. problem C, mission improbable 161
10.5. problem D, money for nothing 166
10.6. problem E, need for speed 173
10.7. problem F, posterize 177
10.8. problem G, replicate replicate rfplicbte 185
10.9. problem H, scenery 194
10.10. problem I, secret chamber at mount rushmore 202
10.11. problem J, son of pipe stream 207
10.12. problem K, tarot sham boast 217
10.13. problem L, visual python++ 223
10.14. across the seven languages 231
11. icpc world finals 2018 233
11.1. the year-local helpers: log factorials, clipped geometry, a counting fenwick, and the tour tables 233
11.2. problem A: catch the plane 243
11.3. problem B: comma sprinkler 253

11.4. problem C: conquer the world 262
11.5. problem D: gem island 271
11.6. problem E: getting a jump on crime 277
11.7. problem F: go with the flow 286
11.8. problem G: panda preserve 293
11.9. problem H: single cut of failure 303
11.10. problem I: triangles 312
11.11. problem J: uncrossed knight's tour 321
11.12. problem K: wireless is the new fiber 330
11.13. across the seven languages 339
12. icpc world finals 2019 341
12.1. the year-local helpers: a scan cursor, an order-statistics multiset, and a run treap 341
12.2. problem A, azulejos 355
12.3. problem B, beautiful bridges 363
12.4. problem C, checks post facto 371
12.5. problem D, circular dna 380
12.6. problem E, dead-end detector 388
12.7. problem F, directing rainfall 397
12.8. problem G, first of her name 406
12.9. problem H, hobson's trains 413
12.10. problem I, karel the robot 422
12.11. problem J, miniature golf 431
12.12. problem K, traffic blights 440
12.13. across the seven languages 449
13. icpc world finals 2022 451
13.1. the year-local helpers: fenwick counting, planar predicates, io cursors, and the sweep engine 451
13.2. p, turning red 459
13.3. q, doing the container shuffle 468
13.4. r, zoo management 474
13.5. s, bridging the gap 483
13.6. t, carl's vacation 491
13.7. u, toy train tracks 500
13.8. v, three kinds of dice 507
13.9. w, riddle of the sphinx 516
13.10. x, quartets 522
13.11. y, compression 532
13.12. z, archaeological recovery 536
13.13. across the seven languages 544
14. icpc world finals 2023 548
14.1. the year-local helpers: the sphinx oracle, ordered subsets, score hulls, pyramid vectors, and the failure functions 548
14.2. the go scan cursor 571
14.3. a, riddle of the sphinx 573
14.4. b, schedule 581
14.5. c, three kinds of dice 588
14.6. d, carl's vacation 598
14.7. e, a recurring problem 607
14.8. f, tilting tiles 616
14.9. g, turning red 626
14.10. h, jet lag 635
14.11. i, waterworld 643

14.12. j, bridging the gap 646
14.13. k, alea iacta est 654
14.14. across the seven languages 663
15. icpc world finals 2025 665
15.1. the year-local helpers: kuhn matchings, the rollback sweep engine, and slot wheels 665
15.2. the go scan cursor 681
15.3. problem a, a-skew-ed reasoning 682
15.4. problem b, blackboard game 692
15.5. problem c, bride of pipe stream 701
15.6. problem d, buggy rover 710
15.7. problem e, delivery service 717
15.8. problem f, herding cats 726
15.9. problem g, lava moat 734
15.10. problem h, score values 744
15.11. problem i, slot machine 751
15.12. problem j, stacking cups 760
15.13. problem k, treasure map 766
15.14. problem l, walking on sunshine 776
15.15. across the seven languages 783
16. fast big arithmetic 785
16.1. the dynamic bigint core 785
16.2. karatsuba multiplication 791
16.3. exact ntt multiplication 800
16.4. fft multiplication, plain and split 806
16.5. long division, knuth algorithm d 814
16.6. measured: four multipliers, seven languages 823
16.7. across the seven languages 825

1. the contest and the house rules

The International Collegiate Programming Contest, the ICPC, is a worldwide contest for university students, run through regional contests that feed a world finals. The finals is a five-hour competition between teams of three, and this book works six of them, 2017, 2018, 2019, 2022, 2023, and 2025, 68 problems in all, in seven languages built from first principles. The chapter you are reading is the contract for how: the shape of a finals, the anatomy of one problem, the content rules that govern everything this book says about the problem sets, the chapter template every later chapter follows, and the integer ground rules the eight languages bring with them. The eight toolbox chapters behind this one, one per language, are the shared substrate the finals chapters import and extend.

1.1. what the contest is

A finals team is three students with one machine. For five hours the team works the whole set at once, ten to thirteen problems, reading them in any order and submitting source code that a judge compiles, runs against hidden inputs, and verdicts. The verdict vocabulary is short: accepted, wrong answer, time limit exceeded, run time error. There is no partial credit and no score for style, the output has to match what the judge expects, and each problem carries its own time limit, a small number of seconds, so a program that is right but too slow scores exactly like a program that is wrong.

The only progress signal on the contest floor is balloons. Each problem has a color, and a balloon is delivered to a team when its submission for that problem is accepted, so the whole room can read who is ahead. Strategy lives under those balloons. Three people, one keyboard, and a single clock mean the hardest skill is deciding what to type next, who types it, and what the other two do while it is being typed.

1.2. the shape of a finals

The format holds constant across the six sets this book works. What moves a little is the set itself. Four sets carry 11 problems and two carry 12, 68 in all, and every problem in every set carries a letter. Five sets letter from A through K or A through L. The 2022 set runs P through Z, which is exactly how the ICPC Foundation published that cycle, and this book keeps the letters as printed, so the 2022 chapter's problem sections run P to Z. The table lists the six finals with their counts, their letters, and both pdfs the Foundation publishes for each:

finals	problems	problemset pdf	solutions pdf
2017	12, A through L	icpc2017.pdf	finals2017solutions.pdf
2018	11, A through K	icpc2018.pdf	finals2018solutions.pdf
2019	11, A through K	icpc2019.pdf	finals2019solutions.pdf
2022	11, P through Z	icpc2022.pdf	finals2022solutions.pdf
2023	11, A through K	icpc2023.pdf	finals2023solutions.pdf
2025	12, A through L	problemset.pdf	2025-icpc-solutions.pdf

Both pdfs were accessed 2026-09-14 from icpc.global and worldfinals.icpc.global, and both are cached under `ref/icpc/` with sha256 digests and byte counts in `ref/icpc/INDEX.md`, so every claim the finals chapters make is checked against a local copy of the exact bytes. The problems pdf is the statement of record: letter, title, story, input and output contracts, constraints, samples, time limit. The solutions pdf is the Foundation's own solution sketch set, one short editorial per problem from the problem setters, and it is the algorithm reference for this book.

A set spreads wide. The 2017 sketch counts one problem accepted by 127 teams and one accepted by none. The 2025 sketch counts four problems accepted by 135 teams or more and one accepted by none. That spread is the design: the first hours belong to the problems most of the room solves, the middle band separates the field, and the last problems decide the title. Working every letter, the solved ones and the unsolved ones, is what makes six sets fill six chapters.

Ranking is arithmetic, and the arithmetic shapes the endgame. A team is ranked by problems solved first, and ties break on total time: for each solved problem, the minute the accepted submission landed plus a penalty for every earlier wrong submission on that problem, summed. A wrong submission on a problem the team never solves costs nothing, which is why teams gamble on hard letters in the last hour, and a wrong submission on a problem the team later solves is minutes on the score for good.

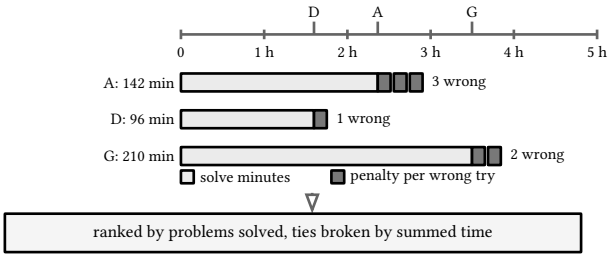


figure 1: how the scoreboard sees one team: solve minutes on the clock plus a penalty block per wrong submission, summed over the solved problems

1.3. the anatomy of a problem

Every problem in every set arrives in the same shape, and the shape is a contract. A letter and a title identify the problem. A story sets the scene, a paragraph or two of prose in the world of the problem. The task states once, precisely, the quantity to compute. The input section names every token the program will read and bounds every value, the output section says exactly what to print, and one or more sample pairs close the statement, small worked inputs with their expected outputs. The time limit rides with the letter.

The bounds deserve their own sentence because they carry the algorithmic content of the statement. A bound of a few hundred items invites exhaustive search, a bound in the hundreds of thousands invites something linear or near-linear in it, and the distance between those two is the distance between accepted and time limit exceeded. Reading the constraints before the story is the first habit of a finals solver, and every walkthrough in this book states the bounds it solved against.

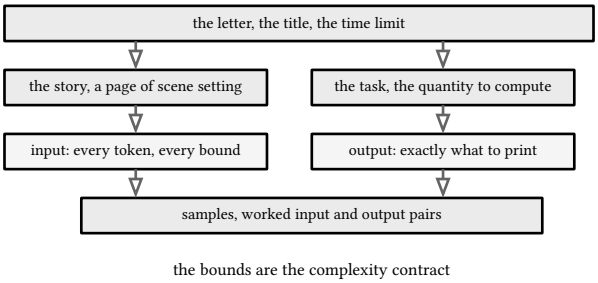


figure 2: the anatomy every problem carries: letter and title, story, task, input with its bounds, output, samples, time limit

The samples are the one part of a statement a solver needs verbatim, so this book carries them. Every problem section reprints at least one official sample pair exactly as the Foundation printed it, and the reprint is checked byte for byte against the cached judge data, so the pair on the page is the pair the judge grades. The story around it stays paraphrase, and the suites still pin crafted fixtures of their own with hand-computed answers, because a fixture whose answer is derived beside it is the teaching artifact. The sample pair is the running target: feed it to any solver in this book and the output matches the printed answer.

1.4. what the problems ask for

Six sets, 68 problems, and the same families keep returning. Every problem starts as input to parse, line lists, token rows, character grids, blank-line groups, which is why every toolbox has an input module that reads lines, a string module or byte-level idiom that splits and trims, and grid modules that carry the r times cols plus c index law with the 4- and 8-neighbor tables ready. Past the parsing, counting problems want a dictionary and a set, search

problems want a heap and a deque, reachability wants walks and flood fills, and the arithmetic problems want gcd and modular exponentiation or a big integer that survives the count.

The sets also reach past the toolbox. World finals problems ask for geometry, network flows, game trees, and reductions the shared modules do not carry, and the house rule for those is year-local: such helpers are built inside the year chapter with a one-line spec in the year's suite. That boundary is deliberate. The toolbox holds what all eight languages need across all six years, and everything specific to one problem lives with that problem.

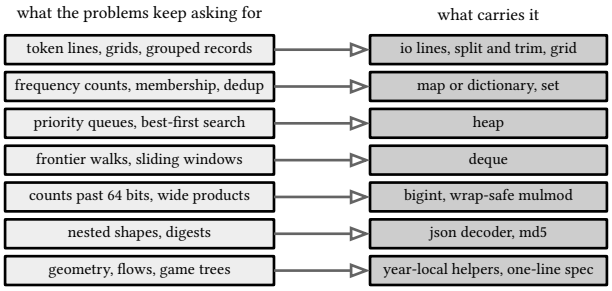


figure 3: the recurring problem families on the left, the toolbox that carries each on the right, year-local where no shared module exists

None of this solves a problem. Knowing a problem wants a heap does not say what the heap should order by, and the hard third of a finals set is hard in the model or the proof first. What the toolbox removes is everything around the problem: parsing that works, containers already tested, arithmetic that cannot wrap. The finals chapters spend their length on the problem and import the rest.

1.5. the content policy

The problems are not this book's to copy. The rulebook the whole book is written under states the policy, and it is restated here because every chapter that mentions a problem is bound by it:

PITFALL: the house rules

Statements are retold in this book's own words and linked to the year's problemset pdf at icpc.global, and the retelling is complete: task, input and output contracts, every bound, the time limit, and at least one official sample pair reprinted exactly. The story prose is never copied. The problemset and solutions pdfs are cached under ref/icpc for private reference, sha256 digests in ref/icpc/INDEX.md. Judge data under ref/icpc//data/ is local only and never committed: its sample files are the byte-exact source for the sample pairs this book prints, and its secret inputs are never quoted. Sources rows attribute the ICPC Foundation and icpc.global with access dates. The official solutions pdf is the algorithm reference for every method and complexity claim, and it is cited where used. The 2025 set postdates training data, so everything this book says about it derives strictly from the cached pdfs.

Each clause has a mechanical consequence in this book, and it is enforced by two gates: every suite pins its solver to crafted fixtures with hand-computed answers, and the committed `ref/icpc/verify-data.ps1` maps every problem letter to its input and answer pairs under the local judge data in `ref/icpc/<year>/data/`, runs the solvers against them, and reports counts.

1.6. the problem-chapter template

Every finals chapter follows the same shape, so a reader who has seen one can navigate all of them. One section per problem, in the letter order of the set, P through Z in the 2022 chapter exactly as the pdf prints them. A section opens with the task in this book's own words and the links, then states the input and output contracts: every token the program reads, every bound on every value, the time limit. An official sample pair follows, reprinted exactly and attributed. Before the derivation comes recognition: the bounds read as a complexity budget, the statement's shape names the structure, the family names the siblings in other years, and the rejected alternatives are priced

out. The derivation comes next, the method built up with the solutions sketch cited. The model is then traced by hand on the official sample before any code is printed, as a figure when the run is visual and a numbered state table when it is tabular, ending at the byte-exact printed answer. After the trace the solver walks in seven listings, one per language, in the fixed order c, go, java, c#, javascript, python, lua. That order is book 10′s order with java slotted after go, the data structures book this series pairs with, and keeping it means a reader moving between the two books always knows where a language's version of a structure lives. A helper section opens the chapter when year-local kits exist, and every file a printed solver imports is itself printed there, so the reader never leaves the book to follow an import. Each problem section carries its worked run as a figure or state table before the listings and one diagram of the method after them. Every walkthrough pins its answer: the official sample above and the crafted fixture beside it, the hand-computed expectation derived in the prose, and the suite assertion that ties them. A closing section compares the seven languages on that set, and a sources paragraph carries the attribution, the urls, the access date, the solutions cite, and the suite counts.

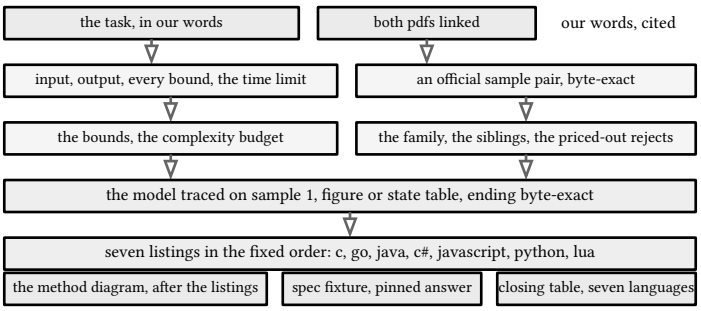


figure 4: the problem-chapter template: the task in our words, the contracts and bounds, an official sample pair, recognition, the worked run, seven listings in the fixed order, pinned answers, the closing table

The fixed order is also a promise about interchangeability. The seven languages do not solve each problem identically, integer semantics differ, container choices differ, and the walkthroughs say so out loud. When a cross-language anchor exists, the same hand-computed value must come out everywhere: 2^100 mod 1e9+7 landing at 976371285, the 92 eight-queens solutions, the failure table of ababa. When fixtures differ between languages, the walkthrough cites its own fixture's answer and the prose never claims the outputs match. That honesty rule holds for the toolbox chapters too.

Behind every chapter stands the judge harness, run locally. It pipes each official input to the solver and diffs the official answer, and the chapter reports those runs as counts. The sample inputs and answers are public statement data and this book prints them, checked byte for byte against the cache; the secret inputs and answers stay on the machine, so the judge data stays a private second gate.

1.7. the eight languages and their integers

The eight languages were not chosen for this book. They are the series: seven language manuals and C, the same eight that book 10 rebuilt every data structure in, in the same order. What differs between them, and what every toolbox chapter has to say out loud, is what happens when a product outgrows the machine word, and a finals set asks for that often enough: combinatorial counts past 64 bits, coordinate arithmetic that overflows mid-computation, answers with dozens of digits. The rules below are binding for the whole book and each toolbox chapter repeats the ones it lives under.

language	machine word	big integers	wrap-safe product
c	`unsigned long long`	toolbox bigint, 12 limbs of 9 digits	add-and-double with a remainder every step
rust	`u64`, `usize` indices	toolbox bigint, the same 12 limbs	one native `u128` expression, widened and reduced
go	`uint64` / `int64`	`math/big` where needed	`math/bits.Mul64` for the wide product

language	machine word	big integers	wrap-safe product
java	`long`	`BigInteger` at the c `__int128` sites	`Math.multiplyHigh` plus the shift-subtract lane
c#	`long`	`System.Numerics.BigInteger`	checked 64-bit, checked bigint
javascript	`Number`	`BigInt` past the safe 53-bit range	`BigInt` at the boundary, stated in a comment
python	native `int`	unbounded, no ceremony	native
lua	int64, wraps silently	toolbox bigint over tables	doubling mulmod, boundary test pinned

Two of those rows carry a story this book tells in full later. C's `__uint128_t` multiplies fine under clang 23 on this machine, but the link then fails for want of `__umodti3`, the runtime helper for a 128-bit remainder, so the C toolbox and every C modular routine use add-and-double instead, and book 11, ch 2 (the icpc world finals, the c toolbox) shows the whole decision. Rust on the same machine never meets that wall: `u128` is a first-class type there, the wide product and its remainder are one expression, and book 11, ch 3 (the icpc world finals, the rust toolbox) spends the saved length on the boundaries rust refuses to hide, the wrapping sites said out loud and the checked adds that refuse to overflow silently. Lua's integers wrap without a sound, and the Lua toolbox pins the proof in a test, `math.maxinteger * 2 == -2`, then builds its mulmod on doubling so no wrap can happen, and book 11, ch 9 (the icpc world finals, the lua toolbox) walks it. Python, C#, and Java sit at the other pole, where big integers are native and the chapter's job is to show what the machine is doing for you. The row for javascript is the one that bites contest solvers most often, because `Number` arithmetic that is fine in the small example loses precision on the judge input's magnitudes with no error raised, so the boundary is stated in a comment at every place a toolbox crosses it.

1.8. the toolbox layer

The eight chapters between this one and the finals chapters are toolboxes, one per language: book 11, ch 2 (the icpc world finals, the c toolbox), book 11, ch 3 (the icpc world finals, the rust toolbox), book 11, ch 4 (the icpc world finals, the go toolbox), book 11, ch 5 (the icpc world finals, the java toolbox), book 11, ch 6 (the icpc world finals, the c# toolbox), book 11, ch 7 (the icpc world finals, the javascript toolbox), book 11, ch 8 (the icpc world finals, the python toolbox), and book 11, ch 9 (the icpc world finals, the lua toolbox). Each is a small from-scratch library covering what the problem sets keep asking for: big integers beyond the machine word, a string-to-value dictionary and a set, a binary min-heap, a deque, grid index math with neighbor tables, input reading, modular arithmetic with a multiplication that cannot wrap, md5 from the RFC, and a json decoder. The finals chapters import and extend these. They never re-derive a container inline, because a chapter about a hard search problem should spend its length on the search, and because a container built once and tested once beats eight private re-derivations with eight private bugs.

The measured size of the layer is part of the story. The C toolbox is 11 header files, 1125 lines, in `books/icpc/samples-c/src/Ch02/`, tested by 10 programs with 251 internal checks. The Rust toolbox is 11 files, 1362 non-blank lines, in `books/icpc/samples-rust/src/Ch03/`, with 278 checks through its runner. The Lua toolbox is 9 modules, 1088 lines, in `books/icpc/samples-lua/`, with 45 checks through its runner. The other five languages carry their own chapters and their own counts. Those numbers are counted from the tree, not estimated, raw line totals for C and Lua and non-blank lines for the other six, and the chapter that owns each suite restates its own.

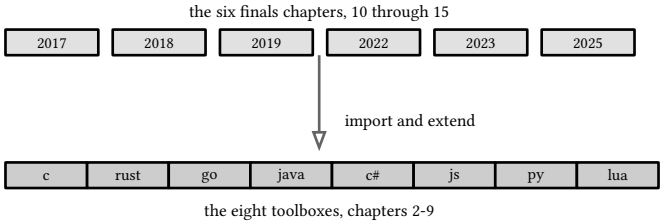


figure 5: the layering: finals chapters import the toolbox of their language, never re-derive containers

The test discipline is the same across all eight, and it is the same discipline book 10′s suites run under. Tests are table-driven over crafted fixtures with hand-computed answers. No network, no clock, no randomness unless it is seeded deterministically. Brute-force cross-checks are welcome and counted, because a clever method that agrees with a naive one on a small fixture is worth more than either alone. Every language's suite prints or asserts so its gate can count: the C gate compiles every module under `-Werror` and counts one ok line per program, the Rust gate does the same with rustc per file under `-D warnings`, the Lua runner counts checks per module, and the managed-language suites run their native test frameworks. The footer of every toolbox chapter states its own verified counts.

Crafted fixtures with hand-computed answers are the committed gate, and the judge data is the second one, run locally forever, not just for the toolboxes but for every finals chapter that follows. The policy above is the reason, and the practice also keeps the suites fast, deterministic, and offline. When a walkthrough needs a wide product or a large count to make its point, it builds a fixture whose answer is derived in the prose beside it, and the reader can check the arithmetic by hand.

sources: ICPC Foundation, icpc.global and worldfinals.icpc.global, the six problemset and solutions pdfs linked in the table above, accessed 2026-09-14 and cached under `ref/icpc/` with sha256 digests in `ref/icpc/INDEX.md`. The solution sketches settle the algorithm and complexity claims made here, and the 2025 set postdates training data, so everything said about it derives from the cached pdfs. This chapter has no sample suite. The suites begin with chapter 2, whose gate line is 10 files and 251 internal checks, and chapter 9, whose runner reports 45 checks across 9 modules.

2. the c toolbox

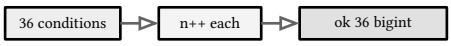
C is where the year chapters pay the highest price for convenience, so the C toolbox pays it once. Eleven headers live in `books/icpc/samples-c/src/Ch02/`, every function `static inline`, no translation units, no build file: a year chapter includes what it needs and compiles one file. Each module ships with a `_test.c` program that runs crafted fixtures with hand-computed answers, and the gate compiles all of them under `-Werror` with the format and address sanitizer legs on top. Ten programs, 251 internal checks, 1125 lines of header. This chapter walks each module with its test, and where a design decision has a story, the story is told here once so the year chapters can just cite it. The Lua twin of this chapter is book 11, ch 9 (the icpc world finals, the lua toolbox), and the two cross-cite freely: same structures, different machines underneath.

2.1. header-only, under the compiler's eye

Every declaration is `static inline` in a header guarded by an `#ifndef`. That choice keeps the toolbox trivial to import, one include line per module, and it also means every including translation unit sees every function body, so the compiler inlines what is small and warns about what is unused or wrong at the call site, and `-Werror` turns every warning into a build failure. The ucrt adds a twist the tests have to live with: the platform marks the unbounded byte helpers, `strcpy`, `strcat`, the `fopen` family, as deprecated, and under `-Werror` a deprecation attribute is fatal. The toolbox does not silence that globally. It does two narrower things: `io.h` defines `_CRT_SECURE_NO_WARNINGS` before any standard header lands, because that module genuinely wants the portable `fopen`, and the modules that copy bytes, `map.h` and `json.h`, carry their own bounded copy loops so no deprecated call exists to warn about. The test harness itself is one macro used by all ten programs.

The dry run: `bigint_test.c`'s 36 checks are the count the chapter's close pins for the module, and the walk is the ledger of how `n` climbs to the line that prints it.

1. The opening block pins one parse seven ways: the parse itself, the round-trip string, `a.n` = 3, the limbs 567890123, 678901234, 12345 from least to most significant, and sign 1; `n` stands at 7.
2. The carry fixture adds 3, parse, printed sum, and limb count, `n` at 10.
3. Plain `123 + 456` and `456 - 123` add 2 string compares, `n` at 12.
4. The signed group adds 6, the `-100` parse and its sign, two `-60` prints, the `0` print, and the positive zero sign, `n` at 18.
5. Comparisons add 7 over `bi_cmp`, `n` at 25.
6. Multiply adds 5, two parses per product and both string compares, `n` at 30.
7. Small division adds 3, parse, quotient string, remainder, `n` at 33.
8. The three garbage refusals close the file, `n` at 36, and the final `printf` prints `ok 36 bigint`.



the opening block alone is 7 checks over one 23-digit parse

nothing else prints, the count rides to the end silently

figure 6: one program under the check macro: every true condition bumps the count, the final printf carries it

The count lands whole in `ok 36 bigint`, and the macro doing the counting is the listing below.

```c
static int n = 0;

#define CHECK(c)                                                    \
  do {                                                              \
    if (!(c)) {                                                     \
      fprintf(stderr, "FAIL %s:%d\n", __FILE__, __LINE__);          \
      return 1;                                                     \
    }                                                               \
    n++;                                                            \
  } while (0)
```

listing 1: c: the check macro every module test is built on, counting as it passes (source: icpc/samples-c/src/Ch02/bigint_test.c, lines 9-18)

A failed check prints the file and line and returns nonzero, the runner reports one ok line per program, and the internal count rides in the `n` that the final `printf` carries. The assertions are the documentation: a fixture like `4 9 1 7 3 8 6 5 2` pushed into the heap and popped back as `1` through `9` says more about the heap than a paragraph would.

2.2. big integers, split in two headers

The finals hand out numbers past 64 bits often enough that the toolbox carries its own bigint rather than reaching for anything external. The representation is sign-magnitude over base one billion limbs, least significant limb first, twelve limbs, 108 decimal digits of headroom. Zero is a magnitude with the positive sign, and the sign rule is enforced at every exit: zero never carries a minus. The arithmetic core, compare, add, subtract, multiply, small division, lives in `bigint.h`, and parsing and printing live in `bigint_io.h`. The split is deliberate: a year chapter that only needs arithmetic on values it built from `unsigned long long` includes one header, and the parse and print half, with its 25-bytes-per-limb buffer contract, stays out of the way.

The dry run: the module's carry fixture, `9999999999999999999 + 1` in `bigint_test.c`, asserts the printed sum and the limb count.

1. `bi_parse` cuts the twenty nines into three limbs from the right: limb 0 = 999999999, limb 1 = 999999999, limb 2 = 99, sign +1, `a.n` = 3.
2. `bi_from_u64(&b, 1)` is the single limb 1.
3. The magnitude add folds limb 0: 999999999 + 1 = 1000000000, one full base, so the limb records 1000000000 - 1000000000 = 0 and carries 1.
4. Limb 1 the same: 999999999 + 1 = 1000000000, limb 1 records 0, carry 1.
5. Limb 2: 99 + 1 = 100, under the base, no carry, and `r.n` stays 3.
6. `bi_to_string` prints the top limb 100 bare and pads the two below to nine digits, 3 + 9 + 9 = 21 characters concatenating to `100000000000000000000`, the pinned string.

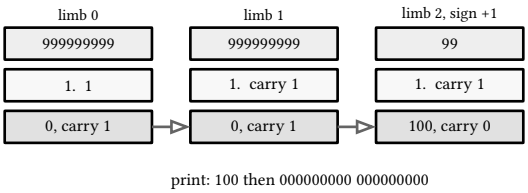


figure 7: the carry chain across all three limbs: each full base records a zero limb and passes one up, the top limb absorbs the last

The three limbs print back as the pinned `100000000000000000000`, and the listings below carry the representation, the add, and the parse that produce it.

```c
typedef struct {
  int sign; // -1 or +1; zero always carries +1
  int n;    // limb count, 1..BI_LIMBS, no high zero limbs except "0"
  unsigned int limb[BI_LIMBS];
} BigInt;

// zero
static inline void bi_zero(BigInt *r) {
  r->sign = 1;
  r->n = 1;
  for (int i = 0; i < BI_LIMBS; i++)
    r->limb[i] = 0;
}

static inline void bi_trim(BigInt *r) {
  while (r->n > 1 && r->limb[r->n - 1] == 0)
    r->n--;
  if (r->n == 1 && r->limb[0] == 0)
    r->sign = 1;
}
```

listing 2: c: the bigint struct, sign-magnitude over 12 base-1e9 limbs, and the trim that keeps zero positive (source: icpc/samples-c/src/Ch02/bigint.h, lines 11-30)

Comparison is magnitude-first, limb count then limb values from the top, and the signed wrapper only flips the sense when both signs are negative. Addition of same signs is the magnitude add with the shared sign, opposite signs route through the magnitude subtract with the larger operand's sign, and subtraction is addition of a negated copy. The carry past the twelfth limb is dropped on the floor, documented, and the tests stay inside the buffer. Multiply is schoolbook, limb times limb accumulated into a `unsigned long long` column array, then one carry pass. A limb product is at most the square of 999999999, just under a quintillion, and the accumulator survives several of those plus a carry inside 64 bits, which is exactly why the base is a billion and not a trillion.

```c
// schoolbook multiply; the product must fit BI_LIMBS
static inline void bi_mul(BigInt *r, const BigInt *a, const BigInt *b) {
  unsigned long long acc[BI_LIMBS] = {0};
  for (int i = 0; i < a->n; i++)
    for (int j = 0; j < b->n && i + j < BI_LIMBS; j++)
      acc[i + j] += (unsigned long long)a->limb[i] * b->limb[j];
  unsigned long long carry = 0;
  int n = 0;
  for (int i = 0; i < BI_LIMBS; i++) {
    unsigned long long s = acc[i] + carry;
    r->limb[n++] = (unsigned int)(s % BI_BASE);
    carry = s / BI_BASE;
  }
  while (carry) // overflow beyond the buffer: keep only the low limbs
    carry /= BI_BASE;
  r->n = n;
  r->sign = a->sign * b->sign;
  bi_trim(r);
}
```

listing 3: c: schoolbook multiply into a 64-bit column accumulator, one carry pass at the end (source: icpc/samples-c/src/Ch02/bigint.h, lines 136-154)

Parsing cuts the decimal string into limbs of nine digits from the right, so `123456789012345678901 23` parses to three limbs, 12345, 678901234, 567890123 from most to least significant, and the test pins that exact layout. Printing is

the reverse: the top limb bare, every limb below it nine digits wide with `%09u`, so the zero padding is what makes the concatenation a correct decimal. The pins are the language-neutral anchors of this toolbox: `123456789 * 987654321` lands at `121932631112635269`, crossing out of one limb, and `10^21` divided by 7 leaves quotient `142857142857142857` and remainder 6.

```c
static inline int bi_parse(BigInt *r, const char *s) {
  bi_zero(r);
  int sign = 1;
  if (*s == '-') {
    sign = -1;
    s++;
  } else if (*s == '+') {
    s++;
  }
  if (!*s)
    return 0;
  // collect the digit string, then cut limbs of nine from the right
  char digits[128];
  int nd = 0;
  for (; *s; s++) {
    if (*s < '0' || *s > '9' || nd ≥ 128)
      return 0;
    digits[nd++] = *s;
  }
  r->n = 0;
  for (int pos = nd; pos > 0 && r->n < BI_LIMBS;) {
    int start = pos - 9 > 0 ? pos - 9 : 0;
    unsigned int limb = 0;
    for (int i = start; i < pos; i++)
      limb = limb * 10u + (unsigned int)(digits[i] - '0');
    r->limb[r->n++] = limb;
    pos = start;
  }
  r->sign = sign;
  bi_trim(r);
  return 1;
}

// write the decimal form into buf (at least BI_LIMBS * 10 + 2 bytes);
// returns buf
static inline char *bi_to_string(const BigInt *a, char *buf) {
  char *p = buf;
  if (a->sign < 0 && !(a->n == 1 && a->limb[0] == 0))
    *p++ = '-';
  p += sprintf(p, "%u", a->limb[a->n - 1]);
  for (int i = a->n - 2; i ≥ 0; i--)
    p += sprintf(p, "%09u", a->limb[i]);
  *p = 0;
  return buf;
}
```

listing 4: c: parse by cutting limbs of nine from the right, print with the nine-digit zero pad (source: icpc/samples-c/src/Ch02/bigint_io.h, lines 9-53)

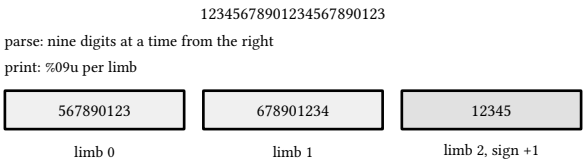


figure 8: the 23-digit fixture as three limbs, least significant first, and the nine-digit print width that reassembles it

Measured against the Lua twin: the arithmetic core plus io is 171 + 55 lines here against one 221-line Lua module that also embeds its tests, 36 internal checks against 7 Lua test blocks, and the same anchors, the 121932631112635269 product and the remainder-6 division, hold in both languages. Lua's version keeps limbs in a plain table and leans on the fact that a billion-base limb product stays under the 62-bit wrap line, which is true on Lua's 64-bit integers and would be false with a trillion base.

2.3. the string map

The map is open addressing, strings to `long long`, with linear probing over a power-of-two capacity. The hash is FNV-1a 64, offset basis 14695981039346656037, prime 1099511628211, the constants from the Fowler, Noll, Vo reference. Keys are copied into fixed 24-byte cells with the bounded copy loop from the introduction, which is what keeps the module free of the ucrt's deprecated calls. Deletion writes a tombstone rather than an empty marker, because a probe walk that stops at a hole would walk past a deleted slot and lose keys that probed through it.

The dry run: the suite's opening block in `map_test.c` walks the basic contract, three keys in, every get back, an overwrite that leaves `used` pinned at 3.

1. A fresh 16-slot map takes `alpha` 1, `beta` 2, `gamma` 3: `used` = 3, `tombs` = 0.
2. The gets return with the values 1, 2, and 3, and `delta`, never inserted, misses with `map_get` returning 0.
3. `map_put("alpha", 10)` walks the probe to the live key and overwrites in place: the next get returns 10 and `used` still reads 3, the asserted balance.
4. The load check reads live plus tombstones, 3 + 0 = 3, against 12, three quarters of the 16 slots, so no rehash fires anywhere in the block.

op	used	tombs	the assert reads
3 puts	3	0	gets return 1, 2, 3
get delta	3	0	miss, returns 0
put alpha 10	3	0	get returns 10, count unchanged
load check	3	0	3 + 0 against 12 of 16

An overwrite never grows the table, the live count pinned at 3 on both sides of it, and the listings below are the hash and the put that keep it.

```c
static inline unsigned long long map_fnv1a(const char *s) {
  unsigned long long h = 14695981039346656037ULL;
  while (*s) {
    h ^= (unsigned char)*s++;
    h *= 1099511628211ULL;
  }
  return h;
}

// bounded byte copy: the platform headers deprecate the unbounded ones
static inline void map_copy_key(char *dst, const char *src) {
  int i = 0;
  for (; i < MAP_KEY_MAX - 1 && src[i]; i++)
    dst[i] = src[i];
  dst[i] = 0;
}
```

listing 5: c: fnv-1a 64 and the bounded key copy, no deprecated byte helpers anywhere (source: icpc/samples-c/src/Ch02/map.h, lines 26-41)

Insertion asks the probe walk for two things at once, the live key if it exists or the first tombstone seen on the way, and reinsertion into a tombstone consumes it: the test deletes `beta`, watches `used` fall and `tombs` rise, reinserts `beta`, and asserts the tombstone count is back to zero. The load check counts live entries plus tombstones against three quarters of capacity, so a delete-heavy workload still triggers the rehash, and growth doubles the capacity and rehashes only the live entries, which clears the tombstones for free.

```c
static inline void map_put(StrMap *m, const char *key, long long val) {
  MapSlot *s = map_find_slot(m, key, 1);
  if (s) {
    if (s->state == MAP_TOMB)
      m->tombs--;
    else
      m->used--; // overwrite path balances the increment below
    map_copy_key(s->key, key);
    s->state = MAP_USED;
    s->val = val;
    m->used++;
    return;
  }
  // no existing key and no tombstone: claim the first empty slot
  unsigned long long i = map_fnv1a(key) & (m->cap - 1);
  while (m->slots[i].state == MAP_USED)
    i = (i + 1) & (m->cap - 1);
  MapSlot *t = &m->slots[i];
  map_copy_key(t->key, key);
  t->state = MAP_USED;
  t->val = val;
  m->used++;
  if (m->used + m->tombs > m->cap * 3 / 4)
    map_grow(m);
}
```

listing 6: c: put with the overwrite balance, tombstone reuse, and the 75 percent growth trigger (source: icpc/samples-c/src/Ch02/map.h, lines 86-110)

The growth fixture is 200 keys with square values inserted from a 16-slot start: the capacity ends at 256 or more, four doublings, all 200 lookups return, `key42` holds exactly 1764, deleting the hundred even keys leaves every

odd value intact. Lua needs no map module at all, its tables are the primitive, and that asymmetry is the honest comparison: C spends 130 lines buying what Lua was born with.

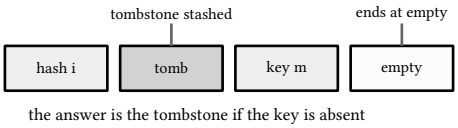


figure 9: one probe walk serving both find and reinsert: the first tombstone is remembered, the walk only ends at a true empty

2.4. the min heap

The heap is an array of key and value pairs ordered by the `unsigned long long` key, doubling its allocation when full. Push sifts up while the parent's key is larger, pop moves the last element to the root and sifts down into the smaller child. Values travel with keys, so a year chapter can push distances with node ids attached and get both back in order. Nothing here is stable by contract, but equal keys come out in insertion order on the fixtures because the sift compares strictly, and the test says so out loud for the two 9s that precede the 10.

The dry run: the suite's first block in `heap_test.c` pushes 5, 1, 3, then 2 and 4, and asserts every pop plus the peeks between.

1. Pushing 5 leaves `[5]`; pushing 1 appends then sifts once, the 5 and 1 swap, `[1, 5]`; pushing 3 finds its parent 1 already smaller, `[1, 5, 3]`, and the asserts read size 3, peek key 1.
2. Pushing 2 sifts past the 5 to `[1, 2, 3, 5]`; pushing 4 finds its parent 2 smaller and stays appended, `[1, 2, 3, 5, 4]`.
3. The first pop returns 1: the last element 4 takes the root, the smaller child 2 swaps up, and 4 settles over the 5, `[2, 4, 3, 5]`.
4. The second pop returns 2: the 5 takes the root, swaps once with the smaller child 3, and rests beside the 4, `[3, 4, 5]`, and the peek assert reads 3.
5. Pops return 3 leaving `[4, 5]`, then 4 leaving `[5]`, then 5, and `heap_empty` reads true.

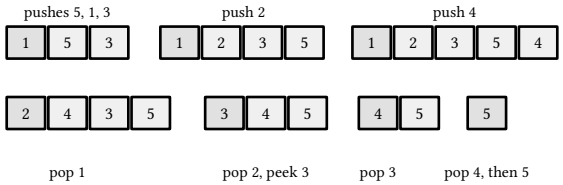


figure 10: the array after each operation, root shaded, the pops draining it in the asserted order

The five pops land 1, 2, 3, 4, 5 exactly as asserted, and the listing below is the sift pair that produces them.

```c
static inline void heap_push(MinHeap *h, unsigned long long key,
                             long long value) {
  if (h->size == h->cap) {
    h->cap *= 2;
    h->items = realloc(h->items, h->cap * sizeof(HeapItem));
  }
  unsigned long long i = h->size++;
  h->items[i].key = key;
  h->items[i].value = value;
  while (i > 0) {
    unsigned long long p = (i - 1) / 2;
    if (h->items[p].key <= h->items[i].key)
      break;
    heap_swap(&h->items[p], &h->items[i]);
    i = p;
  }
}

static inline HeapItem heap_peek(const MinHeap *h) { return h->items[0]; }

static inline HeapItem heap_pop(MinHeap *h) {
  HeapItem top = h->items[0];
  h->items[0] = h->items[--h->size];
  unsigned long long i = 0;
  for (;;) {
    unsigned long long l = 2 * i + 1, r = 2 * i + 2, m = i;
    if (l < h->size && h->items[l].key < h->items[m].key)
      m = l;
    if (r < h->size && h->items[r].key < h->items[m].key)
      m = r;
    if (m == i)
      break;
    heap_swap(&h->items[i], &h->items[m]);
    i = m;
  }
  return top;
}
```

listing 7: c: sift up on push, smaller-child sift down on pop, one struct per element (source: icpc/samples-c/src/Ch02/heap.h, lines 39-75)

The fixture suite is 27 checks: shuffled pushes popped ascending, values tracked through both operations, duplicates, interleaved push and pop holding the minimum on top, a heapsort pass over `4 9 1 7 3 8 6 5 2` that pops 1 through 9 with each value ten times its key, and a growth run to 40 entries from capacity 8. The Go twin makes the same structure generic with `Heap[T]`, and the Lua twin takes a comparator so one heap serves min and max order, 91 lines against this 77-line header, and all three pop the same fixtures in order.

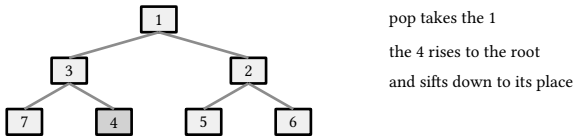


figure 11: one pop, drawn on the heapsort fixture: the last element takes the root and sifts down three times

2.5. the ring deque

The deque is a ring buffer of `long long` with a head index and a count: the back writes at `(head + count) % cap`, the front walks the head backward, and both pops refuse on empty by returning 0 with an out parameter, no

exceptions in C, so refusal is a return value and the test asserts it on both ends. Growth doubles the capacity and rebases: the new buffer is written from logical index zero, so a deque wrapped across the seam lands contiguous again with head at slot 0.

The dry run: the suite's wrap fixture in `deque_test.c` pushes 1, 2, 3 to the back, then 9 and 8 to the front, and asserts the pops read 8, 9, 1.

1. Three back pushes land at slots 0, 1, and 2 of the cap 8 ring, head 0, count 3.
2. `deque_push_front(9)` walks the head to (0 + 8 - 1) % 8 = 7 and writes slot 7, count 4.
3. `deque_push_front(8)` walks again to (7 + 8 - 1) % 8 = 6 and writes slot 6, count 5.
4. The pops read slot 6 then slot 7, returning 8 then 9, the head arriving back at 0.
5. The third pop reads slot 0, the 1, the asserted order across the seam.

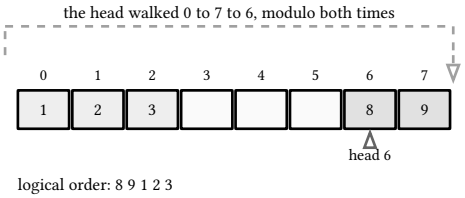


figure 12: the cap 8 ring after both front pushes: the head walked backward past slot 0, the pops read 8, 9, 1

The pops cross the seam in the asserted order, 8, 9, 1, and the listing below is the modulo pair that writes them.

```c
static inline void deque_grow(Deque *d) {
  unsigned long long ncap = d->cap * 2;
  long long *nbuf = malloc(ncap * sizeof(long long));
  for (unsigned long long i = 0; i < d->count; i++)
    nbuf[i] = d->buf[(d->head + i) % d->cap];
  free(d->buf);
  d->buf = nbuf;
  d->cap = ncap;
  d->head = 0;
}

static inline void deque_push_back(Deque *d, long long v) {
  if (d->count == d->cap)
    deque_grow(d);
  d->buf[(d->head + d->count) % d->cap] = v;
  d->count++;
}

static inline void deque_push_front(Deque *d, long long v) {
  if (d->count == d->cap)
    deque_grow(d);
  d->head = (d->head + d->cap - 1) % d->cap;
  d->buf[d->head] = v;
  d->count++;
}
```

listing 8: c: doubling growth rebases the ring to slot 0, both pushes wrap by modulo (source: icpc/samples-c/src/Ch02/deque.h, lines 27-51)

The growth fixture starts at capacity 2 and interleaves 50 pushes at each end, and the drain asserts the whole logical order in one pass, `-50` through `-1` then `1` through `50`, 100 elements that crossed six doublings with their order intact. A full rotation, pop front and push back six times over six elements, restores the original order, which is the property the rotate-heavy puzzles lean on. The C# toolbox's `Ring.cs` grows the same way and the Lua twin carries a `copies` counter the test reads directly, 4 copies when a 4-ring doubles, and that counter is the honest little instrument this header leaves implicit.

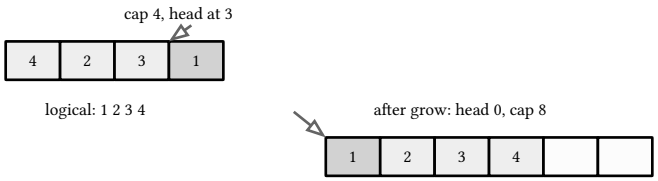


figure 13: a wrapped ring before and after growth: the logical order survives, the head rebases to slot 0

2.6. the vector and the sort idiom

The vector is the dynamic array of the series, `long long` elements, capacity doubling through `reserve`, an insert that shifts the tail right, and a pop that refuses on empty. The capacity pins in the test are the amortized argument made concrete: 4 through the first four pushes, 8 at the fifth, 16 at the ninth, and a `reserve(100)` that jumps past several doublings in one realloc.

The dry run: the suite's insert block in `vec_test.c` walks the capacity pins into the tail shift, then the sort fixture reloads `5 3 9 1 7 2 8 6`.

1. Four pushes 1 through 4 sit inside capacity 4; the fifth doubles to 8 and pushes 9 through 12 double again to 16, both asserted; the pop returns 12, leaving the eight elements `1 2 3 4 5 9 10 11`.
2. `vec_insert(&v, 2, 99)` shifts the six-element tail right one slot and lands 99 at index 2: size 8 + 1 = 9, and the asserts read `vec_get(2)` = 99 with `vec_get(3)` = 3, the old element's new home.
3. Two pops return 11 then 10.
4. The sort fixture pushes `5 3 9 1 7 2 8 6`; `qsort` over the exposed buffer with `cmp_ll` lands `1 2 3 5 6 7 8 9`, all eight positions asserted.
5. The negated comparator sorts the same buffer the other way, and `vec_reserve(&v, 100)` jumps capacity past 100 with size still 8.

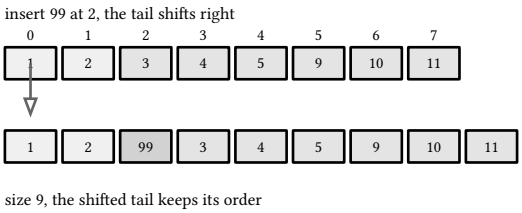


figure 14: the insert at index 2: the six-element tail slides one slot right, 99 lands at the gap

The shifted tail and the sorted buffer are the two idioms the listing below exposes `buf` for, and the sorted read `1 2 3 5 6 7 8 9` is the pinned one.

```c
static inline void vec_reserve(Vec *v, unsigned long long need) {
  if (need ≤ v->cap)
    return;
  while (v->cap < need)
    v->cap *= 2;
  v->buf = realloc(v->buf, v->cap * sizeof(long long));
}

static inline void vec_push(Vec *v, long long x) {
  vec_reserve(v, v->size + 1);
  v->buf[v->size++] = x;
}

// pop refuses on empty and returns 0
static inline int vec_pop(Vec *v, long long *out) {
  if (v->size == 0)
    return 0;
  *out = v->buf[--v->size];
  return 1;
}

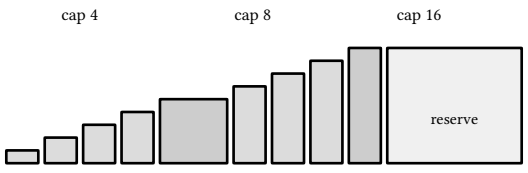
static inline long long vec_get(const Vec *v, unsigned long long i) {
  return v->buf[i];
}

static inline void vec_set(Vec *v, unsigned long long i, long long x) {
  v->buf[i] = x;
}

// insert shifts the tail right; the caller keeps i within bounds
static inline void vec_insert(Vec *v, unsigned long long i, long long x) {
  vec_reserve(v, v->size + 1);
  for (unsigned long long j = v->size; j > i; j--)
    v->buf[j] = v->buf[j - 1];
  v->buf[i] = x;
  v->size++;
}
```

listing 9: c: doubling reserve, refusing pop, and the tail-shifting insert (source: icpc/samples-c/src/Ch02/vec.h, lines 28-64)

The reason a raw buffer earns its place next to the structured containers is `qsort`. The vec exposes `buf`, `size`, and the element width, which is the whole interface `qsort` wants, and the comparator idiom, subtract-free `(x > y) - (x < y)` so no overflow ever enters the comparison, sorts the same buffer ascending through `cmp_ll` and descending through its negation. The fixture `5 3 9 1 7 2 8 6`, eight values with 4 skipped on purpose so an off-by-one cannot hide, sorts to `1 2 3 5 6 7 8 9` both ways.



pale bars: the capacity jumps at pushes 5 and 9

figure 15: size against capacity across the pinned pushes, the doubling staircase under every append

2.7. grid index math

The grid module is the smallest header and the one every year chapter touches first. A grid is a row-major buffer plus `rows` and `cols`, the index law is `r * cols + c`, and the module supplies the inverse, row and column from a flat index, the bounds predicate, and the two delta tables, the 4-neighbor cross and the 8-neighbor square. There is no allocation and no struct: the functions are pure index arithmetic, which is why they can be `static inline` without a thought.

The dry run: the 5 by 7 fixture in `grid_test.c` pins the law and its inverse on index 34, then the neighbor counts at an interior cell, an edge, and a corner.

1. The far corner: `grid_idx(4, 6, 7)` computes 4 x 7 + 6 = 34, asserted.
2. The inverse splits it: 34 / 7 = 4 with 4 x 7 = 28, then 34 - 28 = 6, row 4 column 6, both asserted.
3. Index 13 decodes the same way, 13 = 1 x 7 + 6, row 1 column 6, and the sweep round-trips all 5 x 7 = 35 cells.
4. The interior cell (2, 3) keeps every axis step in bounds, count 4, and all eight with diagonals, count 8, the two asserted together.
5. The top edge (0, 3) loses the up step and the two up-diagonals, 4 - 1 = 3 and 8 - 3 = 5, and the corner (0, 0) keeps 2 and 3, all six counts asserted.

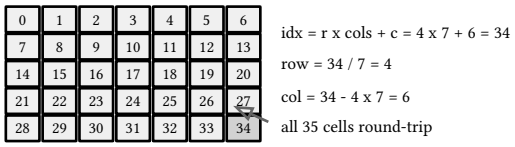


figure 16: the 35 flat indices of the 5 by 7 fixture in five rows, the far corner shaded, the law and its inverse meeting there

Every index decodes back to its cell, 34 the far corner both ways, and the listing below is the law and its inverse.

```c
static const int GRID_D4[4][2] = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
static const int GRID_D8[8][2] = {{-1, 0},  {1, 0},  {0, -1}, {0, 1},
                                  {-1, -1}, {-1, 1}, {1, -1}, {1, 1}};

static inline long long grid_idx(int r, int c, int cols) {
  return (long long)r * cols + c;
}

static inline int grid_row(long long idx, int cols) {
  return (int)(idx / cols);
}

static inline int grid_col(long long idx, int cols) {
  return (int)(idx % cols);
}

static inline int grid_in_bounds(int r, int c, int rows, int cols) {
  return r >= 0 && r < rows && c >= 0 && c < cols;
}

// how many of the 4- or 8-neighbors of (r, c) stay inside the grid
static inline int grid_neighbor_count(int r, int c, int rows, int cols,
                                      int eight) {
  int cnt = 0;
  int dn = eight ? 8 : 4;
  for (int d = 0; d < dn; d++) {
    int dr = eight ? GRID_D8[d][0] : GRID_D4[d][0];
    int dc = eight ? GRID_D8[d][1] : GRID_D4[d][1];
    if (grid_in_bounds(r + dr, c + dc, rows, cols))
      cnt++;
  }
  return cnt;
}
```

listing 10: c: the delta tables, the index law and its inverse, and in-bounds neighbor counting (source: icpc/samples-c/src/Ch02/grid.h, lines 6-38)

The fixture grid is 5 by 7: index 34 decodes to row 4 column 6, every cell's index round-trips, and the neighbor counts land where geometry says, 4 and 8 at the interior, 3 and 5 at an edge, 2 and 3 at a corner. The degree sums over a 5 by 5 grid are the quiet correctness check: 80 directed 4-edges and 144 with diagonals, both derivable by counting shared edges and half-diagonals by hand. The Lua twin returns the in-bounds neighbor coordinates themselves rather than counting them, coordinates a puzzle wants more often than a count, and its tables live as plain Lua arrays.

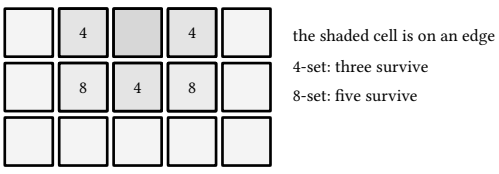


figure 17: the 4- and 8-neighbor deltas around an edge cell, and what the edge removes

2.8. reading input

Every contest problem starts as a file, so `io.h` reads a whole file into one buffer and splits it into lines. The module's first act is the opt-out from the introduction, `_CRT_SECURE_NO_WARNINGS` defined before `stdio.h` is included, because the portable `fopen` is what the module wants and the define has to precede the header that carries the deprecation

attribute. `io_read_file` returns the byte count or −1 when the file cannot be opened, and the buffer is nul-terminated and caller-freed.

The dry run: the suite in `io_test.c` writes the 31-byte fixture and asserts the byte count, the five split lines, and the two edge files.

1. The fixture counts 31 bytes: `alpha` 5, newline 1, `beta` 4, newline 1, the empty line's newline 1, `gamma delta` 11, newline 1, `epsilon` 7, and 5 + 1 + 4 + 1 + 1 + 11 + 1 + 7 = 31, the asserted `len`.
2. The read nul-terminates past the bytes, so `buf[31]` reads 0, and the corner asserts read `buf[0]` as the `a` and `buf[30]` as the final `n`.
3. The splitter walks the four newlines at byte indexes 5, 10, 11, and 23, terminates each line in place, and stores five pointers, the asserted count.
4. `lines[2]` is the empty string between the newlines at 10 and 11, the kept blank, and `lines[4]` points at byte 24, the `epsilon` tail no newline terminates, still counted.
5. The missing path returns −1, the crlf rewrite splits 2 lines with the carriage returns stripped, and the empty file reads 0 bytes and 0 lines.

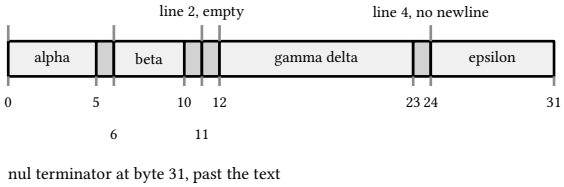


figure 18: the 31 bytes as a ruler: four newlines at 5, 10, 11, 23 cut five lines, the tail counts without one

The read returns the pinned 31 and the split the pinned five lines, and the listings below are the read and the walk.

```c
#ifndef _CRT_SECURE_NO_WARNINGS
#define _CRT_SECURE_NO_WARNINGS
#endif
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// returns the byte count, or -1 when the file cannot be opened; the
// buffer is nul-terminated and caller frees it
static inline long long io_read_file(const char *path, char **out) {
  FILE *f = fopen(path, "rb");
  if (!f)
    return -1;
  fseek(f, 0, SEEK_END);
  long long len = ftell(f);
  fseek(f, 0, SEEK_SET);
  char *buf = malloc((size_t)len + 1);
  size_t got = fread(buf, 1, (size_t)len, f);
  fclose(f);
  buf[got] = 0;
  *out = buf;
  return (long long)got;
}
```

listing 11: c: the ucrt opt-out first, then the whole-file read with byte count (source: icpc/samples-c/src/Ch02/io.h, lines 9-31)

The splitter walks for newlines and strips a trailing carriage return per line, so crlf text from a windows editor is handled where it should be, at the boundary. The fixture is 31 bytes, `alpha`, `beta`, an empty line, `gamma delta`, and an unterminated `epsilon`: five lines, the empty one kept, the tail counted. An empty file is zero bytes and zero lines, and a final line without a newline still counts, the two edge cases a puzzle input will eventually exercise. The Go

twin's ReadLines keeps trailing blanks, a documented divergence between the twins, and the year chapters cite whichever behavior their language's toolbox pins.

```
static inline long long io_read_lines(const char *buf, char **lines,
                                      long long max_lines) {
  long long n = 0;
  const char *p = buf;
  while (*p && n < max_lines) {
    const char *nl = strchr(p, '\n');
    long long len = nl ? nl - p : (long long)strlen(p);
    char *line = malloc((size_t)len + 1);
    for (long long i = 0; i < len; i++)
      line[i] = p[i];
    line[len] = 0;
    // strip a trailing carriage return from crlf text
    if (len && line[len - 1] == '\r')
      line[len - 1] = 0;
    lines[n++] = line;
    p = nl ? nl + 1 : p + len;
  }
  return n;
}
```

listing 12: c: the newline walk, crlf strip, and the unterminated tail that still counts (source: icpc/samples-c/src/Ch02/io.h, lines 35-53)

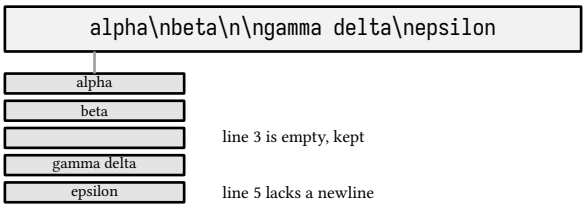


figure 19: the 31-byte fixture to five lines: splits at newlines, the blank survives, the tail counts without one

2.9. number theory with an honest product

This is the module with the story. Modular exponentiation over 64-bit values needs a multiplication whose product can exceed 64 bits, and C offers `__uint128_t` for exactly that. On this toolchain the obvious implementation compiles, clang 23 accepts the type and the arithmetic, and then the link fails: `undefined symbol __umodti3`, the compiler-rt helper that implements a 128-bit remainder, absent from the MSVC plus lld link this machine performs. The chapter 16 probe of the data structures book recorded the same failure through the real gate, and the conclusion is binding here: the toolbox computes products by add-and-double, doubling one operand and reducing modulo `m` every step, so no intermediate ever exceeds two 64-bit values added.

The dry run: `num_test.c` pins the euclid chain on 48 and 18, the bezout identity for 240 and 46, and the classic three-congruence fold.

1. `num_gcd(48, 18)` substitutes: 48 = 2 x 18 + 12, then 18 = 1 x 12 + 6, then 12 = 2 x 6 + 0, gcd 6, asserted.
2. The lcm pins: 4 x 6 = 24 over gcd 2 leaves 12, and 21 x 6 = 126 over gcd 3 leaves 42, both asserted.
3. `num_ext_gcd(240, 46, &x, &y)` returns gcd 2 with x = −9 and y = 47, and the asserted identity reads 240 x (-9) = −2160, 46 x 47 = 2162, −2160 + 2162 = 2.
4. The crt fold: 2 mod 3 with 3 mod 5 combines to 8 mod 15, then 8 + 15 = 23 at modulus 15 x 7 = 105, the asserted pair.
5. The non-coprime pair 1 mod 6 with 3 mod 10 resolves to 13 mod 30, asserted, and the contradiction is rejected by the gcd test with return 0.

step	line	value
euclid	48 = 2 x 18 + 12	12
euclid	18 = 1 x 12 + 6	6
euclid	12 = 2 x 6 + 0	gcd 6
bezout	240 x (-9) + 46 x 47	−2160 + 2162 = 2
crt	(2, 3) then (3, 5)	8 mod 15
crt	8 + 15, modulus 15 x 7	23 mod 105
crt	1 mod 6 with 3 mod 10	13 mod 30

Both anchors hold, 2 as the bezout identity and 23 mod 105 as the fold, and the listings below are the machinery that computes them.

```c
// (a * b) % m without ever forming a 64-bit product
static inline unsigned long long
num_mulmod(unsigned long long a, unsigned long long b, unsigned long long m) {
  unsigned long long r = 0;
  a %= m;
  while (b) {
    if (b & 1)
      r = (r + a) % m;
    a = (a + a) % m;
    b >>= 1;
  }
  return r;
}

static inline unsigned long long num_modpow(unsigned long long base,
                                            unsigned long long exp,
                                            unsigned long long m) {
  unsigned long long r = 1 % m;
  base %= m;
  while (exp) {
    if (exp & 1)
      r = num_mulmod(r, base, m);
    base = num_mulmod(base, base, m);
    exp >>= 1;
  }
  return r;
}
```

listing 13: c: the add-and-double mulmod and the modpow built on it, a reduction every step (source: icpc/samples-c/src/Ch02/num.h, lines 25-51)

The test pins the machinery three ways. The identity case, `mulmod(6000000000, 6000000000, 1000000007)`, is 1764, because a billion is congruent to −7 modulo a billion and seven, so the product is 36 times 49. The exponentiation anchor is `2^100 mod 1000000007` at 976371285, the value every language in this book agrees on. And the chinese remainder theorem fixture is the classic, 2 mod 3, 3 mod 5, 2 mod 7 combining through 8 mod 15 to 23 mod 105, with the consistent non-coprime pair, 1 mod 6 with 3 mod 10, resolving to 13 mod 30 and the inconsistent pair rejected by the gcd test.

```c
// combine (r1, m1) and (r2, m2); returns 1 on success
static inline int num_crt(unsigned long long r1, unsigned long long m1,
                          unsigned long long r2, unsigned long long m2,
                          unsigned long long *r, unsigned long long *m) {
  unsigned long long g = num_gcd(m1, m2);
  if ((r2 + m2 - r1 % m1) % m1 % g != 0)
    return 0;
  unsigned long long l = num_lcm(m1, m2);
  for (unsigned long long x = r1 % m1; x < l; x += m1)
    if (x % m2 == r2 % m2) {
      *r = x;
      *m = l;
      return 1;
    }
  return 0;
}
```

listing 14: c: pairwise crt, the consistency check against gcd, and the scan to the combined residue (source: icpc/samples-c/src/Ch02/num.h, lines 87-102)

The Lua twin arrives at the same add-and-double design from the opposite direction. C never reaches a wrap-around, the 128-bit product simply fails to link, while Lua's integers wrap silently in hardware, and book 11, ch 9 (the icpc world finals, the lua toolbox) pins the proof in a test, `math.maxinteger * 2 == -2`, before building the identical doubling loop. One language cannot link the wide product, the other cannot trust it, and the fix is the same seven lines in both.

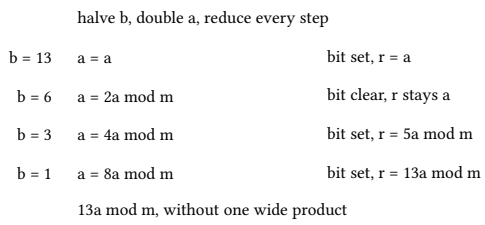


figure 20: one mulmod as the add-and-double ladder: halve b, double a, add when the bit is set, reduce every step

2.10. md5 from the RFC

Some days want a real hash, and the finals' favorite is md5, so the toolbox builds it from RFC 1321 rather than linking a crypto library: the little-endian Merkle-Damgard construction over 512-bit blocks, sixteen words a block, 64 rounds in four nonlinear passes. The padding is the standard, one `0x80` byte, zeros to 56 mod 64, then the bit length in eight little-endian bytes, and each block is read as sixteen 32-bit little-endian words.

The dry run: the suite's 80-digit vector in `md5_test.c` is the two-block case, and the walk is its padding arithmetic.

1. The input is 80 decimal digits, asserted by `strlen`, so the bit length is 80 x 8 = 640.
2. The `0x80` pad byte takes the message to 80 + 1 = 81 bytes, 81 - 64 = 17 bytes into the second block.
3. Zeros follow to 56 mod 64: 56 - 17 = 39 zeros, 17 + 39 = 56.
4. The bit length lands in eight little-endian bytes, 640 as `80 02` then six zero bytes, and 56 + 8 = 64 closes the second block, 128 bytes total.
5. Two blocks fold through the 64 rounds each and the digest prints `57edf4a22be3c955ac49da2e2107b67a`, the pinned vector.

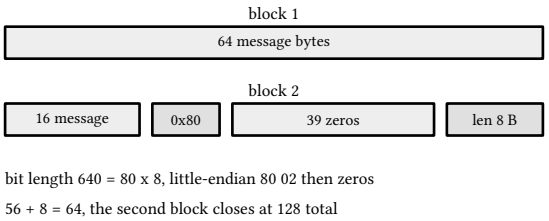


figure 21: the 80-digit input pads into two blocks: 64 message bytes fill the first, the second carries 16 message bytes, the pad, 39 zeros, and the length

The second block closes at 128 bytes and the digest lands on the pinned `57edf4a22be3c955ac49da2e2107b67a`, the listings below.

```c
// padded message: original + 0x80 + zeros + 8-byte little-endian length
unsigned long long padded = ((len + 8) / 64 + 1) * 64;
unsigned char *buf = malloc((size_t)padded);
memset(buf, 0, (size_t)padded);
memcpy(buf, msg, (size_t)len);
buf[len] = 0x80;
unsigned long long bits = len * 8;
for (int i = 0; i < 8; i++)
  buf[padded - 8 + i] = (unsigned char)(bits >> (8 * i));

for (unsigned long long off = 0; off < padded; off += 64) {
  unsigned int w[16];
  for (int i = 0; i < 16; i++)
    w[i] = (unsigned int)buf[off + 4 * i] |
           ((unsigned int)buf[off + 4 * i + 1] << 8) |
           ((unsigned int)buf[off + 4 * i + 2] << 16) |
           ((unsigned int)buf[off + 4 * i + 3] << 24);
  unsigned int a = h0, b = h1, c = h2, d = h3;
```

listing 15: c: pad to 56 mod 64, length little-endian, load sixteen words per block (source: icpc/samples-c/src/Ch02/md5.h, lines 39-56)

The rounds are the four boolean functions of the RFC with their message schedules, indices `i`, `5i+1`, `3i+5`, `7i` modulo 16, and the left-rotation schedule, the dance of `a`, `b`, `c`, `d` that ends by adding the working registers into the chaining state. The sine table and shift table are the RFC's own constants.

```c
    for (int i = 0; i < 64; i++) {
      unsigned int f;
      int g;
      if (i < 16) {
        f = (b & c) | (~b & d);
        g = i;
      } else if (i < 32) {
        f = (d & b) | (~d & c);
        g = (5 * i + 1) % 16;
      } else if (i < 48) {
        f = b ^ c ^ d;
        g = (3 * i + 5) % 16;
      } else {
        f = c ^ (b | ~d);
        g = (7 * i) % 16;
      }
      unsigned int tmp = d;
      d = c;
      c = b;
      b = b + md5_leftrotate(a + f + MD5_K[i] + w[g], MD5_S[i]);
      a = tmp;
    }
    h0 += a;
    h1 += b;
    h2 += c;
    h3 += d;
  }
```

listing 16: c: the 64 rounds, four passes with their word schedules, folded into the state (source: icpc/samples-c/src/Ch02/md5.h, lines 57-83)

The test suite is the RFC 1321 vector set: the empty string `d41d8cd98f00b204e9800998ecf8427e`, `a`, `abc`, `message digest`, the alphabet, and the 80-digit input that crosses the 64-byte block boundary and forces the two-block padding path, `57edf4a22be3c955ac49da2e2107b67a`. Determinism is asserted directly, same input twice, and avalanche by one changed letter, `abcd` against `abce`, digests differing. The Lua twin computes the same construction over 64-bit integers masked to 32 bits after every operation, and all five of its pinned vectors match this suite's, the same RFC read by two machines.

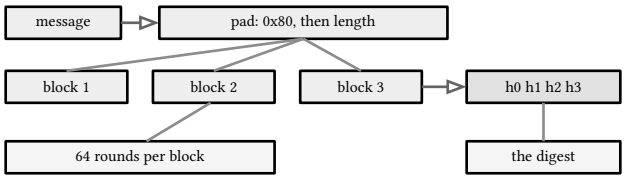


figure 22: merkle-damgard: pad once, then fold block after block into four chaining words

2.11. a json reader

The json module is a recursive-descent parser into a node tree from a static pool, 128 nodes, 16 children per container, 64 bytes per string, fixed bounds chosen for puzzle documents and stated in the test rather than hidden behind allocation. Objects keep their member names beside their children, arrays keep order, strings decode the standard escapes, numbers go through `strtod`, and the three literals become tags. The whole grammar hangs off one dispatch function.

The dry run: the crafted document `{"a": 1, "b": [2, 3.5, true, null, "x"], "c": {"d": -4}}` is the suite's walk fixture, its children asserted by index and its numeric leaves summed.

1. The parse returns a three-child object, `n_children` = 3 asserted, with the keys `a`, `b`, `c` in order.

2. The array under `b` holds five children asserted by index: the numbers 2 and 3.5, the true, the null, and the string `x`, order kept.
3. The nested object carries the `-4`, and the four numeric leaves fold through the sum walk: 1 + 2 = 3, 3 + 3.5 = 6.5, 6.5 - 4 = 2.5, the pinned `json_sum_numbers`.
4. The escape fixture `{"k": "a\"b\\c\nd"}` decodes its three escape pairs to quote, backslash, newline, and the `strcmp` reads the seven-character result.
5. The scalars parse alone, `42` as a number, `true` as boolean 1, `null` as the tag, `"solo"` as the string.

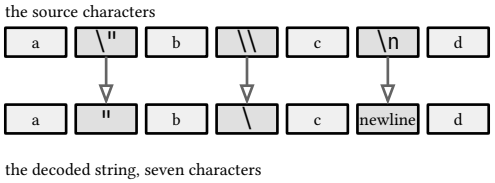


figure 23: the escape decode: three pairs in the source become quote, backslash, and newline in the string, everything else passes through

The leaves fold to the pinned 2.5 and the escapes decode in place, and the listings below are the dispatch and the walks.

```c
static inline JNode *json_parse_value(void) {
  json_skip_ws();
  if (*json_at == '{')
    return json_parse_object();
  if (*json_at == '[')
    return json_parse_array();
  if (*json_at == '"') {
    JNode *n = json_new(JSON_STRING);
    json_parse_string_into(n->string);
    return n;
  }
  if (strncmp(json_at, "true", 4) == 0) {
    json_at += 4;
    JNode *n = json_new(JSON_BOOL);
    n->boolean = 1;
    return n;
  }
  if (strncmp(json_at, "false", 5) == 0) {
    json_at += 5;
    JNode *n = json_new(JSON_BOOL);
    n->boolean = 0;
    return n;
  }
  if (strncmp(json_at, "null", 4) == 0) {
    json_at += 4;
    return json_new(JSON_NULL);
  }
  // number: optional sign, digits, fraction, exponent (strtod handles it)
  if (*json_at == '-' || (*json_at ≥ '0' && *json_at ≤ '9')) {
    char *end;
    double v = strtod(json_at, &end);
    if (end == json_at) {
      json_ok = 0;
      return json_new(JSON_NULL);
    }
    json_at = end;
    JNode *n = json_new(JSON_NUMBER);
    n->number = v;
    return n;
  }
  json_ok = 0;
  return json_new(JSON_NULL);
}
```

listing 17: c: the value dispatch: containers recurse, literals match, numbers take strtod (source: icpc/samples-c/src/Ch02/json.h, lines 169-211)

Parsing is all-or-nothing: a global `json_ok` flag drops on the first malformed construct, and the entry point refuses trailing content, so `{`, `[1, 2`, `tru`, and `1 2` all return a null pointer. The walk helpers are where the puzzle value lives, a sum over every numeric leaf and a count by tag, and the crafted fixture runs both: a document holding `1`, an array of `2, 3.5, true, null, "x"`, and a nested `-4` has four numbers summing to 2.5, one array, two objects, one boolean, one null.

```c
// sum every numeric leaf in the tree
static inline double json_sum_numbers(const JNode *n) {
  if (!n)
    return 0;
  double s = n->tag == JSON_NUMBER ? n->number : 0;
  for (int i = 0; i < n->n_children; i++)
    s += json_sum_numbers(n->children[i]);
  return s;
}

static inline int json_count_tag(const JNode *n, int tag) {
  if (!n)
    return 0;
  int c = n->tag == tag ? 1 : 0;
  for (int i = 0; i < n->n_children; i++)
    c += json_count_tag(n->children[i], tag);
  return c;
}
```

listing 18: c: the numeric-leaf sum and the tag count, the walks the year chapters call (source: icpc/samples-c/src/Ch02/json.h, lines 225-242)

The comparison with Lua is the sharpest in the book. C needs 244 lines, a node pool, and child arrays because it has no dynamic structures of its own, and its null is a tag. Lua decodes the same grammar straight into plain tables in 167 lines, its null is a sentinel table so the value can sit in a field without vanishing into nil, and its errors are raised by name, `expected ':'`, against this module's silent null return. Both pin the same 2.5 walk and both refuse the same malformed documents, each in its language's own idiom, and book 11, ch 9 (the icpc world finals, the lua toolbox) shows the other half of the conversation.

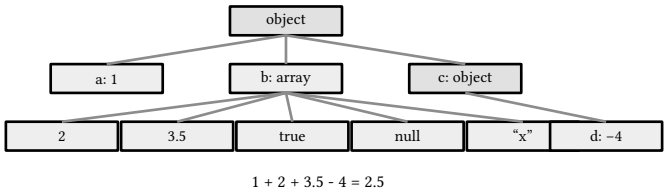


figure 24: the crafted fixture as a node tree: numbers at the leaves, the walk sums 2.5

2.12. across the eight languages

This chapter owns C, and the honest close is the same table every toolbox chapter closes with, the structure, how C builds it, how Lua builds it, and why they differ. The other six languages carry their own chapters, and their rows join the comparison there. Java's chapter 5 sits at the opposite pole and still teaches the same seam: BigInteger ships in its library, but its kit builds the wide product from `Math.multiplyHigh` plus a 128-step shift-subtract remainder lane beside it, add-and-double with a machine that cooperates. Rust's chapter 3 mirrors this one module for module, and the one row above that dissolves there is modular num: `u128` is a first-class type, the wide product and its remainder are one expression, and add-and-double never starts.

structure	c builds it as	lua builds it as	why they differ
bigint	sign-magnitude struct, 12 fixed limbs	sign plus limb table, unbounded	c pins the digit budget, lua lets tables grow
map and set	open addressing, fnv-1a, tombstones	the table primitive itself	c has no hash map, lua has nothing else
min heap	typed array of key-value pairs	array plus injected comparator	one c type order, any lua order

structure	c builds it as	lua builds it as	why they differ
deque	ring with head index and count	ring with head, count, copies	the lua test reads the copy count directly
grid	pure index functions over a buffer	parse to one string, index functions	same law, lua keeps the cells as text
input	whole-file read, newline split	line lists from the runner	c owns its bytes, lua owns its strings
modular num	add-and-double, the link refused the wide product	add-and-double, the wrap is proven	different diseases, same cure
md5	uint32 arithmetic	int64 masked to 32 bits	lua has no 32-bit integers to use
json	node pool with tags	plain tables, null sentinel	c needs the tree, lua is the tree

Verified by `make verify-c`: 10 programs, 251 internal checks, bigint 36, deque 20, grid 33, heap 27, io 15, json 34, map 26, md5 9, num 31, vec 20, plus the format and address-sanitizer legs of the gate.

sources: R. Rivest, RFC 1321, The MD5 Message-Digest Algorithm, April 1992, the K table, shift schedule, and the test vectors this suite pins; the FNV-1a constants from the Fowler, Noll, Vo reference hash page; the `__umodti3` link outcome is the machine fact recorded by the chapter 16 probe in book 10 and reconfirmed by this suite's gate run, 2026-09-14. The Lua comparisons cite the landed `books/icpc/samples-lua/` modules read the same day.

3. the rust toolbox

Rust enters the contest third in the corpus reading order, between C and Go, and its toolbox is the eighth. Eleven files live in `books/icpc/samples-rust/src/Ch03/`, one module per file, each self-contained with its check harness embedded: 1362 non-blank lines carrying 278 checks, zero dependencies, every structure either std or built here, compiled per file under a pinned rustc 1.99.0 with `--edition 2024 -D warnings`. The module list mirrors chapter 2 file for file, bigint through json, with one addition, the token scanner `scan.rs`, which C never needed and Go's `input.go` is the sibling of. The integer story is the reversal of C's. Where C reaches for `__uint128_t` and the link fails, rust owns `u128` as a first-class type and the wide product is one expression. The trap moves to overflow: a plain rustc build keeps overflow checks on, arithmetic that wraps silently in C aborts here, so every wraparound site says `wrapping_` or `checked_` out loud, and a green run is the proof that nothing wraps unannounced.

3.1. one file, one gate

The sample contract is the corpus one, stated in rust terms. Every file embeds the same helper: a static `AtomicUsize` counter and an `ok(cond, name)` that prints `ok N name` on a pass, prints `FAIL:`
`name` to stderr and exits 1 on the first failure. A module-scope counter in rust must be atomic to exist at all, safe shared state is the only kind the language offers, and the suites stay single-threaded regardless. The runner `tools/run-rust-samples.ps1` resolves the rustup-managed stable msvc toolchain from `RUSTUP_HOME` or `~/.rustup`, asserts the version line begins `rustc 1.99.0`
(`b940084d7` and names the install command otherwise, then compiles each file, runs the binary, and counts lines matching `^ok \d+`. Zero checks is red, a FAIL line is red, a nonzero exit is red, and the nightly toolchain this machine also carries never satisfies the gate. No cargo, no clippy, no rustfmt anywhere in the lane: one file, one compiler call, one binary, the same shape a contest judge performs. The toolchain resolution story is the subject of book 3, ch 1 (rust 1.99, toolchain, cargo, and the rustup lane).

The dry run: the opening block of `scan.rs`, nine checks over one five-token fixture, is the ledger of how the counter climbs.

1. `Scanner::load` reads `"3 42\n-7\t0 18446744073709551615"` whole and splits it on whitespace: five tokens. `has_next` passes, ok 1, and the token count asserts 5, ok 2.
2. `next_u64` takes the 3, ok 3, then the 42, ok 4.
3. `next_i64` takes the -7, ok 5, then the 0, ok 6.
4. `next_u64` takes the last token and parses all twenty digits to `u64::MAX`, 18446744073709551615, ok 7.
5. `has_next` now reads false, ok 8, and `next` returns `None` at the end, ok 9, the refusal on record before anything else runs.

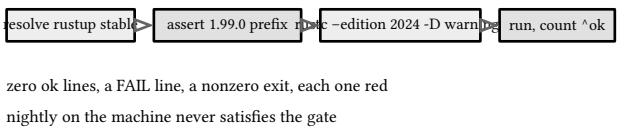


figure 25: the gate chain: toolchain assert, per-file compile under -D warnings, run, count ok lines, red on any failure

Nine checks print, and the helper doing the counting is the listing below, identical bytes in all eleven files.

```rust
static CHECKS: std::sync::atomic::AtomicUsize = std::sync::atomic::AtomicUsize::new(0);

fn ok(cond: bool, name: &str) {
    let i = CHECKS.fetch_add(1, std::sync::atomic::Ordering::SeqCst) + 1;
    if cond {
        println!("ok {i} {name}");
    } else {
        eprintln!("FAIL: {name}");
        std::process::exit(1);
    }
}
```

listing 19: the helper every module embeds, one atomic counter, one ok line per check, exit 1 on the first failure (source: icpc/samples-rust/src/Ch03/vec.rs, lines 1-11)

`-D warnings` is the discipline's teeth: an unused import or a dead variable is a build failure, so the tree carries nothing it does not use. The check names ride the ok lines to the runner, and the per-module counts in this chapter's footer were measured there.

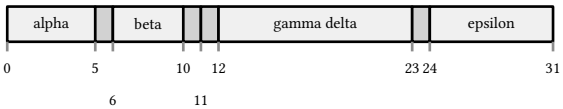
3.2. input over a locked stdin

`io.rs` reads the whole source into one `String` through `impl Read`, so a file handle, a cursor over bytes, and the standard input are the same call, and errors travel as `io::Result`: the missing-path check asserts the error instead of a sentinel. `read_lines` splits the whole buffer on `\n`, pops the single empty piece a trailing newline leaves, and strips one `\r` per line, so windows text needs no pre-pass.

The dry run: the fixture is chapter 2's 31 bytes, the cross-language anchor, and the walk is the split over it.

1. `alpha` 5, newline 1, `beta` 4, newline 1, the empty line's newline 1, `gamma delta` 11, newline 1, `epsilon` 7: 5 + 1 + 4 + 1 + 1 + 11
    1. 1 + 7 = 31, the asserted length, and the corner reads pin byte 0

    as `a` and byte 30 as the final `n`.
2. The split walks the four newlines at byte indexes 5, 10, 11, and 23 and leaves five pieces: the unterminated `epsilon` still counts, and no trailing newline exists for the pop to remove.
3. `lines[2]` is the empty string between the newlines at 10 and 11, the blank kept as data, the same reading Go's `ReadLines` pins and C's splitter arrives at from the byte side.
4. The crlf rewrite `one\r\ntwo\r\n` splits to two lines with both carriage returns stripped, and the empty file reads 0 bytes and 0 lines.
5. `"x\n"` splits to `x` and one empty, the pop fires, and the count is 1: no phantom final line, a check C's walker never needed because its count came from the newline walk itself.



line 2 is the kept blank, line 5 lacks a newline

figure 26: the 31 bytes as a ruler: four newlines at 5, 10, 11, 23 cut five lines, the tail counts without one

The read returns the pinned 31 and the split the pinned five lines, and the listings below are the splitter and the blocking lane.

```rust
fn read_lines(s: &str) -> Vec<String> {
    if s.is_empty() {
        return Vec::new();
    }
    let mut parts: Vec<&str> = s.split('\n').collect();
    if parts.last() == Some(&"") {
        parts.pop();
    }
    parts
        .iter()
        .map(|p| p.strip_suffix('\r').unwrap_or(p).to_string())
        .collect()
}
```

listing 20: split the whole UTF-8 buffer, pop the trailing empty, strip one carriage return per line (source: icpc/samples-rust/src/Ch03/io.rs, lines 25-37)

The stdin lane reads through `std::io::stdin().lock()` into the same splitter, and the whole path sits behind an argv gate, `--stdin`. The runner executes every binary with no arguments, and a bare run that locked stdin would sit waiting on a console that never feeds it, so the blocking lane exists only when asked for. The file's last check writes its own ok line through a locked stdout handle, the output side of the same discipline.

```rust
fn stdin_line_count() -> usize {
    let mut buf = String::new();
    let mut lock = std::io::stdin().lock();
    lock.read_to_string(&mut buf).unwrap();
    read_lines(&buf).len()
}

fn main() {
    if std::env::args().any(|a| a == "--stdin") {
        println!("{}", stdin_line_count());
        return;
    }
```

listing 21: the locked-stdin lane behind an argv gate, the bare runner never blocks (source: icpc/samples-rust/src/Ch03/io.rs, lines 39-50)

One divergence against the C twin is the walker itself. C's splitter is `strlen`-based, so a NUL byte in the buffer would end its text, and the rust split walks every byte of the `String`, NUL included, because a rust string holds them and UTF-8 input arrives whole. Both walkers produce the same five lines on this fixture, and the difference shows only on input no judge would send. The Go twin's trailing-blank behavior is the same deliberate reading, stated in book 11, ch 4 (the icpc world finals, the go toolbox).

3.3. the token scanner

`scan.rs` is the one module with no C twin. C's year chapters parse out of the line list with `sscanf`, and the sibling is Go's `input.go`, whose `ReadInts` returns a whole vector of numbers or an error. The Scanner is a cursor between those readings: `load` reads any `Read` once and splits on whitespace into owned strings, then `has_next`, `next`, `next_u64`, and `next_i64` pull tokens one at a time, every take an `Option`, so an exhausted stream or a malformed token is `None` instead of an exception, a zero, or a sentinel position.

The dry run: the refusal block and the mixed-whitespace block close the file, 10 checks from the garbage fixture to the last token.

1. `"-1 12x3"`: `next_u64` refuses the sign, ok 10, and `next_i64` refuses the letters, ok 11, both refusals asserted, not skipped.

2. The boundaries parse exactly: `i64::MIN` and `i64::MAX`, ok 12 and 13.
3. `"1\r\n2"`: the carriage return is whitespace to the splitter, so the pair parses as 1 then 2, ok 14 and 15, no crlf special case in the token lane at all.
4. `"  lead\ttrail\n\nmid  "`: leading, trailing, tab, and doubled newline runs all collapse, three tokens, ok 16, then `lead`, `trail`, `mid`, ok 17 through 19, the file's end.

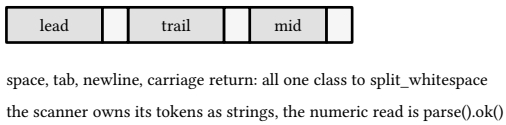


figure 27: the mixed-whitespace buffer: every run of whitespace is a seam, three tokens come out

Three tokens of four seams, and the listing is the whole scanner.

```rust
struct Scanner {
    tokens: Vec<String>,
    pos: usize,
}

impl Scanner {
    fn load(mut r: impl Read) -> Scanner {
        let mut buf = String::new();
        r.read_to_string(&mut buf).unwrap();
        Scanner {
            tokens: buf.split_whitespace().map(|t| t.to_string()).collect(),
            pos: 0,
        }
    }

    fn has_next(&self) -> bool {
        self.pos < self.tokens.len()
    }

    fn next(&mut self) -> Option<&str> {
        let t = self.tokens.get(self.pos)?;
        self.pos += 1;
        Some(t)
    }

    fn next_u64(&mut self) -> Option<u64> {
        let t = self.tokens.get(self.pos)?;
        self.pos += 1;
        t.parse().ok()
    }

    fn next_i64(&mut self) -> Option<i64> {
        let t = self.tokens.get(self.pos)?;
        self.pos += 1;
        t.parse().ok()
    }
}
```

listing 22: split_whitespace once into owned strings, then a cursor whose every take is an Option (source: icpc/samples-rust/src/Ch03/scan.rs, lines 15-51)

Owning the token list up front is the cost: the whole input lives in memory twice, once as text and once as tokens, the price of a cursor with no borrow entanglement, and a contest input is small. The `--scan-stdin` argv gate repeats

io's rule for the same reason, and the numeric conversion behind `next_u64` is the std parse, so the refusal channel and the parse channel are one.

3.4. the collections, borrowed whole

C builds its vector, deque, and heap by hand. Rust borrows all three from std and asserts them from the outside, and the honest comparison is contract against contract. `Vec` growth is amortized by contract: the std's collections page marks push with the amortized star and says the vector allocates a substantially larger array when it does grow, so the tests assert floors, capacity at least 4 through the first four pushes, at least 5 at the fifth, at least 9 at the ninth, where chapter 2 pinned the exact staircase 4, 8, 16. Same pushes, same eight-value sort fixture `5 3 9 1 7 2 8 6`, 4 skipped so an off-by-one cannot hide, sorted both directions, and the same `reserve(100)` jump past the doublings with the length held at 8.

```
    let mut v: Vec<i64> = Vec::with_capacity(4);

    ok(v.capacity() ≥ 4, "capacity-reserved");
    for i in 1..=4 {
        v.push(i as i64);
    }
    ok(v.capacity() ≥ 4, "capacity-held");
    v.push(5);
    ok(v.capacity() ≥ 5, "capacity-grown");
    ok(v.len() == 5, "len-5");
    for i in 9..=12 {
        v.push(i);
    }
    ok(v.capacity() ≥ 9, "capacity-doubled-law");
```

listing 23: capacity asserted as a floor: growth is amortized by contract, not pinned doubling (source: icpc/samples-rust/src/Ch03/vec.rs, lines 14-27)

The deque fixtures are C's. The wrap block pushes 1, 2, 3 to the back then 9 and 8 to the front and pops 8, 9, 1 across the seam, the growth block interleaves 50 pushes at each end from capacity 2 and drains −50 through −1 then 1 through 50 in order, and the rotation block restores order after six pop-front-push-back cycles. The container is `VecDeque`, all std, so the listing is the fixture, not the machinery:

```
    d.push_back(1);
    d.push_back(2);
    d.push_back(3);
    d.push_front(9);
    d.push_front(8);
    ok(d.pop_front() == Some(8), "wrap-8");
    ok(d.pop_front() == Some(9), "wrap-9");
    ok(d.pop_front() == Some(1), "wrap-1");
```

listing 24: the wrap fixture: two front pushes cross the seam, the pops read 8, 9, 1 (source: icpc/samples-rust/src/Ch03/deque.rs, lines 48-56)

The heap is where the borrowing needs one adapter. `BinaryHeap` is a max-heap by its own description, the biggest first, so the min-heap this toolbox wants is `Reverse` over a `(u64, i64)` tuple, and the tuple is the tiebreak: where C's heap compares keys only and equal keys come out in insertion order by the accident of a strict compare, the rust adapter orders equal keys by the value field, on purpose.

The dry run: the duplicate block pushes three pairs and pins the tiebreak.

1. Push (10, 1000), then (9, 91): under `Reverse` the max-heap's greatest is the smallest tuple, so the 9-pair sits on top.
2. Push (9, 92): equal key 9, and the tuple compares the value next, 91 before 92, so (9, 91) stays on top.

3. The pops return (9, 91), then (9, 92), then (10, 1000), the pinned order with the tie broken by value, not by arrival.

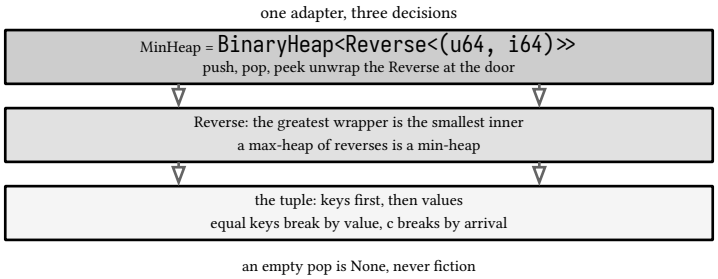


figure 28: three layers: the std max-heap at the bottom, reverse flipping it, the tuple breaking ties

The drain runs the tiebreak, and the listing is the adapter whole.

```rust
type MinHeap = std::collections::BinaryHeap<Reverse<(u64, i64)>>;

fn push(h: &mut MinHeap, key: u64, value: i64) {
    h.push(Reverse((key, value)));
}

fn pop(h: &mut MinHeap) -> Option<(u64, i64)> {
    h.pop().map(|Reverse(p)| p)
}

fn peek(h: &MinHeap) -> Option<(u64, i64)> {
    h.peek().map(|Reverse(p)| *p)
}
```

listing 25: binaryheap is a max-heap, reverse makes it min, the tuple breaks ties by value (source: icpc/samples-rust/src/Ch03/heap.rs, lines 15-27)

The rest of the heap fixtures repeat C's: the values travel with the keys, the interleaved block holds the minimum on top through pushes and pops, the heapsort pass over `4 9 1 7 3 8 6 5 2` pops 1 through 9 with each value ten times its key, and a growth run to 40 entries peeks (1, 39) with the value intact. Two extra checks state the borrowing out loud: the plain heap pops 9 first, the max-heap as documented, and the reversed one pops 3. The sift machinery under all of it, up and down, is book 10, ch 8 (practical data structures and algorithms, heaps and priority queues) territory, built from scratch there.

3.5. index math

The grid module is the one container-shaped piece built rather than borrowed, because it is not a container: pure index arithmetic over `i32` coordinates, the law `r * cols + c` computed in `i64`, the inverse split, the bounds predicate, and the two delta tables as const arrays. The fixture set is chapter 2′s, check for check, the same 33: the 5 by 7 grid, the far corner 4 x 7 + 6 = 34 decoding back to row 4 column 6, all 35 cells round-tripping, the neighbor counts 4 and 8 at the interior, 3 and 5 at an edge, 2 and 3 at a corner, the degree sums 80 and 144 over 5 by 5, both derivable by counting shared edges and half-diagonals by hand, and the degenerate one-row and one-column grids pinning 1 at an end and 2 in the middle.

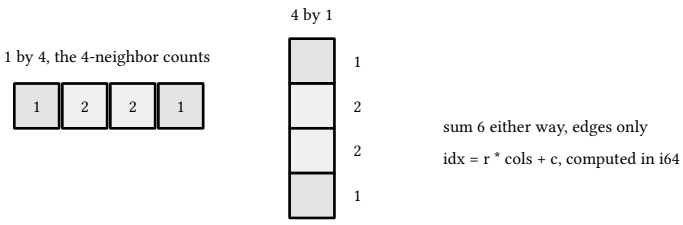


figure 29: the degenerate grids: a 1 by 4 row keeps 1 neighbor at each end and 2 in the middle, a 4 by 1 column the same

```rust
const D4: [(i32, i32); 4] = [(-1, 0), (1, 0), (0, -1), (0, 1)];
const D8: [(i32, i32); 8] = [
    (-1, 0),
    (1, 0),
    (0, -1),
    (0, 1),
    (-1, -1),
    (-1, 1),
    (1, -1),
    (1, 1),
];

fn grid_idx(r: i32, c: i32, cols: i32) -> i64 {
    r as i64 * cols as i64 + c as i64
}

fn grid_row(idx: i64, cols: i32) -> i32 {
    (idx / cols as i64) as i32
}

fn grid_col(idx: i64, cols: i32) -> i32 {
    (idx % cols as i64) as i32
}

fn grid_in_bounds(r: i32, c: i32, rows: i32, cols: i32) -> bool {
    r ≥ 0 && r < rows && c ≥ 0 && c < cols
}

fn grid_neighbor_count(r: i32, c: i32, rows: i32, cols: i32, eight: bool) -> i32 {
    let table = if eight { &D8[..] } else { &D4[..] };
    table
        .iter()
        .filter(|(dr, dc)| grid_in_bounds(r + dr, c + dc, rows, cols))
        .count() as i32
}
```

listing 26: the two delta tables, the index law and its inverse, and the filtered neighbor count (source: icpc/samples-rust/src/Ch03/grid.rs, lines 13-47)

The law is the same one every toolbox in this book pins, the Go twin round-trips a 3 by 3 on it and the C twin a 5 by 7, and the `grid_neighbor_count` shape, a filter over the delta table through the bounds predicate, replaces the C loop ladder with the same arithmetic under it.

3.6. the ordered map, not the hash map

The map is one line of type, `BTreeMap<String, i64>`, against the 130 lines of FNV-1a open addressing chapter 2 spends, and the choice of tree over hash is a ruling, not a taste. The std `HashMap` hashes with a random seed per process, so iteration order changes run to run, and a suite or solver that walked one into its output would pin a coin flip. The std's own page says it plainly: the ordered collections yield their items in sorted order. So the toolbox map is the ordered one, and its determinism is testable in a way a hash map's never is.

The dry run: the 200-key workload, the sorted lane, and the deletion sweep.

1. Inserts `key0` through `key199` with square values: length 200, every lookup returns, `key42` holds 1764, `key200` misses.
2. The keys vector equals its own sorted copy, asserted directly: iteration order is key order, no sort performed.
3. The first key is `key0` and the last is `key99`: the names are digit strings and the tree orders them as text, so `key99` sorts after `key199`, the reading the check pins on purpose.

4. Deleting the hundred even keys halves the map, and all hundred odd squares survive.
5. The `BTreeSet` half: insert reports true then false on the duplicate, iteration collects 3, 5, 9 in order, and remove mirrors the refusal.



text order: key99 after key199, the pinned reading

figure 30: the iteration strip: digit strings sorted as text, key99 last, a hash map would reshuffle this row every run

Deletion is where the C comparison is sharpest. Open addressing cannot empty a slot a probe walked through, so chapter 2 writes tombstones, counts them against the load factor, and consumes them on reinsertion. The tree just deletes: `remove` returns the old value, a reinsert is an ordinary insert, and there is no tombstone surface anywhere to test. The block below pins that absence by behavior, the counts before and after.

```
    ok(m.remove("beta") == Some(2), "remove-beta");
    ok(m.get("beta").is_none(), "removed-misses");
    ok(m.len() == 2, "len-2");
    m.insert("beta".to_string(), 20);
    ok(m.get("beta") == Some(&20), "reinsert-beta");
    ok(m.len() == 3, "len-back-to-3");
    ok(m.remove("beta") == Some(20), "remove-again");
    ok(m.remove("beta").is_none(), "repeat-remove-refused");
    ok(m.len() == 2, "len-final");

    ok(m.get("alpha") == Some(&10), "alpha-survives");
    ok(m.get("gamma") == Some(&3), "gamma-survives");
```

listing 27: remove returns the value and is gone, reinsert is insert, no tombstone surface anywhere (source: icpc/samples-rust/src/Ch03/map.rs, lines 31-42)

```
    let keys: Vec<&String> = m.keys().collect();
    let mut sorted_keys = keys.clone();
    sorted_keys.sort();
    ok(keys == sorted_keys, "iteration-sorted");
    ok(keys.first().map(|s| s.as_str()) == Some("key0"), "first-key");
    ok(keys.last().map(|s| s.as_str()) == Some("key99"), "last-key");
```

listing 28: the determinism lane: iteration is key order, first and last read without sorting (source: icpc/samples-rust/src/Ch03/map.rs, lines 60-65)

The wider std collections story, including the hash map this chapter deliberately declines, belongs to book 3, ch 15 (rust 1.99, the standard library).

3.7. big integers, the same twelve limbs

Rust's standard library ships no bigint, and the zero-dependency lane bars the crate that would provide one, so the toolbox builds the same limbs C builds, a deliberate mirror rather than a private rederivation: sign-magnitude over 12 limbs of base one billion, least significant first, 108 decimal digits of headroom, zero positive at every exit. The anchors are the language-neutral ones, the 23-digit parse into 567890123, 678901234, 12345 from least to most significant, the carry fixture, `123456789 * 987654321` landing at `121932631112635269` across the limb boundary, and `10^21` over 7 leaving quotient `142857142857142857` and remainder 6.

The dry run: the carry fixture `9999999999999999999 + 1`, the same walk chapter 2 traces.

1. The parse cuts twenty nines into three limbs, 999999999, 999999999, 99, sign +1.
2. The magnitude add folds limb 0: 999999999 + 1 = 1000000000, one full base, the limb records 0 and carries 1. Limb 1 the same.

3. Limb 2: 99 + 1 = 100, under the base, no carry, three limbs still.
4. The print gives the top limb bare and pads the two below to nine digits, 3 + 9 + 9 = 21 characters, `100000000000000000000`, the pinned string.

```rust
#[derive(Clone, Copy)]
struct BigInt {
    sign: i32,
    n: usize,
    limb: [u32; LIMBS],
}

impl BigInt {
    fn zero() -> BigInt {
        BigInt {
            sign: 1,
            n: 1,
            limb: [0; LIMBS],
        }
    }

    fn trim(&mut self) {
        while self.n > 1 && self.limb[self.n - 1] == 0 {
            self.n -= 1;
        }
        if self.n == 1 && self.limb[0] == 0 {
            self.sign = 1;
        }
    }
```

listing 29: the bigint struct, sign-magnitude over 12 base-1e9 limbs, the trim that keeps zero positive (source: icpc/samples-rust/src/Ch03/bigint.rs, lines 16-39)

Multiply is the same schoolbook shape as C's, limb times limb into a `u64` column array, one carry pass at the end, and the base stays a billion for the same reason: a limb product stops near `10^18`, and `u64` holds several of those plus a carry with room left.

```
fn mul(a: &BigInt, b: &BigInt) -> BigInt {
    let mut acc = [0u64; LIMBS];
    for i in 0..a.n {
        for j in 0..b.n {
            if i + j < LIMBS {
                acc[i + j] += a.limb[i] as u64 * b.limb[j] as u64;
            }
        }
    }
    let mut r = BigInt::zero();
    let mut carry = 0u64;
    r.n = 0;
    for i in 0..LIMBS {
        let s = acc[i] + carry;
        r.limb[r.n] = (s % BASE) as u32;
        r.n += 1;
        carry = s / BASE;
    }
    r.sign = a.sign * b.sign;
    r.trim();
    r
}
```

listing 30: schoolbook multiply into a u64 column accumulator, one carry pass (source: icpc/samples-rust/src/Ch03/bigint.rs, lines 158-179)

The parse is where the twin diverges. C's `bi_parse` returns 0 on garbage, a status the caller checks, and rust's returns `Option`, `None` for the letter-bearing string, the bare minus, and the empty string, all three asserted. The 128-digit cap is shared: C refuses once its digit count passes 128, and rust at `rest.len() > 128`, a sanity ceiling above the 108 digits the twelve limbs hold. The limb cut keeps the least significant 108, so a 109-to-128-digit input parses as its low 108 digits, the same truncation both trees inherit from the shared budget.

```
fn parse(s: &str) -> Option<BigInt> {
    let mut sign = 1;
    let mut rest = s;
    if let Some(tail) = rest.strip_prefix('-') {
        sign = -1;
        rest = tail;
    } else if let Some(tail) = rest.strip_prefix('+') {
        rest = tail;
    }
    if rest.is_empty() || rest.len() > 128 {
        return None;
    }
    let bytes = rest.as_bytes();
    if bytes.iter().any(|b| !b.is_ascii_digit()) {
        return None;
    }
```

listing 31: parse as option: sign prefixes, the shared 128-digit cap, ascii digits only, None for garbage (source: icpc/samples-rust/src/Ch03/bigint.rs, lines 195-210)

Comparison and the signed routes are the C logic in rust's idiom, the magnitude compare limb count then limb values from the top, `Ordering` returned instead of a sign convention, addition of opposite signs routing through the magnitude subtract with the larger operand's sign, and subtraction as addition of a negated copy, a `Copy` struct so the negation is a stack byte copy rather than an allocation.

3.8. number theory with a native u128

This is the module where rust's story inverts C's. C's mulmod is add-and-double because the 128-bit product compiles and then fails to link, `__umodti3` absent from the MSVC plus lld lane. Go assembles the wide product from `bits.Mul64` halves. Java builds it from `Math.multiplyHigh` plus a shift-subtract lane. Rust widens into a type the machine and the language both carry, and the whole mulmod is one expression: widen both operands to `u128`, multiply, reduce, narrow. The suite pins the type itself before trusting it:

check	expression	pinned
square of max	`u64::MAX as u128` squared	340282366920938463426481119284349108225
mulmod at max	`num_mulmod(u64::MAX, u64::MAX, MOD)`	the direct u128 reduction
overflow refused	`(1 << 127).checked_add(1 << 127)`	`None`
widening boundary	`u64::MAX as u128 + 1`	`1 << 64`

The square is the 39-digit product no 64-bit sibling can form without help, the `checked_add` is the same overflow the plain operator would panic on under this lane's build, and the widening boundary is the arithmetic the narrowing rides back on. Four checks with `u128` in their names, and every one green means the type behaves at its edges.

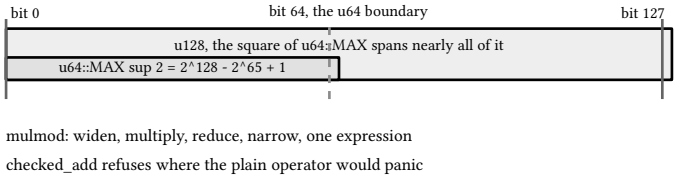


figure 31: the u128 register: the product of two 64-bit maxima fills it past the halfway line, the u64 boundary sits far left

```rust
fn num_mulmod(a: u64, b: u64, m: u64) -> u64 {
    ((a as u128 * b as u128) % m as u128) as u64
}

fn num_modpow(mut base: u64, mut exp: u64, m: u64) -> u64 {
    let mut r = 1 % m;
    base %= m;
    while exp != 0 {
        if exp & 1 != 0 {
            r = num_mulmod(r, base, m);
        }
        base = num_mulmod(base, base, m);
        exp >>= 1;
    }
    r
}
```

listing 32: mulmod is one line of native u128, modpow squares through it (source: icpc/samples-rust/src/Ch03/num.rs, lines 31-46)

`modpow` rides that mulmod at every step, and the anchor is the one every language in this book agrees on, `2^100 mod 1000000007` at 976371285, with the fermat lane `2^(MOD-1)` at 1 and the degenerate modulus 1 pinning 0. The inverse goes through extended euclid with a gcd guard that returns 0 on shared factors, the one sentinel refusal in this toolbox, and the 97 sweep cross-checks it against mulmod for every unit below the prime. The euclid chain and the bezout identity for 240 and 46, x = −9 and y = 47, are chapter 2's pins exactly, and the teaching derivation lives in book 10, ch 16 (practical data structures and algorithms, number theory and modular arithmetic).

The crt fold carries the wave's one caught defect. The pairwise combine needs the residue difference `(r2 mod m1) - (r1 mod m1)`, which is negative whenever the second residue is the smaller, and the plain `u64` form of the normalization panics on that underflow under the lane's overflow checks. The fix widens the normalization through `u128`, add `m1` before subtracting, reduce, narrow, and the blind review that caught it is the reason the wrapped-difference fixtures exist:

1. The classic fold: 2 mod 3 with 3 mod 5 combines to 8 mod 15, then with 2 mod 7 to 23 mod 105, the asserted pair.
2. Shared factors in agreement: 1 mod 6 with 3 mod 10 resolves to 13 mod 30. In conflict: refused.
3. The wrapped difference, crt(7, 10, 0, 3): the difference 0 - 7 is negative, the normalization reads (0 + 10 - 7) mod 10 = 3, and the scan from x = 7 by steps of 10 finds 27, 27 mod 3 = 0, the pair (27, 30). Its refusal twin, crt(9, 10, 0, 5), dies on the gcd test: 1 is not divisible by 5.

```rust
fn num_crt(r1: u64, m1: u64, r2: u64, m2: u64) -> Option<(u64, u64)> {
    let g = num_gcd(m1, m2);
    let lhs = (((r2 % m1) as u128 + m1 as u128 - (r1 % m1) as u128) % m1 as u128) as u64;
    if lhs % g != 0 {
        return None;
    }
    let l = num_lcm(m1, m2);
    let mut x = r1 % m1;
    while x < l {
        if x % m2 == r2 % m2 {
            return Some((x, l));
        }
        x += m1;
    }
    None
}
```

listing 33: the residue difference normalized through u128, the scan to the combined class, None on conflict (source: icpc/samples-rust/src/Ch03/num.rs, lines 80-95)

```rust
    let wide = u64::MAX as u128 * u64::MAX as u128;
    ok(wide == 340282366920938463426481119284349108225, "u128-square-of-max");
    ok(num_mulmod(u64::MAX, u64::MAX, MOD) == (wide % MOD as u128) as u64, "u128-mulmod-max");
    ok(
        (1u128 << 127).checked_add(1u128 << 127).is_none(),
        "u128-overflow-refused",
    );
    ok(u64::MAX as u128 + 1 == 1u128 << 64, "u128-widening-boundary");
```

listing 34: the showcase block: the square of max pinned in full, checked add refusing, the widening boundary (source: icpc/samples-rust/src/Ch03/num.rs, lines 154-161)

3.9. md5 and json leaves

The digest module is RFC 1321 again, the same K and S tables chapters 2 and 9 carry, little-endian Merkle-Damgard over 512-bit blocks, 64 rounds in four passes, and the same vector set including the 80-digit input whose digest `57edf4a22be3c955ac49da2e2107b67a` crosses the block boundary, plus determinism and avalanche, `abcd` against `abce`. The padding arithmetic differs in shape only: one expression, `((len + 8) / 64 + 1) * 64`, computes the padded length where C counted zeros to 56 mod 64, and on the 80 digits it gives (80 + 8) / 64 = 1, plus 1, times 64, 128 bytes, two blocks.

The round math is the tree's one deliberate wrap site. C's `uint32` wraps by definition and Lua masks to 32 bits after every operation, and rust must say the same thing out loud, `wrapping_add` at every add of the round and every fold into the chaining state, because the algorithm is defined on the wrap and a plain rustc build would panic on the first real input without the names.

```
        for i in 0..64u32 {
            let (f, g) = if i < 16 {
                ((b & c) | (!b & d), i as usize)
            } else if i < 32 {
                ((d & b) | (!d & c), ((5 * i + 1) % 16) as usize)
            } else if i < 48 {
                (b ^ c ^ d, ((3 * i + 5) % 16) as usize)
            } else {
                (c ^ (b | !d), ((7 * i) % 16) as usize)
            };
            let tmp = d;
            d = c;
            c = b;
            let sum = a.wrapping_add(f).wrapping_add(K[i as usize]).wrapping_add(w[g]);
            b = b.wrapping_add(sum.rotate_left(S[i as usize]));
            a = tmp;
        }
        h0 = h0.wrapping_add(a);
        h1 = h1.wrapping_add(b);
        h2 = h2.wrapping_add(c);
        h3 = h3.wrapping_add(d);
        off += 64;
    }
```

listing 35: the 64 rounds with wrapping adds, folded into the state, the one wrap site in the tree (source: icpc/samples-rust/src/Ch03/md5.rs, lines 59-81)

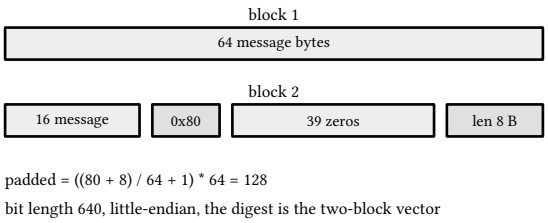


figure 32: the 80-digit input pads into two blocks: 64 message bytes, then 16 message, the pad, 39 zeros, and the length

The json module is a recursive-descent parser into an enum tree, `Null`, `Bool`, `Number`, `String`, `Array`, `Object`, `Option` through the whole parse, all-or-nothing with trailing content refused, so `{`, `[1, 2`, `tru`, and `1 2` all return `None`. Two bounds the C twin arrived at from the pool side arrive here from the recursion side: C's node pool is 128 nodes so depth is bounded by construction, and rust allocates freely, so the parser charges depth per container and refuses at 128, the same number, with the seam pinned both ways, 200-deep refused and 100-deep accepted.

The dry run: the crafted fixture is chapter 2's walk document, and the leaves fold the same sum.

1. `{"a": 1, "b": [2, 3.5, true, null, "x"], "c": {"d": -4}}` parses to a three-member object, keys `a`, `b`, `c` in order.
2. The array under `b` holds five children by index: 2, 3.5, `true`, `null`, `"x"`, order kept, and the escape fixture decodes quote, backslash, newline in place.
3. The variant counts read 4 numbers, 1 array, 2 objects, 1 boolean, 1 null, and the numeric-leaf sum folds 1 + 2 + 3.5 - 4 = 2.5, the pinned value the C and Lua walks also print.
4. The scalars parse alone, `42`, `true`, `null`, `"solo"`, the empty containers parse, and the depth seam holds: 100 brackets accepted, 200 refused.

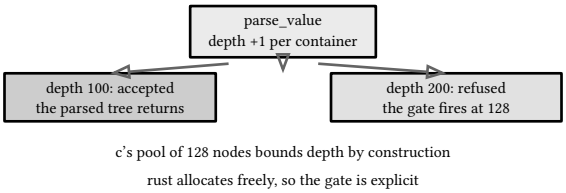


figure 33: the depth gate: recursion charges per container, 100 deep parses, 200 deep is refused at 128

```rust
    fn parse_number(&mut self) -> Option<f64> {
        let first = self.peek()?;
        if first != b'-' && !first.is_ascii_digit() {
            return None;
        }
        let start = self.pos;
        while let Some(b) = self.peek() {
            if b.is_ascii_digit() || matches!(b, b'-' | b'+' | b'.' | b'e' | b'E') {
                self.pos += 1;
            } else {
                break;
            }
        }
        std::str::from_utf8(&self.s[start..self.pos])
            .ok()?
            .parse()
            .ok()
    }
```

listing 36: the RFC-narrow number scan, ten characters, then the f64 parse (source: icpc/samples-rust/src/Ch03/json.rs, lines 75-92)

The number grammar is a reported divergence. The scanner admits the ten characters the RFC grammar uses, digits, minus, plus, dot, `e`, `E`, and the `f64` parse behind it refuses everything else, where C's `strtod` accepts the C locale's wider forms, hexadecimal floats among them. Numbers land in `f64`, the same 53-bit mantissa Go's `float64` leaves land in, so chapter 4's integer-honesty note applies here unchanged.

```rust
    fn parse_value(&mut self) -> Option<JNode> {
        if self.depth >= 128 {
            return None;
        }
        self.depth += 1;
        let v = self.parse_nested();
        self.depth -= 1;
        v
    }

    fn parse_nested(&mut self) -> Option<JNode> {
```

listing 37: the depth gate at 128, recursion charged per container, None past it (source: icpc/samples-rust/src/Ch03/json.rs, lines 94-104)

3.10. built here, cited elsewhere

Eleven files, 1362 non-blank lines, 278 checks, zero dependencies, and the split between what this toolbox builds and what it borrows:

piece	status	where
input over any reader	built, `io.rs`	full-buffer split, argv-gated locked stdin
token scanner	built, `scan.rs`	no c twin, go's `input.go` the sibling

piece	status	where
vector	std, asserted	growth amortized by contract, c pins exact doubling
deque	std, asserted	the wrap fixture 8, 9, 1 matches c's ring
heap	std plus adapter	reverse tuples break ties by value, c compares keys
map and set	std `BTreeMap`	sorted iteration pinned, no tombstone, c hand-rolls fnv
grid	built, `grid.rs`	the index law, degree sums 80 and 144
bigint	built, `bigint.rs`	std has none, crates barred, parse `Option` vs c's 0
mulmod, modpow, crt	built, `num.rs`	native u128, the residue normalized through u128
md5	built, `md5.rs`	rfc 1321, wrapping adds at the round math
json	built, `json.rs`	depth gate 128, rfc-narrow numbers vs strtod
size	1362 non-blank, non-comment	1523 raw lines, 11 files
dependencies	none	per-file rustc, no Cargo.toml, std only
the integer boundary	`u128` native	wrapping where c wraps, crt through u128, md5 the one wrap site

Chapter 2 and this one mirror module for module, and the divergences are named where they teach rather than hidden in the fixtures. Go's chapter 4 splits the same substrate the library way, thin wrappers over `math/big` and `math/bits`, and the Lua twin of chapter 9 builds everything C builds over tables. Java's chapter 5 sits at the opposite pole from all three: `BigInteger` ships, and the kit still builds the wide product from `Math.multiplyHigh` because the language stops at 64 bits. Rust is the pole where the wide lane is simply there.

Verified by `make verify-rust`, whose book-suite leg compiles and runs `books/icpc/samples-rust/src`: 11 files, 278 checks, rustc 1.99.0 pinned, bigint 26, deque 20, grid 33, heap 30, io 17, json 32, map 38, md5 9, num 34, scan 19, vec 20.

sources: doc.rust-lang.org, the standard library pages for `Vec`, `VecDeque`, `BinaryHeap`, `BTreeMap`, and `BTreeSet`, accessed 2026-10-14, the amortized-growth contract, the max-heap description, and the sorted-iteration guarantee read from the collections overview; R. Rivest, RFC 1321, April 1992, the K table, shift schedule, and test vectors this suite pins, the same reference chapters 2 and 9 build from. The overflow-checks behavior is the lane's own fact: an unoptimized rustc build aborts on arithmetic overflow, and the suite's green run, with `wrapping_` only at the md5 round math and the crt fix through `u128`, is the recorded proof. The C and Go comparisons cite the landed `books/icpc/samples-c/src/Ch02/` headers and `books/icpc/samples-go/` modules, read 2026-10-14.

4. the go toolbox

Go enters the contest with a small standard library that trusts the programmer: a heap package that ships an interface instead of a container, a big-integer package that never overflows, and not one puzzle-shaped helper anywhere. So this toolbox is the largest of the six, 399 non-blank lines over 7 files with 29 tests, and it is the one place where generics earn their keep, because a heap of `Coord` ordered by distance and a heap of `int` should share one implementation. The integer story: `int64` and `uint64` are the working types, products that can leave 64-bit range go through `math/bits.Mul64`, which returns the 128-bit product as two halves, and everything unbounded goes to `math/big` rather than hoping. Nothing wraps silently by accident.

4.1. a generic heap over container/heap

`container/heap` has not changed since it landed: it drives any type that implements five methods, and it keeps the ordering logic in the package while the data lives in yours. The adapter below is that contract, a slice plus a `less` function, with `Push` and `Pop` talking the package's `any`-typed protocol.

The dry run: `TestHeapOfStructsByField` in `heap_test.go` pushes four workers, zoe 2, mia 1, lea 2, ada 1, and asserts the drain against the pinned `want` slice.

1. Push zoe, priority 2, lands alone; push mia, priority 1, appends then sifts once, 1 under 2 swapping them to `[mia, zoe]`.
2. Push lea, priority 2, lands under the root's 1 and stays, `[mia, zoe, lea]`.
3. Push ada, priority 1, sifts twice: past zoe on the priority, then past mia on the name, `ada < mia`, into the root, `[ada, mia, lea, zoe]`.
4. The drain pops ada and mia, the two 1s, then lea before zoe, the 2s broken by name, exactly the asserted `want`.

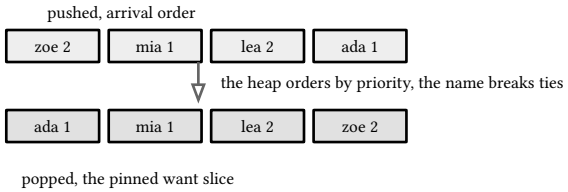


figure 34: four workers in arrival order, the drain order out, priority first and the name breaking ties

The drain runs ada, mia, lea, zoe, the order the test pins, and the listings below build the machinery that produces it.

```
func (h *heapAdapter[T]) Len() int { return len(h.items) }

func (h *heapAdapter[T]) Less(i, j int) bool {
  return h.less(h.items[i], h.items[j])
}

func (h *heapAdapter[T]) Swap(i, j int) {
  h.items[i], h.items[j] = h.items[j], h.items[i]
}

func (h *heapAdapter[T]) Push(x any) { h.items = append(h.items, x.(T)) }
func (h *heapAdapter[T]) Pop() any {
  last := h.items[len(h.items)-1]
  h.items = h.items[:len(h.items)-1]
  return last
}
```

listing 38: the interface dance, five methods over a slice and a less function (source: icpc/samples-go/heap.go, lines 16-31)

The typed front door hides it. `Heap[T]` is generic over any type with a caller-supplied ordering, so the same code is a min-heap of ints, a max-heap by reversed `less`, or a frontier ordered by path cost, and the type assertions `x.(T)` never leak past the adapter:

```
func (h *Heap[T]) Push(x T) {
  heap.Push(&h.inner, x)
}

// Pop removes and returns the smallest element under less. The bool
// is false when the heap is empty, so callers never invent zero
// values.
func (h *Heap[T]) Pop() (T, bool) {
  if h.inner.Len() == 0 {
    var zero T
    return zero, false
  }
  return heap.Pop(&h.inner).(T), true
}
```

listing 39: the generic front door, push and pop return real types (source: icpc/samples-go/heap.go, lines 46-60)

`Pop` returns `(T, bool)` so an empty heap reports itself instead of inventing a zero value, the same refusal the other toolboxes enforce. `Peek` and `Len` close the api. What the package does under these five methods, sift up, sift down, the textbook heap shape, is built from scratch in book 10, ch 8 (practical data structures and algorithms, heaps and priority queues), and the type-parameter machinery itself is the subject of book 4, ch 4 (go 1.27, generics and generic methods).

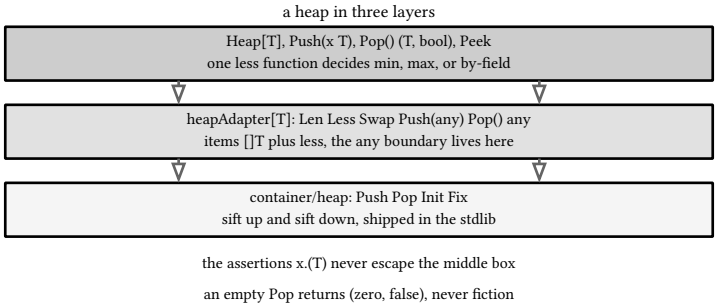


figure 35: three layers, the generic type on top, the any-typed adapter in the middle, the stdlib algorithm below

4.2. reading input the scanner way

Go has no `String.split` culture around files, it has `io.Reader` and `bufio.Scanner`, and the toolbox embraces that: every reader takes a reader. `ReadLines` drains it into one string per line and keeps every line it saw, blank or not, because the blank lines are paragraph boundaries and dropping them here would lose information the group splitter needs later.

The dry run: `TestSplitGroups` in `input_test.go` hands the splitter nine lines with three blanks and asserts the three groups, plus nil for a blanks-only page.

1. The fixture lines run `a1, a2, blank, b1, blank, blank, c1, c2, c3`, the shape `ReadLines` leaves behind.
2. The first group opens at a1 and takes a2, two lines.
3. The blank at index 2 flushes it; b1 opens the second group alone.
4. Two blanks follow: the first flushes b1's group, the second meets an empty pending group and opens nothing, so no empty group appears between them.
5. c1, c2, c3 fill the third group, and the return is the asserted three, `a1 a2`, `b1`, `c1 c2 c3`.
6. On two blanks alone every line is a seam, nothing ever pends, and the return is nil, the pinned refusal.

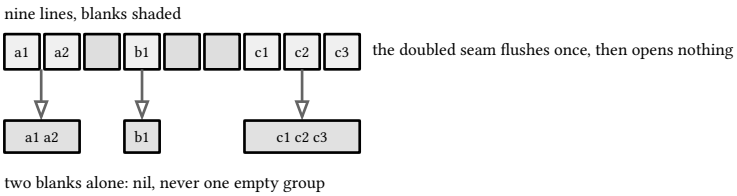


figure 36: nine lines, three seams, three groups: a blank that meets an empty pending group opens nothing

Nine lines fold to three groups, and a blanks-only page returns nil, in the listings below.

```go
func ReadLines(r io.Reader) ([]string, error) {
  scanner := bufio.NewScanner(r)
  var lines []string
  for scanner.Scan() {
    lines = append(lines, scanner.Text())
  }
  return lines, scanner.Err()
}

// ReadInts parses whitespace-separated integers, one or many per
// line. Contest inputs hand out columns of numbers as often as one
// number per line; this reads either.
```

listing 40: drain an io.reader, one string per line, blanks kept (source: icpc/samples-go/input.go, lines 14-25)

`ReadInts` is the column reader. Contest inputs hand out numbers one per line as often as several per line, so it splits each line on whitespace and parses field by field, failing loudly with the offending field in the error:

```go
func ReadInts(r io.Reader) ([]int, error) {
  scanner := bufio.NewScanner(r)
  var ints []int
  for scanner.Scan() {
    for _, field := range strings.Fields(scanner.Text()) {
      v, err := strconv.Atoi(field)
      if err != nil {
        return nil, fmt.Errorf("bad integer %q: %w", field, err)
      }
      ints = append(ints, v)
    }
  }
  return ints, scanner.Err()
}

// GridFromLines builds a byte grid: each line is a row, every
// character a cell. All rows must agree on length.
```

listing 41: whitespace-separated integers, one or many per line (source: icpc/samples-go/input.go, lines 26-42)

The test pins the honesty: input `alpha beta <blank> gamma <blank>` comes back as five lines including both trailing blanks, and `ReadInts` refuses a non-integer field instead of skipping it. That trailing-blank behavior is a deliberate difference from the C# toolbox, which trims the phantom last line at parse time, and it exists because `SplitGroups` consumes blanks as seams rather than noise.

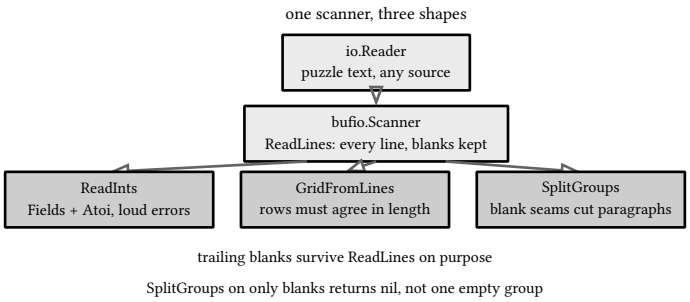


figure 37: one scanner, three downstream shapes

4.3. coordinates on the grid

A grid is a slice of equal-length rows plus coordinate arithmetic, and Go structs make that arithmetic explicit. `Coord` is row then column, `Index` flattens into a row-major offset, and the inverse `CoordAt` rebuilds the coordinate, with the test round-tripping every cell of a small grid and pinning (2, 3) at 5 columns to index 13.

The dry run: `TestNeighborsCornerOf3x3` in `grid_test.go` walks `Coord{0, 0}` on a 3 by 3 fixture under both delta sets and asserts the survivor lists in order.

1. The corner sits at row 0, column 0, where `InBounds` rejects any index below 0.
2. `Deltas4` offers its four steps in order: (-1, 0) and (0, −1) step outside and die, (0, 1) survives as `Coord{0, 1}` and (1, 0) as `Coord{1, 0}`, the asserted pair.
3. `Deltas8` offers eight, row by row: the three with row −1 all die, (0, −1) dies with them, (0, 1) joins, then of row +1 the (1, −1) dies while (1, 0) and (1, 1) join.
4. The 8-delta walk ends with `Coord{0, 1}`, `Coord{1, 0}`, `Coord{1, 1}`, three survivors of the eight offered.

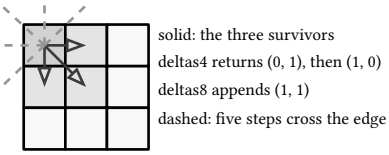


figure 38: the corner's eight offered steps: three survive, five cross the edge

Three of eight survive, the pinned corner lists, and the listings below hold the walk.

```go
func GridFromLines(lines []string) ([][]byte, error) {
  grid := make([][]byte, len(lines))
  for i, line := range lines {
    if len(line) != len(lines[0]) {
      return nil, fmt.Errorf("row %d has %d columns, want %d",
        i, len(line), len(lines[0]))
    }
    grid[i] = []byte(line)
  }
  return grid, nil
}

// SplitGroups cuts the lines into groups wherever a blank line
// appears. Paragraph-structured inputs become one []string each.
```

listing 42: equal-length rows become a byte grid, ragged input refused (source: icpc/samples-go/input.go, lines 43-56)

```go
// Deltas4 steps to the four orthogonal neighbors.
var Deltas4 = [4]Coord{{-1, 0}, {0, 1}, {1, 0}, {0, -1}}

// Deltas8 steps to every neighbor a chess king can reach.
var Deltas8 = [8]Coord{
  {-1, -1}, {-1, 0}, {-1, 1},
  {0, -1}, {0, 1},
  {1, -1}, {1, 0}, {1, 1},
}

// Neighbors returns the in-bounds coordinates reached by stepping
// deltas from c, in delta order.
func (c Coord) Neighbors(rows, cols int, deltas []Coord) []Coord {
  var out []Coord
  for _, d := range deltas {
    n := Coord{Row: c.Row + d.Row, Col: c.Col + d.Col}
    if n.InBounds(rows, cols) {
      out = append(out, n)
    }
  }
  return out
}
```

listing 43: the two delta sets and the in-bounds neighbor walk (source: icpc/samples-go/grid.go, lines 23-44)

`Deltas4` and `Deltas8` are the entire topology of grid puzzles, orthogonal steps for mazes and floods, all eight for visibility and diagonal movement, and `Neighbors` filters them through `InBounds` so every walk starts from a safe enumeration instead of an `if` ladder at each step. The center-cell test pins both neighborhoods exactly, four neighbors under `Deltas4`, eight under `Deltas8`, and a corner keeps only the three that exist.

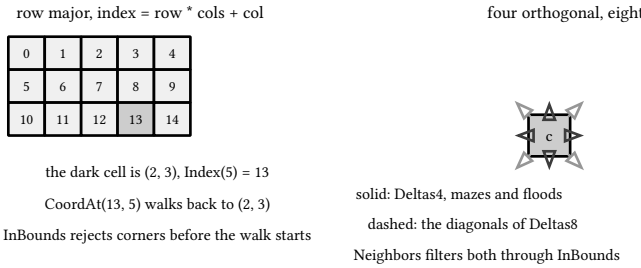


figure 39: row-major flattening and the two neighborhoods

4.4. the math/big wing

When a number will not fit 64 bits, Go's answer is not a trap but a type: `big.Int`, arbitrary-precision limbs with the same operator feel as the rest of the package library. The toolbox keeps thin wrappers so puzzle code says `BigPow` and never allocates limb slices by hand.

The dry run: `TestBigFactorialAndChoose` in `big_test.go` pins `BigChoose(52, 5)` at 2598960, the poker hands, and symmetry keeps k = 5 the shorter side against 52 - 5 = 47, so i runs 0 through 4.

1. i = 0: multiply by 52, divide by 1, running value 52.
2. i = 1: 52 x 51 = 2652, then ÷ 2 = 1326.
3. i = 2: 1326 x 50 = 66300, then ÷ 3 = 22100.
4. i = 3: 22100 x 49 = 1082900, then ÷ 4 = 270725.
5. i = 4: 270725 x 48 = 12994800, then ÷ 5 = 2598960, the pinned poker-hands count.
6. Every division lands exact, each running value itself a binomial coefficient, and the test cross-checks the same shape against the factorial identity at 30 choose 12.

i	x (52 - i)	÷ (i + 1)	running
0	52	1	52
1	51	2	1326
2	50	3	22100
3	49	4	270725
4	48	5	2598960

The fold ends at 2598960 with no factorial ever formed, and the listings below hold the wrapper.

```
func BigPow(base, exp int64) *big.Int {
  return new(big.Int).Exp(big.NewInt(base), big.NewInt(exp), nil)
}

// BigInverse inverts a modulo m, nil when no inverse exists. It is
// the raw big.Int shape; ModInv in num.go wraps it with an error.
```

listing 44: exp with no modulus, limbs grow as needed (source: icpc/samples-go/big.go, lines 11-16)

```
func BigChoose(n, k int) *big.Int {
  if k < 0 || k > n {
    return big.NewInt(0)
  }
  // symmetry: choose the shorter side
  if k > n-k {
    k = n - k
  }
  result := big.NewInt(1)
  for i := 0; i < k; i++ {
    result.Mul(result, big.NewInt(int64(n-i)))
    result.Div(result, big.NewInt(int64(i+1)))
  }
  return result
}

// ParseBig reads decimal, hexadecimal with 0x, and binary with 0b.
```

listing 45: the binomial coefficient, multiplicative form, symmetry first (source: icpc/samples-go/big.go, lines 32-48)

`BigChoose` is the tour's teaching moment: factorials are honest but the multiplicative form, multiply by (n - i), divide by (i + 1), working from the shorter side, keeps every intermediate an exact integer and an order of magnitude smaller. The tests pin the classics, 2^100 as its 31 digits, 10! as 3628800, 52 choose 5 as 2598960, the poker hands, and cross-check choose against the factorial identity. Parsing meets the same standard, decimal, `0x` hex, and `0b` binary all accepted, garbage refused, hex round-tripping through `BigToHex`:

```
func ParseBig(s string) (*big.Int, error) {
  v, ok := new(big.Int).SetString(strings.TrimSpace(s), 0)
  if !ok {
    return nil, fmt.Errorf("not a number: %q", s)
  }
  return v, nil
}

// BigToHex renders a big.Int as 0x-prefixed lowercase hex.
```

listing 46: decimal, hex, and binary in, 0x-prefixed hex out (source: icpc/samples-go/big.go, lines 49-57)

`BigInverse` wraps `big.Int.ModInverse` and returns nil when no inverse exists, gcd 2 for 2 mod 4 in the test, and `BigFactorial` completes the set. The from-scratch limb machinery, sign-magnitude arrays in base 1e9, is built and dissected in book 11, ch 2 (the icpc world finals, the c toolbox) and book 11, ch 9 (the icpc world finals, the lua toolbox), which need it because C and Lua ship nothing comparable.

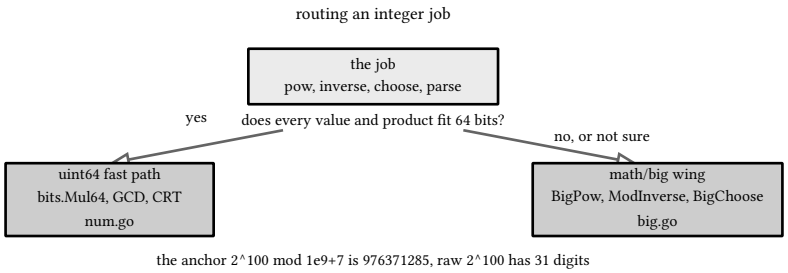


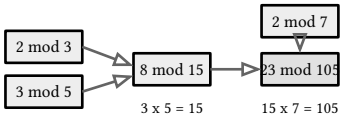
figure 40: routing an integer job, the 64-bit fast path against the unbounded wing

4.5. modular arithmetic through bits

The modular core runs on one observation: `math/bits.Mul64` multiplies two uint64 values and returns the 128-bit product as a high and a low half, and `bits.Div64` divides that pair back down. `MulMod` is therefore three lines, exact for any modulus, no 128-bit type needed.

The dry run: `TestCRTClassicAndRejection` in `num_test.go` folds the system 2 mod 3, 3 mod 5, 2 mod 7 and asserts the pair (23, 105), then both non-coprime branches.

1. The moduli 3 and 5 are coprime, so the first merge combines them into 3 x 5 = 15; the class 8 works, 8 = 2 + 3 x 2 and 8 = 3 + 5 x 1.
2. The second merge combines 15 with 7 into 15 x 7 = 105; the class 23 works, 23 = 8 + 15 x 1 and 23 = 21 + 2, and the test reads back (23, 105).
3. Shared factors in agreement: 5 mod 8 already says 1 mod 4, so the pair collapses to 5 mod 8, the lcm class the test pins.
4. Shared factors in conflict: 3 mod 8 says 3 mod 4, not 1, and the fold refuses the system.



agreeing shared factors: 5 mod 8 says 1 mod 4, merges to 5 mod 8

clashing: 3 mod 8 says 3 mod 4, the system is refused

figure 41: the fold: two merges land the pinned class, and a shared-factor pair either collapses or is refused

The fold ends at the pinned pair, 23 with modulus 105, in the listings below.

```
func MulMod(a, b, m uint64) uint64 {
  hi, lo := bits.Mul64(a, b)
  _, r := bits.Div64(hi, lo, m)
  return r
}

// ModPow is square-and-multiply: O(log exp) modular multiplications.
func ModPow(base, exp, m uint64) uint64 {
  if m == 1 {
    return 0
  }
  result := uint64(1)
  base %= m
  for exp > 0 {
    if exp&1 == 1 {
      result = MulMod(result, base, m)
    }
    base = MulMod(base, base, m)
    exp >>= 1
  }
  return result
}

// ModInv inverts a modulo m through math/big, which handles any
// modulus, not just primes. No inverse exists when a and m share a
// factor.
```

listing 47: mulmod through the 128-bit halves, then square-and-multiply (source: icpc/samples-go/num.go, lines 35-60)

`ModPow` rides `MulMod` at every step, so squaring never truncates, and the suite pins the cross-language anchor, 2^100 mod 1e9+7 is 976371285. The inverse goes through `big.Int.ModInverse`, wrapped with a gcd guard so shared factors report an error instead of a nil dereference, and that choice buys composite moduli for free, where Fermat's little theorem would only work on primes.

```
func CRT(cs []Congruence) (x, combined int64, err error) {
  if len(cs) == 0 {
    return 0, 0, fmt.Errorf("no congruences to combine")
  }
  x = ((cs[0].Rem % cs[0].Mod) + cs[0].Mod) % cs[0].Mod
  combined = cs[0].Mod
  for _, c := range cs[1:] {
    rem := ((c.Rem % c.Mod) + c.Mod) % c.Mod
    g := GCD(combined, c.Mod)
    if (rem-x)%g != 0 {
      return 0, 0, fmt.Errorf("%d mod %d and %d mod %d disagree on %d",
        x, combined, rem, c.Mod, g)
    }
    lcm := combined / g * c.Mod
    // solve combined*t = rem-x (mod c.Mod): divide through by g,
    // then invert combined/g modulo c.Mod/g (coprime now, so the
    // inverse always exists)
    step := c.Mod / g
    inv, err := ModInv(combined/g, step)
    if err != nil {
      return 0, 0, fmt.Errorf("inverting %d mod %d: %w", combined/g, step, err)
    }
    t := ((inv % step) * ((rem - x) / g % step)) % step
    if t < 0 {
      t += step
    }
    x = ((x+combined*t)%lcm + lcm) % lcm
    combined = lcm
  }
  return x, combined, nil
}
```

listing 48: the crt fold, shared factors must agree or the system is refused (source: icpc/samples-go/num.go, lines 80-110)

CRT folds congruences pairwise: coprime moduli combine into their product, shared factors merge through the lcm when the residues agree and are refused when they do not, with the disagreement reported naming both statements. The test pins the classic, 2 mod 3, 3 mod 5, 2 mod 7 folds to 23 mod 105, exactly the value the C# and JavaScript toolboxes pin, and the teaching derivation of every piece used here lives in book 10, ch 16 (practical data structures and algorithms, number theory and modular arithmetic).

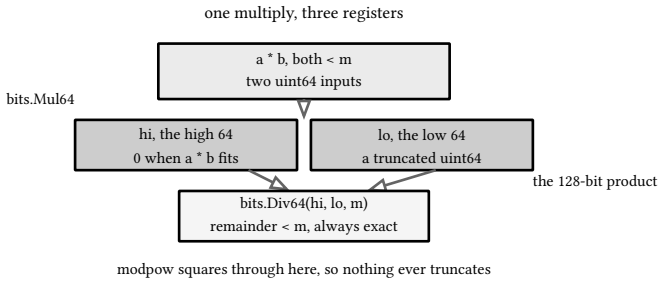


figure 42: the 128-bit product split into halves, divided back to a remainder

4.6. digests and json leaves

Two contest staples are one-liners once the imports are right, and the toolbox keeps them that way. MD5 is broken as security and the contest does not care, the puzzles use it as a deterministic scrambler, so MD5Hex is crypto/md5 plus encoding/hex with a prefix helper for the mining shape, first n hex characters compared against zeros.

The dry run: `TestDecodeStruct` in `json_test.go` unmarshals the passport fixture through the field tags and asserts every mapped field.

1. The document carries id `p7`, issued 2020, tags `north` and `east`, a nested validity with year 2024 and ok true, and one `extra` key no struct field declares.
2. `DecodeStruct` maps tag to field: `p.ID` reads `p7` and `p.Issued` reads 2020.
3. `Tags` lands as a slice of 2, with `Tags[1]` the pinned `east`.
4. `Validity` nests one level down, `Year` 2024 and `OK` true, both asserted.
5. The `extra` key finds no destination and is dropped, and the sibling test refuses garbage with an error instead of guessing.

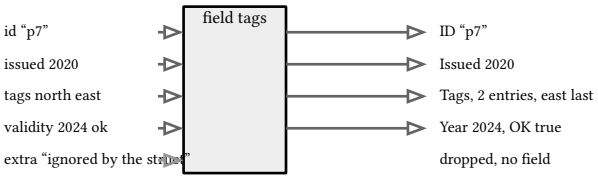


figure 43: the passport through the field tags: four keys map, the undeclared one drops

The struct fills, id `p7`, issued 2020, tags 2, validity 2024, and the listings below hold both halves of the section.

```
func MD5Hex(s string) string {
  sum := md5.Sum([]byte(s))
  return hex.EncodeToString(sum[:])
}

// MD5HexBytes is MD5Hex for a byte slice.
```

listing 49: the scrambler, digest to lowercase hex, prefix sliced (source: icpc/samples-go/digest.go, lines 11-16)

The digest tests pin the standard vectors, the empty string and `abc`, the same constants the from-scratch C and Lua implementations pin, so the three toolboxes cross-check each other, and `MD5Prefix("abc", 5)` is 90015. The json half carries one fact that must be stated out loud: `encoding/json` decodes every number into `float64`, so a schema-free walk over decoded values sees floats at the leaves, integers included:

```
func SumNumbers(v any) float64 {
  switch t := v.(type) {
  case float64:
    return t
  case bool, string, nil:
    return 0
  case map[string]any:
    total := 0.0
    for _, child := range t {
      total += SumNumbers(child)
    }
    return total
  case []any:
    total := 0.0
    for _, child := range t {
      total += SumNumbers(child)
    }
    return total
  }
  return 0
}
```

listing 50: the recursive walk, numeric leaves at any depth, bool string nil contribute zero (source: icpc/samples-go/json.go, lines 44-64)

`SumNumbers` recurses through maps and arrays and adds every numeric leaf, and the test pins the honesty of the walk on a crafted document: scores 1, 2, 3 and a deeply nested 4.5 sum to 10.5, while a `true` flag and a null contribute nothing. For typed shapes the same package fills structs through field tags, `DecodeStruct` maps the passport fixture field by field, and the wider json story, including the v2 api, belongs to book 4, ch 13 (go 1.27, stdlib tour 2: json/v2, uuid, net/http, crypto, simd, os, io, testing).

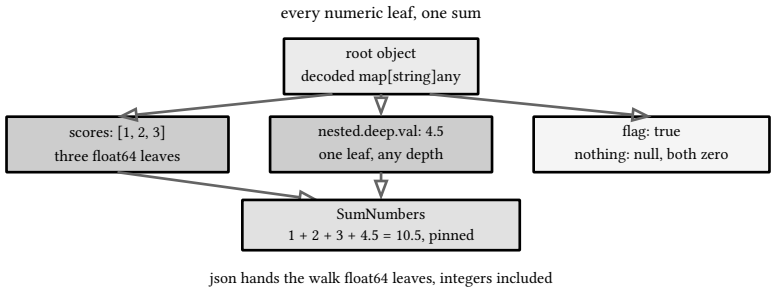


figure 44: the walk sums float64 leaves, the boolean and null are zeros

4.7. built here, cited elsewhere

Seven files, 399 non-blank lines, 29 tests, and the split between what this toolbox owns and what the standard library already did well:

piece	status	where
generic heap	built, `heap.go`	over `container/heap`, sift logic in book 10, ch 8 (practical data structures and algorithms, heaps and priority queues)
input readers	built, `input.go`	5 tests, blanks kept on purpose
coords, deltas, neighbors	built, `grid.go`	4 tests, index round trip pinned
big integer wrappers	built, `big.go`	over `math/big`, from-scratch limbs in book 11, ch 2 (the icpc world finals, the c toolbox)
mulmod, modpow, crt	built, `num.go`	`math/bits` halves, inverse via `ModInverse`
md5	cited	`crypto/md5`, from-scratch in chapters 2 and 9
json decode and walk	built, `json.go`	over `encoding/json`, typed shapes via tags
gcd, lcm, sieve	built where puzzle-shaped	full treatment in book 10, ch 16 (practical data structures and algorithms, number theory and modular arithmetic)

Java's chapter 5 splits the same way this one does: thin over the library, `Math.multiplyHigh` standing where `math/bits.Mul64` stands here, the wide product built from library halves rather than refused or delegated.

sources: pkg.go.dev, `container/heap`, `math/bits`, `math/big`, `crypto/md5`, and `encoding/json` package pages, accessed 2026-09-14. Contest context from the ICPC Foundation, icpc.global world finals problem pages, accessed 2026-09-14. Sample behavior verified by `make verify-go` in `books/icpc/samples-go`, 29 tests in the toolbox module (heap 3, input 5, grid 4, num 4, big 4, digest 4, json 5).

5. the java toolbox

Java enters the contest between go and c# in the corpus reading order, and its toolbox is the seventh. The standard library is deep enough that nothing here hand-builds a container: the heap, the ring deque, the hash map, and the balanced tree all ship, arbitrary precision ships as `BigInteger`, and the work is fluency plus two measured disciplines. One is input: `Scanner` is banned on judge inputs because its regex machinery costs 1.4 to 1.6 times the buffered pair over a 4 MB fixture on a quiet box, 2.4 to 2.8 under load, so the reader is `BufferedReader` under a word-configured `StreamTokenizer` and the writer is a `StringBuilder` flushed by one write. The other is the 64-bit boundary: products that can leave long range travel as `Math.multiplyHigh` plus the low half with a shift-subtract remainder, the same lane the dsa book opened, and `BigInteger` appears exactly where c writes `__int128`. The kit is 772 non-blank lines over 6 files carrying 50 ok-line checks, every number in this chapter measured on this machine under the pinned oracle jdk
27.

5.1. fast input and output

The reader is two objects deep. `BufferedReader` amortizes the syscalls into 8 KB characters, and a `StreamTokenizer` over it, configured word-style, splits on whitespace and hands every token back as a string in `sval`. The configuration is the whole trick: by default the tokenizer parses digit runs into the double `nval`, and a double loses the low bits of any long past `2^53`, so the kit calls `resetSyntax`, declares digits, the minus sign, and letters to be word characters, and parses numbers itself through `Long.parseLong`. `Scanner` looks like the same job and is not: every `hasNextLong` runs its delimiter regex over the stream.

The dry run: `Fastio` builds a 4154731-byte fixture of 420000 numbers, ten per line, from the corpus lcg, then reads it both ways.

1. Both readers return the identical checksum, the sum of all 420000 values, pinned against the value tracked while generating.
2. Warm, then the minimum of 3 timed passes, three fresh JVMs, on a quiet box: Scanner 95.5 to 114.8 ms against the buffered pair's 65.9 to 70.9 ms.
3. The ratio holds at 1.4 to 1.6x: the extra 35 ms per input is real wall clock, multiplied by every case file a finals submission reads, and the gap widens under load, the same fixture measuring 2.4 to 2.8x while four sibling suites compile on the same machine.

fixture	tokens	scanner	buffered	verdict
4154731 bytes, lcg	420000	95.5 to 114.8 ms	65.9 to 70.9 ms	1.4 to 1.6x quiet, 2.4 to 2.8x loaded, Scanner banned

The writer is the mirror discipline, the one-write law: accumulate everything into a `StringBuilder`, convert once, write once, flush once. Interactive problems are the exception the year chapters handle with an explicit flush per turn, never by abandoning the builder.

```
static final class In {
  final StreamTokenizer st;

  In(InputStream src) {
    st = new StreamTokenizer(
        new BufferedReader(new InputStreamReader(src, StandardCharsets.US_ASCII)));
    st.resetSyntax();
    st.wordChars('0', '9');
    st.wordChars('-', '-');
    st.wordChars('a', 'z');
    st.wordChars('A', 'Z');
    st.wordChars('_', '_');
    st.whitespaceChars(0, ' ');
  }
```

listing 51: the word config, digits and minus and letters become words, nval never fires (source: icpc/samples-java/src/Ch04/Fastio.java, lines 38-51)

```
  String nextToken() throws IOException {
    if (st.nextToken() == StreamTokenizer.TT_EOF)
      return null;
    return st.sval;
  }

  boolean hasNext() throws IOException {
    if (st.nextToken() == StreamTokenizer.TT_EOF)
      return false;
    st.pushBack();
    return true;
  }

  long nextLong() throws IOException {
    String t = nextToken();
    if (t == null)
      throw new EOFException("nextLong at end of stream");
    return Long.parseLong(t);
  }
```

listing 52: nextToken nulls at TT_EOF, hasNext peeks through pushBack, longs parse exactly (source: icpc/samples-java/src/Ch04/Fastio.java, lines 53-71)

```
// the one-write writer: everything appends, flush writes once
static final class Out {
  final StringBuilder sb = new StringBuilder();

  Out append(long v) {
    sb.append(v).append('\n');
    return this;
  }

  Out append(String s) {
    sb.append(s).append('\n');
    return this;
  }

  int flush(OutputStream dst) throws IOException {
    byte[] bytes = sb.toString().getBytes(StandardCharsets.US_ASCII);
    dst.write(bytes);
    dst.flush();
    return bytes.length;
  }
}
```

listing 53: the one-write writer, one StringBuilder, one flush, byte count known (source: icpc/samples-java/src/Ch04/Fastio.java, lines 81-101)

The timed main pins the law in checks rather than in prose: both readers must return the checksum, and the one-write flush must carry exactly the builder's bytes:

```
public static void main(String[] args) throws IOException {
  long[] sum = new long[1];
  byte[] bytes = fixture(sum);
  ok(bytes.length == 4154731, "the fixture is 4154731 bytes, about 4 MB");
  ok(drainBuffered(bytes) == sum[0], "buffered pair reads every token, checksum pins");
  ok(drainScanner(bytes) == sum[0], "scanner agrees on the checksum");
  // warm both readers, then time min of 3 each
  drainBuffered(bytes);
  drainScanner(bytes);
  double tb = minOf(3, () -> drainBuffered(bytes));
  double ts = minOf(3, () -> drainScanner(bytes));
  System.out.printf("scanner %.1f ms against buffered %.1f ms over %d tokens, %.1fx%n",
      ts, tb, TOKENS, ts / tb);
  // the one-write law: one flush, one byte array, byte count known
  Out out = new Out();
  out.append(42).append("done");
  int written = out.flush(new ByteArrayOutputStream());
  ok(written == 8 && out.sb.length() == 8, "one write carries the whole output");
  ok(out.sb.toString().equals("42\ndone\n"), "the writer content pins");
```

listing 54: the timed comparison and the one-write check, min of 3 after warmup (source: icpc/samples-java/src/Ch04/Fastio.java, lines 157-176)

5.2. the token scanner discipline

Three contracts live in `Tokens.java`. The first is the `nval` trap stated as a check: the default syntax reads 9007199254740993, which is 2^53 + 1, and hands back 9007199254740992, the double one ulp below it. The

word config reads the same token through `Long.parseLong` and spans the full signed range, `Long.MAX_VALUE` and `Long.MIN_VALUE` round-tripping exactly.

The dry run: the mixed-type fixture `3 alpha -7 12 beta`, then the nine-line group fixture the go toolbox pins.

1. The count-first loop reads 3, then consumes exactly three typed fields, a word length and two numbers, summing 5, −7, 2.
2. `pushBack` rewinds one token, the peek idiom `hasNext` is built on, and the third read past the last token returns `TT_EOF`.
3. Nine lines with three blanks fold to three groups, `a1 a2`, `b1`, `c1 c2 c3`, the doubled blank opening nothing.
4. Two blanks alone fold to zero groups, never one empty group, and a page with no trailing blank still flushes its last group.

token	default nval, cast to long	word config
9007199254740993	9007199254740992, one ulp low	exact, 2^53 + 1
9223372036854775807	9.223372036854778e18, saturating cast	exact by parse
−9223372036854775808	−9.223372036854778e18, saturating cast	exact by parse

```
  static double defaultNval(String token) throws IOException {
    StreamTokenizer st = new StreamTokenizer(new StringReader(token));
    st.nextToken();
    return st.nval;
  }

  // the word-configured reader, the Fastio.In discipline restated
  static String[] readTokens(String text) throws IOException {
    StreamTokenizer st = new StreamTokenizer(new StringReader(text));
    st.resetSyntax();
    st.wordChars('0', '9');
    st.wordChars('-', '-');
    st.wordChars('a', 'z');
    st.wordChars('A', 'Z');
    st.wordChars('_', '_');
    st.whitespaceChars(0, ' ');
    List<String> out = new ArrayList<>();
    while (st.nextToken() != StreamTokenizer.TT_EOF)
      out.add(st.sval);
    return out.toArray(new String[0]);
  }
```

listing 55: the trap demonstrated on the default syntax, then the word config that avoids it (source: icpc/samples-java/src/Ch04/Tokens.java, lines 22-43)

```
static List<List<String>> splitGroups(List<String> lines) {
  List<List<String>> groups = new ArrayList<>();
  List<String> pending = new ArrayList<>();
  for (String line : lines) {
    if (line.isBlank()) {
      if (!pending.isEmpty())
        groups.add(pending);
      pending = new ArrayList<>();
    } else {
      pending.add(line);
    }
  }
  if (!pending.isEmpty())
    groups.add(pending);
  return groups;
}

static int checks = 0;

static void ok(boolean cond, String name) {
```

listing 56: blank lines are seams, maximal non-blank runs become groups (source: icpc/samples-java/src/Ch04/Tokens.java, lines 56-75)

Blank-line groups are the shape the 2019 chapter's board pairs and the 2022 chapter's multi-test blocks parse with, and `readLines` keeps the blanks because the splitter needs them, the same refusal to lose information the go toolbox states. The wider io story, channels and streams and encodings, belongs to book 5, ch 14 (java 27, io and networking).

5.3. big integers, the 128-bit roads

Java is the only toolbox in this set with arbitrary precision in the standard library, so the c `__int128` sites, and the javascript `BigInt` sites beside them, map to `BigInteger` directly. The contest law is narrower: long arithmetic everywhere it fits, the 128-bit product as `Math.multiplyHigh` plus the wrapped low half, and the remainder through a 128-step unsigned shift-subtract loop. That loop is the dsa book's mulmod lane restated for contest shape, built there with `BigInteger.mod` as referee and the same anchor, 2^100 mod 1e9 + 7 = 976371285.

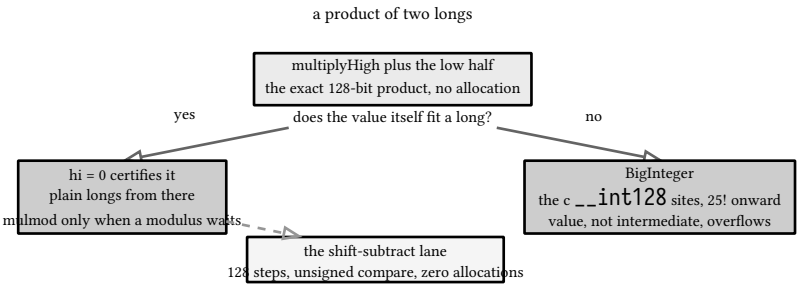


figure 45: routing a product past 64 bits: the fast lane when hi is zero, the shift-subtract lane for mulmod, BigInteger when the value itself overflows

The dry run: `Big.java` runs the roads against each other.

1. `modpow(2, 100, 1e9+7)` returns 976371285 and `BigInteger.TWO.pow(100).mod(...)` agrees, the anchor every language pins.
2. 200 lcg pairs under 1e9+7: the shift-subtract `mulmod` matches the `BigInteger` referee on every one.
3. `multiplyHigh` plus the low half rebuilds 123456789 x 987654321 exactly, the low half joining through its unsigned decimal, because `or`-ing a negative `BigInteger` is the infinite-sign trap.

4. `hi = 0` certifies a product that fit, and plain longs take over.
5. The unsigned laws: `parseUnsignedLong` reads `2^64 - 1` into a negative long, `toUnsignedString` prints it back, `remainderUnsigned` divides the past-`2^63` long correctly, and a `compareUnsigned` comparator puts `2^64 - 1` after 5.

```
static long mulHigh(long a, long b, long[] lo) {
  lo[0] = a * b;
  return Math.multiplyHigh(a, b);
}

// bit i of the 128-bit (hi, lo) pair
static long bit(long hi, long lo, int i) {
  return i ≥ 64 ? (hi >>> (i - 64)) & 1 : (lo >>> i) & 1;
}

// the contest mulmod: 128-step unsigned shift-subtract over the
// (hi, lo) product, every intermediate an unsigned long below 2 * m
static long mulmod(long a, long b, long m) {
  long[] lo = new long[1];
  long hi = mulHigh(a, b, lo);
  long r = 0;
  for (int i = 127; i ≥ 0; i--) {
    r = (r << 1) | bit(hi, lo[0], i);
    if (Long.compareUnsigned(r, m) ≥ 0)
      r -= m;
  }
  return r;
}
```

listing 57: multiplyHigh plus the low half, then the 128-step unsigned shift-subtract remainder (source: icpc/samples-java/src/Ch04/Big.java, lines 19-42)

```
static long modpow(long b, long e, long m) {
  long r = 1 % m;
  b %= m;
  while (e ≠ 0) {
    if ((e & 1) ≠ 0)
      r = mulmod(r, b, m);
    b = mulmod(b, b, m);
    e >>= 1;
  }
  return r;
}
```

listing 58: modpow rides mulmod, squaring never truncates (source: icpc/samples-java/src/Ch04/Big.java, lines 43-53)

```
    // the unsigned laws: parse, print, remainder, order
    long max = Long.parseUnsignedLong("18446744073709551615");
    ok(max == -1L, "parseUnsignedLong reads 2^64-1 into a negative long");
    ok(Long.toUnsignedString(max).equals("18446744073709551615"),
        "toUnsignedString prints it back");
    long seven = Long.parseUnsignedLong("18000000000000000000");
    ok(Long.remainderUnsigned(seven, 7) == new BigInteger("18000000000000000000")
        .mod(BigInteger.valueOf(7)).longValue(),
        "remainderUnsigned divides the >2^63 long correctly");
    ok(Long.compareUnsigned(-1L, 1L) > 0, "compareUnsigned sorts >2^63 above 1");
    Long[] ordered = {-1L, 5L};
    java.util.Arrays.sort(ordered, Long::compareUnsigned);
    ok(ordered[0] == 5L && ordered[1] == -1L,
        "an unsigned comparator puts 2^64-1 last, 5 first");
    // the fast path: multiplyHigh == 0 certifies the product fit in the
```

listing 59: the unsigned laws as checks, parse, print, divide, order (source: icpc/samples-java/src/Ch04/Big.java, lines 104-118)

The from-scratch limb machinery the c and lua toolboxes build, because their libraries ship nothing, is chapter material here too: book 11, ch 2 (the icpc world finals, the c toolbox) carries the base-1e9 limb dissection and book 11, ch 16 (the icpc world finals, fast big arithmetic) ports it to java in this wave. The number theory underneath the anchor belongs to book 10, ch 16 (practical data structures and algorithms, number theory and modular arithmetic), and the language-level treatment of the type lives in book 5, ch 4 (java 27, types and values).

5.4. collections mapping

The mapping is one-to-one because the library did the work: `PriorityQueue` is the binary heap behind c's hand-rolled one, comparator-injected; `ArrayDeque` is stack, queue, and sliding window in one ring; `HashMap` counts; `TreeMap` iterates sorted with successor queries. Two laws come with them. The heap is not stable, so equal priorities drain in arbitrary order unless a sequence counter rides in the comparator, the same counter idiom the python toolbox pins. And boxed collections cost: sorting 200000 values runs 0.11 to 0.12 ms on `int[]` against 0.36 ms on `Integer[]`, about 3x, so hot paths take primitive arrays.

The dry run: `Coll.java` drains its heaps and windows.

1. The min-heap of tasks zoe 2, mia 1, lea 2, ada 1 drains ada, mia, lea, zoe, priority first with the name breaking ties.
2. `Comparator.reverseOrder()` makes the max-heap: 3, 1, 4, 1, 5 yields the top three 5, 4, 3.
3. Four equal-priority tasks drain a, b, c, d under the seq counter, first-in-first-out, the order the bare heap does not promise.
4. The window over 4, 2, 6, 1, 5 tracks the running max 4, 4, 6, 6, 6.
5. Two equal `Pt(2, 3)` records collapse to one map entry at value 9, while the array-keyed map misses the lookup entirely, identity equality against value equality.

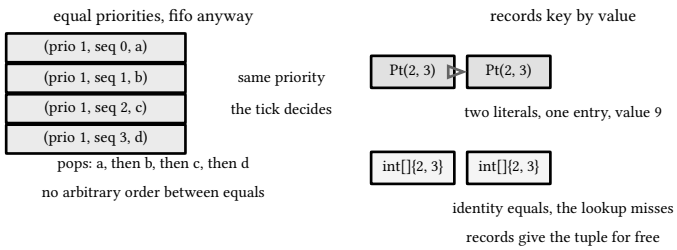


figure 46: equal priorities, the sequence decides: the bare heap leaves equal elements in arbitrary order, the stamped comparator restores arrival order

```
    PriorityQueue<Task> pq = new PriorityQueue<>(
        Comparator.comparingInt(Task::prio).thenComparing(Task::name));
    pq.add(new Task("zoe", 2));
    pq.add(new Task("mia", 1));
    pq.add(new Task("lea", 2));
    pq.add(new Task("ada", 1));
    List<String> drain = new ArrayList<>();
    while (!pq.isEmpty())
      drain.add(pq.poll().name());
    ok(drain.equals(Arrays.asList("ada", "mia", "lea", "zoe")),
        "the min-heap drains ada, mia, lea, zoe, priority then name");
    // the max-heap: one reversed() away
    PriorityQueue<Integer> max = new PriorityQueue<>(Comparator.reverseOrder());
    for (int v : new int[] {3, 1, 4, 1, 5})
      max.add(v);
    List<Integer> top = new ArrayList<>();
    while (top.size() < 3)
      top.add(max.poll());
    ok(top.equals(Arrays.asList(5, 4, 3)), "reversed order makes the max-heap");
    // the tie-breaker: equal priorities drain fifo only with a seq
```

listing 60: the min-heap by record fields, reversed order for the max-heap (source: icpc/samples-java/src/Ch04/Coll.java, lines 40-59)

```
    // counter in the comparator, the heap itself is not stable
    record Stamped(int prio, int seq, String name) {}
    PriorityQueue<Stamped> fifo = new PriorityQueue<>(
        Comparator.comparingInt(Stamped::prio).thenComparingInt(Stamped::seq));
    String[] names = {"a", "b", "c", "d"};
    int seq = 0;
    for (int i = 0; i < 4; i++)
      fifo.add(new Stamped(1, seq++, names[i]));
    List<String> order = new ArrayList<>();
    while (!fifo.isEmpty())
      order.add(fifo.poll().name());
    ok(order.equals(Arrays.asList(names)),
        "the seq counter keeps equal priorities first-in-first-out");
```

listing 61: the seq counter, equal priorities drain first-in-first-out (source: icpc/samples-java/src/Ch04/Coll.java, lines 60-72)

```
        && tm.higherKey(3) == null, "treeMap first and higher walk 1 2 3, higher(3) null");
    // records are the tuple: value equality, hashable, one line
    record Pt(int x, int y) {}
    Map<Pt, Integer> grid = new HashMap<>();
    grid.put(new Pt(2, 3), 7);
    grid.put(new Pt(2, 3), 9);
    grid.put(new Pt(3, 2), 5);
    ok(grid.size() == 2 && grid.get(new Pt(2, 3)) == 9,
        "records key by value, two equal pts one entry");
    // the array-key pitfall records fix: identity equality
    Map<int[], Integer> bad = new HashMap<>();
    bad.put(new int[] {2, 3}, 7);
    ok(bad.get(new int[] {2, 3}) == null,
        "array keys compare by identity, the lookup misses");
    // arrays over boxed collections in hot paths: functional equality,
    // the timing print below carries the measured gap. both legs warm
    // first, then alternate so jit warmth cannot pick the winner
```

listing 62: records key by value, array keys by identity, the pitfall closed (source: icpc/samples-java/src/Ch04/Coll.java, lines 106-122)

The heap under `PriorityQueue`, sift up and sift down, is built from scratch in book 10, ch 8 (practical data structures and algorithms, heaps and priority queues), and the full container tour, including the streams this kit does not need, is book 5, ch 12 (java 27, collections and streams). Records themselves are book 5, ch 5 (java 27, classes and objects) material.

5.5. the judge invocation and the jvm facts

Every java solver in the year chapters carries one main with two modes. An argument means judge mode: read the file at `args[0]`, solve, print the answer exactly as the judge expects, one single write, no ok lines. Bare means self-check mode: the same solve core over crafted fixtures, one `ok N name` line per passing check, exit 1 on the first failure. The harness compiles the chapter directory once with the pinned jdk under `tools/jdk27` and invokes `java -cp build/ChNN Class case.in` per case, byte-comparing stdout against the `.ans` file, so the one-write law is the whole output contract.

The dry run: `Judge.java` demonstrates both legs on a sum-the-input core.

1. The bare leg runs the core through string readers: 7 −2 5 to 10, 2^63 - 1 exactly, the empty case to 0.
2. The argv leg writes a temp file `100 200 300 -50`, reads it back through the same `solveSum`, and prints 550.
3. The proof goes one level deeper: the file spawns `java -cp ... Judge tmp.in` as a real process, and the captured stdout is 550 and nothing else, no ok lines, exit 0.

```
static long solveSum(BufferedReader in) throws IOException {
  int n = Integer.parseInt(in.readLine().trim());
  long sum = 0;
  for (int i = 0; i < n; i++)
    sum += Long.parseLong(in.readLine().trim());
  return sum;
}

static String judgeOutput(String path) throws IOException {
  try (BufferedReader in = Files.newBufferedReader(Path.of(path), StandardCharsets.US_ASCII)) {
    return Long.toString(solveSum(in));
  }
}
```

listing 63: the solve core every mode shares, count line then longs (source: icpc/samples-java/src/Ch04/Judge.java, lines 24-37)

```
public static void main(String[] args) throws Exception {
  if (args.length == 1) {
    // judge mode: one write, the answer, nothing else
    System.out.write(judgeOutput(args[0]).getBytes(StandardCharsets.US_ASCII));
    System.out.flush();
    return;
  }
  // the same core through string readers
  ok(solveSum(new BufferedReader(new StringReader("3\n7\n-2\n5\n"))) == 10,
```

listing 64: the dual-mode dispatch, one argument means judge mode (source: icpc/samples-java/src/Ch04/Judge.java, lines 49-57)

```
        .toString();
    ProcessBuilder pb = new ProcessBuilder(javaExe, "-cp",
        System.getProperty("java.class.path"), "Judge", tmp.toString());
    pb.redirectErrorStream(true);
    Process proc = pb.start();
    String out = new String(proc.getInputStream().readAllBytes(), StandardCharsets.US_ASCII);
    proc.waitFor();
    Files.deleteIfExists(tmp);
    ok(proc.exitValue() == 0 && out.trim().equals("550"),
        "a spawned judge run prints 550 and nothing else");
    ok(!out.contains("ok 1"), "judge mode prints no ok lines");
    // the fast-io shape a real year solver uses: tokens, not lines
    try (BufferedReader br = new BufferedReader(
        new StringReader("2 3 4"))) {
      java.io.StreamTokenizer st = new java.io.StreamTokenizer(br);
      st.resetSyntax();
      st.wordChars('0', '9');
      st.wordChars('-', '-');
```

listing 65: the process-level proof, a spawned judge run byte-compared (source: icpc/samples-java/src/Ch04/Judge.java, lines 74-91)

The JVM facts ride the same file's sibling `Jvm.java`, measured here, never estimated:

probe	command	measured
bare startup	`java -version`, median of 5	53 ms
default stack	recurse to overflow, child jvm	22774 frames
deep stack	`-Xss512m`, same probe	33506905 frames
collectors	the GarbageCollectorMXBeans	G1 Young, Concurrent, Old
serial fallback	`-XX:+UseSerialGC` on the churn	52 ms against 51 ms

```
  static String spawn(List<String> extra, String mode) throws Exception {
    List<String> cmd = new ArrayList<>();
    cmd.add(javaExe());
    cmd.addAll(extra);
    if (mode != null) {
      cmd.add("-cp");
      cmd.add(System.getProperty("java.class.path"));
      cmd.add("Jvm");
      cmd.add(mode);
    } else {
      cmd.add("-version"); // the bare jvm floor, version print and exit
    }
    ProcessBuilder pb = new ProcessBuilder(cmd);
    pb.redirectErrorStream(true);
    Process proc = pb.start();
    String out = new String(proc.getInputStream().readAllBytes(), StandardCharsets.US_ASCII);
    proc.waitFor();
    if (proc.exitValue() != 0)
      throw new IllegalStateException("spawn failed: " + out);
    return out;
  }
```

listing 66: the spawn helper, child jvms so parent frames never pollute the counts (source: icpc/samples-java/src/Ch04/Jvm.java, lines 37-57)

```
    double startup = spawnMedianMs(new ArrayList<>(), null, 5);
    System.out.printf("java -version median %.0f ms%n", startup);
    ok(startup > 20 && startup < 2000, "a bare jvm starts in well under two seconds");
    // stack: default depth against -Xss512m, measured in child jvms so
    // the parent's own frames never pollute the count
    int defDepth = Integer.parseInt(spawn(new ArrayList<>(), "depth").trim());
    int bigDepth = Integer.parseInt(
        spawn(new ArrayList<>(List.of("-Xss512m")), "depth").trim());
    System.out.printf("recursion depth: %d default, %d under -Xss512m%n", defDepth, bigDepth);
    ok(defDepth > 5000, "the default stack still holds thousands of frames");
    ok(bigDepth > defDepth * 4, "-Xss512m lifts the ceiling several-fold");
    // the collector names the run actually uses
    StringBuilder names = new StringBuilder();
    for (GarbageCollectorMXBean b : ManagementFactory.getGarbageCollectorMXBeans())
      names.append(b.getName()).append(' ');
    System.out.printf("collectors: %s%n", names.toString().trim());
```

listing 67: the probes as checks, startup, depth, collectors (source: icpc/samples-java/src/Ch04/Jvm.java, lines 110-125)

Startup at 53 ms means the JVM appears once per case file and the budget holds. The default stack dies at 22774 frames, so the deep dfs families that recurse per cell take `-Xss512m`, worth three orders of magnitude here. The default collector reports as G1 through the MXBeans and `-XX:+UseSerialGC` is the fallback: parity on this box's churn, 52 against 51 ms, single-threaded and lower-footprint where the default's threads contend. The deeper runtime story, memory model and GC mechanics, is book 5, ch 11 (java 27, memory and runtime) the jdk pin itself is book 5, ch 1 (java 27, toolchain and the jdk).

5.6. built here, cited elsewhere

Six kit files, 772 non-blank lines, 50 checks, and the split between what this toolbox owns and what the standard library already ships:

piece	status	where
buffered reader, one-write writer	built, `Fastio.java`	5 checks, Scanner measured and banned
token discipline, group seams	built, `Tokens.java`	9 checks, the nval trap pinned
128-bit product roads	built, `Big.java`	12 checks, `multiplyHigh` over the shift-subtract lane
heap, deque, maps, records	cited	`PriorityQueue` and friends, 11 checks in `Coll.java`
dual-mode judge demo	built, `Judge.java`	8 checks, the spawned-run proof
jvm facts	built, `Jvm.java`	5 checks, startup, stack, collectors measured

The comparison against the other six toolboxes: c and lua hand-build heaps, deques, big integers, and md5 because their libraries ship none of it, go wraps `container/heap` and `math/big` behind generics, c# leans on `System.Numerics` and LINQ, javascript writes its own heap and ring, and python imports its way past all of it. Java's kit is the second lightest after python's, and its two hand-built pieces are disciplines rather than containers: the measured input pair and the measured invocation contract.

sources: docs.oracle.com envelope pages for `StreamTokenizer`, `Scanner`, `BigInteger`, `Math.multiplyHigh`, the unsigned `Long` family, `PriorityQueue`, `ArrayDeque`, `HashMap`, `TreeMap`, and records in the java 27 api specification, accessed 2026-10-06. Contest context from the ICPC Foundation, icpc.global world finals problem pages, accessed 2026-10-06. Sample behavior verified by `pwsh -NoProfile -File tools/run-java-samples.ps1 -SampleRoot books/icpc/samples-java/src -Chapter Ch04`, 50 checks across the 6 kit files (Fastio 5, Tokens 9, Big 12, Coll 11, Judge 8, Jvm 5).

6. the c# toolbox

Every finals lands the same four jobs: read the input, walk a grid, queue work at both ends, do arithmetic past the range a machine word pretends to have. C# starts from an unusual position here because the base class library already ships most of it, so this toolbox builds only what the BCL genuinely lacks and cites the rest by chapter. The integer story: `long` is the working type, signed 64-bit, with `checked` arithmetic where a silent wrap would poison an answer and `System.Numerics.BigInteger` waiting one overload away when products leave long range. Nothing in this suite allocates a file handle, every parse runs over strings the caller already owns, and the whole toolbox is 324 non-blank lines across 4 files with 26 tests behind it.

6.1. input, the shapes puzzles ship in

Puzzle text arrives in four shapes: lines, scattered integers, character grids, and blank-line separated groups. One reader feeds all four, and the reader is deliberately boring, split on newlines, trim the carriage returns, drop the phantom empty line a trailing newline would invent.

The dry run: `InputTests` pins the reader's edge behavior and the signed scan, `1,-2\n3 -4` and the range `2-4,6-8` among the asserts.

1. `Lines("a\r\nb\nc\n")` splits to four pieces, trims the `\r`, and drops the empty tail: a, b, c, asserted.
2. `Lines("a\n\nb")` keeps the interior blank, and `Lines("\r\n")` is one genuinely blank line where `Lines("")` is none, all three asserted.
3. The scan opens `1,-2\n3 -4`: the `1` yields 1, the minus before the `2` has a digit after it and no digit or letter before it, so it signs, −2, then 3 and −4, the asserted four.
4. On `2-4,6-8` both minuses sit between digits and read as dashes: 2, 4, 6, 8, asserted.
5. `x=-17; y=3` yields −17 then 3, and `1000000000000` parses as one long, past int range, asserted.

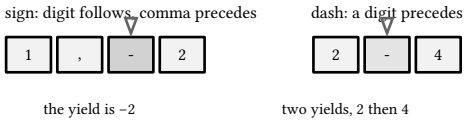


figure 47: the minus rule at character level: a sign needs a digit after and nothing alphanumeric before, a dash has a digit before it

The scan yields the pinned 1, −2, 3, −4 and reads the range as four numbers, and the listings below are the reader and the scanner.

```
    /// <summary>
    /// Lines split on '\n' with trailing '\r' trimmed. A single trailing
    /// newline does not produce a phantom empty line; interior blank lines
    /// are kept for Groups.
    /// </summary>
    public static List<string> Lines(string text)
    {
        var lines = new List<string>();
        foreach (string raw in text.Split('\n'))
            lines.Add(raw.TrimEnd('\r'));
        if (lines.Count > 0 && lines[^1] == "")
            lines.RemoveAt(lines.Count - 1);
        return lines;
    }

    /// <summary>
    /// Every integer in the text, sign only where a minus can legally start one.
    /// </summary>
    public static List<long> Longs(string text) => Scan(text).ToList();

    public static List<int> Ints(string text) =>
        Scan(text).Select(v => (int)v).ToList();

    /// <summary>The non-empty lines as character rows.</summary>
    public static string[] Grid(string text) =>
        Lines(text).Where(line => line.Length > 0).ToArray();
```

listing 68: the line reader and the three shapes it feeds (source: icpc/samples/src/Input.cs, lines 10-35)

The interesting function is the signed number scanner underneath `Longs`. Contest inputs hide minus signs in prose (`x=-17`), in ranges (`2-4` is two numbers, not a subtraction), and in hyphenated noise, so a minus only starts a number when a digit follows it and no digit or letter precedes it:

```
    private static IEnumerable<long> Scan(string text)
    {
        int i = 0;
        while (i < text.Length)
        {
            bool digitNext = i + 1 < text.Length && char.IsDigit(text[i + 1]);
            bool prevAlnum = i > 0 && char.IsLetterOrDigit(text[i - 1]);
            bool signHere = text[i] == '-' && digitNext && !prevAlnum;
            if (char.IsDigit(text[i]) || signHere)
            {
                int start = i;
                i++; // past the first digit or the sign
                while (i < text.Length && char.IsDigit(text[i]))
                    i++;
                yield return long.Parse(text[start..i]);
            }
            else
                i++;
        }
    }
```

listing 69: the signed scan, a minus is a sign only in sign position (source: icpc/samples/src/Input.cs, lines 56-75)

The tests pin the trap cases: `1,-2\n3 -4` parses to 1, −2, 3, −4, the range `2-4,6-8` yields 2, 4, 6, 8 because those minuses sit between digits, and `1000000000000` survives because the scan yields `long`, never `int`. `Groups` splits on blank lines and keeps interior blanks per group, which is what paragraph-shaped inputs actually need.

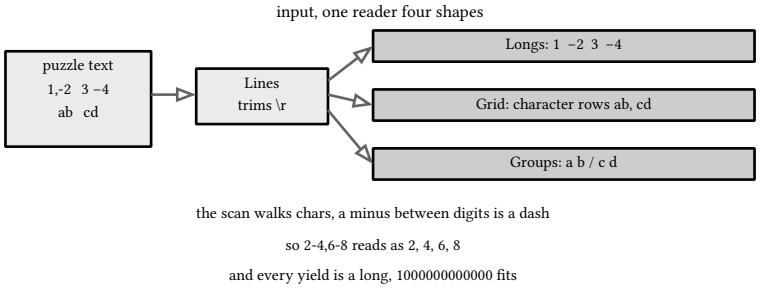


figure 48: one reader, four shapes, the scanner decides what a minus means

6.2. the grid vector

Grid puzzles want a two-component integer that adds, subtracts, scales, and hashes by content. A `readonly record struct` gives all of it at declaration: value semantics, structural equality, and a free hash code that `HashSet` and `Dictionary` accept without ceremony, the same ground book 10, ch 6 (practical data structures and algorithms, hashing) builds from scratch to teach what a hash set is.

The dry run: `VecTests` pins the component arithmetic, the dot and manhattan family, and the quarter-turn cycle on (3, −7).

1. The component asserts: (2, 3) + (1, −1) = (3, 2) and (2, 5) - (1, 1) = (1, 4), both asserted.
2. The dot product of (2, 3) with (4, −1) folds 2 x 4 = 8 and 3 x (-1) = −3, 8 - 3 = 5, asserted, and the length squared of (3, 4) reads 9 + 16 = 25.
3. Manhattan: (2, −3) reads 2 + 3 = 5, and `ManhattanTo` from (-2, −3) to (1, 1) splits 1 - (-2) = 3 over and 1 - (-3) = 4 up, 3 + 4 = 7, asserted.
4. Left maps (x, y) to (-y, x): (1, 0) turns to (0, 1) and (0, 1) to (-1, 0); right maps to (y, -x), (1, 0) to (0, −1), all asserted.
5. Four lefts on (3, −7) cycle (7, 3), (-3, 7), (-7, −3), and back to (3, −7), the asserted identity.

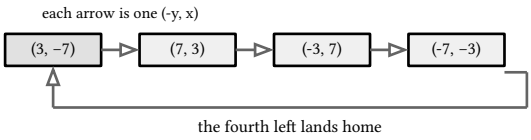


figure 49: the four quarter turns on one vector: each left applies (-y, x), the fourth lands home

Four quarter turns land where they started, (3, −7), and the listings below are the operators that move it.

```
    public static Vec operator +(Vec a, Vec b) => new(a.X + b.X, a.Y + b.Y);

    public static Vec operator -(Vec a, Vec b) => new(a.X - b.X, a.Y - b.Y);

    public static Vec operator *(Vec v, long k) => new(v.X * k, v.Y * k);

    public static Vec operator *(long k, Vec v) => v * k;

    public static Vec operator -(Vec v) => new(-v.X, -v.Y);

    public long Dot(Vec other) => X * other.X + Y * other.Y;

    public long LengthSquared => X * X + Y * Y;

    /// <summary>Grid distance |dx| + |dy|.</summary>
    public long Manhattan => Math.Abs(X) + Math.Abs(Y);

    public long ManhattanTo(Vec other) => (this - other).Manhattan;
```

listing 70: arithmetic plus manhattan distance, value semantics for free (source: icpc/samples/src/Vec.cs, lines 6-23)

Turns are where hand-rolled vectors earn their keep. `R`, `U`, `L`, `D` instructions from wiring and walking puzzles become one multiply-free step: a quarter turn counter-clockwise maps (x, y) to (-y, x), clockwise to (y, -x), and composing those two covers every 90-degree facing a puzzle asks for:

```
    /// <summary>Quarter turn counter-clockwise: (1, 0) becomes (0, 1).</summary>
    public Vec RotateLeft => new(-Y, X);

    /// <summary>Quarter turn clockwise: (1, 0) becomes (0, -1).</summary>
    public Vec RotateRight => new(Y, -X);

    /// <summary>Up, down, left, right.</summary>
    public static readonly Vec[] Deltas4 =
    {
        new(0, -1), new(0, 1), new(-1, 0), new(1, 0),
    };

    /// <summary>The four above plus the diagonals.</summary>
    public static readonly Vec[] Deltas8 =
    {
        new(0, -1), new(0, 1), new(-1, 0), new(1, 0),
        new(-1, -1), new(1, -1), new(-1, 1), new(1, 1),
    };
```

listing 71: quarter turns both ways, then the two neighbor delta sets (source: icpc/samples/src/Vec.cs, lines 25-42)

The tests pin the pairings, turn (1, 0) left for (0, 1), right for (0, −1), and the manhattan family: from (2, 3) to (-1, 5) the distance is 5, three over plus two up, which is the metric every maze step cost assumes.

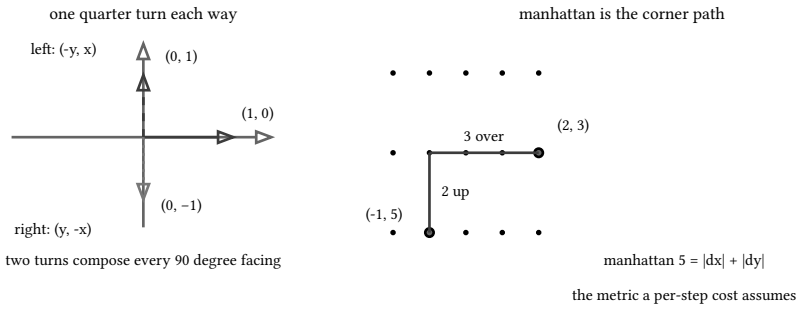


figure 50: quarter turns and the manhattan corner path

6.3. the deque the bcl never shipped

Here is the one real hole. The BCL has `Queue<T>` and `Stack<T>` and, since .NET 6, `PriorityQueue<TElement, TPriority>`, whose chapter is book 10, ch 11 (practical data structures and algorithms, shortest paths). It has no both-end deque, and the contest wants one constantly, sliding windows, worklists that grow at the front, spinners. So the toolbox builds the ring deque that book 10, ch 5 (practical data structures and algorithms, stacks, queues, deques) teaches from first principles, minus the lecture, plus production manners.

The dry run: `RingTests`' rotate fixture walks `abcd` through five asserted states, the step normalization visible in each.

1. The four pushes land `a`, `b`, `c`, `d` at slots 0 through 3, head 0, count 4.
2. `Rotate(1)` computes ((1 % 4) + 4) % 4 = 1, one PopFront-PushBack pair moves the `a`, and `ToList` reads `b c d a`, asserted.
3. `Rotate(-1)` reads the C# remainder −1 % 4 = −1, so ((-1 % 4) + 4) % 4 = 3, three pairs move b, c, d and land back at `a b c d`, asserted.
4. `Rotate(4)` computes ((4 % 4) + 4) % 4 = 0 and runs no pairs, the identity, and `Rotate(0)` the same, both asserted.
5. `Rotate(-5)` normalizes ((-5 % 4) + 4) % 4 = 3, three pairs again, landing `d a b c`, the last asserted state.

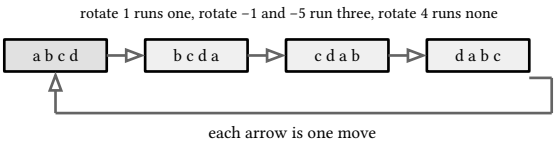


figure 51: the rotation cycle of the fixture: each arrow is one pop-front push-back pair, the normalization picks how many run

The last state lands `d a b c` off three moves, and the listings below are the ring that turns it.

```
    public void PushBack(T item)
    {
        Ensure(_count + 1);
        _items[Wrap(_head + _count)] = item;
        _count++;
    }

    public void PushFront(T item)
    {
        Ensure(_count + 1);
        _head = Wrap(_head - 1);
        _items[_head] = item;
        _count++;
    }

    public T PopFront()
    {
        RequireNonEmpty();
        T item = _items[_head];
        _items[_head] = default!;
        _head = Wrap(_head + 1);
        _count--;
        return item;
    }

    public T PopBack()
    {
        RequireNonEmpty();
        int tail = Wrap(_head + _count - 1);
        T item = _items[tail];
        _items[tail] = default!;
        _count--;
        return item;
    }
```

listing 72: push and pop at both ends, underflow refuses (source: icpc/samples/src/Ring.cs, lines 17-50)

Refusal is a design decision worth stating. An empty pop that returned `default` would invent a zero that flows into an answer silently, so both pops and `Rotate` throw `InvalidOperationException` on an empty ring. The wraparound test walks the seam exactly: fill four slots, pop the front, push a back value into the freed slot, push a front value over the seam, and the contents stay ordered.

```
    private void Ensure(int needed)
    {
        if (needed ≤ _items.Length)
            return;
        int grown = Math.Max(4, _items.Length * 2);
        while (grown < needed)
            grown *= 2;
        var fresh = new T[grown];
        for (int i = 0; i < _count; i++)
            fresh[i] = _items[Wrap(_head + i)]; // growth rebases the ring at slot 0
        _items = fresh;
        _head = 0;
    }

    private void RequireNonEmpty()
    {
        if (_count == 0)
            throw new InvalidOperationException("ring deque is empty");
    }
```

listing 73: doubling growth rebases the ring at slot zero (source: icpc/samples/src/Ring.cs, lines 71-89)

Growth is where naive rings break. Doubling allocates the fresh array and copies elements back to slot 0 in logical order, so `_head` resets and the modulo arithmetic restarts from a clean base, exactly the rebase the dsa chapter pins with its capacity 4, 8, 16, 32 sequence. `Rotate` closes the suite by moving the front to the back one step at a time after normalizing the step count into range, the shape circular-buffer puzzles reach for when a full turn must be the identity.

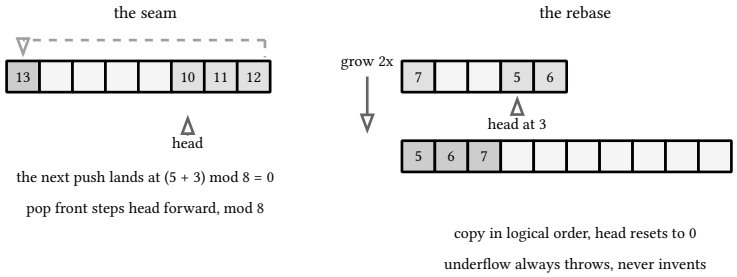


figure 52: the seam and the rebase, two moments a ring must survive

6.4. modular arithmetic, long and biginteger

Modular arithmetic is the backbone of contest math, and the BCL meets it halfway: `BigInteger.ModPow` exists and is cited here rather than rebuilt, while the `long` fast path, the modular inverse, and the chinese remainder fold are worth owning. The suite pins the same anchor the other five languages pin, 2^100 mod 1e9+7 is 976371285, and cross-checks every `long` modpow against the BCL overload.

The dry run: `NumTests` pins the modpow anchor, the inverse identities, and the classic crt fold with its arithmetic.

1. The anchor: `ModPow(2, 100, Prime)` returns 976371285 and equals `BigInteger.ModPow(2, 100, Prime)`, the cross-check assert, with the loop edges pinned, exponent 0 returning 1 and exponent 1 returning the base.
2. `ModInv(3, Prime)` is 333333336, and the asserted product reads 3 x 333333336 = 1000000008 = 1000000007 + 1, one over the modulus.
3. `ModInv(7, 26)` is 15: 7 x 15 = 105 and 105 - 4 x 26 = 1; `ModInv(3, 11)` is 4 with 3 x 4 = 12 = 11 + 1.
4. `ModInv(4, 8)` finds gcd(4, 8) = 4 and throws, the asserted refusal.
5. The crt fold: 2 mod 3 with 3 mod 5 combines to 8 mod 15, then 8 + 15 = 23 at modulus 15 x 7 = 105, the asserted pair, and the consistent non-coprime merge lands 5 mod 12.

6. Past long range, 2^40 against 3^30, the long fold throws `OverflowException` where the biginteger fold returns the product modulus with both congruences verified.

step	line	value
anchor	`ModPow(2, 100, Prime)`	976371285
inverse	3 x 333333336	1000000008, one over m
inverse	7 x 15	105 = 4 x 26 + 1
inverse	3 x 4	12 = 11 + 1
fold	(2, 3), (3, 5), (2, 7)	23 mod 105
boundary	2^40 against 3^30	long throws, biginteger returns

Every inverse reads one over its modulus and the fold lands 23 mod 105, and the listings below are the loops that compute them.

```
    public static long ModPow(long b, long e, long m)
    {
        if (m ≤ 0)
            throw new ArgumentOutOfRangeException(
                nameof(m), "modulus must be positive");
        if (e < 0)
            throw new ArgumentOutOfRangeException(
                nameof(e), "exponent must be non-negative");
        long result = 1;
        b %= m;
        if (b < 0)
            b += m;
        while (e > 0)
        {
            if ((e & 1) == 1)
                result = result * b % m;
            b = b * b % m;
            e >>= 1;
        }
        return result;
    }

    /// <summary>The BCL already carries the unbounded version.</summary>
    public static BigInteger ModPow(BigInteger b, BigInteger e, BigInteger m) ⇒
        BigInteger.ModPow(b, e, m);
```

listing 74: square and multiply on long, then the bcl biginteger version (source: icpc/samples/src/Num.cs, lines 12-36)

The inverse goes through extended Euclid rather than Fermat, because puzzle moduli are composite as often as prime. `ModInv(3, 1000000007)` is 333333336, `ModInv(7, 26)` is 15, and a non-coprime pair throws with the gcd in the message:

```
public static long ModInv(long a, long m)
{
    (long g, long x, _) = ExtGcd(a % m, m);
    if (g != 1)
        throw new InvalidOperationException(
            $"{a} has no inverse mod {m}, gcd is {g}");
    long inv = x % m;
    return inv < 0 ? inv + m : inv;
}

public static BigInteger ModInv(BigInteger a, BigInteger m)
{
    (BigInteger g, BigInteger x, _) = ExtGcd(a % m, m);
    if (g != 1)
        throw new InvalidOperationException(
            $"{a} has no inverse mod {m}, gcd is {g}");
    BigInteger inv = x % m;
    return inv.Sign < 0 ? inv + m : inv;
}
```

listing 75: inverse via extended euclid, long and biginteger overloads (source: icpc/samples/src/Num.cs, lines 39-57)

The CRT fold is the piece that makes giant-cycle puzzles tractable, and it carries the chapter's sharpest integer lesson. The `long` version runs under `checked`, so when a combined modulus leaves long range it fails with `OverflowException` instead of wrapping, and the answer is to switch to the `BigInteger` overload, same fold, unbounded limbs:

```
public static (long Remainder, long Modulus) Crt(
    IEnumerable<(long R, long M)> items)
{
    IEnumerator<(long R, long M)> it = items.GetEnumerator();
    if (!it.MoveNext())
        throw new ArgumentException(
            "at least one congruence required", nameof(items));
    (long r, long m) = it.Current;
    while (it.MoveNext())
    {
        (long r2, long m2) = it.Current;
        long g = Gcd(m, m2);
        if ((r2 - r) % g != 0)
            throw new InvalidOperationException(
                $"no solution: {r} mod {m} and {r2} mod {m2} disagree mod {g}");
        long step = m2 / g;
        (_, long p, _) = ExtGcd(m / g, step); // (m/g)*p = 1 mod step
        long t = (r2 - r) / g % step * p % step;
        if (t < 0)
            t += step;
        long lcm = checked(m / g * m2);
        r = checked(r + m * t) % lcm;
        m = lcm;
    }
    return (r, m);
}
```

listing 76: the checked long fold, consistent non-coprime moduli merge (source: icpc/samples/src/Num.cs, lines 64-89)

```
    public static (BigInteger Remainder, BigInteger Modulus) Crt(
        IEnumerable<(BigInteger R, BigInteger M)> items)
    {
        IEnumerator<(BigInteger R, BigInteger M)> it = items.GetEnumerator();
        if (!it.MoveNext())
            throw new ArgumentException(
                "at least one congruence required", nameof(items));
        (BigInteger r, BigInteger m) = it.Current;
        while (it.MoveNext())
        {
            (BigInteger r2, BigInteger m2) = it.Current;
            BigInteger g = BigInteger.GreatestCommonDivisor(m, m2);
            if ((r2 - r) % g != 0)
                throw new InvalidOperationException(
                    $"no solution: congruences disagree mod {g}");
            BigInteger step = m2 / g;
            (_, BigInteger p, _) = ExtGcd(m / g, step);
            BigInteger t = (r2 - r) / g % step * p % step;
            if (t.Sign < 0)
                t += step;
            BigInteger lcm = m / g * m2;
            r = (r + m * t) % lcm;
            m = lcm;
        }
        return (r, m);
    }
```

listing 77: the biginteger fold for moduli past long range (source: icpc/samples/src/Num.cs, lines 92-117)

The classic fixture agrees everywhere: congruences 2 mod 3, 3 mod 5, 2 mod 7 fold to 23 mod 105, consistent non-coprime pairs merge through the lcm, contradictions throw. The overflow test uses 2^40 against 3^30, whose lcm sits near 2.3e26: the long fold throws, the biginteger fold returns with both congruences verified, and the difference between those two outcomes is the entire argument for keeping both overloads. The teaching version of this material, square-and-multiply derived, binary gcd, the small-primes machinery, lives in book 10, ch 16 (practical data structures and algorithms, number theory and modular arithmetic).

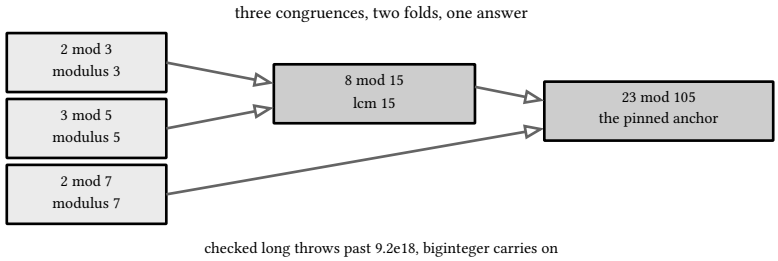


figure 53: the crt fold, pairwise combine grows the modulus toward the lcm

6.5. built here, cited elsewhere

The chapter's contract in one table. Four files, 324 non-blank lines, 26 tests, and a strict rule for what did not get rebuilt:

piece	status	where
input shapes, signed scan	built, `Input.cs`	5 tests, minus-in-sign-position rule
2D vector, turns, deltas	built, `Vec.cs`	6 tests, value-type equality

piece	status	where
both-end ring deque	built, `Ring.cs`	7 tests, teaching build in book 10, ch 5 (practical data structures and algorithms, stacks, queues, deques)
priority queue	cited	book 10, ch 11 (practical data structures and algorithms, shortest paths), `PriorityQueue` since .NET 6
hash sets, dictionaries	cited	book 10, ch 6 (practical data structures and algorithms, hashing), record struct hashing rides the BCL
modpow, modinv, crt	built, `Num.cs`	8 tests, `BigInteger.ModPow` cross-checks
unbounded integers	cited	book 10, ch 16 (practical data structures and algorithms, number theory and modular arithmetic), `System.Numerics.BigInteger`

The from-scratch bigint and md5 stories live in book 11, ch 2 (the icpc world finals, the c toolbox) and book 11, ch 9 (the icpc world finals, the lua toolbox), which need them because their standard libraries do not ship them. Java's chapter 5 sits at this chapter's pole too, BigInteger native, and still builds the 128-bit roads by hand beside the object, `Math.multiplyHigh` plus the shift-subtract lane, because the mechanism is the lesson.

sources: learn.microsoft.com, `BigInteger.ModPow`, `BigInteger.GreatestCommonDivisor`, and `PriorityQueue<TElement, TPriority>` api pages, accessed 2026-09-14. Contest context from the ICPC Foundation, icpc.global world finals problem pages, accessed 2026-09-14. Sample behavior verified by `make verify-csharp`, 26 tests in the icpc toolbox suite (Input 5, Vec 6, Ring 7, Num 8).

7. the javascript toolbox

JavaScript has one number type for everything, a 64-bit double whose integers stop being exact at 2^53, and no syntax error warns you when a computation walks past that line. So this toolbox is the integer story chapter of the set: every structure that stays under the wall, the heap, the deque, the grid, the input kit, runs on plain `Number` arrays, and the number kit does not go near `Number` at all, it is written entirely on `BigInt` with the conversion points pushed to the api edges. The suite is 239 non-blank lines over 6 files with 36 tests, private fields for the invariants, and the same cross-language anchors the other toolboxes pin. The semantics of the wall itself, doubles, safe integers, `BigInt`, belong to book 7, ch 2 (javascript es2026 and typescript 7, values and lexical structure), this chapter is where the policy pays off in contest code.

7.1. the min-heap, comparator included

The heap stores its elements in a plain array, children of i at 2i + 1 and 2i + 2, with the ordering injected as a function so one class serves every puzzle. Push sifts up by swapping with the parent while the comparator says so.

The dry run: the `objects order by a field: dijkstra-style entries` block in `test/heap.test.mjs` pushes three entries through a dist comparator and asserts the pop order by vertex.

1. Push `{v: 2, dist: 7}` lands alone; push `{v: 1, dist: 3}` appends then sifts up, 3 under 7 swapping them to `[3, 7]`.
2. Push `{v: 3, dist: 5}` appends at index 2, where the parent's 3 wins, no swap, `[3, 7, 5]`.
3. Pop returns v 1; the last entry, 5, takes the root, and its only child, 7, does not beat it, `[5, 7]`.
4. The next two pops return v 3 then v 2, dists 5 and 7, the asserted `[1, 3, 2]`.

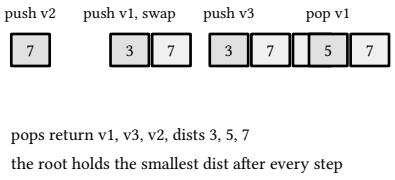


figure 54: the array under the dist comparator, one state per operation, the root always the smallest dist

The frontier pops by dist in order, 3, 5, 7, the asserted vertices 1, 3, 2 in the listings below.

```
export class MinHeap {
  #h = [];
  #less;

  constructor(less = (a, b) => a < b) {
    this.#less = less;
  }

  get size() {
    return this.#h.length;
  }

  peek() {
    return this.#h[0];
  }

  push(v) {
    this.#h.push(v);
    let i = this.#h.length - 1;
    while (i > 0 && this.#less(this.#h[i], this.#h[Math.floor((i - 1) / 2)])) {
      [this.#h[i], this.#h[Math.floor((i - 1) / 2)]] = [
        this.#h[Math.floor((i - 1) / 2)],
        this.#h[i],
      ];
      i = Math.floor((i - 1) / 2);
    }
    return this;
  }
```

listing 78: the class head and push, sifting up one swap at a time (source: icpc/samples-js/src/heap.mjs, lines 4-31)

Pop lifts the last element to the root and sifts down into the smaller child, and both operations return before the heap invariant can break, which the test asserts directly, every parent no greater than its children under the comparator after a shuffled fill:

```
  pop() {
    const top = this.#h[0];
    this.#h[0] = this.#h[this.#h.length - 1];
    this.#h.pop();
    let i = 0;
    for (;;) {
      const l = 2 * i + 1;
      const r = 2 * i + 2;
      let m = i;
      if (l < this.#h.length && this.#less(this.#h[l], this.#h[m])) m = l;
      if (r < this.#h.length && this.#less(this.#h[r], this.#h[m])) m = r;
      if (m === i) return top;
      [this.#h[i], this.#h[m]] = [this.#h[m], this.#h[i]];
      i = m;
    }
  }

  values() {
    return [...this.#h];
  }
}
```

listing 79: pop sifts down into the better child, values dumps the array (source: icpc/samples-js/src/heap.mjs, lines 33-53)

The default comparator is numeric ascending, so pushes of 5, 3, 8, 1, 9, 2 pop back as 1, 2, 3, 5, 8, 9, one line flips it to a max-heap, and a third test orders objects by a cost field, the dijkstra shape, which is exactly how the frontier in book 10, ch 11 (practical data structures and algorithms, shortest paths) would use it. The from-scratch derivation of the sift arithmetic lives in book 10, ch 8 (practical data structures and algorithms, heaps and priority queues).

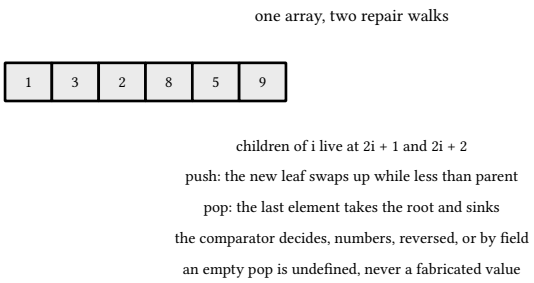


figure 55: the array is the tree, sift up on push, sift down on pop

7.2. the ring deque

The deque is the same ring the C# chapter builds, slots plus head plus count, growth that copies the logical contents back to slot 0, and it is in this suite because sliding windows and both-end worklists appear at every finals.

The dry run: the `the ring wraps: push front across the seam, order survives` block in `test/deque.test.mjs` drives a cap 4 ring through a backward wrap and asserts the drain.

1. pushBack 1, 2, 3 fill slots 0, 1, 2 with head 0, count 3.
2. pushFront 0 sets head to (0 - 1) mod 4 = 3, slot 3 holds 0, and the ring is full at count 4.
3. popBack reads slot (3 + 4 - 1) mod 4 = 2, the 3, and count drops to 3.
4. pushFront 9 sets head to (3 - 1) mod 4 = 2, slot 2 holds 9, the backward wrap past slot 0 the test's comment names.
5. The drain pops 9 from slot 2, 0 from slot 3, then 1 from slot 0, the asserted `[9, 0, 1]`.

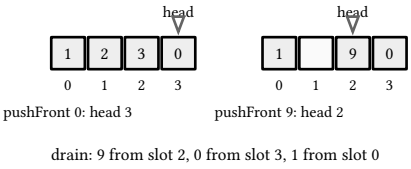


figure 56: the cap 4 ring at the two wraps: slot 3 first, then the backward step to slot 2

Order survives both wraps, the drain reading `[9, 0, 1]`, in the listings below.

```
export class Deque {
  #slots;
  #head = 0;
  #count = 0;

  constructor(cap = 8) {
    this.#slots = new Array(cap);
  }

  get count() {
    return this.#count;
  }

  #grow() {
    const next = new Array(this.#slots.length * 2);
    for (let i = 0; i < this.#count; i++)
      next[i] = this.#slots[(this.#head + i) % this.#slots.length];
    this.#slots = next;
    this.#head = 0;
  }

  pushBack(v) {
    if (this.#count === this.#slots.length) this.#grow();
    this.#slots[(this.#head + this.#count) % this.#slots.length] = v;
    this.#count++;
    return this;
  }

  pushFront(v) {
    if (this.#count === this.#slots.length) this.#grow();
    this.#head = (this.#head + this.#slots.length - 1) % this.#slots.length;
    this.#slots[this.#head] = v;
    this.#count++;
    return this;
  }
```

listing 80: slots, modulo wrap, doubling growth that rebases at slot zero (source: icpc/samples-js/src/deque.mjs, lines 4-38)

```
  popFront() {
    if (this.#count === 0) return undefined;
    const v = this.#slots[this.#head];
    this.#head = (this.#head + 1) % this.#slots.length;
    this.#count--;
    return v;
  }

  popBack() {
    if (this.#count === 0) return undefined;
    const v = this.#slots[(this.#head + this.#count - 1) % this.#slots.length];
    this.#count--;
    return v;
  }
}
```

listing 81: both pops, an empty pop is undefined (source: icpc/samples-js/src/deque.mjs, lines 40-54)

The refusal convention differs from C# on purpose. Where the C# ring throws `InvalidOperationException` on underflow, this deque returns `undefined`, which is JavaScript's native signal for absence, the same value an empty

heap pop returns, and the tests pin it for both ends. The seam test pushes front across the wrap and keeps the order, the growth tests fill from both ends past the initial capacity of 8 and drain in perfect order.

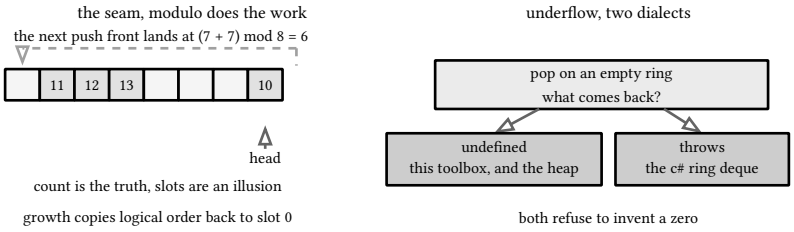


figure 57: head steps backward on push front, the tail wraps into freed slots

7.3. the number kit lives on bigint

Everything in `num.mjs` is `BigInt`, and the reason is a two-line test. `Number(2n ** 53n)` and `Number(2n ** 53n + 1n)` are the same double, the silent collision, while the BigInt values stay distinct, so any modular arithmetic over puzzle-sized numbers has no business running on `Number` at all.

The dry run: the `bezout holds and the classic coefficients pin` block in `test/num.test.mjs` asserts `extGcd(240n, 46n)` deep-equal to `{g: 2n, x: -9n, y: 47n}` and the identity beside it.

1. Euclid walks the magnitudes: 240 = 5 x 46 + 10, 46 = 4 x 10 + 6, 10 = 1 x 6 + 4, 6 = 1 x 4 + 2, and 4 = 2 x 2 + 0, so g is 2.
2. Unwinding, one substitution per step: 2 = 6 - 4, then 2 = 2 x 6
   - 10, then 2 = 2 x 46 - 9 x 10, then 2 = 47 x 46 - 9 x 240.
3. The coefficients land where the assert pins them, x = −9 and y = 47.
4. The check multiplies back: 46 x 47 = 2162, 240 x 9 = 2160, and 2162 - 2160 = 2, the asserted `a * x + b * y = g`.
5. The same identity check runs over 17 and 13, 0 and 7, then 7 and 0, before the file moves to crt.

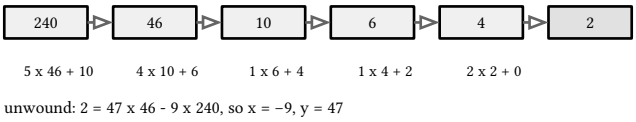


figure 58: the euclid staircase from 240 and 46 down to 2, each step one quotient and one remainder

g is 2 with x = −9 and y = 47, the pinned Bezout pair, in the listings below.

```
export function gcd(a, b) {
  while (b !== 0n) [a, b] = [b, a % b];
  return a;
}

export function lcm(a, b) {
  if (a === 0n || b === 0n) return 0n;
  return (a / gcd(a, b)) * b;
}

// square-and-multiply; mod must be positive
export function modpow(base, exp, mod) {
  let result = 1n % mod;
  let b = ((base % mod) + mod) % mod;
  let e = exp;
  while (e > 0n) {
    if (e & 1n) result = (result * b) % mod;
    b = (b * b) % mod;
    e >>= 1n;
  }
  return result;
}
```

listing 82: gcd, lcm, square-and-multiply, all bigint end to end (source: icpc/samples-js/src/num.mjs, lines 6-27)

The inverse runs extended Euclid recursively and reports failure as `null`, the same absence convention as the heap and the deque, composite moduli included since Euclid does not care about primality:

```
export function extGcd(a, b) {
  if (b === 0n) return { g: a, x: 1n, y: 0n };
  const { g, x, y } = extGcd(b, a % b);
  return { g, x: y, y: x - (a / b) * y };
}

// inverse of a mod m; null when they share a factor
export function modinv(a, m) {
  const { g, x } = extGcd(((a % m) + m) % m, m);
  if (g !== 1n) return null;
  return ((x % m) + m) % m;
}
```

listing 83: bezout coefficients, then the inverse, null on shared factors (source: icpc/samples-js/src/num.mjs, lines 30-41)

The crt fold accepts consistent non-coprime systems through the lcm and returns `null` for contradictions and for empty or non-positive input:

```
export function crt(system) {
  if (system.length === 0) return null;
  let { r: x, m: mod } = system[0];
  if (mod ≤ 0n) return null;
  for (const { r, m } of system.slice(1)) {
    if (m ≤ 0n) return null;
    const g = gcd(mod, m);
    if ((r - x) % g =/= 0n) return null;
    const mOverG = m / g;
    const inv = modinv((mod / g) % mOverG, mOverG);
    const t = ((((r - x) / g) % mOverG) * inv) % mOverG;
    x = x + mod * t;
    mod = (mod / g) * m;
    x = ((x % mod) + mod) % mod;
  }
  return { x, mod };
}
```

listing 84: the fold, consistent non-coprime pairs merge, clashes return null (source: icpc/samples-js/src/num.mjs, lines 44-60)

The anchors are the shared ones: 2^100 mod 1e9+7 is 976371285, extGcd(240, 46) gives coefficients −9 and 47 with gcd 2, the sun tzu fixture folds to 23 mod 105, and 1 mod 4 with 5 mod 8 merges to 5 mod 8. Beyond the anchors, the suite goes where `Number` cannot: 3 to the 2^62 modulo 2^61 stays exact, the lcm of 2^32 and 3^23 towers past 2^53, and a three-congruence system whose combined modulus includes 2^61 - 1 verifies every residue. The mathematics being exercised here is derived in book 10, ch 16 (practical data structures and algorithms, number theory and modular arithmetic).

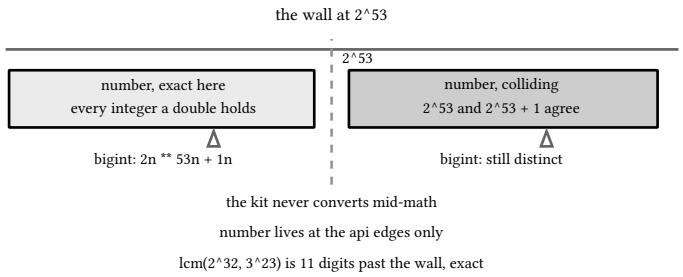


figure 59: the wall at 2^53, number collides, bigint keeps counting

7.4. grid and input, plain data

The grid kit is deliberately untyped: rows are arrays of single character strings, coordinates are two-number arrays, and the two delta sets are exported constants.

The dry run: the `4-neighbors of a center cell and a corner` block in `test/grid.test.mjs` walks both cells of a nine-letter fixture and asserts the neighbor lists with their order.

1. `parseGrid` builds 3 rows of 3 from the abc, def, ghi fixture.
2. From the center `[1, 1]`, N4 walks up, down, left, right, its exported order: `[0, 1]`, `[2, 1]`, `[1, 0]`, `[1, 2]`, all in bounds, the asserted list.
3. From the corner `[0, 0]`, up and left cross the edge; down gives `[1, 0]`, right gives `[0, 1]`, the asserted pair.
4. Under N8 the center keeps all 8 and the corner 3, `[0, 1]`, `[1, 0]`, `[1, 1]`, the exported lengths pinning 4 and 8.
5. `manhattan([0, 0], [3, 4])` adds 3 + 4 = 7, pinned beside the 4 and 0 of the shorter pairs.

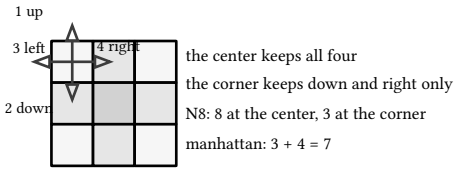


figure 60: the center's four steps in N4 order, up, down, left, right, and the corner keeping two

The lists come back in delta order and manhattan reads 7, the pinned values, in the listings below.

```
export const N4 = [
  [-1, 0],
  [1, 0],
  [0, -1],
  [0, 1],
];

export const N8 = [
  [-1, -1],
  [-1, 0],
  [-1, 1],
  [0, -1],
  [0, 1],
  [1, -1],
  [1, 0],
  [1, 1],
];
```

listing 85: the two delta sets, four orthogonal, eight total (source: icpc/samples-js/src/grid.mjs, lines 5-21)

```
// rows of characters; grid[r][c] is the cell at (r, c)
export function parseGrid(text) {
  return text
    .split("\n")
    .map((line) => line.replace(/\r$/, ""))
    .filter((line) => line.length > 0)
    .map((line) => line.split(""));
}

export function inBounds(grid, r, c) {
  return r >= 0 && r < grid.length && c >= 0 && c < grid[r].length;
}

// the in-bounds neighbors under the given delta set
export function neighbors(grid, [r, c], deltas = N4) {
  return deltas
    .map(([dr, dc]) => [r + dr, c + dc])
    .filter(([nr, nc]) => inBounds(grid, nr, nc));
}

export function manhattan([r1, c1], [r2, c2]) {
  return Math.abs(r1 - r2) + Math.abs(c1 - c2);
}
```

listing 86: parse tolerates crlf, neighbors stay in bounds, manhattan on tuples (source: icpc/samples-js/src/grid.mjs, lines 23-45)

The input kit pairs with it. `splitLines` normalizes `\\r\\n` and the trailing newline in one pass, `blankGroups` reduces the lines into paragraphs, and `intsPerLine` is one regular expression, `/-?\\d+/g`, run through `matchAll`, so every line without numbers yields an empty array rather than a hole:

```
export function splitLines(text) {
  return text
    .replace(/\r\n/g, "\n")
    .replace(/\n$/, "")
    .split("\n");
}

// groups separated by one or more blank lines
export function blankGroups(text) {
  return splitLines(text)
    .reduce(
      (groups, line) => {
        if (line === "") groups.push([]);
        else groups[groups.length - 1].push(line);
        return groups;
      },
      [[]],
    )
    .filter((group) => group.length > 0);
}

// the signed integers on each line; lines without numbers yield []
export function intsPerLine(text) {
  return splitLines(text).map((line) =>
    [...line.matchAll(/-?\d+/g)].map((m) => Number(m[0])),
  );
}
```

listing 87: lines, blank-line groups, signed integers per line (source: icpc/samples-js/src/input.mjs, lines 5-31)

Run-length pairs close the kit, decode expands `[[3, a], [2, b]]` to `aaabb`, encode rebuilds the pairs, and both directions are pinned. The crlf test mixes line endings in one document and the grid parse test does the same, because puzzle files arrive from three operating systems and only ever cause trouble on the third one.

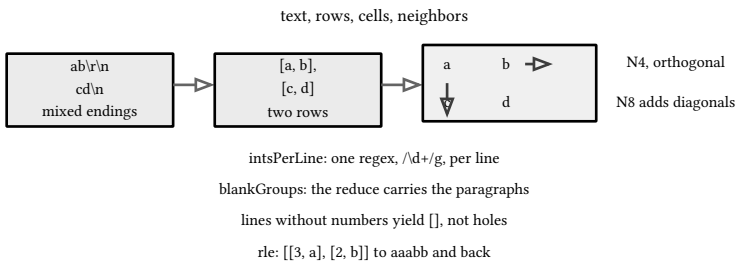


figure 61: text to rows to cells, the deltas ready for the walk

7.5. md5 from node:crypto

Node ships md5 in `node:crypto` behind one line, `createHash("md5")`, and the toolbox wraps exactly that, digest to lowercase hex plus a prefix helper for the mining shape where a puzzle compares the first n hex characters against zeros.

The dry run: the `the prefix helper slices the leading hex characters` block in `test/md5.test.mjs` pins the three slice widths over the `abc` digest the vector blocks already pinned.

1. The vectors land first: the empty string, `abc`, and `message digest` digest to their pinned constants, `abc` to `900150983cd24fb0d6963f7d28e17f72`.
2. `md5Prefix("abc")` takes the default width 8 and reads `90015098`.

3. `md5Prefix("abc", 5)` takes 5 and reads `90015`, the mining shape's own comparison.
4. `md5Prefix("abc", 32)` takes the full width, and a digest is exactly 32 lowercase hex characters, so the slice equals `md5Hex("abc")`.
5. The last block hashes `abcdef609043` against `abcdef609044`, one counter step apart, and the digests differ.

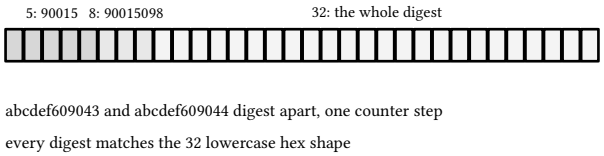


figure 62: the abc digest at three slice widths, 5, 8, and the full 32

The width 5 slice reads `90015`, the pinned mining prefix, in the listing below.

```
import { createHash } from "node:crypto";

export function md5Hex(s) {
  return createHash("md5").update(s).digest("hex");
}

// the first n hex characters, the mined-prefix convention
export function md5Prefix(s, n = 8) {
  return md5Hex(s).slice(0, n);
}
```

listing 88: createhash, hex digest, the mined prefix slice (source: icpc/samples-js/src/md5.mjs, lines 6-15)

The tests pin the canonical vectors, the empty string `d41d8cd98f00b204e9800998ecf8427e`, `abc` as `900150983cd24fb0d6963f7d28e17f72`, the RFC 1321 `message digest` vector, and `md5Prefix("abc")` as `90015098`. Those same constants are pinned by the from-scratch implementations in book 11, ch 2 (the icpc world finals, the c toolbox) and book 11, ch 9 (the icpc world finals, the lua toolbox), so the three toolboxes cross-check each other with every test run, the wrapped one against the hand-rolled two.

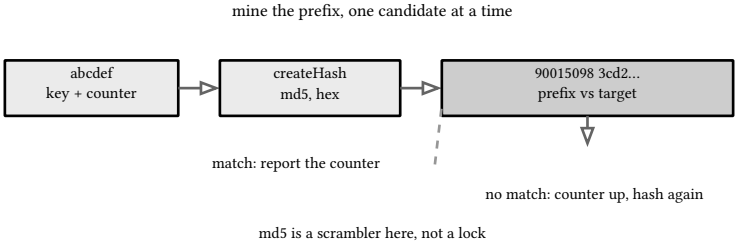


figure 63: the mining loop, hash the candidate, compare the leading hex

7.6. built here, cited elsewhere

Six files, 239 non-blank lines, 36 tests, and what each piece leans on:

piece	status	where
min-heap with comparator	built, `heap.mjs`	5 tests, sift logic derived in book 10, ch 8 (practical data structures and algorithms, heaps and priority queues)
ring deque	built, `deque.mjs`	5 tests, empty pops return undefined
bigint number kit	built, `num.mjs`	10 tests, wall semantics in book 7, ch 2 (javascript es2026 and typescript 7, values and lexical structure)
grid tuples and deltas	built, `grid.mjs`	6 tests, crlf tolerated

piece	status	where
input and rle	built, `input.mjs`	4 tests, one regex for signed ints
md5	cited, `node:crypto`	6 tests, from-scratch in book 11, ch 2 (the icpc world finals, the c toolbox)
crt mathematics	built, `num.mjs`	derived in book 10, ch 16 (practical data structures and algorithms, number theory and modular arithmetic)

Java's chapter 5 never meets the 2^53 wall: its long is a true 64-bit word, so the only boundary it states is the 128-bit product, `Math.multiplyHigh` plus the low half, with BigInteger held for the values that outgrow the word.

sources: nodejs.org docs for `node:crypto`, `Number`, and `BigInt`, developer.mozilla.org for `String.prototype.matchAll`, RFC 1321 for the md5 test vectors, accessed 2026-09-14. Contest context from the ICPC Foundation, icpc.global world finals problem pages, accessed 2026-09-14. Sample behavior verified by `npm run verify` in `books/icpc/samples-js`, 36 tests in the toolbox suite (heap 5, deque 5, num 10, grid 6, input 4, md5 6).

8. the python toolbox

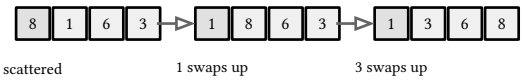
Python carries the lightest toolbox of the eight languages, and that is the point: the standard library already won. A priority queue is `heapq`, a deque is `collections.deque`, memoization is one `@cache` line, the combinatorial loops are `itertools`, the parsing is `re`, and arbitrary-precision integers are not a library at all, they are the int type, so `pow(2, 100, 1_000_000_007)` is a builtin call with no boundary to respect. What this chapter builds is therefore not containers, it is fluency: 274 non-blank lines of idiom checks over 6 files, 67 asserts, every pattern pinned to a fact, and every deeper treatment cited to the python book rather than repeated here. The other six toolboxes in this set exist because their languages made the contest jobs hard. This one exists to show what it looks like when they are easy.

8.1. queues, two import lines

The heap is not a class, it is a discipline over a plain list: `heappush` maintains the invariant, `heap[0]` is always the minimum, and `heappop` drains in order. `heapify` builds the invariant in one in-place pass, `nlargest` answers top-k without ceremony, and the a-star tie-breaker idiom rides along, a counter in the tuple keeps equal priorities first-in-first-out because tuples compare left to right.

The dry run: `queues.py` pushes 7, 2, 9, 4 and drains 2, 4, 7, then feeds `heapify` the scattered list 8, 1, 6, 3 and checks the root it leaves.

1. The four `heappush` calls leave `heap[0]` at 2, the file's first check, and three pops read 2, 4, 7 in order.
2. `heapify` sifts down from the last parent, index 1: its only child, the 3 at index 3, does not beat 1, no swap.
3. Index 0 holds 8 over children 1 and 6: the smaller, 1, swaps up, leaving `[1, 8, 6, 3]`.
4. The sift continues at index 1: the 3 at index 3 swaps up, leaving `[1, 3, 6, 8]`, and the check reads `scattered[0] == 1`.
5. `nlargest(2, ...)` over the 8-element fixture returns 9 and 6, the pinned top two.



the root ends at 1, the pinned check, and nlargest keeps 9 and 6

figure 64: heapify's two sifts over 8, 1, 6, 3: the 1 rises, then the 3, the root ending at 1

Two swaps leave the root at 1 and the drain at 2, 4, 7, both pinned, in the listings below.

```
heap = []
for value in (7, 2, 9, 4):
    heappush(heap, value)
check(heap[0] == 2, "the root is always the minimum")
check([heappop(heap) for _ in range(3)] == [2, 4, 7], "pops drain in order")

scattered = [8, 1, 6, 3]
heapify(scattered)  # one pass, in place
check(scattered[0] == 1, "heapify orders a list in place")
check(nlargest(2, [3, 1, 4, 1, 5, 9, 2, 6]) == [9, 6], "nlargest keeps the top two")

# the a-star tie breaker: a counter keeps equal priorities fifo
queue = []
tick = 0
for label in ("a", "b", "c"):
    heappush(queue, (5, tick, label))  # same priority, insertion ordered
    tick += 1
order = [heappop(queue)[2] for _ in range(3)]
check(order == ["a", "b", "c"], "the counter breaks priority ties fifo")
```

listing 89: the heap discipline, top-k, and the counter tie-breaker (source: icpc/samples-py/src/Ch06/queues.py, lines 16-34)

The deque half is one constructor argument with a personality: `maxlen` turns the deque into a sliding window that silently evicts the far end, and `rotate` moves the whole contents with wraparound in both directions, the operation the C# and JavaScript chapters build by hand:

```
window = deque(maxlen=3)
for value in (1, 2, 3, 4, 5):
    window.append(value)  # the 4th append silently drops the 1
check(list(window) == [3, 4, 5], "a maxlen deque keeps only the last three")
window.appendleft(2)
check(list(window) == [2, 3, 4], "appendleft evicts from the right")

spinner = deque([1, 2, 3, 4])
spinner.rotate(1)  # positive rotates right: the tail wraps to the head
check(list(spinner) == [4, 1, 2, 3], "rotate one step right")
spinner.rotate(-2)
check(list(spinner) == [2, 3, 4, 1], "rotate two steps left")
```

listing 90: the maxlen window evicts silently, rotate wraps both ways (source: icpc/samples-py/src/Ch06/queues.py, lines 36-47)

The checks pin the exact behaviors: pushes of 7, 2, 9, 4 drain as 2, 4, 7, `nlargest(2)` over the fixture returns 9 and 6, a `maxlen=3` window fed 1 through 5 keeps 3, 4, 5 and an `appendleft(2)` evicts from the right back to 2, 3, 4, and `rotate(1)` on 1, 2, 3, 4 gives 4, 1, 2, 3. How these containers are implemented, and when a list is the wrong answer, is the material of book 8, ch 4 (python 3.14, builtin collections, iteration, and generators).

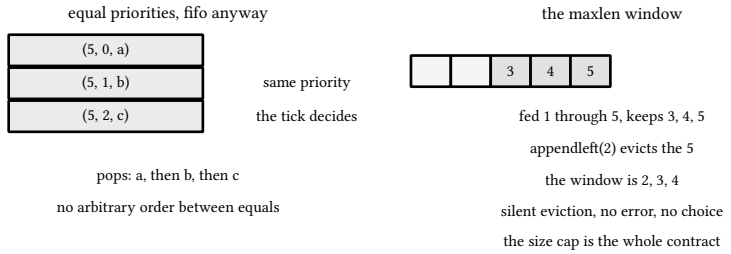


figure 65: the tie-breaker rides tuple comparison, the window forgets the far end

8.2. memoization, three ways

The naive recursion trap, measured: `fib(24)` is 46368 and costs 150049 calls, because the call tree recomputes every subproblem exponentially often. The manual dict memo pays 47 calls for the same answer, linear, and the pair of counters in the check file makes the difference an assertion rather than an observation.

The dry run: `memo.py` runs both recursions on fib(24) and asserts the call counters, 150049 against 47, then the sized cache's evictions.

1. The naive recursion answers 46368 at 150049 calls, both pinned by the opening checks.
2. The manual version descends k - 1 first: 24 entries, k = 24 down to 1, the base case returning under 2 without recursing.
3. Each entry from k = 2 through 24 also calls k - 2 once: 23 more entries, every one a table hit on return.
4. 24 + 23 = 47, the pinned count, for the same 46368.
5. After `cache_clear`, `fib_cached(10)` reads 55 from an empty cache, the reset check.

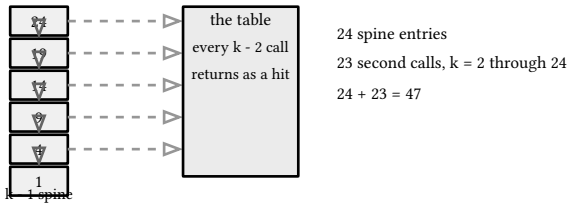


figure 66: the manual recursion: one left spine of 24 entries, 23 second calls hitting the table

The same 46368 for 47 calls instead of 150049, both counters pinned, in the listings below.

```
calls = {"naive": 0, "manual": 0}


def fib_naive(k):
    calls["naive"] += 1
    if k < 2:
        return k
    return fib_naive(k - 1) + fib_naive(k - 2)


check(fib_naive(24) == 46368, "naive fib(24) is still correct")
check(calls["naive"] == 150049, "the trap: 150049 calls, exponential")


def fib_manual(k, table=None):
    if table is None:
        table = {}
    calls["manual"] += 1
    if k < 2:
        return k
    if k not in table:
        table[k] = fib_manual(k - 1, table) + fib_manual(k - 2, table)
    return table[k]


check(fib_manual(24) == 46368, "the manual dict agrees")
check(calls["manual"] == 47, "47 calls: linear, the cache kills the tree")
```

listing 91: the trap and the manual dict, 150049 calls against 47 (source: icpc/samples-py/src/Ch06/memo.py, lines 15-41)

`functools.cache` is the same memo as a decorator, with `cache_info` reporting hits and `cache_clear` resetting it, and `functools.lru_cache(maxsize=2)` is the sized variant whose eviction order the checks trace call by call, 3, 4, 5 computed, 3 evicted and recomputed, 5 still resident, 4 evicted too:

```
@cache
def fib_cached(k):
    return k if k < 2 else fib_cached(k - 1) + fib_cached(k - 2)


check(fib_cached(24) == 46368, "functools.cache wraps the same shape")
check(fib_cached.cache_info().hits ≥ 20, "the wrapper records its hits")
fib_cached.cache_clear()
check(fib_cached(10) == 55, "clear resets the cache harmlessly")

counter = {"sized": 0}


@lru_cache(maxsize=2)
def step(k):
    counter["sized"] += 1
    return k * k


check(step(3) == 9 and step(4) == 16 and step(5) == 25, "the sized cache computes")
check(counter["sized"] == 3, "three distinct keys, three runs")
check(step(3) == 9 and counter["sized"] == 4, "3 was evicted: recomputed")
check(step(5) == 25 and counter["sized"] == 4, "5 is still resident: a hit")
check(step(4) == 16 and counter["sized"] == 5, "4 was evicted too: recomputed")
```

listing 92: cache and the sized variant, evictions traced (source: icpc/samples-py/src/Ch06/memo.py, lines 44-67)

Which wrapper to reach for, and how decorators compose the wrapping at definition time, is the subject of book 8, ch 6 (python 3.14, functions, scopes, and decorators). The dynamic-programming tables this idiom replaces, and when the table beats the memo, belong to the algorithms side of the house.

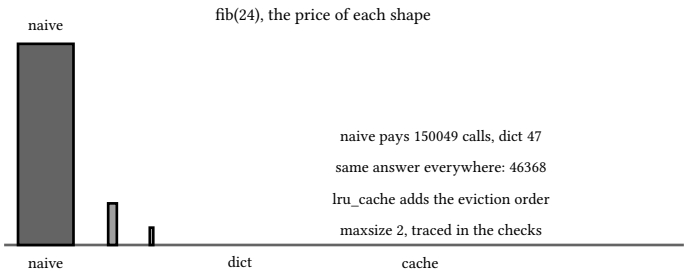


figure 67: one answer, three call counts

8.3. the combinator iterators

Every brute-force finals solution is one `itertools` call. `product` enumerates independent slots, `permutations` cares about order, `combinations` does not, and `pairwise` yields the adjacent gaps that turn positions into differences.

The dry run: `iter.py` splits the records fixture through `groupby` under the key `line ≠ ""` and asserts the three records.

1. `splitlines` yields 7 lines: alpha, beta, blank, gamma, blank, blank, delta.
2. The key turns them into true, true, false, true, false, false, true.
3. `groupby` cuts at every key change: five groups, the doubled blank collapsing into one false group of 2 lines rather than two.
4. The `if key` filter drops both false groups, leaving the pinned records, `alpha beta`, `gamma`, `delta`.

5. Over the words a, b, b, c, c, c the same cut counts runs 1, 2, 3 by `len(list(grp))`.

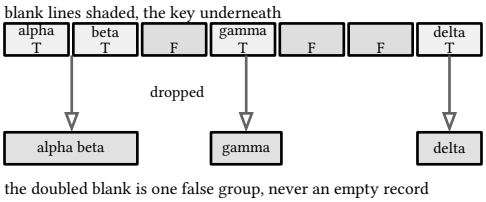


figure 68: seven lines, five groups, three records: the false groups never survive the filter

Seven lines fold to three records, the doubled blank never an empty group, in the listings below.

```
grid = list(product("ab", repeat=2))
check(
    grid == [("a", "a"), ("a", "b"), ("b", "a"), ("b", "b")],
    "product enumerates every slot independently",
)
check(len(list(product(range(4), repeat=3))) == 64, "a 3-digit base-4 space")
check(
    list(permutations("abc", 2))
    == [("a", "b"), ("a", "c"), ("b", "a"), ("b", "c"), ("c", "a"), ("c", "b")],
    "permutations care about order",
)
check(
    list(combinations("abcd", 2))
    == [("a", "b"), ("a", "c"), ("a", "d"), ("b", "c"), ("b", "d"), ("c", "d")],
    "combinations do not",
)
check(
    list(pairwise([1, 3, 6, 10])) == [(1, 3), (3, 6), (6, 10)],
    "pairwise yields the gaps",
)
check(
    [b - a for a, b in pairwise([1, 3, 6, 10])] == [2, 3, 4],
    "the gaps read as differences",
)
```

listing 93: product, permutations, combinations, pairwise, pinned exactly (source: icpc/samples-py/src/Ch06/iter.py, lines 15-38)

The other half of the module that carries puzzle loops is `groupby`, which groups consecutive runs under a key. Over lines classified as blank or not, it splits blank-line separated records, and over a list of words it counts runs, the run-length idiom the JavaScript chapter writes by hand:

```
# puzzle input arrives as blank-line separated records
text = "alpha\nbeta\n\ngamma\n\n\ndelta\n"
records = [
    [line for line in group]
    for key, group in groupby(text.splitlines(), lambda line: line != "")
    if key
]
check(
    records == [["alpha", "beta"], ["gamma"], ["delta"]],
    "groupby splits records",
)

words = ["a", "b", "b", "c", "c", "c"]
runs = [(key, len(list(grp))) for key, grp in groupby(words)]
check(runs == [("a", 1), ("b", 2), ("c", 3)], "groupby also counts runs")
check(
    list(combinations(range(5), 3))[:2] == [(0, 1, 2), (0, 1, 3)],
    "combinations emit lexicographically",
```

listing 94: groupby cuts records and counts runs (source: icpc/samples-py/src/Ch06/iter.py, lines 40-57)

The checks are exhaustive on small cases: `product("ab", repeat=2)` is the four tuples in order, a three-slot base-4 space is exactly 64, `permutations("abc", 2)` lists all six, `combinations("abcd", 2)` lists its six in lexicographic order, and the records fixture with doubled blank lines splits as `alpha beta`, `gamma`, `delta`. Iteration itself, generators, laziness, and the protocols under these functions, is the long form of book 8, ch 4 (python 3.14, builtin collections, iteration, and generators).

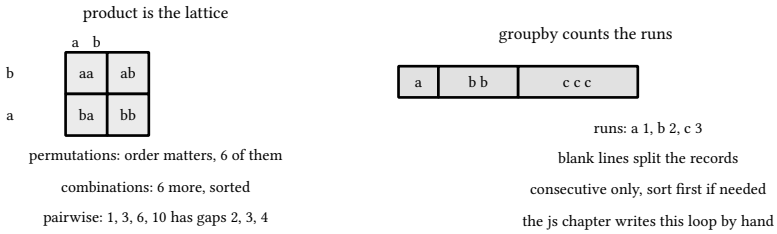


figure 69: four product tuples and three groupby runs

8.4. numbers and line parsing

The number idioms are short because the language does the work. `math.gcd` and `math.lcm`, variadic since they gained the multi-argument form, `divmod` returning the floored pair, and the three-argument `pow` that hashes square-and-multiply into C, with a negative exponent computing the modular inverse and a non-unit raising `ValueError` instead of guessing.

The dry run: `num.py` pins the gcd and lcm fixtures, then the three-argument `pow` family and `divmod`, check by check.

1. `gcd(48, 18)` reads 6, and the variadic `lcm(2, 3, 4)` folds pairwise, `lcm(lcm(2, 3), 4) = lcm(6, 4) = 12`.
2. The anchor `pow(2, 100, 1_000_000_007)` is 976371285; `pow(3, -1, 7)` is 5, verified by the multiply back, 3 x 5 = 15 and 15 - 14 = 1.
3. `pow(2, -1, 4)` raises `ValueError`, 2 and 4 share the factor 2, and the try block turns the refusal into a passing check.
4. `divmod(17, 5)` returns (3, 2): 3 x 5 = 15 and 17 - 15 = 2, the floored quotient beside its remainder.
5. `int("1_000_000")` reads 1000000, `int("z", 36)` reads 35, and the spaces around 42 are forgiven.

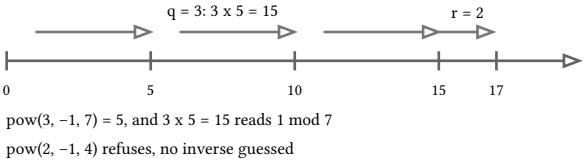


figure 70: divmod(17, 5) on a number line: three hops of 5, then 2 left over

The quotient 3 meets its remainder 2, and the inverse multiplies back to 1, in the listings below.

```
check(gcd(48, 18) == 6, "gcd collapses to the common factor")
check(lcm(4, 6) == 12, "lcm through the gcd")
check(lcm(2, 3, 4) == 12, "lcm variadic: chained pairwise")
check(gcd(0, 7) == 7, "gcd with zero passes the other through")

check(pow(2, 100, 1_000_000_007) == 976371285, "pow hashes the square-and-multiply")
check(pow(3, -1, 7) == 5, "negative exponent computes the modular inverse")
check(3 * pow(3, -1, 7) % 7 == 1, "the inverse verifies")
try:
    pow(2, -1, 4)
    check(False, "a non-unit inverse refuses")
except ValueError:
    check(True, "a non-unit inverse refuses")
```

listing 95: gcd, lcm, three-argument pow with the inverse extension (source: icpc/samples-py/src/Ch06/num.py, lines 15-27)

```
q, r = divmod(17, 5)
check((q, r) == (3, 2), "divmod splits quotient and remainder")
check(
    all(divmod(k, 7) == (k // 7, k % 7) for k in range(-10, 10)),
    "divmod == floored pair",
)
check(divmod(3600, 60) == (60, 0), "exact divisions carry no remainder")

check(int("ff", 16) == 255, "int parses with an explicit base")
check(int("1010", 2) == 10, "binary too")
check(int("1_000_000") == 1000000, "underscores group digits")
check(int("z", 36) == 35, "base 36 covers the alphabet")
check(int("  42  ") == 42, "surrounding whitespace is forgiven")
```

listing 96: divmod is the floored pair, int parses with base and separators (source: icpc/samples-py/src/Ch06/num.py, lines 29-41)

The anchors match the other seven languages, `pow(2, 100, 1_000_000_007)` is 976371285, and `pow(3, -1, 7)` is 5, verified in the next line by multiplying it back to 1. Parsing rides the same theme: `int` takes an explicit base from 2 to 36, accepts underscore separators, forgives surrounding whitespace, and `re` handles the structured lines, `fullmatch` pins the whole shape, capture groups split it, `findall` returns the pairs, and a dict comprehension pivots them:

```
match = re.fullmatch(r"Game (\d+): (.+)", LINE)
check(match is not None and match.group(1) == "7", "fullmatch pins the shape")
check(
    match.group(2) == "3 blue, 4 red, 1 green",
    "the capture groups split cleanly",
)

draws = re.findall(r"(\d+) (blue|red|green)", LINE)
check(
    draws == [("3", "blue"), ("4", "red"), ("1", "green")],
    "findall returns the pairs",
)

pivot = {color: int(count) for count, color in draws}
check(
    pivot == {"blue": 3, "red": 4, "green": 1},
    "the pivot turns pairs into a dict",
)

pairs = dict(re.findall(r"(\w+)=(-?\d+)", "x=3 y=14 z=-2"))
check(pairs == {"x": "3", "y": "14", "z": "-2"}, "key=value lines pivot directly")
```

listing 97: fullmatch, findall pairs, the dict pivot (source: icpc/samples-py/src/Ch06/lines.py, lines 25-45)

The `Game 7` fixture line ends as the dict blue 3, red 4, green 1, and the same pattern pivots `x=3 y=14 z=-2` in one expression. The regex language behind this, and the classic quantifier traps, are chapter material in book 8, ch 5 (python 3.14, strings, bytes, and text), and the numerics module by module is book 8, ch 23 (python 3.14, stdlib numerics: math, statistics, decimal, random).

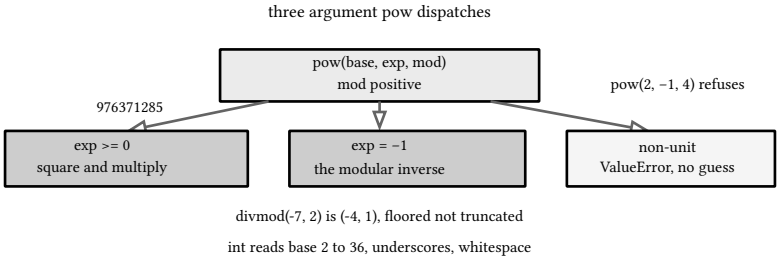


figure 71: one builtin, three exponents, and the floored pair

8.5. digests and json leaves

The digest idioms are two lines each: `hashlib.md5(b"hello")` pins the famous constant, the streaming shape calls `update` twice and lands on the same digest as the one-shot call, and one flipped byte flips the whole digest, the avalanche the mining puzzles rely on.

The dry run: `digest.py` pins the one-shot and streaming digests of `b"hello"`, the flipped byte beside them, and the json leaf sums over the nested fixtures.

1. One shot, `hashlib.md5(b"hello")` is `5d41402abc4b2a76b9719d911017c592`, with the empty-string and sha1 vectors pinned beside it.
2. The streaming shape calls `update(b"hel")` then `update(b"lo")`, 3 + 2 = 5 bytes, and the hexdigest equals the one-shot, asserted.
3. The avalanche check flips one byte, `b"hellp"` against `b"hello"`, and the digests differ.
4. The walk folds the nested list `1, 2, 3, 4` to 10, the strings-and-nulls document to 0, and a bare 42 document is itself.

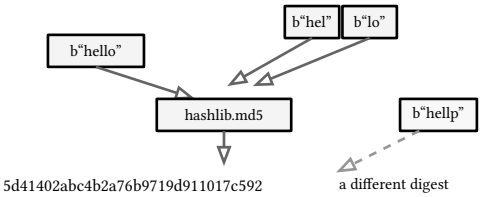


figure 72: the same five bytes by two shapes, one shot and streamed as hel then lo, one digest

Both shapes land on the same `5d41402abc4b2a76b9719d911017c592`, and the listings below hold both halves.

```
check(
    hashlib.md5(b"hello").hexdigest() == "5d41402abc4b2a76b9719d911017c592",
    "the md5 of hello is a famous constant",
)
check(
    hashlib.md5(b"").hexdigest() == "d41d8cd98f00b204e9800998ecf8427e",
    "the empty-input md5 too",
)
got = hashlib.sha1(b"hello").hexdigest()
check(
    got == "aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d",
    "and the sha1 of hello",
)
check(
    hashlib.md5(b"hello").hexdigest() != hashlib.md5(b"hellp").hexdigest(),
    "one byte flips the whole digest",
)

stream = hashlib.md5()  # the streaming shape: update then hexdigest
stream.update(b"hel")
stream.update(b"lo")
check(
    stream.hexdigest() == hashlib.md5(b"hello").hexdigest(),
    "updates concatenate",
)
```

listing 98: the vectors, the streaming shape, the avalanche (source: icpc/samples-py/src/Ch06/digest.py, lines 16-40)

The json half walks a parsed document and sums the numeric leaves at any depth, the same walk the Go toolbox pins at 10.5, here over a fixture whose leaves sum to 15, with the two traps handled out loud, `bool` is an `int` subclass and must be skipped explicitly, and digits inside strings count for nothing:

```
def sum_numbers(node):
    if isinstance(node, bool):  # bool is an int subclass: skip it
        return 0
    if isinstance(node, (int, float)):
        return node
    if isinstance(node, dict):
        return sum(sum_numbers(v) for v in node.values())
    if isinstance(node, list):
        return sum(sum_numbers(item) for item in node)
    return 0  # strings and None contribute nothing


parsed = json.loads(DOC)
check(sum_numbers(parsed) == 15, "the numeric leaves sum to 15")
check(sum_numbers(json.loads("[1, [2, [3, [4]]]]")) == 10, "nested lists recurse")
check(
    sum_numbers(json.loads('{"s": "12", "t": null}')) == 0,
    "digits in strings do not count",
)
check(sum_numbers(json.loads("42")) == 42, "a bare number document")
```

listing 99: the recursive walk, bool skipped, string digits ignored (source: icpc/samples-py/src/Ch06/digest.py, lines 50-69)

The checks pin all four shapes: the nested fixture sums to 15, `[1, [2, [3, [4]]]]` sums to 10, a document of strings and nulls sums to 0, and a bare number document is itself. Where the stdlib's larger streaming and serialization machinery lives is book 8, ch 19 (python 3.14, operational plumbing).

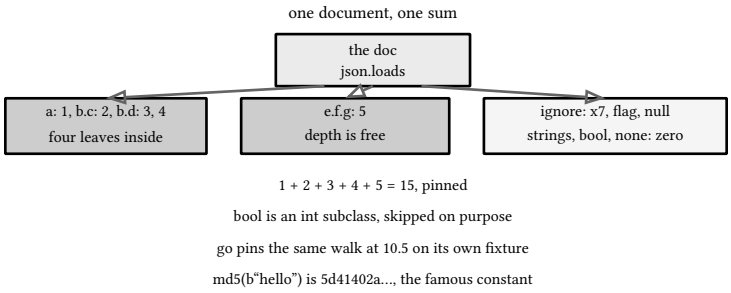


figure 73: the leaves sum to 15, the boolean is not a number

8.6. the lightest toolbox

The whole contract in one table. Six check files, 274 non-blank lines, 67 asserts, and nothing built that the standard library does not already ship:

idiom	status	deep treatment
queues, heapq and deque	stdlib, `queues.py`	9 checks, book 8, ch 4 (python 3.14, builtin collections, iteration, and generators)
memoization	stdlib, `memo.py`	12 checks, book 8, ch 6 (python 3.14, functions, scopes, and decorators)
combinator iterators	stdlib, `iter.py`	9 checks, book 8, ch 4 (python 3.14, builtin collections, iteration, and generators)
numbers, pow, divmod	stdlib, `num.py`	16 checks, book 8, ch 23 (python 3.14, stdlib numerics: math, statistics, decimal, random)
line parsing, regex	stdlib, `lines.py`	12 checks, book 8, ch 5 (python 3.14, strings, bytes, and text)

idiom	status	deep treatment
digests and json walks	stdlib, `digest.py`	9 checks, book 8, ch 19 (python 3.14, operational plumbing)

The counterparts in the other seven toolboxes are the comparison: the c and rust toolboxes hand-build the whole substrate and chapter 3 names every module the standard library was allowed to cover, the go, c#, and javascript toolboxes hand-build heaps, deques, big integers, modular arithmetic, and md5, and the lua toolbox adds a bigint and an md5 for the same reason. Python imports its way past all of it. Java's chapter 5 shows the other face of a rich library: the imports exist, and the kit still measures the seams, Scanner at 1.4 to 1.6x the buffered pair on a quiet box (a 4154731-byte, 420000-token fixture, 2.4 to 2.8x when the box is loaded) and Integer[] sorts at 3x the int[] sort, then bans them by number.

sources: docs.python.org for `heapq`, `collections.deque`, `functools.cache`, `itertools`, `math.gcd`, `re`, `hashlib`, and the builtin `pow` and `int` entries, accessed 2026-09-14. Contest context from the ICPC Foundation, icpc.global world finals problem pages, accessed 2026-09-14. Sample behavior verified by `tools/run-py-samples.ps1 -SampleRoot books/icpc/samples-py/src`, 67 asserts across the 6 check files (queues 9, memo 12, iter 9, num 16, lines 12, digest 9).

9. the lua toolbox

Lua is the language where the machine's arithmetic is the least honest, integers are 64-bit and wrap silently, so the Lua toolbox is built around saying that out loud and then refusing to depend on it. Nine modules live in `books/icpc/samples-lua/`, 1088 lines, each returning a table of named test functions that a single `run.lua` executes and counts: 45 checks across the nine, 7 for bigint down to 4 for heap, deque, and strutil. This chapter walks each module the way book 11, ch 2 (the icpc world finals, the c toolbox) walks the C headers, and the two chapters cross-cite wherever the same structure teaches differently. The C chapter's dilemma was a link failure, a 128-bit product that would not link. Lua's is quieter and worse: the product computes fine and answers wrong.

9.1. modules, assertions, and the runner

Every module follows one shape: helpers at the top, then a returned table of `{name = ..., fn = ...}` test blocks. The runner requires each module by name, runs each block under `pcall`, prints one line per module with the pass count, and exits nonzero on any failure. Assertions come from a `lib.lua` that the suite carries its own copy of: `run.lua` prepends `./?.lua` to `package.path`, so `require("lib")` resolves to the assertion module beside the samples, not to whatever else the machine has. `T.eq` compares with Lua equality first and structurally for tables, keys and values both, so a decoded json tree can be checked against a literal in one call. `T.ok` is the plain assertion, and `T.throws` requires an error whose message contains a given substring, which is how the json module's named errors are tested. The interpreter is the repo's vendored Lua 5.5.1, and the suites under it are pure: no network, no clock, no unseeded randomness, the same discipline every suite in this book runs under.

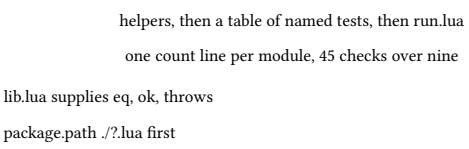


figure 74: one module's shape: helpers, a returned table of named test blocks, one runner line per module

The dry run: no suite file carries `run.lua`, nothing requires it, so this walk is hand-derived from the fixtures it loads. It meets `ch07_heap`, whose returned table holds 4 blocks, and prints the module's line.

1. The require resolves through the prepended `./?.lua` to `ch07_heap.lua`, which returns its 4 blocks: default order, reversed comparator, tuple comparator, interleaved.
2. Each `fn` runs under `pcall`, all four pass, and `cpass` ends at 4.
3. The line prints through `string.format("%-22s %d/%d", name, cpass, #mod)`, the name padded to 22 then `4/4`.
4. The same loop over the chapter's nine modules adds 7 + 4 + 4 + 5 + 5 + 4 + 5 + 5 + 6 = 45 passed, 0 failed, and exits 0.
5. Had a block raised, `pcall` returns false, the error goes to stderr beside the module and block name, `failed` gains 1, and the run ends in `os.exit(1)`.

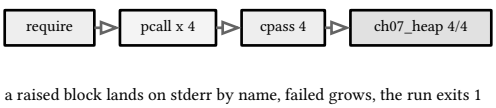


figure 75: one module through the runner loop, from require to the printed line

One ok line per module is the whole output contract, `ch07_heap 4/4` among them, and the two listings below are the files that produce it.

Two files close the runner story. `lib.lua` is the assertion module the opening paragraph describes, the `T.eq`, `T.ok`, and `T.throws` the module tests call, and the most required file in the suite, 82 modules load it. `run.lua` is the top of the graph and is required by nothing: it loads each `chNN` module as its table of `(name, fn)` blocks, runs each block's asserts under `pcall`, and prints one ok line per module with the pass count over the block count, exiting nonzero on any failure. With both printed, every `require` the book's lua solvers make now resolves to a printed file.

```lua
local function deep(a, b)
  if a == b then return true end
  if type(a) ~= "table" or type(b) ~= "table" then return false end
  local seen = 0
  for k, v in pairs(a) do
    if not deep(v, b[k]) then return false end
    seen = seen + 1
  end
  for _ in pairs(b) do seen = seen - 1 end
  return seen == 0
end

-- compares with lua equality first, then structurally for tables
function M.eq(actual, expected, msg)
  if not deep(actual, expected) then
    error((msg or "values differ") .. ": got " .. show(actual) .. ", want " .. show(expected), 2)
  end
end

function M.ok(cond, msg)
  if not cond then error(msg or "assertion failed", 2) end
end

-- requires f to raise, with want as a substring of the message
function M.throws(f, want)
  local ok, err = pcall(f)
  if ok then error("expected an error, call returned " .. show(err), 2) end
  if want and not tostring(err):find(want, 1, true) then
    error("error message " .. show(tostring(err)) .. " lacks " .. show(want), 2)
  end
  return err
end
```

listing 100: lua: the assertion library, deep through tables for eq, ok as the plain gate, throws demanding the named error (source: icpc/samples-lua/lib.lua, lines 17-48)

```lua
package.path = "./?.lua;" .. package.path

local failed = 0
local passed = 0
for _, name in ipairs(chapters) do
  local mod = require(name)
  local cpass = 0
  for _, t in ipairs(mod) do
    local ok, err = pcall(t.fn)
    if ok then
      cpass = cpass + 1
    else
      failed = failed + 1
      io.stderr:write(name .. " :: " .. t.name .. " FAILED: " .. tostring(err) .. "\n")
    end
  end
  passed = passed + cpass
  print(string.format("%-22s %d/%d", name, cpass, #mod))
end
print(string.format("total %d passed, %d failed", passed, failed))
if failed > 0 then os.exit(1) end
```

listing 101: lua: the runner engine, path prepend, one pcall per block, one count line per module, exit 1 on failure (source: icpc/samples-lua/run.lua, lines 88-108)

9.2. big integers over limb tables

The bigint keeps the same representation as the C twin, sign and magnitude over base one billion limbs, least significant first, but the limbs live in a plain Lua table that grows without a declared budget, and zero is `sign = 0`, where C pins zero at sign plus one. Parsing walks the decimal string nine digits at a time from the right, `tonumber` on each slice, and printing zero-pads every limb but the top with `%09d`, the same reassembly trick the C chapter shows. The zero-detection at parse time is a small honesty of its own: an all-zero input like `-000` parses to the zero sign, never to a negative zero.

The dry run: the module's test `small division with pinned quotient and remainder` feeds `smalldiv(parse("1000000000000000000"), 7)` and asserts the pair with `T.eq(tostr(q), "1428571428571428571")` and `T.eq(r, 3)`.

1. Parse cuts the twenty-digit string nine at a time from the right: three limbs, `{0, 0, 10}` little-endian, the top limb carrying the `10` and the two zero limbs the eighteen trailing zeros.
2. Division starts at the top limb with `rem = 0`: `cur = 0 x 1000000000 + 10 = 10`, quotient limb `10 // 7 = 1`, `rem = 10 % 7 = 3`.
3. One limb down: `cur = 3 x 1000000000 + 0 = 3000000000`, quotient limb `3000000000 // 7 = 428571428`, `rem = 3000000000 % 7 = 4`.
4. The bottom limb: `cur = 4 x 1000000000 + 0 = 4000000000`, quotient limb `4000000000 // 7 = 571428571`, `rem = 4000000000 % 7 = 3`.
5. `tostr` prints the top quotient limb bare and zero-pads the rest with `%09d`, so `1`, `428571428`, `571428571` concatenate to 19 digits.

step	limb	cur = rem x 10^9 + limb	out	rem
1	cut	20 digits, right to left	{0, 0, 10}	0
2	10	10	1	3
3	0	3000000000	428571428	4

step	limb	cur = rem x 10^9 + limb	out	rem
4	0	4000000000	571428571	3
5	print	1, 428571428, 571428571	1428571428571428571	3

Every row is one division step or one pad, and the last row is what the two checks read: quotient 1428571428571428571, remainder 3.

```lua
local function parse(s)
  local sign = 1
  if s:sub(1, 1) == "-" then
    sign = -1
    s = s:sub(2)
  end
  local limbs = {}
  -- walk 9 digits at a time from the right: little-endian
  local i = #s
  while i > 0 do
    local lo = math.max(1, i - 8)
    limbs[#limbs + 1] = tonumber(s:sub(lo, i))
    i = lo - 1
  end
  return norm({sign = s:match("^0+$") and 0 or sign, limbs = limbs})
end

local function tostr(b)
  if b.sign == 0 then return "0" end
  local out = {}
  for i = #b.limbs, 1, -1 do
    out[#out + 1] = (i == #b.limbs) and tostring(b.limbs[i])
      or string.format("%09d", b.limbs[i])
  end
  return (b.sign < 0 and "-" or "") .. table.concat(out)
end
```

listing 102: lua: parse cuts nine digits from the right into a limb table, tostring reassembles with the zero pad (source: icpc/samples-lua/ch07_bigint.lua, lines 24-49)

Signed addition dispatches on the signs with two shortcuts C does not need: an operand with `sign == 0` returns the other operand unchanged, and equal opposite magnitudes collapse to zero explicitly. Subtraction is addition of a negated copy, the negation copying the limb table so the original is never mutated.

```lua
local function add(a, b)
  if a.sign == 0 then return b end
  if b.sign == 0 then return a end
  if a.sign == b.sign then
    local r = mag_add(a, b)
    r.sign = a.sign
    return r
  end
  local c = cmp_mag(a, b)
  if c == 0 then return {sign = 0, limbs = {}} end
  if c > 0 then
    local r = mag_sub(a, b)
    r.sign = a.sign
    return r
  end
  local r = mag_sub(b, a)
  r.sign = b.sign
  return r
end

local function sub(a, b)
  return add(a, neg(b))
end
```

listing 103: lua: signed add with the zero shortcuts and the magnitude dispatch, sub as add of negated (source: icpc/samples-lua/ch07_bigint.lua, lines 99-121)

Multiplication is schoolbook with the carry folded inline, limb i of a times limb j of b accumulating at `i + j - 1`, one-based, the carry spilling into the next column as each row completes. The C twin accumulates all columns first and carries once at the end. The comment in the code is the whole safety argument: a product of two base one billion limbs is under `10^18` and under the `2^62` wrap line, so the limb-level arithmetic a bigint needs never wraps even though the machine will wrap a careless product happily.

```lua
local function mul(a, b)
  if a.sign == 0 or b.sign == 0 then return {sign = 0, limbs = {}} end
  local out = {}
  for i = 1, #a.limbs do
    local carry = 0
    for j = 1, #b.limbs do
      local k = i + j - 1
      -- each product of two base-1e9 limbs is < 1e18 < 2^62: no wrap
      local s = (out[k] or 0) + a.limbs[i] * b.limbs[j] + carry
      out[k] = s % BASE
      carry = s // BASE
    end
    out[i + #b.limbs] = (out[i + #b.limbs] or 0) + carry
  end
  return norm({sign = a.sign * b.sign, limbs = out})
end
```

listing 104: lua: schoolbook mul, carry folded per row, the under-2^62 comment carrying the safety argument (source: icpc/samples-lua/ch07_bigint.lua, lines 124-139)

Small division walks from the top limb down, remainder times base plus limb, and its bound is stated beside the multiplication's: the running value stays under d times a billion, still under `2^62`. The fixtures differ from C's on purpose, each language pins its own mini-inputs: here `10^19` over 7 leaves quotient `1428571428571428571` and remainder 3, and `-100` over 7 truncates to `-14` with the magnitude remainder 2, the sign riding the quotient

alone, where the C suite pins `10^21` over 7 with remainder 6. The property test is the one worth stealing: multiply distributes over add, `c*(a+b) == c*a + c*b`, checked at magnitudes past what any int64 holds.

```
local function smalldiv(a, d)
  local out = {}
  local rem = 0
  for i = #a.limbs, 1, -1 do
    local cur = rem * BASE + a.limbs[i] -- rem < d, cur < d*1e9 < 2^62
    out[i] = cur // d
    rem = cur % d
  end
  return norm({sign = a.sign, limbs = out}), rem
end
```

listing 105: lua: small division top-down with the remainder, the bound stated where it is used (source: icpc/samples-lua/ch07_bigint.lua, lines 142-151)

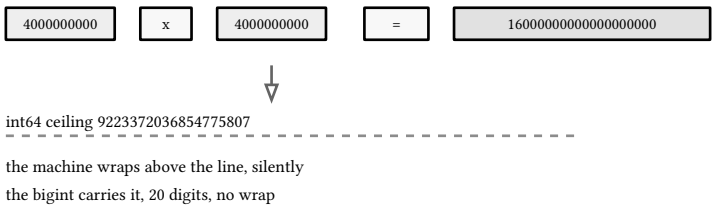


figure 76: why the toolbox needs a bigint: the 4e9 squared product sits past the int64 ceiling, and the machine says nothing

9.3. the heap with a comparator

The Lua heap is the same array-backed binary min-heap as the C one, sift up on push and smaller-child sift down on pop, with one design difference that buys a lot: the comparator is injected at construction. The default is plain `<`, a reversed comparator turns the same structure into a max-heap, and a tuple comparator over `(cost, id)` pairs breaks ties on the id, which is exactly the shape a Dijkstra frontier wants. The C twin hard-codes the order by key type instead, and the year chapters pick per language accordingly.

The dry run: the module's test `interleaved pushes and pops keep the winner on top` mixes 7, 1, 3, and 0 through the default comparator and asserts every return, five `T.eq` pops and peeks closing on `T.eq(Heap.size(h), 0)`.

1. `Heap.push(h, 7)` leaves `[7]`; `Heap.push(h, 1)` appends then sifts once, 1 under 7 swaps them to `[1, 7]`.
2. `Heap.pop(h)` returns 1, the last element 7 takes the root with no challenger below, and the array is `[7]`.
3. `Heap.push(h, 3)` sifts to `[3, 7]`; `Heap.push(h, 0)` appends to `[3, 7, 0]` then swaps twice, 0 past 3 into the root, `[0, 7, 3]`.
4. `Heap.peek(h)` reads that root, 0.
5. The drain pops 0, 3, then 7, the last element walking rootward each time, until the array empties and `Heap.size(h)` reads 0.

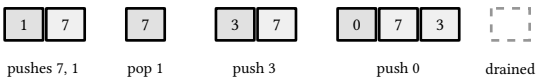


figure 77: the array after each interleaved operation, root shaded, the winner never leaving the top

The winner sits on top after every push and pops in exactly the order the asserts pin, 1 then 0 then 3 then 7, ending at the empty heap's size 0.

```lua
function Heap.new(cmp)
  return {a = {}, cmp = cmp or function(x, y) return x < y end}
end

function Heap.push(h, v)
  local a = h.a
  a[#a + 1] = v
  local i = #a
  while i > 1 and h.cmp(a[i], a[i // 2]) do
    a[i // 2], a[i] = a[i], a[i // 2]
    i = i // 2
  end
end

function Heap.peek(h)
  return h.a[1]
end

function Heap.pop(h)
  local a = h.a
  local top = a[1]
  a[1] = a[#a]
  a[#a] = nil
  local i, n = 1, #a
  while true do
    local l, r, m = 2 * i, 2 * i + 1, i
    if l ≤ n and h.cmp(a[l], a[m]) then m = l end
    if r ≤ n and h.cmp(a[r], a[m]) then m = r end
    if m == i then break end
    a[i], a[m] = a[m], a[i]
    i = m
  end
  return top
end
```

listing 106: lua: heap with an injected comparator, default <, sift up and sift down (source: icpc/samples-lua/ch07_heap.lua, lines 8-41)

The fixture suite runs the default order ascending over `5 3 8 1 9 2`, reverses the comparator for the max-heap, and pins the tuple order: `(4,20)`, `(2,99)`, `(4,10)`, `(2,7)` pushed in that order pop as `(2,7)`, `(2,99)`, `(4,10)`, `(4,20)`. Interleaved pushes and pops keep the winner on top throughout. The module is 91 lines against the C header's 77, and the difference is the comparator plumbing plus Lua's voice, one function per operation instead of one `static inline` block.

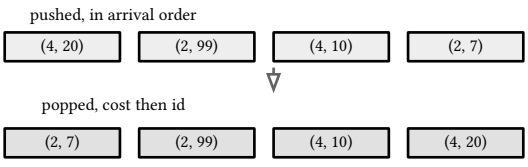


figure 78: the tuple comparator at work: four pushed pairs and the pop order, cost first, id breaking ties

9.4. the ring deque, rotating

The deque mirrors the C ring, head index, count, modulo on both ends, growth by doubling with a rebase to logical index zero, and it adds the one instrument the C header leaves implicit: a `copies` counter that grows by the live count on every doubling. The test reads it directly, a four-slot ring that fills, wraps across the seam, and doubles holds six entries with exactly 4 copies paid, the number the amortized argument predicts for that growth.

The dry run: the module's test `rotate left and right by steps` loads 1 through 5 into a cap 8 ring and asserts the array after each of three rotations with `T.eq(Deque.to_array(q), ...)`.

1. Five `push_back` land at slots 1 through 5 with head 0, count 5, the array `{1, 2, 3, 4, 5}`.
2. `rotate(q, 2)` runs `n = 2 % 5 = 2` pop-front push-back rounds: the 1 moves to slot 6, the 2 to slot 7, head ends at 2, and `to_array` reads slots 3 through 7 as `{3, 4, 5, 1, 2}`.
3. `rotate(q, -2)` takes the floored `n = -2 % 5 = 3`, three rounds the other way: 3 to slot 8, 4 to slot 1, 5 to slot 2, head ends at 5, and the array is `{1, 2, 3, 4, 5}` again.
4. `rotate(q, 5)` computes `n = 5 % 5 = 0` and runs no rounds at all, the pinned full turn that changes nothing.

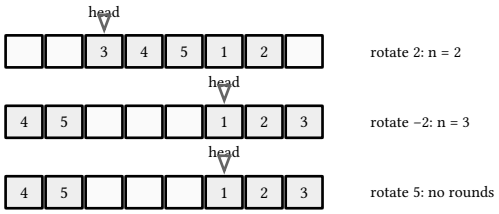


figure 79: the cap 8 slots across the three rotations, live entries and the head, the values wrapping the seam

The ring ends where it began, `{1, 2, 3, 4, 5}`, after two steps left, two right, and a full turn, the three asserts in order.

```lua
local function grow(q)
  local ns = {}
  for i = 0, q.count - 1 do
    ns[i + 1] = q.slots[(q.head + i) % q.cap + 1]
  end
  q.copies = q.copies + q.count
  q.slots = ns
  q.head = 0
  q.cap = q.cap * 2
end

function Deque.push_back(q, v)
  if q.count == q.cap then grow(q) end
  q.slots[(q.head + q.count) % q.cap + 1] = v
  q.count = q.count + 1
end

function Deque.push_front(q, v)
  if q.count == q.cap then grow(q) end
  q.head = (q.head + q.cap - 1) % q.cap
  q.slots[q.head + 1] = v
  q.count = q.count + 1
end
```

listing 107: lua: doubling growth with the copies counter, both pushes wrapping by modulo (source: icpc/samples-lua/ch07_deque.lua, lines 11-33)

Both pops return `nil` on empty, Lua's refusal shape where C returns 0 through an out parameter, and rotation is defined once instead of left to the caller: pop front, push back, `n % count` times, so negative steps rotate right and a full turn changes nothing. The rotate fixture walks `1 2 3 4 5` left two steps to `3 4 5 1 2`, back right two, and around once, ending where it started. The emptying slots are `nil`-ed rather than overwritten, which matters to Lua's `#` and to any iteration that would otherwise see stale values.

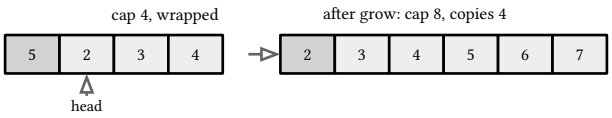


figure 80: the wrapped four-ring that grows: six live entries land contiguous, the copies counter reading exactly 4

9.5. the set, deterministic iteration

A set over a Lua table is membership by presence, `items[v] = true`, with a count kept beside it, and the two interesting decisions are both about honesty. `has` checks `items[v] == true` rather than truthiness, because a set holding `false` stores `true` in its slot and a naive lookup would report the member missing. And iteration is sorted: Lua's `pairs` order is unspecified, so a set whose walk order matters, and puzzle code almost always wants a reproducible walk, collects the keys and sorts them by string form before yielding them. Union and intersect are one pass each over the operands.

The dry run: the module's test `union and intersect on the fixture sets` builds `a = {1, 2, 3, 4}` and `b = {3, 4, 5, 6}` and asserts both results as sorted arrays and counts, four `T.eq` calls.

1. `Set.union` starts empty and adds `a`'s four keys, count 4.
2. `b`'s keys follow: 3 and 4 are members already and `add` skips them, 5 and 6 join, 4 + 2 = 6 members.
3. `Set.intersect` walks `a`'s four keys and keeps the ones `b` has: 1 and 2 fail `has`, 3 and 4 pass, 2 kept.
4. `to_array` sorts by string form, so the asserts read `{1, 2, 3, 4, 5, 6}` with count 6 and `{3, 4}` with count 2.

value	in a	in b	union keeps	intersect keeps
1	yes	no	yes, from a	no
2	yes	no	yes, from a	no
3	yes	yes	yes, from a	yes
4	yes	yes	yes, from a	yes
5	no	yes	yes, from b	no
6	no	yes	yes, from b	no

All six values reach the union and only the shared 3 and 4 survive the intersect, the counts 6 and 2 the last two asserts pin.

```lua
function Set.each(s)
  local keys = {}
  for k in pairs(s.items) do keys[#keys + 1] = k end
  table.sort(keys, function(x, y) return tostring(x) < tostring(y) end)
  local i = 0
  return function()
    i = i + 1
    return keys[i]
  end
end

function Set.to_array(s)
  local out = {}
  for v in Set.each(s) do out[#out + 1] = v end
  return out
end

function Set.union(a, b)
  local out = Set.new()
  for k in pairs(a.items) do Set.add(out, k) end
  for k in pairs(b.items) do Set.add(out, k) end
  return out
end

function Set.intersect(a, b)
  local out = Set.new()
  for k in pairs(a.items) do
    if Set.has(b, k) then Set.add(out, k) end
  end
  return out
end
```

listing 108: lua: sorted each for a deterministic walk, union and intersect in one pass each (source: icpc/samples-lua/ch07_set.lua, lines 29-59)

The fixtures cover the arithmetic, `{1,2,3,4}` union `{3,4,5,6}` is all six and the intersect is `3,4`, the disjoint intersect is empty and union with empty is identity, and the boolean members test pins the `== true` rule with `false` itself in the set. The C twin has no set module at all: its map with ignored values is the set, and the year chapters use whichever shape their language gave them.

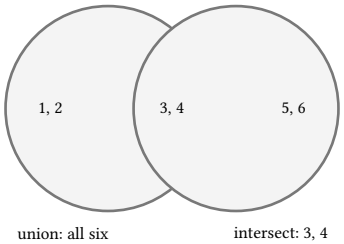


figure 81: union and intersect of the two fixture sets, the two shared members in the middle

9.6. the grid

The grid module parses a list of row strings into one concatenated string and keeps `rows` and `cols` beside it, the same row-major flattening the C header performs on a byte buffer, and the same index law, `r * cols + c`, zero-based, with bounds checks and the two delta tables. The difference is what the neighbor walk returns: C counts the in-bounds neighbors, Lua returns the in-bounds coordinate pairs themselves in delta order, which is what a flood fill or a walk loop consumes directly.

The dry run: the module's test `parse flattens row-major: at() reads through the index math` parses `{"abc", "def"}` and asserts the corners and the raw law itself with `T.eq(Grid.idx(g, 1, 2), 5)`.

1. `Grid.parse` concats the rows, `cells = "abcdef"`, with `rows = 2` and `cols = 3`.
2. `at(0, 0)` computes `idx = 0 x 3 + 0 = 0` and reads `sub(1, 1)`, the `a`.
3. `at(0, 2)` computes `0 x 3 + 2 = 2`, reads `sub(3, 3)`, the `c`; `at(1, 0)` computes `1 x 3 + 0 = 3`, reads the `d`.
4. `at(1, 2)` computes `1 x 3 + 2 = 5`, the assert that pins the law, and reads `sub(6, 6)`, the `f`.

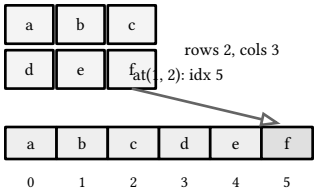


figure 82: the two rows flatten to one string, every (r, c) landing at r x cols + c

Every read lands on its pinned letter through the one law, the `f` at `idx = 5` closing the walk.

```lua
function Grid.parse(lines)
  return {rows = #lines, cols = #lines[1], cells = table.concat(lines)}
end

-- 0-based coordinates everywhere
function Grid.idx(g, r, c)
  return r * g.cols + c
end

function Grid.at(g, r, c)
  return g.cells:sub(Grid.idx(g, r, c) + 1, Grid.idx(g, r, c) + 1)
end

function Grid.in_bounds(g, r, c)
  return r ≥ 0 and r < g.rows and c ≥ 0 and c < g.cols
end

Grid.D4 = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}} -- up down left right

Grid.D8 = {
  {-1, -1}, {-1, 0}, {-1, 1},
  {0, -1}, {0, 1},
  {1, -1}, {1, 0}, {1, 1},
}

-- in-bounds neighbors only, delta order preserved
function Grid.neighbors(g, r, c, deltas)
  local out = {}
  for _, d in ipairs(deltas or Grid.D4) do
    local nr, nc = r + d[1], c + d[2]
    if Grid.in_bounds(g, nr, nc) then
      out[#out + 1] = {nr, nc}
    end
  end
  return out
end
```

listing 109: lua: parse by concat, the index law, both delta tables, neighbors as coordinate pairs (source: icpc/samples-lua/ch07_grid.lua, lines 7-42)

The delta orders are documented per language and they differ: both toolboxes walk the 4-set as up, down, left, right, but the C 8-table lists the four axis steps first and the diagonals after, while the Lua 8-table goes around the cell row by row. Neither order is wrong, each is pinned by its own tests, and the year chapters cite the order they rely on instead of assuming one. The corner fixtures are geometry: a 3 by 3 grid gives the center all eight, a corner three, a 1 by 1 grid none at all.

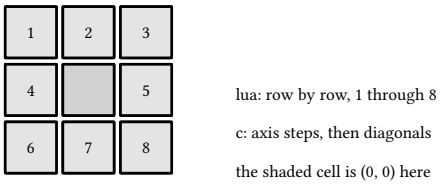


figure 83: the lua 8-neighbor order around a center cell, walked row by row, against the c axis-first order

9.7. string utilities, concat not squares

Four helpers cover what the puzzles keep doing to text. `split` walks a literal separator with `find` in plain mode and keeps empty fields, so `a,b,,c` splits to four strings with the hole preserved, the behavior token-counting loops want. `trim` is one anchored `gsub`. `mapchars` drives character substitution through a table, rot13 being the fixture, and `build` is the idiom this section exists for: assemble a parts table in the loop and `table.concat` once, because every `..` in a loop body allocates a fresh string and a loop of concatenations squares the byte count it moves.

The dry run: the module's test `split keeps empty fields and returns the tail` walks `"a,b,,c"` with `find` in plain mode and pins the walk with `T.eq(split("a,b,,c", ","), {"a", "b", "", "c"})`.

1. The cursor opens at `pos = 1`; the first comma sits at 2, the field is `sub(1, 1) = "a"`, and the cursor moves past it to 3.
2. The comma at 4 yields `sub(3, 3) = "b"`, cursor 5.
3. At 5 the next comma is immediate: the field is `sub(5, 4)`, an empty range, the kept hole, cursor 6.
4. `find` meets no comma from 6, so the tail `sub(6, 6) = "c"` is the fourth field and `split` returns.

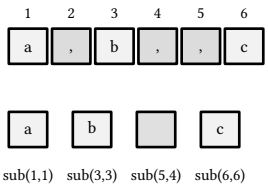


figure 84: the cursor over a,b,,c: three commas, four fields, the adjacent pair at 4 and 5 leaving the hole

Three commas cut four fields, the hole sitting between the adjacent pair, and the pinned table keeps all four.

```lua
local function split(s, sep)
  local out = {}
  local pos = 1
  while true do
    local a, b = s:find(sep, pos, true)
    if not a then
      out[#out + 1] = s:sub(pos)
      return out
    end
    out[#out + 1] = s:sub(pos, a - 1)
    pos = b + 1
  end
end
```

listing 110: lua: split on a literal separator, empty fields kept, the tail returned (source: icpc/samples-lua/ch07_strutil.lua, lines 7-19)

```lua
local function trim(s)
  return (s:gsub("^%s*(.-)%s*$", "%1"))
end

-- map characters through a table: unmapped characters pass through
local function mapchars(s, map)
  local parts = {}
  for i = 1, #s do
    local c = s:sub(i, i)
    parts[#parts + 1] = map[c] or c
  end
  return table.concat(parts)
end

-- build a string from a loop via table.concat, the idiom
local function build(n, f)
  local parts = {}
  for i = 1, n do parts[#parts + 1] = f(i) end
  return table.concat(parts)
end
```

listing 111: lua: trim by anchored gsub, table-driven char map, build through table.concat (source: icpc/samples-lua/ch07_strutil.lua, lines 22-41)

The fixtures pin all four: the split holes, a trim that never touches interior whitespace, rot13 turning `Hello, World!` into `Uryyb, Jbeyq!` with digits untouched, and `build` assembling <1><2><3> in one concat. C has no equivalent module, its string.h idiom is pointers and `strchr`, and the honest comparison is that Lua's string helpers exist because Lua strings are immutable and every operation returns a new one, so the toolbox teaches the allocation-conscious pattern up front.

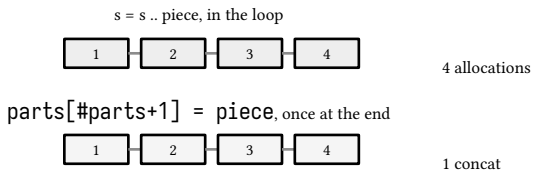


figure 85: two loops over four pieces: a dot in the loop body allocates four fresh strings, the parts table pays one concat

9.8. number theory, wrap stated then avoided

The number module opens by committing the crime it exists to prevent. Its first test asserts `math.maxinteger * 2 == -2`, the wrap made visible: the largest int64 doubled is minus two, two's complement running off the end, and no error, no warning, nothing but a wrong number. That single line is the whole argument for everything below it. The mulmod is the same add-and-double the C chapter landed on, doubling one operand with a reduction every step, arrived at from the opposite direction: C could not link a 128-bit remainder, Lua cannot trust a 64-bit product.

The dry run: the module's test `the wrap boundary stated, then avoided` opens by asserting the wrap, `T.eq(math.maxinteger * 2, -2)`, then pins the two anchors with `T.eq(mulmod(6000000000, 6000000000, 1000000007), 1764)` and `T.eq(modpow(2, 100, 1000000007), 976371285)`.

1. The wrap stated: `2^63 - 1` doubled runs off the int64 end and the machine reports `-2`, no error anywhere.
2. One reduction: `10^9 = 1000000007 - 7`, so `6 x 10^9` is congruent to `6 x (-7) = -42`, and `-42 + 1000000007 = 999999965`.
3. The product never forms: `(-42) x (-42) = 1764`, already under the modulus, the pinned mulmod answer.

4. Inside the loop every intermediate stays under one modulus plus itself, `1000000007 + 1000000007 = 2000000014`, far under the `2^62` wrap line.
5. `modpow(2, 100, 1000000007)` chains the same mulmod through its squarings and lands on the pinned 976371285.

step	line	value
wrap	`(2^63 - 1) x 2`	`-2`
reduce	`10^9 = 1000000007 - 7`, so `6 x 10^9 = 6 x (-7)`	`-42`
normalize	`-42 + 1000000007`	`999999965`
square	`(-42) x (-42)`, under the modulus	`1764`
pow	`2^100 mod 1000000007`, pinned	`976371285`

The fold reads 1764 off a congruence and 976371285 off the pinned power, both asserts satisfied without one wide product ever forming.

```lua
local function mulmod(a, b, m)
  local result = 0
  a = a % m
  while b > 0 do
    if b & 1 == 1 then result = (result + a) % m end
    a = (a + a) % m
    b = b >> 1
  end
  return result
end

local function modpow(base, exp, m)
  local result = 1 % m
  base = base % m
  while exp > 0 do
    if exp & 1 == 1 then result = mulmod(result, base, m) end
    base = mulmod(base, base, m)
    exp = exp >> 1
  end
  return result
end
```

listing 112: lua: the doubling mulmod and modpow, a reduction after every add and every double (source: icpc/samples-lua/ch07_num.lua, lines 12-32)

The anchors match the C suite exactly, the values every language in this book must agree on: `mulmod(6000000000, 6000000000, 1000000007)` is 1764 by the congruence a billion is minus seven, and `modpow(2, 100, 1000000007)` is 976371285. The inverse guard returns `nil` when the gcd is not one, Lua's refusal shape again, and the crt merge is a closed form through a recursive extended euclid, where the C twin scans residues linearly until one fits both congruences. Different algorithms, same answers, and both accept a consistent non-coprime pair, `1 mod 4` with `5 mod 6` resolving to `5 mod 12`, while rejecting the contradiction.

```lua
local function merge(r1, m1, r2, m2)
  local g, p = ext_gcd(m1, m2)
  if (r2 - r1) % g ~= 0 then return nil, nil end
  local lcm = m1 // g * m2
  local t = (r2 - r1) // g % (m2 // g) * p % (m2 // g)
  return (r1 + m1 * t) % lcm, lcm
end

local function crt(congs)
  if #congs == 0 then return nil end
  local r, m = congs[1][1], congs[1][2]
  for i = 2, #congs do
    r, m = merge(r, m, congs[i][1], congs[i][2])
    if r == nil then return nil end
  end
  return r, m
end
```

listing 113: lua: pairwise merge by the extended-euclid closed form, crt folding the list (source: icpc/samples-lua/ch07_num.lua, lines 48-64)

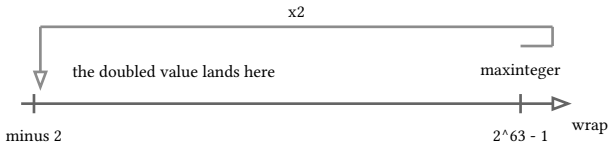


figure 86: the wrap, stated as a number line: maxinteger doubled runs off the end and reappears at minus two

9.9. md5 on 64-bit integers

The md5 module is the RFC 1321 construction again, and the Lua interest is arithmetic, not structure: Lua integers are 64-bit signed, there is no uint32 to compute in, so every operation that must land in 32 bits is masked back with `& 0xFFFFFFFF`, the rotate included, `((x << n) | (x >> (32 - n))) & M32`. The complement in the round functions is safe only because it is immediately intersected with a 32-bit value: `~b` alone is 64 bits of mostly ones, `~b & d` is back in range. The shift schedule is stored compressed, sixteen entries, and indexed `i//16*4 + i%4 + 1`, where the C header spells out all sixty-four.

The dry run: the module's test `md5 of the empty string, the rfc vector` runs the whole construction on zero bytes and pins it with `T.eq(md5(""), "d41d8cd98f00b204e9800998ecf8427e")`.

1. The message is empty, so `bitlen = 0 x 8 = 0` and padding begins with the single `0x80` byte, the table at 1 entry.
2. Zeros follow while `#bytes % 64 ~= 56`, which from 1 entry means 56 - 1 = 55 zeros.
3. Eight little-endian length bytes of zero follow, 56 + 8 = 64, one chunk exactly.
4. The sixteen words are all zero except `M[0]`, which reads the `0x80` little-endian, the value 128.
5. The 64 rounds fold the accumulators with every add masked back to 32 bits, and the four words print little-endian in a b c d order.

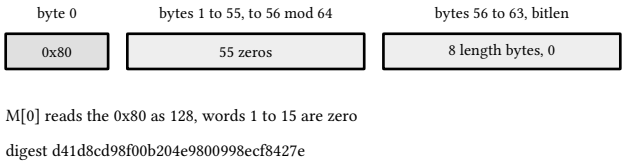


figure 87: the empty message's one chunk: one 0x80 byte, zeros to 56 mod 64, eight zero length bytes

One chunk, one nonzero word, sixty-four masked rounds, and the digest is the RFC's own `d41d8cd98f00b204e9800998ecf8427e`.

```lua
local function md5(msg)
  -- pad: 0x80, zeros to 56 mod 64, then the bit length little-endian
  local bytes = {}
  for i = 1, #msg do bytes[i] = msg:byte(i) end
  local bitlen = #msg * 8
  bytes[#bytes + 1] = 0x80
  while #bytes % 64 ~= 56 do bytes[#bytes + 1] = 0 end
  for i = 0, 7 do
    bytes[#bytes + 1] = (bitlen >> (8 * i)) & 0xFF
  end

  local a0, b0, c0, d0 = 0x67452301, 0xefcdab89, 0x98badcfe, 0x10325476

  for chunk = 1, #bytes // 64 do
    -- sixteen little-endian 32-bit words
    local M = {}
    local base = (chunk - 1) * 64
    for w = 0, 15 do
      local v = 0
      for b = 3, 0, -1 do
        v = (v << 8) | bytes[base + w * 4 + b + 1]
      end
      M[w] = v & M32
    end
```

listing 114: lua: pad to 56 mod 64, the bit length little-endian, sixteen words per chunk (source: icpc/samples-lua/ch07_md5.lua, lines 36-59)

```
    local a, b, c, d = a0, b0, c0, d0
    for i = 0, 63 do
      local f, g
      if i < 16 then
        f = (b & c) | (~b & d)
        g = i
      elseif i < 32 then
        f = (d & b) | (~d & c)
        g = (5 * i + 1) % 16
      elseif i < 48 then
        f = b ~ c ~ d
        g = (3 * i + 5) % 16
      else
        f = c ~ (b | (~d & M32))
        g = (7 * i) % 16
      end
      f = (f + a + K[i + 1] + M[g]) & M32
      a = d
      d = c
      c = b
      -- shift schedule: four values per round, repeating through it
      b = (b + rol(f, SHIFTS[i // 16 * 4 + i % 4 + 1])) & M32
    end
    a0 = (a0 + a) & M32
    b0 = (b0 + b) & M32
    c0 = (c0 + c) & M32
    d0 = (d0 + d) & M32
  end
```

listing 115: lua: the 64 rounds, every add masked back to 32 bits, the schedule indexed by round and step (source: icpc/samples-lua/ch07_md5.lua, lines 61-88)

The pinned vectors are the RFC set and they match the C suite's exactly, both languages reading one standard: the empty string, `a`, `abc`, `message digest`, the alphabet, and the 80-digit input whose padding crosses into a second block, `57edf4a22be3c955ac49da2e2107b67a`. Determinism is checked with the contest's own name in the input, `md5("icpc world finals")` twice, and one extra letter separating digests. Where the C chapter's suite pins ten checks, this one pins five blocks, the same vectors with fewer intermediate assertions, and that split is itself the honest comparison: the C tests assert each vector separately where Lua's blocks group them.

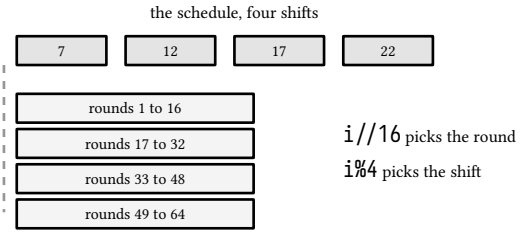


figure 88: the sixteen-entry shift schedule against the four rounds: every round reuses the same four shifts

9.10. json to plain tables

The json decoder is where the two toolboxes diverge the most, because the languages start from opposite places. C needs a node pool, tags, and child arrays to hold a tree at all, 244 lines of it. Lua is given tables, so the decoder walks the text once and returns plain Lua tables directly, objects with string keys, arrays with consecutive integers, and the whole `decode_value` is one function whose arms are the grammar.

The dry run: the module's test `the numeric-leaf sum walks objects and arrays alike` decodes `{"a": 1, "b": [2, 3.5, true]}` and pins the total with `T.eq(JSON.sum_numbers(...), 6.5)`.

1. `decode` meets `{`, reads key `a`, and its value `1` converts through `tonumber` as an integer; key `b` opens the array `[2, 3.5, true]`, both pinned by the earlier `T.eq(v.a, 1)` and `T.eq(v.b, {2, 3.5, true})`.
2. The sum opens at 0; the object's pair `a` yields its leaf, total `0 + 1 = 1`.
3. The pair `b` walks the array: 2 makes `1 + 2 = 3`, 3.5 makes `3 + 3.5 = 6.5`, and `true` is not a number and adds nothing.
4. The nested fixture `[1, [2, [3, [4]]]]` folds the same way, `1 + 2 + 3 + 4 = 10`, a bare string sums to 0, and a bare 7 to 7.

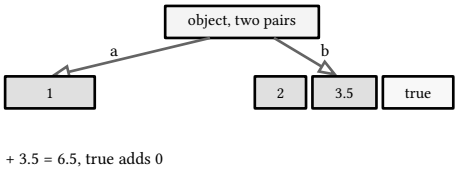


figure 89: the decoded table, numeric leaves shaded, the walk adding 1 + 2 + 3.5 = 6.5 and skipping true

The three numeric leaves total 6.5 and the walk returns it, the pinned sum the year chapters keep calling for.

```
local function decode_value(s, i)
  i = skip_ws(s, i)
  local c = s:sub(i, i)
  if c == "{" then
    local obj = {}
    i = skip_ws(s, i + 1)
    if s:sub(i, i) == "}" then return obj, i + 1 end
    while true do
      i = skip_ws(s, i)
      if s:sub(i, i) ~= '"' then error("expected a key string", 2) end
      local key
      key, i = decode_string(s, i)
      i = skip_ws(s, i)
      if s:sub(i, i) ~= ":" then error("expected ':'", 2) end
      local v
      v, i = decode_value(s, i + 1)
      obj[key] = v
      i = skip_ws(s, i)
      local sep = s:sub(i, i)
      if sep == "," then i = i + 1
      elseif sep == "}" then return obj, i + 1
      else error("expected ',' or '}'", 2) end
    end
  elseif c == "[" then
    local arr = {}
    i = skip_ws(s, i + 1)
    if s:sub(i, i) == "]" then return arr, i + 1 end
    while true do
      local v
      v, i = decode_value(s, i)
      arr[#arr + 1] = v
      i = skip_ws(s, i)
      local sep = s:sub(i, i)
      if sep == "," then i = i + 1
      elseif sep == "]" then return arr, i + 1
      else error("expected ',' or ']'", 2) end
    end
  elseif c == '"' then
    return decode_string(s, i)
  elseif s:sub(i, i + 3) == "true" then
    return true, i + 4
  elseif s:sub(i, i + 4) == "false" then
    return false, i + 5
  elseif s:sub(i, i + 3) == "null" then
    return JSON.null, i + 4
  else
    -- number: integer or decimal, optional minus
    local num = s:match("^-?%d+%.?%d*", i)
    if not num then error("unexpected token at " .. i, 2) end
    return tonumber(num), i + #num
  end
end
```

listing 116: lua: the whole value grammar in one function, objects, arrays, strings, literals, numbers (source: icpc/samples-lua/ch07_json.lua, lines 48-99)

Two decisions inside it carry the module. Null becomes a sentinel table with a tostring of `null`, because a Lua field holding `nil` does not exist, and a decoded `{"a": null}` must distinguish absent from null, which the C tree gets for

free from its explicit tags. And numbers are matched with `^-?%d+%.?%d*` and converted with `tonumber`, so `42` stays an integer and `3.5` a float, where C's `strtod` makes every number a double. The escapes include `\uXXXX`, decoded to two-byte UTF-8 inside the BMP, which the C twin does not attempt, and an unknown escape raises by name, where C passes it through verbatim.

```
function JSON.decode(s)
  local v, i = decode_value(s, 1)
  if skip_ws(s, i) ≤ #s then error("trailing garbage", 2) end
  return v
end

-- sum every number anywhere in the decoded tree
function JSON.sum_numbers(v)
  if type(v) == "number" then return v end
  if type(v) ~= "table" then return 0 end
  local total = 0
  for _, x in pairs(v) do
    total = total + JSON.sum_numbers(x)
  end
  return total
end
```

listing 117: lua: the entry point refusing trailing garbage, and the numeric-leaf sum walk (source: icpc/samples-lua/ch07_json.lua, lines 101-116)

The fixtures pin the decode shapes, the escapes, the sentinel, the integer-or-float distinction, five malformed inputs each refused by name, and the sum walk that this book's year chapters keep calling: `{"a": 1, "b": [2, 3.5, true]}` sums to 6.5, nested arrays to 10, a bare string to 0. The C fixture sums to 2.5 with its own crafted document, and the two suites agree on the method, walk every table, add every number, ignore everything else, while pinning their own mini-inputs, exactly the fixture policy chapter 1 states.

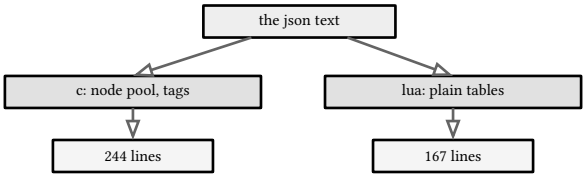


figure 90: the same grammar, two destinations: c parses into a tagged node pool, lua returns the tables the language already has

9.11. across the eight languages

The C chapter closes with the structure table from its side, and this chapter closes with the same table from Lua's. The rows are the same structures, the approaches are what the two chapters showed, and the reasons are the machine facts each language lives under.

structure	c builds it as	lua builds it as	why they differ
bigint	sign-magnitude struct, 12 fixed limbs	sign plus limb table, unbounded	c pins the digit budget, lua lets tables grow
map and set	open addressing, fnv-1a, tombstones	the table primitive, sorted walks	c has no hash map, lua has nothing else
min heap	typed array of key-value pairs	array plus injected comparator	one c type order, any lua order
deque	ring with head index and count	ring with head, count, copies	the lua test reads the copy count directly

structure	c builds it as	lua builds it as	why they differ
grid	pure index functions over a buffer	parse to one string, neighbors as pairs	same law, lua hands back coordinates
strings	pointers and strchr	split, trim, concat idiom	lua strings are immutable, allocation is visible
modular num	add-and-double, the link refused the wide product	add-and-double, the wrap is proven first	different diseases, same cure
md5	uint32 arithmetic	int64 masked to 32 bits	lua has no 32-bit integers to use
json	node pool with tags	plain tables, null sentinel	c needs the tree, lua is the tree

Java's chapter 5 contributes the seam no row above carries: the jvm's default stack dies at 22774 frames where `-Xss512m` holds 33506905, a measured fact its judge invocation pins before any deep recursion runs. Rust's chapter 3 contributes the pole the modular row sits between: native `u128` makes the wide product one expression, so neither c's refused link nor lua's proven wrap applies, and its `BTreeMap` replaces the tombstone walk with iteration that is key order by contract.

Verified by `make verify-lua`: 9 modules, 45 checks, bigint 7, heap 4, deque 4, set 5, grid 5, strutil 4, num 5, md5 5, json 6, on the repo's vendored Lua 5.5.1 interpreter.

sources: R. Rivest, RFC 1321, The MD5 Message-Digest Algorithm, April 1992, the constants, schedule, and test vectors both this suite and the C suite pin; the wrap behavior, the interpreter version, and the per-module check counts are output from the suite run and the vendored interpreter on this machine, 2026-09-14. The C comparisons cite the landed `books/icpc/samples-c/src/Ch02/` headers read the same day.

10. icpc world finals 2017

The 2017 World Finals ran in Rapid City, South Dakota with twelve problems, A through L on the contest pdf. By the solved-by counts printed in the official solutions, E, I and C fell to 127, 127 and 105 teams while H fell to none and J to exactly one, so this set pairs reach problems with two that punished every early guess. It was also the first finals where Python was offered, and the judges wrote more Python solutions than the teams did. The problem statements are paraphrased in this book's own words, and the cached official solutions pdf is the algorithm reference for every method and complexity claim in this chapter, cited per problem. Each section walks one problem through the same seven solvers, pins a crafted fixture against a hand-computed answer, and closes with one diagram of the method.

10.1. the year-local helpers: chord geometry, matching, flow, and rank selection

Four helper kits are year-local to 2017, one per problem that wanted one, and everything else the year's solvers import comes from the wave 1 toolbox chapters 2 through 9. Each kit states its contract and prints its twin files below in the book's fixed language order.

The geom kit backs problem A with the year's hardest contract: given a chord's two supporting vertices, classify every contact the island boundary makes with the chord's line, transversal crossing, touch-and-return, or a run along the line, and return the longest closed segment of that line lying on the island. C keeps every breakpoint an exact fraction n over m compared in `__int128` inside `Ch08/ch08_geom.h`'s `geo_events`, go walks doubles whose integer crosses stay exact at these coordinates through `ch08/ch08_geom.go`, and lua's `ch08_geom.lua` exposes one `best_span` that does the whole parity walk. Java's `Ch09/Geom.java` ports the c contract whole, the same exact fractions with the comparison cross-multiplication in `BigInteger` where c reached for `__int128`. C#, JavaScript and Python keep their own versions inside their problem A solvers. A maximal chain of on-line vertices is what the kit earns its keep on: the chain toggles the crossing parity exactly once, at its entry and its exit, and the lua solver in section A is the thin driver that hands every vertex pair to it.

```c
// collects the boundary events of the line through p_i with direction d
// (d = p_j - p_i). returns the event count; collinear edge spans are
// written to span_lo/span_hi (as fraction pairs), count in *spans
static inline int geo_events(const GPoint *poly, int n, int i, GPoint d,
                             GEvent *ev, GFr *span_lo, GFr *span_hi,
                             int *spans) {
  int cnt = 0;
  *spans = 0;
  long long dd = geo_dot(d, d);
  for (int k = 0; k < n; k++) {
    long long s = geo_cross(d, geo_sub(poly[k], poly[i]));
    if (s != 0)
      continue;
    GFr t = {geo_dot(d, geo_sub(poly[k], poly[i])), dd};
    long long sp = geo_cross(d, geo_sub(poly[(k + n - 1) % n], poly[i]));
    long long sn = geo_cross(d, geo_sub(poly[(k + 1) % n], poly[i]));
    int crossing = sp != 0 && sn != 0 && ((sp > 0) != (sn > 0));
    if (sn == 0 && sp != 0) {
      // start of a boundary run along the line: the boundary crosses
      // here when the side it leaves on after the run is opposite to
      // the side it arrived from before the run
      int j = (k + 1) % n;
      while (geo_cross(d, geo_sub(poly[j], poly[i])) == 0)
        j = (j + 1) % n;
      long long after = geo_cross(d, geo_sub(poly[j], poly[i]));
      crossing = (sp > 0) != (after > 0);
    }
    ev[cnt].t = t;
    ev[cnt].crossing = crossing;
    cnt++;
    if (sp == 0) { // edge (k-1, k) lies on the line
      GFr t2 = {geo_dot(d, geo_sub(poly[(k + n - 1) % n], poly[i])), dd};
      if (geo_fr_cmp(t, t2) > 0) {
        span_lo[*spans] = t2;
        span_hi[*spans] = t;
      } else {
        span_lo[*spans] = t;
        span_hi[*spans] = t2;
      }
      (*spans)++;
    }
  }
```

listing 118: c: the geo_events vertex walk, on-line vertices classified touch or transversal, collinear edges kept as covered spans (source: icpc/samples-c/src/Ch08/ch08_geom.h, lines 46-87)

```go
type vec2 struct{ x, y float64 }

func (a vec2) sub(b vec2) vec2   { return vec2{a.x - b.x, a.y - b.y} }
func (a vec2) dot(b vec2) float64 { return a.x*b.x + a.y*b.y }

func cross3(o, a, b vec2) float64 {
  return (a.x-o.x)*(b.y-o.y) - (a.y-o.y)*(b.x-o.x)
}

func distPointSeg(p, a, b vec2) float64 {
  ab := b.sub(a)
  ap := p.sub(a)
  if ab.dot(ab) == 0 {
    return math.Hypot(ap.x, ap.y)
  }
  t := ap.dot(ab) / ab.dot(ab)
  if t < 0 {
    t = 0
  } else if t > 1 {
    t = 1
  }
  closest := vec2{a.x + t*ab.x, a.y + t*ab.y}
  return math.Hypot(p.x-closest.x, p.y-closest.y)
}

// insidePoly reports whether p lies in the polygon or on its border.
func insidePoly(poly []vec2, p vec2) bool {
  for i := range poly {
    if distPointSeg(p, poly[i], poly[(i+1)%len(poly)]) < 1e-7 {
      return true
    }
  }
  inside := false
  for i := range poly {
    a, b := poly[i], poly[(i+1)%len(poly)]
    if (a.y > p.y) != (b.y > p.y) {
      xHit := a.x + (p.y-a.y)*(b.x-a.x)/(b.y-a.y)
      if xHit > p.x {
        inside = !inside
      }
    }
  }
  return inside
}
```

listing 119: go: cross3 and distPointSeg exact in float64 at these coordinates, insidePoly boundary-inclusive by ray parity (source: icpc/samples-go/ch08/ch08_geom.go, lines 10-53)

```
static int frCmp(Fr a, Fr b) {
  BigInteger l = BigInteger.valueOf(a.n).multiply(BigInteger.valueOf(b.m));
  BigInteger r = BigInteger.valueOf(b.n).multiply(BigInteger.valueOf(a.m));
  return l.compareTo(r);
}

// collects the boundary events of the line through poly[i] with
// direction d (d = poly[j] - poly[i]); returns the event count, writes
// collinear edge spans to spanLo/spanHi with their count in spansOut[0]
static int events(Pt[] poly, int i, Pt d, Ev[] ev, Fr[] spanLo, Fr[] spanHi,
    int[] spansOut) {
  int n = poly.length;
  int cnt = 0;
  spansOut[0] = 0;
  long dd = dot(d, d);
  for (int k = 0; k < n; k++) {
    long s = cross(d, sub(poly[k], poly[i]));
    if (s != 0)
      continue;
    Fr t = new Fr(dot(d, sub(poly[k], poly[i])), dd);
    long sp = cross(d, sub(poly[(k + n - 1) % n], poly[i]));
    long sn = cross(d, sub(poly[(k + 1) % n], poly[i]));
    int crossing = (sp != 0 && sn != 0 && ((sp > 0) != (sn > 0))) ? 1 : 0;
    if (sn == 0 && sp != 0) {
      // start of a boundary run along the line: the boundary crosses
      // here when the side it leaves on after the run is opposite to
      // the side it arrived from before the run
      int j = (k + 1) % n;
      while (cross(d, sub(poly[j], poly[i])) == 0)
        j = (j + 1) % n;
      long after = cross(d, sub(poly[j], poly[i]));
      crossing = ((sp > 0) != (after > 0)) ? 1 : 0;
    }
    ev[cnt].t = t;
    ev[cnt].crossing = crossing;
    cnt++;
    if (sp == 0) { // edge (k-1, k) lies on the line
```

listing 120: java: frCmp cross-multiplies in BigInteger where c wrote __int128, the vertex walk classifying touch, crossing and boundary-run entry (source: icpc/samples-java/src/Ch09/Geom.java, lines 47-83)

```lua
function geom.best_span(px, py, n, i, j)
  local ox, oy = px[i], py[i]
  local dx, dy = px[j] - ox, py[j] - oy
  local c, t = {}, {}
  for k = 1, n do
    local rx, ry = px[k] - ox, py[k] - oy
    c[k] = dx * ry - dy * rx
    t[k] = dx * rx + dy * ry
  end
  local bps, tog, pieces = {}, {}, {}
  local nb, nt = 0, 0
  for k = 1, n do
    local b = k % n + 1
    local ca, cb = c[k], c[b]
    if ca == 0 then nb = nb + 1; bps[nb] = t[k] end
    if cb == 0 then nb = nb + 1; bps[nb] = t[b] end
    if ca * cb < 0 then
      local tc = (ca * t[b] - cb * t[k]) / (ca - cb)
      nb = nb + 1; bps[nb] = tc
      nt = nt + 1; tog[nt] = tc
    elseif ca == 0 and cb == 0 then
      local a1, a2 = t[k], t[b]
      pieces[#pieces + 1] = { a1 < a2 and a1 or a2, a1 < a2 and a2 or a1 }
    end
  end
  -- transversal crossings at on-line vertices: the boundary can run
  -- along the line for a maximal chain of on-line vertices and still
  -- cross it, so a chain toggles once when its entry and exit sides
  -- differ. scan from an off-line vertex so wraparound chains stay
  -- whole.
  local k0 = 1
  while k0 ≤ n and c[k0] == 0 do k0 = k0 + 1 end
  if k0 ≤ n then
    local k = k0
    repeat
      if c[k] == 0 then
        local e = k
        while c[e % n + 1] == 0 and e % n + 1 ~= k0 do e = e % n + 1 end
        local cb, ca = c[(k - 2) % n + 1], c[e % n + 1]
        if cb ~= 0 and ca ~= 0 and (cb > 0) ~= (ca > 0) then
          nt = nt + 1
          tog[nt] = t[k]
        end
        k = e % n + 1
      else
        k = k % n + 1
      end
    until k == k0
  end
  table.sort(bps)
  table.sort(tog)
```

listing 121: lua: best_span collects breakpoints and toggles, a maximal chain of on-line vertices scanned from an off-line start (source: icpc/samples-lua/ch08_geom.lua, lines 12-62)

The match kit backs problem C with Kuhn's augmenting-path bipartite matching over at most 200 plus 200 nodes, all the per-height row and column graphs ever need. C passes a dense n by m byte matrix to `match_kuhn`, java's `Match.kuhn` takes the same dense matrix, go's `kuhnMatch` takes adjacency lists, lua's `match.kuhn` the same lists, and

all four twins fit whole because the contract is one recursive augment. JavaScript inlines the augment over typed arrays, C# augments directly on the column list, and Python recurses through a closure, each inside its problem C solver.

```c
// ch08_match.h: kuhn augmenting-path bipartite matching over a dense
// byte adjacency matrix, for 2017 C (rows and cols both ≤ 100)
#ifndef CH08_MATCH_H
#define CH08_MATCH_H

static int kh_try(int i, int m, const char *adj, int *matchR, int *seen) {
  for (int j = 0; j < m; j++) {
    if (!adj[i * m + j] || seen[j])
      continue;
    seen[j] = 1;
    if (matchR[j] < 0 || kh_try(matchR[j], m, adj, matchR, seen)) {
      matchR[j] = i;
      return 1;
    }
  }
  return 0;
}

// adj is n x m bytes (row-major); returns the matching size
static inline int match_kuhn(int n, int m, const char *adj, int *matchR,
                             int *seen) {
  for (int j = 0; j < m; j++)
    matchR[j] = -1;
  int size = 0;
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++)
      seen[j] = 0;
    size += kh_try(i, m, adj, matchR, seen);
  }
  return size;
}

#endif // CH08_MATCH_H
```

listing 122: c: kh_try augments over the dense byte matrix, match_kuhn resets and drives it per row (source: icpc/samples-c/src/Ch08/ch08_match.h, lines 1-33)

```go
package ch08

// ch08_match is the year-local bipartite matching helper used by
// problem C: Kuhn's augmenting-path algorithm over up to 200+200
// nodes, which is all the per-height row/column graphs ever need.
//
// matchR[r] is the left node matched to right node r, or -1.
func kuhnMatch(left, right int, adj [][]int) int {
  matchR := make([]int, right)
  for r := range matchR {
    matchR[r] = -1
  }
  seen := make([]bool, right)
  var augment func(u int) bool
  augment = func(u int) bool {
    for _, v := range adj[u] {
      if seen[v] {
        continue
      }
      seen[v] = true
      if matchR[v] < 0 || augment(matchR[v]) {
        matchR[v] = u
        return true
      }
    }
    return false
  }
  size := 0
  for u := 0; u < left; u++ {
    for i := range seen {
      seen[i] = false
    }
    if augment(u) {
      size++
    }
  }
  return size
}
```

listing 123: go: kuhnMatch over adjacency lists, the recursive augment closing over matchR and seen (source: icpc/samples-go/ch08/ch08_match.go, lines 1-38)

```java
class Match {
  static boolean tryRow(int i, int m, byte[][] adj, int[] matchR, boolean[] seen) {
    for (int j = 0; j < m; j++) {
      if (adj[i][j] == 0 || seen[j])
        continue;
      seen[j] = true;
      if (matchR[j] < 0 || tryRow(matchR[j], m, adj, matchR, seen)) {
        matchR[j] = i;
        return true;
      }
    }
    return false;
  }

  // adj is n x m bytes; returns the matching size
  static int kuhn(int n, int m, byte[][] adj, int[] matchR, boolean[] seen) {
    for (int j = 0; j < m; j++)
      matchR[j] = -1;
    int size = 0;
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++)
        seen[j] = false;
      size += tryRow(i, m, adj, matchR, seen) ? 1 : 0;
    }
    return size;
  }
}
```

listing 124: java: tryRow augments over the dense byte matrix, kuhn resets and drives it per row (source: icpc/samples-java/src/Ch09/Match.java, lines 3-29)

```lua
-- ch08 year-local bipartite matching helper for problem c: kuhn's
-- augmenting paths over up to 200+200 nodes per height, all the
-- per-height row/column graphs ever need
local match = {}

-- adj[u] lists right nodes; returns the matching size
function match.kuhn(left, right, adj)
  local match_r = {}
  for v = 1, right do match_r[v] = 0 end
  local seen = {}
  local function augment(u)
    for _, v in ipairs(adj[u]) do
      if not seen[v] then
        seen[v] = true
        if match_r[v] == 0 or augment(match_r[v]) then
          match_r[v] = u
          return true
        end
      end
    end
    return false
  end
  local size = 0
  for u = 1, left do
    for v = 1, right do seen[v] = false end
    if augment(u) then size = size + 1 end
  end
  return size
end

return match
```

listing 125: lua: match.kuhn over ipairs adjacency lists, 1-based match_r with 0 for unmatched (source: icpc/samples-lua/ch08_match.lua, lines 1-31)

The dinic kit backs problem J with Dinic's max flow over at most 201 nodes in double capacities and epsilon comparisons, the alpha mix of two integral flows being fractional, edge slots retained so per-pipe flows read off the residual, and a maxflow that accumulates so the section's four runs chain on one network. C carries the paired-arc edge list in `Ch08/ch08_dinic.h`, go, javascript and lua the same shape in `ch08/ch08_dinic.go`, `ch08-dinic.mjs` and `ch08_dinic.lua`, the three with a paired-capacity `addUndirected` for the pipes, java the same paired arcs in `Ch09/Dinic.java` with edge indices retained so per-pipe flows read off the residual, while C#'s dense matrix with conservation asserts and Python's inline class live inside their problem J solvers.

```c
static inline int dinic_bfs(Dinic *g, int s, int t) {
  for (int i = 0; i < g->n; i++)
    g->lvl[i] = -1;
  int qh = 0, qt = 0;
  g->q[qt++] = s;
  g->lvl[s] = 0;
  while (qh < qt) {
    int u = g->q[qh++];
    for (int e = g->head[u]; e >= 0; e = g->e[e].nxt)
      if (g->e[e].cap > DINIC_EPS && g->lvl[g->e[e].to] < 0) {
        g->lvl[g->e[e].to] = g->lvl[u] + 1;
        g->q[qt++] = g->e[e].to;
      }
  }
  return g->lvl[t] >= 0;
}

static inline double dinic_dfs(Dinic *g, int u, int t, double f) {
  if (u == t)
    return f;
  for (int *ie = &g->iter[u]; *ie >= 0; *ie = g->e[*ie].nxt) {
    int e = *ie;
    int v = g->e[e].to;
    if (g->e[e].cap <= DINIC_EPS || g->lvl[v] != g->lvl[u] + 1)
      continue;
    double d = dinic_dfs(g, v, t, f < g->e[e].cap ? f : g->e[e].cap);
    if (d > DINIC_EPS) {
      g->e[e].cap -= d;
      g->e[e ^ 1].cap += d;
      return d;
    }
  }
  return 0;
}

static inline double dinic_maxflow(Dinic *g, int s, int t) {
  double flow = 0;
  while (dinic_bfs(g, s, t)) {
    for (int i = 0; i < g->n; i++)
      g->iter[i] = g->head[i];
    double f;
    while ((f = dinic_dfs(g, s, t, 1e18)) > DINIC_EPS)
      flow += f;
  }
  return flow;
}
```

listing 126: c: dinic_bfs levels the graph, dinic_dfs walks current arcs, dinic_maxflow accumulates, doubles past DINIC_EPS (source: icpc/samples-c/src/Ch08/ch08_dinic.h, lines 64-109)

```go
func (d *dinic) dfs(u, t int, f float64) float64 {
  if u == t {
    return f
  }
  for ; d.iter[u] < len(d.nodes[u]); d.iter[u]++ {
    e := &d.nodes[u][d.iter[u]]
    if e.cap > dinicEps && d.level[e.to] == d.level[u]+1 {
      flow := d.dfs(e.to, t, math.Min(f, e.cap))
      if flow > dinicEps {
        e.cap -= flow
        d.nodes[e.to][e.rev].cap += flow
        return flow
      }
    }
  }
  return 0
}

// maxflow accumulates the flow from s to t on the current network,
// so multiple source nodes can be handled by sequential calls.
func (d *dinic) maxflow(s, t int) float64 {
  flow := 0.0
  for d.bfs(s, t) {
    d.iter = make([]int, len(d.nodes))
    for {
      f := d.dfs(s, t, math.Inf(1))
      if f <= dinicEps {
        break
      }
      flow += f
    }
  }
  return flow
}
```

listing 127: go: the level-graph dfs on current-arc iter, then maxflow accumulating across sequential source runs (source: icpc/samples-go/ch08/ch08_dinic.go, lines 64-97)

```
double dfs(int u, int t, double f) {
  if (u == t)
    return f;
  for (; iter[u] ≥ 0; iter[u] = e[iter[u]].nxt) {
    int ed = iter[u];
    int v = e[ed].to;
    if (e[ed].cap ≤ EPS || lvl[v] ≠ lvl[u] + 1)
      continue;
    double d = dfs(v, t, Math.min(f, e[ed].cap));
    if (d > EPS) {
      e[ed].cap -= d;
      e[ed ^ 1].cap += d;
      return d;
    }
  }
  return 0;
}

double maxflow(int s, int t) {
  double flow = 0;
  while (bfs(s, t)) {
    for (int i = 0; i < n; i++)
      iter[i] = head[i];
    double f;
    while ((f = dfs(s, t, 1e18)) > EPS)
      flow += f;
  }
  return flow;
}
```

listing 128: java: the current-arc dfs with eps comparisons, maxflow accumulating so the section's runs chain on one network (source: icpc/samples-java/src/Ch09/Dinic.java, lines 67-95)

```
  bfs(s, t) {
    const n = this.nodes.length;
    this.level = new Int32Array(n).fill(-1);
    this.level[s] = 0;
    const queue = [s];
    for (let head = 0; head < queue.length; head++) {
      const u = queue[head];
      for (const e of this.nodes[u]) {
        if (e.cap > dinicEps && this.level[e.to] < 0) {
          this.level[e.to] = this.level[u] + 1;
          queue.push(e.to);
        }
      }
    }
    return this.level[t] ≥ 0;
  }

  dfs(u, t, f) {
    if (u === t) return f;
    for (; this.iter[u] < this.nodes[u].length; this.iter[u]++) {
      const e = this.nodes[u][this.iter[u]];
      if (e.cap > dinicEps && this.level[e.to] === this.level[u] + 1) {
        const flow = this.dfs(e.to, t, Math.min(f, e.cap));
        if (flow > dinicEps) {
          e.cap -= flow;
          this.nodes[e.to][e.rev].cap += flow;
          return flow;
        }
      }
    }
    return 0;
  }

  // accumulates flow from s to t on the current network, so multiple
  // source nodes are handled by sequential calls
  maxflow(s, t) {
    let flow = 0;
    while (this.bfs(s, t)) {
      this.iter = new Int32Array(this.nodes.length);
      for (;;) {
        const f = this.dfs(s, t, Infinity);
        if (f ≤ dinicEps) break;
        flow += f;
      }
    }
    return flow;
  }
```

listing 129: javascript: the same bfs, dfs and accumulating maxflow over adjacency arrays of paired arcs (source: icpc/samples-js/src/ch08-dinic.mjs, lines 29-75)

```
function dinic.dfs(d, u, t, f)
  if u == t then return f end
  local adj = d.nodes[u]
  local i = d.it[u]
  while i ≤ #adj do
    local e = adj[i]
    if e.cap > EPS and d.level[e.to] == d.level[u] + 1 then
      local flow = dinic.dfs(d, e.to, t, f < e.cap and f or e.cap)
      if flow > EPS then
        e.cap = e.cap - flow
        d.nodes[e.to][e.rev].cap = d.nodes[e.to][e.rev].cap + flow
        d.it[u] = i
        return flow
      end
    end
    i = i + 1
    d.it[u] = i
  end
  return 0.0
end

-- accumulates flow from s to t, so several sources can run in turn
function dinic.maxflow(d, s, t)
  local flow = 0.0
  while dinic.bfs(d, s, t) do
    for i = 1, #d.nodes do d.it[i] = 1 end
    repeat
      local f = dinic.dfs(d, s, t, 1e18)
      if f > EPS then flow = flow + f end
    until f ≤ EPS
  end
  return flow
end
```

listing 130: lua: dinic.dfs on current-arc iterators, dinic.maxflow accumulating across the four runs (source: icpc/samples-lua/ch08_dinic.lua, lines 57-89)

The fenwick kit backs problem L with a fenwick of counts over compressed row ranks: select by binary descent for the k-th active rank, `pred` for the largest active row at most the queried corner, and `succ` for the smallest above it, the order statistics the column sweep matches corners with. C's `Ch08/ch08_fenwick.h` carries the whole contract in fifty lines, go and lua print the descent and the two queries, javascript the identical class, java's `Ch09/Fenwick.java` the descent and the predecessor, while Python inlines the tree in its solver and C# keeps a private nested Fenwick with Pred and Succ inside PL.

```c
// ch08_fenwick.h: fenwick tree of counts over compressed ranks with
// select (k-th active index) and predecessor (largest active ≤ x),
// for 2017 L
#ifndef CH08_FENWICK_H
#define CH08_FENWICK_H

#include <stdlib.h>

typedef struct {
  int n;
  int *t; // t[1..n]
} Fenwick;

static inline void fw_init(Fenwick *f, int n) {
  f->n = n;
  f->t = calloc((size_t)n + 1, sizeof(int));
}

static inline void fw_free(Fenwick *f) { free(f->t); }

static inline void fw_add(Fenwick *f, int i, int d) {
  for (; i ≤ f->n; i += i & (-i))
    f->t[i] += d;
}

static inline int fw_prefix(Fenwick *f, int i) {
  int s = 0;
  for (; i > 0; i -= i & (-i))
    s += f->t[i];
  return s;
}

// smallest index with prefix count ≥ k, or -1 when none (k ≥ 1)
static inline int fw_select(Fenwick *f, int k) {
  int idx = 0;
  for (int step = 1 << 20; step > 0; step >>= 1)
    if (idx + step ≤ f->n && f->t[idx + step] < k) {
      idx += step;
      k -= f->t[idx];
    }
  return idx + 1 ≤ f->n ? idx + 1 : -1;
}

// largest active index ≤ x, or -1
static inline int fw_pred(Fenwick *f, int x) {
  int c = fw_prefix(f, x);
  return c > 0 ? fw_select(f, c) : -1;
}

#endif // CH08_FENWICK_H
```

listing 131: c: fw_select by binary descent down the counting tree, fw_pred the largest active index at most x (source: icpc/samples-c/src/Ch08/ch08_fenwick.h, lines 1-50)

```
// selectK returns the rank of the k-th active element (k ≥ 1).
func (f *fenwick) selectK(k int) int {
  pos := 0
  bit := 1
  for bit<<1 < len(f.tree) {
    bit <<= 1
  }
  for ; bit > 0; bit >>= 1 {
    next := pos + bit
    if next < len(f.tree) && k > f.tree[next] {
      pos = next
      k -= f.tree[next]
    }
  }
  return pos
}

func (f *fenwick) pred(x int) int {
  k := f.prefix(x)
  if k == 0 {
    return -1
  }
  return f.selectK(k)
}

func (f *fenwick) succ(x int) int {
  k := f.prefix(x)
  if f.prefix(len(f.tree)-2) == k {
    return -1
  }
  return f.selectK(k + 1)
}
```

listing 132: go: selectK descends the counting tree, pred and succ answer the sweep's corner queries (source: icpc/samples-go/ch08/ch08_fenwick.go, lines 29-60)

```
void add(int i, int d) {
  for (; i ≤ n; i += i & -i)
    t[i] += d;
}

int prefix(int i) {
  int s = 0;
  for (; i > 0; i -= i & -i)
    s += t[i];
  return s;
}

// smallest index with prefix count ≥ k, or -1 when none (k ≥ 1)
int select(int k) {
  int idx = 0;
  for (int step = 1 << 20; step > 0; step >>= 1)
    if (idx + step ≤ n && t[idx + step] < k) {
      idx += step;
      k -= t[idx];
    }
  return idx + 1 ≤ n ? idx + 1 : -1;
}

// largest active index ≤ x, or -1
int pred(int x) {
  int c = prefix(x);
  return c > 0 ? select(c) : -1;
}
```

listing 133: java: select descends the counting tree by binary steps, pred answers the sweep's corner query (source: icpc/samples-java/src/Ch09/Fenwick.java, lines 13-40)

```javascript
// ch08-fenwick is the year-local order-statistics helper used by
// problem l: a fenwick tree over compressed row ranks supporting
// insert, erase, pred (largest active rank ≤ x), and succ (smallest
// active rank > x) via rank selection.
export class Fenwick {
  constructor(n) {
    this.tree = new Int32Array(n + 1);
  }

  add(i, delta) {
    for (i++; i < this.tree.length; i += i & -i) this.tree[i] += delta;
  }

  prefix(i) {
    let sum = 0;
    for (i++; i > 0; i -= i & -i) sum += this.tree[i];
    return sum;
  }

  // the rank of the k-th active element (k ≥ 1)
  selectK(k) {
    let pos = 0;
    let bit = 1;
    while ((bit << 1) < this.tree.length) bit <<= 1;
    for (; bit > 0; bit >>= 1) {
      const next = pos + bit;
      if (next < this.tree.length && k > this.tree[next]) {
        pos = next;
        k -= this.tree[next];
      }
    }
    return pos;
  }

  pred(x) {
    const k = this.prefix(x);
    return k === 0 ? -1 : this.selectK(k);
  }

  succ(x) {
    const k = this.prefix(x);
    if (this.prefix(this.tree.length - 2) === k) return -1;
    return this.selectK(k + 1);
  }
}
```

listing 134: javascript: the same class, selectK by descent, pred and succ over prefix counts (source: icpc/samples-js/src/ch08-fenwick.mjs, lines 1-45)

```
-- rank (0-based) of the k-th active element, k ≥ 1
function fenwick.select_k(f, k)
  local pos, bit = 0, 1
  while (bit << 1) < f.n + 1 do bit = bit << 1 end
  while bit > 0 do
    local nxt = pos + bit
    if nxt ≤ f.n + 1 and k > (f.tree[nxt] or 0) then
      pos = nxt
      k = k - (f.tree[nxt] or 0)
    end
    bit = bit >> 1
  end
  return pos
end

function fenwick.pred(f, x)
  local k = fenwick.prefix(f, x)
  if k == 0 then return -1 end
  return fenwick.select_k(f, k)
end

function fenwick.succ(f, x)
  local k = fenwick.prefix(f, x)
  if fenwick.prefix(f, f.n - 1) == k then return -1 end
  return fenwick.select_k(f, k + 1)
end
```

listing 135: lua: select_k descends the 1-based counting tree, pred and succ wrapping it (source: icpc/samples-lua/ch08_fenwick.lua, lines 29-54)

C# and Python carry no twin file in any of the four kits: their solvers inline the predicates, classes and closures in the listings of sections A, C, J and L.

10.2. problem A, airport construction

Piconesia is a tropical island nation that wants its first airport, and since a longer landing strip accommodates larger airplanes it wants the longest strip it can build. The island's boundary is modeled as a polygon, the strip is a straight line segment that must lie on the island, it may touch or run along the boundary but must never intersect the sea, and the task is to print the length of the longest such segment (statement).

The input is one integer n, 3 <= n <= 200 by the problemset pdf, then n lines of two integers x and y with |x| and |y| at most 10^6, the polygon's vertices in counterclockwise order. The pdf promises the polygon is simple, vertices distinct, edges touching only where consecutive edges share their common vertex, and no two consecutive edges collinear, a promise that tames every degenerate touching case. The output is the single length within an absolute or relative error of 10^-6, the judge answers printed to nine decimals, under the pdf's 2 second time limit.

Official sample 1, reprinted byte for byte from the judge data pair `A-airport/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: the seven-vertex island's winning strip runs vertex to vertex for 76.157731059.

input:

```
7
0 20
40 0
40 20
70 50
50 70
```

```
30 50
0 50
```

expected output:

```
76.157731059
```

Recognition: the bounds choose the search shape before any geometry does. With n <= 200 under the 2 second limit, the 200 choose 2 = 19900 vertex-pair lines are enumerable outright, and classifying one line against the boundary walks the 200 edges once, about 4 million exact integer crosses for the whole search, an O(n^3) shape with two orders of magnitude to spare.

The statement's own wording, the longest segment lying on the island, hands over the candidate set: an optimal strip can always be slid and stretched until its supporting line pins against two polygon vertices, the cue that turns a continuous placement search into 19900 discrete lines. Problem A is the 2017 face of the geometric critical-point enumeration family. Geometric critical-point enumeration is the shared shape of chapter 10 problem D, chapter 11 problem E, chapter 11 problem G, chapter 13 problem T, and chapter 14 problem D.

The tempting shortcut, rotating calipers over the convex hull, is priced out by the shape itself: the hull pads every bay with sea, sample 1′s own hull already fills the reflex notch at (40,20), so a caliper chord can cross water and certifying any hull pair still needs the O(n) walk against the original boundary that the full enumeration already pays 19900 times inside the budget, while a continuous ternary search over strip placement has no unimodality on a non-convex boundary to exploit.

The solutions pdf, page 2, settles the shape of an optimum: the strip's line passes through two polygon vertices. So the search tries all O(n^2) vertex pairs, tests that the chord between a pair lies on the island, and extends that chord along its line out to the polygon boundary in both directions, O(n^3) total for n <= 200 and comfortably inside the limit. The whole difficulty sits in the extension: the chord's line can touch an edge at a point, run along it, and veer back to its own side, or cross over to the far side, and only the crossing toggles the inside parity, so the classifier has to tell a touch-and-return from a touch-and-cross using exact integer cross products on the given coordinates. Book 10′s chapter 24, lattice geometry and grid algorithms, develops that lattice polygon arithmetic from scratch. Seven solvers, seven exactness strategies on that classification.

The worked run: trace the model on sample 1. The island is the seven printed vertices, so the enumeration visits 7 choose 2 = 21 lines, and three of them carry the story. The pair (0,50), (70,50) is the horizontal y = 50: the boundary lies along it over the edge from (0,50) to (30,50), that on-line chain toggles the crossing parity once because its entry and exit sides differ, and the boundary stretch merges with the interior stretch out to (70,50) for a run of exactly 70 - 0 = 70. The pair (40,20), (50,70) exercises the extension: at (40,20) both neighboring edges sit on the same side of the line, a touch and not a crossing, so the run does not stop at the vertex but continues to where the line leaves the island through the edge (0,20)-(40,0), y = 20 - x/2 meeting y = 5x - 180 at (400/11, 20/11), and from there to (50,70) the horizontal reach is 50 - 400/11 = 150/11 at slope 5, a run of (150/11) sqrt(26) = 69.532. The runner-up is the pair (0,20), (50,70), a plain chord of legs 50 - 0 = 50 and 70 - 20 = 50 worth 50 sqrt(2) = 70.711. The winner is the pair (0,20), (70,50): legs 70 - 0 = 70 and 50 - 20 = 30, and at both endpoints the neighboring edges sit on opposite sides of the line, crosses +210 against −260 at (0,20) and −120 against +200 at (70,50), so both ends are transversal crossings, the extension adds nothing, and the run is exactly the chord, sqrt(70^2 + 30^2) = sqrt(5800) = 76.1577310586, and the trace ends at the printed answer `76.157731059`.

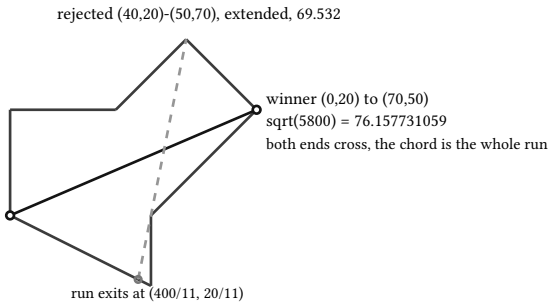


figure 91: the sample-1 island, the winning chord from (0,20) to (70,50), the rejected (40,20)-(50,70) line extended to its boundary exit

```c
qsort(ev, (size_t)cnt, sizeof(GEvent), geo_ev_cmp);
// dedup: a shared parameter keeps the crossing flag
int m = 0;
for (int k = 0; k < cnt; k++) {
  if (m > 0 && geo_fr_cmp(ev[k].t, ev[m - 1].t) == 0)
    ev[m - 1].crossing |= ev[k].crossing;
  else
    ev[m++] = ev[k];
}
// gap k (between events k and k+1) covered: odd crossing prefix, or
// the gap lies inside a boundary run along the line
char covered[512];
int cross = 0;
for (int k = 0; k + 1 < m; k++) {
  cross += ev[k].crossing;
  covered[k] = cross & 1;
}
for (int s = 0; s < spans; s++)
  for (int k = 0; k + 1 < m; k++)
    if (!covered[k] && geo_fr_cmp(ev[k].t, slo[s]) ≥ 0 &&
        geo_fr_cmp(ev[k + 1].t, shi[s]) ≤ 0)
      covered[k] = 1;
// locate the defining vertices (params 0 and 1 on this line)
GFr t0 = {0, 1}, t1 = {1, 1};
int ia = -1, ib = -1;
for (int k = 0; k < m; k++) {
  if (geo_fr_cmp(ev[k].t, t0) == 0)
    ia = k;
  if (geo_fr_cmp(ev[k].t, t1) == 0)
    ib = k;
}
if (ia < 0 || ib < 0 || ia > ib)
  return;
for (int k = ia; k < ib; k++)
  if (!covered[k])
    return;
int a = ia, b = ib;
while (a > 0 && covered[a - 1])
  a--;
while (b + 1 < m && covered[b])
  b++;
```

listing 136: c: events as exact fractions sorted in `__int128`, parity plus boundary runs decide the covered gaps (source: icpc/samples-c/src/Ch08/pA.c, lines 24-64)

```
      for k := 0; k < n; k++ {
        a, b := poly[k], poly[(k+1)%n]
        ca := d.x*(a.y-o.y) - d.y*(a.x-o.x)
        cb := d.x*(b.y-o.y) - d.y*(b.x-o.x)
        ta := a.sub(o).dot(d)
        tbk := b.sub(o).dot(d)
        if ca == 0 {
          ts = append(ts, ta/dd)
        }
        if cb == 0 {
          ts = append(ts, tbk/dd)
        }
        if ca != 0 && cb != 0 && (ca > 0) != (cb > 0) {
          u := ca / (ca - cb)
          ts = append(ts, (ta+u*(tbk-ta))/dd)
        }
      }
      sortFloats(ts)
      root := math.Sqrt(dd)
      for k := 0; k+1 < len(ts); {
        if !intervalInside(poly, o, d, ts[k], ts[k+1]) {
          k++
          continue
        }
        start := k
        for k+1 < len(ts) && intervalInside(poly, o, d, ts[k], ts[k+1]) {
          k++
        }
        if span := (ts[k] - ts[start]) * root; span > best-1e-5 {
          if span > best {
            best = span
          }
          cands = append(cands, cand{i, j, start, k})
        }
      }
```

listing 137: go: float breakpoints and midpoint containment tests collect candidate runs (source: icpc/samples-go/ch08/pa.go, lines 41-75)

```
    Arrays.sort(ev, 0, cnt, (a, b) -> Geom.frCmp(a.t, b.t));
    // dedup: a shared parameter keeps the crossing flag
    int m = 0;
    for (int k = 0; k < cnt; k++) {
      if (m > 0 && Geom.frCmp(ev[k].t, ev[m - 1].t) == 0)
        ev[m - 1].crossing |= ev[k].crossing;
      else
        ev[m++] = ev[k];
    }
    // gap k (between events k and k+1) covered: odd crossing prefix, or
    // the gap lies inside a boundary run along the line
    boolean[] covered = new boolean[512];
    int cross = 0;
    for (int k = 0; k + 1 < m; k++) {
      cross += ev[k].crossing;
      covered[k] = (cross & 1) == 1;
    }
    for (int s = 0; s < spans[0]; s++)
      for (int k = 0; k + 1 < m; k++)
        if (!covered[k] && Geom.frCmp(ev[k].t, slo[s]) ≥ 0 &&
            Geom.frCmp(ev[k + 1].t, shi[s]) ≤ 0)
          covered[k] = true;
    // locate the defining vertices (params 0 and 1 on this line)
    Geom.Fr t0 = new Geom.Fr(0, 1), t1 = new Geom.Fr(1, 1);
    int ia = -1, ib = -1;
    for (int k = 0; k < m; k++) {
      if (Geom.frCmp(ev[k].t, t0) == 0)
        ia = k;
      if (Geom.frCmp(ev[k].t, t1) == 0)
        ib = k;
    }
    if (ia < 0 || ib < 0 || ia > ib)
      return;
    for (int k = ia; k < ib; k++)
      if (!covered[k])
        return;
    int a = ia, b = ib;
    while (a > 0 && covered[a - 1])
      a--;
    while (b + 1 < m && covered[b])
      b++;
    BigInteger dn = BigInteger.valueOf(ev[b].t.n).multiply(BigInteger.valueOf(ev[a].t.m))
        .subtract(BigInteger.valueOf(ev[a].t.n).multiply(BigInteger.valueOf(ev[b].t.m)));
    if (dn.signum() ≤ 0)
      return;
```

listing 138: java: events sorted and deduped through the helper's frCmp, gaps covered by crossing parity or boundary runs, the chord-length difference cross-multiplied in BigInteger (source: icpc/samples-java/src/Ch09/PA.java, lines 44-88)

```
                long s0 = Cross(dx, dy, px[k] - px[i], py[k] - py[i]);
                long s1 = Cross(dx, dy, px[k2] - px[i], py[k2] - py[i]);
                double t0 =
                    ((px[k] - px[i]) * dx + (py[k] - py[i]) * dy) / len2;
                if (s0 == 0)
                {
                    if (runLen == 0)
                        runLo = runHi = t0;
                    else
                    {
                        runLo = Math.Min(runLo, t0);
                        runHi = Math.Max(runHi, t0);
                    }
                    runLen++;
                }
                else if (runLen > 0)
                {
                    runs.Add((runLo, runHi));
                    runLen = 0;
                }
                bool crossing = (s0 < 0 && s1 ≥ 0) || (s0 ≥ 0 && s1 < 0);
                if (!crossing)
                    continue;
                long ex = px[k2] - px[k], ey = py[k2] - py[k];
                events[evCnt++] = s0 == 0
                    ? t0
                    : s1 == 0
                        ? ((px[k2] - px[i]) * dx + (py[k2] - py[i]) * dy) / len2
                        : CrossV(px[k] - px[i], py[k] - py[i], ex, ey)
                            / CrossV(dx, dy, ex, ey);
            }
            if (runLen > 0)
                runs.Add((runLo, runHi));
            Array.Sort(events, 0, evCnt);

            var spans = new List<(double Lo, double Hi)>(runs);
            int q = 0;
            int parity = 0;
            double prevOdd = double.NegativeInfinity;
            while (q < evCnt)
            {
                int r2 = q;
                while (r2 < evCnt && events[r2] == events[q])
                    r2++;
                if (((r2 - q) & 1) == 1)
                {
                    if ((parity & 1) == 1)
                        spans.Add((prevOdd, events[q]));
                    parity++;
                    prevOdd = events[q];
                }
                q = r2;
```

listing 139: c#: integer cross products classify edges, double t sorts events, parity walks build the inside spans (source: icpc/samples/src/Ch08/PA.cs, lines 42-93)

```
const ts = new Float64Array(2 * n + 2), tags = new Int32Array(2 * n + 2);
const cands = [];
let best = 0;
for (let i = 0; i < n; i++) {
  for (let j = i + 1; j < n; j++) {
    const ox = px[i], oy = py[i], dx = px[j] - ox, dy = py[j] - oy;
    const dd = dx * dx + dy * dy;
    const m = lineBreaks(px, py, n, i, j, ts, tags);
    sortTs(ts, tags, m);
    const root = Math.sqrt(dd);
    for (let k = 0; k + 1 < m; ) {
      const mid = (ts[k] + ts[k + 1]) / 2;
      if (!insidePoly(px, py, n, ox + mid * dx, oy + mid * dy)) {
        k++;
        continue;
      }
      const start = k;
      while (k + 1 < m) {
        const mi = (ts[k] + ts[k + 1]) / 2;
        if (!insidePoly(px, py, n, ox + mi * dx, oy + mi * dy)) break;
        k++;
      }
      const span = (ts[k] - ts[start]) * root;
      if (span > best) best = span;
      if (span > best - 1e-5) cands.push(i, j, start, k);
    }
  }
}
```

listing 140: javascript: the same float walk, survivors re-evaluated later in bigint rationals (source: icpc/samples-js/src/ch08-pa-airport.mjs, lines 128-155)

```
    for k in range(n):
        ax, ay = poly[k]
        bx, by = poly[(k + 1) % n]
        ca = dx * (ay - oy) - dy * (ax - ox)
        cb = dx * (by - oy) - dy * (bx - ox)
        if ca == 0 and cb == 0:
            ta = (ax - ox) * dx + (ay - oy) * dy
            tb = (bx - ox) * dx + (by - oy) * dy
            if ta > tb:
                ta, tb = tb, ta
            collinear.append((ta, tb))
        elif ca and cb and (ca > 0) != (cb > 0):
            ta = (ax - ox) * dx + (ay - oy) * dy
            tb = (bx - ox) * dx + (by - oy) * dy
            toggles.append((ta * (ca - cb) + ca * (tb - ta)) / (ca - cb))
    for v in range(n):
        vx, vy = poly[v]
        if dx * (vy - oy) - dy * (vx - ox):
            continue
        px, py = poly[v - 1]
        nx, ny = poly[(v + 1) % n]
        sp = dx * (py - oy) - dy * (px - ox)
        sn = dx * (ny - oy) - dy * (nx - ox)
        t = (vx - ox) * dx + (vy - oy) * dy
        if sp == 0:
            # the previous edge runs along the line; it toggles when
            # the boundary veers to the interior (left) side
            if (vx - px) * (ny - vy) - (vy - py) * (nx - vx) > 0:
                toggles.append(t)
        elif sn == 0:
            if (nx - vx) * (py - vy) - (ny - vy) * (px - vx) > 0:
                toggles.append(t)
        elif sp * sn < 0:
            toggles.append(t)
```

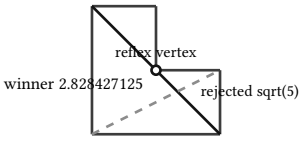
listing 141: python: exact integer crosses drive toggles, collinear edges add boundary pieces (source: icpc/samples-py/src/Ch08/pa.py, lines 20-53)

```lua
local geom = require("ch08_geom")

local function solve(input)
  local nums = {}
  for tok in input:gmatch("%S+") do
    nums[#nums + 1] = tonumber(tok) + 0.0
  end
  local n = nums[1]
  local px, py = {}, {}
  for i = 1, n do
    px[i] = nums[2 * i]
    py[i] = nums[2 * i + 1]
  end
  local best = 0.0
  for i = 1, n do
    for j = i + 1, n do
      local s = geom.best_span(px, py, n, i, j)
      if s > best then best = s end
    end
  end
  return string.format("%.9f\n", best)
end
```

listing 142: lua: the solver as a thin driver, every vertex pair handed to the year-local geom helper's best_span (source: icpc/samples-lua/ch08_pa.lua, lines 8-29)

The C solver carries its classification in `ch08_geom.h` as fractions n over m compared in `__int128`, then deliberately drops to doubles for the final metric, and the comparison cannot ask for more: the official answer values sit beyond double precision in both directions, so the harness verifies problem A in its float mode, `2017/A` pinned to `float` in compare.json, the last numeric token checked within the statement's 1e-6 rather than a nine-decimal byte match. Go and JavaScript walk floats, collect every run within an epsilon of the best, and re-evaluate the survivors exactly, Go in `big.Rat` with a 200-bit square root and JavaScript in BigInt rationals with an integer square root and half-up rounding. C# keeps the per-edge classification in exact integer cross products while sorting double parameters, Java ports the c line whole, the same fractions with `frCmp` and the chord-length difference in `BigInteger` at exactly the two places c wrote `__int128`, then the same drop to doubles for the printed metric, and Python and Lua run the parity argument on exact integer crosses end to end. The measured fixture is the six-vertex L island `0 0 / 2 0 / 2 1 / 1 1 / 1 2 / 0 2`, where the diagonal through the reflex vertex has length 2*sqrt(2), and every suite asserts the exact output `2.828427125`: the C `CHECK(strcmp(...))`, the C# `Assert.Equal`, Go's table row, the JavaScript `assert.equal`, the Python `check`, the Lua `T.eq`, and java's `ok` lines. All seven suites also pin the 3 by 4 rectangle answering `5.000000000`, and the C and java suites add a skinny triangle answering `1000.000000000`.



optimal strips pass through two vertices, then extend to the boundary

figure 92: the l island, the winning diagonal through the reflex vertex, one rejected shorter chord

10.3. problem B, get a clue!

The game is Cluedo: twenty-one cards, six persons A to F, six weapons G to L, nine rooms M to U. One person, one weapon and one room card are drawn and hidden as the answer, the remaining eighteen are dealt round-robin to the four players starting with player 1, us, and moving to the right, hands of 5, 5, 4 and 4. Play rotates the same way from player 1, and a turn is a suggestion naming a person, a weapon and a room. The suggester first asks the player to their right for evidence, then onward around the table: a responder holding exactly one suggested

card must show it, a responder holding several may show any one, and the questioning stops at the first evidence or when everyone has passed. The record marks a pass as a dash, an evidence card we ourselves see, because we made the suggestion or provided the card, by its letter, and evidence nobody shows us as a star. Given our five cards and the full history, the task is to print the three-character answer string, the forced murderer, weapon and room letters or a question mark per category that is still open (statement).

The input is one integer n, 1 <= n <= 50 suggestions by the problemset pdf, then a line with our five dealt cards as uppercase letters A through U, then n suggestion lines: three characters, person, weapon and room in that order, followed by up to three responses in questioning order starting from the suggester's right neighbor, each a dash or an evidence character, and only the last response can be evidence. Only valid histories appear. The output is the three character string under the pdf's 4 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `B-clue/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: we hold none of A, G or M and all three opponents pass, so the hidden answer is exactly that triple.

input:

```
1
B I P C F
A G M - - -
```

expected output:

```
AGM
```

Official sample 2, same source: this is the input that pins the conventions. The first suggestion is ours and player 2 shows us M, so player 2 holds M. If the second suggestion were also ours, player 2 would have to show M again rather than pass, which contradicts the record, so suggestions must rotate and the second is player 2′s own, asked of players 3 then 4: player 3 passes on F H M, player 4′s unseen star can only be F, since we hold H and player 2 holds M. Every person but E is then accounted for, weapon and room are not, and the pdf prints E??.

input:

```
2
A B C D H
F G M M
F H M - *
```

expected output:

```
E??
```

Recognition: the bounds sit on the enumeration, not on any search through it. The answer triple is one of 6 x 6 x 9 = 324 candidates, and once a triple is fixed the thirteen cards we never see split into opponent hands of 5, 4 and 4, C(13,5) x C(8,4) = 1287 x 70 = 90090 deals to test against the history, about 2.9 x 10^7 raw worlds across all triples, and clause pruning cuts that far below the 4 second limit.

The statement's own shape, print the forced letters or a question mark per category that is still open, is a consistency census: every question mark must survive because at least one consistent deal exists, and the rule that a responder holding several suggested cards may show any one means no local deduction closes a category on its own. Problem B is the 2017 face of the constraint repair with backtracking family. Chapter 11 problem C is the other constraint repair with backtracking member.

The tempting shortcut, a forward rule engine that propagates forced cards and answers from what it can see, is priced out by that same freedom: whether a category stays open can hinge on a deal that exists but no recorded suggestion ever exposes, so the census over deals is the only decider, and the 90090 deal test per triple is what fits the budget.

Page 3 of the solutions pdf settles the deduction: enumerate the consistent worlds. The judges' fast shape computes the candidate hand sets per opponent that are compatible with the history, guesses the answer triple, then joins, for a player 2 set Si and a player 3 set Tj it looks up X xor Si xor Tj among the player 4 sets through a bitmask dictionary over 21-bit integers, worst case bounded on the C(13,5) scale and fast enough in plain CPython, while a directly pruned backtracking over the two unknown hands also fits the 4 second limit. The seven solvers split three ways.

The worked run: trace the model on sample 1. Our hand is the printed B I P C F, the single suggestion A G M is ours because play starts at player 1, and the three printed dashes are the answers of players 2, 3 and 4 in questioning order. A dash means the responder holds none of the three suggested cards, so no opponent holds A, G or M, and checking them against the printed hand, B, I, P, C and F leave all three untouched there too. Every card is either dealt into one of the four hands or hidden as the answer, so all three must be the hidden answer, person A, weapon G and room M, no category stays open, and the trace ends at the printed answer `AGM`.

card	category	in our hand	responder	therefore
A	person	no	player 2 passes, holds none	hidden
G	weapon	no	player 3 passes, holds none	hidden
M	room	no	player 4 passes, holds none	hidden

```c
static int dfs(Inst *s, int idx) {
  if (idx == s->ncards) {
    for (int c = 0; c < s->clcnt; c++)
      if (!(s->cl[c].mask & s->hold[s->cl[c].player]))
        return 0;
    return 1;
  }
  int card = s->cards[idx];
  for (int p = 0; p < 3; p++) {
    if (s->cap[p] <= 0 || (s->neg[p] >> card) & 1)
      continue;
    s->hold[p] |= 1 << card;
    s->cap[p]--;
    int ok = 1;
    for (int c = 0; c < s->clcnt && ok; c++) {
      int q = s->cl[c].player;
      if (s->cap[q] == 0 && !(s->cl[c].mask & s->hold[q]))
        ok = 0;
    }
    if (ok && dfs(s, idx + 1))
      return 1;
    s->cap[p]++;
    s->hold[p] &= ~(1 << card);
  }
  return 0;
}
```

listing 143: c: pruned backtracking over the dealt cards, clause cards ordered first (source: icpc/samples-c/src/Ch08/pB.c, lines 23-48)

```
    allowed := remaining &^ nothold[player]
    if popcount32(allowed) < sizes[player] {
      return false
    }
    free := allowed &^ need[player]
    k := sizes[player] - popcount32(need[player])
    if k < 0 {
      return false
    }
    var bits []uint32
    m := free
    for m ≠ 0 {
      b := m & -m
      bits = append(bits, b)
      m &= m - 1
    }
    // suffix[i] = bits still pickable from position i on
    suffix := make([]uint32, len(bits)+1)
    for i := len(bits) - 1; i ≥ 0; i-- {
      suffix[i] = suffix[i+1] | bits[i]
    }
    base := need[player]
    var comb func(start, chosen int, acc uint32) bool
    comb = func(start, chosen int, acc uint32) bool {
      if chosen == k {
        hand := acc | base
        for _, o := range ors[player] {
          if hand&o == 0 {
            return false
          }
        }
        return rec(player+1, remaining&^hand)
      }
      reach := acc | base | suffix[start]
      for _, o := range ors[player] {
        if o&reach == 0 {
          return false
        }
      }
      if k-chosen > len(bits)-start {
        return false
      }
      for i := start; i < len(bits); i++ {
        if comb(i+1, chosen+1, acc|bits[i]) {
          return true
        }
      }
      return false
    }
    return comb(0, 0, 0)
```

listing 144: go: per-player hand enumeration with coverage and suffix-reach pruning (source: icpc/samples-go/ch08/pb.go, lines 119-168)

```java
static boolean dfs(Inst s, int idx) {
  if (idx == s.ncards) {
    for (int c = 0; c < s.clcnt; c++)
      if ((s.cl[c].mask & s.hold[s.cl[c].player]) == 0)
        return false;
    return true;
  }
  int card = s.cards[idx];
  for (int p = 0; p < 3; p++) {
    if (s.cap[p] ≤ 0 || (s.neg[p] >> card & 1) ≠ 0)
      continue;
    s.hold[p] |= 1 << card;
    s.cap[p]--;
    boolean ok = true;
    for (int c = 0; c < s.clcnt && ok; c++) {
      int q = s.cl[c].player;
      if (s.cap[q] == 0 && (s.cl[c].mask & s.hold[q]) == 0)
        ok = false;
    }
    if (ok && dfs(s, idx + 1))
      return true;
    s.cap[p]++;
    s.hold[p] &= ~(1 << card);
  }
  return false;
}
```

listing 145: java: pruned backtracking over the dealt cards, clause cards ordered first, capacity, negative-mask and clause-reach pruning at every placement (source: icpc/samples-java/src/Ch09/PB.java, lines 30-55)

```
        int[] cap, int[] owner, bool[,] mustNot)
    {
        if (idx == stars.Count)
            return Hall(cap, owner, mustNot);
        var (p, set) = stars[idx];
        foreach (int c in set)
        {
            if (mustNot[p, c])
                continue;
            if (owner[c] == 0)
            {
                if (cap[p] == 0)
                    continue;
                owner[c] = p;
                cap[p]--;
                if (PlaceStars(stars, idx + 1, cap, owner, mustNot))
                    return true;
                owner[c] = 0;
                cap[p]++;
            }
            else if (owner[c] == p)
            {
                if (PlaceStars(stars, idx + 1, cap, owner, mustNot))
                    return true;
            }
        }
        return false;
    }
```

listing 146: c#: star evidence placed by backtracking, then Hall's condition over the three hands (source: icpc/samples/src/Ch08/PB.cs, lines 141-168)

```
  const rec = (player, remaining) => {
    if (player === 5) return remaining === 0;
    let cover = 0;
    for (let j = player; j <= 4; j++) cover |= remaining & ~nothold[j];
    if (cover !== remaining) return false;
    const allowed = remaining & ~nothold[player];
    if (popcount(allowed) < SIZES[player]) return false;
    const free = allowed & ~need[player];
    const k = SIZES[player] - popcount(need[player]);
    if (k < 0) return false;
    const bits = [];
    for (let m = free; m !== 0; m &= m - 1) bits.push(m & -m);
    const suffix = new Array(bits.length + 1).fill(0);
    for (let i = bits.length - 1; i >= 0; i--) suffix[i] = suffix[i + 1] | bits[i];
    const base = need[player];
    const comb = (start, chosen, acc) => {
      if (chosen === k) {
        const hand = acc | base;
        for (const o of ors[player]) if ((hand & o) === 0) return false;
        return rec(player + 1, remaining & ~hand);
      }
      const reach = acc | base | suffix[start];
      for (const o of ors[player]) if ((o & reach) === 0) return false;
      if (k - chosen > bits.length - start) return false;
      for (let i = start; i < bits.length; i++) {
        if (comb(i + 1, chosen + 1, acc | bits[i])) return true;
      }
      return false;
    };
    return comb(0, 0, 0);
  };
  return rec(2, pool);
}
```

listing 147: javascript: the same pruned deal search, 21-bit masks as plain numbers (source: icpc/samples-js/src/ch08-pb-clue.mjs, lines 41-73)

```
    full = (1 << 21) - 1 & ~ours
    two = _hands(full, 2, need, nothold, ors)
    three = _hands(full, 3, need, nothold, ors)
    four = set(_hands(full, 4, need, nothold, ors))
    person_ok = [False] * 6
    weapon_ok = [False] * 6
    room_ok = [False] * 9
    for s in two:
        for t in three:
            if s & t:
                continue
            rest = full ^ s ^ t
            # the rest splits into the answer triple plus player 4's
            # four cards; try every 3-subset as the answer
            bits = []
            m = rest
            while m:
                b = m & -m
                bits.append(b)
                m ^= b
            for pick in combinations(bits, 3):
                answer = 0
                for b in pick:
                    answer |= b
                if answer ^ rest not in four:
                    continue
                pfield = answer & 0x3F
                wfield = answer >> 6 & 0x3F
                if pfield.bit_count() != 1 or wfield.bit_count() != 1:
                    continue
                if answer >> 12 & -(1 << 9):
                    continue
                person_ok[pfield.bit_length() - 1] = True
                weapon_ok[wfield.bit_length() - 1] = True
                room_ok[(answer >> 12).bit_length() - 1] = True
    return _pick(person_ok, 65) + _pick(weapon_ok, 71) + _pick(room_ok, 77) + "\n"
```

listing 148: python: the judges' join, player 2 and 3 hands enumerated, the rest looked up in a set (source: icpc/samples-py/src/Ch08/pb.py, lines 62-97)

```
  local function rec(player, remaining)
    if player == 5 then return remaining == 0 end
    local cover = 0
    for j = player, 4 do cover = cover | (remaining & ~nothold[j]) end
    if cover ~= remaining then return false end
    local allowed = remaining & ~nothold[player]
    if popcount(allowed) < sizes[player] then return false end
    local free = allowed & ~need[player]
    local k = sizes[player] - popcount(need[player])
    if k < 0 then return false end
    local bits = {}
    local m = free
    while m ~= 0 do
      local b = m & (-m)
      bits[#bits + 1] = b
      m = m & (m - 1)
    end
    local suffix = {}
    suffix[#bits + 1] = 0
    for i = #bits, 1, -1 do
      suffix[i] = suffix[i + 1] | bits[i]
    end
    local base = need[player]
    local function comb(start, chosen, acc)
      if chosen == k then
        local hand = acc | base
        for _, o in ipairs(ors[player]) do
          if hand & o == 0 then return false end
        end
        return rec(player + 1, remaining & ~hand)
      end
      local reach = acc | base | suffix[start]
      for _, o in ipairs(ors[player]) do
        if o & reach == 0 then return false end
      end
      if k - chosen > #bits - start + 1 then return false end
      for i = start, #bits do
        if comb(i + 1, chosen + 1, acc | bits[i]) then return true end
      end
      return false
    end
    return comb(1, 0, 0)
  end
  return rec(2, pool)
```

listing 149: lua: the pruned deal search with bit operators on 64-bit integers (source: icpc/samples-lua/ch08_pb.lua, lines 26-69)

C backtracks over the thirteen undealt cards with response clauses checked at the leaves and clause cards ordered first so dead branches die early, and java walks the same shape, the clause cards hoisted to the front of the deal order before the recursion starts. C# enumerates all 324 answer triples and tests each with forced-card placement, star disjunctions placed by recursion, and a Hall condition over the three unknown hands. Go, JavaScript and Lua share one algorithm: for each answer triple, a per-player recursive hand builder prunes on coverage, capacity, and whether the evidence sets can still be reached. Python is the judges' shape verbatim, opponent hands enumerated as 21-bit masks, player 2 crossed with player 3, the remainder looked up in the player 4 dictionary, then three-bit slices tried as the answer. The measured fixture is `2 suggestions, our hand A B D E F`, the first suggestion passing C, G and M around the whole table so all three are the hidden answer, and every suite asserts the exact string `CGM`,

with the second case forcing `A??` from a hand holding every person but A. Cards are bits 0 to 20 in every language, machine ints throughout, no bigint anywhere.

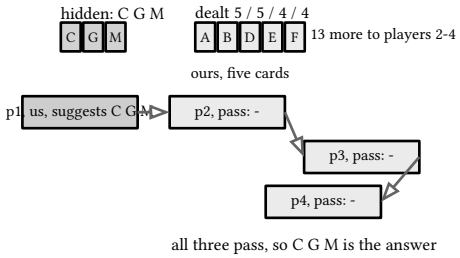


figure 93: 21 cards into 3 hidden plus hands of 5, 5, 4, 4, one suggestion passing three times then unseen evidence

10.4. problem C, mission improbable

Crates of identical size stand in piles on an r by c grid inside a warehouse. Three security cameras photograph the piles hourly: the front camera records the height of the tallest pile in each column, the side camera the tallest pile in each row, and the top camera which piles are nonempty, and any change in any image sounds the alarm. Patrick can carry crates out and rearrange what remains, but he can never add a crate or refill an empty pile, so the task is the largest number of crates that can leave while all three camera images stay exactly what they were (statement).

The input is two integers r and c, each 1 to 100 by the problemset pdf, then r lines of c integers, the pile heights, each 0 to 10^9 inclusive. The output is one integer, the maximum number of crates that can be stolen without detection, under the pdf's 1 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `C-improbable/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints with its before and after figures: the grid holds 45 crates on 20 nonempty piles, the camera views pin 23 of them in seven tall piles plus 13 single-crate leftovers, so exactly 9 can go.

input:

```
5 5
1 4 0 5 2
2 1 2 0 1
0 2 3 4 4
0 3 0 3 1
1 2 2 1 1
```

expected output:

```
9
```

Recognition: the bounds make every stratum cheap. With r and c at most 100 the grid holds at most 10^4 nonempty cells, each distinct height runs one matching over at most 100 rows plus 100 columns, and the strata are disjoint in their edges, a cell can serve only the stratum of its own height, so all the matchings together touch at most those 10^4 cells against the 1 second limit.

The statement's shape, remove the most crates while all three camera images stay exactly what they were, splits by the images themselves: every row and every column pins its maximum, and one tall pile can satisfy a row and a column at once, which is a matching question per height over the nonempty cells. Problem C is the 2017 face of the bipartite assignment feasibility family. The bipartite assignment feasibility family collects chapter 13 problem X, chapter 15 problem B, and chapter 15 problem F.

The tempting shortcut, the max(m, n) tally that keeps one pile per row or column maximum, is priced out by exactly the matching: on sample 1 the height-3 stratum has its row and column crossing on an empty cell, so it

needs two piles of 3 where max(1, 1) buys one, the tally answers 12 against the printed 9, and on a full grid the error compounds across every unmatched stratum.

Pages 3 and 4 of the solutions pdf give the method. Process the distinct heights in decreasing order. For height H, let m be the columns whose maximum is H and n the rows whose maximum is H: a pile of height H can satisfy one row and one column at once, but only where it stands on a cell the top camera sees as nonempty, so a bipartite matching between those rows and columns over the nonempty cells places m plus n minus the matching size piles of height H. After every height is processed, each still-empty originally nonempty cell keeps exactly one crate, so the kept total is the sum of H times (m + n - match(H)) over all heights plus the leftover singles, and the answer is the initial total minus that. Each height runs its own matching on at most 100 plus 100 nodes, well inside the limit. Book 10′s chapter 38, network flows ii, teaches Kuhn's augmenting-path bipartite matching from scratch. The matching is what separates this from the naive max(m, n) tally.

The worked run: trace the model on sample 1. The printed grid sums to 45 crates on 20 nonempty piles, the row maxima read 5, 2, 4, 3 and 2, the column maxima 2, 4, 3, 5 and 4, and the distinct heights descend through 5, 4, 3 and 2 with height 1 pinning nothing. Height 5: row 1 and column 4 cross at the printed 5 there, a matching of 1, so 1 + 1 - 1 = 1 pile of 5. Height 4: row 3 against columns 2 and 5, and the printed 4 at row 3 column 5 serves both, 1 + 2 - 1 = 2 piles of 4. Height 3: row 4 against column 3, and the cell where they cross holds 0, no matching, 1 + 1 - 0 = 2 piles of 3 on the printed 3s of row 4 and column 3. Height 2: rows 2 and 5 against column 1, the printed 2 at row 2 column 1 serves both, 2 + 1 - 1 = 2 piles of
2. Seven tall piles hold 5 + 8 + 6 + 4 = 23 crates, the other 13

nonempty cells keep one crate each, 23 + 13 = 36 stay, 45 - 36 = 9 can leave, and the trace ends at the printed answer 9.

height	rows at max	cols at max	match	piles	crates
5	1	4	1	1	5
4	3	2, 5	1	2	8
3	4	3	0	2	6
2	2, 5	1	1	2	4
1	none	none	0	0	0

```
  for (int t = 0; t < u; t++) {
    long long H = hs[t];
    int rows[128], nr = 0, cols[128], nc = 0;
    for (int i = 0; i < r; i++)
      if (rmax[i] == H)
        rows[nr++] = i;
    for (int j = 0; j < c; j++)
      if (cmax[j] == H)
        cols[nc++] = j;
    memset(adj, 0, (size_t)nr * (size_t)nc);
    for (int a = 0; a < nr; a++)
      for (int b = 0; b < nc; b++)
        if (h[rows[a] * c + cols[b]] > 0)
          adj[a * nc + b] = 1;
    int mt = match_kuhn(nr, nc, adj, matchR, seen);
    long long piles = nr + nc - mt;
    kept += H * piles;
    piles_total += piles;
  }
  kept += nonempty - piles_total;
  long long answer = total - kept;
```

listing 150: c: per height, rows and columns matched over nonempty cells, kept crates tallied (source: icpc/samples-c/src/Ch08/pC.c, lines 57-77)

```
  var kept int64
  piles := 0
  for _, h := range distinct {
    rows, cols := []int{}, []int{}
    for i := 0; i < r; i++ {
      if rowMax[i] == h {
        rows = append(rows, i)
      }
    }
    for j := 0; j < c; j++ {
      if colMax[j] == h {
        cols = append(cols, j)
      }
    }
    adj := make([][]int, len(rows))
    for a, i := range rows {
      for b, j := range cols {
        if grid[i][j] > 0 {
          adj[a] = append(adj[a], b)
        }
      }
    }
    match := kuhnMatch(len(rows), len(cols), adj)
    placed := len(rows) + len(cols) - match
    kept += int64(h) * int64(placed)
    piles += placed
  }
  kept += int64(nonempty - piles)
  return fmt.Sprintf("%d\n", total-kept)
}
```

listing 151: go: distinct heights descending, kuhn over adjacency lists, kept tallied (source: icpc/samples-go/ch08/pc.go, lines 60-89)

```
    for (int t = 0; t < u; t++) {
      long height = hs[t];
      int[] rows = new int[128], cols = new int[128];
      int nr = 0, nc = 0;
      for (int i = 0; i < r; i++)
        if (rmax[i] == height)
          rows[nr++] = i;
      for (int j = 0; j < c; j++)
        if (cmax[j] == height)
          cols[nc++] = j;
      for (int a = 0; a < nr; a++)
        for (int b = 0; b < nc; b++)
          adj[a][b] = h[rows[a]][cols[b]] > 0 ? (byte) 1 : 0;
      int mt = Match.kuhn(nr, nc, adj, matchR, seen);
      long piles = nr + nc - mt;
      kept += height * piles;
      pilesTotal += piles;
    }
    kept += nonempty - pilesTotal;
    return (total - kept) + "\n";
```

listing 152: java: per height, rows and columns matched through the year's dense-matrix helper, kept crates tallied (source: icpc/samples-java/src/Ch09/PC.java, lines 66-85)

```
        foreach (long h in heights.Reverse())
        {
            var rows = new List<int>();
            var cols = new List<int>();
            for (int i = 0; i < r; i++)
                if (rowMax[i] == h)
                    rows.Add(i);
            for (int j = 0; j < c; j++)
                if (colMax[j] == h)
                    cols.Add(j);
            // kuhn matching between these rows and cols over nonempty cells
            var matchCol = new int[c];
            Array.Fill(matchCol, -1);
            int matched = 0;
            foreach (int j in cols)
                matchCol[j] = -2;
            foreach (int i in rows)
            {
                var seen = new bool[c];
                if (Augment(i, cols, g, matchCol, seen))
                    matched++;
            }
            long placed = rows.Count + cols.Count - matched;
            kept += h * placed;
            piles += placed;
        }
        kept += nonempty - piles;
        return (total - kept).ToString() + "\n";
```

listing 153: c#: the same loop, augmenting paths over the column list (source: icpc/samples/src/Ch08/PC.cs, lines 40-67)

```
function kuhnMatch(left, right, adj) {
  const matchR = new Int32Array(right).fill(-1);
  const seen = new Uint8Array(right);
  const augment = (u) => {
    for (const v of adj[u]) {
      if (seen[v]) continue;
      seen[v] = 1;
      if (matchR[v] < 0 || augment(matchR[v])) {
        matchR[v] = u;
        return true;
      }
    }
    return false;
  };
  let size = 0;
  for (let u = 0; u < left; u++) {
    seen.fill(0);
    if (augment(u)) size++;
  }
  return size;
}
```

listing 154: javascript: the kuhn augmenting path inlined, matchR and seen as typed arrays (source: icpc/samples-js/src/ch08-pc-improbable.mjs, lines 13-33)

```
def _match(adj, right):
    match_r = [-1] * right
    seen = [False] * right

    def augment(u):
        for v in adj[u]:
            if not seen[v]:
                seen[v] = True
                if match_r[v] < 0 or augment(match_r[v]):
                    match_r[v] = u
                    return True
        return False

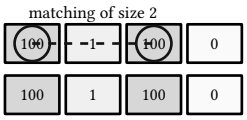
    size = 0
    for u in range(len(adj)):
        for i in range(right):
            seen[i] = False
        if augment(u):
            size += 1
    return size
```

listing 155: python: recursive augment over adjacency lists (source: icpc/samples-py/src/Ch08/pc.py, lines 11-30)

```
  local heights = {}
  for h in pairs(seen) do heights[#heights + 1] = h end
  table.sort(heights, function(a, b) return a > b end)
  local kept, piles = 0, 0
  for _, h in ipairs(heights) do
    local rows, cols = {}, {}
    for i = 1, r do
      if rowmax[i] == h then rows[#rows + 1] = i end
    end
    for j = 1, c do
      if colmax[j] == h then cols[#cols + 1] = j end
    end
    local adj = {}
    for a = 1, #rows do
      adj[a] = {}
      for b = 1, #cols do
        if grid[rows[a]][cols[b]] > 0 then adj[a][#adj[a] + 1] = b end
      end
    end
    local m = mt.kuhn(#rows, #cols, adj)
    local placed = #rows + #cols - m
    kept = kept + h * placed
    piles = piles + placed
  end
  kept = kept + (nonempty - piles)
  return string.format("%d\n", total - kept)
```

listing 156: lua: heights descending from a set, matching from the ch08_match module (source: icpc/samples-lua/ch08_pc.lua, lines 42-67)

Every language runs Kuhn augmenting paths on at most 100 plus 100 nodes per height, and the differences are carrier only: C calls its `ch08_match.h` over a dense byte matrix, java requires the `Match` helper over the same dense matrix, C# augments directly on the column list, Go feeds its `ch08_match` adjacency lists, JavaScript inlines the helper with typed arrays, Python recurses through a closure, Lua requires `ch08_match`. The measured fixture is the 2 by 4 grid `100 1 100 0 / 100 1 100 0`, total 402 crates over 6 nonempty cells, where the two rows and two 100-columns match perfectly so 2 piles of 100 stay, one pile of 1, and three single-crate leftovers, 204 kept and 198 stolen, asserted as an exact string in all seven suites. Totals reach 10^13, int64 in C, Go, java, C# and Lua, and JavaScript stays exact below 2^53, so no bigint is needed anywhere.



row maxima 100 100, column maxima 100 1 100 0, six nonempty cells
kept 200 + 1 + 3, stolen 198
a pile covers one row and one column at once only where the top view is nonempty

figure 94: the 2 by 4 grid, two 100 piles matched to both rows and both 100 columns, filler cells, three views unchanged

10.5. problem D, money for nothing

We are a middleman in the widget market and may sign with exactly one producer company and one consumer company. Each producer has a price per widget and a day its first widget is available, each consumer has a price it pays and the day immediately after its last required delivery, and a signed pair means one widget per day bought at the producer's price and sold at the consumer's, from the producer's first day through the day before the consumer's end. The profit is (q - p) times (e - d) where that is positive, and the task is the maximum over all pairs, 0 if no pair pays (statement).

The input is two integers m and n, 1 to 500000 each by the problemset pdf, then m producer lines of two integers pi and di, then n consumer lines of qj and ej, all four values 1 to 10^9. The output is one integer, the maximum profit or 0, under the pdf's 5 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `D-money/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: the producer at price 2 from day 1 against the consumer at price 7 ending after day 2 earns (7 - 2) times (2 - 1) = 5, the best contract on the table.

input:

```
2 2
1 3
2 1
3 5
7 2
```

expected output:

```
5
```

Recognition: the bounds kill the pairwise scan outright. With m and n each up to 500000, all m x n = 2.5 x 10^11 producer-consumer pairs is far past any budget, while sorting both sides, pruning dominated corners to staircases and dividing the argmax search costs O((m + n) log n), about 10^6 corners through 20 levels, or 2 x 10^7 touches, trivial inside the 5 second limit.

The statement's shape, one producer and one consumer quantity multiplied together, is a rectangle in disguise: plotting a producer at (d, p) and a consumer at (e, q), the profit (q - p)(e - d) is exactly the axis-aligned rectangle between the two corners, and the maximum is attained at finitely many staircase corners. Problem D is the 2017 face of the geometric critical-point enumeration family. Geometric critical-point enumeration is the shared shape of chapter 11 problem A, chapter 11 problem E, chapter 11 problem G, chapter 13 problem T, and chapter 14 problem D.

The tempting shortcut, a ternary search over each producer's stretch of the consumer staircase, is priced out by shape: along the staircase the span e - d grows while the margin q - p shrinks, and their product is not unimodal, spans 1, 2, 3, 10 against margins 5, 2.4, 2, 1.9 give 5, 4.8, 6 and 19, a dip at the second step before the peak, so a ternary search can settle on the wrong hill.

Pages 4 and 5 of the solutions pdf recast the search as geometry. Plot each producer as the lower-left corner (d, p) and each consumer as the upper-right corner (e, q): the profit of a pair is exactly the axis-aligned rectangle between the two corners, so the task is the largest such rectangle. Corners dominated in both coordinates by another corner of their own kind can never win, which prunes both sides to staircases, and then a divide and conquer finds every producer's best consumer: for the middle producer scan the consumers linearly, and because the optimal consumer index is monotone across the recursion, each level touches each consumer once, O((m + n) log n) in total. Book 10′s chapter 32, dynamic programming ii, develops the divide-and-conquer over a monotone argmax that this sweep is the bare-bones shape of. An alternative half-plane sweep with the same bound exists. The seven solvers agree on the recursion and split on the objective the argmax tracks.

The worked run: trace the model on sample 1. The two printed producers plot at (3, 1) and (1, 2), the two consumers at (5, 3) and (2, 7), and neither side holds a dominated corner, so both staircases keep everything. Producer (3, 1) scans its valid consumers, those with e past 3 and q past 1: (5, 3) pays (3 - 1) x (5 - 3) = 4, and (2, 7) is invalid because e = 2 never reaches d = 3. Producer (1, 2) scans both: (5, 3) pays (3 - 2) x (5 - 1) = 4, and (2, 7) pays (7 - 2) x (2 - 1) = 5, the winner. The recursion splits the two producers one per half, each half scanning the staircase once, and the trace ends at the printed answer 5.

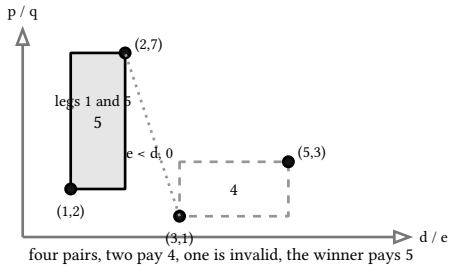


figure 95: sample 1′s four corners, the winning rectangle from producer (1,2) to consumer (2,7), the invalid pair marked

```
// the delivery window is nonempty (e > d), else negative infinity.
// plain max(0, .) clamping breaks the Monge property the recursion
// needs; pairs with e > d but q ≤ p give a nonpositive product and
// can never win against the final max with 0.
#define D_NEGBIG (-(1LL << 62))
static long long profit(Prod a, Cons b) {
  if (b.e ≤ a.d)
    return D_NEGBIG;
  return (b.q - a.p) * (b.e - a.d);
}

static void solve_dc(int ilo, int ihi, int jlo, int jhi) {
  if (ilo > ihi)
    return;
  int imid = (ilo + ihi) / 2;
  long long bv = D_NEGBIG - 1;
  int bj = jlo;
  for (int j = jlo; j ≤ jhi; j++) {
    long long v = profit(prods[imid], conss[j]);
    if (v ≥ bv) { // ties break right so the left half keeps its range
      bv = v;
      bj = j;
    }
  }
  if (bv > best)
    best = bv;
  solve_dc(ilo, imid - 1, jlo, bj);
  solve_dc(imid + 1, ihi, bj, jhi);
}
```

listing 157: c: the argmax tracks the raw signed product, ties break right, clamping is deferred to the end (source: icpc/samples-c/src/Ch08/pD.c, lines 22-50)

```
var search func(plo, phi, clo, chi int)
search = func(plo, phi, clo, chi int) {
  if plo > phi {
    return
  }
  mid := (plo + phi) / 2
  // the argmax tracks the raw product so the optimum index
  // stays monotone; only positive-margin pairs feed the answer
  bestJ, rawBest := clo, int64(math.MinInt64)
  for j := clo; j <= chi; j++ {
    dh := C[j][0] - P[mid][0]
    dv := C[j][1] - P[mid][1]
    if dh*dv > rawBest {
      rawBest = dh * dv
      bestJ = j
    }
    if dh > 0 && dv > 0 && dh*dv > best {
      best = dh * dv
    }
  }
  search(plo, mid-1, clo, bestJ)
  search(mid+1, phi, bestJ, chi)
}
if len(P) > 0 && len(C) > 0 {
  search(0, len(P)-1, 0, len(C)-1)
}
```

listing 158: go: one closure, the argmax on the raw product, positive-margin pairs feed the answer (source: icpc/samples-go/ch08/pd.go, lines 34-59)

```
static final long NEGBIG = -(1L << 62);

static long profit(int i, int j) {
  if (consE[j] <= prodD[i])
    return NEGBIG;
  return (consQ[j] - prodP[i]) * (consE[j] - prodD[i]);
}

static void solveDc(int ilo, int ihi, int jlo, int jhi) {
  if (ilo > ihi)
    return;
  int imid = (ilo + ihi) / 2;
  long bv = NEGBIG - 1;
  int bj = jlo;
  for (int j = jlo; j <= jhi; j++) {
    long v = profit(imid, j);
    if (v >= bv) { // ties break right so the left half keeps its range
      bv = v;
      bj = j;
    }
  }
  if (bv > best)
    best = bv;
  solveDc(ilo, imid - 1, jlo, bj);
  solveDc(imid + 1, ihi, bj, jhi);
}
```

listing 159: java: the argmax on the raw signed product past a negative-infinity sentinel, ties break right, the staircases sorted over boxed indices (source: icpc/samples-java/src/Ch09/PD.java, lines 37-62)

```
    private static void Divide(List<(long D, long P)> ps, List<(long E, long Q)> cs,
        int plo, int phi, int clo, int chi, ref long best)
    {
        if (plo > phi || clo > chi)
            return;
        int pm = (plo + phi) / 2;
        // valid consumers for ps[pm]: e > d (e asc) and q > p (q desc)
        var (d, p) = ps[pm];
        int jE = LowerE(cs, d);
        int jQ = UpperQ(cs, p);
        int lo = Math.Max(clo, jE);
        int hi = Math.Min(chi, jQ);
        if (lo > hi)
        {
            // windows are monotone in the producer index, so the halves keep
            // the correspondingly shifted consumer ranges
            Divide(ps, cs, plo, pm - 1, clo, Math.Min(chi, hi), ref best);
            Divide(ps, cs, pm + 1, phi, Math.Max(clo, lo), chi, ref best);
            return;
        }
        long bestV = 0;
        int bestJ = lo;
        for (int j = lo; j ≤ hi; j++)
        {
            long v = (cs[j].Q - p) * (cs[j].E - d);
            if (v > bestV)
            {
                bestV = v;
                bestJ = j;
            }
        }
        if (bestV > best)
            best = bestV;
        Divide(ps, cs, plo, pm - 1, clo, bestJ, ref best);
        Divide(ps, cs, pm + 1, phi, bestJ, chi, ref best);
    }
```

listing 160: c#: divide and conquer with binary-searched valid consumer windows per producer (source: icpc/samples/src/Ch08/PD.cs, lines 43-78)

```
  let best = 0n;
  // the argmax tracks the raw product so the optimum index stays
  // monotone; only positive-margin pairs feed the answer
  const search = (plo, phi, clo, chi) => {
    if (plo > phi) return;
    const mid = (plo + phi) >> 1;
    const pd = P[mid][0], pq = P[mid][1];
    let bestJ = clo, rawBest = null;
    for (let j = clo; j ≤ chi; j++) {
      const dh = C[j][0] - pd, dv = C[j][1] - pq;
      const prod = BigInt(dh) * BigInt(dv);
      if (rawBest === null || prod > rawBest) {
        rawBest = prod;
        bestJ = j;
      }
      if (dh > 0 && dv > 0 && prod > best) best = prod;
    }
    search(plo, mid - 1, clo, bestJ);
    search(mid + 1, phi, bestJ, chi);
  };
  if (P.length > 0 && C.length > 0) search(0, P.length - 1, 0, C.length - 1);
  return `${best}\n`;
```

listing 161: javascript: the same search with every product in bigint (source: icpc/samples-js/src/ch08-pd-money.mjs, lines 39-60)

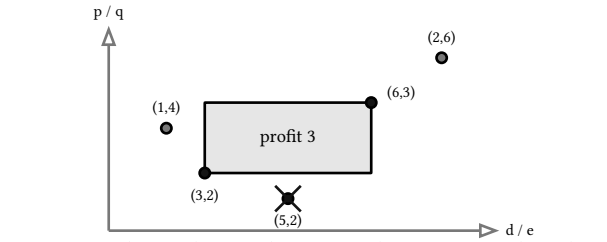
```
    best = 0
    stack = [(0, len(prod) - 1, 0, len(cons) - 1)] if prod and cons else []
    while stack:
        plo, phi, clo, chi = stack.pop()
        if plo > phi:
            continue
        mid = (plo + phi) // 2
        dm, pm = prod[mid]
        best_j, raw = clo, -(1 << 62)
        for j in range(clo, chi + 1):
            e, q = cons[j]
            dh = e - dm
            dv = q - pm
            area = dh * dv
            if area > raw:
                raw = area
                best_j = j
            if dh > 0 and dv > 0 and area > best:
                best = area
        stack.append((plo, mid - 1, clo, best_j))
        stack.append((mid + 1, phi, best_j, chi))
    return f"{best}\n"
```

listing 162: python: an explicit stack replaces the recursion (source: icpc/samples-py/src/Ch08/pd.py, lines 41-62)

```
  local P, C = staircase(prod, false), staircase(cons, true)
  local best = 0
  local function search(plo, phi, clo, chi)
    if plo > phi then return end
    local mid = (plo + phi) // 2
    local bestj, raw = clo, -math.maxinteger
    for j = clo, chi do
      local dh = C[j][1] - P[mid][1]
      local dv = C[j][2] - P[mid][2]
      if dh * dv > raw then
        raw = dh * dv
        bestj = j
      end
      if dh > 0 and dv > 0 and dh * dv > best then
        best = dh * dv
      end
    end
    search(plo, mid - 1, clo, bestj)
    search(mid + 1, phi, bestj, chi)
  end
  if #P > 0 and #C > 0 then search(1, #P, 1, #C) end
  return string.format("%d\n", best)
end
```

listing 163: lua: 64-bit integers, raw product argmax, max with zero at the end (source: icpc/samples-lua/ch08_pd.lua, lines 64-86)

Clamping at zero inside the recursion would break the monotonicity the divide and conquer feeds on, so all seven track the raw signed product for the index and only let positive-margin pairs update the answer, with C, C# and java holding the invariant through a negative-infinity sentinel and window bounds. The measured fixture is three producers, two consumers, `4 1 / 2 3 / 5 2` against `6 2 / 3 6`, where the producer (5, 2) is dominated, and the rectangle between (3, 2) and (6, 3) wins with area 3, asserted exactly in all seven suites, alongside the no-profit pair answering `0`. This is the chapter's hard integer boundary: (q - p)(e - d) reaches 10^18, past 2^53, so C, Go, java, C# and Lua multiply in 64-bit integers, 10^18 still under 2^63, while JavaScript converts both factors to BigInt for every product, never a JavaScript number.



staircases prune dominated corners, the optimum index is monotone in the producer

figure 96: two staircases, the winning rectangle between producer (3,2) and consumer (6,3), the dominated producer crossed out

10.6. problem E, need for speed

Sheila's student car lost its speedometer needle and she glued it back at a possibly wrong angle, so when the speedometer reads s the true speed is s + c for an unknown constant c that may be negative. She recorded a journey of n segments, distance and reading per segment, the whole journey took time t, and her true speed was positive on every segment. Distances are in miles, time in hours, speeds in miles per hour, and the task is to recover c (statement).

The input is two integers n and t by the problemset pdf, n from 1 to 1000 the number of segments and t from 1 to 10^6 the total time, then n lines of two integers each, the distance with 1 <= d <= 1000 and the reading with |s| <= 1000. The output is c in miles per hour within an absolute or relative error of 10^-6, under the pdf's 1 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `E-speed/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: 4 over c minus 1, plus 4 over c, plus 10 over c plus 3 sums to exactly 5 at c = 3.

input:

```
3 5
4 -1
4 0
10 3
```

expected output:

```
3.000000000
```

Recognition: the bounds invite the cheapest possible search. The unknown c is one real number, a probe costs n = 1000 fraction additions, and two hundred bisection iterations cost 2 x 10^5 divisions, nothing against the 1 second limit, while the bracket is honest: strictly above minus the smallest reading, where the slowest true speed reaches zero, up to 10^6 + 1000.

The statement's shape hands over the monotonicity: every segment contributes its distance over its true speed, d over s + c, and every such term strictly decreases as c grows across the valid range, so the whole journey took time t pins one crossing of one falling curve. Problem E is the 2017 face of the one-dimensional answer-space search family. The one-dimensional answer-space search family also collects chapter 11 problem F and chapter 15 problem C.

The tempting shortcut, solving the equation in closed form, is priced out by degree: the sum clears to a degree-n polynomial, cubic already at n = 3 and degree 1000 here, and a Newton walk needs derivatives and can jump across the pole at c = -min s, while two hundred monotone probes bracket the root to 10^-9 by construction.

Page 5 of the solutions pdf is the whole story: the time sum, each segment's distance over its true speed, is strictly decreasing in c across the valid range, so bisect the sum against t. The bounds matter more than the loop: the low bound is minus the smallest reading, exclusive, where a speed hits zero, and the high bound is 10^6 + 1000 rather than 10^6, because a journey of a thousand 1000-mile segments inside one hour forces true speeds up to a million against readings as low as minus a thousand. The pdf prescribes one hundred iterations of a linear evaluation, doubled to two hundred in every landed suite. The set's easiest problem is the chapter's control group, seven languages, one identical algorithm.

The worked run: trace the model on sample 1. The readings are −1, 0 and 3, so the pole sits at c = 1 and the bracket opens just above 1, where the first term 4 over c - 1 blows past t = 5, and closes at 10^6 + 1000, where the sum is near zero. Probe 1 at (1 + 1001000) / 2 = 500500.5 sums about 18 / 500500 = 3.6 x 10^-5, below 5, so the high half dies. Probe 2 at (1 + 500500.5) / 2 = 250250.75 sums about 18 / 250250 = 7.2 x 10^-5, again below, and the bracket keeps halving, 999999 / 2^k wide after k probes, under 10^-9 by probe 50. The root itself is exact: 4 / (3 - 1) + 4 / 3 + 10 / (3 + 3) = 2 + 4/3 + 5/3 = 2 + 3 = 5 = t, so the falling sum crosses t precisely at c = 3, and the trace ends at the printed answer `3.000000000`.

probe	mid	sum, about	against t = 5	bracket top
open	1 to 1001000	past 5 down to 0	crosses once	1001000
1	500500.5	18 / 500500 = 3.6 x 10^-5	below	500500.5
2	250250.75	18 / 250250 = 7.2 x 10^-5	below	250250.75
50	halved 50 times	999999 / 2^50 = 8.9 x 10^-10 wide	below	1 + 9 x 10^-10
root	3	2 + 4/3 + 5/3 = 5	equals t	c = 3

```
  double mins = 1e30;
  for (int i = 0; i < n; i++) {
    d[i] = strtod(p, &p);
    s[i] = strtod(p, &p);
    if (s[i] < mins)
      mins = s[i];
  }
  double lo = -mins, hi = 1e6 + 1000.0;
  for (int it = 0; it < 200; it++) {
    double mid = (lo + hi) / 2, f = 0;
    int inf = 0;
    for (int i = 0; i < n; i++) {
      if (s[i] + mid ≤ 0) {
        inf = 1;
        break;
      }
      f += d[i] / (s[i] + mid);
    }
    if (inf || f > t)
      lo = mid;
    else
      hi = mid;
  }
```

listing 164: c: bisect with a nonpositive-speed guard pushing the low bound up (source: icpc/samples-c/src/Ch08/pE.c, lines 24-46)

```
  total := func(c float64) float64 {
    sum := 0.0
    for i := 0; i < n; i++ {
      sum += float64(ints[2+2*i]) / (float64(ints[3+2*i]) + c)
    }
    return sum
  }
  lo, hi := float64(-minS), 1e6+1000.0
  for iter := 0; iter < 200; iter++ {
    mid := (lo + hi) / 2
    if total(mid) > float64(t) {
      lo = mid
    } else {
      hi = mid
    }
  }
  // samples are %.9f while the secrets are stored in Python-repr
  // form; %.9f is within the 1e-6 error bound of both
  return fmt.Sprintf("%.9f\n", (lo+hi)/2)
}
```

listing 165: go: the time sum as a closure, the high-bound comment naming 1e6+1000 (source: icpc/samples-go/ch08/pe.go, lines 26-45)

```
    double lo = -mins, hi = 1e6 + 1000.0;
    for (int it = 0; it < 200; it++) {
      double mid = (lo + hi) / 2, f = 0;
      boolean inf = false;
      for (int i = 0; i < n; i++) {
        if (s[i] + mid ≤ 0) {
          inf = true;
          break;
        }
        f += d[i] / (s[i] + mid);
      }
      if (inf || f > t)
        lo = mid;
      else
        hi = mid;
    }
    return String.format(Locale.ROOT, "%.9f", hi) + "\n";
  }
```

listing 166: java: the same loop, nonpositive-speed guard pushing the low bound up, the answer printed %.9f through String.format with Locale.ROOT (source: icpc/samples-java/src/Ch09/PE.java, lines 39-56)

```
        double lo = -s.Min() + 1e-9; // speeds must stay positive
        double hi = 1e6 + 1000;
        for (int it = 0; it < 200; it++)
        {
            double mid = (lo + hi) / 2;
            double time = 0;
            for (int i = 0; i < n; i++)
                time += d[i] / (s[i] + mid);
            if (time > t)
                lo = mid; // too slow, need larger c
            else
                hi = mid;
        }
        return ((lo + hi) / 2).ToString("F9", CultureInfo.InvariantCulture) + "\n";
```

listing 167: c#: the same loop, low starts one epsilon above minus the smallest reading (source: icpc/samples/src/Ch08/PE.cs, lines 20-33)

```
  const total = (c) => {
    let sum = 0;
    for (let i = 0; i < n; i++) sum += ints[2 + 2 * i] / (ints[3 + 2 * i] + c);
    return sum;
  };
  let lo = -minS, hi = 1e6 + 1000;
  for (let iter = 0; iter < 200; iter++) {
    const mid = (lo + hi) / 2;
    if (total(mid) > t) lo = mid;
    else hi = mid;
  }
  return `${((lo + hi) / 2).toFixed(9)}\n`;
}
```

listing 168: javascript: total as an arrow function, toFixed(9) for the output (source: icpc/samples-js/src/ch08-pe-speed.mjs, lines 14-26)

```
    lo, hi = float(-min(ss)), 1e6 + 1000.0
    for _ in range(200):
        mid = (lo + hi) / 2
        total = 0.0
        for d, s in zip(ds, ss):
            total += d / (s + mid)
        if total > t:
            lo = mid
        else:
            hi = mid
    return f"{(lo + hi) / 2:.9f}\n"
```

listing 169: python: the bisection in eleven lines (source: icpc/samples-py/src/Ch08/pe.py, lines 15-25)

```
  local function total(c)
    local sum = 0.0
    for i = 1, n do sum = sum + d[i] / (s[i] + c) end
    return sum
  end
  local lo, hi = -mins + 0.0, 1e6 + 1000.0
  for _ = 1, 200 do
    local mid = (lo + hi) / 2
    if total(mid) > t then
      lo = mid
    else
      hi = mid
    end
  end
  return string.format("%.9f\n", (lo + hi) / 2)
end
```

listing 170: lua: the time sum closure and the %.9f format (source: icpc/samples-lua/ch08_pe.lua, lines 31-46)

The measured fixture is two segments, `4 -1` and `8 1` with t equal to 4, where 4 over c minus 1 plus 8 over c plus 1 equals 4 exactly at c = 3, and every suite asserts `3.000000000` as an exact string, alongside the single fast segment forcing the negative answer `-1.000000000`. Everything runs in doubles, and the only divergence is the guard at the pole, C and java check for a nonpositive speed mid-loop, C# lifts the low bound by 1e-9, the others bisect between the strict bounds and let the sum blow up past t. Output formatting differs per language, `%.9f`, `F9`, `toFixed(9)`, java's `String.format` under `Locale.ROOT`, all landing on the same nine decimals.

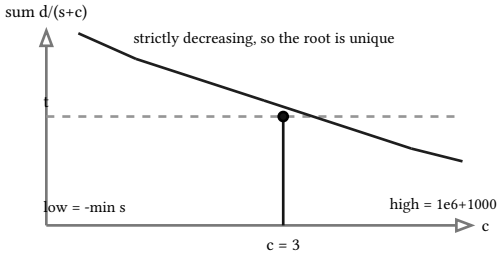


figure 97: the c axis, the strictly decreasing time sum against the horizontal t, the bracket closing on 3

10.7. problem F, posterize

Posterizing a picture replaces each color channel by at most k chosen integer levels, and this problem tracks only the red channel, values 0 to 255. Each pixel's red intensity maps to the nearest allowed level, and the editing tool picks the k levels that minimize the sum of squared errors across all pixels. The picture arrives compressed as d distinct red values with their pixel counts, and the task is that minimum sum (statement).

The input is two integers d and k by the problemset pdf, d from 1 to 256 the number of distinct red values and k from 1 to d the number of levels allowed, then d lines of two integers each, the red value r from 0 to 255 and the pixel count p from 1 to 2^26, the lines in increasing order of r. The output is one integer, the minimum achievable sum of squared errors, under the pdf's 2 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `F-posterize/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: one level for 20000 pixels at 50 and 10000 at 150 sits at the weighted mean 83.33, and the level 83 costs 20000 times 33 squared plus 10000 times 67 squared, exactly 66670000.

input:

```
2 1
50 20000
150 10000
```

expected output:

```
66670000
```

Recognition: the bounds are the whole dynamic program case. Distinct values d at most 256 and levels k at most d make the transition table O(d^2 k) = 256^2 x 256, about 1.7 x 10^7 cells against the 2 second limit, with the d^2 one-color costs precomputed in one pass each.

The statement's shape, pick k levels so each pixel maps to its nearest one for the least squared error, is clustering on a line: the values arrive sorted, an optimal level set never needs two levels sharing a boundary with no value between them, so the clusters are contiguous runs and the last chosen breakpoint drives each transition. Problem F is the 2017 face of the chain dp over sorted positions family. Chapter 11 problem B carries the same chain dp over sorted positions shape.

The tempting shortcut, generic k-means with Lloyd iterations, is priced out twice: it promises no global optimum, and with pixel counts to 2^26 every recentering pass multiplies through huge weights, while the exact level-set enumeration faces C(256,128), about 10^76 choices, and the dp is both exact and 1.7 x 10^7 cells.

Pages 5 and 6 of the solutions pdf give the dynamic program over the sorted values. Optimal clusters are contiguous runs on the sorted list, so C(i, j), the best cost of covering the first i values with j levels, is the minimum over i' < i of C(i', j-1) + F(i'+1, i), where F(a, b) is the one-color cost of the run r_a through r_b. That cost is a quadratic in the chosen level, minimized at the run's pixel-weighted mean rounded to the nearest integer, and all d squared F values precompute in one pass, leaving the dp itself O(d^2 k). The seven solvers differ on how the one-color cost is evaluated.

The worked run: trace the model on sample 1. The printed values carry weights 20000 at 50 and 10000 at 150, and k = 1 forces the whole sorted list into one cluster, C(2, 1) = F(1, 2). The one-color moments are sum p = 20000 + 10000 = 30000 and sum p x r = 20000 x 50
1. 10000 x 150 = 2500000, so the weighted mean sits at 2500000 /

30000 = 83.33 and the level rounds to 83. The cost at 83 is 20000 x 33^2 + 10000 x 67^2 = 21780000 + 44890000 = 66670000, the neighbor 84 costs 20000 x 34^2 + 10000 x 66^2 = 66680000, one more, so 83 stands and the trace ends at the printed answer `66670000`.

step	quantity	value	source
1	sum p	30000	printed counts
2	sum p x r	2500000	20000 x 50 + 10000 x 150
3	mean, level	83.33, 83	2500000 / 30000, rounded
4	cost at 83	66670000	20000 x 1089 + 10000 x 4489
5	cost at 84	66680000	neighbor test, rejected

step	quantity	value	source
6	C(2, 1)	66670000	one cluster, k = 1

```
  for (int a = 1; a ≤ d; a++) {
    long long w = 0, ws = 0;
    for (int b = a; b ≤ d; b++) {
      w += cnt[b];
      ws += cnt[b] * r[b];
      // best integer x is the rounded weighted mean; test both sides
      long long x0 = ws / w;
      if (x0 < r[a])
        x0 = r[a];
      if (x0 > r[b])
        x0 = r[b];
      long long bestc = -1;
      for (long long x = x0 - 1; x ≤ x0 + 1; x++) {
        if (x < r[a] || x > r[b])
          continue;
        long long c = 0;
        for (int i = a; i ≤ b; i++)
          c += cnt[i] * (r[i] - x) * (r[i] - x);
        if (bestc < 0 || c < bestc)
          bestc = c;
      }
      cost[a][b] = bestc;
    }
  }
  for (int i = 0; i ≤ d; i++)
    dp[0][i] = (i == 0) ? 0 : -1;
  for (int j = 1; j ≤ k; j++) {
    dp[j][0] = 0;
    for (int i = 1; i ≤ d; i++) {
      long long bestv = -1;
      for (int a = 1; a ≤ i; a++) {
        if (dp[j - 1][a - 1] < 0)
          continue;
        long long v = dp[j - 1][a - 1] + cost[a][i];
        if (bestv < 0 || v < bestv)
          bestv = v;
      }
      dp[j][i] = bestv;
    }
  }
```

listing 171: c: F(a,b) at the clamped weighted mean, both neighbors tested, then the dp (source: icpc/samples-c/src/Ch08/pF.c, lines 22-61)

```
  F := make([][]int64, d)
  for i := 0; i < d; i++ {
    F[i] = make([]int64, d)
    var sp, spr, spr2 int64
    for j := i; j < d; j++ {
      sp += p[j]
      spr += p[j] * r[j]
      spr2 += p[j] * r[j] * r[j]
      x := spr / sp
      rem := spr % sp
      if rem*2 ≥ sp {
        x++
      }
      F[i][j] = x*x*sp - 2*x*spr + spr2
    }
  }
  const inf = int64(1) << 60
  prev := make([]int64, d+1)
  cur := make([]int64, d+1)
  for i := 1; i <= d; i++ {
    prev[i] = inf
  }
  prev[0] = 0
  for kk := 1; kk <= k; kk++ {
    cur[0] = inf
    for i := 1; i <= d; i++ {
      cur[i] = inf
      for j := 0; j < i; j++ {
        if prev[j] < inf && F[j][i-1] < inf {
          if v := prev[j] + F[j][i-1]; v < cur[i] {
            cur[i] = v
          }
        }
      }
    }
    prev, cur = cur, prev
  }
  return fmt.Sprintf("%d\n", prev[d])
```

listing 172: go: three running moments give F in closed form, rolling dp arrays (source: icpc/samples-go/ch08/pf.go, lines 29-66)

```
  for (int a = 1; a ≤ d; a++) {
    long w = 0, ws = 0;
    for (int b = a; b ≤ d; b++) {
      w += cnt[b];
      ws += cnt[b] * r[b];
      // best integer x is the rounded weighted mean; test both sides
      long x0 = ws / w;
      if (x0 < r[a])
        x0 = r[a];
      if (x0 > r[b])
        x0 = r[b];
      long bestc = -1;
      for (long x = x0 - 1; x ≤ x0 + 1; x++) {
        if (x < r[a] || x > r[b])
          continue;
        long c = 0;
        for (int i = a; i ≤ b; i++)
          c += cnt[i] * (r[i] - x) * (r[i] - x);
        if (bestc < 0 || c < bestc)
          bestc = c;
      }
      cost[a][b] = bestc;
    }
  }
```

listing 173: java: F(a,b) evaluated at the clamped weighted mean, the mean and both neighbors tested inside the run's value range (source: icpc/samples-java/src/Ch09/PF.java, lines 36-59)

```
      // F[i,j] = one-color cost of r_i..r_j, best integer center
      var f = new long[d, d];
      for (int i = 0; i < d; i++)
      {
          long sp = 0, sr = 0;
          for (int j = i; j < d; j++)
          {
              sp += p[j];
              sr += p[j] * r[j];
              long lo = sr / sp; // floor and ceil of the weighted mean
              long hi = (sr + sp - 1) / sp;
              f[i, j] = Math.Min(Cost(i, j, lo, r, p), Cost(i, j, hi, r, p));
          }
      }

      const long inf = long.MaxValue / 4;
      var dp = new long[k + 1, d + 1];
      for (int j = 0; j ≤ d; j++)
          dp[0, j] = j == 0 ? 0 : inf;
      for (int c = 1; c ≤ k; c++)
          for (int j = 1; j ≤ d; j++)
          {
              long best = inf;
              for (int i = 0; i < j; i++)
                  if (dp[c - 1, i] < inf && f[i, j - 1] < inf)
                      best = Math.Min(best, dp[c - 1, i] + f[i, j - 1]);
              dp[c, j] = best;
          }
```

listing 174: c#: floor and ceil of the mean, explicit cost loops, dp over at most k levels (source: icpc/samples/src/Ch08/PF.cs, lines 18-45)

```
// F[i][j] = cost of clustering values i..j into one level
const F = new Array(d);
for (let i = 0; i < d; i++) {
  F[i] = new Float64Array(d);
  let sp = 0, spr = 0, spr2 = 0;
  for (let j = i; j < d; j++) {
    sp += p[j];
    spr += p[j] * r[j];
    spr2 += p[j] * r[j] * r[j];
    let x = Math.floor(spr / sp);
    if ((spr % sp) * 2 ≥ sp) x++;
    F[i][j] = x * x * sp - 2 * x * spr + spr2;
  }
}
const inf = Infinity;
let prev = new Float64Array(d + 1).fill(inf);
prev[0] = 0;
let cur = new Float64Array(d + 1);
for (let kk = 1; kk ≤ k; kk++) {
  cur[0] = inf;
  for (let i = 1; i ≤ d; i++) {
    cur[i] = inf;
    for (let j = 0; j < i; j++) {
      if (prev[j] < inf) {
        const v = prev[j] + F[j][i - 1];
        if (v < cur[i]) cur[i] = v;
      }
    }
  }
  [prev, cur] = [cur, prev];
}
return `${prev[d]}\n`;
```

listing 175: javascript: the same closed form, exact in doubles under 2^53 (source: icpc/samples-js/src/ch08-pf-posterize.mjs, lines 20-51)

```
    # F[i][j] = one-level cost of values i..j, filled incrementally in j
    cost = []
    for i in range(d):
        row = [0] * d
        sp = spr = spr2 = 0
        for j in range(i, d):
            pj, rj = p[j], r[j]
            sp += pj
            spr += pj * rj
            spr2 += pj * rj * rj
            x = spr // sp
            if spr % sp * 2 ≥ sp:
                x += 1
            row[j] = x * x * sp - 2 * x * spr + spr2
        cost.append(row)
    prev = [INF] * (d + 1)
    prev[0] = 0
    for _ in range(k):
        cur = [INF] * (d + 1)
        for i in range(1, d + 1):
            im1 = i - 1
            cand = [INF] + [
                pv + row[im1] for pv, row in zip(prev[:i], cost) if pv < INF
            ]
            cur[i] = min(cand)
        prev = cur
    return f"{prev[d]}\n"
```

listing 176: python: the cost rows as lists, the dp as a min comprehension (source: icpc/samples-py/src/Ch08/pf.py, lines 20-46)

```
local F = {}
for i = 1, d do
  F[i] = {}
  local sp, spr, spr2 = 0, 0, 0
  for j = i, d do
    sp = sp + p[j]
    spr = spr + p[j] * r[j]
    spr2 = spr2 + p[j] * r[j] * r[j]
    local x = spr // sp
    if (spr % sp) * 2 ≥ sp then x = x + 1 end
    F[i][j] = x * x * sp - 2 * x * spr + spr2
  end
end
local inf = 1 << 60
local prev, cur = {}, {}
for i = 0, d do prev[i] = inf end
prev[0] = 0
for kk = 1, k do
  cur[0] = inf
  for i = 1, d do
    cur[i] = inf
    for j = 0, i - 1 do
      if prev[j] < inf and F[j + 1][i] < inf then
        local v = prev[j] + F[j + 1][i]
        if v < cur[i] then cur[i] = v end
      end
    end
  end
  prev, cur = cur, prev
end
return string.format("%d\n", prev[d])
```

listing 177: lua: closed-form moments, integer division and rounding by hand (source: icpc/samples-lua/ch08_pf.lua, lines 18-48)

Go, JavaScript, Python and Lua accumulate the three moments sum p, sum p times r, sum p times r squared and evaluate the cost as x squared times sum p minus two x times the weighted sum plus the raw moment sum, in one pass per starting index. C, C# and java instead evaluate the cost explicitly at the rounded weighted mean, C and java testing x and both neighbors clamped into the run's value range, C# testing the floor and the ceiling. The measured fixture is three values with counts `0 2 / 10 3 / 11 2` and k equal to 2, where the split at 0 and a shared level for 10 and 11 costs 2 and every suite asserts the exact integer 2, alongside k equal to d answering `0`. The integer boundary is real but one-sided: d at most 256 values at p at most 2^26 each parks up to 2^34 pixels in one cluster, and their error around the cluster's weighted mean averages at most 255^2 over 4 per pixel, so the worst cluster costs about 2.8 times 10^14, int64 territory in C, Go, java, C# and Lua, and every intermediate in the closed form stays below 2^53, so JavaScript's doubles are exact and no bigint appears.

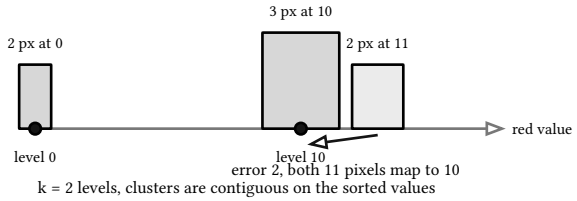


figure 98: the intensity line, three pixel masses, allowed values at 0 and 10, the error bar from 11

10.8. problem G, replicate replicate rfplicbte

A cellular lattice of two-state cells replicates parts: at each discrete step every cell updates simultaneously, becoming filled exactly when an odd number of the nine cells of its 3 by 3 neighborhood, itself included, are filled. A bug crept in: after each step at most one cell in the lattice may spontaneously flip its state. The original patterns are lost and only a possibly corrupted final pattern remains, and the task is a smallest possible nonempty initial pattern, by bounding-box area, that could have produced it, printed with '.' for empty and '#' for filled in the fewest rows and columns that display it. If several minima exist any one is accepted, though the answer is in fact unique (statement).

The input is two integers w and h, each 1 to 300 by the problemset pdf, the width and height of the final pattern's bounding box, then h lines of w characters, '.' or '#', with at least one filled cell on every side of the box. The output is the minimum-size pattern in its tight bounding box, under the pdf's 3 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `G-replicate/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints in its corrupted-replication figure: the ten by ten board, a flipped cell in two of its four steps, walks back to a two by two seed.

input:

```
10 10
.#...#...#
##..##..##
##.#.##...
##.#.##...
.#...#####
...##..#.#
......###.
##.#.##...
#..#..#..#
##..##..##
```

expected output:

```
.#
##
```

Recognition: the bounds price the backward walk directly. The box is at most 300 by 300, every forward step grows it by at least one cell per side, so at most min((w+1)/2, (h+1)/2) = 150 backward steps exist and each is a linear sweep with at most one repair, the chapter's O((w+h)^3) worst case about 2 x 10^8 byte xors against the 3 second limit.

The statement's shape is the inversion itself: the forward rule, filled exactly when an odd number of the nine neighborhood cells are filled, is fully known and deterministic, and a smallest initial pattern asks which earlier states could have produced the final one, so the run walks the rule backwards and repairs the at most one flipped cell each step leaves. Problem G is the 2017 face of the inverting a deterministic process family. Chapter 14 problem A is the other half of the inverting a deterministic process pair.

The tempting shortcut, searching forward from candidate seeds, is priced out by the space: patterns inside a 300 by 300 box run 2^90000 deep before the per-step flips, unenumerable at any limit, while the backward recurrence hands out every earlier cell in one xor of already-known neighbors.

Page 6 of the solutions pdf runs the rule backwards. Each step grows the bounding box by at least one cell in each dimension, so at most min((w+1)/2, (h+1)/2) backward steps exist, and each step inverts the rule cell by cell: sweeping row-major, X(r,c) equals the stored Y(r-1,c-1) up-left of it xor the eight X cells around that up-left position, all of them already reconstructed. A flipped cell poisons exactly two cells of this reconstruction and then

self-heals on the third, so the two cells just past each row's end verify every row, and on failure a second column-major sweep pinpoints the flipped cell by where the two sweeps disagree, undo it, and redo the step. The sweep stops at a single filled cell, worst case O((n+m)^3), and the answer is unique, so byte comparison verifies.

The worked run: trace the model on sample 1. The reconstruction sweeps the printed ten by ten box padded by one ring on every side, solving X(r, c) = Y(r - 1, c - 1) xor the eight already-rebuilt neighbors of that up-left cell, and the two cells past every row end and column end must come out empty. The first pass is clean, all sentinels zero, and the ten by ten collapses to an eight by eight. The second step fails its row sentinels first at row 4 and its column sentinels at column 6, so the crossed sweeps pin the bug's flip at Y(4, 6), a cell the bug filled that the clean pattern leaves empty: undo it, redo, and the eight by eight walks back to a six by six. The third pass is clean again, six by six to four by four. The fourth fails both checks at row 2, column 0, a cell the bug emptied that the clean pattern fills, and undoing it lands on the two by two `.#` over `##`. The fifth step still fails, its one localized repair tried and discarded, so the two by two has no consistent pre-image and is itself the minimum, and the trace ends at the printed answer `##`, the second of its two lines.

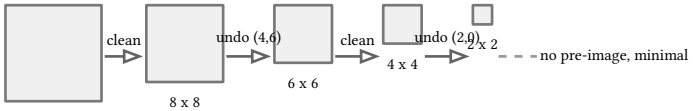


figure 99: sample 1 walked backwards, clean and repaired steps alternating, the 2x2 seed certified by the failed fifth

```c
static int reconstruct(char *g, int gw, int r0, int c0, int h, int w, int *er,
                       int *ec) {
  *er = -1;
  *ec = -1;
  for (int r = -1; r ≤ h + 1; r++) {
    char *row = X[r + 1] + 1;
    for (int c = -1; c ≤ w + 1; c++) {
      int v = 0;
      if (r ≥ 1 && c ≥ 1 && r - 1 < h && c - 1 < w)
        v = g[(r0 + r - 1) * gw + c0 + c - 1];
      for (int rr = r - 2; rr ≤ r; rr++)
        for (int cc = c - 2; cc ≤ c; cc++) {
          if ((rr == r && cc == c) || rr < -1 || cc < -1)
            continue;
          v ^= X[rr + 1][cc + 1];
        }
      row[c] = (char)v;
    }
  }
  for (int r = -1; r ≤ h + 1; r++)
    if (X[r + 1][w + 1] || X[r + 1][w + 2]) {
      *er = r - 1; // a flip at Y(er, .) poisons rows er+1, er+2
      break;
    }
  for (int c = -1; c ≤ w + 1; c++)
    if (X[h + 1][c + 1] || X[h + 2][c + 1]) {
      *ec = c - 1;
      break;
    }
  for (int r = -1; r ≤ h + 1; r++)
    if (X[r + 1][w + 1] || X[r + 1][w + 2])
      return 0;
  for (int c = -1; c ≤ w + 1; c++)
    if (X[h + 1][c + 1] || X[h + 2][c + 1])
      return 0;
  return 1;
}
```

listing 178: c: reconstruct over the padded frame, the two sentinel cells past every row and column end (source: icpc/samples-c/src/Ch08/pG.c, lines 20-56)

```go
func stepBack(y [][]byte) ([][]byte, bool) {
  h, w := len(y), len(y[0])
  if h < 3 || w < 3 {
    return nil, false
  }
  x, failRow := rowSweep(y)
  if failRow < 0 {
    if mismatches(x, y) <= 1 {
      return box(x), true
    }
    return nil, false
  }
  _, failCol := colSweep(y)
  tryFlip := func(zr, zc int) ([][]byte, bool) {
    if zr < 0 || zr ≥ h || zc < 0 || zc ≥ w {
      return nil, false
    }
    y2 := make([][]byte, h)
    for i := range y {
      y2[i] = append([]byte(nil), y[i]...)
    }
    y2[zr][zc] ^= 1
    x2, fail2 := rowSweep(y2)
    if fail2 ≥ 0 || mismatches(x2, y2) ≠ 0 {
      return nil, false
    }
    return box(x2), true
  }
  if failCol ≥ 0 {
    if out, ok := tryFlip(failRow-1, failCol-1); ok {
      return out, true
    }
    return nil, false
  }
  // the flip sits in one of the two columns only the row sweep
  // visits; try both
  for zc := w - 2; zc <= w-1; zc++ {
    if out, ok := tryFlip(failRow-1, zc); ok {
      return out, true
    }
  }
  return nil, false
}
```

listing 179: go: one backward step, the flip localized by the crossed sweeps and retried (source: icpc/samples-go/ch08/pg.go, lines 185-227)

```java
static int reconstruct(byte[][] g, int gw, int r0, int c0, int h, int w,
    int[] err, int[] errc) {
  err[0] = -1;
  errc[0] = -1;
  for (int r = -1; r ≤ h + 1; r++) {
    byte[] row = xg[r + 1];
    for (int c = -1; c ≤ w + 1; c++) {
      int v = 0;
      if (r ≥ 1 && c ≥ 1 && r - 1 < h && c - 1 < w)
        v = g[r0 + r - 1][c0 + c - 1];
      for (int rr = r - 2; rr ≤ r; rr++)
        for (int cc = c - 2; cc ≤ c; cc++) {
          if ((rr == r && cc == c) || rr < -1 || cc < -1)
            continue;
          v ^= xg[rr + 1][cc + 1];
        }
      row[c + 1] = (byte) v;
    }
  }
  for (int r = -1; r ≤ h + 1; r++)
    if (xg[r + 1][w + 1] ≠ 0 || xg[r + 1][w + 2] ≠ 0) {
      err[0] = r - 1; // a flip at Y(er, .) poisons rows er+1, er+2
      break;
    }
  for (int c = -1; c ≤ w + 1; c++)
    if (xg[h + 1][c + 1] ≠ 0 || xg[h + 2][c + 1] ≠ 0) {
      errc[0] = c - 1;
      break;
    }
  for (int r = -1; r ≤ h + 1; r++)
    if (xg[r + 1][w + 1] ≠ 0 || xg[r + 1][w + 2] ≠ 0)
      return 0;
  for (int c = -1; c ≤ w + 1; c++)
    if (xg[h + 1][c + 1] ≠ 0 || xg[h + 2][c + 1] ≠ 0)
      return 0;
  return 1;
}
```

listing 180: java: reconstruct over the padded frame, the two sentinel cells past every row and column end checked, the flip position returned for the undo (source: icpc/samples-java/src/Ch09/PG.java, lines 21-57)

```
    // row-major reconstruction over the full [y0+1..y1+1] x [x0+1..x1+1]
    // rectangle: every box equation gets enforced, so a clean sweep is an exact
    // preimage. any nonzero outside the region flags the first bad row; the
    // poison from a flipped Y cell (zr, zc) lives at rows ≥ zr+1 and
    // cols ≥ zc+1, so the first bad row is zr+1 (column-major: zc+1)
    private static (bool[][] X, bool Ok, int BadRow) RowSweep(bool[][] y,
        int y0, int y1, int x0, int x1)
    {
        var x = new bool[y.Length][];
        for (int i = 0; i < x.Length; i++)
            x[i] = new bool[y[0].Length];
        for (int r = y0 + 1; r ≤ y1 + 1; r++)
        {
            bool bad = false;
            for (int c = x0 + 1; c ≤ x1 + 1; c++)
            {
                x[r][c] = y[r - 1][c - 1]
                    ^ x[r - 2][c - 2] ^ x[r - 2][c - 1] ^ x[r - 2][c]
                    ^ x[r - 1][c - 2] ^ x[r - 1][c - 1] ^ x[r - 1][c]
                    ^ x[r][c - 2] ^ x[r][c - 1];
                if ((c > x1 - 1 || r > y1 - 1) && x[r][c])
                    bad = true;
            }
            if (bad)
                return (x, false, r);
        }
        return (x, true, -1);
    }
```

listing 181: c#: the row sweep enforcing every box equation, the first bad row reported (source: icpc/samples/src/Ch08/PG.cs, lines 88-115)

```
// one backward step, repairing at most one flipped cell
function stepBack(y) {
  const h = y.length, w = y[0].length;
  if (h < 3 || w < 3) return [null, false];
  let [x, failRow] = rowSweep(y);
  if (failRow < 0) {
    if (mismatches(x, y) ≤ 1) return [box(x), true];
    return [null, false];
  }
  const [, failCol] = colSweep(y);
  const tryFlip = (zr, zc) ⇒ {
    if (zr < 0 || zr ≥ h || zc < 0 || zc ≥ w) return [null, false];
    const y2 = y.map((row) ⇒ row.slice());
    y2[zr][zc] ^= 1;
    const [x2, fail2] = rowSweep(y2);
    if (fail2 ≥ 0 || mismatches(x2, y2) ≠ 0) return [null, false];
    return [box(x2), true];
  };
  if (failCol ≥ 0) return tryFlip(failRow - 1, failCol - 1);
  // the flip sits in one of the two columns only the row sweep visits
  for (let zc = w - 2; zc ≤ w - 1; zc++) {
    const [out, ok] = tryFlip(failRow - 1, zc);
    if (ok) return [out, true];
  }
  return [null, false];
}
```

listing 182: javascript: the same repair, typed-array rows (source: icpc/samples-js/src/ch08-pg-replicate.mjs, lines 100-125)

```python
def _scan3(e, w):
    # x_k = x_{k-3} ^ e_k: groups of three bits prefix-xor, four
    # groups at a time through a precomputed transition table
    global _SCAN
    if _SCAN is None:
        tab = []
        for idx in range(1 << 15):
            state, chunk, out = idx >> 12, idx & 0xFFF, 0
            for g in range(4):
                state ^= chunk >> (3 * g) & 7
                out |= state << (3 * g)
            tab.append(out << 3 | state)
        _SCAN = tab
    tab = _SCAN
    out = state = base = 0
    while base < w:
        v = tab[state << 12 | (e >> base & 0xFFF)]
        out |= v >> 3 << base
        state = v & 7
        base += 12
    return out


def _w3(v):
    return v ^ v << 1 ^ v << 2


def _row_sweep(yrows, h, w):
    # returns (x rows as ints, first failing 1-based row or -1)
    xrows = []
    p2 = p1 = 0
    for r in range(h - 2):
        d = yrows[r] ^ _w3(p2) ^ _w3(p1)
        xr = _scan3(d ^ (d << 1), w)
        xrows.append(xr)
        if xr >> (w - 2) & 3:
            return xrows, r + 1
        p2, p1 = p1, xr
    return xrows, -1
```

listing 183: python: rows as ints, groups of three bits prefix-xored through a 32768-entry transition table (source: icpc/samples-py/src/Ch08/pg.py, lines 15-53)

```lua
local function row_sweep(y)
  local x = newgrid(y.h - 2, y.w)
  for r = 1, y.h - 2 do
    for cc = 1, y.w do
      local v = cell(y, r, cc)
      for dr = -2, 0 do
        for dc = -2, 0 do
          if not (dr == 0 and dc == 0) then
            v = v ~ cell(x, r + dr, cc + dc)
          end
        end
      end
      setcell(x, r, cc, v)
    end
    if cell(x, r, y.w - 1) ~= 0 or cell(x, r, y.w) ~= 0 then
      return x, r
    end
  end
  return x, 0
end

-- transposed reconstruction, first failing column in the same style
local function col_sweep(y)
  local x = newgrid(y.h, y.w - 2)
  for cc = 1, y.w - 2 do
    for r = 1, y.h do
      local v = cell(y, r, cc)
      for dr = -2, 0 do
        for dc = -2, 0 do
          if not (dr == 0 and dc == 0) then
            v = v ~ cell(x, r + dr, cc + dc)
          end
        end
      end
      setcell(x, r, cc, v)
    end
    if cell(x, y.h - 1, cc) ~= 0 or cell(x, y.h, cc) ~= 0 then
      return x, cc
    end
  end
  return x, 0
end
```

listing 184: lua: row and column sweeps over flat 1-based grids, zero outside (source: icpc/samples-lua/ch08_pg.lua, lines 54-95)

Six of the seven walk cells in nested loops, C and java over one padded frame with the sentinels folded into the same grid, C# over the full shrunk rectangle, Go, JavaScript and Lua over sentinel columns per row. Python is the outlier: each row is one arbitrary-precision integer, the three-cell recurrence becomes a prefix-xor over bit groups, and a precomputed table advances four groups at a time, the same transition-table trick the Python judge solutions favored. The measured fixture is the solid 3 by 3 block, one clean backward step collapses it to the single center cell, and every suite asserts the one-character output `#`, alongside the already-minimal 1 by 1 block. Cells are bytes or bits everywhere, no integer concerns.

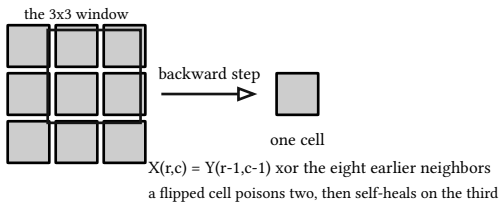


figure 100: the 3x3 block with the xor stencil centered on (1,1) collapsing to one cell, the arrow pointing backwards

10.9. problem H, scenery

A day in Rapid City, one camera, n photographs of Badlands features. Feature i has an earliest start time a and a completion deadline b, and each shot occupies the photographer for t consecutive time units that must fall entirely inside its window, one shot at a time. The task is to decide whether every photo fits in the day, yes or no in lowercase (statement). Zero teams solved it in contest.

The input is two integers n and t by the problemset pdf, n from 1 to 10^4 the number of photographs and t from 1 to 10^5 the time each one takes, then n lines of two nonnegative integers a and b with a + t <= b <= 10^9, the earliest start and the completion deadline of each shot. The output is the single word yes or no under the pdf's 6 second limit.

Official sample 2, reprinted byte for byte from the judge data pair `H-scenery/sample-2.in` and `sample-2.ans`, the judge data's only infeasible sample: the [1,15] photo's shot spans [s, s+10] for some s from 1 to 5, which cuts the day into end pieces of at most 5 and 9 time units, never the 10 consecutive units the [0,20] photo still needs, so no.

input:

```
2 10
1 15
0 20
```

expected output:

```
no
```

Recognition: the bounds pin the method to bookkeeping over photos, never over time. Deadlines reach 10^9, so anything that walks clock ticks is dead on arrival, while n = 10^4 photos make the forbidden-region phase O(n^2) = 10^8 integer comparisons and the greedy pass O(n log n), together comfortable inside the 6 second limit.

The statement's shape, does every photo fit when each shot takes the same t units somewhere inside its own window, is interval scheduling with equal lengths, and the certified answer runs an earliest deadline pass that avoids regions proven to starve a subproblem, an exchange argument behind every placement. Problem H is the 2017 face of the greedy with a safety proof family. Greedy with a safety proof is the family of chapter 11 problem K, chapter 12 problem A, and chapter 14 problem H.

The tempting shortcut, the plain greedy that shoots at time zero whenever the clock allows, is priced out by the chapter's own fixture, windows 0 8, 1 3 and 4 6 with t = 2, where opening at [0,2] strands [1,3], and the 10^9-wide deadline axis prices out every per-tick dynamic program in the same breath.

Pages 7 and 8 of the solutions pdf, adapted from Garey, Johnson, Simons and Tarjan 1981, give two phases. Phase one computes forbidden regions: for each interval [s, e] with s a photo's start time and e any photo's end time, processed with s latest-first, take the photos whose windows sit inside [s, e], schedule them as late as possible while respecting the regions already found, and let C be the first shot's start time. If C falls below s the day is infeasible, and if C sits within t of s then starting a shot in [C - t, s) starves this subproblem, so that span is marked forbidden for starts, collapsed per s. Incremental bookkeeping makes the phase O(n^2). Phase two runs a forward earliest-deadline greedy that never starts a shot inside a forbidden region, and the solutions prove that a greedy

failure always exhibits a phase-one failure, so a greedy success is final. Book 10′s chapter 40, game theory and scheduling, collects the Garey-Johnson-Simons-Tarjan window scheduling family this comes from. A greedy that shoots at time zero on the fixture is exactly the trap.

The worked run: trace the model on the section's printed sample, official sample 2, the judge data's only infeasible one. The photos are [1,15] and [0,20] with t = 10, the distinct start times 1 and 0 are swept latest first, and the distinct ends are 15 and 20. At s = 1 the subproblem [1,15] holds only the [1,15] photo, stacked as late as it fits, the shot [5,15] with C = 5, and [1,20] holds the same photo stacked [10,20] with C = 10; the per-s collapse takes minC = 5, and since 5 - 1 = 4 < 10, every start in (C - t, s) = (-5, 1) starves a subproblem, which forbids the one nonnegative integer start 0. At s = 0 the subproblem [0,20] holds both photos, stacked [10,20] and then [0,10], but the start 0 is forbidden, the second shot slides to −5, and C = −5 < s = 0 declares the day infeasible before the greedy ever runs, and the trace ends at the printed answer `no`.

[s, e]	photos	as-late stack	C	verdict
[1, 15]	1	5 to 15	5	slack 4 < 10, forbid start 0
[1, 20]	1	10 to 20	10	collapsed into minC = 5
[0, 15]	1	5 to 15	5	holds, never binds
[0, 20]	1, 2	10 to 20, then −5 to 5	−5	C < s: infeasible

```
// activates slot idx in the deadline-ordered chain and restores the
// as-late invariant downward (every start = min(own deadline, the
// start above) pushed out of forbidden regions)
static void chain_activate(int idx, int n, long long t) {
  dsu_n[idx] = idx;
  dsu_p[idx] = idx;
  int up = find_prev(idx - 1);
  int down = find_next(idx + 1);
  long long cand = ph[idx].b - t;
  if (up > 0 && cst[up] - t < cand)
    cand = cst[up] - t;
  cst[idx] = adjust_down(cand);
  cnb[up] = idx;
  cpv[idx] = up;
  cnb[idx] = down;
  cpv[down] = idx;
  if (down == n + 1)
    lowest = idx;
  int prev = idx, node = down;
  while (node ≤ n) {
    long long np = cst[node];
    if (cst[prev] - t < np)
      np = cst[prev] - t;
    np = adjust_down(np);
    if (np == cst[node])
      break;
    cst[node] = np;
    prev = node;
    node = cnb[node];
  }
}
```

listing 185: c: the as-late chain kept incrementally, activation and cascade over deadline order (source: icpc/samples-c/src/Ch08/pH.c, lines 110-140)

```
  for i := 0; i < n; {
    s := byA[i][0]
    j := i
    for j < n && byA[j][0] == s {
      j++
    }
    for k := i; k < j; k++ {
      from := sort.SearchInts(distinct, byA[k][1])
      for e := from; e < ne; e++ {
        // pack one more photo in front of C, pushed below
        // any region it would start inside
        C[e] = int64(regions.below(int(C[e]) - t))
        cnt[e]++
      }
    }
    minC := int64(1) << 62
    for e := 0; e < ne; e++ {
      if cnt[e] == 0 {
        continue
      }
      if C[e] < int64(s) {
        return false
      }
      if C[e] < minC {
        minC = C[e]
      }
    }
    if minC < 1<<62 && minC-int64(s) < int64(t) {
      regions.add(int(minC)-t, s)
    }
    i = j
  }
  return true
}
```

listing 186: go: the latest-first sweep with C pushed below regions per packed photo (source: icpc/samples-go/ch08/ph.go, lines 111-144)

```
static void chainActivate(int idx, int n, long t) {
  dsuN[idx] = idx;
  dsuP[idx] = idx;
  int up = findPrev(idx - 1);
  int down = findNext(idx + 1);
  long cand = pb[idx] - t;
  if (up > 0 && cst[up] - t < cand)
    cand = cst[up] - t;
  cst[idx] = adjustDown(cand);
  cnb[up] = idx;
  cpv[idx] = up;
  cnb[idx] = down;
  cpv[down] = idx;
  if (down == n + 1)
    lowest = idx;
  int prev = idx, node = down;
  while (node ≤ n) {
    long np = cst[node];
    if (cst[prev] - t < np)
      np = cst[prev] - t;
    np = adjustDown(np);
    if (np == cst[node])
      break;
    cst[node] = np;
    prev = node;
    node = cnb[node];
  }
}
```

listing 187: java: chainActivate splices the slot into the deadline-ordered chain and cascades the as-late invariant downward (source: icpc/samples-java/src/Ch09/PH.java, lines 125-152)

```
        foreach (long s in byStart.Keys.OrderByDescending(x => x))
        {
            foreach (int p in byStart[s])
            {
                int lo = LowerBound(ends, b[p]);
                for (int e = lo; e < m; e++)
                {
                    big[e] = has[e]
                        ? Left(big[e] - t, regions)
                        : Left(ends[e] - t, regions);
                    has[e] = true;
                }
            }
            long minC = long.MaxValue;
            for (int e = 0; e < m; e++)
            {
                if (!has[e])
                    continue;
                if (big[e] < s)
                    return false;
                minC = Math.Min(minC, big[e]);
            }
            if (minC < long.MaxValue && minC - s < t)
                AddRegion(regions, minC - t, s);
        }
        return true;
    }
```

listing 188: c#: phase one over distinct ends, big arrays of first starts, regions merged sorted (source: icpc/samples/src/Ch08/PH.cs, lines 41-67)

```
function collectRegions(photos, t, regions) {
  const n = photos.length;
  const byA = [...photos].sort((x, y) => y[0] - x[0]);
  const ends = [...new Set(photos.map((p) => p[1]))].sort((a, b) => a - b);
  const ne = ends.length;
  const C = Float64Array.from(ends);
  const cnt = new Int32Array(ne);
  const lowerBound = (arr, v) => {
    let a = 0, b = arr.length;
    while (a < b) {
      const m = (a + b) >> 1;
      if (arr[m] < v) a = m + 1;
      else b = m;
    }
    return a;
  };
  for (let i = 0; i < n; ) {
    const s = byA[i][0];
    let j = i;
    while (j < n && byA[j][0] === s) j++;
    for (let k = i; k < j; k++) {
      const from = lowerBound(ends, byA[k][1]);
      for (let e = from; e < ne; e++) {
        // pack one more photo in front of C, pushed below any region
        // it would start inside
        C[e] = regions.below(C[e] - t);
        cnt[e]++;
      }
    }
    let minC = Infinity;
    for (let e = 0; e < ne; e++) {
      if (cnt[e] === 0) continue;
      if (C[e] < s) return false;
      if (C[e] < minC) minC = C[e];
    }
    if (minC < Infinity && minC - s < t) regions.add(minC - t, s);
    i = j;
  }
  return true;
}
```

listing 189: javascript: phase one with a region set answered by binary search (source: icpc/samples-js/src/ch08-ph-scenery.mjs, lines 66-105)

```python
def _collect_regions(photos, t, regions):
    # c[e] tracks the as-late first start for the photos inside
    # [s, ends[e]]; it only ever decreases, so the per-group minimum
    # and the C ≥ s check reduce to one running minimum
    n = len(photos)
    by_a = sorted(photos, key=lambda p: -p[0])
    ends = sorted({b for _, b in photos})
    ne = len(ends)
    idx_of = {v: i for i, v in enumerate(ends)}
    c = list(ends)
    below = regions.below
    min_c = BIG
    i = 0
    while i < n:
        s = by_a[i][0]
        j = i
        while j < n and by_a[j][0] == s:
            j += 1
        for k in range(i, j):
            for e in range(idx_of[by_a[k][1]], ne):
                v = below(c[e] - t)
                c[e] = v
                if v < min_c:
                    min_c = v
        if min_c < BIG:
            if min_c < s:
                return False
            if min_c - s < t:
                regions.add(min_c - t, s)
        i = j
    return True
```

listing 190: python: the per-s minimum kept running, regions as descending interval lists (source: icpc/samples-py/src/Ch08/ph.py, lines 63-93)

```
  local i = 1
  while i ≤ n do
    local s = by_a[i][1]
    local j = i
    while j ≤ n and by_a[j][1] == s do j = j + 1 end
    for k = i, j - 1 do
      -- first index of an end ≥ this photo's end (ascending array)
      local from = 1
      local l, r = 1, ne + 1
      while l < r do
        local m = (l + r) // 2
        if distinct[m] ≥ by_a[k][2] then r = m else l = m + 1 end
      end
      from = l
      for e = from, ne do
        C[e] = regions:below(C[e] - t)
        cnt[e] = cnt[e] + 1
      end
    end
    local min_c = math.maxinteger
    for e = 1, ne do
      if cnt[e] > 0 then
        if C[e] < s then return false end
        if C[e] < min_c then min_c = C[e] end
      end
    end
    if min_c < math.maxinteger and min_c - s < t then
      regions:add(min_c - t, s)
    end
    i = j
  end
  return true
end
```

listing 191: lua: the same sweep, the region metatable carrying below and above (source: icpc/samples-lua/ch08_ph.lua, lines 107-139)

C and java are the structural pair, keeping the as-late schedule as an incrementally maintained chain over deadline order with a successor structure, so each activation cascades downward instead of rebuilding, java's forbidden regions merged into sorted arrays answered by binary search. The other five share the [s, e] subproblem sweep over distinct end times with a region set answered by binary search, the region container differing per language, sorted arrays in C#, a descending-interval struct in Go and JavaScript and Python and Lua with bisect in Python and a metatable in Lua. The measured fixture is three windows `0 8 / 1 3 / 4 6` with t equal to 2, where [1,3] and [4,6] lock and [0,8] takes [6,8], so every suite asserts `yes`, alongside the two identical windows answering `no` and the pair forcing a start at zero answering `yes`. Times stay under a billion and n times t under a billion, int64-safe in the four int64 languages and exact in JavaScript's doubles.

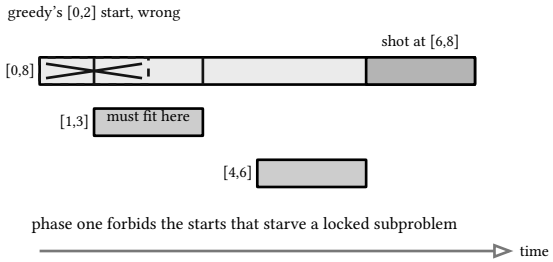


figure 101: three windows, the two short shots locked late, the [0,8] shot at [6,8], the greedy's wrong [0,2] start crossed out

10.10. problem I, secret chamber at mount rushmore

A hidden chamber at Mount Rushmore holds documents enciphered letter by letter. The key is a list of directed translations between lowercase letters, some letters with several translations, some with none, and applying them repeatedly turns one letter into another. A pair of words matches when the words have equal length and each letter of the first can be turned into the letter at the same position of the second through zero or more translations, and the task is to answer yes or no for every pair (statement).

The input is two integers m and n by the problemset pdf, m from 1 to 500 the number of translations and n from 1 to 50 the number of word pairs, then m lines of two distinct letters each, a can translate to b, every ordered pair appearing at most once, then n word-pair lines. All words are lowercase and 1 to 50 letters long. The output is one yes or no line per pair under the pdf's 1 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `I-secretchamber/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: the identical pair needs zero translations, out matches the through o to c to t to e, u to h and t to e, while can fails on a, which has no translation at all, and work fails on r reaching no l, giving yes no no yes yes.

input:

```
9 5
c t
i r
k p
o c
r o
t e
t f
u h
w p
we we
can the
work people
it of
out the
```

expected output:

```
yes
no
no
yes
yes
```

Recognition: the bounds collapse the universe. Up to 500 translations live on 26 letters, and 50 word pairs of at most 50 letters each are at most 2500 position checks, so the whole task is one 26^3 = 17576-operation closure and then constant-time lookups, trivially inside the 1 second limit.

The statement's shape, each letter of the first word can be turned into the letter at the same position of the second through zero or more translations, is per-position reachability: a length mismatch fails at once, and otherwise the only question is which of the 26 letters reach which. Problem I is the 2017 face of the algebraic reduction to a small universe family. Algebraic reduction to a small universe covers chapter 12 problem K and chapter 13 problem Z.

The tempting shortcut, checking one translation step per position or walking the 500-edge graph per position, is priced out both ways: one step is refuted by the sample itself, o reaches t only through o to c to t, and a per-position walk pays 2500 x 500 = 1.25 x 10^6 edge visits for what the 17576-operation closure answers once.

Page 9 of the solutions pdf closes the letter graph transitively: build the directed graph on the 26 letters, run Floyd-Warshall in 26 cubed, and every query position becomes one reachability lookup, with a length mismatch an immediate no. Tied with E at the set's highest solve count, 127 teams, this is the chapter's baseline, and all seven solvers are the same program in seven spellings.

The worked run: trace the model on sample 1. The nine printed translations close over the 26 letters: c reaches t, then e through t, i reaches r then o, o reaches c then t, then e and f through the t edges, r reaches o, u reaches h, w and k reach p, and every letter reaches itself in zero steps. The five pairs then read off position by position. we against we needs zero translations, yes. can against the fails at the first a, which has no translation at all, no. work against people fails on length, 4 against 6, no. it against of needs i to o, which runs i to r to o, and t to f directly, yes. out against the needs o to t through c, u to h, and t to e, all of them hold, yes, and the trace ends at the printed answer yes.

pair	lengths	positions	verdict
we we	2 = 2	w to w, e to e, zero steps	yes
can the	3 = 3	a reaches nothing	no
work people	4 and 6	mismatch	no
it of	2 = 2	i to r to o, t to f	yes
out the	3 = 3	o to c to t, u to h, t to e	yes

```
char reach[26][26];
memset(reach, 0, sizeof(reach));
for (int i = 0; i < 26; i++)
  reach[i][i] = 1;
for (int i = 0; i < m; i++)
  reach[lines[1 + i][0] - 'a'][lines[1 + i][2] - 'a'] = 1;
for (int k = 0; k < 26; k++)
  for (int i = 0; i < 26; i++)
    if (reach[i][k])
      for (int j = 0; j < 26; j++)
        if (reach[k][j])
          reach[i][j] = 1;
char *out = malloc((size_t)n * 5);
char *q = out;
for (int i = 0; i < n; i++) {
  char *w1 = lines[1 + m + i];
  char *sp = strchr(w1, ' ');
  *sp = 0;
  char *w2 = sp + 1;
  int ok = strlen(w1) == strlen(w2);
  for (int j = 0; ok && w1[j]; j++)
    if (!reach[w1[j] - 'a'][w2[j] - 'a'])
      ok = 0;
  if (ok) {
    memcpy(q, "yes\n", 4);
    q += 4;
  } else {
    memcpy(q, "no\n", 3);
    q += 3;
  }
}
```

listing 192: c: self-reach seeded, warshall over the byte matrix, per-pair scan (source: icpc/samples-c/src/Ch08/pI.c, lines 20-50)

```
  reach := make([][26]bool, 26)
  for i := 0; i < 26; i++ {
    reach[i][i] = true // zero translations always suffice
  }
  for i := 0; i < m; i++ {
    f := strings.Fields(lines[1+i])
    a, b := f[0][0]-'a', f[1][0]-'a'
    reach[a][b] = true
  }
  for k := 0; k < 26; k++ {
    for i := 0; i < 26; i++ {
      if !reach[i][k] {
        continue
      }
      for j := 0; j < 26; j++ {
        if reach[k][j] {
          reach[i][j] = true
        }
      }
    }
  }
  var out strings.Builder
  for q := 0; q < n; q++ {
    f := strings.Fields(lines[1+m+q])
    a, b := f[0], f[1]
    ok := len(a) == len(b)
    if ok {
      for i := 0; i < len(a); i++ {
        if !reach[a[i]-'a'][b[i]-'a'] {
          ok = false
          break
        }
      }
    }
    if ok {
      out.WriteString("yes\n")
    } else {
      out.WriteString("no\n")
    }
  }
  return out.String()
```

listing 193: go: arrays of 26 bools, zero translations always suffice (source: icpc/samples-go/ch08/pi.go, lines 19-59)

```
    boolean[][] reach = new boolean[26][26];
    for (int i = 0; i < 26; i++)
      reach[i][i] = true;
    for (int i = 0; i < m; i++)
      reach[lines.get(1 + i).charAt(0) - 'a'][lines.get(1 + i).charAt(2) - 'a'] = true;
    for (int k = 0; k < 26; k++)
      for (int i = 0; i < 26; i++)
        if (reach[i][k])
          for (int j = 0; j < 26; j++)
            if (reach[k][j])
              reach[i][j] = true;
    StringBuilder out = new StringBuilder(n * 5);
    for (int i = 0; i < n; i++) {
      String line = lines.get(1 + m + i);
      int sp = line.indexOf(' ');
      String w1 = line.substring(0, sp), w2 = line.substring(sp + 1);
      boolean ok = w1.length() == w2.length();
      for (int j = 0; ok && j < w1.length(); j++)
        if (!reach[w1.charAt(j) - 'a'][w2.charAt(j) - 'a'])
          ok = false;
      out.append(ok ? "yes\n" : "no\n");
    }
    return out.toString();
  }
```

listing 194: java: self-reach seeded, warshall over the boolean matrix, per-pair scan (source: icpc/samples-java/src/Ch09/PI.java, lines 20-43)

```
        var reach = new bool[26, 26];
        for (int i = 0; i < m; i++)
        {
            var p = lines[1 + i].Split(' ');
            reach[p[0][0] - 'a', p[1][0] - 'a'] = true;
        }
        for (int k = 0; k < 26; k++)
            for (int i = 0; i < 26; i++)
                for (int j = 0; j < 26; j++)
                    if (reach[i, k] && reach[k, j])
                        reach[i, j] = true;
        var sb = new System.Text.StringBuilder();
        for (int i = 0; i < q; i++)
        {
            var p = lines[1 + m + i].Split(' ');
            string x = p[0], y = p[1];
            bool ok = x.Length == y.Length;
            if (ok)
                for (int j = 0; j < x.Length; j++)
                    if (x[j] != y[j] && !reach[x[j] - 'a', y[j] - 'a'])
                    {
                        ok = false;
                        break;
                    }
            sb.Append(ok ? "yes" : "no").Append('\n');
        }
        return sb.ToString();
    }
```

listing 195: c#: the 26 by 26 bool matrix, same triple loop (source: icpc/samples/src/Ch08/PI.cs, lines 11-38)

```
  const reach = Array.from({ length: 26 }, () => new Uint8Array(26));
  for (let i = 0; i < 26; i++) reach[i][i] = 1; // zero translations suffice
  for (let i = 0; i < m; i++) {
    const f = lines[1 + i].split(/\s+/);
    reach[f[0].charCodeAt(0) - 97][f[1].charCodeAt(0) - 97] = 1;
  }
  for (let k = 0; k < 26; k++) {
    for (let i = 0; i < 26; i++) {
      if (!reach[i][k]) continue;
      for (let j = 0; j < 26; j++) {
        if (reach[k][j]) reach[i][j] = 1;
      }
    }
  }
  let out = "";
  for (let q = 0; q < n; q++) {
    const f = lines[1 + m + q].split(/\s+/);
    const a = f[0], b = f[1];
    let ok = a.length === b.length;
    if (ok) {
      for (let i = 0; i < a.length; i++) {
        if (!reach[a.charCodeAt(i) - 97][b.charCodeAt(i) - 97]) {
          ok = false;
          break;
        }
      }
    }
    out += ok ? "yes\n" : "no\n";
  }
  return out;
}
```

listing 196: javascript: typed-array rows, same closure (source: icpc/samples-js/src/ch08-pi-secretchamber.mjs, lines 12-42)

```
    reach = [[i == j for j in range(26)] for i in range(26)]
    for i in range(m):
        a, b = lines[1 + i].split()
        reach[ord(a) - 97][ord(b) - 97] = True
    for k in range(26):
        rk = reach[k]
        for ri in reach:
            if ri[k]:
                for j in range(26):
                    if rk[j]:
                        ri[j] = True
    out = []
    for q in range(n):
        a, b = lines[1 + m + q].split()
        ok = len(a) == len(b) and all(
            reach[ord(x) - 97][ord(y) - 97] for x, y in zip(a, b)
        )
        out.append("yes" if ok else "no")
    return "\n".join(out) + "\n"
```

listing 197: python: the per-pair check as one all() generator (source: icpc/samples-py/src/Ch08/pi.py, lines 10-28)

```
  local reach = {}
  for i = 0, 25 do
    reach[i] = {}
    reach[i][i] = true
  end
  for k = 1, m do
    local a, b = lines[1 + k]:match("(%a) (%a)")
    reach[a:byte() - 97][b:byte() - 97] = true
  end
  for k = 0, 25 do
    for i = 0, 25 do
      if reach[i][k] then
        for j = 0, 25 do
          if reach[k][j] then reach[i][j] = true end
        end
      end
    end
  end
  local out = {}
  for q = 1, n do
    local a, b = lines[1 + m + q]:match("(%S+) (%S+)")
    local ok = #a == #b
    if ok then
      for i = 1, #a do
        if not reach[a:byte(i) - 97][b:byte(i) - 97] then
          ok = false
          break
        end
      end
    end
    out[#out + 1] = ok and "yes" or "no"
  end
  return table.concat(out, "\n") .. "\n"
end
```

listing 198: lua: pattern-matched tokens, same closure and scan (source: icpc/samples-lua/ch08_pi.lua, lines 12-45)

The one honest detail is the diagonal, Go, JavaScript and Lua seed i reaching i before the closure, C prints the diagonal in its initialization loop, java seeds it in its own pass, C# lets the loop's k equal i case do it, Python builds the matrix with i equal j. The measured fixture is four translations `a b / b c / x y / y x` against four query pairs, the chain answers `ab` versus `cc` with yes, the cross-family pair and the length mismatch answer no, asserted as the exact four-line string `no / yes / no / no` in all seven suites, alongside the two-letter cycle answering `yes / yes`. Booleans only, no integer concerns.

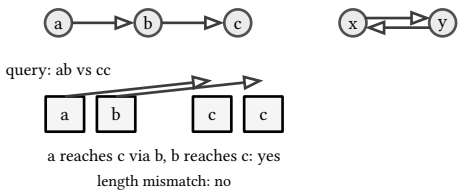


figure 102: the letter digraph, the a-b-c chain, the x-y cycle, one query pair aligned position by position

10.11. problem J, son of pipe stream

The hometown flubber network needs a routing. Node 1 is the flubber factory, node 2 the water source, node 3 the flubber department that consumes both, and the p bidirectional pipes carry both fluids at once. Flubber is sluggish with viscosity v, so a pipe whose water capacity is c liters per second moves only 1 liter per second of

pure flubber and scales linearly for mixtures: rates f and w in one pipe must obey v times f plus w at most c. The two fluids may never flow in opposite directions through the same pipe, and membranes at every node separate and reorganize incoming mixtures at will, so each fluid conserves at every node except its own source and the department. Maximize the value F to the a times W to the 1 minus a of the delivered rates, print the flubber and water rate of every pipe in input order, negative when the fluid moves from the pipe's second node to its first, then print the maximum value (statement).

The input is four values by the problemset pdf: n from 3 to 200 locations, p from n-1 up to n(n-1)/2 pipes, the viscosity v with 1 <= v <= 10 and the mixture weight a with 0.01 <= a <= 0.99, the two reals carrying at most 10 digits after the decimal point. Then p lines of j < k and the integer capacity c with 1 <= c <= 10. No two pipes join the same pair and the network is connected. The output is p pipe lines then the value, everything within an absolute error of 10^-4, any valid optimal routing accepted, under the pdf's 5 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `J-sonofpipestream/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: the cut into node 3 totals Z = 4 water-equivalents, Fmax is 4 and Wmax is 4, a times Z = 2.64 lands inside [0, 4], so flubber leaves node 1 at 2.64 water-equivalents, 0.88 liters per second at v = 3, against 1.36 of water, and the six pipe lines end in the value 1.02037965897.

input:

```
6 6 3.0 0.66
2 4 8
4 6 1
3 6 1
4 5 5
1 5 7
3 5 3
```

expected output:

```
0.000000000 1.360000000
0.000000000 1.000000000
0.000000000 -1.000000000
0.000000000 0.360000000
0.880000000 0.000000000
-0.880000000 -0.360000000
1.02037965897
```

Recognition: the bounds fit four max flows and nothing fancier. The network runs at most 200 nodes and 19900 pipes, one Dinic pass over 201 nodes and about 39800 arcs is 10^9 double operations at the O(V^2 E) worst case, four passes fit the 5 second limit with capacity doubles and an epsilon, and the integer capacities keep every water-equivalent flow the mix quotes exact.

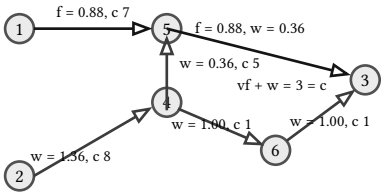
The statement's shape, two fluids sharing pipes under the linear law v f + w at most c with a product objective, is supply moving through shared capacity: bill one flubber liter as v capacity units and both fluids become one commodity whose total the max flow fixes and whose split between the sources the objective chooses. Problem J is the 2017 face of the flow modeling on a network family. Chapter 10 problem C is the other half of the flow modeling on a network pair.

The tempting shortcut, a genuine two-commodity flow that routes the fluids independently, is priced out before any bound is read: two-commodity integral flow has no max-flow integrality to lean on and the no-opposing-flows rule is not convex, while the water-equivalent billing makes one commodity that four Dinic runs answer exactly.

Pages 9 and 10 of the solutions pdf show the viscosity is a red herring once flubber is billed v capacity units per liter. Work in water-equivalent units, where a pipe holds c units total and a flubber liter counts as v of them,

and the integer capacities make every max flow integral. Three flows settle the optimum: Z, the max flow from a super source feeding both node 1 and node 2 into node 3, Fmax from node 1 alone, Wmax from node 2 alone. The optimal total is always the full Z, with flubber clamped into the interval from Z - Wmax to Fmax, F = **clamp(a times Z, Z - Wmax, Fmax) and water Z - F**, and the solutions' convexity argument shows the no-opposing-flows constraint never binds at that split. The routing is built from two combined routings: max flubber first with the residual carrying water gives f1, max water first with the residual carrying flubber gives f2, mixing alpha times f1 plus 1 minus alpha times f2 reaches the target totals, and one final max flubber flow from node 1 through a digraph whose arcs are the mixed usages splits each pipe into flubber plus leftover water. Book 10′s chapter 38, network flows ii, builds Dinic's algorithm from scratch. Four Dinic runs per input.

The worked run: trace the model on sample 1. The cut into node 3 is the printed pipes (3,5) with capacity 3 and (3,6) with capacity 1, so Z = 4 water-equivalents and nothing beats it. Flubber alone from node 1 pushes 3 through (1,5) and (3,5) plus 1 more through (5,4), (4,6) and (3,6), so Fmax = 4; water alone from node 2 pushes the mirror routes, 3 through (2,4), (4,5) and (3,5) plus 1 through (2,4), (4,6) and (3,6), so Wmax = 4. The clamp interval is [Z - Wmax, Fmax] = [0, 4], and a Z = 0.66 x 4 = 2.64 lands inside: F* = 2.64 water-equivalents of flubber, 2.64 / 3 = 0.88 liters per second at v = 3, against W = 4 - 2.64 = 1.36 of water. The printed routing carries exactly that, flubber 0.88 along (1,5) into (3,5), water 1.00 along (2,4), (4,6) into (3,6) and 0.36 along (2,4), (4,5) into (3,5), where v f + w = 3 x 0.88 + 0.36 = 3 saturates the capacity, and the value 0.88^0.66 x 1.36^0.34 reads 1.02037965897, and the trace ends at the printed answer `1.02037965897`.



Z = 4, F* = 2.64 water-equivalents, W = 1.36, value 1.02037965897

figure 103: sample 1′s six pipes, flubber thick along 1-5-3, water split 1.00 through 4-6 and 0.36 through 4-5

```
// Z: super source 0 feeds nodes 1 and 2
build(&g, n, np);
dinic_edge(&g, 0, 1, 1e9);
dinic_edge(&g, 0, 2, 1e9);
double Z = dinic_maxflow(&g, 0, 3);
dinic_free(&g);
// flubber-first routing: Fmax flubber, then water on the residual
build(&g, n, np);
double Fmax = dinic_maxflow(&g, 1, 3);
dinic_maxflow(&g, 2, 3);
for (int i = 0; i < np; i++)
  u1[i] = usage(&g, i);
dinic_free(&g);
// water-first routing: Wmax water, then flubber on the residual
build(&g, n, np);
double Wmax = dinic_maxflow(&g, 2, 3);
dinic_maxflow(&g, 1, 3);
for (int i = 0; i < np; i++)
  u2[i] = usage(&g, i);
dinic_free(&g);
double lo = Z - Wmax;
if (lo > Fmax)
  lo = Fmax;
double Fwe = a * Z;
if (Fwe < lo)
  Fwe = lo;
if (Fwe > Fmax)
  Fwe = Fmax;
double alpha = Fmax > lo + 1e-12 ? (Fwe - lo) / (Fmax - lo) : 0.0;
for (int i = 0; i < np; i++)
  U[i] = alpha * u1[i] + (1 - alpha) * u2[i];
```

listing 199: c: Z, the two orderings f1 and f2, the clamp, the convex mix (source: icpc/samples-c/src/Ch08/pJ.c, lines 44-74)

```
  // Z from a super source feeding both wells
  dz := newDinic(n + 1)
  for k := 0; k < p; k++ {
    dz.addUndirected(pa[k], pb[k], pc[k])
  }
  dz.addEdge(n, src1, infCap)
  dz.addEdge(n, src2, infCap)
  Z := dz.maxflow(n, sink)

  dF, _ := fresh()
  Fmax := dF.maxflow(src1, sink)
  dW, _ := fresh()
  Wmax := dW.maxflow(src2, sink)

  // f1 pushes flubber first, then water through the residual;
  // f2 does the reverse
  df1, x1 := fresh()
  df1.maxflow(src1, sink)
  df1.maxflow(src2, sink)
  net1 := netFlow(df1, x1)
  df2, x2 := fresh()
  df2.maxflow(src2, sink)
  df2.maxflow(src1, sink)
  net2 := netFlow(df2, x2)

  lo := Z - Wmax
  Fstar := alpha * Z
  if Fstar < lo {
    Fstar = lo
  }
  if Fstar > Fmax {
    Fstar = Fmax
  }
  mix := 1.0
  if Fmax-lo > 1e-9 {
    mix = (Fstar - lo) / (Fmax - lo)
  }
  fstar := make([]float64, p)
  for k := range fstar {
    fstar[k] = mix*net1[k] + (1-mix)*net2[k]
  }
```

listing 200: go: paired-capacity undirected edges, net usage read off the residual (source: icpc/samples-go/ch08/pj.go, lines 61-101)

```
    // Z: super source 0 feeds nodes 1 and 2
    Dinic g = build(n, np, pj, pk, pc);
    g.edge(0, 1, 1e9);
    g.edge(0, 2, 1e9);
    double Z = g.maxflow(0, 3);
    // flubber-first routing: Fmax flubber, then water on the residual
    g = build(n, np, pj, pk, pc);
    double Fmax = g.maxflow(1, 3);
    g.maxflow(2, 3);
    double[] u1 = new double[np];
    for (int i = 0; i < np; i++)
      u1[i] = usage(g, i);
    // water-first routing: Wmax water, then flubber on the residual
    g = build(n, np, pj, pk, pc);
    double Wmax = g.maxflow(2, 3);
    g.maxflow(1, 3);
    double[] u2 = new double[np];
    for (int i = 0; i < np; i++)
      u2[i] = usage(g, i);
    double lo = Z - Wmax;
    if (lo > Fmax)
      lo = Fmax;
    double Few = a * Z;
    if (Few < lo)
      Few = lo;
    if (Few > Fmax)
      Few = Fmax;
    double alpha = Fmax > lo + 1e-12 ? (Few - lo) / (Fmax - lo) : 0.0;
    double[] U = new double[np];
    for (int i = 0; i < np; i++)
      U[i] = alpha * u1[i] + (1 - alpha) * u2[i];
```

listing 201: java: Z, the two orderings f1 and f2, the clamp, the convex mix, usage read off retained edge indices (source: icpc/samples-java/src/Ch09/PJ.java, lines 61-91)

```
        var g1 = Pipes(n, p, eu, ev, ec);
        double fMax = Dinic(n, 0, 2, g1, double.MaxValue).flow;
        var f1F = PipeNet(n, p, eu, ev, g1);
        if (Dinic(n, 1, 2, g1, z - fMax).flow < z - fMax - 1e-6)
            throw new InvalidOperationException("f1 water phase fell short");
        var f1W = Sub(PipeNet(n, p, eu, ev, g1), f1F);

        var g2 = Pipes(n, p, eu, ev, ec);
        double wMax = Dinic(n, 1, 2, g2, double.MaxValue).flow;
        var f2W = PipeNet(n, p, eu, ev, g2);
        if (Dinic(n, 0, 2, g2, z - wMax).flow < z - wMax - 1e-6)
            throw new InvalidOperationException("f2 flubber phase fell short");
        var f2F = Sub(PipeNet(n, p, eu, ev, g2), f2W);

        double lo = z - wMax, hi = fMax;
        double fWe = Math.Clamp(alpha * z, lo, hi);
        double wWe = z - fWe;
        double mix = hi - lo > Eps ? (fWe - lo) / (hi - lo) : 0;

        var use = new double[p];
        for (int i = 0; i < p; i++)
            use[i] = mix * (f1F[i] + f1W[i]) + (1 - mix) * (f2F[i] + f2W[i]);

        // push flubber from node 0 inside the mixed usage digraph, water
        // fills the rest
        var gu = new double[n, n];
        for (int i = 0; i < p; i++)
            if (use[i] > Eps)
                gu[eu[i], ev[i]] = use[i];
            else if (use[i] < -Eps)
                gu[ev[i], eu[i]] = -use[i];
        if (Dinic(n, 0, 2, gu, fWe).flow < fWe - 1e-6)
            throw new InvalidOperationException("flubber repush fell short");
```

listing 202: c#: the same four runs over a dense capacity matrix, with conservation asserts (source: icpc/samples/src/Ch08/PJ.cs, lines 41-73)

```
  // Z from a super source feeding both wells
  const dz = new Dinic(n + 1);
  for (let k = 0; k < p; k++) dz.addUndirected(pa[k], pb[k], pc[k]);
  dz.addEdge(n, src1, infCap);
  dz.addEdge(n, src2, infCap);
  const Z = dz.maxflow(n, sink);

  const [dF] = fresh();
  const Fmax = dF.maxflow(src1, sink);
  const [dW] = fresh();
  const Wmax = dW.maxflow(src2, sink);

  // f1 pushes flubber first, then water through the residual; f2 the
  // reverse
  const [df1, x1] = fresh();
  df1.maxflow(src1, sink);
  df1.maxflow(src2, sink);
  const net1 = netFlow(df1, x1);
  const [df2, x2] = fresh();
  df2.maxflow(src2, sink);
  df2.maxflow(src1, sink);
  const net2 = netFlow(df2, x2);

  const lo = Z - Wmax;
  let Fstar = alpha * Z;
  if (Fstar < lo) Fstar = lo;
  if (Fstar > Fmax) Fstar = Fmax;
  let mix = 1;
  if (Fmax - lo > 1e-9) mix = (Fstar - lo) / (Fmax - lo);
  const fstar = new Float64Array(p);
  for (let k = 0; k < p; k++) fstar[k] = mix * net1[k] + (1 - mix) * net2[k];

  // split the mixed usage into flubber plus leftover water
  const dd = new Dinic(n + 1);
  const arc = new Int32Array(p);
  for (let k = 0; k < p; k++) {
    if (fstar[k] > dinicEps) arc[k] = dd.addEdge(pa[k], pb[k], fstar[k]);
    else if (fstar[k] < -dinicEps) arc[k] = dd.addEdge(pb[k], pa[k], -fstar[k]);
    else arc[k] = -1;
  }
  dd.addEdge(n, src1, Fstar);
  dd.maxflow(n, sink);
```

listing 203: javascript: the same shape over the ch08-dinic module (source: icpc/samples-js/src/ch08-pj-sonofpipestream.mjs, lines 46-87)

```
    dz = _Dinic(n + 1)
    for k in range(p):
        dz.add_undirected(pa[k], pb[k], pc[k])
    dz.add_edge(n, 0, INF_CAP)
    dz.add_edge(n, 1, INF_CAP)
    z = dz.maxflow(n, 2)

    df1, x1 = fresh()
    df1.maxflow(0, 2)
    df1.maxflow(1, 2)
    net1 = net_flow(df1, x1)
    df2, x2 = fresh()
    df2.maxflow(1, 2)
    df2.maxflow(0, 2)
    net2 = net_flow(df2, x2)
    d_f, _ = fresh()
    fmax = d_f.maxflow(0, 2)
    d_w, _ = fresh()
    wmax = d_w.maxflow(1, 2)

    lo = z - wmax
    fstar = min(max(alpha * z, lo), fmax)
    mix = 1.0 if fmax - lo ≤ 1e-9 else (fstar - lo) / (fmax - lo)
    fstar_net = [mix * a + (1 - mix) * b for a, b in zip(net1, net2)]

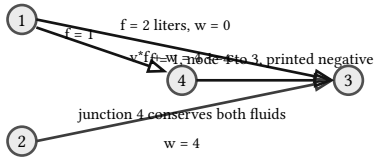
    dd = _Dinic(n + 1)
    arc = [0] * p
    for k in range(p):
        if fstar_net[k] > EPS:
            arc[k] = dd.add_edge(pa[k], pb[k], fstar_net[k])
        elif fstar_net[k] < -EPS:
            arc[k] = dd.add_edge(pb[k], pa[k], -fstar_net[k])
        else:
            arc[k] = -1
    dd.add_edge(n, 0, fstar)
    dd.maxflow(n, 2)
```

listing 204: python: the two orderings, the clamp, the repush digraph (source: icpc/samples-py/src/Ch08/pj.py, lines 108-143)

```
  local dz = dn.new(n + 1)
  for k = 1, p do dn.add_undirected(dz, pa[k], pb[k], pc[k]) end
  dn.add_edge(dz, n + 1, src1, inf_cap)
  dn.add_edge(dz, n + 1, src2, inf_cap)
  local Z = dn.maxflow(dz, n + 1, sink)
  local dF = fresh()
  local Fmax = dn.maxflow(dF, src1, sink)
  local dW = fresh()
  local Wmax = dn.maxflow(dW, src2, sink)
  local df1, x1 = fresh()
  dn.maxflow(df1, src1, sink)
  dn.maxflow(df1, src2, sink)
  local net1 = net_flow(df1, x1)
  local df2, x2 = fresh()
  dn.maxflow(df2, src2, sink)
  dn.maxflow(df2, src1, sink)
  local net2 = net_flow(df2, x2)
  local lo = Z - Wmax
  local fstar = alpha * Z
  if fstar < lo then fstar = lo end
  if fstar > Fmax then fstar = Fmax end
  local mix = 1.0
  if Fmax - lo > EPS then mix = (fstar - lo) / (Fmax - lo) end
  local fs = {}
  for k = 1, p do fs[k] = mix * net1[k] + (1.0 - mix) * net2[k] end
  local dd = dn.new(n + 1)
  local arc = {}
  for k = 1, p do
    if fs[k] > EPS then
      arc[k] = dn.add_edge(dd, pa[k], pb[k], fs[k])
    elseif fs[k] < -EPS then
      arc[k] = dn.add_edge(dd, pb[k], pa[k], -fs[k])
    else
      arc[k] = 0
    end
  end
  dn.add_edge(dd, n + 1, src1, fstar)
  dn.maxflow(dd, n + 1, sink)
```

listing 205: lua: the year's ch08_dinic module under the same four runs (source: icpc/samples-lua/ch08_pj.lua, lines 39-76)

All seven carry the same two combined routings, flubber-first and water-first, mix them to the clamped target, and push max flubber through a digraph whose arcs are the mixed usages, water per pipe is what remains. The carrier differs, C, java and the Go, JavaScript, Lua and Python helpers keep paired arcs in edge lists or adjacency arrays with capacities as doubles and an epsilon, java reading each pipe's usage off its retained edge indices, C# runs a dense n by n matrix and throws when a phase falls short, which is how it self-checks conservation. The measured fixture is the four-pipe network `1 3 4 / 2 3 4 / 1 4 2 / 3 4 2` with v equal to 2 and a equal to 0.5, Z = 10, the interval collapses to [6, 6], flubber saturates its cut at 3 liters, and every suite asserts all four pipe lines plus the value `3.46410161514` exactly, alongside the interior-alpha junction case ending in `2.82842712475`. That second fixture's bytes are not shared by every tree: the c and java inputs print the junction pipe as `3 4 8` and expect `-2 -4` on its line, the python and go siblings print `4 3 8` and expect `2 4`, two distinct self-consistent inputs, and java mirrors the c bytes. Capacities are integers but the mix is decimal, so the max flows run in doubles with epsilon comparisons throughout, no bigint.



Z = 10 water-equivalents, flubber clamped to 6, value 3.46410161514

figure 104: the four-node network, flubber liters thick, water thin, the node 4 junction, the budget on pipe (1,3)

10.12. problem K, tarot sham boast

The rival plays n rounds of rock paper scissors, choosing uniformly at random and independently every round. Each of s fortune-tellers predicted one string of the rival's choices, all predictions the same length, and the task is to rank the predictions by the probability that each appears somewhere in the match as a contiguous block of the rival's sequence, most likely first, ties kept in input order (statement).

The input is two integers n and s by the problemset pdf, n from 1 to 10^6 the number of rounds and s from 1 to 10 the number of predictions, then s lines, each a string over R, P and S, all of one common length from 1 up to the smaller of n and 10^5. The output is the s predictions in sorted order under the pdf's 2 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `K-tarotshamboast/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: with n = 3 and length 2 the truncation threshold 2 l - n is 1, PS carries no border and leads, then PP, RR and SS tie on border length 1 and keep their input order.

input:

```
3 4
PP
RR
PS
SS
```

expected output:

```
PS
PP
RR
SS
```

Recognition: the bounds keep it linear. Predictions are at most 10 strings of at most 10^5 letters over n = 10^6 rounds, so each failure function costs one O(l) pass and the stable sort costs s log s, about 33 comparisons, a few million character touches inside the 2 second limit.

The statement's shape, rank predictions by the probability each appears as a contiguous block of the random sequence, hides a collapse: for one common length the first-order inclusion-exclusion terms are identical across strings, so only self-overlap separates them, and self-overlap is exactly the border sequence read off the KMP failure chain. Problem K is the 2017 face of the string borders, rotations, and automata family. String borders, rotations, and automata connect chapter 12 problem G and chapter 14 problem F.

The tempting shortcut, computing the probabilities exactly, is priced out by the state space: the rival's sequence ranges over 3^1000000 outcomes and exact inclusion-exclusion over the n - l + 1 placements walks 2^999999 subset terms, while the truncated border sequences rank the same order in one failure pass per string.

Pages 10 and 11 of the solutions pdf give the ranking rule. The first-order inclusion-exclusion terms are identical for all strings of one length, so the differences come entirely from self-overlaps: the rank key of a prediction X is its border sequence, the border lengths of X in decreasing order, equivalently the shifts at which X overlaps itself, read off the KMP failure chain, truncated at the threshold 2 l - n because two occurrences cannot both fit in

n rounds below it, and a uniform trailing zero entry never affects the order, so it is dropped. A lexicographically smaller border sequence means a more likely prediction, so the sort is ascending and stable, O(l + s log s) overall. Book 10′s chapter 35, combinatorics, carries the inclusion-exclusion machinery the ranking's first-order terms are an instance of. Both official samples fall out of the rule, the second one ranking the border-free PRSPS first and the 4-3-2-1 chain of SSSSS last. The correctness proof lives in a separate document the solutions cite.

The worked run: trace the model on sample 1. The common length is l = 2 against n = 3 rounds, so the truncation threshold is 2l - n = 1 and borders at or above 1 count. The failure pass walks each printed string: PP's prefix P equals its suffix P, border length 1; RR the same, 1; PS's P and S differ, no border at all; SS the same as PP, 1. Truncated, the rank keys read PS [], PP [1], RR [1] and SS [1], ascending lexicographic order puts the empty sequence first, PS, and the three equal [1] sequences keep their input order PP, RR, SS, and the trace ends at the printed answer `SS`.

prediction	border walk	key at 2l - n = 1	output rank
PP	P equals P, length 1	[1]	2
RR	R equals R, length 1	[1]	3
PS	P against S, none	[]	1
SS	S equals S, length 1	[1]	4

```
  for (int i = 0; i < s; i++) {
    strcpy(pred[i], lines[1 + i]);
    int l = (int)strlen(pred[i]);
    if (l > maxlen)
      maxlen = l;
    // KMP failure function
    int *f = kmpf;
    f[0] = 0;
    int k = 0;
    for (int c = 1; c < l; c++) {
      while (k && pred[i][c] ≠ pred[i][k])
        k = f[k - 1];
      if (pred[i][c] == pred[i][k])
        k++;
      f[c] = k;
    }
    long long thr = 2LL * l - n;
    int cnt = 0;
    for (int b = f[l - 1]; b > 0; b = f[b - 1])
      if ((long long)b ≥ thr)
        bord[i][cnt++] = b;
    blen[i] = cnt;
  }
```

listing 206: c: the failure function per prediction, borders kept at or above 2l-n (source: icpc/samples-c/src/Ch08/pK.c, lines 43-65)

```
func borderSeq(x string, floor int) []int {
  fail := make([]int, len(x))
  for i := 1; i < len(x); i++ {
    j := fail[i-1]
    for j > 0 && x[i] ≠ x[j] {
      j = fail[j-1]
    }
    if x[i] == x[j] {
      j++
    }
    fail[i] = j
  }
  var borders []int
  for b := fail[len(x)-1]; b > 0; b = fail[b-1] {
    if b ≥ floor {
      borders = append(borders, b)
    }
  }
  return borders
}

func lexLess(a, b []int) bool {
  for i := 0; i < len(a) && i < len(b); i++ {
    if a[i] ≠ b[i] {
      return a[i] < b[i]
    }
  }
  return len(a) < len(b)
}
```

listing 207: go: borderSeq off the failure chain, sort.SliceStable for ties (source: icpc/samples-go/ch08/pk.go, lines 46-74)

```
    for (int i = 0; i < s; i++) {
      String w = pred[i];
      int l = w.length();
      int[] f = new int[l];
      f[0] = 0;
      int k = 0;
      for (int c = 1; c < l; c++) {
        while (k > 0 && w.charAt(c) ≠ w.charAt(k))
          k = f[k - 1];
        if (w.charAt(c) == w.charAt(k))
          k++;
        f[c] = k;
      }
      long thr = 2L * l - n;
      int cnt = 0;
      for (int b = f[l - 1]; b > 0; b = f[b - 1])
        if (b ≥ thr)
          borders[i][cnt++] = b;
      blen[i] = cnt;
    }
    Integer[] idx = new Integer[s];
    for (int i = 0; i < s; i++)
      idx[i] = i;
    Arrays.sort(idx, (x, y) -> {
      int a = 0, b = 0;
      while (a < blen[x] && b < blen[y]) {
        if (borders[x][a] ≠ borders[y][b])
          return Integer.compare(borders[x][a], borders[y][b]);
        a++;
        b++;
      }
      if (a < blen[x])
        return 1; // longer, equal-prefixed borderser list sorts later
      if (b < blen[y])
        return -1;
      return Integer.compare(x, y); // stable for equal sequences
    });
```

listing 208: java: the failure function per prediction, borders kept at or above 2l-n, the index tiebreak spelled out in the boxed sort (source: icpc/samples-java/src/Ch09/PK.java, lines 30-66)

```
    private static List<long> Borders(string s, long n)
    {
        long l = s.Length;
        long cut = 2 * l - n; // overlaps that cannot fit two occurrences
        var f = new int[s.Length + 1];
        f[0] = 0;
        if (s.Length > 1)
            f[1] = 0;
        for (int i = 2; i ≤ s.Length; i++)
        {
            int k = f[i - 1];
            while (k > 0 && s[i - 1] ≠ s[k])
                k = f[k];
            if (s[i - 1] == s[k])
                k++;
            f[i] = k;
        }
        var res = new List<long>();
        for (int b = f[s.Length]; b > 0; b = f[b])
            if (b ≥ cut)
                res.Add(b);
        return res;
    }
```

listing 209: c#: borders as a list, compare by element then length (source: icpc/samples/src/Ch08/PK.cs, lines 40-62)

```
function borderSeq(x, floor) {
  const fail = new Int32Array(x.length);
  for (let i = 1; i < x.length; i++) {
    let j = fail[i - 1];
    while (j > 0 && x[i] !== x[j]) j = fail[j - 1];
    if (x[i] === x[j]) j++;
    fail[i] = j;
  }
  const borders = [];
  for (let b = fail[x.length - 1]; b > 0; b = fail[b - 1]) {
    if (b ≥ floor) borders.push(b);
  }
  return borders;
}

const lexLess = (a, b) => {
  for (let i = 0; i < a.length && i < b.length; i++) {
    if (a[i] !== b[i]) return a[i] < b[i];
  }
  return a.length < b.length;
};
```

listing 210: javascript: the same border walk, the stable array sort (source: icpc/samples-js/src/ch08-pk-tarotshamboast.mjs, lines 13-33)

```
    keyed = []
    for idx, pred in enumerate(preds):
        fail = [0] * length
        j = 0
        for i in range(1, length):
            while j and pred[i] != pred[j]:
                j = fail[j - 1]
            if pred[i] == pred[j]:
                j += 1
            fail[i] = j
        borders = []
        b = fail[-1]
        while b > 0:
            if b ≥ floor:
                borders.append(b)
            b = fail[b - 1]
        keyed.append((borders, idx))
    keyed.sort()
    return "".join(f"{preds[i]}\n" for _, i in keyed)
```

listing 211: python: borders and index as one sorted tuple (source: icpc/samples-py/src/Ch08/pk.py, lines 15-33)

```
local function border_seq(x, floor)
  local n = #x
  local fail = { 0 }
  for i = 2, n do
    local j = fail[i - 1] or 0
    while j > 0 and x:byte(i) ~= x:byte(j + 1) do j = fail[j] end
    if x:byte(i) == x:byte(j + 1) then j = j + 1 end
    fail[i] = j
  end
  local out = {}
  local b = fail[n] or 0
  while b > 0 do
    if b ≥ floor then out[#out + 1] = b end
    b = fail[b] or 0
  end
  return out
end

local function lex_less(a, b)
  for i = 1, math.min(#a, #b) do
    if a[i] ~= b[i] then return a[i] < b[i] end
  end
  return #a < #b
end
```

listing 212: lua: table.sort is not stable, the index tiebreak is spelled out (source: icpc/samples-lua/ch08_pk.lua, lines 10-33)

The interesting per-language question is who guarantees the tie order. C, java and Lua spell it, C's comparator falls back to the input index, java's boxed `Integer` sort closes on the input index, and Lua's falls back to a.idx, Go uses `sort.SliceStable` and JavaScript the stable array sort, Python sorts (borders, index) tuples so the tiebreak is positional, C# compares border lists with `List.Sort` and no index fallback. The measured fixture is n equal to 10 with `RRR / RPR / PPP / PPR / RPP`, border-free PPR and RPP first in input order, then RPR, then RRR before PPP, asserted as the exact five-line ranking in all seven suites, alongside the truncation case n equal to 4 where the 2l minus n threshold erases the length-1 borders and `RPR / RRP / RRR` falls out. Border lengths fit any integer type, no concerns.

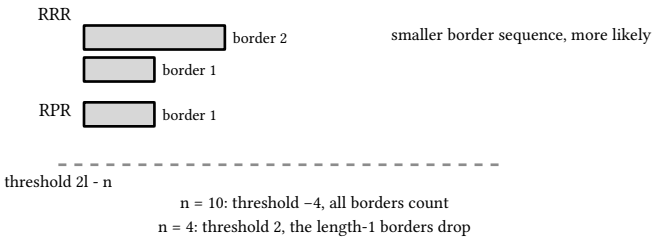


figure 105: RRR's failure chain 2-1-0 against RPR's 1-0, the truncation threshold drawn at 2l-n

10.13. problem L, visual python++

Visual Python++ draws every statement block as a rectangle of characters with a top-left corner and a bottom-right corner. In a syntactically correct program every two blocks are either nested, one inside the other, or disjoint, and in both cases their borders may not overlap. Programmers only mark the top-left and bottom-right corners, the parser must match them up, and the task is exactly that parser step: n top-left corners and n bottom-right corners arrive unmatched, and the output is the matching as a permutation, or the literal syntax error when no valid nesting exists (statement).

The input is one integer n, 1 to 10^5 by the problemset pdf, then n lines of two integers r and c, 1 to 10^9, the top-left corners in input order, then n lines of bottom-right corners the same way, all 2n locations distinct. The output is n lines, line i naming the bottom-right corner matched to top-left corner i, corners of each kind numbered from 1 in input order, any valid matching accepted, or the two words syntax error, under the pdf's 5 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `L-visual/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: the (9,8) top-left closes first against the (14,17) bottom-right, so line 2 answers 1 before line 1 answers 2, and the rows 9 to 14 rectangle nests inside rows 4 to 19.

input:

```
2
4 7
9 8
14 17
19 18
```

expected output:

```
2
1
```

Official sample 3, same source, the one that needs both sweeps: the first sweep pairs both corners without complaint, but the rectangles it forms, rows 9 to 19 against rows 4 to 14, cross instead of nesting, and the second sweep rejects the program.

input:

```
2
4 8
9 7
14 18
19 17
```

expected output:

```
syntax error
```

Recognition: the bounds demand one sorted pass. Corners reach 10^5 of each kind, so 2 x 10^5 events through an ordered structure is about 2 x 10^6 set operations at n log n, comfortably inside the 5 second limit, while the naive pairing of every top-left with every bottom-right is n^2 = 10^10 candidates before any nesting check runs.

The statement's shape, every two blocks nested or disjoint with borders never overlapping, hands over the discipline: sweeping columns left to right, the unmatched top-left corners form a set ordered by row, and a bottom-right corner at row r2 matches the largest open row at most r2, exactly the predecessor question an ordered active set answers. Problem L is the 2017 face of the sweep line with an ordered active set family. Chapter 11 problem F is the other member of the sweep line with an ordered active set family.

The tempting shortcut, pairing each top-left with the bottom-right of nearest column and testing the n^2 leftovers, is priced out at the same 10^10 with no pruning, and column proximity alone picks wrong whenever an inner rectangle closes before an outer one, which the sample's own nesting does.

Page 11 of the solutions pdf matches the corners by sweeping columns left to right over the 2n events with an active set of unmatched top-left corners ordered by row. On a top-left corner, a duplicate row already in the set fails. On a bottom-right corner at row r2, match the active top-left with the largest row at most r2, none available fails. That first sweep is O(n log n) with an ordered structure, and a second sweep over the now-known rectangles checks that adjacent active pairs never let borders touch or cross. The matching is unique when it exists, so byte comparison verifies.

The worked run: trace the model on sample 1. The printed corners give four events, top-left (4,7), top-left (9,8), bottom-right (14,17) and bottom-right (19,18), and the sweep reads them in column order 7, 8, 17, 18 with no column ties to break. Column 7 opens row 4, column 8 opens row 9, the active set holds the two distinct rows 4 and 9. Column 17 closes at row 14: the largest active row at most 14 is 9, so top-left 2 pairs with bottom-right 1 and line 2 answers

1. Column 18 closes at row 19: the only open row left is 4, top-left

1 pairs with bottom-right 2 and line 1 answers 2. The second sweep checks the two rectangles it just formed, rows 4 to 19 against rows 9 to 14 and columns 7 to 18 against 8 to 17, the inner sits strictly inside with no borders touching, the output reads 2 then 1 down the lines, and the trace ends at the printed answer 1.

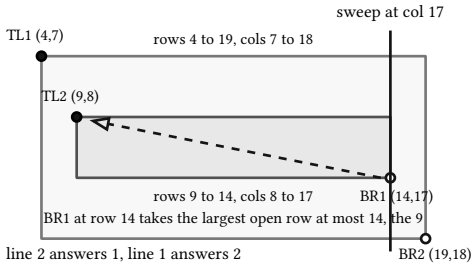


figure 106: the sample 1 rectangles nested, the sweep line on column 17 closing the inner block first

```
  // sweep 1: match by column, largest open top row ≤ the closing row
  memcpy(tlrow, tlr, sizeof(int) * (size_t)n);
  qsort(tlrow, (size_t)n, sizeof(int), cmp_int);
  int un = 0;
  for (int i = 0; i < n; i++)
    if (i == 0 || tlrow[i] != tlrow[i - 1])
      tlrow[un++] = tlrow[i];
  Fenwick fw;
  fw_init(&fw, un);
  for (int i = 0; i < un; i++)
    tlidx[i] = -1;
  int ne = 0;
  for (int i = 0; i < n; i++) {
    evs[ne].col = tlc[i];
    evs[ne].type = 0;
    evs[ne].idx = i;
    ne++;
    evs[ne].col = brc[i];
    evs[ne].type = 1;
    evs[ne].idx = i;
    ne++;
  }
  int cmp_ev(const void *a, const void *b);
  qsort(evs, (size_t)ne, sizeof(Ev), cmp_ev);
  for (int e = 0; e < ne && !bad; e++) {
    if (evs[e].type == 0) {
      int i = evs[e].idx;
      int pos = bisect(tlrow, un, tlr[i]) + 1; // 1-based fenwick rank
      if (fw_pred(&fw, pos) == pos) {
        bad = 1; // same row already open
        break;
      }
      fw_add(&fw, pos, 1);
      tlidx[pos - 1] = i;
    } else {
      int i = evs[e].idx;
      int cnt = bisect(tlrow, un, brr[i] + 1); // tl rows ≤ brr
      int pos = fw_pred(&fw, cnt);
      if (pos < 1) {
        bad = 1;
        break;
      }
      ans[tlidx[pos - 1]] = i;
      fw_add(&fw, pos, -1);
      tlidx[pos - 1] = -1;
    }
  }
```

listing 213: c: sweep one over compressed row ranks, fenwick predecessor picks the match (source: icpc/samples-c/src/Ch08/pL.c, lines 96-142)

```
  events := make([]event, 0, 2*n)
  for i := 0; i < n; i++ {
    events = append(events, event{tlCol[i], i, true})
  }
  for i := 0; i < n; i++ {
    events = append(events, event{brCol[i], i, false})
  }
  // top-left corners open before bottom-right corners close at the
  // same column: this is the tie order the official sweep uses
  // (judge-verified on L secrets 004 and 016)
  sort.Slice(events, func(a, b int) bool {
    if events[a].col != events[b].col {
      return events[a].col < events[b].col
    }
    return events[a].open && !events[b].open
  })
  rows := append(append([]int(nil), tlRow...), brRow...)
  rankOf, sortedRows := compressRows(rows)
  fen := newFenwick(len(rows))
  match := make([]int, n)
  openAt := make(map[int]int, n)
  for _, e := range events {
    if e.open {
      if _, dup := openAt[tlRow[e.idx]]; dup {
        return "syntax error\n"
      }
      openAt[tlRow[e.idx]] = e.idx
      fen.add(rankOf[tlRow[e.idx]], 1)
    } else {
      p := fen.pred(rankOf[brRow[e.idx]])
      if p < 0 {
        return "syntax error\n"
      }
      fen.add(p, -1)
      match[openAt[sortedRows[p]]] = e.idx
      delete(openAt, sortedRows[p])
    }
  }
  if len(openAt) != 0 {
    return "syntax error\n"
  }
```

listing 214: go: events sorted opens-first on equal columns, the helper's pred answers matches (source: icpc/samples-go/ch08/pl.go, lines 41-81)

```
    Arrays.sort(evs, 0, ne, (x, y) -> x.col != y.col ? Integer.compare(x.col, y.col)
        : Integer.compare(x.type, y.type));
    for (int e = 0; e < ne && !bad; e++) {
      if (evs[e].type == 0) {
        int i = evs[e].idx;
        int pos = bisect(tlrow, un, tlr[i]) + 1; // 1-based fenwick rank
        if (fw.pred(pos) == pos) {
          bad = true; // same row already open
          break;
        }
        fw.add(pos, 1);
        tlidx[pos - 1] = i;
      } else {
        int i = evs[e].idx;
        int cnt = bisect(tlrow, un, brr[i] + 1); // tl rows ≤ brr
        int pos = fw.pred(cnt);
        if (pos < 1) {
          bad = true;
          break;
        }
        ans[tlidx[pos - 1]] = i;
        fw.add(pos, -1);
        tlidx[pos - 1] = -1;
      }
    }
```

listing 215: java: events sorted opens-first on equal columns over boxed indices, matches answered by the Fenwick helper's pred (source: icpc/samples-java/src/Ch09/PL.java, lines 126-150)

```
        var openAt = new Dictionary<long, int>(n);
        foreach (var (_, idx, open) in events)
        {
            if (open)
            {
                if (openAt.ContainsKey(tlRow[idx]))
                    return "syntax error\n";
                openAt[tlRow[idx]] = idx;
                fen.Add(Rank(tlRow[idx]), 1);
            }
            else
            {
                int p = fen.Pred(Rank(brRow[idx]));
                if (p < 0)
                    return "syntax error\n";
                fen.Add(p, -1);
                match[openAt[rows[p]]] = idx;
                openAt.Remove(rows[p]);
            }
        }
        if (openAt.Count != 0)
            return "syntax error\n";
```

listing 216: c#: the open set as a dictionary, matching through the nested fenwick's Pred (source: icpc/samples/src/Ch08/PL.cs, lines 72-93)

```
  const events = [];
  for (let i = 0; i < n; i++) events.push({ col: tlCol[i], idx: i, open: true });
  for (let i = 0; i < n; i++) events.push({ col: brCol[i], idx: i, open: false });
  // top-left corners open before bottom-right corners close at the
  // same column: the tie order the official sweep uses
  events.sort((a, b) => {
    if (a.col !== b.col) return a.col - b.col;
    if (a.open === b.open) return 0;
    return a.open ? -1 : 1;
  });
  const rows = [...tlRow, ...brRow];
  const sortedRows = [...rows].sort((a, b) => a - b);
  const rankOf = new Map();
  for (let i = 0; i < sortedRows.length; i++) {
    if (i === 0 || sortedRows[i] !== sortedRows[i - 1])
      rankOf.set(sortedRows[i], i);
  }
  const fen = new Fenwick(rows.length);
  const match = new Int32Array(n);
  const openAt = new Map();
  for (const e of events) {
    if (e.open) {
      if (openAt.has(tlRow[e.idx])) return "syntax error\n";
      openAt.set(tlRow[e.idx], e.idx);
      fen.add(rankOf.get(tlRow[e.idx]), 1);
    } else {
      const p = fen.pred(rankOf.get(brRow[e.idx]));
      if (p < 0) return "syntax error\n";
      fen.add(p, -1);
      match[openAt.get(sortedRows[p])] = e.idx;
      openAt.delete(sortedRows[p]);
    }
  }
  if (openAt.size !== 0) return "syntax error\n";
```

listing 217: javascript: rank map over both corner kinds, same sweep (source: icpc/samples-js/src/ch08-pl-visual.mjs, lines 30-63)

```
    # top-left corners open before bottom-right corners close at the
    # same column (judge-verified on L secrets 004 and 016)
    opens = [(tl_col[i], 0, i) for i in range(n)]
    closes = [(br_col[i], 1, i) for i in range(n)]
    events = sorted(opens + closes)
    rank_of = {v: i for i, v in enumerate(sorted(tl_row + br_row))}
    by_rank = {i: v for v, i in rank_of.items()}
    fen = _Fenwick(2 * n)
    match = [0] * n
    open_at = {}
    for col, kind, idx in events:
        if kind == 0:
            if tl_row[idx] in open_at:
                return "syntax error\n"
            open_at[tl_row[idx]] = idx
            fen.add(rank_of[tl_row[idx]], 1)
        else:
            p = fen.pred(rank_of[br_row[idx]])
            if p < 0:
                return "syntax error\n"
            fen.add(p, -1)
            match[open_at.pop(by_rank[p])] = idx
    if open_at:
        return "syntax error\n"
    inv = [0] * n
    for i in range(n):
        inv[match[i]] = i
```

listing 218: python: events as tuples, the inline fenwick's pred and later succ (source: icpc/samples-py/src/Ch08/pL.py, lines 66-92)

```
  local events = {}
  for i = 1, n do events[#events + 1] = { tlc[i], 1, i } end
  for i = 1, n do events[#events + 1] = { brc[i], 0, i } end
  table.sort(events, function(a, b)
    if a[1] ~= b[1] then return a[1] < b[1] end
    return a[2] > b[2]
  end)
  local rows = {}
  for i = 1, n do rows[#rows + 1] = tlr[i] end
  for i = 1, n do rows[#rows + 1] = brr[i] end
  table.sort(rows)
  local rank_of, sorted_rows = {}, {}
  for i = 1, #rows do
    if i == 1 or rows[i] ~= rows[i - 1] then
      rank_of[rows[i]] = #sorted_rows + 1
      sorted_rows[#sorted_rows + 1] = rows[i]
    end
  end
  local fen = fw.new(#sorted_rows)
  local match = {}
  local open_at = {}
  for _, e in ipairs(events) do
    if e[2] == 1 then
      if open_at[tlr[e[3]]] then return "syntax error\n" end
      open_at[tlr[e[3]]] = e[3]
      fw.add(fen, rank_of[tlr[e[3]]] - 1, 1)
    else
      local p = fw.pred(fen, rank_of[brr[e[3]]] - 1)
      if p < 0 then return "syntax error\n" end
      fw.add(fen, p, -1)
      match[open_at[sorted_rows[p + 1]]] = e[3]
      open_at[sorted_rows[p + 1]] = nil
    end
  end
  if next(open_at) then return "syntax error\n" end
```

listing 219: lua: the tie order judge-verified on the secrets, the ch08 fenwick required (source: icpc/samples-lua/ch08_pl.lua, lines 29-63)

One tie order decides correctness on the official data, top-left corners open before bottom-right corners close on the same column, and Go and Lua carry the comment naming the two secret files that pinned it. The order-statistics carrier is a Fenwick tree of counts over compressed row ranks with predecessor and successor everywhere, a shared helper for Go, JavaScript and Lua, an inline class in Python, a private nested class in C#, and in both c and java a fenwick helper for the first sweep plus a max segment tree for the second. The measured fixture is six corners matching as `2 / 3 / 1` with the rows 2 to 3 rectangle nested inside rows 1 to 8, asserted exactly in all seven suites, alongside two syntax-error cases, two top-lefts sharing a row, and crossing rectangles caught by the second sweep. Rows reach a billion and row differences two billion, so C stores long long and the other int64 languages match, JavaScript's doubles are exact at this range.

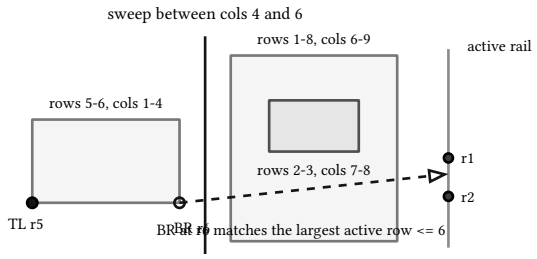


figure 107: the column sweepline between cols 4 and 6, the active top-left set as a row-ordered rail, one match arrow

10.14. across the seven languages

One set, twelve problems, and the same seven solvers every time. The table reads the chapter back from the language axis, what each problem was and where the seven implementations genuinely diverged, which is rarely the algorithm and always the exactness carrier, the sort contract, or the data structure holding the state:

problem	the method (solutions cite)	where the seven languages split
A airport	vertex-pair chords, boundary classification, O(n^3)	exactness ladder: c fractions in `__int128`, java the same fractions in BigInteger, go big.Rat and js bigint re-evaluation, cs integer crosses, py and lua exact toggles
B clue	enumerate answer triples and consistent deals	c and java clause backtracking, cs Hall condition, go/js/lua pruned deal search, py the judges' dictionary join
C improbable	per-height bipartite matching, m+n-match piles	carrier only: dense byte matrix in c and java, column lists, adjacency lists, inlined, closure, module
D money	staircase prune, divide and conquer, O((m+n) log n)	the 1e18 product: int64 in c/cs/go/java/lua, bigint multiply in js
E speed	bisection on a monotone time sum	identical, only the pole guard and the nine-decimal formatting differ
F posterize	d^2 one-color costs, O(d^2 k) dp	closed-form moments in go/js/py/lua, explicit cost loops at the rounded mean in c, cs and java
G replicate	backward xor sweeps, one localized repair	nested-loop sweeps everywhere but py, which runs bit-parallel rows through a transition table
H scenery	forbidden regions, then earliest-deadline greedy	c and java keep an incremental as-late chain, the other five sweep [s,e] subproblems over distinct ends
I chamber	26-closure by floyd-warshall, O(1) lookups	nothing but spelling
J pipe stream	four dinic runs in water-equivalent units	edge-list doubles with eps in c/go/java/js/py/lua, dense matrix with asserts in cs
K tarot	border sequences off the kmp chain, stable sort	c, java and lua spell the index tiebreak, go and js use stable sorts, py sorts tuples, cs relies on List.Sort
L visual	column sweep, fenwick predecessor matching, second sweep	helper modules in go/js/lua, inline class in py, nested class in cs, helper plus segment tree for sweep two in c and java

The pattern the table keeps finding is that the 2017 problems punished arithmetic and bookkeeping, not algorithm invention, so the seven-language spread lands on which exactness a language can afford. Four problems needed more than a double or a 32-bit word, D past 2^53 where only JavaScript had to switch types, C past int32 but under 2^53, F under 2^53 but past int32, and L past int32 in the row differences. Two needed doubles with epsilon discipline end to end, A's boundary chords and J's flows, and both are exactly where the suites re-derive the printed decimals from exact arithmetic rather than trusting the float walk. The remaining six ran on machine ints, bytes, or booleans, E alone among them in plain doubles, a bisection with no integer boundary to police, and there the seven listings are the same program wearing seven syntaxes, which is its own finding about the set.

sources: the problems are icpc2017.pdf and the algorithm reference for every method and complexity claim in this chapter is finals2017solutions.pdf, both from the ICPC Foundation at icpc.global, both accessed 2026-09-14 and cached under ref/icpc with sha256 digests eb78eeba and 599e518a recorded in ref/icpc/INDEX.md, the solved-by counts and the first-python-finals note from the solutions pdf's first page. Judge data is local-only under ref/icpc/2017/data, never committed, 778 input files mapped to answer pairs by the committed ref/icpc/verify-data.ps1, and every sample pair reprinted in this chapter is the byte-exact content of that problem's sample-N.in and sample-N.ans files, attributed in place, with three problems on non-exact compare modes recorded in ref/icpc/compare.json: A in float, the last numeric token within the statement's 1e-6, because the official answers sit beyond double precision in both directions, E in float because the secrets mix shortest-form floats with %.9f samples and one answer sits 4e-7 off exact, verified 18 of 18, and J in last-line-float, the last non-empty line's last numeric token within 1e-4, because per-pipe routings are arbitrary among valid ones and only the maximum value is unique. The listings are sliced from the frozen solvers in books/icpc/samples-c/src/Ch08, books/icpc/samples/src/Ch08 with its tests under books/icpc/samples/tests/Ch08, books/icpc/samples-go/ch08, books/icpc/samples-js/src and test, books/icpc/samples-py/src/Ch08, and books/icpc/samples-lua, plus the java tree at books/icpc/samples-java/src/Ch09, java alone carrying the post-renumber chapter numbers while every sibling tree keeps its historical Ch08 directory, and every fixture this chapter pins is crafted, not taken from the pdfs. The java judge mode reads its whole input through `Files.readString` and walks a cursor over the text, the mirror of c's io.h pointer, closing with one `System.out.print` no 2017 solver streams incrementally, so BufferedReader and StreamTokenizer never appear and Scanner is absent by the toolbox law, while the float answers of A, E and J print through `String.format` under Locale.ROOT. Suite counts, measured from those files: C 40 `CHECK` assertions, C# 26 `[Fact]` test methods, Go 26 table-driven cases in ch08_test.go, JavaScript 26 node:test assertions across the twelve ch08 test files, Python 26 self-check `check` calls, and Lua 26 `T.eq` blocks through run.lua, plus java's 40 `ok` lines across the 16 files of samples-java Ch09, twelve solvers and the four year helpers, the last three verified green on this machine on 2026-09-18, the first three counted from the frozen sources, java's suite green under tools/run-java-samples.ps1 on the pinned oracle jdk 27 with the judge sweep returning 778 of 778 under ref/icpc/verify-data.ps1's `-Lang java` mode.

11. icpc world finals 2018

The 2018 world finals set is eleven problems, A through K, published by the ICPC Foundation at icpc.global, and this chapter works every letter in seven languages, 77 solvers plus the year-local helpers they share. The solutions pdf is the algorithm reference for every method and complexity claim below, exactly as the house rules in chapter 1 require, and the spread its scoreboard page reports for this set is wide: C and J were accepted by no team, D by 5, G by 7. The C and J walkthroughs are therefore the longest, because a problem nobody solved at the contest deserves the fuller derivation. Inputs run to 52.8 MB for A, so whole-file readers are load-bearing across the set, and the judge data that backs the fixtures stays local, its official sample pairs reprinted below byte for byte with attribution.

11.1. the year-local helpers: log factorials, clipped geometry, a counting fenwick, and the tour tables

Four helper kits are year-local to 2018, each named in the year spec with a one-line contract and none duplicating a wave 1 toolbox module, and everything else the year's solvers import comes from the toolbox chapters 2 through 9. Each kit states its contract and prints its twin files below in the book's fixed language order.

The lnfact kit backs problem D and exists for the year's overflow trap: a table of `ln(k!)` with `lnC(a,b)` as three subtractions over it, so the gem island recursion evaluates every binomial in log space and the raw `C(999,500)`, near 1e299, is never built. C's `Ch09/ch09_lnfact.h` fills the table to 1023 behind `lnfact_init`, go's `ch09_lnfact.go` builds 1100 entries as a package var, java's `Ch10/Lnfact.java` fills the table behind `init`, javascript's `ch09-lnfact.mjs` wraps one `Float64Array` in a load-time closure, and lua's `ch09_lnfact.lua` builds its table at require time, while c# and python inline the same table inside their D solvers sized to `n+d+2`.

```c
// ch09_lnfact.h: ln-factorial table and log-space binomials for 2018 D
#ifndef CH09_LNFACT_H
#define CH09_LNFACT_H

#include <math.h>

static double lnf[1024];

static inline void lnfact_init(int n) {
  lnf[0] = 0;
  for (int i = 1; i <= n; i++)
    lnf[i] = lnf[i - 1] + log((double)i);
}

// ln C(a, b); -infinity (caller treats exp as 0) when out of range
static inline double lnC(int a, int b) {
  if (b < 0 || b > a)
    return -1e100;
  return lnf[a] - lnf[b] - lnf[a - b];
}

#endif // CH09_LNFACT_H
```

listing 220: c: lnfact_init fills the table, lnC as three subtractions, out of range held at −1e100 (source: icpc/samples-c/src/Ch09/ch09_lnfact.h, lines 1-22)

```go
package ch09

import "math"

// ch09_lnfact is the year-local gem island helper: a table of
// ln(k!) used to evaluate binomials in log space, where every
// coefficient stays a plain float and C(999, 500) could never fit.
var lnFact = buildLnFact(1100)

func buildLnFact(n int) []float64 {
  table := make([]float64, n+1)
  for i := 2; i <= n; i++ {
    table[i] = table[i-1] + math.Log(float64(i))
  }
  return table
}

// lnC returns ln(C(a, b)), with -Inf when b is out of range.
func lnC(a, b int) float64 {
  if b < 0 || b > a {
```

listing 221: go: buildLnFact as a package var, lnC with -Inf outside the range (source: icpc/samples-go/ch09/ch09_lnfact.go, lines 1-20)

```java
  static void init(int n) {
    lnf[0] = 0;
    for (int i = 1; i ≤ n; i++)
      lnf[i] = lnf[i - 1] + Math.log(i);
  }

  // ln C(a, b); -infinity (caller treats exp as 0) when out of range
  static double lnC(int a, int b) {
    if (b < 0 || b > a)
      return -1e100;
    return lnf[a] - lnf[b] - lnf[a - b];
  }
```

listing 222: java: init fills the table, lnC as three subtractions, out of range held at −1e100 (source: icpc/samples-java/src/Ch10/Lnfact.java, lines 5-16)

```javascript
// ch09-lnfact is the year-local gem island helper: a table of
// ln(k!) used to evaluate binomials in log space, where every
// coefficient stays a plain float and C(999, 500) could never fit.
const lnFact = (() => {
  const table = new Float64Array(1101);
  for (let i = 2; i ≤ 1100; i++) table[i] = table[i - 1] + Math.log(i);
  return table;
})();

// ln(C(a, b)), with -infinity when b is out of range
export function lnC(a, b) {
  if (b < 0 || b > a) return -Infinity;
  return lnFact[a] - lnFact[b] - lnFact[a - b];
}
```

listing 223: javascript: one Float64Array built in a closure, the same exported lnC (source: icpc/samples-js/src/ch09-lnfact.mjs, lines 1-14)

```lua
-- ch09 year-local gem island helper: ln(k!) table so binomials
-- evaluate in log space, where every coefficient stays a plain float
-- and C(999, 500) could never fit
local lnfact = {}

local fact

local function build()
  fact = { [0] = 0.0 }
  for i = 1, 1100 do fact[i] = fact[i - 1] + math.log(i) end
end

build()

-- ln(C(a, b)), or -inf when b is out of range
function lnfact.ln_c(a, b)
  if b < 0 or b > a then return -math.huge end
  return fact[a] - fact[b] - fact[a - b]
end

return lnfact
```

listing 224: lua: fact built at require time, ln_c over three table lookups (source: icpc/samples-lua/ch09_lnfact.lua, lines 1-21)

The geo kit backs problem G with the 2d vector algebra no wave 1 toolbox chapter carried, cross products, ray-cast point-in-polygon, segment intersection, and the half-plane clip of a convex cell against another site's perpendicular bisector. C's `Ch09/ch09_geo.h` walks flat `G2` structs with an on-edge rule that counts the closed park as inside, go's `ch09_geo.go` carries the same predicates over `pt2` values, java's `Ch10/Geo.java` over mutable point objects with the clip's scratch preallocated because the clip runs O(n^2) times per solve, javascript's `ch09-geo.mjs` exports them over plain point objects, and lua's `ch09_geo.lua` spells them over flat coordinate arrays, while c# and python inline the predicates inside their G solvers.

```c
// point in (or on) a simple polygon, ray cast with on-edge check
static inline int g2inpolygon(G2 p, const G2 *poly, int n) {
  for (int i = 0; i < n; i++) { // on-edge counts as inside (closed park)
    G2 a = poly[i], b = poly[(i + 1) % n];
    double cr = g2cross(a, b, p);
    if (cr > -1e-9 && cr < 1e-9 && p.x ≥ (a.x < b.x ? a.x : b.x) - 1e-9 &&
        p.x ≤ (a.x > b.x ? a.x : b.x) + 1e-9 &&
        p.y ≥ (a.y < b.y ? a.y : b.y) - 1e-9 &&
        p.y ≤ (a.y > b.y ? a.y : b.y) + 1e-9)
      return 1;
  }
  int cnt = 0;
  for (int i = 0; i < n; i++) {
    G2 a = poly[i], b = poly[(i + 1) % n];
    if ((a.y > p.y) ≠ (b.y > p.y)) {
      double xx = a.x + (p.y - a.y) * (b.x - a.x) / (b.y - a.y);
      if (p.x < xx)
        cnt++;
    }
  }
  return cnt & 1;
}

// proper intersection of segments (a,b) and (c,d)
static inline int g2segcross(G2 a, G2 b, G2 c, G2 d) {
  double d1 = g2cross(a, b, c), d2 = g2cross(a, b, d);
  double d3 = g2cross(c, d, a), d4 = g2cross(c, d, b);
  return ((d1 > 0) ≠ (d2 > 0)) && ((d3 > 0) ≠ (d4 > 0));
}

// clip the convex polygon (n vertices) by the half-plane left of the
// directed line through a toward b; returns the new vertex count
static inline int g2clip(G2 *p, int n, G2 a, G2 b) {
  G2 out[8192];
  int m = 0;
  for (int i = 0; i < n; i++) {
    G2 cur = p[i], nxt = p[(i + 1) % n];
    double cc = g2cross(a, b, cur), cn = g2cross(a, b, nxt);
    int in1 = cc ≥ -1e-9, in2 = cn ≥ -1e-9;
    if (in1)
      out[m++] = cur;
    if (in1 ≠ in2) {
      double t = cc / (cc - cn);
      G2 x = {cur.x + (nxt.x - cur.x) * t, cur.y + (nxt.y - cur.y) * t};
      out[m++] = x;
    }
  }
  for (int i = 0; i < m; i++)
    p[i] = out[i];
  return m;
}
```

listing 225: c: g2inpolygon by ray cast with the closed-park on-edge rule, g2segcross, then the g2clip half-plane cut (source: icpc/samples-c/src/Ch09/ch09_geo.h, lines 17-67)

```go
// clipCell keeps the part of a convex cell closer to site a than to
// site b (the perpendicular bisector half-plane).
func clipCell(cell []pt2, a, b pt2) []pt2 {
  m := pt2{(a.x + b.x) / 2, (a.y + b.y) / 2}
  d := sub2(b, a)
  var out []pt2
  n := len(cell)
  for i := 0; i < n; i++ {
    cur, next := cell[i], cell[(i+1)%n]
    sc := dot2(sub2(cur, m), d)
    sn := dot2(sub2(next, m), d)
    keepCur, keepNext := sc <= geoEps, sn <= geoEps
    switch {
    case keepCur && keepNext:
      out = append(out, next)
    case keepCur && !keepNext:
      out = append(out, intersectionLine(cur, next, m, d))
    case !keepCur && keepNext:
      out = append(out, intersectionLine(cur, next, m, d), next)
    }
  }
  return out
}

// intersectionLine cuts the segment cur-next at the line through m
// with normal d.
func intersectionLine(cur, next, m, d pt2) pt2 {
  edge := sub2(next, cur)
  den := dot2(d, edge)
  if math.Abs(den) < 1e-18 {
    return cur
  }
  t := dot2(d, sub2(m, cur)) / den
  if t < 0 {
    t = 0
  } else if t > 1 {
    t = 1
  }
  return pt2{cur.x + t*edge.x, cur.y + t*edge.y}
}
```

listing 226: go: clipCell keeping the half-plane closer to one site, and the line cut it inserts (source: icpc/samples-go/ch09/ch09_geo.go, lines 79-118)

```java
static boolean inPolygon(Pt p, Pt[] poly, int n) {
  for (int i = 0; i < n; i++) { // on-edge counts as inside (closed park)
    Pt a = poly[i], b = poly[(i + 1) % n];
    double cr = cross(a, b, p);
    if (cr > -1e-9 && cr < 1e-9 && p.x ≥ Math.min(a.x, b.x) - 1e-9 &&
        p.x ≤ Math.max(a.x, b.x) + 1e-9 &&
        p.y ≥ Math.min(a.y, b.y) - 1e-9 &&
        p.y ≤ Math.max(a.y, b.y) + 1e-9)
      return true;
  }
  int cnt = 0;
  for (int i = 0; i < n; i++) {
    Pt a = poly[i], b = poly[(i + 1) % n];
    if ((a.y > p.y) ≠ (b.y > p.y)) {
      double xx = a.x + (p.y - a.y) * (b.x - a.x) / (b.y - a.y);
      if (p.x < xx)
        cnt++;
    }
  }
  return (cnt & 1) == 1;
}

// proper intersection of segments (a,b) and (c,d)
static boolean segCross(Pt a, Pt b, Pt c, Pt d) {
  double d1 = cross(a, b, c), d2 = cross(a, b, d);
  double d3 = cross(c, d, a), d4 = cross(c, d, b);
  return ((d1 > 0) ≠ (d2 > 0)) && ((d3 > 0) ≠ (d4 > 0));
}
```

listing 227: java: inPolygon by ray cast with the closed-park on-edge rule, then segCross's proper-crossing test (source: icpc/samples-java/src/Ch10/Geo.java, lines 23-50)

```javascript
export function distPointSeg(p, a, b) {
  const abx = b.x - a.x, aby = b.y - a.y;
  const ee = abx * abx + aby * aby;
  if (ee === 0) return Math.hypot(p.x - a.x, p.y - a.y);
  let t = ((p.x - a.x) * abx + (p.y - a.y) * aby) / ee;
  if (t < 0) t = 0;
  else if (t > 1) t = 1;
  return Math.hypot(p.x - (a.x + t * abx), p.y - (a.y + t * aby));
}

// is p inside the polygon or on its boundary (ray cast plus an
// on-edge tolerance)?
export function pointInPoly(poly, p) {
  let inside = false;
  const n = poly.length;
  for (let i = 0; i < n; i++) {
    const a = poly[i], b = poly[(i + 1) % n];
    if (distPointSeg(p, a, b) < geoEps) return true;
    if (a.y > p.y !== b.y > p.y) {
      const xHit = a.x + ((p.y - a.y) * (b.x - a.x)) / (b.y - a.y);
      if (xHit > p.x) inside = !inside;
    }
  }
  return inside;
}

// the intersection point of two segments when they share one,
// handling touching endpoints
export function segIntersect(p1, p2, p3, p4) {
  const cross = (ax, ay, bx, by) => ax * by - ay * bx;
  const d1 = cross(p2.x - p1.x, p2.y - p1.y, p3.x - p1.x, p3.y - p1.y);
  const d2 = cross(p2.x - p1.x, p2.y - p1.y, p4.x - p1.x, p4.y - p1.y);
  const d3 = cross(p4.x - p3.x, p4.y - p3.y, p1.x - p3.x, p1.y - p3.y);
  const d4 = cross(p4.x - p3.x, p4.y - p3.y, p2.x - p3.x, p2.y - p3.y);
  if (((d1 > geoEps && d2 < -geoEps) || (d1 < -geoEps && d2 > geoEps)) &&
      ((d3 > geoEps && d4 < -geoEps) || (d3 < -geoEps && d4 > geoEps))) {
    const s = d3 / (d3 - d4);
    return [{ x: p1.x + s * (p2.x - p1.x), y: p1.y + s * (p2.y - p1.y) }, true];
  }
  if (distPointSeg(p3, p1, p2) < geoEps) return [p3, true];
  if (distPointSeg(p4, p1, p2) < geoEps) return [p4, true];
  if (distPointSeg(p1, p3, p4) < geoEps) return [p1, true];
  if (distPointSeg(p2, p3, p4) < geoEps) return [p2, true];
  return [null, false];
}
```

listing 228: javascript: distPointSeg feeding pointInPoly, segIntersect keeping touching endpoints (source: icpc/samples-js/src/ch09-geo.mjs, lines 11-55)

```
-- keeps the part of a convex cell closer to site a than site b: the
-- perpendicular-bisector half-plane clip
function geo.clip_cell(cell, ax, ay, bx, by)
  local mx, my = (ax + bx) / 2, (ay + by) / 2
  local dx, dy = bx - ax, by - ay
  local n = #cell
  local out = {}
  for i = 1, n do
    local cx, cy = cell[i][1], cell[i][2]
    local j = i % n + 1
    local nx, ny = cell[j][1], cell[j][2]
    local sc = (cx - mx) * dx + (cy - my) * dy
    local sn = (nx - mx) * dx + (ny - my) * dy
    local keep_c, keep_n = sc ≤ EPS, sn ≤ EPS
    if keep_c and keep_n then
      out[#out + 1] = { nx, ny }
    elseif keep_c and not keep_n then
      out[#out + 1] = { geo.line_cut(cx, cy, nx, ny, mx, my, dx, dy) }
    elseif not keep_c and keep_n then
      out[#out + 1] = { geo.line_cut(cx, cy, nx, ny, mx, my, dx, dy) }
      out[#out + 1] = { nx, ny }
    end
  end
  return out
end

-- cuts segment cur-next at the line through m with normal d
function geo.line_cut(cx, cy, nx, ny, mx, my, dx, dy)
  local ex, ey = nx - cx, ny - cy
  local den = dx * ex + dy * ey
  if math.abs(den) < 1e-18 then return cx, cy end
  local t = (dx * (mx - cx) + dy * (my - cy)) / den
  if t < 0 then t = 0 elseif t > 1 then t = 1 end
  return cx + t * ex, cy + t * ey
end
```

listing 229: lua: clip_cell keeping the side closer to one site, line_cut clamped onto the crossing edge (source: icpc/samples-lua/ch09_geo.lua, lines 65-99)

The fenwick kit backs problem I with the year's order statistic, point add and prefix sum over shifted picture columns, an active base end entering the tree at its corner and leaving when its run expires. C's `Ch09/ch09_fenwick.h` carries the whole contract in thirty-one lines, go's `ch09_fenwick.go` walks the same lowbit steps as `fenwick9`, java's `Ch10/Fenwick.java` the same two functions over a plain int array, and lua's `ch09_fenwick.lua` exposes the module `fen` over a sparse table, while javascript inlines a `Fenwick9` class in its solver, python replaces the tree with one big-int bitmask and counts windows by `bit_count`, and c# folds the arrays into its `Sweep`.

```c
// ch09_fenwick.h: fenwick tree with point add and prefix sum over
// shifted columns, for 2018 I
#ifndef CH09_FENWICK_H
#define CH09_FENWICK_H

#include <stdlib.h>
#include <string.h>

typedef struct {
  int n;
  int *t;
} FW;

static inline void fw_init(FW *f, int n) {
  f->n = n;
  f->t = calloc((size_t)n + 1, sizeof(int));
}

static inline void fw_free(FW *f) { free(f->t); }

static inline void fw_add(FW *f, int i, int d) {
  for (; i ≤ f->n; i += i & (-i))
    f->t[i] += d;
}

static inline int fw_prefix(FW *f, int i) {
  int s = 0;
  for (; i > 0; i -= i & (-i))
    s += f->t[i];
  return s;
}
```

listing 230: c: the whole fenwick header, init, point add, prefix sum, three functions (source: icpc/samples-c/src/Ch09/ch09_fenwick.h, lines 1-31)

```
package ch09

// ch09_fenwick is the year-local order helper used by problem I:
// point add and prefix sum over shifted columns, sized once per
// picture row.
type fenwick9 struct {
  tree []int
}

func newFenwick9(n int) *fenwick9 {
  return &fenwick9{tree: make([]int, n+1)}
}

func (f *fenwick9) add(i, delta int) {
  for i++; i < len(f.tree); i += i & -i {
    f.tree[i] += delta
  }
}

func (f *fenwick9) prefix(i int) int {
  sum := 0
  for i++; i > 0; i -= i & -i {
```

listing 231: go: fenwick9 over shifted columns, add and prefix on the lowbit walk (source: icpc/samples-go/ch09/ch09_fenwick.go, lines 1-22)

```
  Fenwick(int n) {
    this.n = n;
    t = new int[n + 1];
  }

  void add(int i, int d) {
    for (; i ≤ n; i += i & -i)
      t[i] += d;
  }

  int prefix(int i) {
    int s = 0;
    for (; i > 0; i -= i & -i)
      s += t[i];
    return s;
  }
}
```

listing 232: java: the whole helper, point add and prefix sum on the lowbit walk (source: icpc/samples-java/src/Ch10/Fenwick.java, lines 7-23)

```lua
-- ch09 year-local order helper for problem i: point add and prefix
-- sum over shifted picture columns, sized once per row
local fen = {}

function fen.new(n)
  return { n = n, tree = {} }
end

function fen.add(f, i, delta)
  i = i + 1
  local t = f.tree
  while i ≤ f.n + 1 do
    t[i] = (t[i] or 0) + delta
    i = i + (i & -i)
  end
end

function fen.prefix(f, i)
  i = i + 1
  local t, sum = f.tree, 0
  while i > 0 do
    sum = sum + (t[i] or 0)
    i = i - (i & -i)
  end
  return sum
end

return fen
```

listing 233: lua: fen.add and fen.prefix on the 1-based lowbit walk, empty slots held by `or 0` (source: icpc/samples-lua/ch09_fenwick.lua, lines 1-28)

The jtable kit embeds problem J's per-m base tables and cycle parameters, `Ch09/ch09_jtable.h`, `ch09_jtable.go`, `ch09-jtable.mjs`, `ch09_jtable.lua` and `Ch10/Jtable.java`, and all five twins already print in full inside problem J's own section, so no listing repeats here. C# and Python carry no twin file in any of the year's four kits: their solvers inline the log table in D, the predicates in G, the arrays and bitmask in I, and the embedded tour tables in J.

11.2. problem A: catch the plane

The plane to the ICPC Finals leaves soon and the only way to the airport is by bus, but the drivers are considering a strike: every bus in the schedule runs with its own probability, the events independent, and you learn whether the bus you tried to board actually runs only at that moment. You stand at station 0, the airport is station 1, and the schedule lists each bus's start station, destination, departure time, arrival time, and run probability. A transfer needs an arrival strictly earlier than the next bus's departure, arriving exactly at a departure is too late to board, and among the buses leaving one station at the same instant you can try to get on only one of them. Plan the journey that maximizes the probability of standing at the airport by time k (statement).

The input is two integers m and n by the problemset pdf, 1 <= m <= 10^6 buses and 2 <= n <= 10^6 stations, then one integer k, 1 <= k <= 10^18, the time by which you must arrive at the airport, then m bus lines: stations a and b with 0 <= a, b < n and a != b, the departure and arrival times s and t with 0 <= s < t <= k, and p, the probability the bus runs, 0 <= p <= 1 written with at most 10 digits after the decimal point. The output is the single probability of catching the plane under an optimal course of action, correct to an absolute error of 1e-6, under the pdf's 10 second limit, and the harness verifies it in float mode, `2018/A` pinned to `float` in `ref/icpc/compare.json`.

Official sample 1, reprinted byte for byte from the judge data pair `A-catch/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints and walks through beside its schedule figure: gamble on the 0.2 direct flight and fall

back to the station 3 route, itself worth `0.5*0.181 + 0.5*0.1` through its 0.1 airport bus and its 0.9 return loop to the last direct, for `0.2 + 0.8*0.1405 = 0.3124`, while the certain bus into station 2 arrives exactly at its best onward's departure.

input:

```
8 4
1000
0 1 0 900 0.2
0 2 100 500 1.0
2 1 500 700 1.0
2 1 501 701 0.1
0 3 200 400 0.5
3 1 500 800 0.1
3 0 550 650 0.9
0 1 700 900 0.1
```

expected output:

```
0.3124
```

Recognition: the bounds set the sweep's budget before the probabilities do. With m <= 10^6 buses under the 10 second limit, one sort of the 10^6 events costs about 2*10^7 comparisons and the backward pass folds one proposal per event, 2*10^6 probability updates, so O(m log m) is the whole affordable shape and everything else must fall out of the sorted order itself.

The statement's cue is its timing discipline: you learn whether a bus runs only at the moment you try to board, an arrival exactly at a departure is too late, and one try per simultaneous departure, all constraints about a journey read backwards from the deadline k, so each station carries one scalar, its onward success probability, refreshed by the events that pass it. Problem A is the 2018 face of the event sweep over a sorted axis family, and the same event sweep over a sorted axis cue drives chapter 11 problem H, chapter 12 problem D, chapter 12 problem J, and chapter 14 problem L.

The tempting alternative, a forward grid over time instants, dies on k <= 10^18, and enumerating journeys prices out at 2^1000000 strategies against 10^6 buses, so the sorted backward event axis is the only budget that closes.

The sweep runs backwards over sorted events with every station's onward success probability initialized to 0 and the airport's to 1: an arrival snapshots its station's onward success probability, a departure proposes `r*q + (1-r)*p[station]`, and simultaneous departures from one station batch so only the best proposal lands, with arrivals at time t snapshotted before t-departures fold in, which is exactly the rule that kills the same-instant transfer. The answer is station 0's probability once the time 0 batch lands, O(m log m) for the sort and linear past it (solutions.pdf pp. 1-2). The fixture is four buses where the certain bus into station 2 arrives exactly when its onward connection leaves, so the direct flight with a weak fallback wins at 0.55, and the sweep below lands it.

The worked run: trace the model on sample 1. The airport seeds `p[1] = 1` and every other station 0, and the backward sweep folds the printed buses latest first. Bus 8's arrival at 900 snapshots `p[1] = 1`, so its 700 departure from station 0 lands `0.1*1 +`
`0.9*0 = 0.1`. Bus 7's 650 arrival into station 0 snapshots that 0.1, and its 550 departure from station 3 lands `0.9*0.1 + 0.1*0 =`
`0.09`. Bus 6's 800 arrival snapshotted 1, so its 500 departure from station 3 lifts `p[3]` to `0.1*1 + 0.9*0.09 = 0.181`, which bus 5's 400 arrival snapshots before its 200 departure from station 0 lands `0.5*0.181 + 0.5*0.1 = 0.1405`. The same-instant rule then decides the sample: bus 2 arrives at station 2 exactly at 500, its snapshot sees `p[2] = 0` because bus 3's 500 departure from station 2 folds only afterwards and lands `1.0*1 + 0.0*0 =`
1, so bus 2's 100 departure proposes `1.0*0 + 0.0*0.1405 = 0` and waiting keeps 0.1405. The time 0 batch finally proposes bus 1 as `0.2*1 + 0.8*0.1405 = 0.3124`, and the trace ends at the printed answer `0.3124`.

instant	event	fold	station value
900	arrival, bus 8	snapshot `p1`	q = 1
700	departure, bus 8	`0.1*1 + 0.9*0`	`p0` = 0.1
650	arrival, bus 7	snapshot `p0`	q = 0.1
550	departure, bus 7	`0.9*0.1 + 0.1*0`	`p3` = 0.09
500	arrival, bus 2	`p2` before the 500 batch	q = 0
500	departure, bus 6	`0.1*1 + 0.9*0.09`	`p3` = 0.181
500	departure, bus 3	`1.0*1 + 0.0*0`	`p2` = 1
200	departure, bus 5	`0.5*0.181 + 0.5*0.1`	`p0` = 0.1405
100	departure, bus 2	`1.0*0 + 0.0*0.1405`, waiting wins	`p0` = 0.1405
0	departure, bus 1	`0.2*1 + 0.8*0.1405`	`p0` = 0.3124

```c
  static int idx[AM * 2];
  for (int i = 0; i < nev; i++)
    idx[i] = i;
  qsort(idx, (size_t)nev, sizeof(int), cmp_key);
  int maxst = 2;
  for (int i = 0; i < m; i++) {
    if (ba[i] + 1 > maxst)
      maxst = ba[i] + 1;
    if (bb[i] + 1 > maxst)
      maxst = bb[i] + 1;
  }
  for (int i = 0; i < maxst; i++)
    prob[i] = 0;
  prob[1] = 1;
  double *q = malloc((size_t)m * sizeof(double));
  int e = 0;
  while (e < nev) {
    int i = ord[idx[e]];
    long long k = key[idx[e]];
    if (k & 1) { // arrival: record onward probability
      q[i] = prob[bb[i]];
      e++;
      continue;
    }
    // batch all departures from the same station at the same time
    int st = ba[i];
    long long tm = bs[i];
    double base = prob[st], best = base;
    while (e < nev) {
      int j = ord[idx[e]];
      if (!(key[idx[e]] & 1) && ba[j] == st && bs[j] == tm) {
        double prop = bp[j] * q[j] + (1 - bp[j]) * base;
        if (prop > best)
          best = prop;
        e++;
      } else {
        break;
      }
    }
    prob[st] = best;
  }
  char *out = malloc(32);
  snprintf(out, 32, "%.9f\n", prob[0]);
  free(q);
  free(buf);
  return out;
```

listing 234: c: index sort by packed key, arrivals recorded, one batched best proposal per station and instant (source: icpc/samples-c/src/Ch09/pA.c, lines 63-108)

```
  type event struct {
    t    int64
    kind int // 0 arrival, 1 departure
    idx  int
  }
  events := make([]event, 0, 2*m)
  for i := 0; i < m; i++ {
    events = append(events, event{arr[i], 0, i}, event{dep[i], 1, i})
  }
  sort.Slice(events, func(a, b int) bool {
    x, y := events[a], events[b]
    if x.t ≠ y.t {
      return x.t > y.t
    }
    if x.kind ≠ y.kind {
      return x.kind < y.kind
    }
    if x.kind == 1 && src[x.idx] ≠ src[y.idx] {
      return src[x.idx] < src[y.idx]
    }
    return false
  })
  p := make([]float64, n)
  p[1] = 1
  arrVal := make([]float64, m)
  i := 0
  for i < len(events) {
    t := events[i].t
    for ; i < len(events) && events[i].t == t && events[i].kind == 0; i++ {
      b := events[i].idx
      arrVal[b] = p[dst[b]]
    }
    for ; i < len(events) && events[i].t == t; {
      st := src[events[i].idx]
      old, best := p[st], p[st]
      for ; i < len(events) && events[i].t == t && src[events[i].idx] == st; i++ {
        b := events[i].idx
        if v := prob[b]*arrVal[b] + (1-prob[b])*old; v > best {
          best = v
        }
      }
      p[st] = best
    }
  }
  return fmt.Sprintf("%.9f\n", p[0])
}
```

listing 235: go: one sort key orders time, kind, station, the sweep reads it straight through (source: icpc/samples-go/ch09/pa.go, lines 33-78)

```
    for (int i = 0; i < m; i++) {
      ba[i] = (int) num();
      bb[i] = (int) num();
      bs[i] = num();
      bt[i] = num();
      bp[i] = dbl();
      key[nev++] = bt[i] * 2 + 1; // arrival of bus i
      key[nev++] = bs[i] * 2; // departure of bus i
    }
    Integer[] idx = new Integer[nev];
    for (int i = 0; i < nev; i++)
      idx[i] = i;
    // descending time (key packs t*2 + type); arrivals before departures
    // at equal times; then departures grouped by station so one station's
    // simultaneous departures stay contiguous
    Arrays.sort(idx, (a, b) -> {
      if (key[a] != key[b])
        return Long.compare(key[b], key[a]);
      int ta = (int) (key[a] & 1), tb = (int) (key[b] & 1);
      if (ta != tb)
        return Integer.compare(tb, ta);
      int ia = a >> 1, ib = b >> 1;
      if (ta == 0 && ba[ia] != ba[ib])
        return Integer.compare(ba[ia], ba[ib]);
      return 0;
    });
```

listing 236: java: keys packed t*2 + type so arrivals sort before departures, the boxed index sort grouping one station's simultaneous departures (source: icpc/samples-java/src/Ch10/PA.java, lines 49-74)

```
        // latest first; at equal times departures must see pre-batch values, so
        // all proposals of one instant are collected before applying
        var order = Enumerable.Range(0, (int)(2 * m))
            .OrderByDescending(i => times[i])
            .ToArray();
        var prob = new double[n + 1];
        prob[1] = 1.0;
        // value of being at the arrival station, at the arrival
        // instant, strictly before same-time buses
        var q = new double[m];
        int total = (int)(2 * m);
        int e = 0;
        while (e < total)
        {
            long cur = times[order[e]];
            int d = e;
            while (d < total && times[order[d]] == cur)
                d++;
            for (int i = e; i < d; i++)
                if (evBus[order[i]] < 0)
                    q[-1 - evBus[order[i]]] = prob[to[-1 - evBus[order[i]]]];
            var best = new Dictionary<long, double>();
            for (int i = e; i < d; i++)
            {
                if (evBus[order[i]] < 0)
                    continue;
                int bus = evBus[order[i]];
                double proposal = p[bus] * q[bus] + (1 - p[bus]) * prob[from[bus]];
                if (!best.TryGetValue(from[bus], out double b) || proposal > b)
                    best[from[bus]] = proposal;
            }
            foreach (var (st, val) in best)
                if (val > prob[st])
                    prob[st] = val;
            e = d;
        }
        return prob[0].ToString("F9", CultureInfo.InvariantCulture) + "\n";
```

listing 237: c#: instant groups collect every arrival before the best-proposal dictionary lands (source: icpc/samples/src/Ch09/PA.cs, lines 29-65)

```
  // exact descending rank per distinct time (times reach 1e18, past
  // the exact float64 range)
  const times = new BigInt64Array(2 * m);
  times.set(dep);
  times.set(arr, m);
  times.sort();
  const rankOf = new Map();
  for (let i = 2 * m - 1, r = 0; i ≥ 0; i--) {
    if (i === 2 * m - 1 || times[i] =/= times[i + 1]) rankOf.set(times[i], r++);
  }
  // key = rank * 2^22 + kind * 2^21 + src: arrivals before departures
  // at one time, same-station departures contiguous
  const keyOf = (t, kind, st) => rankOf.get(t) * 4194304 + kind * 2097152 + st;
  const keys = new Float64Array(2 * m);
  for (let i = 0; i < m; i++) {
    keys[i] = keyOf(arr[i], 0, 0);
    keys[m + i] = keyOf(dep[i], 1, src[i]);
  }
  const ord = Array.from({ length: 2 * m }, (_, i) => i);
  ord.sort((a, b) => keys[a] - keys[b]);
  const p = new Float64Array(n);
  p[1] = 1;
  const arrVal = new Float64Array(m);
  const busOf = (e) => (e < m ? e : e - m);
  let i = 0;
  const ordLen = ord.length;
  while (i < ordLen) {
    const hi = Math.floor(keys[ord[i]] / 4194304) * 4194304;
    // arrivals at this time snapshot first
    while (i < ordLen && keys[ord[i]] < hi + 2097152) {
      const b = busOf(ord[i]);
      arrVal[b] = p[dst[b]];
      i++;
    }
    // departures, batched per station: proposals compete against the
    // pre-batch value and only the best lands
    while (i < ordLen && keys[ord[i]] < hi + 4194304) {
      const st = src[busOf(ord[i])];
      const old = p[st];
      let best = p[st];
      while (i < ordLen && keys[ord[i]] < hi + 4194304 &&
             src[busOf(ord[i])] === st) {
        const b = busOf(ord[i]);
        const v = prob[b] * arrVal[b] + (1 - prob[b]) * old;
        if (v > best) best = v;
        i++;
      }
      p[st] = best;
    }
  }
  return `${p[0].toFixed(9)}\n`;
```

listing 238: javascript: exact descending time ranks packed as `rank*2^22 + kind*2^21 + src`, then the same sweep (source: icpc/samples-js/src/ch09-pa-catch.mjs, lines 71-121)

```
    pos = 3
    for i in range(m):
        src[i] = int(data[pos])
        dst[i] = int(data[pos + 1])
        dep = int(data[pos + 2])
        arr = int(data[pos + 3])
        prob[i] = float(data[pos + 4])
        pos += 5
        # reversed iteration flips kind to arrivals-first; sorting by
        # station keeps the departure batches contiguous
        events[2 * i] = (arr, 1, 0, i)
        events[2 * i + 1] = (dep, 0, src[i], i)
    events.sort()
    p = [0.0] * n
    p[1] = 1.0
    arr_val = [0.0] * m
    total = 2 * m
    while total:
        total -= 1
        t, kind, st, i = events[total]
        if kind:
            arr_val[i] = p[dst[i]]
            continue
        old = p[st]
        best = old
        while total ≥ 0:
            t2, kind2, st2, j = events[total]
            if kind2 or t2 ≠ t or st2 ≠ st:
                break
            q = prob[j]
            v = q * arr_val[j] + (1 - q) * old
            if v > best:
                best = v
            total -= 1
        total += 1
        p[st] = best
    return f"{p[0]:.9f}\n"
```

listing 239: python: a flipped kind bit in the tuple sort puts arrivals first, the batch loop walks backwards (source: icpc/samples-py/src/Ch09/pa.py, lines 19-55)

```
  -- events sorted by time descending, arrivals before departures at
  -- equal time; batching by station makes the within-batch order
  -- irrelevant, so a plain key sort suffices
  local ev = {}
  for i = 1, m do
    ev[#ev + 1] = { arr[i], 0, i }
    ev[#ev + 1] = { dep[i], 1, i }
  end
  table.sort(ev, function(a, b)
    if a[1] ~= b[1] then return a[1] > b[1] end
    if a[2] ~= b[2] then return a[2] < b[2] end
    return src[a[3]] < src[b[3]]
  end)
  local p = {}
  for i = 1, n do p[i] = 0.0 end
  p[2] = 1.0 -- station 2 is the airport in 1-based numbering
  local arr_val = {}
  local i = 1
  local ne = #ev
  while i ≤ ne do
    local t = ev[i][1]
    while i ≤ ne and ev[i][1] == t and ev[i][2] == 0 do
      local b = ev[i][3]
      arr_val[b] = p[dst[b]]
      i = i + 1
    end
    while i ≤ ne and ev[i][1] == t do
      local st = src[ev[i][3]]
      local old, best = p[st], p[st]
      while i ≤ ne and ev[i][1] == t and src[ev[i][3]] == st do
        local b = ev[i][3]
        local v = pr[b] * arr_val[b] + (1.0 - pr[b]) * old
        if v > best then best = v end
        i = i + 1
      end
      p[st] = best
    end
  end
  return string.format("%.9f\n", p[1])
```

listing 240: lua: byte-scanner parse, key sort, station 2 is the airport in 1-based numbering (source: icpc/samples-lua/ch09_pa.lua, lines 42-80)

The suites pin the fixture `4 3 / 10 / …` at `0.550000000\n`: three CHECKs in c (the fixture plus two hand cases, one a same-instant batch), three [Fact] methods in c#, four table cases in go, four ok lines in java, one assertion in the javascript node:test file, four checks in python, and four `T.eq` blocks in lua, the extra cases each covering a certain direct bus, a best-of-batch, and a failed first leg caught by a later flight. Integer notes: times run to 1e18, so every language keeps them as 64-bit integers or exact integers, and javascript goes one step further because a float64 sort key loses integer exactness past 2^53, so it ranks the distinct times exactly with a `BigInt64Array` first and packs `rank`, kind, and station into one float key, while java packs `t*2 + type` into one long, exact at 1e18, and sorts boxed indices through a comparator that keeps one station's simultaneous departures contiguous, python sorts tuples whose kind bit is flipped and lua sorts tables with the plain comparison chain.

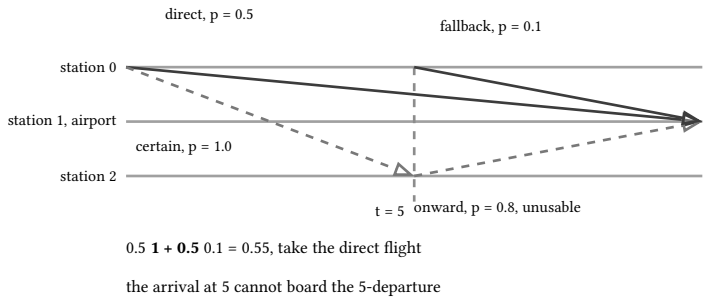


figure 108: the fixture on one clock: the certain bus reaches station 2 exactly at its onward departure, so the direct flight with the 0.1 fallback is the strategy that survives

11.3. problem B: comma sprinkler

Doctor Comma Sprinkler claims to have fixed English comma placement, frustrating and ambiguous as it is, with two rewrite rules. Rule one: if a word anywhere in the text is preceded by a comma, put a comma before every occurrence of that word, except an occurrence that opens a sentence or already carries one. Rule two: if a word anywhere in the text is succeeded by a comma, put a comma after every occurrence of that word, except an occurrence that closes a sentence or already carries one. The two rules apply repeatedly until neither can add a comma anywhere, and the task is to print the final text (statement).

The input is one line of text, at least 2 and at most 10^6 characters by the problemset pdf, every character a lowercase letter, a comma, a period, or a space. A word is a maximal run of letters, the text begins with a word, between every two words sits exactly one of a single space, a comma and a space, or a period and a space ending one sentence and opening the next, and the last word of the text is followed by a period with nothing after it. The output is the text after both rules run to their fixed point, under the pdf's 8 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `B-comma/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints and works through sentence by sentence in its statement: the comma after spot in the middle sentence plants one after the third sentence's spot, which is not sentence-final there, and one before the first sentence's sit, which is not sentence-initial, and that new comma before the first here forces one before the second here.

input:

```
please sit spot. sit spot, sit. spot here now here.
```

expected output:

```
please, sit spot. sit spot, sit. spot, here now, here.
```

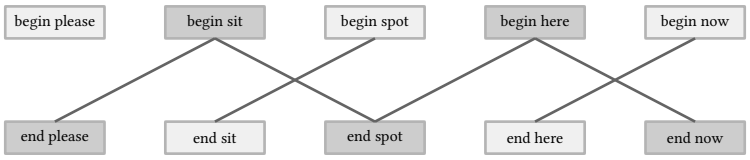
Recognition: the text runs to 10^6 characters, say 5*10^5 words and as many within-sentence adjacencies, under the 8 second limit, and one closure pass over that graph is linear, so the budget demands the whole fixed point fall out of a single reachability sweep rather than from rule applications.

The cue sits in the rules' own wording, every occurrence of that word, a quantifier over occurrences, with the only exemptions positional, sentence-initial and sentence-final, so what propagates is a property of word positions in the separator graph, not of the text layout. Problem B is the 2018 face of the component structure and connectivity family, and the component structure and connectivity family runs through chapter 12 problem E, chapter 12 problem H, chapter 13 problem R, chapter 15 problem E, and chapter 14 problem G.

Reapplying the two rewrite rules textually until quiescence is the priced-out alternative: one planted comma can force a change in another sentence, so a text of s sentences can take s full 10^6-character scans, 2.5*10^11 character touches at the bound, against the closure's single linear pass.

The model is two graph vertices per distinct word, begin and end, with an edge joining end(prev) to begin(next) for every within-sentence adjacency: the input's own commas seed marks on end(prev) for a comma after prev and on begin(next) for a comma before next, a mark on a begin vertex can only ever force marks on the end vertices joined to it and vice versa, so one closure over the seeded components decides every separator, O(n log n) with a dictionary (solutions.pdf p. 2). The closure is union-find in c, c# and java, java's `find` path-halving, and a stack walk in go, javascript, python, and lua, and both vehicles land the same bytes on the fixture, whose only seed `ab,` spreads to three more vertices and stops.

The worked run: trace the model on sample 1. The three sentences carry five distinct words, please, sit, spot, here and now, so the graph is ten vertices with an edge joining end(prev) to begin(next) for each of the seven within-sentence adjacencies, and the input's one comma, the one after the middle sentence's spot, seeds two marks, end(spot) and begin(sit). The closure spreads along adjacency edges: end(spot) reaches begin(here) across the third sentence's spot-here, begin(sit) reaches end(please) across the first sentence's opening, begin(here) reaches end(now) across the closing now-here, and the component closes at five marked vertices. Reading marks back to text, end(please) plants the comma after the first please, begin(sit) plants one before the first sentence's sit, end(spot) plants one after the third sentence's spot, which is not sentence-final there, begin(here) plants one before both of the third sentence's heres, and end(now) plants one after now, while the first sentence's spot stays bare as sentence-final and the middle spot already carries its comma, and the trace ends at the printed answer `please, sit spot. sit spot, sit. spot, here now, here.`.



seeded: the comma in spot, sit.

five marked vertices, four commas printed, three of them new

figure 109: sample 1′s closure graph: the seeded comma closes five vertices of one component, and those five marks print four commas, three of them new

```
  for (int v = 0; v < 2 * nwords; v++)
    parent[v] = v;
  for (int k = 0; k + 1 < nocc; k++)
    if (!ost[k + 1])
      uni(2 * oid[k] + 1, 2 * oid[k + 1]);
  for (int k = 0; k < nocc; k++) {
    if (obc[k] && !ost[k])
      seeded[find(2 * oid[k])] = 1;
    if (oac[k] && !oen[k])
      seeded[find(2 * oid[k] + 1)] = 1;
  }
  size_t outcap = len + nocc + 16;
  char *out = malloc(outcap);
  size_t o = 0;
  for (int k = 0; k < nocc; k++) {
    if (k > 0) {
      // one comma per separator: the before/after flags agree because
      // the separator's endpoints are unioned
      if (!ost[k] && seeded[find(2 * oid[k])])
        out[o++] = ',';
      out[o++] = ' ';
    }
    memcpy(out + o, buf + olo[k], (size_t)(ohi[k] - olo[k]));
    o += (size_t)(ohi[k] - olo[k]);
    if (oen[k])
      out[o++] = '.';
  }
  out[o++] = '\n';
  out[o] = 0;
```

listing 241: c: fnv-1a interning into an open-addressed table, union-find closure, one comma per separator (source: icpc/samples-c/src/Ch09/pB.c, lines 100-128)

```
begin := func(w int) int { return 2 * w }
end := func(w int) int { return 2*w + 1 }
// adjacency in CSR over 2*nWords vertices
deg := make([]int, 2*nWords)
type slot struct{ a, b, seed int }
var slots []slot
for k := 0; k+1 < len(toks); k++ {
  if toks[k].sep == 2 {
    continue
  }
  a, b := end(toks[k].word), begin(toks[k+1].word)
  slots = append(slots, slot{a, b, toks[k].sep})
  deg[a]++
  deg[b]++
}
starts := make([]int, 2*nWords+1)
for v, d := range deg {
  starts[v+1] = starts[v] + d
}
fill := append([]int(nil), starts[:len(starts)-1]...)
adj := make([]int, starts[2*nWords])
for _, sl := range slots {
  adj[fill[sl.a]] = sl.b
  fill[sl.a]++
  adj[fill[sl.b]] = sl.a
  fill[sl.b]++
}
marked := make([]bool, 2*nWords)
queue := make([]int, 0, 2*nWords)
for _, sl := range slots {
  if sl.seed == 1 && !marked[sl.a] {
    marked[sl.a] = true
    queue = append(queue, sl.a)
  }
}
for len(queue) > 0 {
  v := queue[len(queue)-1]
  queue = queue[:len(queue)-1]
  for _, u := range adj[starts[v]:starts[v+1]] {
    if !marked[u] {
      marked[u] = true
      queue = append(queue, u)
    }
  }
}
```

listing 242: go: the adjacency as a CSR array pair, seeds enqueued, stack closure over it (source: icpc/samples-go/ch09/pb.go, lines 51-95)

```
      int s = i;
      long h = FNV_OFFSET;
      while (i < len && buf.charAt(i) ≥ 'a' && buf.charAt(i) ≤ 'z') {
        h ^= buf.charAt(i);
        h *= FNV_PRIME;
        i++;
      }
      int slot = (int) (h & (tcap - 1));
      int id;
      for (;;) {
        if (tabId[slot] < 0) {
          tabH[slot] = h;
          tabId[slot] = nwords;
          id = nwords++;
          break;
        }
        if (tabH[slot] == h) {
          id = tabId[slot];
          break;
        }
        slot = (slot + 1) & (tcap - 1);
      }
```

listing 243: java: fnv-1a interning into an open-addressed table, the offset basis parsed unsigned so the multiply wraps like c's (source: icpc/samples-java/src/Ch10/PB.java, lines 57-78)

```
        // two vertices per distinct word: begin and end; marks spread along the
        // end(prev)-begin(next) edges, so one union-find component per closure
        var id = new Dictionary<string, int>();
        int Vid(string w, bool end)
        {
            string key = w + (end ? "#" : "^");
            if (!id.TryGetValue(key, out int v))
                id[key] = v = id.Count;
            return v;
        }
        var beg = new int[k2];
        var end = new int[k2];
        for (int x = 0; x < k2; x++)
        {
            beg[x] = Vid(pieces[x].Word, false);
            end[x] = Vid(pieces[x].Word, true);
        }
        var uf = new int[id.Count];
        for (int x = 0; x < uf.Length; x++)
            uf[x] = x;
        int Find(int x)
        {
            while (uf[x] ≠ x)
            {
                uf[x] = uf[uf[x]];
                x = uf[x];
            }
            return x;
        }
        for (int x = 0; x + 1 < k2; x++)
        {
            if (pieces[x].Period)
                continue;
            uf[Find(end[x])] = Find(beg[x + 1]);
        }
        var marked = new bool[id.Count];
        for (int x = 0; x < k2; x++)
            if (pieces[x].Comma)
                marked[Find(end[x])] = true;
```

listing 244: c#: dictionary vertices `word^` and `word#`, path-halving find, marks seed whole components (source: icpc/samples/src/Ch09/PB.cs, lines 28-66)

```
  const nWords = ids.size;
  const begin = (w) => 2 * w;
  const end = (w) => 2 * w + 1;
  const adj = Array.from({ length: 2 * nWords }, () => []);
  const slots = [];
  for (let k = 0; k + 1 < toks.length; k++) {
    if (toks[k][1] === 2) continue; // sentence boundary, no edge
    const a = end(toks[k][0]), b = begin(toks[k + 1][0]);
    slots.push([a, b, toks[k][1]]);
    adj[a].push(b);
    adj[b].push(a);
  }
  const marked = new Uint8Array(2 * nWords);
  const queue = [];
  for (const [a, , seed] of slots) {
    if (seed === 1 && !marked[a]) {
      marked[a] = 1;
      queue.push(a);
    }
  }
  while (queue.length > 0) {
    const v = queue.pop();
    for (const u of adj[v]) {
      if (!marked[u]) {
        marked[u] = 1;
        queue.push(u);
      }
    }
  }
```

listing 245: javascript: a Map interns words, adjacency lists in two arrays of arrays, the stack closure (source: icpc/samples-js/src/ch09-pb-comma.mjs, lines 38-66)

```
    nvert = 2 * len(ids)
    adj = [[] for _ in range(nvert)]
    seeds = []
    for k in range(len(toks) - 1):
        wid, sep = toks[k]
        if sep == 2:
            continue
        a, b = 2 * wid + 1, 2 * toks[k + 1][0]
        adj[a].append(b)
        adj[b].append(a)
        if sep == 1:
            seeds.append(a)
    marked = bytearray(nvert)
    queue = []
    for a in seeds:
        if not marked[a]:
            marked[a] = 1
            queue.append(a)
    while queue:
        v = queue.pop()
        for u in adj[v]:
            if not marked[u]:
                marked[u] = 1
                queue.append(u)
    out = []
    for k, (wid, sep) in enumerate(toks):
        out.append(words[wid])
        if sep == 2:
            out.append(". " if k + 1 < len(toks) else ".")
            continue
        comma = (
            sep == 1
            or marked[2 * wid + 1]
            or (k + 1 < len(toks) and marked[2 * toks[k + 1][0]])
        )
        out.append(", " if comma else " ")
    out.append("\n")
    return "".join(out)
```

listing 246: python: bytearray marks, list-stack closure, rebuild with the exempt cases split out (source: icpc/samples-py/src/Ch09/pb.py, lines 38-75)

```lua
local nv = 2 * #ids
local adj = {}
local seeds = {}
for k = 1, #toks - 1 do
  if toks[k][2] ~= 2 then
    local a = 2 * toks[k][1] -- end(prev)
    local b = 2 * toks[k + 1][1] - 1 -- begin(next)
    adj[a] = adj[a] or {}
    adj[b] = adj[b] or {}
    adj[a][#adj[a] + 1] = b
    adj[b][#adj[b] + 1] = a
    if toks[k][2] == 1 then seeds[#seeds + 1] = a end
  end
end
local marked = {}
local stack = {}
for _, v in ipairs(seeds) do
  if not marked[v] then
    marked[v] = true
    stack[#stack + 1] = v
  end
end
while #stack > 0 do
  local v = table.remove(stack)
  for _, u in ipairs(adj[v] or {}) do
    if not marked[u] then
      marked[u] = true
      stack[#stack + 1] = u
    end
  end
end
local out = {}
for k = 1, #toks do
  out[#out + 1] = ids[toks[k][1]]
  if toks[k][2] == 2 then
    out[#out + 1] = (k == #toks) and "." or ". "
  else
    local comma = toks[k][2] == 1 or marked[2 * toks[k][1]] or
      (k < #toks and marked[2 * toks[k + 1][1] - 1])
    out[#out + 1] = comma and ", " or " "
  end
end
return table.concat(out) .. "\n"
```

listing 247: lua: ids double as a rank list, closure over adjacency tables, table.concat output (source: icpc/samples-lua/ch09_pb.lua, lines 41-83)

Every suite pins the fixture `ab cd. ef ab, gh. ef cd.` at `ab, cd. ef, ab, gh. ef, cd.\n` and the two boundary exemptions, a sentence-final word that never takes a trailing comma and a sentence-initial word that never takes a leading one: three CHECKs in c (the fixture plus both official samples), three [Fact] methods in c#, five cases in go, five ok lines in java, five assertions in javascript, five checks in python, and five `T.eq` blocks in lua, the shared extras being the two exemptions, a seed whose closure changes nothing, and a comma-free text that is already a fixed point. One integer note, and it is java's: the fnv-1a offset basis `14695981039346656037` crosses 2^63, so it enters through `Long.parseUnsignedLong` and every plain long multiply then wraps exactly like c's unsigned arithmetic, and no other unsigned site, `remainderUnsigned` included, appears in the year's java solvers.

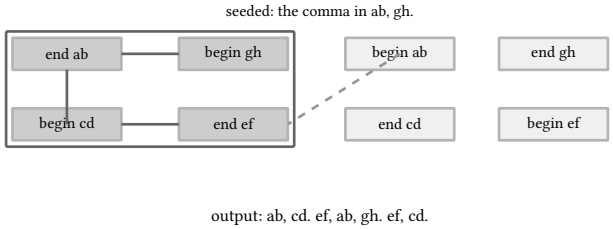


figure 110: the fixture's closure graph: the seeded end(ab) reaches three more vertices, and those four marks are exactly the printed commas

11.4. problem C: conquer the world

The volcano lair's deathtraps have, you assume, finally caught the dashingly handsome spy Waco Powers, so nothing remains but to CONQUER THE WORLD, and the only obstacle is the travel budget: the lair is expensive and the evil armies refuse to keep carving their relentless path of destruction across the puny nations without being paid. The nations of the world form a tree of transport routes, each route connecting two nations at a fixed cost per army that uses it and leaving exactly one way to travel between any two nations. Nation i currently hosts `x_i` armies and needs at least `y_i` stationed there in the final configuration. Move the armies into place as cheaply as possible and report the minimum total cost (statement).

The input is one integer n, 1 <= n <= 250000 nations by the problemset pdf, then n-1 route lines u, v, and c with 1 <= u, v <= n and 1 <= c <= 10^6, the cost per army on the bidirectional route, then n lines of two non-negative integers `x_i` and `y_i`, the armies now in nation i and the number that must end up there. The pdf promises the total army count is at least the total requirement and no more than 10^6. The output is the single minimum cost, under the pdf's 8 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `C-conquertheworld/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: nation 2 holds five spare armies, nation 3 is two short, and one spare rides 2->1->3 for 5+5 while nation 1's own spare army covers the second slot for 5, total 15.

input:

```
3
1 2 5
3 1 5
2 1
5 0
1 3
```

expected output:

```
15
```

Recognition: n <= 250000 nations with at most 10^6 armies total under the 8 second limit prices the job at one tree pass, and the convex tree dp runs O(n + X log^2 X) for X the army count, about 10^6 heap operations inside a log squared, while anything that routes armies one at a time cannot fit.

The cue is the movement language, move the armies as cheaply as possible against per-route costs and per-nation minimums, supply and demand carried on a shared tree, the min-cost flow shape with all capacity sitting on the nodes. Problem C is the 2018 face of the flow modeling on a network family, and the same flow modeling on a network cue drives chapter 10 problem J.

The wrong tool is a literal min-cost flow with unit augmentations, 10^6 armies each rerouted by an augmenting path over 250000 nodes for 2.5*10^11 node touches, and even the cheap naive greedy that matches heaps at every

least common ancestor dies on correctness, 16 failures in 300 small random trees, so cost and correctness kill the alternatives together.

With armies on the nodes of a weighted tree and a garrison requirement at every node, the task is a minimum-cost flow on a tree where only sources and sinks carry capacity. Book 10′s chapter 38, network flows ii, develops the min-cost flow formulation this task reduces to. The shipped vehicle in all seven languages is the convex tree dp from the sketch: each subtree keeps its cost function as marginal savings f, a non-increasing list stored as a low side below the split point and a high side above it, each a lazily shifted heap, plus a base cost, extending across an edge of cost c maps `f(a) = max(0, f(a) - sgn(delta+a)*c)`, and joins merge the two lists small-to-large, the extends O(n + X log X) and the joins O(n + X log^2 X) for X the total army count, with the answer read off at the root where delta = 0 (solutions.pdf pp. 2-4). A naive greedy that matches supply and demand heaps at every least common ancestor is wrong, it fails 16 of 300 small random trees while passing both official samples, and none of the seven solvers ships it.

The worked run: trace the model on sample 1. The printed tree is the chain 2-1-3 with both routes costing 5, nation 1 holds `x = 2` against `y = 1` for one spare, nation 2 holds 5 against 0 for five spares, and nation 3 holds 1 against 3 for a shortfall of two. The shortfall pulls two armies across the 3-1 route at 5 each, and the dp reads the marginal cost of each unit at node 1: its own spare crosses for 5, while a nation 2 spare crosses 2-1 for 5 then 1-3 for 5 more, 10 in total, so meeting both units costs 5 + 10 = 15 against 10 + 10 = 20 for two imported spares, and the trace ends at the printed answer 15.

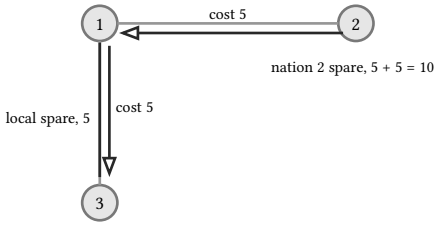


figure 111: sample 1′s two shipments: the local spare crosses one edge for 5, the nation 2 spare rides both edges for 10

```c
// insert one value among the klow largest (low side) or the rest
static void ctw_route(Ctw *t, long long v, long long c) {
  if (!t->high.n || v ≥ high_actual(t, 0)) {
    push_low(t, v, c);
    t->klow += c;
  } else {
    push_high(t, v, c);
  }
}

// extend t across an edge of the given cost toward a clean parent:
// rebalance materialized entries against the split p = max(0, -delta),
// then shift the two sides lazily by +-cost
static void ctw_extend(Ctw *t, long long cost) {
  long long p = -t->delta;
  if (p < 0)
    p = 0;
  long long v, c;
  while (t->nlow < p && t->high.n) {
    long long need = p - t->nlow;
    pop_high(t, &v, &c);
    long long take = c < need ? c : need;
    push_low(t, v, take);
    if (take < c)
      push_high(t, v, c - take);
  }
  while (t->nlow > p) {
    long long need = t->nlow - p; // captured before the pop: an
    pop_low(t, &v, &c);           // overshoot must still move `need`
    long long take = c < need ? c : need;
    push_high(t, v, take);
    if (take < c)
      push_low(t, v, c - take);
  }
  if (t->nlow < p)
    push_low(t, 0, p - t->nlow); // implicit zeros materialize
  t->klow = p;
  t->low.off += cost;
  t->high.off -= cost;
  if (t->delta ≥ 0)
    t->base += t->delta * cost;
  else
    t->base -= t->delta * cost;
}
```

listing 248: c: routing into the k-low largest, and extend's rebalance plus the two lazy shifts, runs carry multiplicities (source: icpc/samples-c/src/Ch09/pC.c, lines 124-167)

```go
// ctwExtend extends subtree t across an edge of the given cost to a
// clean parent: the split point moves to max(0, -delta), the two
// sides get +-cost lazy shifts, dead values clamp at zero when they
// surface, and the base grows by |delta|*cost. Positions below the
// split that hold no materialized value are zeros that the +cost
// shift must raise to cost, so they are materialized explicitly.
func ctwExtend(t *ctwNode, cost int) *ctwNode {
  p := -t.delta
  if p < 0 {
    p = 0
  }
  for len(t.low) < p && len(t.high) > 0 {
    v, _ := t.popHigh()
    t.pushLow(v)
  }
  for len(t.low) > p {
    v, _ := t.popLow()
    t.pushHigh(v)
  }
  for len(t.low) < p {
    t.pushLow(0)
  }
  t.kLow = p
  t.lowShift += int64(cost)
  t.highShift -= int64(cost)
  if t.delta ≥ 0 {
    t.base += int64(t.delta) * int64(cost)
  } else {
    t.base -= int64(t.delta) * int64(cost)
  }
  return t
}
```

listing 249: go: the extend map, split to `max(0, -delta)`, materialize implicit zeros, shift both sides (source: icpc/samples-go/ch09/pc.go, lines 182-213)

```
static void ctwExtend(Ctw t, long cost) {
  long p = -t.delta;
  if (p < 0)
    p = 0;
  long v, c;
  while (t.nlow < p && t.high.n > 0) {
    long need = p - t.nlow;
    popHigh(t);
    v = popScratch[0];
    c = popScratch[1];
    long take = Math.min(c, need);
    pushLow(t, v, take);
    if (take < c)
      pushHigh(t, v, c - take);
  }
  while (t.nlow > p) {
    long need = t.nlow - p; // captured before the pop: an overshoot
    popLow(t); // must still move `need`
    v = popScratch[0];
    c = popScratch[1];
    long take = Math.min(c, need);
    pushHigh(t, v, take);
    if (take < c)
      pushLow(t, v, c - take);
  }
  if (t.nlow < p)
    pushLow(t, 0, p - t.nlow); // implicit zeros materialize
  t.klow = p;
  t.low.off += cost;
  t.high.off -= cost;
  if (t.delta ≥ 0)
    t.base += t.delta * cost;
  else
    t.base -= t.delta * cost;
}
```

listing 250: java: extend's rebalance against the split point plus the two lazy shifts, runs carry multiplicities on a hand-rolled heap (source: icpc/samples-java/src/Ch10/PC.java, lines 160-194)

```
        public void Extend(long w)
        {
            B += Math.Abs(Delta) * w;
            SigA += w;
            SigB -= w;
            while (BSet.Count > 0 && BSet.Min.V + SigB ≤ 0)
                BSet.Remove(BSet.Min);
            long p = -Delta;
            if (ACnt < p && BSet.Count == 0)
                for (long q = ACnt; q < p; q++)
                {
                    A.Enqueue(w - SigA, w - SigA);
                    ACnt++;
                }
        }

        public void Absorb(Sub s)
        {
            while (s.A.TryDequeue(out long raw, out _))
            {
                long v = raw + s.SigA - SigB;
                BSet.Add((v, _tie++));
            }
            foreach (var (v0, _) in s.BSet)
                BSet.Add((v0 + s.SigB - SigB, _tie++));
            B += s.B;
            Delta += s.Delta;
            Rebalance();
        }
```

listing 251: c#: BCL PriorityQueue and SortedSet, lazy shifts SigA and SigB, Extend clamps dead highs away (source: icpc/samples/src/Ch09/PC.cs, lines 20-48)

```
// extend subtree t across an edge of the given cost to a clean
// parent: the split point moves to max(0, -delta), the two sides get
// +-cost lazy shifts, dead values clamp at zero when they surface,
// and the base grows by |delta|*cost. positions below the split that
// hold no materialized value are zeros that the +cost shift must
// raise to cost, so they are materialized explicitly.
function extend(t, cost) {
  let p = -t.delta;
  if (p < 0) p = 0;
  while (t.low.length < p && t.high.length > 0) t.pushLow(t.popHigh());
  while (t.low.length > p) t.pushHigh(t.popLow());
  while (t.low.length < p) t.pushLow(0);
  t.kLow = p;
  t.lowShift += cost;
  t.highShift -= cost;
  t.base += BigInt(Math.abs(t.delta)) * BigInt(cost);
  return t;
}

// merge b into the larger of the two nodes, small-to-large
function join(a, b) {
  if (a.low.length + a.high.length < b.low.length + b.high.length) [a, b] = [b, a];
  a.delta += b.delta;
  a.base += b.base;
  for (;;) {
    const v = b.popLow();
    if (v === null) break;
    a.route(v);
  }
  for (;;) {
    const v = b.popHigh();
    if (v === null) break;
    a.route(v);
  }
  // b's implicit low zeros follow its own entries
  const implicit = b.kLow - b.low.length;
  for (let i = 0; i < implicit; i++) a.route(0);
  return a;
}
```

listing 252: javascript: extend and join, the cost accumulator is BigInt because 2.5e17 exceeds 2^53 (source: icpc/samples-js/src/ch09-pc-conquertheworld.mjs, lines 121-159)

```
    def extend(self, cost):
        p = -self.delta if self.delta < 0 else 0
        while len(self.low) < p and self.high:
            self.push_low(self.pop_high())
        while len(self.low) > p:
            self.push_high(self.pop_low())
        while len(self.low) < p:
            self.push_low(0)
        self.k_low = p
        self.low_shift += cost
        self.high_shift -= cost
        if self.delta ≥ 0:
            self.base += self.delta * cost
        else:
            self.base -= self.delta * cost

    def route(self, v):
        if not self.high or v ≥ self.high_top():
            self.push_low(v)
            self.k_low += 1
        else:
            self.push_high(v)

    def answer(self):
```

listing 253: python: extend, route, answer, and the small-to-large join over heapq lists, one file, no class ceremony (source: icpc/samples-py/src/Ch09/pc.py, lines 41-64)

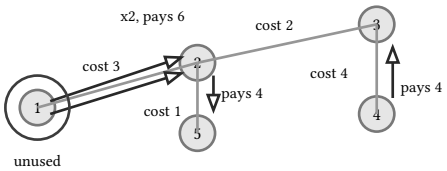
```lua
-- extend across an edge of the given cost to a clean parent: the
-- split moves to max(0, -delta), both sides shift lazily, dead
-- values clamp at zero when they surface, and the base grows by
-- |delta|*cost. low-side zeros materialize so the +cost shift can
-- raise them.
local function extend(t, cost)
  local p = -t.delta
  if p < 0 then p = 0 end
  while #t.low < p and #t.high > 0 do
    push_low(t, pop_high(t))
  end
  while #t.low > p do
    push_high(t, pop_low(t))
  end
  while #t.low < p do
    push_low(t, 0)
  end
  t.k_low = p
  t.low_shift = t.low_shift + cost
  t.high_shift = t.high_shift - cost
  if t.delta ≥ 0 then
    t.base = t.base + t.delta * cost
  else
    t.base = t.base - t.delta * cost
  end
  return t
end

-- insert one value into the low side when it belongs among the
-- k_low largest, else the high side
local function route(t, v)
  local top = t.high[1]
  if top == nil then
    push_low(t, v)
    t.k_low = t.k_low + 1
    return
  end
  local actual = top + t.high_shift
  if actual < 0 then actual = 0 end
  if v ≥ actual then
    push_low(t, v)
    t.k_low = t.k_low + 1
    return
  end
  push_high(t, v)
end
```

listing 254: lua: the same extend and route on hand-rolled 1-based heaps, post-order over an explicit stack (source: icpc/samples-lua/ch09_pc.lua, lines 82-127)

The crafted fixture is the 5-node tree below with answer 14, verified against an exact assignment brute force during spec derivation: two armies ride 1->2 at cost 3 each, one rides on to node 5 for 4 more, node 4's spare covers node 3 for 4, and covering node 3 from node 1 instead would cost 15. The suites pin `14\n` plus a leaf-to-leaf case, a surplus-stays-home case, a balanced singleton at `0\n`, and a chain: three CHECKs in c, three [Fact] methods in c#, five table cases in go, five ok lines in java, five assertions in javascript, five checks in python, five `T.eq` blocks in lua. Integer notes: costs and answers are int64 everywhere with the judge maximum at 2.5e17, javascript accumulates the base cost and the answer in BigInt and prints through `toString`, and every other language rides its native 64-

bit type, c and java packing run multiplicities so a 1e6-army node costs one heap entry per distinct value, java's on a hand-rolled binary heap of value-and-count entries.



2*3 + 4 + 4 = 14, the brute force agrees

figure 112: the fixture's optimal shipment: two armies down the cost-3 edge, one continuing to node 5, node 4's spare crossing to node 3, one army never leaves

11.5. problem D: gem island

Gem Island's n inhabitants each woke up one morning holding a single sparkling gem that had appeared overnight, and every night afterwards exactly one gem on the island, uniformly random among all the gems there, split into two. After d nights the holdings vary widely, some islanders rich and many with few, and the elders want to know whether pure chance explains the spread before tensions rise. List the final holdings in non-increasing order `a_1` ≥ `a_2` ≥ … ≥ `a_n` and report the expected value of `a_1 + … + a_r`, the gems held collectively by the r richest islanders (statement).

The input is a single line with three integers by the problemset pdf, n, d, and r, with 1 <= n, d <= 500 and 1 <= r <= n. The output is the expected number of gems the top r islanders hold after d nights, an absolute or relative error of at most 1e-6 accepted, under the pdf's 3 second limit, and the harness verifies it in float mode, `2018/D` pinned to `float` in `ref/icpc/compare.json`.

Official sample 1, reprinted byte for byte from the judge data pair `D-gemisland/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: three splits over two islanders leave holdings (4,1), (3,2), (2,3), (1,4), the four compositions of 3 extra gems over 2 islanders each equally likely, so the richest holds 4, 3, 3, 4 in turn and the expectation is exactly 3.5.

input:

```
2 3 1
```

expected output:

```
3.5
```

Recognition: n and d both at most 500 under the 3 second limit leave room for the O(n^2 d) memo table, 1.25*10^8 float terms at the corner, and nothing more, while the family it sums over holds C(n+d-1, d) compositions, C(999,500) near 1e299 at the bound, a count no enumeration ever touches.

The cue is the word expected over the r richest holdings, an expectation over an astronomically large implicit family, exactly the shape a recurrence answers once every composition is known to be equally likely, the theorem the sketch proves by induction. Problem D is the 2018 face of the counting and expectation without enumeration family, and the same counting and expectation without enumeration cue drives chapter 11 problem I, chapter 13 problem Q, chapter 14 problem E, and chapter 14 problem K.

The priced-out alternative is enumeration itself, of compositions or of the d split sequences, and Monte Carlo simulation is just as dead, 1e299 outcomes sampled nowhere near tightly enough for a 1e-6 answer.

The sketch proves by induction that every composition of the d extra gems over the n islanders is equally likely, so the answer is a top-r sum over all `C(n+d-1, d)` compositions divided by that count. Write S(n,d) for that un-

normalized sum: it collapses to `(n+d)*C(n+d-1,d)` when n <= r, because then everyone counts, and otherwise `S(n,d) = r*C(n+d-1,d) + sum_g C(n,g)*S(n-g, d-n+g)`, a memoized recursion over states with `a+b` ≤ `n+d`, O(n^2 d) terms (solutions.pdf pp. 5-6). The shipped recursion is the normalized form `g(a,b) = r + sum coef(x) g(a-x, b-a+x)` with `coef(x) = C(a,x) C(b-1,a-x-1) / C(a+b-1,b)`. All seven languages evaluate it in floats with log-space binomials from a shared ln-factorial table, coefficients in [0,1] summing to 1, so the total error stays near 1e-11 against a 1e-6 bound. Book 10′s chapter 35, combinatorics, builds the log-space binomial table and the stars-and-bars counts this recursion draws from.

The worked run: trace the model on sample 1. With n = 2, d = 3 and r = 1 the equally-likely-compositions theorem is small enough to see whole, the four compositions of the 3 extra gems and their richest holdings in the table below, and the recursion compresses exactly that average. At the state (a,b) = (2,3) the two coefficients are `C(2,0)*C(2,1)/C(4,3) = 2/4` and `C(2,1)*C(2,0)/C(4,3) = 2/4`, so `g(2,3) = 1 + 0.5*g(2,1) + 0.5*g(1,2)`. The branch `g(1,2)` is the everyone-counts base, a <= r, where the total holding is the deterministic `a + b = 3`, and `g(2,1)` folds once more with its single coefficient `C(2,1)*C(0,0)/C(2,1) = 1`, giving `g(2,1) = 1 + 1*g(1,0) = 1 + 1 = 2`. So `g(2,3) = 1 + 0.5*2 + 0.5*3 = 3.5`, the same average the four compositions hand out, and the trace ends at the printed answer `3.5`.

extra gems	holdings	richest holds
(3,0)	(4,1)	4
(2,1)	(3,2)	3
(1,2)	(2,3)	3
(0,3)	(1,4)	4
expectation	4 compositions	(4+3+3+4)/4 = 3.5

```
static double G(int a, int b) {
  if (a ≤ R)
    return a + b;
  if (a == 0)
    return 0;
  if (seen[a][b])
    return memo[a][b];
  double lden = lnC(a + b - 1, b);
  double res = R;
  int k0 = a - b > 0 ? a - b : 0;
  for (int k = k0; k ≤ a - 1; k++) {
    double c = exp(lnC(a, k) + lnC(b - 1, a - k - 1) - lden);
    res += c * G(a - k, b - a + k);
  }
  if (b == 0) // everyone collapses to one gem: k = a, G(0,0) = 0
    ;
  seen[a][b] = 1;
  memo[a][b] = res;
  return res;
}
```

listing 255: c: the memoized G over `seen` and `memo`, binomials through the lnfact header's lnC (source: icpc/samples-c/src/Ch09/pD.c, lines 17-36)

```
var g func(a, b int) float64
g = func(a, b int) float64 {
  if a <= r {
    return float64(a + b)
  }
  if b == 0 {
    return float64(r)
  }
  if memo[a][b] ≥ 0 {
    return memo[a][b]
  }
  total := float64(r)
  lnDen := lnC(a+b-1, b)
  // x = islanders stuck at one gem; the rest keep splitting,
  // so x runs where C(b-1, a-x-1) is nonzero
  lo, hi := 0, a-1
  if a-b > lo {
    lo = a - b
  }
  for x := lo; x <= hi; x++ {
    c := lnC(a, x) + lnC(b-1, a-x-1) - lnDen
    if c > -700 {
      total += math.Exp(c) * g(a-x, b-a+x)
    }
  }
  memo[a][b] = total
  return total
}
return fmt.Sprintf("%.7f\n", g(n, d))
```

listing 256: go: the closure g over a −1-filled memo table, exponent guard at −700 (source: icpc/samples-go/ch09/pd.go, lines 26-54)

```
static double G(int a, int b) {
  if (a ≤ r)
    return a + b;
  if (a == 0)
    return 0;
  if (seen[a][b])
    return memo[a][b];
  double lden = Lnfact.lnC(a + b - 1, b);
  double res = r;
  int k0 = Math.max(a - b, 0);
  for (int k = k0; k ≤ a - 1; k++) {
    double c = Math.exp(Lnfact.lnC(a, k) + Lnfact.lnC(b - 1, a - k - 1) - lden);
    res += c * G(a - k, b - a + k);
  }
  // when b == 0 everyone collapses to one gem: k = a, G(0,0) = 0
  seen[a][b] = true;
  memo[a][b] = res;
  return res;
}
```

listing 257: java: the memoized G over `seen` and `memo`, binomials through the Lnfact helper's lnC (source: icpc/samples-java/src/Ch10/PD.java, lines 32-50)

```
        // normalized recursion g(a,b) over weak compositions, log-space binomials
        int maxN = n, maxD = d;
        var lnf = new double[n + d + 2];
        for (int i = 2; i < lnf.Length; i++)
            lnf[i] = lnf[i - 1] + Math.Log(i);
        double LnC(int a, int b)
        {
            if (b < 0 || b > a)
                return double.NegativeInfinity;
            return lnf[a] - lnf[b] - lnf[a - b];
        }
        var memo = new double[maxN + 1, maxD + 1];
        var done = new bool[maxN + 1, maxD + 1];
        double G(int a, int b)
        {
            if (a ≤ r)
                return a + b;
            if (b == 0)
                return r;
            if (done[a, b])
                return memo[a, b];
            double sum = r;
            int lo = Math.Max(0, a - b);
            for (int g = lo; g ≤ a - 1; g++)
            {
                double coef = Math.Exp(
                    LnC(a, g) + LnC(b - 1, a - g - 1) - LnC(a + b - 1, b));
                sum += coef * G(a - g, b - a + g);
            }
            memo[a, b] = sum;
            done[a, b] = true;
            return sum;
        }
        double ans = G(n, d);
        return ans.ToString("F7", CultureInfo.InvariantCulture) + "\n";
```

listing 258: c#: local function LnC over an inline table, recursion G with the same guard rails (source: icpc/samples/src/Ch09/PD.cs, lines 12-46)

```
export function solve(input) {
  const [n, d, r] = input.match(/\d+/g).map(Number);
  const memo = new Float64Array((n + 1) * (d + 1)).fill(-1);
  const at = (a, b) => a * (d + 1) + b;
  const g = (a, b) => {
    if (a ≤ r) return a + b;
    if (b === 0) return r;
    const key = at(a, b);
    if (memo[key] ≥ 0) return memo[key];
    let total = r;
    const lnDen = lnC(a + b - 1, b);
    // x = islanders stuck at one gem; the rest keep splitting, so x
    // runs where C(b-1, a-x-1) is nonzero
    let lo = 0;
    if (a - b > lo) lo = a - b;
    for (let x = lo; x ≤ a - 1; x++) {
      const c = lnC(a, x) + lnC(b - 1, a - x - 1) - lnDen;
      if (c > -700) total += Math.exp(c) * g(a - x, b - a + x);
    }
    memo[key] = total;
    return total;
  };
  return `${g(n, d).toFixed(7)}\n`;
}
```

listing 259: javascript: solve with the imported lnC, one flat Float64Array memo (source: icpc/samples-js/src/ch09-pd-gemisland.mjs, lines 12-35)

```
def solve(text):
    data = text.split()
    n, d, r = int(data[0]), int(data[1]), int(data[2])
    lnf = [0.0] * (n + d + 2)
    for i in range(2, n + d + 2):
        lnf[i] = lnf[i - 1] + log(i)

    def ln_c(a, b):
        if b < 0 or b > a:
            return NEG
        return lnf[a] - lnf[b] - lnf[a - b]

    g = [[0.0] * (d + 1) for _ in range(n + 1)]
    for a in range(r + 1):
        row = g[a]
        for b in range(d + 1):
            row[b] = a + b
    for a in range(r + 1, n + 1):
        g[a][0] = r
    for b in range(1, d + 1):
        for a in range(r + 1, n + 1):
            total = r
            ln_den = ln_c(a + b - 1, b)
            lo = a - b if a - b > 0 else 0
            row = g[a]
            for x in range(lo, a):
                c = ln_c(a, x) + ln_c(b - 1, a - x - 1) - ln_den
                if c > -700:
                    total += exp(c) * g[a - x][b - a + x]
            row[b] = total
    return f"{g[n][d]:.7f}\n"
```

listing 260: python: the table filled by increasing b so the recursion never nests, lnC inline (source: icpc/samples-py/src/Ch09/pd.py, lines 14-44)

```
local function solve(input)
  local n, d, r
  local it = input:gmatch("%S+")
  n, d, r = tonumber(it()), tonumber(it()), tonumber(it())
  local memo = {}
  local W = d + 1
  local g
  g = function(a, b)
    if a ≤ r then return a + b end
    if b == 0 then return r end
    local key = a * W + b
    local hit = memo[key]
    if hit then return hit end
    local total = r + 0.0
    local ln_den = lnfact.ln_c(a + b - 1, b)
    local lo, hi = 0, a - 1
    if a - b > lo then lo = a - b end
    for x = lo, hi do
      local c = lnfact.ln_c(a, x) + lnfact.ln_c(b - 1, a - x - 1) - ln_den
      if c > -700 then
        total = total + math.exp(c) * g(a - x, b - a + x)
      end
    end
    memo[key] = total
    return total
  end
  return string.format("%.7f\n", g(n, d))
end
```

listing 261: lua: the required lnfact module, flat (n+1)x(d+1) memo, allocation-free inner loop (source: icpc/samples-lua/ch09_pd.lua, lines 13-40)

The fixture `3 2 1` walks all six compositions of 2 extra gems over 3 islanders, the (3,1,1) family three times and the (2,2,1) family three times, so the richest holds 3 or 2 and the expectation is exactly `2.5000000`. The suites pin that value and the three official samples as printed-tolerance cases: four CHECKs in c, two [Fact] methods in c# whose second runs all three samples, four table cases in go, four ok lines in java, four assertions in javascript, four checks in python, four `T.eq` blocks in lua, the shared extras being all-counted, single-islander, and small split cases. Integer notes are the warning instead of a divergence: the raw binomials must never be built, `C(999,500)` sits near 1e299 and overflows a double before any division, which is the entire reason the helper exists.

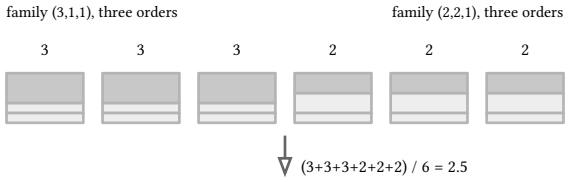


figure 113: the fixture's six compositions as bar stacks, the richest islander's holding shaded, expectation 2.5 across the family

11.6. problem E: getting a jump on crime

Robin the superhero patrols the hometown by jumping from roof to roof, and the hometown is a square grid of dx by dy blocks, each block one w by w meter building with its own height. Every jump goes from the center of one roof to the center of another, not necessarily adjacent, leaves the ground at one fixed takeoff speed v whose horizontal and vertical components trade off as `vd^2 + vh^2 = v^2`, flies with constant horizontal velocity under gravity g = 9.80665 m/s^2 with the cape handling air resistance, and may not collide with any building along the way, where a jump passing over a corner at which four buildings meet must clear all four. Robin picks the takeoff

angle and cannot change direction mid-air, so from the secret hideout's roof the task is the minimum jump count to every other roof, or X when no chain of jumps reaches it (statement).

The input is one line of six integers by the problemset pdf, dx, dy, w, v, and the hideout coordinates lx, ly, with 1 <= dx, dy <= 20, the building width 1 <= w <= 10^3 meters, the takeoff velocity 1 <= v <= 10^3 meters per second, and 1 <= lx <= dx, 1 <= ly <= dy, then dy lines of dx non-negative integer heights in meters, each at most 10^3, line j carrying buildings (1, j) through (dx, j). The pdf guarantees the answers would not change if any building's height moved by up to 1e-6, so borderline geometry never occurs. The output is dy lines of dx values in input order, each the minimum jump count from the hideout or X, under the pdf's 2 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `E-gettingjump/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints and draws in its cross-section figure: with 100 m blocks and v = 55 the flat range is 3025/9.80665 = 308.46 m, so the 300 m jump from the height 10 hideout roof to the far roof exists, its green arc in the figure clearing the 40 m and 60 m roofs mid-flight, and every roof costs one jump.

input:

```
4 1 100 55 1 1
10 40 60 10
```

expected output:

```
0 1 1 1
```

Recognition: dx and dy at most 20 mean at most 400 roofs and 79800 ordered pairs, each pair's feasibility one quadratic and each clearance walk O(dx+dy) binding instants, 3.2*10^6 checks inside the 2 second limit, an O(dx^3 dy^3) worst case of 6.4*10^7 that still fits, so the budget invites enumerating every roof pair outright.

The cue is the physics sentence, one fixed takeoff speed whose components trade off, which hands each jump's feasibility to a quadratic in the takeoff angle, finitely many critical candidates per pair instead of a continuum of arcs. Problem E is the 2018 face of the geometric critical-point enumeration family, and the same geometric critical-point enumeration cue drives chapter 10 problem A, chapter 10 problem D, chapter 11 problem G, chapter 13 problem T, and chapter 14 problem D; a secondary thread, bfs on the implicit jump graph, covers reaching the door once jump feasibility is known.

The priced-out wrong tool is a continuous search over takeoff angles, fine sampling or ternary search per pair: the clearance margin along an arc family has no single peak to converge on, and 79800 pairs times a 1000-step angle scan times the 40-instant walk is 3.2*10^9 float evaluations, an order past the limit for nothing the closed form does not give exactly.

The jump graph connects two roof centers when a parabola clears every other building. The takeoff condition is a quadratic in `x = vd^2/v^2`, read off the height equation `dh = (d/vd)*sqrt(v^2 - vd^2) - g*d^2/(2*vd^2)`: the steeper arc is the one that matters, and the managed languages recover it through the larger root and the root product because the smaller root itself cancels catastrophically when the roots straddle 0 and 1, while c and java solve for `vd^2` directly and take the smaller root, and c# front-loads the closed-form existence test `v^2 ≥ g*(hypot(d,dh)+dh)` and still solves the quadratic for the arc it flies, in every case leaning on the fact that the steepest feasible arc sits pointwise above every other (solutions.pdf pp. 6-7). Concavity makes the first and last instant over each footprint the binding checks, the graph build costing O(dx^2 dy^2 (dx+dy)) or O(dx^3 dy^3), and a breadth-first search over the resulting graph is the whole solver past that.

The worked run: trace the model on sample 1. The hideout roof sits at height 10 with the row's other centers 100, 200 and 300 meters out at heights 40, 60 and 10, v = 55 gives `v^2 = 3025`, and the existence test `v^2 ≥ g*(hypot(d,dh)+dh)` prices each jump in one line. Roof 2: `hypot(100,30) = sqrt(10900) = 104.403`, so `9.80665*(104.403+30) = 1318.0 ≤ 3025`. Roof 3: `hypot(200,50) = sqrt(42500) = 206.155`, so `9.80665*(206.155+50) = 2512.1 ≤`

3025. Roof 4: `hypot(300,0) = 300`, so `9.80665*300 = 2942.0` $\leq$
3025. All three jumps exist from the hideout, the steepest arc clearing the 40 m and 60 m footprints mid-flight by concavity, so the bfs opens all three roofs at depth 1 and no roof needs a second jump, and the trace ends at the printed answer `0 1 1 1`.

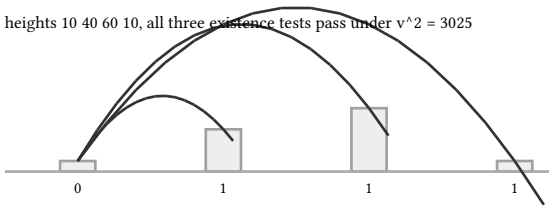


figure 114: sample 1's cross-section: all three jumps exist from the height 10 hideout, the steepest arcs clearing the 40 m and 60 m roofs mid-flight

```c
// solve vd^2 for horizontal distance d and height difference dh;
// returns the high arc's vd (larger vh), or -1 if impossible
static double launch_vd(double d, double dh) {
  // dh = (d/√x)·sqrt(v²-x) - g·d²/(2x), x = vd²
  double A = d * d + dh * dh;
  double B = G * d * d * dh - d * d * V * V;
  double C = G * G * d * d * d * d / 4.0;
  double disc = B * B - 4 * A * C;
  if (disc < 0)
    return -1;
  double sq = sqrt(disc);
  double x1 = (-B - sq) / (2 * A), x2 = (-B + sq) / (2 * A);
  double x = x1 < x2 ? x1 : x2; // smaller vd: the higher arc
  if (x ≤ 0 || x > V * V)
    return -1;
  return sqrt(x);
}

// height of the arc at horizontal distance s from the source
static inline double arc_y(double vd, double s, double y0) {
  double vh = sqrt(V * V - vd * vd);
  return y0 + vh * (s / vd) - G * (s / vd) * (s / vd) / 2.0;
}
```

listing 262: c: launch_vd solves the quadratic for `vd^2` and takes the smaller root, arc_y evaluates the parabola (source: icpc/samples-c/src/Ch09/pE.c, lines 20-42)

```go
func canJump(a, b, dx int, h []float64, w, v, g float64) bool {
  ax, ay := float64(a%dx), float64(a/dx)
  bx, by := float64(b%dx), float64(b/dx)
  x1, y1 := (ax+0.5)*w, (ay+0.5)*w
  x2, y2 := (bx+0.5)*w, (by+0.5)*w
  d := math.Hypot(x2-x1, y2-y1)
  dh := h[b] - h[a]
  bigG := g * d * d / (2 * v * v)
  aa := d*d + dh*dh
  bb := 2*dh*bigG - d*d
  disc := bb*bb - 4*aa*bigG*bigG
  if disc < 0 {
    return false
  }
  // the smaller root suffers catastrophic cancellation when the
  // two roots straddle near 0 and 1; take it through the stable
  // larger root and the product of roots
  xPlus := (-bb + math.Sqrt(disc)) / (2 * aa)
  if xPlus <= 0 || xPlus > 1+1e-12 {
    return false
  }
  x := (bigG * bigG) / (aa * xPlus)
  if x <= 1e-12 || x > 1 {
    return false
  }
  vd := v * math.Sqrt(x)
  heightAt := func(dist float64) float64 {
    t := dist / vd
    return h[a] + v*math.Sqrt(1-x)*t - g*t*t/2
  }
  // every building the path passes over: the steep arc must clear
  // the takeoff roof's far half, intervening buildings, and the
  // landing roof's near half (corner points count for all four
  // adjacent buildings)
  for k := range h {
    t1, t2, ok := clipPathToCell(x1, y1, x2, y2, float64(k%dx), float64(k/dx), w)
    if !ok {
      continue
    }
    // 1e-6 keeps float cancellation at the landing point from
    // rejecting exact landings; the statement guarantees answers
    // are stable under that perturbation
    if heightAt(t1*d) < h[k]-1e-6 || heightAt(t2*d) < h[k]-1e-6 {
      return false
    }
  }
  return true
}
```

listing 263: go: canJump with the stable-root recovery and the two binding instants per footprint (source: icpc/samples-go/ch09/pe.go, lines 76-123)

```java
static double launchVd(double d, double dh) {
  // dh = (d/sqrt(x))·sqrt(v^2-x) - g·d^2/(2x), x = vd^2
  double A = d * d + dh * dh;
  double B = GRAV * d * d * dh - d * d * launchV * launchV;
  double C = GRAV * GRAV * d * d * d * d / 4.0;
  double disc = B * B - 4 * A * C;
  if (disc < 0)
    return -1;
  double sq = Math.sqrt(disc);
  double x1 = (-B - sq) / (2 * A), x2 = (-B + sq) / (2 * A);
  double x = Math.min(x1, x2); // smaller vd: the higher arc
  if (x ≤ 0 || x > launchV * launchV)
    return -1;
  return Math.sqrt(x);
}

// height of the arc at horizontal distance s from the source
static double arcY(double vd, double s, double y0) {
  double vh = Math.sqrt(launchV * launchV - vd * vd);
  return y0 + vh * (s / vd) - GRAV * (s / vd) * (s / vd) / 2.0;
}
```

listing 264: java: launchVd solves the quadratic for vd^2 and takes the smaller root, arcY evaluates the parabola (source: icpc/samples-java/src/Ch10/PE.java, lines 37-57)

```
    // a jump of horizontal distance d and rise dh exists iff
    // v^2 ≥ g*(hypot(d, dh) + dh); the steepest feasible arc is pointwise above
    // every other, so checking it against intervening buildings decides existence
    private static bool JumpClears(int ci, int cj, int i2, int j2, double[,] h,
        int dx, int dy, double w, double v)
    {
        double d = w * Math.Sqrt((i2 - ci) * (i2 - ci) + (j2 - cj) * (j2 - cj));
        if (d == 0)
            return false;
        double rise = h[j2, i2] - h[cj, ci];
        if (v * v < G * (Math.Sqrt(d * d + rise * rise) + rise))
            return false;
        double k = G * d * d / (2 * v * v);
        double disc = d * d - 4 * k * (rise + k);
        if (disc < 0)
            return false;
        // steep root: larger vertical speed
        double a = (d + Math.Sqrt(disc)) / (2 * k);
        double b = G * (1 + a * a) / (2 * v * v);
        double y0 = h[cj, ci];

        // the steepest arc is pointwise above every other feasible arc, so
        // checking it against every roof (including source and destination,
        // with equality at the endpoints) decides the jump
        for (int j = 0; j < dy; j++)
            for (int i = 0; i < dx; i++)
            {
                // arc parameter interval where the jump segment is over the
                // building footprint: intersect both axis slabs
                double sx0 = (ci + 0.5) * w, sx1 = (i2 + 0.5) * w;
                double sy0 = (cj + 0.5) * w, sy1 = (j2 + 0.5) * w;
                double lo, hi;
                if (!Slab(sx0, sx1, i * w, (i + 1) * w, d,
                        out double sxLo, out double sxHi)
                    || !Slab(sy0, sy1, j * w, (j + 1) * w, d,
                        out double syLo, out double syHi))
                    continue;
                lo = Math.Max(0, Math.Max(sxLo, syLo));
                hi = Math.Min(d, Math.Min(sxHi, syHi));
                if (lo > hi + 1e-9)
                    continue;
                // concave margin: binding at the interval ends
                if (y0 + a * lo - b * lo * lo < h[j, i] - 1e-9
                    || y0 + a * hi - b * hi * hi < h[j, i] - 1e-9)
                    return false;
            }
        return true;
    }
```

listing 265: c#: the closed-form existence test, steepest arc `y0 + a*s - b*s^2`, slab clipping per footprint (source: icpc/samples/src/Ch09/PE.cs, lines 84-131)

```
function canJump(a, b, dx, h, w, v) {
  const ax = a % dx, ay = Math.floor(a / dx);
  const bx = b % dx, by = Math.floor(b / dx);
  const x1 = (ax + 0.5) * w, y1 = (ay + 0.5) * w;
  const x2 = (bx + 0.5) * w, y2 = (by + 0.5) * w;
  const d = Math.hypot(x2 - x1, y2 - y1);
  const dh = h[b] - h[a];
  const bigG = (G * d * d) / (2 * v * v);
  const aa = d * d + dh * dh;
  const bb = 2 * dh * bigG - d * d;
  const disc = bb * bb - 4 * aa * bigG * bigG;
  if (disc < 0) return false;
  // the smaller root suffers catastrophic cancellation when the two
  // roots straddle near 0 and 1; take it through the stable larger
  // root and the product of roots
  const xPlus = (-bb + Math.sqrt(disc)) / (2 * aa);
  if (xPlus ≤ 0 || xPlus > 1 + 1e-12) return false;
  const x = (bigG * bigG) / (aa * xPlus);
  if (x ≤ 1e-12 || x > 1) return false;
  const vd = v * Math.sqrt(x);
  const heightAt = (dist) => {
    const t = dist / vd;
    return h[a] + v * Math.sqrt(1 - x) * t - (G * t * t) / 2;
  };
  // every building the path passes over: the steep arc must clear the
  // takeoff roof's far half, intervening buildings, and the landing
  // roof's near half (corner points count for all four adjacent
  // buildings). 1e-6 keeps float cancellation at the landing point
  // from rejecting exact landings; the statement guarantees answers
  // are stable under that perturbation
  for (let k = 0; k < h.length; k++) {
    const clip = clipPathToCell(
      x1, y1, x2, y2, k % dx, Math.floor(k / dx), w,
    );
    if (clip === null) continue;
    if (heightAt(clip[0] * d) < h[k] - 1e-6 ||
      heightAt(clip[1] * d) < h[k] - 1e-6) {
      return false;
    }
  }
  return true;
}
```

listing 266: javascript: the same canJump, endpoint clearance at 1e-6 as the statement's stability bound demands (source: icpc/samples-js/src/ch09-pe-gettingjump.mjs, lines 43-84)

```
def _can_jump(a, b, dx, h, w, v):
    ax, ay = a % dx, a // dx
    bx, by = b % dx, b // dx
    x1, y1 = (ax + 0.5) * w, (ay + 0.5) * w
    x2, y2 = (bx + 0.5) * w, (by + 0.5) * w
    d = hypot(x2 - x1, y2 - y1)
    dh = h[b] - h[a]
    big = G * d * d / (2 * v * v)
    aa = d * d + dh * dh
    bb = 2 * dh * big - d * d
    disc = bb * bb - 4 * aa * big * big
    if disc < 0:
        return False
    # the smaller root suffers cancellation when the roots straddle
    # near 0 and 1; take it through the stable larger root
    x_plus = (-bb + sqrt(disc)) / (2 * aa)
    if x_plus ≤ 0 or x_plus > 1 + 1e-12:
        return False
    x = big * big / (aa * x_plus)
    if x ≤ 1e-12 or x > 1:
        return False
    vd = v * sqrt(x)
    sq = v * sqrt(1 - x)
    ha = h[a]

    def height_at(dist):
        t = dist / vd
        return ha + sq * t - G * t * t / 2

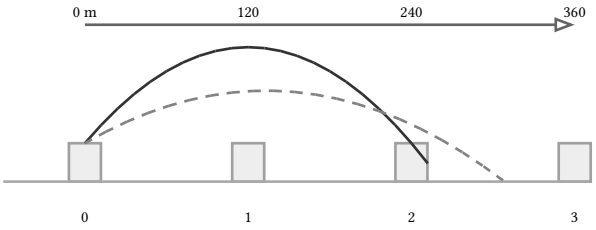
    for ky in range(min(ay, by), max(ay, by) + 1):
        base = ky * dx
        for kx in range(min(ax, bx), max(ax, bx) + 1):
            k = base + kx
            span = _clip(x1, y1, x2, y2, kx, ky, w)
            if span is None:
                continue
            if height_at(span[0] * d) < h[k] - 1e-6:
                return False
            if height_at(span[1] * d) < h[k] - 1e-6:
                return False
    return True
```

listing 267: python: `_can_jump` with the product-of-roots recovery, bounding-box footprint scan (source: icpc/samples-py/src/Ch09/pe.py, lines 39-79)

```lua
local function can_jump(a, b)
  local ax, ay = (a - 1) % dx, (a - 1) // dx
  local bx, by = (b - 1) % dx, (b - 1) // dx
  local x1, y1 = (ax + 0.5) * w, (ay + 0.5) * w
  local x2, y2 = (bx + 0.5) * w, (by + 0.5) * w
  local ddx, ddy = x2 - x1, y2 - y1
  local d = math.sqrt(ddx * ddx + ddy * ddy)
  local dh = h[b] - h[a]
  local big_g = G * d * d / (2 * v * v)
  local aa = d * d + dh * dh
  local bb = 2 * dh * big_g - d * d
  local disc = bb * bb - 4 * aa * big_g * big_g
  if disc < 0 then return false end
  local x_plus = (-bb + math.sqrt(disc)) / (2 * aa)
  if x_plus ≤ 0 or x_plus > 1 + 1e-12 then return false end
  local x = (big_g * big_g) / (aa * x_plus)
  if x ≤ 1e-12 or x > 1 then return false end
  local vd = v * math.sqrt(x)
  local vh = v * math.sqrt(1 - x)
  local function height_at(dist)
    local t = dist / vd
    return h[a] + vh * t - G * t * t / 2
  end
  for k = 1, cells do
    local t1, t2 = clip_cell(x1, y1, x2, y2,
      (k - 1) % dx, (k - 1) // dx, w)
    if t1 then
      if height_at(t1 * d) < h[k] - 1e-6 or
         height_at(t2 * d) < h[k] - 1e-6 then
        return false
      end
    end
  end
  return true
end
```

listing 268: lua: can_jump over the shared clip_cell, 1-based grid arithmetic throughout (source: icpc/samples-lua/ch09_pe.lua, lines 22-56)

The fixture is one flat row, `4 1 120 55 1 1` with all heights 0, so only range matters: flat-ground reach is `v^2/g = 308.46` m, roofs sit 120, 240, and 360 m out, the first two jumps exist and a flat-to-flat parabola stays strictly above zero between its ends, and the expected output is `0 1 1 2`. Suites: three CHECKs in c, three [Fact] methods in c#, five table cases in go, five ok lines in java, five assertions in javascript, five checks in python, five `T.eq` blocks in lua, the shared extras a beyond-range tower, a wall between neighbors, a step-up past the apex, and a mid-row hideout. No integer notes, every value is a double and g is the statement's 9.80665 exactly, with the 1e-6 endpoint slack justified by the statement's own perturbation guarantee.



flat range v^2/g = 308.46 m, the 360 m jump does not exist

figure 115: cross-section of the flat fixture: the 240 m parabola exists, the 360 m attempt falls to the ground at the 308.46 m range limit

11.7. problem F: go with the flow

Typesetters call a river a string of spaces between words that runs down several lines of text, and celebrated river authority Flo Ng wants her rivers-of-the-world book to contain the longest typographic rivers possible. The text is set in a monospaced font, left-aligned in a column of some fixed width, words packed as tightly as possible on each line, never split across lines, with exactly one space between words and no right alignment, and the width is ours to choose, at least as long as the longest word. For Flo a river is a sequence of spaces in consecutive lines where each space after the first sits at most one column away from the space in the line above, trailing whitespace never joins one. Find the width that produces the longest river, the smallest such width on ties, and print the width and the river length (statement).

The input is one integer n, 2 <= n <= 2500 words by the problemset pdf, then the words on the following lines, letters only, a single space between words on the same line, and no word longer than 80 characters. The output is one line, the winning width, a space, and the longest river's length, under the pdf's 12 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `F-gowithflow/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints and sets twice in its own figure: the 21-word Yangtze text wraps at width 15 into a river of 5, one longer than width 14 manages, so the answer is 15 5.

input:

```
21
The Yangtze is the third longest
river in Asia and the longest in
the world to flow
entirely in one country
```

expected output:

```
15 5
```

Recognition: n <= 2500 words of at most 80 letters under the 12 second limit, and the naive all-widths sweep costs theta(n^2 L^2) column steps, `2500^2 * 80^2 = 4*10^10`, which time-limits, so the budget itself forces the sweep's two prunes, the line-count stop and the jump between wrap-changing widths.

The cue is the phrase the width is ours to choose, a single scalar parameter whose every value runs one full greedy simulation, the one-dimensional answer-space search shape over the widths from the longest word upward. Problem F is the 2018 face of the one-dimensional answer-space search family, and the same one-dimensional answer-space search cue drives chapter 10 problem E and chapter 15 problem C.

The priced-out alternative is the unpruned sweep itself, every width from 81 to the whole text on one line each re-wrapping all 2500 words, 4*10^10 column steps at the bound against the pruned sweep's few dozen wrap-changing widths, each still one linear river pass.

Wrapping is greedy, and the naive sweep over all widths costs theta(n^2 L^2) for L the longest word and time-limits, so the pinned vehicle carries the sketch's first and third fixes together: stop once the best river reaches the current line count, since wider widths only shrink the line count, and jump straight to the next width that changes the wrap, the smallest width at which any line would take one more word (solutions.pdf p. 7). Per width the river is one column dp pass, a space at column c of line k continuing any space at `c-1`, `c`, or `c+1` of line k-1.

The worked run: trace the model on sample 1. At the winning width 15 the greedy wrap packs the printed 21 words into eight lines, each line the most words that fit, `The Yangtze is` at 14 columns, `the third` at 9, `longest river` at 13, `in Asia and the` at 15, `longest in the` at 14, `world to flow` at 13, `entirely in one` at 15 and `country`, and the interior space columns per line are listed below. The column dp then walks them: the column 8 chain runs line 3 to line 5 at lengths 1, 2 and 3, steps to column 9 for line 6 at 4, holds column 9 for line 7 at 5, and every other column dies earlier, the column 4 chain at 2, the column 12 restart at 2, the column 3 restart at 1, so the river is 5.

The widths below 15 top out at 4, width 14′s own count in the sample walkthrough, and past 15 the wrap's line count only shrinks toward 5, where the stop rule ends the sweep with the best still 5, so the smallest winning width holds, and the trace ends at the printed answer `15 5`.

line	packed words	space columns	river ending on the line
1	The Yangtze is	4, 12	1
2	the third	4	2
3	longest river	8	1
4	in Asia and the	3, 8, 12	2
5	longest in the	8, 11	3
6	world to flow	6, 9	4
7	entirely in one	9, 12	5
8	country	none	0

```c
  int maxw = 0;
  for (int i = 0; i < nw; i++)
    if (wl[i] > maxw)
      maxw = wl[i];
  int ls[FN], L = 0;
  int bestw = -1, bestr = -1;
  int W = maxw;
  while (W ≤ 1000000) {
    int nextw;
    wrap(W, ls, &L, &nextw);
    int r = river(ls, L, W);
    if (r > bestr) {
      bestr = r;
      bestw = W;
    }
    if (bestr ≥ L) // wider widths have fewer lines: cannot improve
      break;
    if (nextw == 1 << 30)
      break;
    W = nextw;
  }
  char *out = malloc(32);
  snprintf(out, 32, "%d %d\n", bestw, bestr);
  for (int i = 0; i < nw; i++)
    free(words[i]);
  free(buf);
  return out;
```

listing 269: c: the width sweep with both prunes, wrap computing the next wrap-changing width as a side effect (source: icpc/samples-c/src/Ch09/pF.c, lines 100-126)

```go
// riverOf computes the longest river of one wrap: a space at column
// c of line k continues any line k-1 space at c-1, c, or c+1.
func riverOf(lengths, firsts, words, prev, cur []int) int {
  best := 0
  prevTouched := make([]int, 0, 64)
  for k := range lengths {
    curTouched := make([]int, 0, 16)
    end := len(words)
    if k+1 < len(firsts) {
      end = firsts[k+1]
    }
    col := 0
    for w := firsts[k]; w+1 < end; w++ {
      col += words[w]
      val := 1
      for dc := -1; dc <= 1; dc++ {
        if prev[col+dc]+1 > val {
          val = prev[col+dc] + 1
        }
      }
      cur[col] = val
      curTouched = append(curTouched, col)
      if val > best {
        best = val
      }
      col++
    }
    for _, c := range prevTouched {
      prev[c] = 0
    }
    prev, cur = cur, prev
    prevTouched = curTouched
  }
  for _, c := range prevTouched {
    prev[c] = 0
  }
  return best
}
```

listing 270: go: riverOf, the touched-column reset trick keeping the two row arrays reusable (source: icpc/samples-go/ch09/pf.go, lines 72-109)

```
  static int wrapInto(int width) {
    int lines = 0, i = 0;
    lsScratch[lines++] = 0;
    while (i < nw) {
      int j = i, len = wl[i];
      while (j + 1 < nw && len + 1 + wl[j + 1] ≤ width) {
        len += 1 + wl[j + 1];
        j++;
      }
      i = j + 1;
      if (i < nw)
        lsScratch[lines++] = i;
    }
    return lines;
  }

  static int nextChangeWidth(int width) {
    int i = 0, nextw = 1 << 30;
    while (i < nw) {
      int j = i, len = wl[i];
      while (j + 1 < nw && len + 1 + wl[j + 1] ≤ width) {
        len += 1 + wl[j + 1];
        j++;
      }
      if (j + 1 < nw) {
        int change = len + 1 + wl[j + 1];
        if (change < nextw)
          nextw = change;
      }
      i = j + 1;
    }
    return nextw;
  }
```

listing 271: java: the wrap split into wrapInto and nextChangeWidth, the same greedy walk read as line starts and as the next wrap-changing width (source: icpc/samples-java/src/Ch10/PF.java, lines 31-63)

```
    private static (int, int) WrapRiver(int[] len, int cnt, int w, out int nextW)
    {
        // greedy wrap; jump straight to the next width that changes any line;
        // river dp keeps, per column, the longest chain ending there
        var spaceCols = new List<int[]>();
        int i = 0;
        nextW = int.MaxValue;
        while (i < cnt)
        {
            int j = i;
            long used = len[i];
            while (j + 1 < cnt && used + 1 + len[j + 1] ≤ w)
            {
                used += 1 + len[j + 1];
                j++;
            }
            var cols = new int[j - i];
            int c = len[i];
            for (int k = i + 1; k ≤ j; k++)
            {
                cols[k - i - 1] = c;
                c += 1 + len[k];
            }
            spaceCols.Add(cols);
            if (j + 1 < cnt)
                nextW = Math.Min(nextW, (int)(used + 1 + len[j + 1]));
            i = j + 1;
        }
        if (nextW == int.MaxValue)
            nextW = -1;
        int lines = spaceCols.Count;
        int river = 0;
        var prev = new int[w + 2];
        var cur = new int[w + 2];
        for (int li = 0; li < lines; li++)
        {
            foreach (int col in spaceCols[li])
                cur[col] = 0;
            foreach (int col in spaceCols[li])
            {
                int best = li > 0 ? Math.Max(prev[col],
                    Math.Max(col > 0 ? prev[col - 1] : 0,
                        col < w ? prev[col + 1] : 0)) + 1 : 1;
                cur[col] = Math.Max(cur[col], best);
                river = Math.Max(river, best);
            }
            (prev, cur) = (cur, prev);
            Array.Clear(cur);
        }
        return (lines, river);
    }
```

listing 272: c#: WrapRiver returns lines, river, and the next width, per-line space columns listed while wrapping (source: icpc/samples/src/Ch09/PF.cs, lines 39-89)

```javascript
export function solve(input) {
  // the word list spans multiple input lines, so tokenize everything
  const toks = input.trim().split(/\s+/);
  const n = Number(toks[0]);
  const words = toks.slice(1, 1 + n).map((w) => w.length);
  let maxLen = 0, total = 0;
  for (const w of words) {
    if (w > maxLen) maxLen = w;
    total += w;
  }
  let bestRiver = 0, bestWidth = maxLen;
  if (n > 1) {
    const prev = new Int32Array(total + n + 2), cur = new Int32Array(total + n + 2);
    let width = maxLen;
    while (width ≤ total + n - 1) {
      const [lengths, firsts] = wrap(words, width);
      const river = riverOf(lengths, firsts, words, prev, cur);
      if (river > bestRiver) {
        bestRiver = river;
        bestWidth = width;
      }
      if (bestRiver ≥ lengths.length) break;
      // widths below the smallest merge threshold keep the wrap
      let next = 0;
      for (let i = 0; i + 1 < lengths.length; i++) {
        const need = lengths[i] + 1 + words[firsts[i + 1]];
        if (next === 0 || need < next) next = need;
      }
      if (lengths.length === 1) break;
      width = next;
    }
  }
  return `${bestWidth} ${bestRiver}\n`;
}
```

listing 273: javascript: solve with the line-count stop and the merge-threshold jump (source: icpc/samples-js/src/ch09-pf-gowithflow.mjs, lines 58-91)

```python
def _river(lengths, firsts, words, prev, cur):
    best = 0
    prev_touched = []
    lines = len(lengths)
    for k in range(lines):
        cur_touched = []
        end = firsts[k + 1] if k + 1 < lines else len(words)
        col = 0
        for w in range(firsts[k], end - 1):
            col += words[w]
            val = 1
            v = prev[col - 1] + 1
            if v > val:
                val = v
            v = prev[col] + 1
            if v > val:
                val = v
            v = prev[col + 1] + 1
            if v > val:
                val = v
            cur[col] = val
            cur_touched.append(col)
            if val > best:
                best = val
            col += 1
        for c in prev_touched:
            prev[c] = 0
        prev, cur = cur, prev
        prev_touched = cur_touched
    for c in prev_touched:
        prev[c] = 0
    return best
```

listing 274: python: `_river`, the same touched-column reset, one line of dp per space (source: icpc/samples-py/src/Ch09/pf.py, lines 27-58)

```lua
local function river_of(lengths, firsts, words, prev, cur)
  local best = 0
  local prev_touched = {}
  for k = 1, #lengths do
    local cur_touched = {}
    local endw = (k < #firsts) and firsts[k + 1] or #words + 1
    local col = 0
    for w = firsts[k], endw - 2 do
      col = col + words[w]
      local val = 1
      for dc = -1, 1 do
        local p = prev[col + dc]
        if p and p + 1 > val then val = p + 1 end
      end
      cur[col] = val
      cur_touched[#cur_touched + 1] = col
      if val > best then best = val end
      col = col + 1
    end
    for _, c in ipairs(prev_touched) do prev[c] = nil end
    prev, cur = cur, prev
    prev_touched = cur_touched
  end
  for _, c in ipairs(prev_touched) do prev[c] = nil end
  return best
end
```

listing 275: lua: river_of over sparse tables, `prev[c] = nil` instead of a zeroing pass (source: icpc/samples-lua/ch09_pf.lua, lines 27-52)

The fixture is six alternating words `aa bbb` where width 6 wraps to three identical lines whose single spaces all sit in column 3, a river of 3, while widths 3 to 5 put one word per line with no spaces at all and widths from 8 pack at least two words per line, capping the line count at 3, so the answer is `6 3`. Suites: three CHECKs in c, three [Fact] methods in c#, four table cases in go, four ok lines in java, four assertions in javascript, four checks in python, four `T.eq` blocks in lua, the shared extras a two-word forced space, a single word with river 0, and a tall column of one-letter words. Java is the one tree that splits c's wrap into two functions, `wrapInto` collecting the line starts and `nextChangeWidth` computing the jump, where c returns the next width as its wrap's side effect. No integer notes, everything is a length and a column index well inside every machine word.

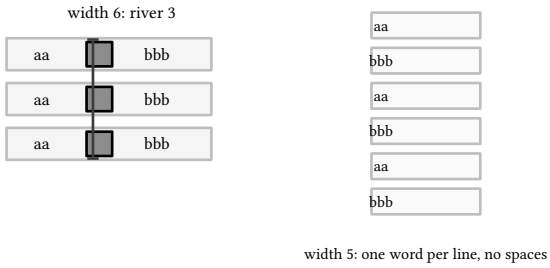


figure 116: the fixture at width 6: three lines, one space each, all in column 3, a river of length 3, against the degenerate width-5 wrap beside it

11.8. problem G: panda preserve

Sichuan province has funding for the Great Panda National Park, a preserve for more than 1800 giant pandas surrounded by a polygonal fence, and every panda will wear a wireless transmitter. Researchers place one receiver at each vertex of the enclosing polygon, every receiver covers a circular area of one shared range centered on itself, and smaller receivers are cheaper. Find the smallest range that covers the entire park (statement).

The input is one integer n, 3 <= n <= 2000 vertices by the problemset pdf, then n lines of two integers x and y with |x|, |y| <= 10^4, the polygon's vertices in counter-clockwise order. The pdf promises a simple polygon, every vertex distinct, and no two edges intersecting or touching except consecutive edges at their common vertex. The output is the minimum wireless range, an absolute or relative error of at most 1e-6 accepted, under the pdf's 10 second limit, and the harness verifies it in float mode, `2018/G` pinned to `float` in `ref/icpc/compare.json`.

Official sample 1, reprinted byte for byte from the judge data pair `G-pandapreserve/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints and draws at ranges 35 and 50 in its own figure: the pentagon's critical point is (100,0) on the bottom edge, exactly 50 from the two plateau vertices (60,30) and (140,30), so here the covering radius is decided by a voronoi edge crossing a polygon edge rather than by an interior voronoi vertex like the fixture's center.

input:

```
5
0 0
170 0
140 30
60 30
0 70
```

expected output:

```
50
```

Recognition: n <= 2000 vertices under the 10 second limit prices the per-vertex cell build at O(n^2 log n), 4*10^6 bisector clips behind an 11-entry sort each, about 4.4*10^7 comparisons, and the crossing tests add O(n^2), so enumerating both candidate families per vertex is the affordable shape.

The cue is smallest range that covers the entire park, an extremal covering radius, attained at finitely many critical points, interior voronoi vertices and voronoi-polygon edge crossings, the enumerate-and-test shape over O(n^2) geometric candidates. Problem G is the 2018 face of the geometric critical-point enumeration family, and the same geometric critical-point enumeration cue drives chapter 10 problem A, chapter 10 problem D, chapter 11 problem E, chapter 13 problem T, and chapter 14 problem D.

Sampling the park instead is the priced-out wrong tool: a 2000 by 2000 grid over the 10^4 coordinate range resolves 10-unit cells, four orders too coarse for the 1e-6 tolerance, and refining to 10^-3 spacing is a 2*10^13-point scan, while a boundary-only scan misses the interior voronoi vertices outright.

Receivers sit at every vertex and cover the disc of radius r around themselves, so the covering radius is the largest over park points of the distance to the nearest vertex, attained either at a voronoi vertex of the polygon's vertices that lies inside the polygon or where a voronoi edge crosses a polygon edge. Each solver builds every vertex's cell by clipping a box against the perpendicular bisectors of the other vertices, nearest first with a distance-ordered early break, then tests those two candidate families, O(n^2 log n) for the cell build and O(n^2) for the crossing tests (solutions.pdf pp. 7-9). Book 10′s chapter 37, computational geometry ii, develops the half-plane intersection this per-vertex clipping rests on. This is the problem that forced a year-local geometry helper, cross products, ray-cast point-in-polygon, segment intersection, and the half-plane clip, because no wave 1 toolbox carried 2d vector algebra.

The worked run: trace the model on sample 1. The plateau vertices (60,30) and (140,30) share the perpendicular bisector `x = 100`, and that bisector crosses the bottom edge from (0,0) to (170,0) at (100,0), on the park's boundary, so it is a crossing candidate. Its covering radius is `sqrt(40^2 + 30^2) = sqrt(2500) = 50` from either plateau vertex, and the two voronoi vertices on the same bisector fall outside the park: with (0,0) the bisector `2x + y = 75` meets `x = 100` at (100,-125), with (0,70) the bisector `3x - 2y = -10` meets it at (100,155), above the top edge. Every other cell peaks lower, the left-edge crossing of the (0,0)-(0,70) bisector `y = 35` answers 35 and the (170,0)-(140,30)

cut `y = x - 140` meets the bottom edge 30 out at (140,0), so the binding candidate is the edge crossing at (100,0), and the trace ends at the printed answer `50`.

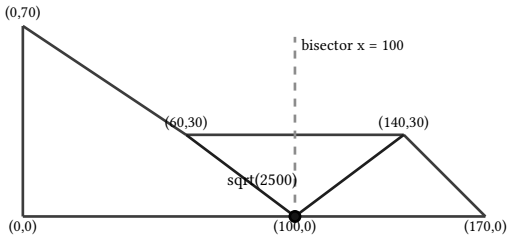


figure 117: sample 1′s pentagon: the plateau bisector x = 100 crosses the bottom edge at (100,0), exactly 50 from both plateau vertices, both triple points on the bisector falling outside

```
for (int i = 0; i < n; i++) {
  int m = 4;
  double B = 4e4;
  cell[0].x = -B;
  cell[0].y = -B;
  cell[1].x = B;
  cell[1].y = -B;
  cell[2].x = B;
  cell[2].y = B;
  cell[3].x = -B;
  cell[3].y = B;
  for (int j = 0; j < n && m > 0; j++) {
    if (j == i)
      continue;
    // keep {x : |x - pi| ≤ |x - pj|} = {x : nrm.(x - mid) ≤ 0}
    // with nrm = pj - pi; the left side of the directed line
    // mid -> mid + (-nrm.y, nrm.x) is exactly that half-plane
    G2 mid = {(pt[i].x + pt[j].x) / 2, (pt[i].y + pt[j].y) / 2};
    double nx = (double)pt[j].x - pt[i].x, ny = (double)pt[j].y - pt[i].y;
    G2 a = mid, b = {mid.x - ny, mid.y + nx};
    int allin = 1, anyin = 0;
    for (int k = 0; k < m; k++) {
      // side ≥ 0 means kept (left of a->b, same as g2clip)
      double side = g2cross(a, b, cell[k]);
      if (side < 0)
        allin = 0;
      else
        anyin = 1;
    }
    if (allin)
      continue;
    if (!anyin) {
      m = 0;
      break;
    }
    m = g2clip(cell, m, a, b);
  }
  if (m ≤ 0)
    continue;
  // cell vertices inside the park
  for (int k = 0; k < m; k++) {
    if (g2inpolygon(cell[k], pt, n)) {
      double dx = cell[k].x - pt[i].x, dy = cell[k].y - pt[i].y;
      double d = dx * dx + dy * dy;
      if (d > best)
        best = d;
    }
  }
```

listing 276: c: the per-vertex cell build by g2clip against a big box, then in-park cell vertices as candidates (source: icpc/samples-c/src/Ch09/pG.c, lines 38-85)

```
    // a modest box (five times the coordinate range) keeps the
    // clip arithmetic well conditioned; every bisector crossing
    // the park stays inside it, and far circumcenters only ever
    // produce candidates outside the park
    const box = 5e4
    cell := []pt2{{-box, -box}, {box, -box}, {box, box}, {-box, box}}
    radius := box * math.Sqrt2
    for _, j := range byDist {
      if j == site {
        continue
      }
      if distSq(s, poly[j]) ≥ 4*radius*radius {
        // the bisector lies beyond the whole current cell
        break
      }
      cell = clipCell(cell, s, poly[j])
      if len(cell) == 0 {
        break
      }
      radius = 0
      for _, c := range cell {
        if d := distSq(s, c); d > radius {
          radius = d
        }
      }
      radius = math.Sqrt(radius)
    }
    if len(cell) == 0 {
      continue
    }
    // Voronoi vertices inside the park
    for _, c := range cell {
      if pointInPoly(poly, c) {
        if d := math.Sqrt(distSq(s, c)); d > best {
          best = d
        }
      }
    }
    // Voronoi edges crossing polygon edges
    for k := 0; k < len(cell); k++ {
      a, b := cell[k], cell[(k+1)%len(cell)]
      for e := 0; e < n; e++ {
        if p, ok := segIntersect(a, b, poly[e], poly[(e+1)%n]); ok {
          if d := math.Sqrt(distSq(s, p)); d > best {
            best = d
          }
        }
      }
    }
  }
  return fmt.Sprintf("%.9f\n", best)
```

listing 277: go: the box comment, nearest-first clipping with the early break, both candidate scans (source: icpc/samples-go/ch09/pg.go, lines 38-88)

```
    for (int i = 0; i < n; i++) {
      int m = 4;
      cell[0].x = -box;
      cell[0].y = -box;
      cell[1].x = box;
      cell[1].y = -box;
      cell[2].x = box;
      cell[2].y = box;
      cell[3].x = -box;
      cell[3].y = box;
      for (int j = 0; j < n && m > 0; j++) {
        if (j == i)
          continue;
        // keep {x : |x - pi| ≤ |x - pj|} = {x : nrm.(x - mid) ≤ 0}
        // with nrm = pj - pi; the left side of the directed line
        // mid -> mid + (-nrm.y, nrm.x) is exactly that half-plane
        Geo.Pt mid = new Geo.Pt((pt[i].x + pt[j].x) / 2, (pt[i].y + pt[j].y) / 2);
        double nx = pt[j].x - pt[i].x, ny = pt[j].y - pt[i].y;
        Geo.Pt a = mid, b = new Geo.Pt(mid.x - ny, mid.y + nx);
        boolean allin = true, anyin = false;
        for (int k = 0; k < m; k++) {
          // side ≥ 0 means kept (left of a->b, same as clip)
          double side = Geo.cross(a, b, cell[k]);
          if (side < 0)
            allin = false;
          else
            anyin = true;
        }
        if (allin)
          continue;
        if (!anyin) {
          m = 0;
          break;
        }
        m = Geo.clip(cell, m, a, b);
      }
```

listing 278: java: the per-vertex cell build against a big box, half-planes that keep the whole cell skipped, empty cells dropped early (source: icpc/samples-java/src/Ch10/PG.java, lines 45-80)

```
            foreach (int j in others)
            {
                Clip(cell, px[v], py[v], px[j], py[j]);
                if (cell.Count == 0)
                    break;
            }
            for (int c = 0; c < cell.Count; c++)
            {
                var a = cell[c];
                var b = cell[(c + 1) % cell.Count];
                if (Math.Abs(a.X) < Box / 2 && Math.Abs(a.Y) < Box / 2
                    && InPolygon(a.X, a.Y, px, py))
                    best = Math.Max(best, Dist2(a.X, a.Y, px[v], py[v]));
                for (int e = 0; e < n; e++)
                {
                    var p = SegCross(a, b, px[e], py[e],
                        px[(e + 1) % n], py[(e + 1) % n]);
                    if (p.HasValue)
                        best = Math.Max(best,
                            Dist2(p.Value.X, p.Value.Y, px[v], py[v]));
                }
            }
        }
        return Math.Sqrt(best).ToString("F9", CultureInfo.InvariantCulture) + "\n";
```

listing 279: c#: clip by the algebraic half-plane $f(p) \le 0$, candidates from crossings and in-polygon circumcenters (source: icpc/samples/src/Ch09/PG.cs, lines 41-64)

```
export function solve(input) {
  const ints = input.match(/-?\d+/g).map(Number);
  const n = ints[0];
  const poly = [];
  for (let i = 0; i < n; i++) poly.push({ x: ints[1 + 2 * i], y: ints[2 + 2 * i] });
  let best = 0;
  for (let site = 0; site < n; site++) {
    const s = poly[site];
    const byDist = poly
      .map((_, j) => j)
      .sort((a, b) => distSq(s, poly[a]) - distSq(s, poly[b]));
    // a modest box (five times the coordinate range) keeps the clip
    // arithmetic well conditioned; every bisector crossing the park
    // stays inside it, and far circumcenters only ever produce
    // candidates outside the park
    const box = 5e4;
    let cell = [
      { x: -box, y: -box }, { x: box, y: -box },
      { x: box, y: box }, { x: -box, y: box },
    ];
    let radius = box * Math.SQRT2;
    for (const j of byDist) {
      if (j === site) continue;
      if (distSq(s, poly[j]) ≥ 4 * radius * radius) {
        // the bisector lies beyond the whole current cell
        break;
      }
      cell = clipCell(cell, s, poly[j]);
      if (cell.length === 0) break;
      radius = 0;
      for (const c of cell) radius = Math.max(radius, distSq(s, c));
      radius = Math.sqrt(radius);
    }
    if (cell.length === 0) continue;
    // voronoi vertices inside the park
    for (const c of cell) {
      if (pointInPoly(poly, c)) best = Math.max(best, Math.sqrt(distSq(s, c)));
    }
    // voronoi edges crossing polygon edges
    for (let k = 0; k < cell.length; k++) {
      const a = cell[k], b = cell[(k + 1) % cell.length];
      for (let e = 0; e < n; e++) {
        const [p, ok] = segIntersect(a, b, poly[e], poly[(e + 1) % n]);
        if (ok) best = Math.max(best, Math.sqrt(distSq(s, p)));
      }
    }
  }
  return `${best.toFixed(9)}\n`;
}
```

listing 280: javascript: solve importing clipCell, pointInPoly, segIntersect from the ch09-geo module (source: icpc/samples-js/src/ch09-pg-pandapreserve.mjs, lines 12-60)

```
def solve(text):
    data = text.split()
    n = int(data[0])
    poly = [(float(data[1 + 2 * i]), float(data[2 + 2 * i])) for i in range(n)]
    best = 0.0
    for site in range(n):
        sx, sy = poly[site]
        by_dist = sorted(
            range(n),
            key=lambda j: (poly[j][0] - sx) ** 2 + (poly[j][1] - sy) ** 2,
        )
        # a modest box (five times the coordinate range) keeps the clip
        # arithmetic well conditioned; far circumcenters only ever
        # produce candidates outside the park
        cell = [(-BOX, -BOX), (BOX, -BOX), (BOX, BOX), (-BOX, BOX)]
        radius = BOX * sqrt(2)
        for j in by_dist:
            if j == site:
                continue
            bx, by = poly[j]
            if (bx - sx) ** 2 + (by - sy) ** 2 ≥ 4 * radius * radius:
                break
            cell = _clip(cell, sx, sy, bx, by)
            if not cell:
                break
            radius = 0.0
            for cx, cy in cell:
                d = (cx - sx) ** 2 + (cy - sy) ** 2
                if d > radius:
                    radius = d
            radius = sqrt(radius)
        if not cell:
            continue
        for cx, cy in cell:
            if _in_poly(poly, cx, cy):
                d = sqrt((cx - sx) ** 2 + (cy - sy) ** 2)
                if d > best:
                    best = d
        for k in range(len(cell)):
            a, b = cell[k], cell[(k + 1) % len(cell)]
            for e in range(n):
                p = _seg_x(a, b, poly[e], poly[(e + 1) % n])
                if p is not None:
                    d = sqrt((p[0] - sx) ** 2 + (p[1] - sy) ** 2)
                    if d > best:
                        best = d
    return f"{best:.9f}\n"
```

listing 281: python: the same solve with the geometry inlined, per-site sorted neighbors and the box (source: icpc/samples-py/src/Ch09/pg.py, lines 95-141)

```
  for site = 1, n do
    local sx, sy = px[site], py[site]
    -- rank pack: distance-squared (integer-exact below 2^30) times
    -- 2048 plus the vertex id, sorted with the default order
    local bydist = {}
    local cnt = 0
    for j = 1, n do
      if j ~= site then
        cnt = cnt + 1
        local dx, dy = sx - px[j], sy - py[j]
        bydist[cnt] = (dx * dx + dy * dy) * 2048 + j
      end
    end
    table.sort(bydist)
    local cell = { { -box, -box }, { box, -box }, { box, box }, { -box, box } }
    local radius = box_corner
    for q = 1, cnt do
      local j = bydist[q] % 2048
      local dx, dy = sx - px[j], sy - py[j]
      local dsq = dx * dx + dy * dy
      if dsq >= 4 * radius * radius then break end
      cell = geo.clip_cell(cell, sx, sy, px[j], py[j])
      if #cell == 0 then break end
      radius = 0.0
      for _, c in ipairs(cell) do
        local cx, cy = c[1] - sx, c[2] - sy
        local dd = cx * cx + cy * cy
        if dd > radius then radius = dd end
      end
      radius = math.sqrt(radius)
    end
    if #cell == 0 then goto nextsite end
    for _, c in ipairs(cell) do
      if geo.point_in_poly(px, py, n, c[1], c[2]) then
        local d = math.sqrt(geo.dist_sq(sx, sy, c[1], c[2]))
        if d > best then best = d end
      end
    end
    for k = 1, #cell do
      local a, b = cell[k], cell[k % #cell + 1]
      for e = 1, n do
        local ix, iy = geo.seg_intersect(a[1], a[2], b[1], b[2],
          px[e], py[e], px[e % n + 1], py[e % n + 1])
        if ix then
          local d = math.sqrt(geo.dist_sq(sx, sy, ix, iy))
          if d > best then best = d end
        end
      end
    end
    ::nextsite::
  end
  return string.format("%.9f\n", best)
```

listing 282: lua: distance-squared rank-packing into one exact integer sort key, then the shared clip loop (source: icpc/samples-lua/ch09_pg.lua, lines 24-75)

The fixture is the 10 by 6 rectangle, whose four nearest-corner quadrants meet at the center, so the answer is the half diagonal `sqrt(34) = 5.830951895`, printed `%.9f`. Suites: three CHECKs in c, two [Fact] methods in c# whose

second runs all three official samples, four table cases in go, four ok lines in java each a tolerance compare, four assertions in javascript, four checks in python, and four `T.eq` blocks in lua, the shared extras a square, a thin strip peaking at `50.002499938`, and a right triangle whose peak sits on the hypotenuse. Integer notes: the input coordinates are integers and the distances are doubles everywhere, and lua packs `(dx^2+dy^2) * 2048 + id` into one sort key that stays exact as an integer, where a float key could tie-break wrongly at these magnitudes.

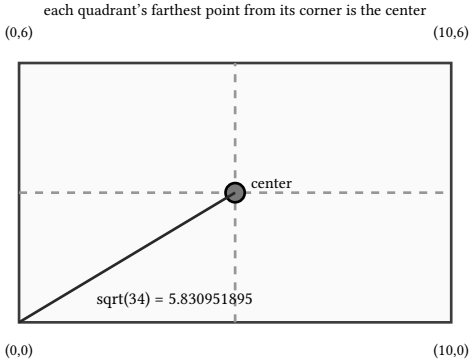


figure 118: the fixture rectangle: the four voronoi cells meet at the center, the covering radius is the half diagonal sqrt(34)

11.9. problem H: single cut of failure

The Intrusion and Crime Prevention Company is bidding to guard the room holding next year's World Finals problem set, a room with a single door and no other exits, defended against window rappels, air ducts, impersonated contest directors, and attack submarines by wiring the door itself: sensors on the four sides, pairs joined by straight wires, and any still-connected pair sounds the alarm when the door opens. The one design flaw is that an intruder might cut the wires first, and assessing that threat is the task: n wires cross the rectangular door, each anchored at integer points on two different sides, never at a corner, all anchor locations distinct. A cut is a straight line segment that starts and ends on different sides of the door and stays at least 1e-6 away from every wire anchor and every corner, and a minimum-size set of cuts must intersect every wire (statement).

The input is one line with n, w, and h by the problemset pdf, 1 <= n <= 10^6 wires on a door of width 1 <= w <= 10^8 and height 1 <= h <= 10^8, then n lines of four integers x1, y1, x2, and y2 with 0 <= x1, x2 <= w and 0 <= y1, y2 <= h, the two anchor points of one wire. Each wire connects two different sides, no wire is anchored at any of the four corners, and all locations in the input are distinct. The output is the number of cuts, then one cut per line as `x1 y1 x2 y2`, the coordinates may be real, cuts may appear in any order and each cut's endpoints in either order, any minimum-size set accepted, under the pdf's 6 second limit. The harness grades this problem on its first line, `2018/H` pinned to `first-line` in `ref/icpc/compare.json`, the cut lines beside the count accepted as one valid witness.

Official sample 1, reprinted byte for byte from the judge data pair `H-single/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints beside its door figure: one cut from (0,4) on the left side to (4,3) on the right crosses all four wires, so the minimum is 1.

input:

```
4 4 6
0 1 4 4
0 5 2 0
0 3 3 6
2 6 4 2
```

expected output:

```
1
0 4 4 3
```

Recognition: n <= 10^6 wires under the 6 second limit prices one sort of the 2*10^6 anchors plus a linear two-pointer window, about 4*10^7 comparisons with the fold, and any per-pair geometry over wires, 10^12 pairwise intersections, is dead on arrival.

The cue is the reformulation the statement invites: a cut is an arc once the boundary is unrolled onto a perimeter circle, and it splits exactly the wires with one anchor strictly inside, so the whole task becomes one sweep over sorted anchor positions. Problem H is the 2018 face of the event sweep over a sorted axis family, and the same event sweep over a sorted axis cue drives chapter 11 problem A, chapter 12 problem D, chapter 12 problem J, and chapter 14 problem L.

The priced-out alternative is treating cuts as arbitrary segments and searching a minimum line cover over 10^6 wires, pairwise candidate generation alone costing 5*10^11 pairs, while a per-wire interval formulation loses the arc structure that makes the one-cut test a single window.

Wires cross a rectangular door anchored at integer points on two different sides, and an intruder cuts them with straight segments that start and end on different sides, keeping 1e-6 clearance from every anchor and corner. Unrolling the boundary to a perimeter circle turns a cut into an open arc: it crosses exactly the wires with one anchor strictly inside the arc, one cut suffices iff some arc splits every wire, and the sweep over sorted anchors finds it with a two-pointer window that never lets a wire fall fully inside, O(n log n) (solutions.pdf pp. 9-10). The candidate endpoints are the midpoints of the open gaps between consecutive anchors, in quarter units, with a midpoint landing exactly on a door corner shifted a quarter forward, and the one-cut acceptance additionally demands endpoints on different door sides.

Two corner rules in that acceptance are the story of this problem, and all seven languages landed the same fix family for them. First, a gap that strictly contains a corner lets its endpoint sit on either side of that corner, so only gaps lying strictly inside one side carry a side label, and the wrap gap between the last and first anchor always strictly contains ring corner 0 because anchors never sit at corners, so it always straddles even when no midpoint corner lies inside: labeling it single-side under-reports single cuts, 113 of 6000 mismatches against a quarter-grid brute, and the regression door `2 10 10` with anchors `0 3 5 0` and `7 0 8 0` is the pinned witness, one cut from `0 0.25` to `7.5 0`, its s endpoint snapped one quarter past corner 0 onto the left side. A wrap midpoint landing exactly one lap, which is corner 0 again, reduces modulo the ring before the corner test. Second, the witness render snaps: a straddling endpoint whose gap midpoint happens to render on the same side as the other endpoint slides one quarter past its contained corner onto the differing side, so the two printed endpoints really differ.

The worked run: trace the model on sample 1. The 4 by 6 door has perimeter `2*(4+6) = 20`, and unrolling counter-clockwise from corner (0,0) puts the eight printed anchors at ring positions: wire 1′s (0,1) at `20-1 = 19` and (4,4) at `4+4 = 8`, wire 2′s (0,5) at 15 and (2,0) at 2, wire 3′s (0,3) at 17 and (3,6) at `14-3 = 11`, wire 4′s (2,6) at `14-2 = 12` and (4,2) at 6. The sweep slides a window that never contains both anchors of one wire, and the winning arc runs from the midpoint of gap (15,17), position 16, wrapping forward to the midpoint of gap (6,8), position 7. Inside that arc sit anchors 17, 19, 2 and 6, one per wire, wire 1′s 19, wire 2′s 2, wire 3′s 17, wire 4′s 6, so a single arc splits all four wires, zero cuts never guards anything with wires present, and the minimum is 1. The arc's endpoints render back on the boundary as (0,4) on the left side and (4,3) on the right side, `20-4 = 16` and `4+3 = 7`, two different sides, and the trace ends at the printed answer `0 4 4 3`.

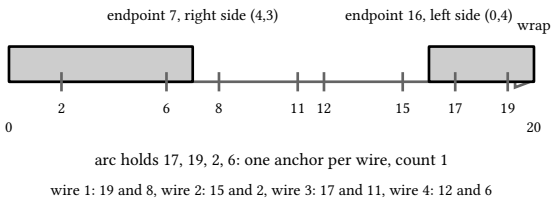


figure 119: sample 1′s anchors unrolled on the perimeter ring: the winning arc from midpoint 16 wraps past corner 0 to midpoint 7 and holds exactly one anchor of each wire

```
// per-gap candidate points and corner-straddle flags: the gap after
// anchor k (wrapping at k = 2n-1) has its midpoint in quarter
// units; a gap strictly containing a door corner lets the cut
// endpoint sit on either side of it, so the differ-sides test runs
// per gap, never at the exact midpoint
static long long gap_after[2000008];
static long long gap_corner[2000008];
static char gap_single[2000008];
static int gap_side[2000008];
long long corners[4] = {0, 4 * W, 4 * W + 4 * H, 8 * W + 4 * H};
for (int k = 0; k < 2 * n; k++) {
  long long next = A[(k + 1) % (2 * n)].p;
  if (k == 2 * n - 1)
    next += P4;
  long long lo = A[k].p, hi = next;
  long long mid = (lo + hi) / 2; // both multiples of 4: mid is even
  if (mid ≥ P4)
    mid -= P4; // wrap first (≥ : a midpoint of exactly one lap is
  // the corner at 0 and must shift), then the corner shift
  for (int ci = 0; ci < 4; ci++)
    if (mid == corners[ci]) {
      mid += 1;
      break;
    }
  gap_after[k] = mid;
  gap_single[k] = 1;
  gap_corner[k] = -1;
  for (int ci = 0; ci < 4; ci++)
    if (corners[ci] > lo && corners[ci] < hi) {
      gap_single[k] = 0;
      gap_corner[k] = corners[ci];
    }
  if (k == 2 * n - 1 && gap_single[k]) {
    // the wrap gap always strictly contains ring corner 0 (anchors
    // never sit at corners), so it always straddles even when no
    // mid corner lies inside
    gap_single[k] = 0;
    gap_corner[k] = 0;
  }
```

listing 283: c: gap midpoints with the one-lap reduction, corner-straddle flags, and the wrap gap forced to straddle (source: icpc/samples-c/src/Ch09/pH.c, lines 109-147)

```
  // gapAfter[k] = perimeter position (in quarter units) of the gap
  // between anchor k and k+1, with the wrap gap re-referenced
  gapAfter := make([]int64, 2*n)
  gapSingle := make([]bool, 2*n) // gap lies on one door side only
  gapSide := make([]int, 2*n)
  gapCorner := make([]int64, 2*n) // a corner strictly inside, or -1
  corners := []int64{0, int64(w) * 4, int64(w+h) * 4, int64(2*w + h) * 4}
  for k := 0; k < 2*n; k++ {
    next := int64(sortedPos[(k+1)%(2*n)])
    if k == 2*n-1 {
      next += int64(perim)
    }
    lo := int64(sortedPos[k]) * 4
    hi := next * 4
    mid := (int64(sortedPos[k]) + next) * 2 // quarter units
    mid %= int64(perim) * 4
    for _, corner := range corners {
      if mid == corner {
        mid++
        break
      }
    }
    gapAfter[k] = mid
    // a gap strictly containing a corner lets the cut endpoint
    // sit on either side of it
    gapSingle[k] = true
    gapCorner[k] = -1
    for _, c := range corners {
      if c > lo && c < hi {
        gapSingle[k] = false
        gapCorner[k] = c
      }
    }
    if k == 2*n-1 && gapSingle[k] {
      // the wrap gap always strictly contains ring corner 0
      // (anchors never sit at corners), so it always straddles
      // even when no mid corner lies inside
      gapSingle[k] = false
      gapCorner[k] = 0
    }
    if gapSingle[k] {
      gapSide[k] = sideOfQuarter(mid, w, h)
    }
  }
```

listing 284: go: gapAfter, gapSingle, gapCorner, gapSide, with the wrap-gap block verbatim (source: icpc/samples-go/ch09/ph.go, lines 56-99)

```
    for (int k = 0; k < 2 * n; k++) {
      long next = anchor[(k + 1) % (2 * n)][0];
      if (k == 2 * n - 1)
        next += p4;
      long lo = anchor[k][0], hi = next;
      long mid = (lo + hi) / 2; // both multiples of 4: mid is even
      if (mid ≥ p4)
        mid -= p4; // wrap first (≥ : a midpoint of exactly one lap is
      // the corner at 0 and must shift), then the corner shift
      for (long corner : corners) {
        if (mid == corner) {
          mid += 1;
          break;
        }
      }
      gapAfter[k] = mid;
      gapSingle[k] = true;
      gapCorner[k] = -1;
      for (long corner : corners)
        if (corner > lo && corner < hi) {
          gapSingle[k] = false;
          gapCorner[k] = corner;
        }
      if (k == 2 * n - 1 && gapSingle[k]) {
        // the wrap gap always strictly contains ring corner 0 (anchors
        // never sit at corners), so it always straddles even when no
        // mid corner lies inside
        gapSingle[k] = false;
        gapCorner[k] = 0;
      }
      if (gapSingle[k]) {
        long pv = mid;
        int side;
        if (pv ≤ 4 * w)
          side = 0;
        else if (pv ≤ 4 * w + 4 * h)
          side = 1;
        else if (pv ≤ 8 * w + 4 * h)
          side = 2;
        else
          side = 3;
        gapSide[k] = side;
      }
    }
```

listing 285: java: gap midpoints with the one-lap reduction, straddle flags per gap, the wrap gap forced to straddle corner 0 (source: icpc/samples-java/src/Ch10/PH.java, lines 113-156)

```
        // gap midpoints in x4 perimeter units; one landing on a door
        // corner slides a quarter unit forward. a gap strictly containing
        // a door corner lets its cut endpoint sit on either side of that
        // corner, so only gaps inside one door side carry a side label
        long[] corner4 = { 0, 4 * w, 4 * (w + h), 4 * (2 * w + h) };
        var gap = new long[2 * n];
        var gapLo = new long[2 * n];
        var gapHi = new long[2 * n];
        var gapCorner = new long[2 * n];
        var gapSingle = new bool[2 * n];
        var gapSide = new int[2 * n];
        for (int k = 0; k < 2 * n; k++)
        {
            long a = anchors[k];
            long b = k + 1 < 2 * n ? anchors[k + 1] : anchors[0] + per;
            gapLo[k] = 4 * a;
            gapHi[k] = 4 * b;
            long m4 = 2 * (a + b); // (lo4 + hi4) / 2 in x4 units
            long p4 = m4 % (4 * per);
            if (Array.IndexOf(corner4, p4) ≥ 0)
                p4++;
            gap[k] = p4;
            gapSingle[k] = true;
            gapCorner[k] = -1;
            foreach (long c in corner4)
                if (c > gapLo[k] && c < gapHi[k])
                {
                    gapSingle[k] = false;
                    gapCorner[k] = c;
                }
            if (k == 2 * n - 1 && gapSingle[k])
            {
                // the wrap gap always strictly contains ring corner 0
                // (anchors never sit at corners), so it always
                // straddles even when no mid corner lies inside
                gapSingle[k] = false;
                gapCorner[k] = 0;
            }
            if (gapSingle[k])
                gapSide[k] = SideOf(p4, w, h);
        }
```

listing 286: c#: the same gap table in x4 units, corner4 test, wrap rule last (source: icpc/samples/src/Ch09/PH.cs, lines 25-65)

```
  // s iterates the gaps in ascending order: the wrap gap first, then
  // gapAfter[0..2n-2], with start anchors 0..2n-1
  for (let si = 0; si < m; si++) {
    if (si > 0 && si - 1 ≤ j) {
      // anchor si-1 drops out of the window
      cnt[sortedWire[si - 1]]--;
      inside--;
    }
    if (j < si - 1) j = si - 1; // the window restarts empty at si
    while (j + 1 < m && cnt[sortedWire[j + 1]] === 0) {
      j++;
      cnt[sortedWire[j]]++;
      inside++;
    }
    if (inside === n) {
      const sGap = si === 0 ? m - 1 : si - 1;
      const tGap = j;
      const differ =
        !gapSingle[sGap] || !gapSingle[tGap] || gapSide[sGap] =/= gapSide[tGap];
      if (differ) {
        let sPos = gapAfter[sGap], tPos = gapAfter[tGap];
        // a straddling endpoint whose gap midpoint renders on the
        // same side as the other endpoint slides one quarter past a
        // contained corner onto the differing side, so both printed
        // endpoints really differ
        if (sideOfQuarter(sPos, w, h) === sideOfQuarter(tPos, w, h)) {
          let g = sGap;
          let avoid = sideOfQuarter(tPos, w, h);
          if (gapSingle[sGap] && !gapSingle[tGap]) {
            g = tGap;
            avoid = sideOfQuarter(sPos, w, h);
          }
          const ring = perim * 4;
          const a = (gapCorner[g] + 1) % ring;
          const b = ((gapCorner[g] - 1) % ring + ring) % ring;
          const pick = sideOfQuarter(a, w, h) === avoid ? b : a;
          if (g === sGap) sPos = pick;
          else tPos = pick;
        }
        const [x1, y1] = perimXY(sPos, w, h);
        const [x2, y2] = perimXY(tPos, w, h);
        return `1\n${renderQuarter(x1)} ${renderQuarter(y1)} ` +
          `${renderQuarter(x2)} ${renderQuarter(y2)}\n`;
      }
    }
  }
```

listing 287: javascript: the window sweep, the differ-sides test, and the witness snap past the corner (source: icpc/samples-js/src/ch09-ph-single.mjs, lines 94-139)

```
    cnt = [0] * n
    inside = 0
    j = -1
    for si in range(2 * n):
        if si > 0 and si - 1 ≤ j:
            cnt[swire[si - 1]] -= 1
            inside -= 1
        if j < si - 1:
            j = si - 1
        while j + 1 < 2 * n and cnt[swire[j + 1]] == 0:
            j += 1
            cnt[swire[j]] += 1
            inside += 1
        if inside == n:
            s_gap = si - 1 if si else 2 * n - 1
            t_gap = j
            differ = (
                not gap_single[s_gap]
                or not gap_single[t_gap]
                or gap_side[s_gap] ≠ gap_side[t_gap]
            )
            if differ:
                s_pos, t_pos = gap_after[s_gap], gap_after[t_gap]
                # a straddling endpoint whose gap midpoint renders on
                # the same side as the other endpoint slides one
                # quarter past a contained corner onto the differing
                # side, so both printed endpoints really differ
                if _side(s_pos, w, h) == _side(t_pos, w, h):
                    g, avoid = s_gap, _side(t_pos, w, h)
                    if gap_single[s_gap] and not gap_single[t_gap]:
                        g = t_gap
                        avoid = _side(s_pos, w, h)
                    ring = 4 * perim
                    a = (gap_corner[g] + 1) % ring
                    b = ((gap_corner[g] - 1) % ring + ring) % ring
                    if _side(a, w, h) == avoid:
                        a = b
                    if g == s_gap:
                        s_pos = a
                    else:
                        t_pos = a
                x1, y1 = _perim_xy(s_pos, w, h)
                x2, y2 = _perim_xy(t_pos, w, h)
                return (
                    f"1\n{_quarter(x1)} {_quarter(y1)} "
                    + f"{_quarter(x2)} {_quarter(y2)}\n"
                )
    return f"2\n0 0.5 {w} {_quarter(4 * h - 2)}\n0 {_quarter(4 * h - 2)} {w} 0.5\n"
```

listing 288: python: the same sweep and snap, quarter formatter, two-cut fallback (source: icpc/samples-py/src/Ch09/ph.py, lines 98-145)

```
  local corners = { 0, w * 4, (w + h) * 4, (2 * w + h) * 4 }
  for k = 1, 2 * n do
    local nxt = spos[k % (2 * n) + 1]
    if k == 2 * n then nxt = nxt + perim end
    local lo = spos[k] * 4
    local hi = nxt * 4
    local mid = (spos[k] + nxt) * 2
    mid = mid % (perim * 4)
    for _, corner in ipairs(corners) do
      if mid == corner then
        mid = mid + 1
        break
      end
    end
    gap_after[k] = mid
    gap_single[k] = true
    gap_corner[k] = -1
    for _, c in ipairs(corners) do
      if c > lo and c < hi then
        gap_single[k] = false
        gap_corner[k] = c
      end
    end
    if k == 2 * n and gap_single[k] then
      -- the wrap gap always strictly contains ring corner 0 (anchors
      -- never sit at corners), so it always straddles even when no
      -- mid corner lies inside
      gap_single[k] = false
      gap_corner[k] = 0
    end
    if gap_single[k] then gap_side[k] = side_of(mid, w, h) end
  end
  local cnt = {}
  local inside = 0
  local j = 0
  local function perim_xy(q)
    if q ≤ w * 4 then return q, 0 end
    if q ≤ (w + h) * 4 then return 4 * w, q - 4 * w end
    if q ≤ (2 * w + h) * 4 then return (2 * w + h) * 4 - q, 4 * h end
    return 0, 2 * (w + h) * 4 - q
```

listing 289: lua: the gap classification in quarter units, wrap rule included (source: icpc/samples-lua/ch09_ph.lua, lines 49-88)

The fixture is two wires crossing at the center of a 4 by 6 door, both split by the vertical cut through the crossing, expected `1\n2 0 2 6\n`. The suites pin that fixture, the two-cut fallback for parallel wires, a half-unit gap midpoint, a single wire, and three parallel wires cut at the middle gap: three CHECKs in c, five [Fact] methods in c#, six table cases in go, six ok lines in java, six assertions in the javascript node:test file, six checks in python, and six `T.eq` blocks in lua, and the wrap-gap regression door above is a pinned case in all seven suites. Integer notes: everything runs in quarter-unit integer arithmetic, coordinates and the ring fit int64, javascript's plain Numbers are exact at these magnitudes, and the clearance is structural, gap midpoints stay at least a quarter unit from every integer anchor and corner, so no epsilon ever appears.

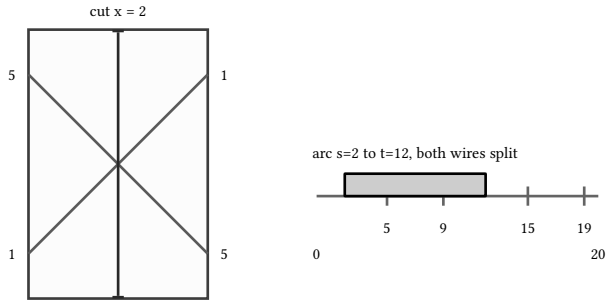


figure 120: the fixture door with its crossing wires and the winning vertical cut, and below it the boundary unrolled with the winning arc shaded

11.10. problem I: triangles

The trip to Beijing comes with plenty of puzzle books, and the puzzle is the oldest one in them: how many triangles are in the picture? The picture here is an ascii rendering of a triangular lattice with r rows and c columns of vertices. Every vertex is marked x, a horizontal unit edge between neighboring vertices is three dashes, and a diagonal unit edge is one / or one \, on staggered vertex rows: picture lines 1, 5, 9, and so on carry vertices in columns 1, 5, 9, and so on, picture lines 3, 7, 11 carry vertices in columns 3, 7, 11, and the even lines between carry the diagonal edge characters exactly between the vertices they join. Count every triangle of any size whose sides are straight grid lines (statement).

The input is two integers r and c by the problemset pdf, 1 <= r <= 3000 vertex rows and 1 <= c <= 6000 vertex columns, then 2r-1 picture lines of at most 2c-1 characters each, spaces everywhere except the x vertices, the dash triples of horizontal edges, and the single slash or backslash of each diagonal edge. The pdf notes that trailing whitespace on a line may be omitted, which is why the leading blanks carry the geometry. The output is the single count of triangles of any size, under the pdf's 6 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `I-triangles/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: the top dashes and the two upper diagonals close one downward triangle, the bottom vertex row carries no dashes so the upward orientation never forms, and the count is 1.

input:

```
3 3
x---x
 \ /
  x
 / \
x   x
```

expected output:

```
1
```

Recognition: r <= 3000 and c <= 6000 mean 1.8*10^7 vertices under the 6 second limit, the run tables cost one O(rc) pass, and each row's order statistic is an O(log c) fenwick step per corner, about 2.3*10^8 tree touches across both orientations, inside the budget only because the sweep is linear past the sort.

The cue is the count-every-triangle wording over an ascii lattice, a count over an implicit family far larger than any listing of triangles, and the sheared-grid recurrence turns it into run lengths plus an order statistic with scheduled removals. Problem I is the 2018 face of the counting and expectation without enumeration family, and the same counting and expectation without enumeration cue drives chapter 11 problem D, chapter 13 problem Q, chapter 14 problem E, and chapter 14 problem K.

Brute force prices out plainly: every corner at every size is `r*c*min(r,c) = 5.4*10^10` candidate triangles at the bound, each verified against runs, thirty times the budget even at one nanosecond per candidate.

Shearing the staggered grid turns one diagonal family vertical, a corner with an s-step horizontal run and an s-step same-family diagonal run forms a triangle exactly when the opposite family's run from the far base end covers s steps, and the count per row is an order statistic over active base ends with scheduled removals, O(n^2 log n) over both orientations, the second counted by the mirrored sweep (solutions.pdf pp. 10-11). The pictures are ragged, leading spaces are significant and trailing spaces may be gone, so every parser reads raw lines by position, never by token.

The worked run: trace the model on sample 1. The picture puts two vertices on the top row, the apex between on the middle row and two on the bottom row, and the run tables read straight off the printed characters: the top dashes give one horizontal run of one step, the bottom row prints no dashes at all, so its horizontal runs are empty, and each diagonal family carries the two one-step runs the printed slashes draw through the apex. The downward sweep opens at the top row's left corner: it holds an s = 1 horizontal run and an s = 1 same-family diagonal run, and the opposite family's run from the far base end covers 1 step, so the sweep counts exactly one triangle, the apex-closing one the picture draws. The upward sweep needs a bottom horizontal run to open any corner and finds none, the bare dash row kills every one, so the total is 1 + 0, and the trace ends at the printed answer 1.

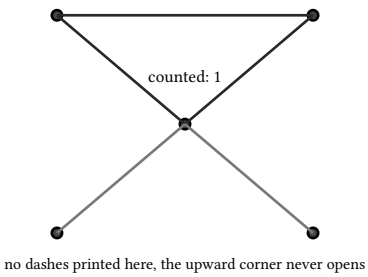


figure 121: sample 1's lattice: the one downward triangle the sweep counts, the bottom row's missing dash run killing the upward orientation

```
  long long ans = 0;
  for (int c = 0; c < n; c++) {
    for (int i = Srem[c]; i ≥ 0;) {
      fw_add(&SFW, i + 1, -1);
      int nx = Srm[i];
      Srm[i] = -1;
      i = nx;
    }
    for (int i = Sact[c]; i ≥ 0;) {
      fw_add(&SFW, i + 1, 1);
      int nx = Snxt[i];
      Snxt[i] = -1;
      i = nx;
    }
    if (bud[c] ≤ 0)
      continue;
    int m = bud[c];
    if (dir > 0) {
      int lo = c + 1, hi = c + m;
      if (hi > n - 1)
        hi = n - 1;
      if (lo > hi)
        continue;
      ans += fw_prefix(&SFW, hi + 1) - fw_prefix(&SFW, lo);
    } else {
      int lo = c - m, hi = c - 1;
      if (lo < 0)
        lo = 0;
      if (hi < 0)
        continue;
      ans += fw_prefix(&SFW, hi + 1) - fw_prefix(&SFW, lo);
    }
  }
  fw_free(&SFW);
  return ans;
}
```

listing 290: c: the sweep with chained scheduled adds and removals over the fenwick header (source: icpc/samples-c/src/Ch09/pI.c, lines 100-135)

```
  var total int64
  for i := 0; i < r; i++ {
    // base end w covers corner jv iff jv ≥ jw - dd[w]
    fen := newFenwick9(c)
    rems := make([]triRemoval, 0, c)
    for j := 0; j < c; j++ {
      if g.vert[i][j] && dd[i][j] ≥ 1 {
        if t := j - dd[i][j] - 1; t ≥ 0 {
          rems = append(rems, triRemoval{t, j})
        }
      }
    }
    sortTriRemovals(rems)
    p := len(rems) - 1
    for jv := c - 1; jv ≥ 0; jv-- {
      for p ≥ 0 && rems[p].at ≥ jv {
        fen.add(rems[p].col, -1)
        p--
      }
      total += cornerCount(g, rr, uu, fen, i, jv, c, false)
      if g.vert[i][jv] && dd[i][jv] ≥ 1 {
        fen.add(jv, 1)
      }
    }
  }
  return total
}
```

listing 291: go: countRightDown's row pass, a fresh fenwick9 per row, removals fired off a sorted list (source: icpc/samples-go/ch09/pi.go, lines 149-175)

```
static long sweepAxis(int n, short[] wr, short[] bud, int dir) {
  sweepTree = new Fenwick(n);
  for (int i = 0; i ≤ n + 1; i++)
    sact[i] = srem[i] = -1;
  for (int i = 0; i < n; i++) {
    int w = wr[i];
    if (w ≤ 0)
      continue;
    int a, r;
    if (dir > 0) {
      a = i - w;
      if (a < 0)
        a = 0;
      r = i;
    } else {
      a = i;
      r = i + w + 1;
    }
    if (r ≤ a)
      continue;
    if (a ≥ n)
      continue;
    if (r > n + 1)
      r = n + 1;
    snxt[i] = sact[a];
    sact[a] = i;
    srme[i] = srem[r];
    srem[r] = i;
  }
```

listing 292: java: the sweep with chained scheduled adds and removals over the Fenwick helper (source: icpc/samples-java/src/Ch10/PI.java, lines 61-89)

```
    private static long Sweep(int[] fen, int[] expire, int[] expireHead,
        int[] l, int[] u, int[] d, int c)
    {
        Array.Clear(fen);
        Array.Fill(expireHead, -1);
        for (int i = 0; i < c; i++)
        {
            int ex = i + d[i];
            if (d[i] > 0 && ex < c)
            {
                expire[i] = expireHead[ex];
                expireHead[ex] = i;
            }
        }
        long total = 0;
        for (int i = 0; i < c; i++)
        {
            int reach = Math.Min(l[i], u[i]);
            if (reach > 0)
                total += Prefix(fen, i) - Prefix(fen, i - reach);
            if (d[i] > 0)
                FenAdd(fen, i + 1, 1);
            for (int a = expireHead[i]; a != -1;)
            {
                FenAdd(fen, a + 1, -1);
                a = expire[a];
            }
        }
        return total;
    }
```

listing 293: c#: Sweep, the shared engine both orientations call, expiry lists bucketed by firing column (source: icpc/samples/src/Ch09/PI.cs, lines 127-156)

```
function countRightDown(g, r, c) {
  const rr = [], uu = [], dd = []; // right, down "\", down "/"
  for (let i = 0; i < r; i++) {
    rr.push(new Int32Array(c));
    uu.push(new Int32Array(c));
    dd.push(new Int32Array(c));
  }
  for (let i = 0; i < r; i++) {
    for (let j = c - 2; j ≥ 0; j--) {
      if (g.hEdge[i][j]) rr[i][j] = 1 + rr[i][j + 1];
    }
  }
  for (let i = r - 2; i ≥ 0; i--) {
    for (let j = 0; j < c; j++) {
      const ks = g.downSlash[i][j], kb = g.downBack[i][j];
      if (ks ≥ 0) uu[i][j] = 1 + uu[i + 1][ks];
      if (kb ≥ 0) dd[i][j] = 1 + dd[i + 1][kb];
    }
  }
  let total = 0;
  for (let i = 0; i < r; i++) {
    // base end w covers corner jv iff jv ≥ jw - dd[w]
    const fen = new Fenwick9(c);
    const rems = [];
    for (let j = 0; j < c; j++) {
      if (g.vert[i][j] && dd[i][j] ≥ 1) {
        const t = j - dd[i][j] - 1;
        if (t ≥ 0) rems.push([t, j]);
      }
    }
    rems.sort((a, b) ⇒ a[0] - b[0]);
    let p = rems.length - 1;
    for (let jv = c - 1; jv ≥ 0; jv--) {
      while (p ≥ 0 && rems[p][0] ≥ jv) {
        fen.add(rems[p][1], -1);
        p--;
      }
      total += cornerCount(g, rr, uu, fen, i, jv, c, false);
      if (g.vert[i][jv] && dd[i][jv] ≥ 1) fen.add(jv, 1);
    }
  }
  return total;
}
```

listing 294: javascript: countRightDown with the inlined Fenwick9 class, run tables filled bottom-up (source: icpc/samples-js/src/ch09-pi-triangles.mjs, lines 93-135)

```
    # pass 1, bottom-up: corners with the horizontal side right and
    # the "\" side down; base ends are "/" runs from the far end
    uu_next = [0] * w1
    dd_next = [0] * w2
    for i in range(r - 1, -1, -1):
        p = i & 1
        line = pic[2 * i]
        vbits = line[2 * p :: 4].ljust(c, b"0").translate(_T_X)
        rr = _runs_desc(line[2 * p + 2 :: 4].ljust(c, b"0").translate(_T_DASH), c)
        if i < r - 1:
            uu = [n + 1 if ch == 49 else 0 for ch, n in zip(ds[i], uu_next)]
            dd = [n + 1 if ch == 49 else 0 for ch, n in zip(db[i], dd_next)]
        else:
            uu = uu_next
            dd = dd_next
        s1 = off - i // 2
        s2 = (i + 1) // 2
        rem_at = [0] * (c + 1)
        for j in range(c):
            if vbits[j] == 49 and dd[j + s2] >= 1:
                t = j - dd[j + s2] - 1
                if t >= 0:
                    rem_at[t] |= 1 << j
        mask = 0
        for jv in range(c - 1, -1, -1):
            r_ = rem_at[jv]
            if r_:
                mask &= ~r_
            if vbits[jv] == 49:
                m_ = rr[jv]
                u = uu[jv + s1]
                if u < m_:
                    m_ = u
                if m_:
                    hi = jv + m_
                    if hi > c - 1:
                        hi = c - 1
                    total += (mask >> (jv + 1)).bit_count() - (
                        mask >> (hi + 1)
                    ).bit_count()
                if dd[jv + s2] >= 1:
                    mask |= 1 << jv
        uu_next = uu
        dd_next = dd
```

listing 295: python: pass 1 with an integer bitmask replacing the fenwick, window counts by `bit_count` (source: icpc/samples-py/src/Ch09/pi.py, lines 73-116)

```
    -- per-row sweep right to left with a fenwick of base ends.
    -- bucket the scheduled removals by firing column: each fires at
    -- exactly one jv of the sweep, so per-at lists replace go's sort
    local f = fen.new(c)
    local rems = {}
    for j = 1, c do
      if vert[j] and dd[j] ≥ 1 then
        local t = j - dd[j] - 1
        if t ≥ 1 then
          local bucket = rems[t]
          if bucket then bucket[#bucket + 1] = j else rems[t] = { j } end
        end
      end
    end
    for jv = c, 1, -1 do
      local bucket = rems[jv]
      if bucket then
        for w = 1, #bucket do fen.add(f, bucket[w] - 1, -1) end
      end
      if vert[jv] and rr[jv] > 0 and uu[jv] > 0 then
        local m = math.min(rr[jv], uu[jv])
        local hi = jv + m
        if hi > c then hi = c end
        total = total + fen.prefix(f, hi - 1) - fen.prefix(f, jv - 1)
      end
      if vert[jv] and dd[jv] ≥ 1 then
        fen.add(f, jv - 1, 1)
```

listing 296: lua: the fenwick sweep with removals bucketed per firing column instead of a sort (source: icpc/samples-lua/ch09_pi.lua, lines 114-140)

Six languages answer the order statistic with the year-local fenwick helper, point add and prefix sum over shifted columns, and python replaces it with a big-int bitmask over the row, activation and removal as bit sets and clears, and the window count as a shift plus `bit_count`, which is the same data structure wearing python's arbitrary-precision integers. The fixture is the three-row picture that carries exactly one triangle per orientation, expected `2\n`: the top dashes plus the two upper diagonals close a downward triangle, the bottom dashes plus the two lower diagonals close an upward one, and no two-step straight run exists. Suites: one CHECK in c on the fixture, three [Fact] methods in c#, four table cases in go, four ok lines in java, four assertions in javascript, four checks in python, four `T.eq` blocks in lua, the shared extras a lattice with only horizontals, an empty lattice of bare vertices, and long dashes without diagonals. Integer notes: the count reaches 1.1e11, int64 in c, c#, go, and lua, and javascript's plain Numbers stay exact below 2^53 so nothing special is needed.

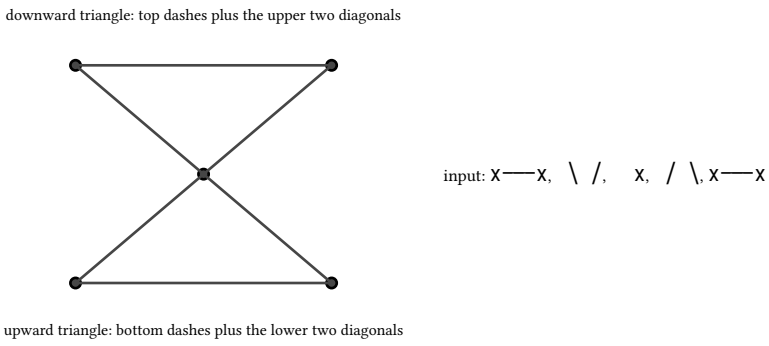


figure 122: the fixture rendered as a lattice: one downward triangle and one upward triangle share the middle apex

11.11. problem J: uncrossed knight's tour

The classic puzzle tours every square of an 8 by 8 chessboard with a knight, no repeats, ending back at the start. This harder version runs on a rectangular m by n board, a knight jump being one square in one direction and two in an orthogonal direction, and imagines the path as the straight line segments joining the centers of consecutive squares: those segments must form a simple polygon, no two of them crossing or touching except consecutive segments at their shared endpoint, and the knight must still return to its starting square. The no-crossing constraint makes visiting every square impossible, so the task is the maximum number of squares one closed, non-crossing tour can visit, 0 when no such tour exists at all (statement).

The input is a single line with two integers by the problemset pdf, m and n, 1 <= m <= 8 and 1 <= n <= 10^15. The output is the largest number of squares a knight can visit in a tour that does not cross its own path, or 0 if no such tour exists, under the pdf's 2 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `J-uncrossedknights/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints with its figure of an optimal tour: 12 squares of the 6 by 6 board close one simple polygon.

input:

```
6 6
```

expected output:

```
12
```

Official sample 2, same source, reprinted because it is the transpose witness: `v(8,3) = 6 = v(3,8)`, the one official sample whose answer per-m tables looked up without swapping the dimensions get wrong.

input:

```
8 3
```

expected output:

```
6
```

Recognition: m <= 8 but n <= 10^15 under a 2 second limit, so no per-query dp over columns can ever run, 10^15 columns times even one profile state is 10^17 transitions, five orders past the budget, and the only affordable shape is a table built offline plus an O(1) lookup with cycle extrapolation.

The cue is the pair of bounds itself, one dimension tiny and one astronomical, which forces an engineered compact state per row, visitation kind, vertical jump pattern, canonicalized connectivity, whose values fall into a cycle `v(n+P) = v(n)+A` once the window is deep enough. Problem J is the 2018 face of the dp over an engineered state space family, and the same dp over an engineered state space cue drives chapter 12 problem I, chapter 13 problem S, chapter 14 problem J, and chapter 15 problem H.

The priced-out alternative is running that profile dp at judge time or searching tours directly, and the contest itself priced it: no team accepted the problem, and the official solution literally says precompute locally then submit a table.

A knight tours an m by n board visiting as many squares as possible in a closed loop whose straight jump segments never cross or touch, and the count is wanted for n up to 1e15, which is why nobody solved it at the finals and why the official solution is literally precompute locally then submit a table (solutions.pdf pp. 11-12). The derivation is a broken-profile dp over rows, state mixing visitation kind, vertical jump pattern, and canonicalized connectivity, run offline until each width's values fall into a cycle `v(n+P) = v(n)+A`, and the shipped solvers are exactly that final

step: per-m base tables plus cycle extrapolation, one algorithm and one table shared by all seven languages, with every value cross-checked against all 277 judge cases.

One correction to the naive table model is load-bearing: boards transpose, `v(m,n) = v(n,m)`, so the transpose happens before any lookup, and `v(8,3) = 6 = v(3,8)` is the one official sample the unswapped tables get wrong. Width 7 hides the other surprise, its cycle runs period 33 with add 142, and width 8 has no linear closed form at all, `v(17) = 80` and `v(19) = 92` both sit below any `6n - c` line, so the table window plus the cycle is the whole answer.

The worked run: trace the model on sample 1. The query is m = 6, n = 6, the transpose leaves it unchanged, and the width-6 branch reads its base window: n <= 15, so the answer is the table entry `j6_base[6-3]`, the value 12, with the cycle machinery idle. The regime boundary checks in one step on each side: the window's last entry `j6_base[12] = 46` serves n = 15, and the linear law `4n - 14` gives `4*15 - 14 = 46` there, so beyond the window the formula continues the table seamlessly. The entry itself is what the offline dp froze, the 12-square simple polygon of the section's figure, and the trace ends at the printed answer 12.

width m	base window	beyond the window
3	0 below 5, then 4 at n = 5	period 4, add 4, phases −2, −2, 2, 2
4	0 below 4	`2n - 4` from n = 4
5	`j5_base`, n = 3 to 14	add 26 per 10
6	`j6_base`, n = 3 to 15	`4n - 14`
7	`j7_base`, n = 3 to 65	add 142 per 33
8	`j8_base`, n = 4 to 19	add 36 per 6

```c
// v(m, n): boards transpose, so swap to m ≤ n first
static inline long long jtour(long long m, long long n) {
  if (m > n) {
    long long t = m;
    m = n;
    n = t;
  }
  if (m ≤ 2)
    return 0;
  if (m == 3) {
    if (n < 5)
      return 0;
    if (n == 5)
      return 4;
    // period 4, add 4, phase c[0..3] = -2, -2, 2, 2 around 4*floor(n/4)
    long long c[4] = {-2, -2, 2, 2};
    return 4 * (n / 4) + c[n % 4];
  }
  if (m == 4) {
    if (n < 4)
      return 0;
    return 2 * n - 4;
  }
  if (m == 5) {
    if (n ≤ 14)
      return j5_base[n - 3];
    long long k = (n - 15) / 10 + 1; // v(n) = v(n - 10k) + 26k
    return j5_base[n - 10 * k - 3] + 26 * k;
  }
  if (m == 6) {
    if (n ≤ 15)
      return j6_base[n - 3];
    return 4 * n - 14;
  }
  if (m == 7) {
    if (n ≤ 65)
      return j7_base[n - 3];
    long long k = (n - 66) / 33 + 1;
    return j7_base[n - 33 * k - 3] + 142 * k;
  }
  if (n < 4)
    return 0;
  if (n ≤ 19)
    return j8_base[n - 4];
  long long k = (n - 20) / 6 + 1;
  return j8_base[n - 6 * k - 4] + 36 * k;
}
```

listing 297: c: jtour, transpose first, per-m base arrays, cycle extrapolation by one division (source: icpc/samples-c/src/Ch09/ch09_jtable.h, lines 20-66)

```
  switch m {
  case 1, 2:
    return 0
  case 3:
    if n < 5 {
      return 0
    }
    if n == 5 {
      return 4
    }
    return 4*(n/4) + jtC3[n%4]
  case 4:
    if n < 4 {
      return 0
    }
    return 2*n - 4
  case 5:
    if n < 3 {
      return 0
    }
    if n <= 14 {
      return jtBase5[n-3]
    }
    k := (n - 5) / 10
    return jtBase5[n-10*k-3] + 26*k
  case 6:
    if n < 3 {
      return 0
    }
    if n <= 12 {
      return jtBase6[n-3]
    }
    return 4*n - 14
  case 7:
    if n < 3 {
      return 0
    }
    if n <= 65 {
      return jtBase7[n-3]
    }
    k := (n - 33) / 33
    return jtBase7[n-33*k-3] + 142*k
  default:
    if n < 4 {
      return 0
    }
```

listing 298: go: knightTour, the switch over m with each cycle's period and add (source: icpc/samples-go/ch09/ch09_jtable.go, lines 30-75)

```
static long jtour(long m, long n) {
  if (m > n) {
    long t = m;
    m = n;
    n = t;
  }
  if (m ≤ 2)
    return 0;
  if (m == 3) {
    if (n < 5)
      return 0;
    if (n == 5)
      return 4;
    // period 4, add 4, phase c[0..3] = -2, -2, 2, 2 around 4*floor(n/4)
    long[] c = {-2, -2, 2, 2};
    return 4 * (n / 4) + c[(int) (n % 4)];
  }
  if (m == 4) {
    if (n < 4)
      return 0;
    return 2 * n - 4;
  }
  if (m == 5) {
    if (n ≤ 14)
      return BASE5[(int) (n - 3)];
    long k = (n - 15) / 10 + 1; // v(n) = v(n - 10k) + 26k
    return BASE5[(int) (n - 10 * k - 3)] + 26 * k;
  }
  if (m == 6) {
    if (n ≤ 15)
      return BASE6[(int) (n - 3)];
    return 4 * n - 14;
  }
```

listing 299: java: jtour, transpose first, per-m base arrays, cycle extrapolation by one division (source: icpc/samples-java/src/Ch10/Jtable.java, lines 17-49)

```
    public static string Solve(string input)
    {
        var tok = Input.Longs(input);
        long m = tok[0], n = tok[1];
        if (m > n)
            (m, n) = (n, m);
        return V((int)m, n).ToString() + "\n";
    }

    private static long V(int m, long n)
    {
        switch (m)
        {
            case 1:
            case 2:
                return 0;
            case 3:
                if (n < 5)
                    return 0;
                if (n == 5)
                    return 4;
                long q = n / 4;
                return 4 * q + (n % 4 is 0 or 1 ? -2 : 2);
            case 4:
                return n < 4 ? 0 : 2 * n - 4;
            case 5:
                return Cycle(M5, 3, 14, 10, 26, n);
            case 6:
                if (n < 3)
                    return 0;
                return n ≤ 12 ? M6[n - 3] : 4 * n - 14;
            case 7:
                return Cycle(M7, 3, 65, 33, 142, n);
            default:
                return Cycle(M8, 4, 19, 6, 36, n);
        }
    }

    // v(n) = v(n - period) + add past hi; reduce with one division
    private static long Cycle(long[] tab, int lo, int hi,
        long period, long add, long n)
    {
        if (n < lo)
            return 0;
        if (n ≤ hi)
            return tab[n - lo];
        long k = (n - hi + period - 1) / period;
        long r = n - k * period;
        return tab[r - lo] + add * k;
    }
```

listing 300: c#: V over embedded tables, the Cycle helper reducing past the window with one division (source: icpc/samples/src/Ch09/PJ.cs, lines 25-74)

```
export function knightTour(m, n) {
  if (n < m) [m, n] = [n, m];
  switch (m) {
    case 1:
    case 2:
      return 0;
    case 3:
      if (n < 5) return 0;
      if (n === 5) return 4;
      return 4 * Math.floor(n / 4) + jtC3[n % 4];
    case 4:
      if (n < 4) return 0;
      return 2 * n - 4;
    case 5: {
      if (n < 3) return 0;
      if (n ≤ 14) return jtBase5[n - 3];
      const k = Math.floor((n - 5) / 10);
      return jtBase5[n - 10 * k - 3] + 26 * k;
    }
    case 6:
      if (n < 3) return 0;
      if (n ≤ 12) return jtBase6[n - 3];
      return 4 * n - 14;
    case 7: {
      if (n < 3) return 0;
      if (n ≤ 65) return jtBase7[n - 3];
      const k = Math.floor((n - 33) / 33);
      return jtBase7[n - 33 * k - 3] + 142 * k;
    }
    default: {
      if (n < 4) return 0;
      if (n ≤ 19) return jtBase8[n - 4];
      const k = Math.floor((n - 14) / 6);
      return jtBase8[n - 6 * k - 4] + 36 * k;
    }
  }
}
```

listing 301: javascript: knightTour, values stay under 2^53 so plain Numbers carry them exactly (source: icpc/samples-js/src/ch09-jtable.mjs, lines 20-56)

```
def _tour(m, n):
    if n < m:
        m, n = n, m
    if m in (1, 2):
        return 0
    if m == 3:
        if n < 5:
            return 0
        if n == 5:
            return 4
        return 4 * (n // 4) + _C3[n % 4]
    if m == 4:
        return 0 if n < 4 else 2 * n - 4
    if m == 5:
        if n < 3:
            return 0
        if n ≤ 14:
            return _BASE5[n - 3]
        k = (n - 5) // 10
        return _BASE5[n - 10 * k - 3] + 26 * k
    if m == 6:
        if n < 3:
            return 0
        if n ≤ 12:
            return _BASE6[n - 3]
        return 4 * n - 14
    if m == 7:
        if n < 3:
            return 0
        if n ≤ 65:
            return _BASE7[n - 3]
        k = (n - 33) // 33
        return _BASE7[n - 33 * k - 3] + 142 * k
    if n < 4:
        return 0
    if n ≤ 19:
        return _BASE8[n - 4]
    k = (n - 14) // 6
    return _BASE8[n - 6 * k - 4] + 36 * k
```

listing 302: python: `_tour`, the same tables and cycles embedded in the solver (source: icpc/samples-py/src/Ch09/pj.py, lines 97-135)

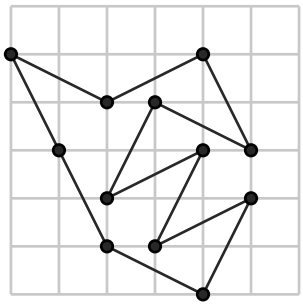
```
function jt.tour(m, n)
  if n < m then m, n = n, m end
  if m ≤ 2 then return 0 end
  if m == 3 then
    if n < 5 then return 0 end
    if n == 5 then return 4 end
    return 4 * (n // 4) + cyc3[(n % 4) + 1]
  elseif m == 4 then
    if n < 4 then return 0 end
    return 2 * n - 4
  elseif m == 5 then
    if n < 3 then return 0 end
    if n ≤ 14 then return base5[n - 2] end
    local k = (n - 5) // 10
    return base5[n - 10 * k - 2] + 26 * k
  elseif m == 6 then
    if n < 3 then return 0 end
    if n ≤ 12 then return base6[n - 2] end
    return 4 * n - 14
  elseif m == 7 then
    if n < 3 then return 0 end
    if n ≤ 65 then return base7[n - 2] end
    local k = (n - 33) // 33
    return base7[n - 33 * k - 2] + 142 * k
  else
    if n < 4 then return 0 end
    if n ≤ 19 then return base8[n - 3] end
    local k = (n - 14) // 6
    return base8[n - 6 * k - 3] + 36 * k
  end
end

return jt
```

listing 303: lua: jt.tour, 1-based table indexing, floor division in the extrapolation (source: icpc/samples-lua/ch09_jtable.lua, lines 21-53)

The fixture is the official sample `6 6`, expected `12\n`, and optimality is not hand-provable, the offline dp is the only witness, so the suites pin the sample plus the other official boards and one extrapolation case drawn from judge data: five CHECKs in c, two [Fact] methods in c# whose first runs all four official samples, five table cases in go, five ok lines in java, five assertions in javascript, five checks in python, five `T.eq` blocks in lua, the transpose case `8 3` at 6, `7 20` at 80, `2 6` at 0, and `3 1000000000000000` at 999999999999998. Integer notes: values reach 6e15, int64 in the native languages, and javascript documents the bound instead of switching types, plain Numbers are exact below `2^53 = 9.007e15` and `v(8, 10^15) = 5999999999999980` stays under it.

12 squares, every jump a knight move, no two segments crossing or touching



v(6,6) = 12, the table's pinned optimum

figure 123: the 6 x 6 board with a 12-square uncrossed closed tour, found by search and pinned here as one simple polygon

11.12. problem K: wireless is the new fiber

A new unbounded-bandwidth wireless technology is replacing the struggling fiber network. The old network is a set of n nodes and m fiber links, each link joining two different nodes, with at least one route between every pair of nodes and sometimes several links between the same pair for bandwidth. The replacement uses as few wireless links as possible while keeping exactly one way to travel between every pair of nodes, and every node was built with its current number of connections in mind: a node whose link count changes must be reorganized, which is costly. Design the new network so the fewest nodes change their number of connections, and print the count plus any achieving layout (statement).

The input is two integers n and m by the problemset pdf, 2 <= n <= 10^4 nodes numbered 0 to n-1 and 1 <= m <= 10^5 fiber links, then m lines of two distinct integers a and b, the endpoints of one link, with the pdf guaranteeing a path between every pair of nodes. The output is the smallest number of nodes whose link count changes, then a line with the node count and the wireless link count, then the links in the input's own format, any valid layout accepted, under the pdf's 2 second limit. The harness grades this problem on its first line, `2018/K` pinned to `first-line` in `ref/icpc/compare.json`, any achieving tree passing beside the count.

Official sample 1, reprinted byte for byte from the judge data pair `K-newfiber/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints with its before and after figures, the answer file's final blank line included in the bytes: the seven degrees are 4, 5, 5, 1, 1, 3, 3, the two degree-3 raises cost 2 each and fit inside the n-2 = 5 spare budget while neither degree-5 raise fits after them, so exactly three nodes change, nodes 1, 2, and 5 in the printed layout.

input:

```
7 11
0 1
0 2
0 5
0 6
1 3
2 4
1 2
1 2
1 5
2 6
5 6
```

expected output:

```
3
7 6
0 1
0 2
0 5
0 6
3 6
4 6
```

Recognition: n <= 10^4 nodes and m <= 10^5 links under the 2 second limit price the whole job at one degree sort plus one fifo build, `10^4 log 10^4` comparisons and a linear pass, with the counting argument, tree degrees sum to 2n-2, doing all the real work.

The cue is the double objective, fewest wireless links and fewest nodes whose link count changes, and the second reduces to handing out the n-2 spare degree increments cheapest-first, a local rule made safe by the exchange argument that any kept set of size k costs at least the k cheapest increments. Problem K is the 2018 face of the greedy with a safety proof family, and the same greedy with a safety proof cue drives chapter 10 problem H, chapter 12 problem A, and chapter 14 problem H.

The priced-out alternative is searching over subsets of nodes to keep, `2^10000` subsets, or building the tree by min-cost flow with degree bonuses, `10^4` nodes of flow against a sort, both paying orders more for an answer the exchange argument certifies directly.

A connected multigraph on n nodes is to be replaced by a tree, a node is happy when its tree degree equals its old degree, and the task is the fewest unhappy nodes plus any achieving tree. Only the input degrees matter: a tree's degrees are at least 1 and sum to 2n-2, so after the mandatory 1 per node there are n-2 spare increments, handing them out to keep degrees cheapest-first, by `(d_i - 1, id)` ascending, maximizes the kept set because any kept set of size k costs at least the k cheapest increments, and a leftover budget dumps onto one changed node (solutions.pdf pp. 12-13). The tree then builds by adding nodes in decreasing target degree, ties by id, onto a fifo of outstanding slots.

The worked run: trace the model on sample 1. The eleven printed links give degrees 4, 5, 5, 1, 1, 3 and 3 for nodes 0 through 6, a tree needs `2n-2 = 12` degree units against the mandatory 7, so `n-2 = 5` spare increments, and keeping node i costs `d_i - 1` of them. Cheapest first, nodes 3 and 4 cost 0 each, nodes 5 and 6 cost 2 each, spending 4 of 5, and the next keep, node 0, costs 3 more, 7 against 5, so at most four nodes keep their degrees and the count is `7 - 4 = 3`. The printed witness spends the same budget the other way, keeping nodes 0, 3, 4 and 6 for `3 + 0 + 0 + 2 = 5`, exactly the whole budget with no leftover to raise, and its six edges give node 0 degree 4, node 6 degree 3 and the rest 1, precisely those targets, any such fifo tree accepted beside the first-line count, and the trace ends at the printed answer `4 6`.

node	input degree	keep cost `d-1`	printed witness
0	4	3	kept, degree 4
1	5	4	changed, degree 1
2	5	4	changed, degree 1
3	1	0	kept, degree 1
4	1	0	kept, degree 1
5	3	2	changed, degree 1
6	3	2	kept, degree 3

```
  }
  int order[KN];
  for (int i = 0; i < n; i++)
    order[i] = i;
  qsort(order, (size_t)n, sizeof(int), cmp_keep);
  int target[KN];
  long long used = 0;
  int kept = 0;
  for (int i = 0; i < n; i++) {
    int id = order[i];
    int cost = deg[id] - 1;
    if (cost < 0)
      cost = 0;
    if (used + cost ≤ n - 2) {
      used += cost;
      target[id] = deg[id];
      kept++;
    } else {
      target[id] = 1;
    }
  }
  long long rho = (n - 2) - used;
  if (rho > 0) {
    // raise the smallest-id non-kept node with d ≠ 1
    for (int i = 0; i < n && rho > 0; i++)
      if (target[i] == 1 && deg[i] ≠ 1) {
        target[i] = 1 + (int)rho;
        rho = 0;
      }
  }
```

listing 304: c: the cheapest-first keep loop over the id-sorted order, then the leftover rho raise (source: icpc/samples-c/src/Ch09/pK.c, lines 55-84)

```
  sort.Slice(order, func(a, b int) bool {
    x, y := order[a], order[b]
    if deg[x] ≠ deg[y] {
      return deg[x] < deg[y]
    }
    return x < y
  })
  target := make([]int, n)
  for i := range target {
    target[i] = 1
  }
  budget := n - 2
  kept := make([]bool, n)
  for _, v := range order {
    cost := deg[v] - 1
    if cost ≥ 0 && cost <= budget {
      budget -= cost
      target[v] = deg[v]
      kept[v] = true
    }
  }
  if budget > 0 {
    // raise the smallest-id non-kept node with degree ≠ 1
    for v := 0; v < n; v++ {
      if !kept[v] && deg[v] ≠ 1 {
        target[v] = 1 + budget
        break
      }
    }
  }
  unhappy := 0
  for v := range target {
    if target[v] ≠ deg[v] {
      unhappy++
    }
  }
```

listing 305: go: the keep-set sort and budget walk, the raise for leftover budget (source: icpc/samples-go/ch09/pk.go, lines 33-68)

```
    Integer[] order = new Integer[n];
    for (int i = 0; i < n; i++)
      order[i] = i;
    // cheapest spare first: (d_i - 1) ascending, then id ascending
    Arrays.sort(order, (x, y) -> deg[x] - 1 != deg[y] - 1
        ? Integer.compare(deg[x] - 1, deg[y] - 1) : Integer.compare(x, y));
    int[] target = new int[n];
    int used = 0, kept = 0;
    for (int i = 0; i < n; i++) {
      int id = order[i];
      int cost = Math.max(deg[id] - 1, 0);
      if (used + cost ≤ n - 2) {
        used += cost;
        target[id] = deg[id];
        kept++;
      } else {
        target[id] = 1;
      }
    }
    int rho = (n - 2) - used;
    if (rho > 0) {
      // raise the smallest-id non-kept node with d ≠ 1
      for (int i = 0; i < n && rho > 0; i++)
        if (target[i] == 1 && deg[i] ≠ 1) {
          target[i] = 1 + rho;
          rho = 0;
        }
    }
```

listing 306: java: the cheapest-first keep loop over the boxed id sort, then the leftover rho raise (source: icpc/samples-java/src/Ch10/PK.java, lines 41-68)

```
        // keep the cheapest degree increments (d_i - 1, id) while they fit in n-2
        var order = Enumerable.Range(0, n).OrderBy(i => (long)(deg[i] - 1) * n + i);
        var target = new int[n];
        Array.Fill(target, 1);
        long budget = n - 2;
        foreach (int i in order)
        {
            if (deg[i] - 1 > budget)
                break;
            target[i] = deg[i];
            budget -= deg[i] - 1;
        }
        int raised = -1;
        if (budget > 0)
        {
            raised = Enumerable.Range(0, n)
                .First(i => target[i] == 1 && deg[i] != 1);
            if (raised >= 0)
                target[raised] = (int)(1 + budget);
        }

        // attach in (target desc, id asc) order to a fifo of open slots
        int changed = Enumerable.Range(0, n).Count(i => target[i] != deg[i]);
        var attach = Enumerable.Range(0, n)
            .OrderByDescending(i => target[i])
            .ThenBy(i => i)
            .ToList();
        var sb = new StringBuilder();
        sb.Append(changed).Append('\n')
            .Append(n).Append(' ').Append(n - 1).Append('\n');
        var fifo = new LinkedList<(int Node, int Left)>();
        fifo.AddLast((attach[0], target[attach[0]]));
        foreach (int i in Enumerable.Skip(attach, 1))
        {
            while (fifo.First!.Value.Left == 0)
                fifo.RemoveFirst();
            var head = fifo.First!.Value;
            sb.Append(head.Node).Append(' ').Append(i).Append('\n');
            fifo.First!.Value = (head.Node, head.Left - 1);
            fifo.AddLast((i, target[i] - 1));
        }
        return sb.ToString();
```

listing 307: c#: keep order as one long key, the raise, and the LinkedList fifo attach (source: icpc/samples/src/Ch09/PK.cs, lines 21-62)

```
  // keep-set: cheapest increments first, ties by id
  const order = deg.map((_, v) => v).sort((x, y) => deg[x] - deg[y] || x - y);
  const target = new Int32Array(n).fill(1);
  let budget = n - 2;
  const kept = new Uint8Array(n);
  for (const v of order) {
    const cost = deg[v] - 1;
    if (cost ≥ 0 && cost ≤ budget) {
      budget -= cost;
      target[v] = deg[v];
      kept[v] = 1;
    }
  }
  if (budget > 0) {
    // raise the smallest-id non-kept node with degree ≠ 1
    for (let v = 0; v < n; v++) {
      if (!kept[v] && deg[v] =/= 1) {
        target[v] = 1 + budget;
        break;
      }
    }
  }
  let unhappy = 0;
  for (let v = 0; v < n; v++) {
    if (target[v] =/= deg[v]) unhappy++;
  }
  // construction: decreasing target, ties by id ascending
  const build = target.map((_, v) => v).sort(
    (x, y) => target[y] - target[x] || x - y,
  );
  const out = [`${unhappy}\n`, `${n} ${n - 1}\n`];
  const queue = [[build[0], target[build[0]]]];
  for (let i = 1; i < build.length; i++) {
    const v = build[i];
    const head = queue[0];
    out.push(`${head[0]} ${v}\n`);
    head[1]--;
    if (head[1] === 0) queue.shift();
    const t = target[v] - 1;
    if (t > 0) queue.push([v, t]);
  }
  return out.join("");
```

listing 308: javascript: keep-set, raise, and the fifo construction in one pass (source: icpc/samples-js/src/ch09-pk-newfiber.mjs, lines 19-60)

```
    target = [1] * n
    budget = n - 2
    kept = [False] * n
    for v in sorted(range(n), key=lambda i: (deg[i], i)):
        cost = deg[v] - 1
        if 0 ≤ cost ≤ budget:
            budget -= cost
            target[v] = deg[v]
            kept[v] = True
    if budget > 0:
        for v in range(n):
            if not kept[v] and deg[v] ≠ 1:
                target[v] = 1 + budget
                break
    unhappy = sum(1 for v in range(n) if target[v] ≠ deg[v])
    build = sorted(range(n), key=lambda i: (-target[i], i))
    out = [str(unhappy), f"{n} {n - 1}"]
    queue = [[build[0], target[build[0]]]]
    for v in build[1:]:
        head = queue[0]
        out.append(f"{head[0]} {v}")
        head[1] -= 1
        if head[1] == 0:
            queue.pop(0)
        if target[v] - 1 > 0:
            queue.append([v, target[v] - 1])
    return "\n".join(out) + "\n"
```

listing 309: python: the whole solver, sort by degree then id, fifo of [node, remaining] (source: icpc/samples-py/src/Ch09/pk.py, lines 18-44)

```lua
local target = {}
for i = 0, n - 1 do target[i] = 1 end
local budget = n - 2
local kept = {}
for _, v in ipairs(order) do
  local cost = deg[v] - 1
  if cost ≥ 0 and cost ≤ budget then
    budget = budget - cost
    target[v] = deg[v]
    kept[v] = true
  end
end
if budget > 0 then
  -- raise the smallest-id non-kept node with degree ~= 1
  for v = 0, n - 1 do
    if not kept[v] and deg[v] ~= 1 then
      target[v] = 1 + budget
      break
    end
  end
end
local unhappy = 0
for v = 0, n - 1 do
  if target[v] ~= deg[v] then unhappy = unhappy + 1 end
end
local build = {}
for i = 0, n - 1 do build[i + 1] = i end
table.sort(build, function(a, b)
  if target[a] ~= target[b] then return target[a] > target[b] end
  return a < b
end)
local out = {}
out[#out + 1] = string.format("%d", unhappy)
out[#out + 1] = string.format("%d %d", n, n - 1)
-- fifo of {node, remaining} pairs
local qnode, qrem = { build[1] }, { target[build[1]] }
local qh, qt = 1, 1
for bi = 2, n do
  local v = build[bi]
  out[#out + 1] = string.format("%d %d", qnode[qh], v)
  qrem[qh] = qrem[qh] - 1
  if qrem[qh] == 0 then qh = qh + 1 end
  local t = target[v] - 1
  if t > 0 then
    qt = qt + 1
    qnode[qt] = v
    qrem[qt] = t
  end
end
return table.concat(out, "\n") .. "\n"
```

listing 310: lua: target fill, keep walk, raise, and the two-array fifo (source: icpc/samples-lua/ch09_pk.lua, lines 44-93)

The fixture is a 4-cycle where every node has degree 2, so every keep costs one increment and the budget `n-2 = 2` keeps exactly nodes 0 and 1, three keeps would cost 3, and the targets are `[2,2,1,1]` with no leftover: expected count 2 and the edges `0 1`, `0 2`, `1 3`. Suites: two CHECKs in c, five [Fact] methods in c#, four table cases in go, six ok lines in java, four assertions in javascript, six checks in python, six `T.eq` blocks in lua, the shared extras a path and a star that are already happy trees at count 0 and a five-cycle, with python, lua and java additionally validating

the five-cycle's output structurally, spanning, hanging off the root, and counting two. No integer notes, degrees and counts sit far below every machine word.

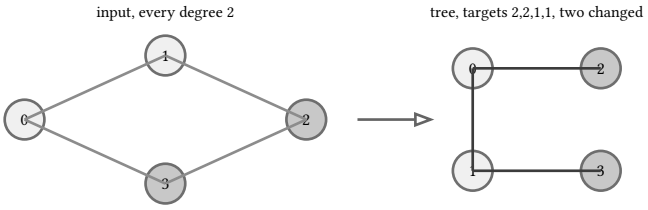


figure 124: the input 4-cycle beside the constructed tree, the two changed nodes shaded

11.13. across the seven languages

The same eleven problems, solved seven times each, and the table reads the year off the solvers' own choices:

language	size	integer vehicle	suite
c	11 solvers, 1979 lines, plus 192 lines of year headers	`long long` everywhere, run-length heaps with multiplicities in C, quarter units in H	33 CHECKs across 11 self-checks
go	package ch09, 1948 lines including the test table	`int64` exact, one hand-rolled `itoa64`	50 table cases
java	11 solvers and 4 helper twins, 1758 non-comment lines	`long` everywhere, the fnv-1a offset basis through `Long.parseUnsignedLong`, no BigInteger this year	52 ok lines across 11 self-checks
c#	11 solvers, 1233 lines, 329 test lines	`long`, big integers never needed this year	34 [Fact] methods
javascript	11 solvers and 3 modules, 1561 lines, 347 test lines	`BigInt` accumulator in C, exact BigInt64 time ranks in A, plain Numbers below 2^53 elsewhere	47 assertions in node:test
python	11 solvers, 1499 lines	native int, the I bitmask replacing the fenwick	52 checks
lua	11 solvers and 4 modules, 1765 lines	int64 exact, rank-packed sort keys in G	52 T.eq blocks

Two divergences shaped the year. Problem A's 1e18 timestamps forced javascript to rank times exactly before sorting while every other tree sorted native integers, java packing time and kind into one long, and problem C's 2.5e17 cost ceiling forced the BigInt accumulator in javascript alone. Everywhere else the seven languages agreed on the arithmetic, and the disagreements were structural instead: union-find against stack closure in B, four takes on one quadratic in E, a fenwick against a bitmask in I, and two inline table styles in D and G where the compiled languages reached for helper files. 320 suite assertions stand behind the chapter, 33 in c, 50 cases in go, 52 ok lines in java, 34 test methods in c#, 47 assertions in javascript, 52 checks in python, and 52 `T.eq` blocks in lua, counted from the frozen sources, not estimated.

sources: ICPC Foundation, icpc.global, the 2018 problemset at icpc2018.pdf and the solution sketches at finals2018solutions.pdf, both accessed 2026-09-14 and cached under `ref/icpc/2018/` with sha256 digests in `ref/icpc/INDEX.md`. The solutions pdf is the algorithm reference for every method and complexity claim in this chapter, cited where used. Judge data is local-only under `ref/icpc/2018/data/`, never committed, 931 input files mapped to answer pairs by the committed `ref/icpc/verify-data.ps1`, and every sample pair reprinted in this chapter is the byte-exact content of that problem's `sample-N.in` and `sample-N.ans` files, attributed in place, with the float-tolerant answers of A, D, and G and the any-layout outputs of H and K graded through the non-exact compare modes recorded in `ref/icpc/compare.json`. The committed suites carry the reader with crafted fixtures whose expectations are derived in the prose beside them. The java tree lands at books/icpc/samples-java/src/Ch10, java alone carrying

the post-renumber chapter numbers while every sibling tree keeps its historical Ch09 directory: 15 files, the eleven solvers plus the lnfact, geo, fenwick and jtable twins, 1758 non-comment source lines, 52 ok-line checks green under tools/run-java-samples.ps1 on the pinned oracle jdk 27, and the judge sweep returning 931 of 931 under ref/icpc/verify-data.ps1′s `-Lang java` mode. Every java judge mode reads its whole input through `Files.readString` and walks a cursor over the text, the mirror of c's io.h pointer, closing with one `System.out.print` no 2018 solver streams incrementally, so BufferedReader and StreamTokenizer never appear and Scanner is absent by the toolbox law, while the float answers of A, D and G print through `String.format` under Locale.ROOT.

12. icpc world finals 2019

The 2019 finals set 11 problems, letters A through K, and this chapter walks every letter in seven languages, the fixed order c, go, java, c#, javascript, python, and lua that the whole book uses. The statements are paraphrased in this book's own words and read at the source, the problemset pdf at icpc.global. The Foundation's own solution sketch set, the solutions pdf, is the algorithm reference for this chapter: every method and complexity claim below is settled against it, and the sketches' own account of the contest gives the set its shape, A the easiest letter with 128 accepted teams while C was accepted by none, I by five teams, K by four, and F by three, with Moscow State defending the title on a last-hour F. The set skews toward implementation, and the sketches admit the judges badly misjudged two letters, C and I, both of which this chapter works in full. Each section retells its letter in full, states the input and output contract with every bound, and reprints the letter's official sample pair byte for byte from the cached judge data before the seven solvers run.

12.1. the year-local helpers: a scan cursor, an order-statistics multiset, and a run treap

Three helper kits are year-local to 2019, and everything else the year's solvers import comes from the wave 1 toolbox chapters 2 through 8. The scan kit exists because the 2019 inputs reach 19.6 MB and a million tokens, past what the toolbox line splitter tolerates: the C solvers read the whole file into one buffer and a const cursor walks it, `next_int64` parsing digits in place, inside `Ch10/ch10_scan.h`, the twin the year's A, B and E solvers include. The other six languages carry no twin file here: C# splits through the toolbox `Input.Longs`, go's `scan10` and javascript's `makeScanner` are byte cursors inside the solver files, python splits the buffer natively, lua walks a byte cursor of its own, and java reads the whole file through a `BufferedReader` into one string that a static cursor and `nextLong` walk inside each solver, the scan family subsumed rather than ported. The cursor outlived its year: chapters 12, 13 and 14 face the same oversized inputs and their C solvers include the `ch11_scan.h` twin of this kit.

```c
typedef struct {
  const char *p;
} Scan;

static inline void scan_init(Scan *s, const char *buf) { s->p = buf; }

static inline void scan_skip(Scan *s) {
  while (*s->p == ' ' || *s->p == '\n' || *s->p == '\r' || *s->p == '\t')
    s->p++;
}

static inline long long next_int64(Scan *s) {
  scan_skip(s);
  long long sign = 1;
  if (*s->p == '-') {
    sign = -1;
    s->p++;
  }
  long long v = 0;
  while (*s->p ≥ '0' && *s->p ≤ '9')
    v = v * 10 + (*s->p++ - '0');
  return sign * v;
}

static inline double next_double(Scan *s) {
  scan_skip(s);
  char *end;
  double v = strtod(s->p, &end);
  s->p = end;
  return v;
}
```

listing 311: c: scan_skip steps the cursor past whitespace, next_int64 folds digits in place, next_double hands off to strtod (source: icpc/samples-c/src/Ch10/ch10_scan.h, lines 6-36)

The oset kit backs problem A with the year's trickiest queries: the order-statistics multiset over tiles ranked by height, with insert, erase, max, and the two queries the greedy lives on, `pred_strict(t)` for the tallest tile strictly below t, tie to the largest tile index, and `succ_strict(v)` for the shortest tile strictly above v, tie to the smallest index. Every language builds it the same way, a fenwick of counts over compressed tile ranks plus rank and select by binary descent down the counting tree, and the tie rules are load-bearing: they are what makes the seven walkthroughs print the judge's own witness byte for byte. The representations differ where the languages do. C and java compress heights to int32 buckets and keep per-bucket index stacks, java's sorted descending so the bucket's top and bottom index read off the ends, C# keeps a `SortedSet` of indexes per bucket, and the other four rank tiles by the composite key height shifted left 32 bits or index, so the tie rules fall out of the sort order. That composite key is a 64-bit product-shaped value, which is where the languages split on integers: go reads it as `uint64`, lua as an exact 64-bit integer, python as a native int, and javascript must leave `Number`, whose 53-bit mantissa cannot hold it, and sort `BigInt64Array` keys instead.

```c
static inline int oset_prefix(Oset *o, int i) {
  int s = 0;
  for (; i > 0; i -= i & (-i))
    s += o->cnt[i];
  return s;
}

// the k-th smallest height (1-based rank) as a 1-based bucket id
static inline int oset_select(Oset *o, int k) {
  int idx = 0;
  for (int step = 1 << 20; step > 0; step >>= 1)
    if (idx + step ≤ o->n && o->cnt[idx + step] < k) {
      idx += step;
      k -= o->cnt[idx];
    }
  return idx + 1;
}
```

listing 312: c: prefix count by fenwick sum, then select by binary descent over the counting tree, the k-th smallest height as a bucket id (source: icpc/samples-c/src/Ch10/ch10_oset.h, lines 69-85)

```go
// predStrict returns the largest key with height < t; among equal
// heights that is the largest index.
func (o *oset) predStrict(t uint64) (uint64, bool) {
  lo := o.rankOf(t << 32) // first rank with height ≥ t
  c := o.prefix(lo)
  if c == 0 {
    return 0, false
  }
  return o.ranks[o.selectK(c)], true
}

// succStrict returns the smallest key with height > v; among equal
// heights that is the smallest index.
func (o *oset) succStrict(v uint64) (uint64, bool) {
  hi := o.rankOf((v + 1) << 32) // first rank with height ≥ v+1
  c := o.prefix(hi)
  if c == o.size() {
    return 0, false
  }
  return o.ranks[o.selectK(c+1)], true
}
```

listing 313: go: pred and succ over composite uint64 ranks, the tie encoded in the key order (source: icpc/samples-go/ch10/ch10_oset.go, lines 81-101)

```
int prefix(int i) {
  int s = 0;
  for (; i > 0; i -= i & (-i))
    s += cnt[i];
  return s;
}

// the k-th smallest height (1-based rank) as a 1-based bucket id
int select(int k) {
  int idx = 0;
  for (int step = 1 << 20; step > 0; step >>= 1)
    if (idx + step ≤ n && cnt[idx + step] < k) {
      idx += step;
      k -= cnt[idx];
    }
  return idx + 1;
}

// largest tile index in the bucket (buckets sort descending)
int firstIdx(int bucket) {
  return stack[bucket][0];
}

// smallest tile index in the bucket
int lastIdx(int bucket) {
  return stack[bucket][stop[bucket] - 1];
}
```

listing 314: java: prefix count by fenwick sum, select by binary descent, the bucket's top and bottom index off the descending stack (source: icpc/samples-java/src/Ch11/Oset.java, lines 58-84)

```
  /// <summary>
  /// largest tile with height strictly below rank(t), largest index tie
  /// </summary>
  public (int Rank, int Idx)? PredStrict(int t)
  {
      int k = Prefix(t);
      if (k == 0)
          return null;
      int r = FindKth(k);
      return (r, _bucket[r].Max);
  }

  /// <summary>
  /// smallest tile with height strictly above rank(v), smallest index tie
  /// </summary>
  public (int Rank, int Idx)? SuccStrict(int v)
  {
      int k = Prefix(v + 1);
      if (k == Count)
          return null;
      int r = FindKth(k + 1);
      return (r, _bucket[r].Min);
  }
```

listing 315: c#: pred strict takes the k-th of the prefix, largest index from the SortedSet bucket, succ strict mirrors it (source: icpc/samples/src/Ch10/Oset.cs, lines 70-92)

```
  // the largest key with height < t; among equal heights that is the
  // largest index
  predStrict(t) {
    const lo = this.rankOf(t << 32n); // first rank with height ≥ t
    const c = this.prefix(lo);
    if (c === 0) return null;
    return this.ranks[this.selectK(c)];
  }
  // the smallest key with height > v; among equal heights that is the
  // smallest index
  succStrict(v) {
    const hi = this.rankOf((v + 1n) << 32n); // first rank with height ≥ v+1
    const c = this.prefix(hi);
    if (c === this.size()) return null;
    return this.ranks[this.selectK(c + 1)];
  }
```

listing 316: javascript: the same two queries over bigint composite keys, counts and ranks plain numbers (source: icpc/samples-js/src/ch10-oset.mjs, lines 57-72)

```
    def pred_strict(self, t):
        # largest key with height < t; equal heights tie to the
        # largest index, which the composite key already encodes
        lo, hi = 0, self.n
        ranks = self.ranks
        key = t << 32
        while lo < hi:
            mid = (lo + hi) // 2
            if ranks[mid] < key:
                lo = mid + 1
            else:
                hi = mid
        c = self.prefix(lo)
        if not c:
            return None
        return ranks[self.select_k(c)]

    def succ_strict(self, v):
        # smallest key with height > v; equal heights tie to the
        # smallest index
        lo, hi = 0, self.n
        ranks = self.ranks
        key = (v + 1) << 32
        while lo < hi:
            mid = (lo + hi) // 2
            if ranks[mid] < key:
                lo = mid + 1
            else:
                hi = mid
        c = self.prefix(lo)
        if c == self.size():
            return None
        return ranks[self.select_k(c + 1)]
```

listing 317: python: pred strict binary-searches the rank below t, succ strict the rank from v+1, select closes both (source: icpc/samples-py/src/Ch10/oset.py, lines 61-93)

```lua
-- largest live key with height < t
function oset.pred_strict(o, t)
  local lo = oset.rank_of(o, t << 32)
  local c = oset.prefix(o, lo - 1)
  if c == 0 then return nil end
  return o.ranks[oset.select_k(o, c) + 1]
end

-- smallest live key with height > v
function oset.succ_strict(o, v)
  local hi = oset.rank_of(o, (v + 1) << 32)
  local c = oset.prefix(o, hi - 1)
  if c == oset.size(o) then return nil end
  return o.ranks[oset.select_k(o, c + 1) + 1]
end
```

listing 318: lua: the same pair over exact 64-bit composite keys, one-based rank tables (source: icpc/samples-lua/ch10_oset.lua, lines 65-79)

The python helper carries its own self-check, six assertions pinning the tie rules directly: max picks the tallest tile with the largest index, pred below 9 ties to index 2, nothing lives strictly below 3, succ above 3 ties to index 0, nothing lives above 9, and size counts every insertion. Those six checks run beside the 11 problem suites and are counted separately in the sources paragraph. Java's twin carries no self-check of its own: its tie rules are pinned through A's suite, the only solver that queries it.

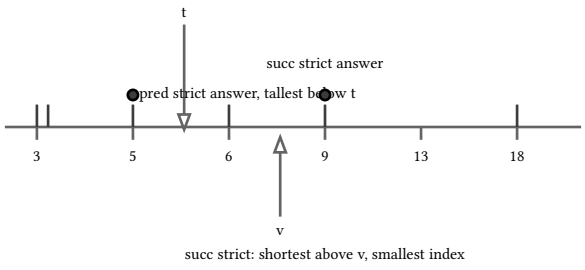


figure 125: one query pair of the multiset: pred strict takes the tallest rank below t, succ strict the shortest above v, both resolved as a fenwick select

The rtreap kit backs problem F with the year's heaviest contract: V, the piecewise-constant minimum-puncture function over the compressed cells, lives as a treap of equal-value runs with subtree min and max, lazy addition, and cell-boundary splits and merges, plus the balanced rebuild that repairs the heap order run cuts break. C carries the vehicle whole in `Ch10/ch10_rtreap.h`, C# splits problem F's sweep from the vehicle across `PFSweep.cs` and `PFRuns.cs`, go keeps `pf_runs.go` beside the solver in the year package, java keeps the `RTreap` class beside the solver the same way, javascript imports the `RunTreap` class from `ch10-rtreap.mjs`, python from `pf_runs.py`, and lua requires `ch10_pf_runs`. Java's priorities are uint32 `splitMix32` draws, so every priority comparison routes through `Integer.compareUnsigned` and the rebuild pins `0xFFFFFFFF - depth` so the rebuilt core sinks under fresh nodes; F's own tarp treap draws 31-bit priorities, where plain signed compares are already exact. The rebuild trigger is the split the F section measures: c, C#, go, java, and javascript flatten and rebuild on the fixed 4096-tarp cadence, python and lua flag the structure dirty when a walk passes the depth gate of 96, and lua's rebuilt core carries near-max priorities so the fresh 28-bit nodes sink under it instead of chaining at the root.

```
static void rpull(int x) {
  long long lo = rval[x], hi = rval[x];
  for (int i = 0; i < 2; i++) {
    int c = i ? rr[x] : rl[x];
    if (!c)
      continue;
    if (rmn[c] + radd[x] < lo)
      lo = rmn[c] + radd[x];
    if (rmx[c] + radd[x] > hi)
      hi = rmx[c] + radd[x];
  }
  rmn[x] = lo;
  rmx[x] = hi;
}

static void rapply(int x, long long d) {
  if (!x)
    return;
  rval[x] += d;
  radd[x] += d;
  rmn[x] += d;
  rmx[x] += d;
}

// move a node's pending addition onto its children so runs cut out of
// the subtree carry absolute values
static void rpush(int x) {
  if (radd[x]) {
    rapply(rl[x], radd[x]);
    rapply(rr[x], radd[x]);
    radd[x] = 0;
  }
}

static int rmerge(int a, int b) {
  if (!a)
    return b;
  if (!b)
    return a;
  rpush(a);
  rpush(b);
  if (rpri[a] > rpri[b]) {
    rr[a] = rmerge(rr[a], b);
    rpull(a);
    return a;
  }
  rl[b] = rmerge(a, rl[b]);
  rpull(b);
  return b;
}
```

listing 319: c: rpull folds child min and max past the lazy add, rapply and rpush keep cut runs absolute, rmerge joins disjoint cell ranges (source: icpc/samples-c/src/Ch10/ch10_rtreap.h, lines 41-90)

```c
// split so the left result holds exactly the cells below c and the
// right result the cells from c on; recursion depth is the treap
// height, capped by the periodic rebuild
static void rsplit(int x, int c, int *left, int *right) {
  if (!x) {
    *left = *right = 0;
    return;
  }
  rpush(x);
  if (rpos[x] ≥ c) {
    int a, b;
    rsplit(rl[x], c, &a, &b);
    rl[x] = b;
    rpull(x);
    *left = a;
    *right = x;
    return;
  }
  if (rend[x] ≤ c) {
    int a, b;
    rsplit(rr[x], c, &a, &b);
    rr[x] = a;
    rpull(x);
    *left = x;
    *right = b;
    return;
  }
  // the run straddles c: cut its tail into a fresh right-side node
  int y = ralloc(c, rend[x], rval[x]);
  rend[x] = c;
  int a, b;
  rsplit(rr[x], c, &a, &b);
  rr[x] = a;
  rpull(x);
  *left = x;
  *right = rmerge(y, b);
}
```

listing 320: c: rsplit cuts at cell boundary c, a straddling run's tail cut into a fresh right-side node (source: icpc/samples-c/src/Ch10/ch10_rtreap.h, lines 92-128)

```go
// splitCell splits the runs so the left result holds exactly the
// cells below c and the right result the cells from c on.
func (t *runTreap) splitCell(x, c int32) (int32, int32) {
  if x < 0 {
    return -1, -1
  }
  t.push(x)
  if t.pos[x] ≥ c {
    a, b := t.splitCell(t.l[x], c)
    t.l[x] = b
    t.pull(x)
    return a, x
  }
  if t.end[x] <= c {
    a, b := t.splitCell(t.r[x], c)
    t.r[x] = a
    t.pull(x)
    return x, b
  }
  // the run straddles c: cut its tail into a fresh right-side node
  y := t.alloc(c, t.end[x], t.val[x])
  t.end[x] = c
  a, b := t.splitCell(t.r[x], c)
  t.r[x] = a
  t.pull(x)
  return x, t.merge(y, b)
}

func (t *runTreap) merge(a, b int32) int32 {
  if a < 0 {
    return b
  }
  if b < 0 {
    return a
  }
  t.push(a)
  t.push(b)
  if t.pri[a] > t.pri[b] {
    t.r[a] = t.merge(t.r[a], b)
    t.pull(a)
    return a
  }
  t.l[b] = t.merge(a, t.l[b])
  t.pull(b)
  return b
}
```

listing 321: go: splitCell and merge over int32 run ids, the straddle cut into a fresh node, pushes before each walk (source: icpc/samples-go/ch10/pf_runs.go, lines 143-188)

```
void rpull(int x) {
  long lo = rval[x], hi = rval[x];
  for (int i = 0; i < 2; i++) {
    int c = i != 0 ? rr[x] : rl[x];
    if (c == 0)
      continue;
    if (rmn[c] + radd[x] < lo)
      lo = rmn[c] + radd[x];
    if (rmx[c] + radd[x] > hi)
      hi = rmx[c] + radd[x];
  }
  rmn[x] = lo;
  rmx[x] = hi;
}

void rapply(int x, long d) {
  if (x == 0)
    return;
  rval[x] += d;
  radd[x] += d;
  rmn[x] += d;
  rmx[x] += d;
}

// move a node's pending addition onto its children so runs cut out of
// the subtree carry absolute values
void rpush(int x) {
  if (radd[x] != 0) {
    rapply(rl[x], radd[x]);
    rapply(rr[x], radd[x]);
    radd[x] = 0;
  }
}

int rmerge(int a, int b) {
  if (a == 0)
    return b;
  if (b == 0)
    return a;
  rpush(a);
  rpush(b);
  if (Integer.compareUnsigned(rpri[a], rpri[b]) > 0) {
    rr[a] = rmerge(rr[a], b);
    rpull(a);
    return a;
  }
  rl[b] = rmerge(a, rl[b]);
  rpull(b);
  return b;
}
```

listing 322: java: rpull folds child min and max past the lazy add, rpush keeps cut runs absolute, rmerge joins on unsigned splitMix32 priorities through Integer.compareUnsigned (source: icpc/samples-java/src/Ch11/RTreap.java, lines 44-93)

```
    public int[] Order()
    {
        int n = _x1.Length;
        var keys = new long[2 * n];
        var ids = new int[2 * n];
        for (int i = 0; i < n; i++)
        {
            keys[2 * i] = _x1[i] << 1;
            keys[2 * i + 1] = (_x2[i] << 1) | 1;
            ids[2 * i] = i;
            ids[2 * i + 1] = i;
        }
        Array.Sort(keys, ids);
        var indeg = new int[n];
        var adj = new List<int>[n];
        for (int i = 0; i < n; i++)
            adj[i] = new List<int>(4);
        void Edge(int lo, int hi)
        {
            if (lo ≥ 0 && hi ≥ 0)
            {
                adj[lo].Add(hi);
                indeg[hi]++;
            }
        }
        for (int ev = 0; ev < 2 * n; ev++)
        {
            int t = ids[ev];
            _curX = keys[ev] >> 1;
            if ((keys[ev] & 1) == 0)
            {
                Insert(t);
                Edge(PredNode(t), t);
                Edge(t, SuccNode(t));
            }
            else
            {
                Edge(PredNode(t), SuccNode(t));
                Remove(t);
            }
        }
```

listing 323: c#: the endpoint sweep, live tarps ordered by height at the current x, predecessor and successor edges feeding the dag (source: icpc/samples/src/Ch10/PFSweep.cs, lines 132-172)

```
    // split so the left result holds exactly the cells below c and the
    // right result the cells from c on
    public (int, int) SplitCell(int x, int c)
    {
        if (x < 0)
            return (-1, -1);
        Push(x);
        if (_pos[x] ≥ c)
        {
            var (a, b) = SplitCell(_l[x], c);
            _l[x] = b;
            Pull(x);
            return (a, x);
        }
        if (_end[x] ≤ c)
        {
            var (a, b) = SplitCell(_r[x], c);
            _r[x] = a;
            Pull(x);
            return (x, b);
        }
        // this run straddles c: cut its tail into a fresh right node
        int y = Alloc(c, _end[x], _val[x]);
        _end[x] = c;
        var (a2, b2) = SplitCell(_r[x], c);
        _r[x] = a2;
        Pull(x);
        return (x, Merge(y, b2));
    }

    public int Merge(int a, int b)
    {
        if (a < 0)
            return b;
        if (b < 0)
            return a;
        Push(a);
        Push(b);
        if (_pri[a] > _pri[b])
        {
            _r[a] = Merge(_r[a], b);
            Pull(a);
            return a;
        }
        _l[b] = Merge(a, _l[b]);
        Pull(b);
        return b;
    }
```

listing 324: c#: SplitCell cuts at c with the straddle tail, Merge joins on priorities, pushes before either walk (source: icpc/samples/src/Ch10/PFRuns.cs, lines 138-185)

```
  // split the runs so the left result holds exactly the cells below c
  // and the right result the cells from c on
  splitCell(x, c) {
    if (x < 0) return [-1, -1];
    this.push(x);
    if (this.pos[x] ≥ c) {
      const [a, b] = this.splitCell(this.l[x], c);
      this.l[x] = b;
      this.pull(x);
      return [a, x];
    }
    if (this.end[x] ≤ c) {
      const [a, b] = this.splitCell(this.r[x], c);
      this.r[x] = a;
      this.pull(x);
      return [x, b];
    }
    // the run straddles c: cut its tail into a fresh right-side node
    const y = this.alloc(c, this.end[x], this.val[x]);
    this.end[x] = c;
    const [a, b] = this.splitCell(this.r[x], c);
    this.r[x] = a;
    this.pull(x);
    return [x, this.merge(y, b)];
  }

  merge(a, b) {
    if (a < 0) return b;
    if (b < 0) return a;
    this.push(a);
    this.push(b);
    if (this.pri[a] > this.pri[b]) {
      this.r[a] = this.merge(this.r[a], b);
      this.pull(a);
      return a;
    }
    this.l[b] = this.merge(a, this.l[b]);
    this.pull(b);
    return b;
  }
```

listing 325: javascript: the same splitCell and merge over plain arrays, values exact below the 2^53 sentinel (source: icpc/samples-js/src/ch10-rtreap.mjs, lines 124-163)

```
    def merge(self, a, b):
        # treap join over disjoint cell ranges: every key of a precedes
        # every key of b. the higher-priority root keeps its outer
        # subtree and the walk steps into its inner child; the walked
        # path is pulled bottom-up on the unwind
        if a < 0:
            return b
        if b < 0:
            return a
        pri, left, right = self.pri, self.left, self.right
        val, add, mn, mx = self.val, self.add, self.mn, self.mx
        path = []
        ap = path.append
        while a ≥ 0 and b ≥ 0:
            d = add[a]
            if d:
                for ch in (left[a], right[a]):
                    if ch ≥ 0:
                        val[ch] += d
                        add[ch] += d
                        mn[ch] += d
                        mx[ch] += d
                add[a] = 0
            d = add[b]
            if d:
                for ch in (left[b], right[b]):
                    if ch ≥ 0:
                        val[ch] += d
                        add[ch] += d
                        mn[ch] += d
                        mx[ch] += d
                add[b] = 0
            if pri[a] > pri[b]:
                ap(a)
                a = right[a]
            else:
                ap(~b)
                b = left[b]
        if len(path) > _GATE:
            self.dirty = True
```

listing 326: python: the iterative merge, the path stack sign-encoded, a walk past depth 96 flags the structure dirty (source: icpc/samples-py/src/Ch10/pf_runs.py, lines 77-116)

```
function rt.rebuild(t)
  local rpos, rfin, rval = {}, {}, {}
  local nr = 0
  local function walk(x, shift)
    if x > 0 then
      walk(t.l[x], shift + t.addv[x])
      nr = nr + 1
      rpos[nr], rfin[nr], rval[nr] = t.pos[x], t.fin[x], t.val[x] + shift
      walk(t.r[x], shift + t.addv[x])
    end
  end
  walk(t.root, 0)
  t.pos, t.fin, t.val, t.addv = rpos, rfin, rval, {}
  t.mn, t.mx, t.l, t.r, t.pri = {}, {}, {}, {}, {}
  t.free = {}
  t.next = 0
  t.dirty = false
  if nr == 0 then
    t.root = 0
    return
  end
  local function build(lo, hi, depth)
    if lo > hi then return 0 end
    local mid = (lo + hi) // 2
    local id = mid -- reuse the flattened slot as the new node id
    t.pos[id], t.fin[id], t.val[id] = rpos[mid], rfin[mid], rval[mid]
    t.addv[id], t.mn[id], t.mx[id] = 0, rval[mid], rval[mid]
    t.l[id], t.r[id] = 0, 0
    -- near-max priorities rank the rebuilt shape above every freshly
    -- allocated node, so new nodes sink instead of chaining at the
    -- root (go uses MaxUint32 - depth)
    t.pri[id] = 4294967295 - depth
    t.l[id] = build(lo, mid - 1, depth + 1)
    t.r[id] = build(mid + 1, hi, depth + 1)
    pull(t, id)
    return id
  end
  t.root = build(1, nr, 0)
end
```

listing 327: lua: the rebuild, a flat in-order walk then a depth-ranked core with near-max priorities (source: icpc/samples-lua/ch10_pf_runs.lua, lines 82-120)

12.2. problem A, azulejos

Maria and Joao are opening a small azulejo store in Porto, and the window display must fit on a single shelf holding exactly two rows: one of Joao's tiles in the back with one of Maria's tiles in front of it, every back tile hiding one front tile and vice versa. The hand-crafted tiles come in many sizes and both rows must stay visible to passers-by, so every back-row tile has to stand strictly taller than the front-row tile at its position. For the shoppers, each row must read in non-decreasing order of price from left to right, tiles of one price freely reordered subject to the height condition. The task is an ordering of the two rows meeting both constraints, or a verdict of impossibility (statement).

The input is one integer n, 1 <= n <= 5e5 by the problemset pdf, then four lines of n integers each: the back row's prices, the back row's heights, the front row's prices, the front row's heights, tiles numbered 1 to n in input order within each row and every price and height between 1 and 1e9. The output is two lines of n integers when

an ordering exists, a permutation of the back tile numbers over a permutation of the front ones, any valid pair accepted since the letter runs on a checker, or the single word impossible, under the pdf's 10 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `A-azulejos/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: the cheapest back block holds the lone price-1 tile of height 4, and its answer pairs front tile 4 under it before the price-2 blocks refill both rows.

input:

```
4
3 2 1 2
2 3 4 3
2 1 2 1
2 2 1 3
```

expected output:

```
3 2 4 1
4 2 1 3
```

Recognition: the bounds pay for the whole loop up front. With n <= 5e5 under the 10 second limit, sorting both rows costs about 4e5 log 5e5 key comparisons and each of the n positions spends two binary descents down a fenwick of counts, roughly 4e6 tree steps for the pairing, an O(n log n) shape with two orders of magnitude to spare.

The statement's cue is the double ordering: each row must read in non-decreasing price while every back tile stands strictly taller than the front tile it hides, so tiles inside one price block are interchangeable and the whole task is which front sits under which back, a strict-dominance pairing one order-statistics query answers per position. Problem A is the 2019 face of the greedy with a safety proof family, and greedy with a safety proof is the family of chapter 10 problem H, chapter 11 problem K, and chapter 14 problem H.

The tempting alternative, a bipartite matching over heights, is priced out by its own graph: at n = 5e5 the pairing graph holds up to 2.5e11 edges before any augmenting walk starts, and the price-block ordering is not an edge property a matching preserves, while the exchange argument certifying the greedy is a few lines and free.

The pinned greedy sorts each row by price, so positions fill cheapest block first and tiles inside one price block are interchangeable, then fills one position at a time: let t be the tallest back tile of the current blocks, take the front tile of largest height strictly below t, largest index on a tie, and pair it with the back tile of smallest height strictly above that front height, smallest index on a tie. Consuming the tallest coverable front and the shortest usable back leaves the shortest front leftovers and the tallest back ones, which the exchange argument shows optimal, and the moment no front tile sits strictly below t the answer is impossible (solutions.pdf p. 2, O(n log n) with the balanced-tree queries). The seven solvers run the same loop over the year-local multiset of the helper section.

The worked run: trace the model on sample 1. Both rows sort by price into blocks, back price 1 then 2, 2, 3 and front price 1, 1 then 2, 2. Position 1 tops out at the lone price-1 back tile 3, height 4, so the front pick is the tallest front strictly below 4, tile 4 at height 3, and the back pick the shortest strictly above 3, tile 3 itself. The back block empties and refills to the price-2 backs 2 and 4, both height 3. Position 2 queries t = 3, takes the price-1 front leftover tile 2 at height 2, and pairs it with back tile 2, smallest index on the height tie. The front block empties and refills to the price-2 fronts, tile 1 at height 2 and tile 3 at height 1. Position 3 queries t = 3 over the remaining back 4, takes front 1. Position 4 refills the backs to the price-3 tile 1 at height 2 and takes front 3 at height 1. The rows read back 3 2 4 1 over front 4 2 1 3, prices non-decreasing and every back strictly taller, and the trace ends at the printed answer `4 2 1 3`.

position	t, tallest back	front pick	back pick	pairing
1	4, tile 3	4, height 3	3, height 4	3 over 4

position	t, tallest back	front pick	back pick	pairing
2	3, tiles 2 and 4	2, height 2	2, height 3, tie	2 over 2
3	3, tile 4	1, height 2	4, height 3	4 over 1
4	2, tile 1	3, height 1	1, height 2	1 over 3

```
    // t = the tallest tile in the back block
    if (*ob.tot == 0)
      break;
    int top = oset_select(&ob, *ob.tot);
    // front tile with the largest height strictly below t
    int below = oset_prefix(&of, top - 1);
    if (below == 0)
      break; // impossible
    int fbucket = oset_select(&of, below);
    int fidx = of.stack[fbucket][0]; // largest index (bucket sorted desc)
    int fh = rank[fbucket - 1];
    // back tile with the smallest height strictly above fh
    int above = oset_prefix(&ob, brank(rank, hn, fh));
    int bcnt = *ob.tot - above;
    if (bcnt == 0)
      break; // impossible
    int bbucket = oset_select(&ob, above + 1);
    int bidx = ob.stack[bbucket][ob.stop[bbucket] - 1]; // smallest idx
    oset_del(&of, fbucket, fidx);
    oset_del(&ob, bbucket, bidx);
    ansf[fidx] = pos;
    ansb[bidx] = pos;
  }
```

listing 328: c: per position, the tallest back tile tops the query, the largest front bucket below it and the smallest back bucket above it answer, both erased (source: icpc/samples-c/src/Ch10/pA.c, lines 99-121)

```
  for pos := 0; pos < n; pos++ {
    if frontO.size() == 0 {
      price := frontP[frontOrder[fi]]
      for fi < n && frontP[frontOrder[fi]] == price {
        t := frontOrder[fi]
        frontO.add(rankIn(frontKeys, key(frontH[t], t)), 1)
        fi++
      }
    }
    if backO.size() == 0 {
      price := backP[backOrder[bi]]
      for bi < n && backP[backOrder[bi]] == price {
        t := backOrder[bi]
        backO.add(rankIn(backKeys, key(backH[t], t)), 1)
        bi++
      }
    }
    top, _ := backO.max()
    fk, ok := frontO.predStrict(top >> 32)
    if !ok {
      return "impossible\n"
    }
    bk, ok := backO.succStrict(fk >> 32)
    if !ok {
      return "impossible\n"
    }
    outBack[pos] = int(bk & 0xffffffff)
    outFront[pos] = int(fk & 0xffffffff)
    frontO.add(rankIn(frontKeys, fk), -1)
    backO.add(rankIn(backKeys, bk), -1)
  }
```

listing 329: go: block refill by price order, the composite key split back into height and tile index after each query (source: icpc/samples-go/ch10/pa.go, lines 72-102)

```
    for (int p = 0; p < n; p++) {
      if (ob.total() == 0) {
        if (bp ≥ n)
          break;
        int pr = back[bp].price();
        while (bp < n && back[bp].price() == pr) {
          ob.add(brank(rank, back[bp].height()), back[bp].idx());
          bp++;
        }
      }
      if (of.total() == 0) {
        if (fp ≥ n)
          break;
        int pr = front[fp].price();
        while (fp < n && front[fp].price() == pr) {
          of.add(brank(rank, front[fp].height()), front[fp].idx());
          fp++;
        }
      }
      // t = the tallest tile in the back block
      if (ob.total() == 0)
        break;
      int top = ob.select(ob.total());
      // front tile with the largest height strictly below t
      int below = of.prefix(top - 1);
      if (below == 0)
        break; // impossible
      int fbucket = of.select(below);
      int fidx = of.firstIdx(fbucket);
      int fh = rank[fbucket - 1];
      // back tile with the smallest height strictly above fh
      int above = ob.prefix(brank(rank, fh));
      int bcnt = ob.total() - above;
      if (bcnt == 0)
        break; // impossible
      int bbucket = ob.select(above + 1);
      int bidx = ob.lastIdx(bbucket);
      of.del(fbucket, fidx);
      ob.del(bbucket, bidx);
      ansf[fidx] = p;
      ansb[bidx] = p;
    }
```

listing 330: java: block refill per price, then per position the tallest back tops the query, firstIdx and lastIdx read the bucket's tie indexes, both erased (source: icpc/samples-java/src/Ch11/PA.java, lines 85-126)

```
        for (int pos = 0; pos < n; pos++)
        {
            if (front.Count == 0)
            {
                int price = fp[frontOrder[fi]];
                while (fi < n && fp[frontOrder[fi]] == price)
                {
                    int t = frontOrder[fi++];
                    front.Insert(Rank(fh[t]), t);
                }
            }
            if (back.Count == 0)
            {
                int price = bp[backOrder[bi]];
                while (bi < n && bp[backOrder[bi]] == price)
                {
                    int t = backOrder[bi++];
                    back.Insert(Rank(bh[t]), t);
                }
            }
            int top = vals[back.MaxRank()];
            var fHit = front.PredStrict(Rank(top));
            if (fHit == null)
                return "impossible\n";
            int fTile = fHit.Value.Idx;
            var bHit = back.SuccStrict(Rank(fh[fTile]));
            int bTile = bHit!.Value.Idx;
            front.Erase(fHit.Value.Rank, fTile);
            back.Erase(bHit.Value.Rank, bTile);
            backRow[pos] = bTile;
            frontRow[pos] = fTile;
        }
```

listing 331: c#: the same loop over SortedSet buckets, pred strict then succ strict, erase both hits (source: icpc/samples/src/Ch10/PA.cs, lines 45-76)

```
  for (let pos = 0; pos < n; pos++) {
    if (frontO.size() === 0) {
      const price = frontP[frontOrder[fi]];
      while (fi < n && frontP[frontOrder[fi]] === price) {
        const t = frontOrder[fi];
        frontO.add(rankIn(frontKeys, key(frontH[t], t)), 1);
        fi++;
      }
    }
    if (backO.size() === 0) {
      const price = backP[backOrder[bi]];
      while (bi < n && backP[backOrder[bi]] === price) {
        const t = backOrder[bi];
        backO.add(rankIn(backKeys, key(backH[t], t)), 1);
        bi++;
      }
    }
    const top = backO.max();
    const fk = frontO.predStrict(top >> 32n);
    if (fk === null) return "impossible\n";
    const bk = backO.succStrict(fk >> 32n);
    if (bk === null) return "impossible\n";
    outBack[pos] = Number(bk & 0xffffffffn);
    outFront[pos] = Number(fk & 0xffffffffn);
    frontO.add(rankIn(frontKeys, fk), -1);
    backO.add(rankIn(backKeys, bk), -1);
  }
```

listing 332: javascript: the loop with bigint composite keys, heights and indexes masked back out of each answer (source: icpc/samples-js/src/ch10-pa-azulejos.mjs, lines 69-95)

```
    for pos in range(n):
        if front_o.size() == 0:
            price = front_p[front_order[fi]]
            while fi < n and front_p[front_order[fi]] == price:
                t = front_order[fi]
                front_o.add(bisect_left(front_keys, (front_h[t] << 32) | t), 1)
                fi += 1
        if back_o.size() == 0:
            price = back_p[back_order[bi]]
            while bi < n and back_p[back_order[bi]] == price:
                t = back_order[bi]
                back_o.add(bisect_left(back_keys, (back_h[t] << 32) | t), 1)
                bi += 1
        top = back_o.max()
        fk = front_o.pred_strict(top >> 32)
        if fk is None:
            return "impossible\n"
        bk = back_o.succ_strict(fk >> 32)
        if bk is None:
            return "impossible\n"
        out_back[pos] = bk & 0xFFFFFFFF
        out_front[pos] = fk & 0xFFFFFFFF
        front_o.add(bisect_left(front_keys, fk), -1)
        back_o.add(bisect_left(back_keys, bk), -1)
```

listing 333: python: refill, max, pred strict, succ strict, then erase by rank, positions filled in one pass (source: icpc/samples-py/src/Ch10/pa.py, lines 30-53)

```
for p = 1, n do
  if oset.size(front_o) == 0 then
    local price = frontp[forder[fi]]
    while fi ≤ n and frontp[forder[fi]] == price do
      local t = forder[fi]
      oset.add(front_o, oset.rank_of(front_o, key(fronth[t], t)), 1)
      fi = fi + 1
    end
  end
  if oset.size(back_o) == 0 then
    local price = backp[border[bi]]
    while bi ≤ n and backp[border[bi]] == price do
      local t = border[bi]
      oset.add(back_o, oset.rank_of(back_o, key(backh[t], t)), 1)
      bi = bi + 1
    end
  end
  local top = back_o.ranks[oset.select_k(back_o, oset.size(back_o)) + 1]
  local fk = oset.pred_strict(front_o, top // 4294967296)
  if not fk then return "impossible\n" end
  local bk = oset.succ_strict(back_o, fk // 4294967296)
  if not bk then return "impossible\n" end
  out_back[p] = bk % 4294967296
  out_front[p] = fk % 4294967296
  oset.add(front_o, oset.rank_of(front_o, fk), -1)
  oset.add(back_o, oset.rank_of(back_o, bk), -1)
end
```

listing 334: lua: the same loop, integer division peeling the height off each composite key (source: icpc/samples-lua/ch10_pa.lua, lines 62-88)

Prices and heights stay under 1e9, so the arithmetic fits int32 everywhere, and javascript's `Number` is exact at that magnitude: the only wide value in the problem is the composite rank key, which javascript carries as a bigint and the other six as ordinary 64-bit or native integers.

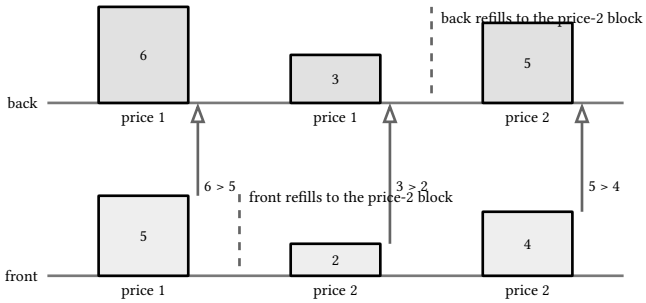


figure 126: the fixture's three positions: back tiles over front tiles, prices non-decreasing along both rows, the dashed lines where each row's price block refills

The crafted fixture is three tiles per row, back prices 1 1 2 with heights 3 6 5, front prices 1 2 2 with heights 5 2 4. The expectation, derived in the spec and pinned by every suite, is the back permutation 2 1 3 over the front permutation 1 2 3, both price rows non-decreasing, heights 6 over 5, 3 over 2, and 5 over 4. The C suite's three `CHECK` compares pin the fixture, official sample 1′s canonical 3 2 4 1 over 4 2 1 3, and official sample 2′s impossible, the C# suite's `Fixture` and `TallestCoverablePairsSortedOrder` facts pin it and an equal-price block, Go's two table cases take the fixture and the crafted impossible board, the javascript `it` blocks assert the fixture strings, python's first check and lua's `three`
`tiles` `pair` `tallest-coverable` `first` do the same, java's six pin the fixture, sample 1′s canonical pair, both impossible boards, and equal-price and cross-block refill chains, and every suite past the C one carries the crafted n = 2 board where a front 6 towers over every back leftover.

12.3. problem B, beautiful bridges

The Arch Bridges Construction company builds the classical form: pillars rise from the ground to a deck at height h above sea level, and consecutive pillars are joined by a semicircular arch of radius half their horizontal separation, its crown touching the deck at midspan and its springs sitting on the two pillars half a span below the deck. The ground profile is a piecewise-linear function given by n key points, intermediate pillars may stand only at key points with the first and last mandatory, and an arch may touch the ground but never extend below it, which rules some placements out entirely. A bridge with k pillars of heights h1 through hk at separations d1 through d(k-1) costs alpha times the pillar heights plus beta times the squared separations, and the task is the cheapest valid bridge across the whole profile, or impossible (statement).

The input is four integers n, h, alpha, beta on one line, 2 <= n <= 1e4 key points, 1 <= h <= 1e5 deck height, and cost factors 1 <= alpha, beta <= 1e4 by the problemset pdf, then n lines of two integers xi, yi, the key points with 0 <= x1 < x2 < ... < xn <= 1e5 and elevations 0 <= yi < h. The output is the minimum cost as one integer or the word impossible, under the pdf's 10 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `B-beautifulbridges/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: a five-point valley under a 60 high deck with alpha 18 and beta 2, and the cheapest bridge trades pillar shafts against squared spans for 6460.

input:

```
5 60 18 2
0 0
20 20
30 10
50 30
70 20
```

expected output:

```
6460
```

Recognition: n <= 1e4 key points under the 10 second limit prices the direct quadratic walk: 1e4 anchors each scanning leftward with running extrema, about 1e8 amortized-constant steps with room for two orders of magnitude more, provided the arch test itself collapses to O(1) per pair, since testing every interior point naively per pair is 1e12 visits.

The statement's cue is ordered placement with pairwise validity: pillars may stand only at key points, which arrive sorted by x, and an arch's legality couples exactly its two endpoints and the points between them, the shape of a dp over the last chosen pillar with a per-pair validity window. Problem B is the 2019 face of the chain dp over sorted positions family, and chapter 10 problem F carries the same shape.

The tempting alternative, a knuth-style or divide-and-conquer speedup on the transitions, is priced out by the windows: the bounds L(u,d) to R(u,d) do not satisfy the monotonicity the quadrangle inequality feeds on, and the plain O(n^2) already fits the limit a hundred times over, so the machinery buys nothing.

The model is the one the judge data settled on 2026-09-17 after an earlier deck-centered reading failed: the arch of span s between j and i is the upper semicircle with center (mid, h - s/2), a key point at anchor-distance u and depth d clears when d >= s/2 or it lies inside the arch's disk, which rearranges in s to L(u,d) <= s <= R(u,d) with L and R equal to 2(u+d) minus and plus 2 sqrt(2ud), and the far anchor reduces to s <= 2 d, an anchor shallower than half its span unable to reach its own springing. The disk is convex, so checking the key points suffices, no interpolated ground segment can cut the arc where the key points on both of its ends do not. The dp walks the last pillar with those bounds, dp i the minimum over valid predecessors j of dp j plus alpha times the last pillar height plus beta times the squared span, per anchor i moving j leftward while tracking the running minimum R and

maximum L over the interior points entered and breaking once the half-span passes the minimum R (solutions.pdf p. 2, O(n^2)). The seven dp loops follow.

The worked run: trace the model on sample 1. Pillar heights above the five points are 60, 40, 50, 30, 40, so dp starts at `18 * 60 = 1080` for the mandatory west pillar. Pillar 2 admits only j = 1 with span 20, dp `1080 + 720 + 2 * 20^2 = 2600`. Pillar 3 takes j = 2, dp 2600 + 900 + 200 = 3700, under j = 1′s 3780. Pillar 4 takes j = 2 with span 30, dp 2600 + 540 + 1800 = 4940, under j = 3′s 5040 and j = 1′s 6620. Pillar 5 takes j = 4 with span 20, dp 4940 + 720 + 800 = 6460, under j = 3′s 7620, j = 2′s 8320, and j = 1′s 11600: the whole-profile arch of span 70 is valid, points 2 and 3 deep enough at 40 and 50 against the springing depth 35, point 4 inside the disk with u = 50, d = 30, L = 160 - 2 sqrt(3000) = 50.46 <= 70, but its span alone costs `2 * 70^2 = 9800`. The winning chain 1, 2, 4, 5 pays `18 * (60 + 40 + 30 + 40) = 3060` in shafts plus `2 * (20^2 + 30^2 + 20^2) = 3400` in spans, and the trace ends at the printed answer `6460`.

pillar	shaft, `18 * height`	winning j, span	dp
1	`18 * 60 = 1080`	mandatory	1080
2	`18 * 40 = 720`	j = 1, span 20	2600
3	`18 * 50 = 900`	j = 2, span 10	3700
4	`18 * 30 = 540`	j = 2, span 30	4940
5	`18 * 40 = 720`	j = 4, span 20	6460

```
const long long INF = (long long)4e18;
DP[0] = alpha * D[0];
for (int i = 1; i < n; i++) {
  DP[i] = INF;
  // halves of the surd bounds: s/2 compared against (u+d) +- sqrt(2ud)
  double min_r = (double)D[i]; // near anchor: s ≤ 2*d_i
  double max_l = 0.0;
  for (int j = i - 1; j ≥ 0; j--) {
    if (j < i - 1) { // key point k = j+1 just became interior
      long long u = X[i] - X[j + 1];
      long long A = u + D[j + 1];
      double b = sqrt(2.0 * (double)u * (double)D[j + 1]);
      if (A + b < min_r)
        min_r = A + b;
      if (u > 2 * D[j + 1] && A - b > max_l)
        max_l = A - b; // only then is L binding (else L ≤ 2d ≤ s)
    }
    long long s = X[i] - X[j];
    if ((double)s / 2 > min_r + 1e-6)
      break; // spans only grow, minimum R only shrinks
    if (s ≤ 2 * D[j] && (double)s / 2 ≥ max_l - 1e-6 && DP[j] < INF) {
      long long cost = DP[j] + alpha * D[i] + beta * s * s;
      if (cost < DP[i])
        DP[i] = cost;
    }
  }
}
```

listing 335: c: dp over the last pillar, half-surds compared in double, the leftward walk breaking when the span outgrows the shrinking minimum R (source: icpc/samples-c/src/Ch10/pB.c, lines 43-69)

```
const inf = int64(1) << 62
dp := make([]int64, n)
dp[0] = alpha * d[0]
for i := 1; i < n; i++ {
  dp[i] = inf
  // running extrema over the entered points, evaluated as the
  // surds R = 2(A + sqrt(B)), L = 2(A - sqrt(B)); the near
  // anchor contributes R = 2*d_i and no L
  minRf := surdSum(d[i], 0)
  maxLf := 0.0
  for j := i - 1; j ≥ 0; j-- {
    if j < i-1 {
      k := j + 1 // key point that just became interior
      u := x[i] - x[k]
      A := u + d[k]
      B := 2 * u * d[k]
      if f := surdSum(A, B); f < minRf-1e-6 {
        minRf = f
      }
      if u > 2*d[k] {
        // far shallow point: blocks the arch until the
        // span grows past its L
        if f := surdDiff(A, B); f > maxLf+1e-6 {
          maxLf = f
        }
      }
    }
    s := x[i] - x[j]
    half := float64(s) / 2
    if half > minRf+1e-6 {
      break // span only grows, minimum R only shrinks
    }
    if s <= 2*d[j] && half ≥ maxLf-1e-6 && dp[j] < inf {
      if cost := dp[j] + alpha*d[i] + beta*s*s; cost < dp[i] {
        dp[i] = cost
      }
    }
  }
}
```

listing 336: go: the surd bounds factored into surdSum and surdDiff, the walk with the 1e-6 epsilon (source: icpc/samples-go/ch10/pb.go, lines 37-75)

```
final long INF = 4_000_000_000_000_000_000L;
DP[0] = alpha * D[0];
for (int i = 1; i < n; i++) {
  DP[i] = INF;
  // halves of the surd bounds: s/2 compared against (u+d) +- sqrt(2ud)
  double minR = D[i]; // near anchor: s ≤ 2*d_i
  double maxL = 0.0;
  for (int j = i - 1; j ≥ 0; j--) {
    if (j < i - 1) { // key point k = j+1 just became interior
      long u = X[i] - X[j + 1];
      long A = u + D[j + 1];
      double b = Math.sqrt(2.0 * u * D[j + 1]);
      if (A + b < minR)
        minR = A + b;
      if (u > 2 * D[j + 1] && A - b > maxL)
        maxL = A - b; // only then is L binding (else L ≤ 2d ≤ s)
    }
    long s = X[i] - X[j];
    if (s / 2.0 > minR + 1e-6)
      break; // spans only grow, minimum R only shrinks
    if (s ≤ 2 * D[j] && s / 2.0 ≥ maxL - 1e-6 && DP[j] < INF) {
      long cost = DP[j] + alpha * D[i] + beta * s * s;
      if (cost < DP[i])
        DP[i] = cost;
    }
  }
}
return DP[n - 1] ≥ INF ? "impossible\n" : DP[n - 1] + "\n";
```

listing 337: java: the dp over the last pillar, half-surds in double with the 1e-6 epsilon, the walk breaking when the half-span passes the shrinking minimum R (source: icpc/samples-java/src/Ch11/PB.java, lines 52-79)

```
        var dp = new long[n];
        Array.Fill(dp, Inf);
        dp[0] = alpha * d[0];
        for (int i = 1; i < n; i++)
        {
            // the walk keeps half-surds: s/2 against (u+d) -/+ sqrt(2ud);
            // the near anchor contributes R/2 = d_i and no L
            double minRHalf = d[i];
            double maxLHalf = 0;
            for (int j = i - 1; j ≥ 0; j--)
            {
                if (j < i - 1)
                {
                    int k = j + 1; // key point that just became interior
                    long u = x[i] - x[k];
                    long a = u + d[k];
                    double root = Math.Sqrt(2 * u * d[k]);
                    minRHalf = Math.Min(minRHalf, a + root);
                    if (u > 2 * d[k])
                        maxLHalf = Math.Max(maxLHalf, a - root);
                }
                long s = x[i] - x[j];
                double half = s / 2.0;
                if (half > minRHalf + Eps)
                    break; // spans only grow leftward, minimum R only shrinks
                if (s ≤ 2 * d[j] && half ≥ maxLHalf - Eps && dp[j] < Inf)
                {
                    long cost = dp[j] + alpha * d[i] + beta * s * s;
                    if (cost < dp[i])
                        dp[i] = cost;
                }
            }
        }
        return dp[n - 1] ≥ Inf ? "impossible\n" : dp[n - 1].ToString() + "\n";
```

listing 338: c#: the same walk keeping half-surds, far anchor checked as s <= 2 d at j, L only binding when u > 2 d (source: icpc/samples/src/Ch10/PB.cs, lines 32-65)

```
const inf = Infinity;
const dp = new Float64Array(n);
dp[0] = alpha * d[0];
for (let i = 1; i < n; i++) {
  dp[i] = inf;
  // running extrema over the entered points; the near anchor
  // contributes R = 2*d_i and no L
  let minRf = surdSum(d[i], 0);
  let maxLf = 0;
  for (let j = i - 1; j ≥ 0; j--) {
    if (j < i - 1) {
      const k = j + 1; // key point that just became interior
      const u = x[i] - x[k];
      const A = u + d[k];
      const B = 2 * u * d[k];
      const f = surdSum(A, B);
      if (f < minRf - 1e-6) minRf = f;
      if (u > 2 * d[k]) {
        // far shallow point: blocks the arch until the span grows
        // past its L
        const g = surdDiff(A, B);
        if (g > maxLf + 1e-6) maxLf = g;
      }
    }
    const s = x[i] - x[j];
    const half = s / 2;
    if (half > minRf + 1e-6) break; // span only grows, min R only shrinks
    if (s ≤ 2 * d[j] && half ≥ maxLf - 1e-6 && dp[j] < inf) {
      const cost = dp[j] + alpha * d[i] + beta * s * s;
      if (cost < dp[i]) dp[i] = cost;
    }
  }
}
if (dp[n - 1] ≥ inf) return "impossible\n";
return `${dp[n - 1]}\n`;
```

listing 339: javascript: the dp in Float64Array cells, costs under 2^53 so plain Numbers stay exact (source: icpc/samples-js/src/ch10-pb-beautifulbridges.mjs, lines 35-69)

```
    dp = [0] * n
    dp[0] = alpha * d[0]
    for i in range(1, n):
        best = INF
        # running extrema over the points entered so far; the near
        # anchor contributes R = d_i and no L
        min_rf = d[i]
        max_lf = 0.0
        for j in range(i - 1, -1, -1):
            if j < i - 1:
                k = j + 1
                u = x[i] - x[k]
                a = u + d[k]
                b = 2 * u * d[k]
                f = a + sqrt(b)
                if f < min_rf - EPS:
                    min_rf = f
                if u > 2 * d[k]:
                    # a far shallow point blocks the arch until the
                    # span grows past its L
                    f = a - sqrt(b)
                    if f > max_lf + EPS:
                        max_lf = f
            s = x[i] - x[j]
            half = s / 2
            if half > min_rf + EPS:
                break
            if s ≤ 2 * d[j] and half ≥ max_lf - EPS and dp[j] < INF:
                cost = dp[j] + alpha * d[i] + beta * s * s
                if cost < best:
                    best = cost
        dp[i] = best
    if dp[n - 1] ≥ INF:
        return "impossible\n"
    return f"{dp[n - 1]}\n"
```

listing 340: python: running extrema per anchor, the break when the half-span passes the shrinking minimum, costs in native ints (source: icpc/samples-py/src/Ch10/pb.py, lines 28-62)

```
local inf = 1 << 62
local dp = {}
dp[1] = alpha * ds[1]
for i = 2, n do
  dp[i] = inf
  -- running extrema over the interior points entered; the near
  -- anchor contributes R = 2*d_i and no L
  local min_rf = ds[i] + 0.0
  local max_lf = 0.0
  for j = i - 1, 1, -1 do
    if j < i - 1 then
      local k = j + 1
      local u = xs[i] - xs[k]
      local a = u + ds[k]
      local b2 = 2 * u * ds[k]
      local f = a + math.sqrt(b2)
      if f < min_rf - 1e-6 then min_rf = f end
      if u > 2 * ds[k] then
        f = a - math.sqrt(b2)
        if f > max_lf + 1e-6 then max_lf = f end
      end
    end
    local s = xs[i] - xs[j]
    local half = s / 2.0
    if half > min_rf + 1e-6 then break end
    if s ≤ 2 * ds[j] and half ≥ max_lf - 1e-6 and dp[j] < inf then
      local cost = dp[j] + alpha * ds[i] + beta * s * s
      if cost < dp[i] then dp[i] = cost end
    end
  end
end
if dp[n] ≥ inf then return "impossible\n" end
return string.format("%d\n", dp[n])
```

listing 341: lua: the same walk, inf at 1 << 62, surds through math.sqrt (source: icpc/samples-lua/ch10_pb.lua, lines 38-70)

Costs stay under alpha times 1e9 plus beta times 1e10, below 2e14, so int64 carries them in c, C#, go, and java, lua and python natively, and javascript's `Number` is exact below 2^53 with room to spare, which is why its dp cells are plain doubles. The surds compare in float64 with a 1e-6 epsilon: distinct values of A plus or minus sqrt(B) with integer A and B up to 2 times 1e5 squared differ by far more than that, so the epsilon decides order and ties exactly.

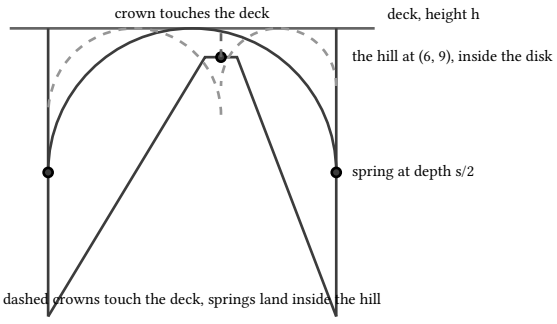


figure 127: the fixture in cross-section: the deck, the one valid arch whose springs sit at depth s/2, the hill passing under the arc, and the dashed short arches a shallow middle pillar cannot spring

The crafted fixture is a three-point ground, 0 0, 6 9, 10 0 under h = 10 with alpha = beta = 1. The single arch spans the whole profile, its springs at depth 5, the hill point at u = 6, d = 1 sits below the arc because L = 7.07 <= 10 <= R = 20.93, and the middle-pillar alternative dies on both short arches since 2 d = 2 there. Expected cost 120, two pillar shafts of 10 plus one span squared of 100. The C suite's four `CHECK` cases pin 120, the high-alpha 344 board,

official sample 1′s 6460, and official sample 2′s impossible, while the C# facts, Go's table cases, the javascript `it` blocks, and python's and lua's checks pin the same 120, 344, and impossible answers with the crafted ground-touching 1100 board sitting in sample 1′s slot, so the six sibling suites agree on three pins and split on the fourth here, java's five checks carrying both 6460 and 1100 and closing the split.

12.4. problem C, checks post facto

The board game club's draughts records fell into a puddle, and all that survived of a game is its first mover and a middle fragment of the move list. English draughts is played on the dark squares of an 8 by 8 board, numbered 1 to 32 row-wise from the top with Black at the top and White at the bottom. Black and White alternate turns, each piece is a man or a king, a man slides one dark square diagonally toward the far side or jumps an adjacent enemy piece to the empty square immediately beyond, removing it, while a king slides or jumps in any of the four diagonal directions. A jump chain repeats with the same piece as long as properly positioned victims remain. Captures are forced: if any jump exists at the start of a turn, the mover must jump and may not stop jumping with that piece while jumps remain, and a man reaching the farthest row is promoted to a king with the turn ending immediately, so a freshly promoted king cannot jump backward in the same turn. A simple move from a to b is written a-b and a jump chain a, b1, ..., bk as axb1xb2x...xbk. The task is a setup of pieces from which the recorded moves replay as a legal game, printed before and after, and the setup may not hold black men on the bottom row or white men on the top row, since those would already have been promoted (statement).

The input is a character c, B or W, naming the first mover, and an integer n, 1 <= n <= 100 moves by the problemset pdf, then n move lines in the notation above over the squares 1 to 32 numbered row-wise from the top, the rows holding 1 to 4, 5 to 8, and so on. The input is guaranteed to admit a legal reconstruction. The output is eight lines, each the before-board's row, one space, and the after-board's row, cells written with a dash for light squares, a dot for empty dark squares, lowercase b and w for the two colors' men and uppercase B and W for their kings, any valid setup accepted since the letter runs on a checker, all under the pdf's 1 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `C-checks/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: White steps 21-17, a black king chains the three jumps 13x22x31x24 sweeping all three white men it meets, and the white king answers 19x28 over the visiting king, the judge's witness keeping two bystander men that no rule needs.

input:

```
W 3
21-17
13x22x31x24
19x28
```

expected output:

```
-b-.-.-. -b-.-.-.
.-.-.-.- .-.-.-.-
-.-.-.-. -.-.-.-.
B-.-w-.- .-.-w-.-
-.-.-W-. -.-.-.-.
w-.-.-.- .-.-.-.-
-.-w-w-. -.-.-.-W
.-.-.-.- .-.-.-.-
```

Recognition: the board is 32 squares and the log at most 100 moves, so the replay itself is microscopic, 100 moves over a handful of squares each, and the only budget question is the search's branching: every repair adds one piece or one promotion and restarts, at most one piece per rule the log can trip, so a consistent witness costs a few dozen adds and the sole forks are blocker colors, binary and few.

The statement's cue is its reconstruction wording: the task wants a setup of pieces from which the recorded moves replay as a legal game, a hidden initial state rebuilt from a partial observation log, every rule the replay trips over turning into a constraint that adds a witness piece or crowns one. Problem C is the 2019 face of the constraint repair with backtracking family, and chapter 10 problem B is its only sibling.

The tempting alternative, enumerating setups directly, is priced out by the universe: five piece states over 32 squares is about 10^22 boards before the log gets a vote, while the replay-driven repair touches a dozen squares because every add is forced by a rule the log itself breaks.

The pinned solver replays from an empty setup and repairs: a missing source piece adds a man of the mover's color, a simple move while any capture exists blocks the smallest landing square of the cheapest kill, a man moving backward must have been a king at the start and is promoted in the setup, a jump over an empty square adds an enemy man there, a man landing on its promotion row mid-sequence must have been a king, and a jump chain that could continue is blocked at its smallest landing square. Every repair restarts the replay, blocker colors are choice points tried black then white, and the search forks over them depth first (solutions.pdf p. 3). The seven repair drivers follow.

The worked run: trace the model on sample 1. The replay from an empty setup repairs as the log trips it. Move 21-17 adds the white man at 21 and walks it to 17. Move 13x22x31x24 adds a black man at 13, jumps the fresh white at 17, finds victim 26 empty and adds a white man there, then on the restart crowns the mover, 31 to 24 being a backward hop, before victim 27 turns out empty too and adds a second white man. Move 19x28 adds a white man at 19. The next restart changes move 2′s ending: the stopped chain sits on 24 with the white man on 19 one diagonal away, so the no-continuation rule blocks its landing square 15, two rows up from 19, and tries black first. That blocker arms a white capture at move 1, the man on 19 jumping it forward onto 10, so the simple 21-17 is illegal and the forced-capture rule blocks square 10, black first again. One more restart crowns the mover on 19, its hop onto 28 running backward, and with both blockers and both kings aboard the replay runs consistent: the chain sweeps 17, 26 and 27, 19x28 sweeps the visitor, and the white king ends on 28. The model's witness carries the two black blockers where the judge's carries two bystander men, both boards legal under the checker, and the shared empty last row reads byte for byte, so the trace ends at the printed answer `.-.-.-.- .-.-.-.-`.

move	rule tripped	repair	restarts
21-17	source empty	add w at 21	yes
13x22x31x24	source empty	add b at 13	yes
13x22x31x24	victim 26 empty	add w at 26	yes
13x22x31x24	backward hop 31 to 24	crown 13	yes
13x22x31x24	victim 27 empty	add w at 27	yes
19x28	source empty	add w at 19	yes
13x22x31x24	chain could continue onto 19	block 15, black	yes
21-17	capture now exists, landing 10	block 10, black	yes
19x28	backward hop onto 28	crown 19	yes

```c
// drive deterministic repairs to completion, forking on blockers
static int solve_state(State *st) {
  for (int iter = 0; iter < 500; iter++) {
    int r = replay(st);
    if (r == 1)
      return 1;
    if (r == 0)
      return 0;
    if (r == 3) { // choice: black blocker, then white; a blocker on
      // its own promotion row is a king (men may not stand there).
      // csq is saved: the recursive first branch clobbers the global
      int csq = choice_sq;
      int kingb = sqrow(csq) == 7, kingw = sqrow(csq) == 0;
      State save = *st;
      add_piece(st, csq, kingb ? BK : BM, 0, kingb);
      if (solve_state(st))
        return 1;
      *st = save;
      add_piece(st, csq, kingw ? WK : WM, 1, kingw);
      if (solve_state(st))
        return 1;
      return 0;
    }
  }
  return 0;
}
```

listing 342: c: the repair driver forks on a blocker square black then white, kings on the promotion rows, restarts re-replay from the setup (source: icpc/samples-c/src/Ch10/pC.c, lines 227-252)

```
func pcPlayDepth(b0 map[int]pcPiece, moves [][]int, isCapture []bool, first string,
  depth int) (pcResult, bool) {
  for {
    res, repair := pcSimulate(b0, moves, isCapture, first)
    if repair == nil {
      return res, true
    }
    if debugC {
      println("DBG d", depth, "repair sq", repair.square,
        "black", repair.black, "king", repair.king,
        "choice", repair.choice, "b0", len(b0))
    }
    if repair.square < 0 {
      return pcResult{}, false
    }
    if repair.choice {
      if _, occupied := b0[repair.square]; occupied {
        // a blocker must land on an empty square
        return pcResult{}, false
      }
      for _, black := range []bool{true, false} {
        nb := make(map[int]pcPiece, len(b0)+1)
        for k, v := range b0 {
          nb[k] = v
        }
        nb[repair.square] = pcPiece{black: black, home: repair.square,
          king: pcPromo(black, pcRowOf(repair.square))}
        if out, ok := pcPlayDepth(nb, moves, isCapture, first,
          depth+1); ok {
          return out, true
        }
      }
      return pcResult{}, false
    }
    if existing, occupied := b0[repair.square]; occupied {
      if existing.black ≠ repair.black {
        return pcResult{}, false
      }
      if existing.king || !repair.king {
        // no progress: same piece already there (and a
        // promotion of a king, or a re-add, goes nowhere)
        return pcResult{}, false
      }
      existing.king = true
      b0[repair.square] = existing
      continue
    }
    b0[repair.square] = pcPiece{black: repair.black,
      king: repair.king, home: repair.square}
  }
}
```

listing 343: go: the drive loop, blocker choice points cloned as fresh maps, an occupied promotion restart dies as a no-progress branch (source: icpc/samples-go/ch10/pc.go, lines 117-167)

```java
// drive deterministic repairs to completion, forking on blockers
static boolean solveState(State st) {
  for (int iter = 0; iter < 500; iter++) {
    int r = replay(st);
    if (r == 1)
      return true;
    if (r == 0)
      return false;
    if (r == 3) { // choice: black blocker, then white; a blocker on
      // its own promotion row is a king (men may not stand there).
      // csq is saved: the recursive first branch clobbers the field
      int csq = choiceSq;
      boolean kingb = sqrow(csq) == 7, kingw = sqrow(csq) == 0;
      State save = new State();
      save.copyFrom(st);
      addPiece(st, csq, kingb ? BK : BM, 0, kingb);
      if (solveState(st))
        return true;
      st.copyFrom(save);
      addPiece(st, csq, kingw ? WK : WM, 1, kingw);
      if (solveState(st))
        return true;
      return false;
    }
  }
  return false;
}
```

listing 344: java: the repair driver forks on the blocker square black then white, kings on their promotion rows, a saved State replayed per restart (source: icpc/samples-java/src/Ch11/PC.java, lines 248-274)

```
        for (int m = 0; m < _moves.Count; m++)
        {
            char mover = (m % 2 == 0) ? _first : Opponent(_first);
            int[] sq = _moves[m];
            if (run[sq[0]] == '.')
            {
                // (a) missing source: the mover's man, king on own promotion row
                if (b0[sq[0]] != '.')
                    return (Verdict.Fail, null);
                return (Verdict.RepairNow,
                    new Repair(sq[0], Upgrade(sq[0], mover)));
            }
            if (char.ToLower(run[sq[0]]) != mover)
                return (Verdict.Fail, null);
            if (!_isCapture[m])
            {
                // simple move: captures are forced at turn start
                int? blockSq = ForcedCaptureBlock(run, mover);
                if (blockSq != null)
                    return (Verdict.RepairNow, new Repair(blockSq.Value, '?'));
                char piece = run[sq[0]];
                if (char.IsLower(piece)
                    && Math.Sign(RowOf(sq[1]) - RowOf(sq[0])) != Dir(mover))
                {
                    // (c) a man moving backward was a king at the start
                    int org = origin[sq[0]];
                    if (!char.IsLower(b0[org]))
                        return (Verdict.Fail, null);
                    return (Verdict.RepairNow, new Repair(org, 'P'));
                }
                if (run[sq[1]] != '.')
                    return (Verdict.Fail, null);
                run[sq[1]] = piece;
                origin[sq[1]] = origin[sq[0]];
                origin[sq[0]] = 0;
                run[sq[0]] = '.';
                if (char.IsLower(piece) && IsOwnPromotion(mover, sq[1]))
                    run[sq[1]] = char.ToUpper(piece);
                continue;
            }
```

listing 345: c#: the replay ladder, missing source, forced-capture block, backward man promoted at its origin square (source: icpc/samples/src/Ch10/PC.cs, lines 108-147)

```
// replay the moves from b0, repairing and restarting; choice points
// (blocker colors) are searched b then w depth-first, earliest first
function pcPlayDepth(b0, moves, isCapture, first) {
  for (;;) {
    const [res, repair] = pcSimulate(b0, moves, isCapture, first);
    if (repair === null) return [res, true];
    if (repair.square < 0) return [null, false];
    if (repair.choice) {
      if (b0.has(repair.square)) return [null, false]; // a blocker needs room
      for (const black of [true, false]) {
        const nb = new Map(b0);
        nb.set(repair.square, {
          black,
          king: pcPromo(black, pcRowOf(repair.square)),
          home: repair.square,
        });
        const [out, ok] = pcPlayDepth(nb, moves, isCapture, first);
        if (ok) return [out, true];
      }
      return [null, false];
    }
    const existing = b0.get(repair.square);
    if (existing) {
      if (existing.black !== repair.black) return [null, false];
      if (existing.king || !repair.king) return [null, false]; // no progress
      b0.set(repair.square, { ...existing, king: true });
      continue;
    }
    b0.set(repair.square, {
      black: repair.black,
      king: repair.king,
      home: repair.square,
    });
  }
}
```

listing 346: javascript: the same loop over Map boards, choice points black first, a re-add that makes no progress fails the branch (source: icpc/samples-js/src/ch10-pc-checks.mjs, lines 159-193)

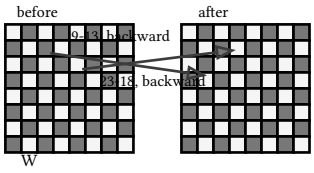
```
def _play(b0, moves, is_capture, black_first):
    b0 = dict(b0)
    while True:
        after, repair = _simulate(b0, moves, is_capture, black_first)
        if repair is None:
            return b0, after
        square, black, king, choice = repair
        if square < 0:
            return None
        if choice:
            if square in b0:
                return None
            for pick in (True, False):
                nb = dict(b0)
                nb[square] = (pick, _promo(pick, _row_of(square)), square)
                out = _play(nb, moves, is_capture, black_first)
                if out is not None:
                    return out
            return None
        existing = b0.get(square)
        if existing is not None:
            if existing[0] != black:
                return None
            if existing[1] or not king:
                return None
            b0[square] = (black, True, existing[2])
            continue
        b0[square] = (black, king, square)
```

listing 347: python: the drive loop restarts on every repair, forking over blocker colors with copied dicts, home squares carried in the piece tuple (source: icpc/samples-py/src/Ch10/pc.py, lines 144-171)

```
-- returns the repaired before-board and the after-board, or nil
local function play_depth(b0, moves, iscap, black_first)
  while true do
    local after, repair = simulate(b0, moves, iscap, black_first)
    if not repair then return b0, after end
    if repair[1] < 0 then return nil end
    if repair[4] then -- choice point: blocker color, b then w
      if b0[repair[1]] then return nil end
      for _, black in ipairs({ true, false }) do
        local nb = {}
        for k, v in pairs(b0) do nb[k] = v end
        nb[repair[1]] = { black = black,
                          king = pc_promo(black, row_of(repair[1])),
                          home = repair[1] }
        local b, a = play_depth(nb, moves, iscap, black_first)
        if a then return b, a end
      end
      return nil
    end
    local existing = b0[repair[1]]
    if existing then
      if existing.black ~= repair[2] then return nil end
      if existing.king or not repair[3] then return nil end
      existing.king = true
    else
      b0[repair[1]] = { black = repair[2], king = repair[3],
                        home = repair[1] }
    end
  end
end
```

listing 348: lua: the drive loop over sparse table boards, the same b-then-w fork at choice squares (source: icpc/samples-lua/ch10_pc.lua, lines 144-173)

The board is 32 entries and pieces are four kinds, so no language touches its integer edges here. The representations do split: C and java keep piece ids with a per-piece kind and color, C# renders pieces as chars and marks promotions with a dedicated 'P' repair, and the other four carry a {black, king, home} record per piece in a sparse map, the home square being what lets a backward-moving man be crowned where it started. Python's suite adds the rule checker the judge harness itself needs: it replays the fixture's pinned before-board through the move list and asserts the after-board comes out legal.



W

both movers stepped toward their own back row, so both were kings in the setup

B

B

figure 128: the fixture's two boards side by side, the black king and the white king each stepping backward, the moves a legal game only because both were kings at the start

The crafted fixture is the two-move game B, 23-18 then 9-13. Both moves run toward decreasing rows, black from row 6 and white from row 3, so both movers were kings at the start, and the pinned output is the eight before-

and-after row pairs with a white king on 9 and a black king on 23 moving to 13 and 18. The C suite's two `CHECK` string compares pin the fixture and the official first sample's canonical board, the C# facts pin the fixture plus three derived boards including a jump that adds its victim, Go's single table case carries the fixture, the javascript `it` blocks pin the fixture, a forward white man, and a backward jump, python's four checks add the forward man, the jump with two kings, and the rule-checker replay, and lua's four checks add a forward jump promoting on the last row and a no-capture game whose minimal board suffices, and java's four take the fixture, the official first sample, a forward white man, and a jump that crowns two kings.

12.5. problem D, circular dna

A bioinformatics group has reduced a strand of DNA to its gene markers: a sequence of n markers in a reading direction fixed by molecular properties, each marker the start or the end marker of a numbered gene type, the two marker spellings unique to their type. A type is properly nested in the sequence when the subsequence of all its markers, none left out, is built from the grammar siei, or siNei wrapping another nested structure N, or the concatenation AB of two nested structures, which is exactly the bracket condition that the type's running balance never dips below zero and ends at zero. Book 10′s chapter 35, combinatorics, counts these balanced-bracket families exactly. The strand under study is circular DNA, a closed loop, and whether a type nests depends on where the loop is cut. The task is the cut, made just before the p-th input marker, that maximizes the number of properly nested gene types, the smallest p breaking ties (statement).

The input is one integer n, 1 <= n <= 1e6 by the problemset pdf, then one line of n markers, each the letter s or e followed by a gene type id from 1 to 1e6, the sequence as cut from the circle at an arbitrary point. The output is p and m on one line, the best cut position and the number of types nesting there, under the pdf's 3 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `D-circular/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: nine markers in which type 2′s end lands before its start and type 42 stands alone, so only type 1′s three s and three e markers can ever close, and they close exactly when the circle is cut before marker 3.

input:

```
9
e1 e1 s1 e2 s1 s2 e42 e1 s1
```

expected output:

```
3 1
```

Recognition: n <= 1e6 markers under the 3 second limit prices exactly two linear passes: 1e6 steps to collect per type the total and the minimum prefix balance, 1e6 more to rotate the cut past every marker, an O(n) shape with three orders of magnitude to spare, so the solution never touches a marker twice after the first pass.

The statement's cue is the circular cut: whether a type nests depends on where the loop is cut, nesting is the bracket condition on one type's running balance, and the whole question is which cuts leave every type's balance at its minimum, one rotating aggregate per type. Problem D is the 2019 face of the event sweep over a sorted axis family, and the same event sweep over a sorted axis cue drives chapter 11 problem A, chapter 11 problem H, chapter 12 problem J, and chapter 15 problem L.

The tempting alternative, rotating the marker string per cut and re-checking bracket balance per type, is priced out at 1e6 cuts times 1e6 markers, 1e12 visits against a 3 second budget that holds about 1e9, three orders of magnitude past the limit.

The one-pass form: a balanced type is nested at cut p exactly when its prefix balance at p-1 equals its minimum prefix balance over all prefixes, so the solver precomputes per type the total and the minimum, then rotates the

cut past each marker and maintains the count of types whose running balance equals its minimum (solutions.pdf p. 3, O(n)). The seven sweeps follow.

The worked run: trace the model on sample 1. The first pass collects per type the total and minimum prefix balance: type 1's markers at positions 1, 2, 3, 5, 8, 9 run the balance −1, −2, −1, 0, −1, 0, total 0 and minimum −2 after position 2; type 2 runs −1 then 0, total 0, minimum −1; type 42 totals −1 and never nests. A type nests at cut p exactly when total 0 meets prefix balance p-1 equal to the minimum, so the sweep starts at cut 1 with every balance 0 against minima −2 and −1, nothing nested. Cut 2 passes one e of type 1, balance −1, still short. Cut 3 has passed both e's, balance −2 equals the minimum, and type 2's balance 0 misses its −1, so exactly one type nests and m = 1. Type 2 reaches its minimum only at cut 5, where type 1 sits at −1, so no cut nests two and the smallest best cut wins, and the trace ends at the printed answer 3 1.

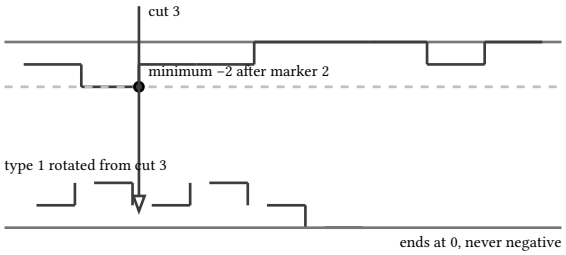


figure 129: type 1's prefix balance over the nine markers dipping to its minimum −2 after marker 2, and the rotated balance from cut 3 staying at or above zero

```
// cut 1 (before marker 1): count types with total 0 and bal == min
for (int t = 1; t ≤ 1000000; t++)
  if (isType[t] && tot[t] == 0 && mnr[t] == 0)
    cnt++;
int best = cnt, bestp = 1;
if (best == 0) {
  best = 0;
  bestp = 1;
}
// sweep the cut past each marker q: apply sgn[q] to cur[type]
for (int q = 1; q ≤ n; q++) {
  int t = typ[q - 1], s = sgn[q - 1];
  if (tot[t] == 0) {
    int before = (cur[t] == mnr[t]);
    cur[t] += s;
    int after = (cur[t] == mnr[t]);
    cnt += after - before;
  }
  if (cnt > best) {
    best = cnt;
    bestp = q + 1;
  }
}
```

listing 349: c: the cut-1 count over types with total 0 and min 0, then one sweep applying each passed marker's sign to its type's balance (source: icpc/samples-c/src/Ch10/pD.c, lines 69-91)

```
  for i := 0; i < n; i++ {
    t := types[i]
    cur[t] += sign[i]
    total[t] += sign[i]
    if cur[t] < minBal[t] {
      minBal[t] = cur[t]
    }
  }
  // count for cut 1: cur = 0 per type (balances before marker 1);
  // only types that actually appear can nest
  present := make([]bool, m)
  for _, t := range types {
    present[t] = true
  }
  nested := make([]bool, m)
  count := 0
  for t := 1; t < m; t++ {
    if present[t] && total[t] == 0 && minBal[t] == 0 {
      nested[t] = true
      count++
    }
  }
  bestCount, bestCut := count, 1
  for i := 0; i < n; i++ {
    // moving the cut past marker i adds the marker to the
    // balances that precede the new cut
    t := types[i]
    if total[t] ≠ 0 {
      continue
    }
    was := nested[t]
    cur[t] += sign[i]
    now := cur[t] == minBal[t]
    if now && !was {
      nested[t] = true
      count++
    } else if !now && was {
      nested[t] = false
      count--
    }
    if count > bestCount {
      bestCount, bestCut = count, i+2
    }
  }
```

listing 350: go: int32 per-type arrays sized by the largest type id, the sweep entering each marker into cur and toggling nested (source: icpc/samples-go/ch10/pd.go, lines 32-75)

```
    // per-type minimum prefix balance
    int[] pt = new int[1_000_001];
    for (int i = 0; i < n; i++) {
      pt[typ[i]] += sgn[i];
      if (pt[typ[i]] < mnr[typ[i]])
        mnr[typ[i]] = pt[typ[i]];
    }
    // cut 1 (before marker 1): count types with total 0 and bal == min
    int cnt = 0;
    for (int t = 1; t ≤ maxType; t++)
      if (isType[t] && tot[t] == 0 && mnr[t] == 0)
        cnt++;
    int best = cnt, bestp = 1;
    // sweep the cut past each marker q: apply sgn[q] to cur[type]
    for (int q = 1; q ≤ n; q++) {
      int t = typ[q - 1], s = sgn[q - 1];
      if (tot[t] == 0) {
        boolean before = cur[t] == mnr[t];
        cur[t] += s;
        boolean after = cur[t] == mnr[t];
        cnt += (after ? 1 : 0) - (before ? 1 : 0);
      }
      if (cnt > best) {
        best = cnt;
        bestp = q + 1;
      }
    }
    return bestp + " " + best + "\n";
```

listing 351: java: per-type minimum prefix balances, the cut-1 count over types with total 0 and min 0, the sweep toggling nested as each marker moves its type's balance (source: icpc/samples-java/src/Ch11/PD.java, lines 49-76)

```
        // balanced types nested at cut p iff bal(p-1) == min; sweep the cut
        foreach (int t in total.Keys)
            cur[t] = 0;
        int nested = 0;
        foreach (var (t, tot) in total)
            if (tot == 0 && cur[t] == minBal[t])
                nested++;
        int bestCount = nested, bestCut = 1;
        for (int q = 1; q ≤ n; q++)
        {
            int t = type[q - 1];
            bool was = total[t] == 0 && cur[t] == minBal[t];
            cur[t] += sign[q - 1];
            bool now = total[t] == 0 && cur[t] == minBal[t];
            if (!was && now)
                nested++;
            else if (was && !now)
                nested--;
            if (q < n && nested > bestCount)
            {
                bestCount = nested;
                bestCut = q + 1;
            }
        }
```

listing 352: c#: dictionaries per type, the nested flag recomputed as cur crosses min, best cut updated only on strict improvement (source: icpc/samples/src/Ch10/PD.cs, lines 39-62)

```
  for (let i = 0; i < n; i++) {
    const t = types[i];
    cur[t] += sign[i];
    total[t] += sign[i];
    if (cur[t] < minBal[t]) minBal[t] = cur[t];
  }
  // count for cut 1: cur = 0 per type (balances before marker 1);
  // only types that actually appear can nest
  const present = new Uint8Array(m);
  for (let i = 0; i < n; i++) present[types[i]] = 1;
  const nested = new Uint8Array(m);
  let count = 0;
  for (let t = 1; t < m; t++) {
    if (present[t] && total[t] === 0 && minBal[t] === 0) {
      nested[t] = 1;
      count++;
    }
  }
  let bestCount = count, bestCut = 1;
  for (let i = 0; i < n; i++) {
    // moving the cut past marker i adds the marker to the balances
    // that precede the new cut
    const t = types[i];
    if (total[t] !== 0) continue;
    const was = nested[t];
    cur[t] += sign[i];
    const now = cur[t] === minBal[t];
    if (now && !was) {
      nested[t] = 1;
      count++;
    } else if (!now && was) {
      nested[t] = 0;
      count--;
    }
    if (count > bestCount) {
      bestCount = count;
      bestCut = i + 2;
    }
  }
  return `${bestCut} ${bestCount}\n`;
```

listing 353: javascript: typed arrays for balances and a Uint8Array of nested flags, the same rotation sweep (source: icpc/samples-js/src/ch10-pd-circular.mjs, lines 38-77)

```
    for i in range(n):
        t = types[i]
        cur[t] += sign[i]
        total[t] += sign[i]
        if cur[t] < min_bal[t]:
            min_bal[t] = cur[t]
        present[t] = 1
    nested = bytearray(m)
    count = 0
    for t in range(1, m):
        if present[t] and total[t] == 0 and min_bal[t] == 0:
            nested[t] = 1
            count += 1
    best_count, best_cut = count, 1
    for i in range(n):
        t = types[i]
        if total[t]:
            continue
        was = nested[t]
        cur[t] += sign[i]
        now = cur[t] == min_bal[t]
        if now and not was:
            nested[t] = 1
            count += 1
        elif not now and was:
            nested[t] = 0
            count -= 1
        if count > best_count:
            best_count, best_cut = count, i + 2
    return f"{best_cut} {best_count}\n"
```

listing 354: python: one pass for total and minimum prefix balance, one sweep moving the cut, the count flipped per type (source: icpc/samples-py/src/Ch10/pd.py, lines 23-52)

```lua
local total, minbal, bal = {}, {}, {}
for i = 1, n do
  local t = typ[i]
  local b = (bal[t] or 0) + sg[i]
  bal[t] = b
  if total[t] == nil then
    total[t] = 0
    minbal[t] = 0
  end
  total[t] = total[t] + sg[i]
  if b < minbal[t] then minbal[t] = b end
end
-- cut 1 counts types whose minimum prefix balance is 0 (the
-- balance before marker 1); the sweep then adds each passed
-- marker's sign, so cur tracks bal(p-1) for balanced types
local cur, nested, cnt = {}, {}, 0
for t, b in pairs(bal) do
  cur[t] = b
  if total[t] == 0 and minbal[t] == 0 then
    nested[t] = true
    cnt = cnt + 1
  end
end
local best_cnt, best_cut = cnt, 1
for q = 1, n do
  local t = typ[q]
  if total[t] == 0 then
    local was = nested[t] or false
    cur[t] = cur[t] + sg[q]
    local now = cur[t] == minbal[t]
    if now and not was then
      nested[t] = true
      cnt = cnt + 1
    elseif not now and was then
      nested[t] = false
      cnt = cnt - 1
    end
    if cnt > best_cnt then
      best_cnt = cnt
      best_cut = q + 1
    end
  end
end
return string.format("%d %d\n", best_cut, best_cnt)
```

listing 355: lua: hash tables per type over sparse ids, cur seeded from the final balances, the sweep in marker order (source: icpc/samples-lua/ch10_pd.lua, lines 36-79)

Balances live in plus or minus n, so int32 carries them and no language crosses an integer edge. The representation splits on how the per-type arrays are sized: c, go, javascript, and java allocate by the largest type id up to 1e6, C# and lua keep dictionaries over the ids that appear, and python sizes its lists off max(types), the dense and sparse wings of the same sweep.

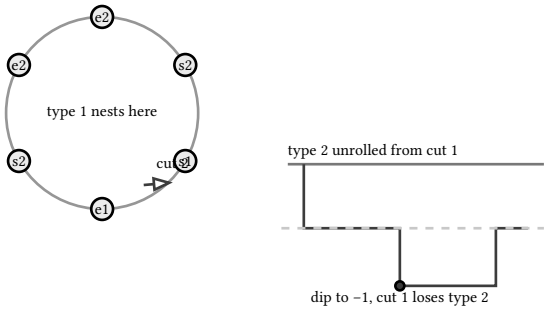


figure 130: the fixture as a ring of six markers with the winning cut before s1, and the unrolled balance of type 2 dipping to −1 at the losing linearization

The crafted fixture is six markers, e1 s1 s2 e2 e2 s2. Type 1 has balance −1 then 0 with minimum −1, type 2 runs 1, 0, −1, 0 with minimum −1, both totals zero, and the sweep finds cut 2 first holding one nested type, so the pinned answer is 2 1. The C suite runs five `CHECK` cases including this fixture, both official samples, the single marker s7 pinning 1 0, and the wrap pair e1 s1 pinning 2 1, the C# suite runs five with the same marker and wrap pins over crafted boards, Go's two table cases take the fixture and the lone marker, the javascript `it` blocks take those plus the wrap pair, and python and lua each run five checks with the wrap pair aboard, java's six pinning the fixture, both samples, the lone marker, the wrap pair, and an unbalanced type, so the wrap case where the cut travels all the way around to nest a reversed pair is pinned in six of the seven suites and the fixture in all seven.

12.6. problem E, dead-end detector

The town council wants dead-end signs placed as sparingly as the rules allow. The road map is a collection of locations joined by two-way streets, and a street's x-entrance earns a sign when, after entering that street from x, a driver cannot come back to x without making a U-turn, an immediate 180-degree reversal. Signs are then pruned for redundancy: if street S carries a sign at its x-entrance and street T one at its y-entrance, and a driver entering S from x can reach y and enter T without any U-turn, the sign at y on T is redundant and stays unbuilt. The task is the complete placement of non-redundant signs (statement).

The input is two integers n and m, 1 <= n <= 5e5 locations and 0 <= m <= 5e5 streets by the problemset pdf, then m lines of two integers v and w, 1 <= v < w <= n, every location pair named at most once. The output is the sign count k on one line, then k lines of v and w marking a sign at the v-entrance of the street joining v and w, sorted ascending by v then by w, under the pdf's 5 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `E-deadend/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: a triangle, where no entrance is a dead end, plus a loose two-street path, where only the two leaf entrances keep their signs.

input:

```
6 5
1 2
1 3
2 3
4 5
5 6
```

expected output:

```
2
4 5
6 5
```

Recognition: n and m both <= 5e5 under the 5 second limit price one linear decomposition: a degree pass, a peel queue over degree-at-most-one vertices, and the boundary sign pass, about 2e6 steps against a budget that holds 1e9, three orders of magnitude to spare.

The statement's cue is its reachability phrasing: a sign is earned when a driver cannot come back without a U-turn and pruned when another entrance reaches this one without turning, both facts about whole components of the road graph rather than any single street, and the components that matter are exactly the 2-core and what peels off it. Problem E is the 2019 face of the component structure and connectivity family, and the component structure and connectivity family runs through chapter 11 problem B, chapter 12 problem H, chapter 13 problem R, chapter 15 problem E, and chapter 15 problem G.

The tempting alternative, a reachability walk per entrance, is priced out at up to 1e6 entrances times an O(n + m) traversal each, 1e12 steps, while the peel answers every entrance in one sweep because U-turn-freedom is exactly membership in the same 2-core boundary.

The solver peels degree-at-most-one vertices, the 2-core: a component that peels entirely is a tree, where only the original leaf entrances survive redundancy pruning, and otherwise every street with exactly one peeled endpoint keeps its sign at the core endpoint, since everything on the peeled branch reaches that entrance without a U-turn (solutions.pdf p. 4, O(n + m)). Book 10′s chapter 39, graph connectivity and decomposition, builds this 2-core peel, degree queues and all, from scratch. The seven peelings follow.

The worked run: trace the model on sample 1. Degrees start at 2, 2, 2, 1, 2, 1, so the peel queue holds 4 and 6. Removing 4 and 6 drops location 5 to degree 0, and 5 peels too, leaving the triangle 1, 2, 3 as the one surviving core. The path component 4-5-6 peeled entirely, so it is a tree and only its original leaf entrances keep signs: a driver entering street 4-5 from 4 reaches 5 and can only U-turn back, and the mirror holds at 6, so signs 4 5 and 6 5 survive redundancy pruning, each leaf entrance reaching the other's street through the tree without turning. The triangle keeps nothing, every entrance leaving and returning around the cycle. Two signs sorted by v then w, and the trace ends at the printed answer `6 5`.

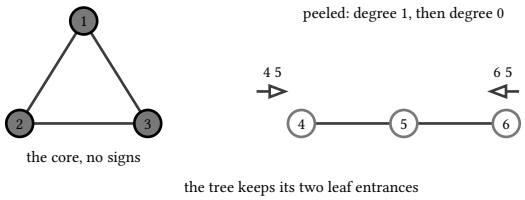


figure 131: sample 1: the triangle survives as the 2-core, the path 4-5-6 peels to a tree keeping only its two leaf signs

```c
int qh = 0, qt = 0;
for (int v = 1; v ≤ n; v++)
  if (deg[v] ≤ 1)
    q[qt++] = v;
while (qh < qt) {
  int v = q[qh++];
  if (peeled[v])
    continue;
  peeled[v] = 1;
  for (int e = head[v]; e ≥ 0; e = nxt[e]) {
    int i = edg[e], w = eu[i] ^ ev[i] ^ v;
    if (peeled[w])
      continue;
    if (--deg[w] ≤ 1)
      q[qt++] = w;
  }
}
// mark components that kept a core: their peeled branches are all
// reachable from a core sign without a U-turn, so they emit nothing
static char mixed[500005];
memset(mixed, 0, (size_t)n + 1);
qh = qt = 0;
for (int v = 1; v ≤ n; v++)
  if (!peeled[v]) {
    mixed[v] = 1;
    q[qt++] = v;
  }
while (qh < qt) {
  int v = q[qh++];
  for (int e = head[v]; e ≥ 0; e = nxt[e]) {
    int i = edg[e], w = eu[i] ^ ev[i] ^ v;
    if (!mixed[w]) {
      mixed[w] = 1;
      q[qt++] = w;
    }
  }
}
```

listing 356: c: the peel queue over degree counts, then the flood that marks which components kept a core (source: icpc/samples-c/src/Ch10/pE.c, lines 62-98)

```
  for e := 0; e < m; e++ {
    a, b := us[e], vs[e]
    switch {
    case peeled[a] && peeled[b]:
      // inside a fully peeled tree component only original
      // leaf entrances survive
      if treeComp[comp[a]] {
        if deg[a] == 1 {
          signs = append(signs, sign{a, b})
        }
        if deg[b] == 1 {
          signs = append(signs, sign{b, a})
        }
      }
    case peeled[a]:
      signs = append(signs, sign{b, a})
    case peeled[b]:
      signs = append(signs, sign{a, b})
    }
  }
  sort.Slice(signs, func(x, y int) bool {
    if signs[x].u ≠ signs[y].u {
      return signs[x].u < signs[y].u
    }
    return signs[x].v < signs[y].v
  })
```

listing 357: go: the sign emission, tree leaves inside fully peeled components, core-end signs on boundary streets (source: icpc/samples-go/ch10/pe.go, lines 99-124)

```
    boolean[] peeled = new boolean[n + 1];
    // a vertex can enqueue up to three times: once for a degree that
    // starts at ≤ 1, once when a decrement lands on 1, once on 0
    int[] q = new int[3 * n + 4];
    int qh = 0, qt = 0;
    for (int v = 1; v ≤ n; v++)
      if (deg[v] ≤ 1)
        q[qt++] = v;
    while (qh < qt) {
      int v = q[qh++];
      if (peeled[v])
        continue;
      peeled[v] = true;
      for (int e = head[v]; e ≥ 0; e = nxt[e]) {
        int i = edg[e], w = eu[i] ^ ev[i] ^ v;
        if (peeled[w])
          continue;
        if (--deg[w] ≤ 1)
          q[qt++] = w;
      }
    }
    // mark components that kept a core: their peeled branches are all
    // reachable from a core sign without a U-turn, so they emit nothing
    boolean[] mixed = new boolean[n + 1];
    qh = qt = 0;
    for (int v = 1; v ≤ n; v++)
      if (!peeled[v]) {
        mixed[v] = true;
        q[qt++] = v;
      }
    while (qh < qt) {
      int v = q[qh++];
      for (int e = head[v]; e ≥ 0; e = nxt[e]) {
        int i = edg[e], w = eu[i] ^ ev[i] ^ v;
        if (!mixed[w]) {
          mixed[w] = true;
          q[qt++] = w;
        }
      }
    }
```

listing 358: java: the peel queue sized 3n + 4 over degree counts, then the flood marking which components kept a core (source: icpc/samples-java/src/Ch11/PE.java, lines 56-95)

```
        // 2-core: peel degree ≤ 1 vertices repeatedly
        var core = (int[])deg.Clone();
        var queue = new Queue<int>();
        for (int v = 1; v ≤ n; v++)
            if (core[v] ≤ 1)
                queue.Enqueue(v);
        while (queue.Count > 0)
        {
            int u = queue.Dequeue();
            core[u] = 0;
            for (int e = Head(u); e < Till(u); e++)
            {
                int v = adj[e];
                if (core[v] > 1 && --core[v] == 1)
                    queue.Enqueue(v);
            }
        }

        // boundary streets of the remaining core: sign at the core endpoint
        var signs = new List<(int U, int V)>();
        for (int u = 1; u ≤ n; u++)
        {
            if (core[u] == 0)
                continue;
            for (int e = Head(u); e < Till(u); e++)
                if (core[adj[e]] == 0)
                    signs.Add((u, adj[e]));
        }
```

listing 359: c#: symmetric csr adjacency, the 2-core peel, and the boundary streets signed at their core endpoints (source: icpc/samples/src/Ch10/PE.cs, lines 38-65)

```
  const peeled = new Uint8Array(n + 1);
  const d = deg.slice();
  const queue = [];
  for (let v = 1; v ≤ n; v++) {
    if (d[v] ≤ 1) queue.push(v);
  }
  while (queue.length > 0) {
    const v = queue.pop();
    peeled[v] = 1;
    for (let h = adjHead[v]; h =/= -1; h = halfNext[h]) {
      const w = halfTo[h];
      if (!peeled[w]) {
        d[w]--;
        if (d[w] === 1) queue.push(w);
      }
    }
  }
  // label components; a component that kept any core vertex makes
  // every sign on its peeled branches redundant
  const comp = new Int32Array(n + 1).fill(-1);
  const treeComp = [];
  for (let s = 1; s ≤ n; s++) {
    if (comp[s] ≥ 0) continue;
    const id = treeComp.length;
    comp[s] = id;
    let allPeeled = peeled[s] === 1;
    const stack = [s];
    while (stack.length > 0) {
      const v = stack.pop();
      for (let h = adjHead[v]; h =/= -1; h = halfNext[h]) {
        const w = halfTo[h];
        if (comp[w] < 0) {
          comp[w] = id;
          if (!peeled[w]) allPeeled = false;
          stack.push(w);
        }
      }
    }
    treeComp.push(allPeeled);
  }
```

listing 360: javascript: the peel over typed arrays, then component labels tracking which components kept a core (source: icpc/samples-js/src/ch10-pe-deadend.mjs, lines 32-71)

```
    peeled = bytearray(n + 1)
    d = deg[:]
    queue = [v for v in range(1, n + 1) if d[v] ≤ 1]
    while queue:
        v = queue.pop()
        peeled[v] = 1
        for w in adj[v]:
            if not peeled[w]:
                d[w] -= 1
                if d[w] == 1:
                    queue.append(w)
    comp = [-1] * (n + 1)
    tree_comp = bytearray(n + 1)
    n_comp = 0
    for s in range(1, n + 1):
        if comp[s] ≥ 0:
            continue
        comp[s] = n_comp
        all_peeled = bool(peeled[s])
        stack = [s]
        while stack:
            v = stack.pop()
            for w in adj[v]:
                if comp[w] < 0:
                    comp[w] = n_comp
                    if not peeled[w]:
                        all_peeled = False
                    stack.append(w)
        tree_comp[n_comp] = all_peeled
        n_comp += 1
```

listing 361: python: peel by stack, then a flood fill per component recording all-peeled trees (source: icpc/samples-py/src/Ch10/pe.py, lines 24-53)

```
local peeled = {}
local d = {}
for i2 = 1, n do d[i2] = deg[i2] end
local queue = {}
local qn = 0
for i2 = 1, n do
  if d[i2] ≤ 1 then
    qn = qn + 1
    queue[qn] = i2
  end
end
while qn > 0 do
  local vv = queue[qn]
  queue[qn] = nil
  qn = qn - 1
  peeled[vv] = true
  local h = ehead[vv]
  while h do
    local w = eto[h]
    if not peeled[w] then
      d[w] = d[w] - 1
      if d[w] == 1 then
        qn = qn + 1
        queue[qn] = w
      end
    end
    h = enext[h]
  end
end
```

listing 362: lua: the same peel queue over half-edge lists, adjacency keyed by node (source: icpc/samples-lua/ch10_pe.lua, lines 46-74)

Vertices and degrees stay under 5e5, so counts fit int32 and the languages differ only in adjacency shape: C XORs the two endpoints out of a packed half-edge, C# builds a symmetric csr array, go, js, and lua thread half-edge lists, java threading them too and XORing its far endpoint back out the same way, and python keeps per-vertex lists. Java also replaces the c qsort-on-structs sign list with one packed long per sign, from times 1e6 plus to, so a plain `Arrays.sort` lands in output order, and sizes its peel queue at 3n + 4 and its sign array at 2m + 1, documented headroom with no behavior change.

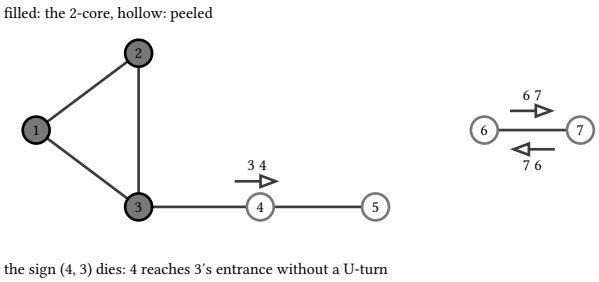


figure 132: the fixture graph: the triangle core keeps the sign at its boundary street, the two peeled vertices render hollow, and the isolated tree keeps both leaf signs

The crafted fixture is seven vertices and six streets, a triangle with a two-edge tail plus one isolated edge. The triangle survives as the core, street 3-4 keeps its sign at 3, and the isolated tree keeps both leaf signs, so the pinned output is three lines, 3 4, 6 7, 7 6. The C suite's four `CHECK` cases pin the fixture, the two official samples, and the two-vertex street, the C# facts pin four answers over the same family, Go's table takes the fixture and the isolated edge, the javascript `it` blocks take those two, python's four checks add a star tree whose leaves all survive, and

lua's four run the same family, java's six adding the star tree and a core that absorbs three branches, so the both-directions rule for an isolated street is pinned in every suite.

12.7. problem F, directing rainfall

The International Consortium of Port Connoisseurs has hung sun tarps over its Douro Valley vineyards, and the tarps, entirely waterproof, now threaten the harvest. In the two-dimensional model the vineyard is an interval on the x-axis and every tarp is a slanted line segment above it. Rain falls straight down from infinitely high, rain landing on a tarp flows along it toward the tarp's lower end and falls off there, unless it meets a puncture between its landing point and the lower end, in which case it drops vertically through the puncture and keeps falling, landing on whatever is below and repeating the same logic until it reaches the ground. The consortium wants the fewest punctures that bring some rain to the vineyard, and legality adds one twist: at least some of the rain reaching the vineyard must have started from directly above the vineyard itself, so a plan that only steals a neighboring vineyard's rain does not count (statement).

The input is three integers l, r, and n, the vineyard 0 <= l < r <= 1e9 and the tarp count 0 <= n <= 5e5 by the problemset pdf, then n lines of four integers x1 y1 x2 y2, each tarp's lower end (x1, y1) and higher end (x2, y2) with 0 <= x1, x2 <= 1e9, x1 never equal to x2, and 0 < y1 < y2 <= 1e9, every x-coordinate in the input including l and r distinct, the tarps pairwise non-intersecting and no tarp endpoint lying on another tarp. The output is one integer, the minimum number of punctures, an answer always existing since puncturing straight down through everything above any field point works, under the pdf's 15 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `F-directingrainfall/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: five tarps stacked over the interval 10 to 20, and two punctures walk one drop from above the field all the way down into it.

input:

```
10 20 5
32 50 12 60
30 60 8 70
25 70 0 80
15 30 28 40
5 20 14 25
```

expected output:

```
2
```

Recognition: n <= 5e5 tarps under the 15 second limit price an endpoint sort plus one ordered-set step per endpoint, about 1e6 live-structure operations at 19 tree levels each, then one batch dp update per tarp over run-encoded cells, an O(n log n) shape with an order of magnitude to spare.

The statement's cue is its layered geometry: tarps are non-crossing slanted segments, water lands on one, flows to the lower end, or drops through a puncture, all verbs of vertical stacking order at a moving x, so the shape is a left-to-right sweep holding the live tarps ordered by height, with the dp the drops feed sitting underneath. Problem F is the 2019 face of the sweep line with an ordered active set family, chapter 10 problem L is its only sibling, and a secondary thread, dp over a piecewise-constant function, covers the water trajectory under the tarps.

The tempting alternative, processing the tarps in order of lower-end height, is priced out by correctness rather than time: a puncture near a tarp's high end can drop water onto a tarp whose lower end sits higher, so no static height key orders the dp, and only the sweep's adjacency dag does.

The pinned solution runs in three parts. Tarps are first ordered by vertical adjacency, an endpoint sweep over a live structure ordered by height whose adjacency edges feed a topological order, and the solver must never sort by

lower-end height alone, since a puncture near a high end can drop water onto a tarp whose lower end is higher. The dp then runs over the piecewise-constant function V on the compressed cells between tarp endpoints, V zero on the field cells and infinity outside, each tarp updating its span by the batch contract: water at a cell either drips off the tarp's low end for the drip value vd, or exits through the first puncture for 1 plus the minimum below, so the update flattens the far side of the valley to min(vd, vmin + 1), lifts the near-drip slope short of vd by one, and flattens the rest to vd. The drip value itself reads the covered side: on a left drip vd is the value of the first covered cell of the span, equal to the outside-left cell unless the field edge coincides with the drip coordinate, and on a right drip the first cell past the span, while the initial field is zero on exactly its own cells plus one extra initial cell past the right edge, which is what makes an edge drip free instead of infinite. Answers that come out too small trace to four spots, the argmin taken on the wrong side and the flat region widened with it, vd read from the wrong side, a missing +1 on the lifted run, or that extra initial cell. The contract was pinned by a property sweep against a built puncture-set oracle over random valid instances, and the oracle choice is load-bearing: a naive reimplementation that reads, on a left drip, the cell left of the tarp agrees on valid boards but diverges from the first-covered-cell semantics on synthetic shapes outside the statement's domain, which is why the sweep compares against the built oracle and not a naive one. The answer is the minimum V over the field cells (solutions.pdf pp. 4-5, amortized O(n log n) in the official two-ordered-set structure). The seven batch updates follow.

The worked run: trace the model on sample 1. Compressed between the tarp endpoints and the field edges, the x-axis splits into cells, and V starts at 0 on the field cells 10 to 20 plus one extra cell past 20, infinity elsewhere. Bottom-up, tarp 5 first: it spans 5 to 14, drips left at 5 into nothing, vd infinity, so puncturing is the only exit and the two field cells under it lift from 0 to 1. Tarp 4 spans 15 to 28, drips left at 15 where its first covered cell, the field cell 15 to 20, holds 0, so vd = 0 flattens the whole span: anything on tarp 4 reaches the field free. Tarp 1 spans 12 to 32, drips right at 32 off the world, vd infinity, so each cell takes 1 plus the running minimum below: cells 14 to 28 drop to 1 through tarp 4′s flat 0, cells 28 to 32 stay infinite. Tarp 2 spans 8 to 30, drips right at 30 onto tarp 1′s infinite tail, and lifts its span to 2 over those 1s, cells 28 to 30 staying infinite. Tarp 3 spans 0 to 25, drips right at 25 where tarp 2 reads 2, so vd = 2 caps its whole span, the puncture option 3 never beating the drip. The field cells 10 to 20 all read 2, and the trace ends at the printed answer 2.

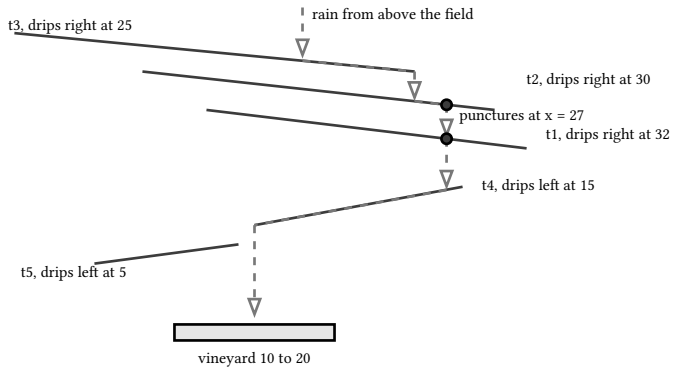


figure 133: sample 1′s five tarps and the two-puncture trajectory: onto tarp 3, drip at 25 onto tarp 2, puncture onto tarp 1, puncture onto tarp 4, drip left at 15 into the field

```
  long long vd;
  if (left_drip) {
    vd = point_val(rroot, a);
  } else if (ib ≥ C) {
    vd = (L ≤ hi && hi ≤ R) ? 0 : INF;
  } else {
    vd = point_val(rroot, ib);
  }
  int lr, mr, m, rrp;
  rsplit(rroot, a, &lr, &mr);
  rsplit(mr, b + 1, &m, &rrp);
  if (left_drip) {
    // water runs to a: cells right of the last valley cell take c,
    // the nonincreasing prefix up to the last value ≥ vd flattens
    // to vd, the rest lifts by one
    int g_end;
    long long vmin = min_at_right(m, &g_end);
    long long c = vd < vmin + 1 ? vd : vmin + 1;
    int lo2, rest;
    rsplit(m, g_end + 1, &lo2, &rest);
    int q;
    right_ge(lo2, vd, &q);
    int junk, l2;
    rsplit(lo2, q + 1, &junk, &l2);
    rapply(l2, 1);
    m = (q ≥ a) ? rmerge(ralloc(a, q + 1, vd), l2) : l2;
    if (g_end < b)
      m = rmerge(m, ralloc(g_end + 1, b + 1, c));
  } else {
    // water runs to b: cells left of the first valley cell take c,
    // the nondecreasing suffix from the first value ≥ vd flattens
    // to vd, the rest lifts by one
    int g;
    long long vmin = min_at(m, &g);
    long long c = vd < vmin + 1 ? vd : vmin + 1;
    int junk, rest;
    rsplit(m, g, &junk, &rest);
    int p;
    left_ge(rest, vd, &p);
    if (p < 0)
      p = b + 1;
    int s1, junk2;
    rsplit(rest, p, &s1, &junk2);
    rapply(s1, 1);
    m = (g > a) ? ralloc(a, g, c) : 0;
    m = rmerge(m, s1);
    if (p ≤ b)
      m = rmerge(m, ralloc(p, b + 1, vd));
  }
  rroot = rmerge(lr, rmerge(m, rrp));
}
```

listing 363: c: vd read at the drip-side cell, then the left-drip and right-drip batch forms over the run treap, splits, lifts, and flat runs (source: icpc/samples-c/src/Ch10/pF.c, lines 291-341)

```
    lr, mr := rt.splitCell(rt.root, a)
    m, rr := rt.splitCell(mr, b+1)
    if leftDrip {
      // water runs to a: cells right of the last valley cell take
      // c, the nonincreasing prefix up to the last value ≥ vd
      // flattens to vd, the rest lifts by one
      vmin, gEnd := rt.minAtRight(m)
      c := vd
      if vmin+1 < c {
        c = vmin + 1
      }
      lo, _ := rt.splitCell(m, gEnd+1)
      q := rt.rightGe(lo, vd)
      _, l2 := rt.splitCell(lo, q+1)
      rt.apply(l2, 1)
      if q ≥ a {
        m = rt.merge(rt.alloc(a, q+1, vd), l2)
      } else {
        m = l2
      }
      if gEnd < b {
        m = rt.merge(m, rt.alloc(gEnd+1, b+1, c))
      }
    } else {
      // water runs to b: cells left of the first valley cell take
      // c, the nondecreasing suffix from the first value ≥ vd
      // flattens to vd, the rest lifts by one
      vmin, g := rt.minAt(m)
      c := vd
      if vmin+1 < c {
        c = vmin + 1
      }
      _, rest := rt.splitCell(m, g)
      p := rt.leftGe(rest, vd)
      if p < 0 {
        p = b + 1
      }
      s1, _ := rt.splitCell(rest, p)
      rt.apply(s1, 1)
      if g > a {
        m = rt.alloc(a, g, c)
      } else {
        m = -1
      }
      m = rt.merge(m, s1)
      if p <= b {
        m = rt.merge(m, rt.alloc(p, b+1, vd))
      }
    }
    rt.root = rt.merge(lr, rt.merge(m, rr))
```

listing 364: go: min at the valley edge, the first or last cell at or above vd, the lifted run and the flat c and vd tails (source: icpc/samples-go/ch10/pf.go, lines 148-197)

```
      int a = lowerBound(xs, C, lo);
      int ib = lowerBound(xs, C, hi);
      int b = ib - 1; // covered cells [a, b] = runs [a, ib)
      long vd;
      if (leftDrip) {
        vd = rt.pointVal(rt.rroot, a);
      } else if (ib ≥ C) {
        vd = (L <= hi && hi ≤ R) ? 0 : INF;
      } else {
        vd = rt.pointVal(rt.rroot, ib);
      }
      int[] sp = rt.rsplit(rt.rroot, a);
      int lr = sp[0], mr = sp[1];
      sp = rt.rsplit(mr, b + 1);
      int m = sp[0], rrp = sp[1];
      if (leftDrip) {
        // water runs to a: cells right of the last valley cell take c,
        // the nonincreasing prefix up to the last value ≥ vd flattens
        // to vd, the rest lifts by one
        long vmin = rt.minAtRight(m, cell);
        int gEnd = cell[0];
        long cv = Math.min(vd, vmin + 1);
        sp = rt.rsplit(m, gEnd + 1);
        int lo2 = sp[0], rest = sp[1];
        int q = rt.rightGe(lo2, vd);
        sp = rt.rsplit(lo2, q + 1);
        int l2 = sp[1];
        rt.rapply(l2, 1);
        m = q ≥ a ? rt.rmerge(rt.ralloc(a, q + 1, vd), l2) : l2;
        if (gEnd < b)
          m = rt.rmerge(m, rt.ralloc(gEnd + 1, b + 1, cv));
      } else {
        // water runs to b: cells left of the first valley cell take c,
        // the nondecreasing suffix from the first value ≥ vd flattens
        // to vd, the rest lifts by one
        long vmin = rt.minAt(m, cell);
        int g = cell[0];
        long cv = Math.min(vd, vmin + 1);
        sp = rt.rsplit(m, g);
        int rest = sp[1];
        int p = rt.leftGe(rest, vd);
        if (p < 0)
          p = b + 1;
        sp = rt.rsplit(rest, p);
        int s1seg = sp[0];
        rt.rapply(s1seg, 1);
        m = g > a ? rt.ralloc(a, g, cv) : 0;
        m = rt.rmerge(m, s1seg);
        if (p ≤ b)
          m = rt.rmerge(m, rt.ralloc(p, b + 1, vd));
      }
      rt.rroot = rt.rmerge(lr, rt.rmerge(m, rrp));
    }
```

listing 365: java: the tarp's cells found by binary search, vd read at the drip-side cell, then both drip forms over the run treap, splits, lifts, and flat runs merged back (source: icpc/samples-java/src/Ch11/PF.java, lines 261-313)

```
            if (leftDrip)
            {
                vd = rt.PointVal(rt.Root, a);
            }
            else
            {
                int ib = Idx(x2[t]);
                vd = ib ≥ cells
                    ? (l ≤ x2[t] && x2[t] ≤ r ? 0 : Inf)
                    : rt.PointVal(rt.Root, ib);
            }
            var (before, mid0) = rt.SplitCell(rt.Root, a);
            var (m1, after) = rt.SplitCell(mid0, b + 1);
            int mid;
            if (leftDrip)
            {
                // water runs to a: cells right of the last valley cell
                // take c, the nonincreasing prefix up to the last value
                // ≥ vd flattens to vd, the rest lifts by one
                var (vmin, gEnd) = rt.MinAtRight(m1);
                long c = Math.Min(vd, vmin + 1);
                var (lo0, _) = rt.SplitCell(m1, gEnd + 1);
                int q = rt.RightGe(lo0, vd);
                var (_, l2) = rt.SplitCell(lo0, q + 1);
                rt.Apply(l2, 1);
                mid = q ≥ a ? rt.Merge(rt.Alloc(a, q + 1, vd), l2) : l2;
                if (gEnd < b)
                    mid = rt.Merge(mid, rt.Alloc(gEnd + 1, b + 1, c));
            }
            else
            {
                // water runs to b: cells left of the first valley cell
                // take c, the nondecreasing suffix from the first value
                // ≥ vd flattens to vd, the rest lifts by one
                var (vmin, g) = rt.MinAt(m1);
                long c = Math.Min(vd, vmin + 1);
                var (_, rest) = rt.SplitCell(m1, g);
                int p = rt.LeftGe(rest, vd);
                if (p < 0)
                    p = b + 1;
                var (s1, _) = rt.SplitCell(rest, p);
                rt.Apply(s1, 1);
                mid = g > a ? rt.Alloc(a, g, c) : -1;
                mid = rt.Merge(mid, s1);
                if (p ≤ b)
                    mid = rt.Merge(mid, rt.Alloc(p, b + 1, vd));
            }
            rt.Root = rt.Merge(before, rt.Merge(mid, after));
```

listing 366: c#: the same contract on its RunTreap, the sweep and vehicle split into PFSweep.cs and PFRuns.cs (source: icpc/samples/src/Ch10/PF.cs, lines 101-148)

```
    const [lr, mr] = rt.splitCell(rt.root, a);
    const [m0, rr] = rt.splitCell(mr, b + 1);
    let m = m0;
    if (leftDrip) {
      // water runs to a: cells right of the last valley cell take c,
      // the nonincreasing prefix up to the last value ≥ vd flattens
      // to vd, the rest lifts by one
      const [vmin, gEnd] = rt.minAtRight(m);
      const c = Math.min(vd, vmin + 1);
      const [lo] = rt.splitCell(m, gEnd + 1);
      const q = rt.rightGe(lo, vd);
      const [, l2] = rt.splitCell(lo, q + 1);
      rt.apply(l2, 1);
      m = q ≥ a ? rt.merge(rt.alloc(a, q + 1, vd), l2) : l2;
      if (gEnd < b) m = rt.merge(m, rt.alloc(gEnd + 1, b + 1, c));
    } else {
      // water runs to b: cells left of the first valley cell take c,
      // the nondecreasing suffix from the first value ≥ vd flattens
      // to vd, the rest lifts by one
      const [vmin, g] = rt.minAt(m);
      const c = Math.min(vd, vmin + 1);
      const [, rest] = rt.splitCell(m, g);
      let p = rt.leftGe(rest, vd);
      if (p < 0) p = b + 1;
      const [s1] = rt.splitCell(rest, p);
      rt.apply(s1, 1);
      m = g > a ? rt.alloc(a, g, c) : -1;
      m = rt.merge(m, s1);
      if (p ≤ b) m = rt.merge(m, rt.alloc(p, b + 1, vd));
    }
    rt.root = rt.merge(lr, rt.merge(m, rr));
  }
```

listing 367: javascript: the mirrored left and right drip updates over the shared run treap module (source: icpc/samples-js/src/ch10-pf-directingrainfall.mjs, lines 238-269)

```
    for i in order:
        if rt.dirty:
            rt.rebuild()
        a = idx_of(tx1[i])
        b = idx_of(tx2[i]) - 1
        left_drip = ty1[i] < ty2[i]
        if left_drip:
            vd = rt.point_val(rt.root, a)
        else:
            ib = idx_of(tx2[i])
            if ib ≥ total_cells:
                vd = 0 if field_l ≤ tx2[i] ≤ field_r else INF
            else:
                vd = rt.point_val(rt.root, ib)
        lr, mr = rt.split_cell(rt.root, a)
        m, rr = rt.split_cell(mr, b + 1)
        if left_drip:
            vmin, g_end = rt.min_at_right(m)
            c = min(vd, vmin + 1)
            lo, _ = rt.split_cell(m, g_end + 1)
            q2 = rt.right_ge(lo, vd)
            _, l2 = rt.split_cell(lo, q2 + 1)
            rt.apply(l2, 1)
            m = rt.merge(rt.alloc(a, q2 + 1, vd), l2) if q2 ≥ a else l2
            if g_end < b:
                m = rt.merge(m, rt.alloc(g_end + 1, b + 1, c))
        else:
            vmin, g = rt.min_at(m)
            c = min(vd, vmin + 1)
            _, rest = rt.split_cell(m, g)
            p = rt.left_ge(rest, vd)
            if p < 0:
                p = b + 1
            s1, _ = rt.split_cell(rest, p)
            rt.apply(s1, 1)
            m = rt.alloc(a, g, c) if g > a else -1
            m = rt.merge(m, s1)
            if p ≤ b:
                m = rt.merge(m, rt.alloc(p, b + 1, vd))
        rt.root = rt.merge(lr, rt.merge(m, rr))
```

listing 368: python: the dp loop over the sweep order, rebuild gated on the dirty flag, both batch forms (source: icpc/samples-py/src/Ch10/pf.py, lines 149-188)

```
    local left_drip = ty1[i] < ty2[i]
    local vd
    if left_drip then
      vd = rt.point_val(t, t.root, a)
    else
      local ib = idx_of(tx2[i])
      if ib ≥ C + 1 then
        vd = (l ≤ tx2[i] and tx2[i] ≤ r) and 0 or inf
      else
        vd = rt.point_val(t, t.root, ib)
      end
    end
    local lr, mr = rt.split_cell(t, t.root, a)
    local m, rr = rt.split_cell(t, mr, b + 1)
    if left_drip then
      local vmin, g_end = rt.min_at_right(t, m)
      local c = vd
      if vmin + 1 < c then c = vmin + 1 end
      local lo = rt.split_cell(t, m, g_end + 1)
      local q = rt.right_ge(t, lo, vd)
      local _, l2 = rt.split_cell(t, lo, q + 1)
      rt.apply(t, l2, 1)
      if q ≥ a then
        m = rt.merge(t, rt.alloc(t, a, q + 1, vd), l2)
      else
        m = l2
      end
      if g_end < b then
        m = rt.merge(t, m, rt.alloc(t, g_end + 1, b + 1, c))
      end
    else
      local vmin, g = rt.min_at(t, m)
      local c = vd
      if vmin + 1 < c then c = vmin + 1 end
      local _, rest = rt.split_cell(t, m, g)
      local p = rt.left_ge(t, rest, vd)
      if p < 0 then p = b + 1 end
      local s1 = rt.split_cell(t, rest, p)
      rt.apply(t, s1, 1)
      if g > a then
        m = rt.alloc(t, a, g, c)
      else
        m = 0
      end
      m = rt.merge(t, m, s1)
      if p ≤ b then
        m = rt.merge(t, m, rt.alloc(t, p, b + 1, vd))
      end
    end
    t.root = rt.merge(t, lr, rt.merge(t, m, rr))
  end
```

listing 369: lua: the same loop, one extra initial cell so right-edge drips read a real value, rebuild when the depth gate fired (source: icpc/samples-lua/ch10_pf.lua, lines 302-352)

The vehicle is the year's own: V lives as a run treap, equal-value runs in a treap with subtree min and max, lazy addition, and cell-boundary splits and merges, ported from the judge-green go implementation. Book 10′s chapter 30, balanced trees and order statistics, builds this treap from scratch, lazy adds and subtree aggregates included.

Run cuts cannot keep a treap heap-consistent under adversarial cut orders, so the vehicle rebuilds, and the rebuild trigger is language-tunable: c, C#, go, java, and javascript rebuild on a fixed cadence, every 4096 tarps, one linear flattening pass, python and lua gate on recursion depth instead, an operation walking deeper than 96 flags the structure dirty and the dp loop rebuilds at the next tarp boundary, which lua cannot afford at the fixed cadence and which never fired on the judge data anyway, the measured maximum depth there being 52, while the gate cut lua's worst secret from 141.4 s to 30.0 s and go passed 96 of 96 secrets at a 4.42 s maximum on the fixed cadence. The sweep itself also splits by integer honesty: c compares tarp heights by exact cross products in `__int128`, java the same in `BigInteger`, the band's one such site, the products reaching 4e27 while the scaled heights stay under 2e18 in long, and python by exact integer products over small ordered blocks, while go, javascript, C#, and lua order the live set by the float height at the query x. Java also replaces c's qsort-on-structs event array with one packed long per event, x shifted up 21 bits over the end flag and tarp index, so a plain `Arrays.sort` reproduces the c tie order. All values stay at most n and coordinates fit int32, so no language crosses a product edge in the dp itself, and C# runs the recursive splits on a dedicated 64 MB thread stack because cut orders between rebuilds can chain the treap depth.

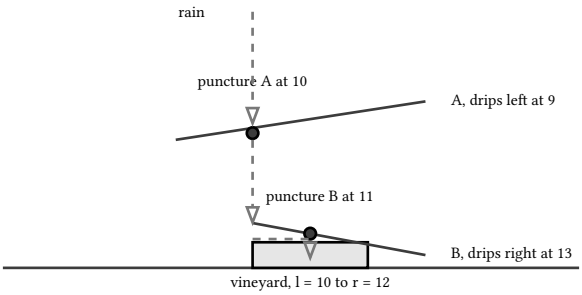


figure 134: the fixture's two tarps over the vineyard, the two punctures as dots, and the dashed drop path from above the field through both into the vineyard

The crafted fixture is the vineyard 10 12 with two tarps, 13 1 10 2 and 9 3 13 4. Tarp A sits above B on the shared span and drips left outside the field, B drips right, and two punctures walk the water down, one in A at x = 10 dropping onto B and one in B at x = 11 inside the vineyard, so the pinned answer is 2. The C suite pins it with three more `CHECK` cases, the official samples answering 2 and 1 and the empty-tarp board answering 0, the C# facts pin four boards over the same family, Go's table takes the fixture and the empty board, the javascript `it` blocks take those two, python's four checks add a free in-field drip and a one-puncture board whose drip lands outside, and lua's four add a tarp clear of the field and one tarp over the whole field, java's six pinning the fixture, both official samples, the free in-field drip, the open field, and the one-puncture board, so the empty and one-tarp edges are pinned across all seven suites.

12.8. problem G, first of her name

The Royal Historian analyzes the Royal Ladies' names. There have been n ladies, and each lady's name is a single uppercase letter concatenated in front of her mother's name, lady 1 being the founder whose name is that one letter alone, so the mother pointers generate every name and all names are distinct, AENERYS the daughter of ENERYS for one. For each of k interesting strings the task is the number of ladies whose name has that string as a prefix (statement).

The input is two integers n and k, both between 1 and 1e6 by the problemset pdf, then n lines of an uppercase letter ci and an integer pi naming lady i's mother, with p1 = 0 and 1 <= pi < i for i > 1, then k nonempty uppercase query strings whose total length is at most 1e6. The output is k lines, one count per query in order, under the pdf's 10 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `G-firstofhername/sample-1.in` and `sample-1.ans`, the only sample the letter carries, the same pair the problems pdf prints: a royal line running S, YS, RYS, ERYS, NERYS, ENERYS down to AENERYS, who has the two daughters DAENERYS and YAENERYS, the latter with daughter RYAENERYS, and the five queries RY, E, N, S, AY answer 2, 2, 1, 1, 0.

input:

```
10 5
S 0
Y 1
R 2
E 3
N 4
E 5
A 6
D 7
Y 7
R 9
RY
E
N
S
AY
```

expected output:

```
2
2
1
1
0
```

Recognition: n and the total query length both <= 1e6 under the 10 second limit price one automaton build over the reversed queries and one walk of the mother tree, a goto step per lady and a fold per state, about 2e6 automaton steps with two orders of magnitude to spare, so nothing that respells a name per query survives.

The statement's cue is the direction of construction: each name is one letter concatenated in front of the mother's, so a prefix query on names is a suffix query on the root-to-node paths of the mother tree, and counting every lady's path against all queries at once is exactly what a pattern automaton with suffix links does. Problem G is the 2019 face of the string borders, rotations, and automata family, and string borders, rotations, and automata connect chapter 10 problem K and chapter 14 problem F.

The tempting alternative, materializing the names, is priced out at about 5e11 characters for one chain of 1e6 ladies before any query lands, and scanning per query multiplies that by k, while the automaton never spells a name at all.

A name has prefix s exactly when the reversed name ends with reversed s, and the reversed names are the root-to-node paths of the mother tree, so the solver builds an aho-corasick automaton of the reversed queries, walks it down the mother tree carrying each lady's state from her mother's, adds one at each lady's state, and propagates the counts up the suffix-link tree in reverse bfs order (solutions.pdf p. 5, O(n + sum of query lengths)). The seven walkers follow.

The worked run: trace the model on sample 1. The five queries reverse into YR, E, N, S and YA and build a six-state trie, root children Y, E, N and S, with R and A under Y, every fail link running to the root. Each lady's state is one goto step from her mother's: down the founder's line the states run S, Y, YR, E, N, lady 6′s ENERYS fails N to root and lands on E, lady 7′s AENERYS finds no A under root and stays root, lady 8′s DAENERYS stays root on D, lady 9′s YAENERYS climbs onto Y, and lady 10′s RYAENERYS ends on YR. One hit per lady at her end state, folded up the fail tree, which is flat here: the states read S 1, Y 2, YR 2, E 2, N 1, root 2, so the answers are YR 2, E 2, N 1, S 1 and YA 0, and the trace ends at the printed answer `0`.

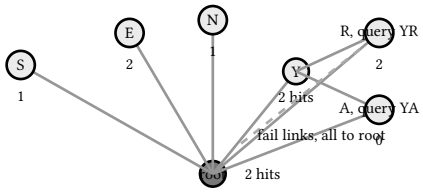


figure 135: the sample's query trie over the reversed queries with the hit counts, every fail link running to the root, so each query state answers its own hits

```
for (int i = 0; i < n; i++) {
  int v = order2[i];
  int st;
  if (mother[v] < 0) {
    st = 0;
  } else {
    st = state[mother[v]];
  }
  int c = letter[v] - 'A';
  state[v] = ch[(size_t)st * 26 + c]; // dense goto from build
  cnt[state[v]]++;
}
// propagate up suffix links in reverse bfs
for (int i = nnode - 1; i ≥ 0; i--) {
  int v = order_[i];
  if (v && suff[v] ≥ 0)
    cnt[suff[v]] += cnt[v];
}
```

listing 370: c: preorder over the mother tree by explicit stack, one dense-goto step per lady, then counts summed up the suffix links in reverse bfs (source: icpc/samples-c/src/Ch10/pG.c, lines 112-129)

```
  state := make([]int32, n+1)
  count := make([]int64, len(nodes))
  type frame struct{ v, next int }
  stack := make([]frame, 0, 64)
  for h := childrenHead[0]; h < childrenHead[1]; h++ {
    c := childNode[h]
    stack = append(stack, frame{c, childrenHead[c]})
  }
  for len(stack) > 0 {
    fr := &stack[len(stack)-1]
    if fr.next == childrenHead[fr.v] {
      st := nodes[state[mother[fr.v]]].next[letters[fr.v]-'A']
      state[fr.v] = st
      count[st]++
    }
    if fr.next < childrenHead[fr.v+1] {
      c := childNode[fr.next]
      fr.next++
      stack = append(stack, frame{c, childrenHead[c]})
    } else {
      stack = stack[:len(stack)-1]
    }
  }
  for i := len(order) - 1; i ≥ 0; i-- {
    v := order[i]
    if v ≠ 0 {
      count[nodes[v].fail] += count[v]
    }
  }
```

listing 371: go: the iterative dfs with child cursors, state of each lady read off its mother's row, propagation in reverse bfs (source: icpc/samples-go/ch10/pg.go, lines 103-131)

```
    // preorder over the mother tree: each lady's state needs its mother's
    // computed first: process in preorder using an explicit stack
    int[] order2 = new int[n];
    int no = 0;
    int[] stack2 = new int[n];
    int st2 = 0;
    // find children lists via counting sort (mothers arbitrary)
    int[] firstc = new int[n + 1], nextc = new int[n];
    java.util.Arrays.fill(firstc, -1);
    for (int i = n - 1; i ≥ 0; i--)
      if (mother[i] ≥ 0) {
        nextc[i] = firstc[mother[i]];
        firstc[mother[i]] = i;
      }
    for (int i = 0; i < n; i++)
      if (mother[i] < 0)
        stack2[st2++] = i;
    while (st2 > 0) {
      int v = stack2[--st2];
      order2[no++] = v;
      for (int c = firstc[v]; c ≥ 0; c = nextc[c])
        stack2[st2++] = c;
    }
    for (int i = 0; i < n; i++) {
      int v = order2[i];
      int st;
      if (mother[v] < 0) {
        st = 0;
      } else {
        st = state[mother[v]];
      }
      int c = letter[v] - 'A';
      state[v] = ch[st * 26 + c]; // dense goto from build
      cnt[state[v]]++;
    }
    // propagate up suffix links in reverse bfs
    for (int i = nnode - 1; i ≥ 0; i--) {
      int v = order[i];
      if (v ≠ 0 && suff[v] ≥ 0)
        cnt[suff[v]] += cnt[v];
    }
```

listing 372: java: preorder over the mother tree by explicit stack, one dense-goto row lookup per lady, counts summed up the suffix links in reverse bfs (source: icpc/samples-java/src/Ch11/PG.java, lines 101-141)

```
        // walk the mother tree iteratively; state(u) = goto(state(mother), letter)
        var state = new int[n + 1];
        var kidsHead = new int[n + 1];
        Array.Fill(kidsHead, -1);
        var kidsNext = new int[n + 1];
        for (int v = n; v ≥ 2; v--)
        {
            kidsNext[v] = kidsHead[mother[v]];
            kidsHead[mother[v]] = v;
        }
        var proc = new int[n];
        var st2 = new int[n];
        int sp = 0, top = 0;
        st2[sp++] = 1;
        while (sp > 0)
        {
            int u = st2[--sp];
            proc[top++] = u;
            for (int v = kidsHead[u]; v ≥ 0; v = kidsNext[v])
                st2[sp++] = v;
        }
        foreach (int u in proc)
            state[u] = u == 1
                ? gotoNext[letter[1]]
                : gotoNext[state[mother[u]] * 26 + letter[u]];

        var occur = new long[cnt];
        foreach (int u in proc)
            occur[state[u]]++;
        // propagate up suffix links in reverse bfs order
        for (int i = cnt - 1; i ≥ 1; i--)
            occur[fail[bfs[i]]] += occur[bfs[i]];
```

listing 373: c#: kids lists built reversed for preorder, each state one goto row lookup, occurrences pushed into fail parents (source: icpc/samples/src/Ch10/PG.cs, lines 76-107)

```
while (stack.length > 0) {
  const fr = stack[stack.length - 1];
  if (fr[1] === childrenHead[fr[0]]) {
    const st = nxt[26 * state[mother[fr[0]]] + (letters[fr[0]] - 65)];
    state[fr[0]] = st;
    count[st]++;
  }
  if (fr[1] < childrenHead[fr[0] + 1]) {
    const c = childNode[fr[1]];
    fr[1]++;
    stack.push([c, childrenHead[c]]);
  } else {
    stack.pop();
  }
}
for (let i = olen - 1; i ≥ 0; i--) {
  const v = order[i];
  if (v !== 0) count[fail[v]] += count[v];
}
```

listing 374: javascript: the same stack walk over the dense Int32Array goto table, counts folded up suffix links (source: icpc/samples-js/src/ch10-pg-firstofhername.mjs, lines 95-113)

```
    state = [0] * (n + 1)
    count = [0] * node_count
    stack = [[c, chead[c]] for c in child[chead[0] : chead[1]]]
    while stack:
        fr = stack[-1]
        v = fr[0]
        if fr[1] == chead[v]:
            st = go[state[mother[v]] * 26 + letters[v]]
            state[v] = st
            count[st] += 1
        if fr[1] < chead[v + 1]:
            c = child[fr[1]]
            fr[1] += 1
            stack.append([c, chead[c]])
        else:
            stack.pop()
    for i in range(len(order) - 1, 0, -1):
        v = order[i]
        count[fail[v]] += count[v]
```

listing 375: python: one stack of frame cursors walks the family, states counted, then reverse-bfs propagation (source: icpc/samples-py/src/Ch10/pg.py, lines 87-105)

```
local trans = {}
local function goto_state(s, c)
  local key = s * 32 + c
  local hit = trans[key]
  if hit then return hit end
  local path = { s }
  local t = s
  while true do
    local u = child[t][c]
    if u then t = u break end
    local m = trans[t * 32 + c]
    if m then t = m break end
    if t == 1 then break end
    t = fail[t]
    path[#path + 1] = t
  end
  -- every state on the chain misses c, so they share the target
  for i = 1, #path do trans[path[i] * 32 + c] = t end
  return t
end
```

listing 376: lua: the memoized transition walks the fail chain and stamps the target on every state it passed, keeping the amortization alive (source: icpc/samples-lua/ch10_pg.lua, lines 87-106)

The automaton splits the languages in two. C, C#, go, java, javascript, and python build the dense 26-wide goto table, each trie node owning a full row either as an array slice or an int32 span, which is the upgrade the sketch's second approach allows when memory holds it. Lua keeps hash children and a memoized transition instead, and the memo must saturate: every state on the fail chain a query walks shares the target, so a cold chain is stamped end to end in one pass, the failure mode being a re-walk per step that degrades the walk quadratically. Counts reach 1e6, inside int32 and exact in javascript's `Number`.

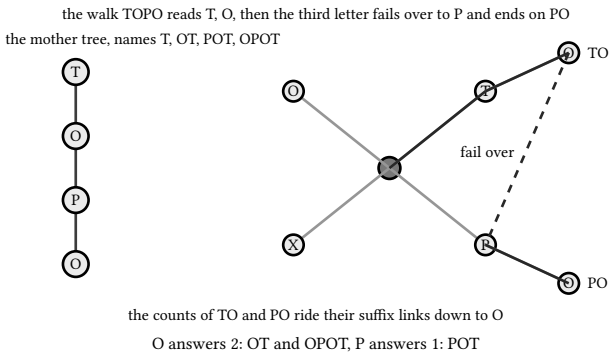


figure 136: the fixture's mother chain on the left, the query trie of reversed queries on the right, the name OPOT walking the heavy path TOPO whose suffix links feed the O and P queries

The crafted fixture is the four-lady family T, then O under T, P under O, O under P, with the five queries P, O, OT, OP, X. The names are T, OT, POT, and OPOT, so the pinned answers are 1, 2, 1, 1, and 0, one per line. The C suite pins it with a founder-only case and the AB versus BA pair that anchors prefix direction, three `CHECK` compares, the C# facts run four over the same family, Go's table carries the fixture, the javascript `it` blocks take the fixture, the founder, and AB against BA, python's four checks add a deep fifth lady, and lua's four run the same family, java's four taking the fixture, the founder, AB against BA, and the deep fifth lady, so the direction case, where AB matches the name AB but BA does not, is pinned in every suite that carries more than the fixture.

12.9. problem H, hobson's trains

Mr. Hobson has traded his stable for a rail network and kept his commitment to sparing passengers any choice: the network has n stations, and from each station a passenger can catch exactly one train, one leg, to exactly one other station, a one-way trip that might have no way back. The only ticket on sale allows up to k legs in one trip, and the single reader at each station's exit must carry the list of starting stations it accepts, so the task is, for every station A, the number of stations including A itself from which A is reachable in at most k legs (statement).

The input is two integers n and k, 2 <= n <= 5e5 stations and 1 <= k <= n - 1 legs by the problemset pdf, then n lines of one integer di, 1 <= di <= n and di never equal to i, the one-leg destination from station i. The output is n lines, the ith holding the count for station i, under the pdf's 5 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `H-hobsonstrains/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: a three-station chain and a side tail feeding a two-station ring, and station 4 tops the table at five, its ring partner, two chain stations, and the tail station all reaching it within two legs.

input:

```
6 2
2
3
4
5
4
3
```

expected output:

```
1
2
4
5
3
1
```

Recognition: n <= 5e5 stations under the 5 second limit price one functional-graph decomposition: an indegree pass, a peel queue, and two tree folds, about 2e6 steps against a budget holding 1e9, three orders of magnitude to spare.

The statement's cue is the exactly-one-train rule: one outgoing leg per station makes the network a functional graph, rings with trees hanging off them, and a station's reachers within k legs are its own subtree count plus an arc of its ring, component structure answering a question that reads like a search. Problem H is the 2019 face of the component structure and connectivity family, and the component structure and connectivity family runs through chapter 11 problem B, chapter 12 problem E, chapter 13 problem R, chapter 15 problem E, and chapter 15 problem G.

The tempting alternative, a backward breadth-first search per station, is priced out at 5e5 stations times up to 5e5 reached each, 2.5e11 leg walks, while the peel answers all n stations in one pass because the k-leg reachers are subtree counts plus wrapped ring arcs.

A functional graph gives every station exactly one outgoing leg, and the pinned solver peels trees off the rings with an indegree queue, then counts in two layers: per tree an iterative dfs carries the ancestor path on a stack, so f of v counts subtree nodes at exact distance k by a k-deep lookback, and a children-before-parents pass folds g of v, the subtree nodes within k, as one plus the sum over children of g minus f. Every node whose ring entry is reached within k legs, ring nodes at distance zero included, adds one to the forward ring arc of length min(k - dist + 1, ring length) starting at its entry, accumulated with a wrapped difference array, and ring answers are the arc prefix sums while tree answers are g (solutions.pdf pp. 5-6, O(n)). Book 10′s chapter 39, graph connectivity and decomposition, develops the functional graph itself, this ring-with-hanging-trees decomposition included. The seven tree walks follow.

The worked run: trace the model on sample 1. The successor list 2, 3, 4, 5, 4, 3 closes the ring 4-5 and hangs the trees under 4: 3 feeds it with 6 and 2 feeding 3, and 1 feeds 2. The tree folds give g, the reachers within k = 2 legs inside each subtree: g of 1 is 1, g of 2 is 2, itself and 1, g of 6 is 1, and g of 3 is 1 + 2 + 1 = 4 after dropping each child's exact-distance-2 count, zero on these small subtrees. The ring arcs then charge every node that reaches its entry within 2 legs: entries 4 and 5 themselves contribute length min(2 + 1, 2) = 2 each, node 3 at distance 1 contributes length 2, nodes 6 and 2 at distance 2 contribute length 1 onto station 4 alone, and node 1 at distance 3 never reaches the ring. Station 4′s arc sums to 1 + 1 + 1 + 1 + 1 = 5, station 5′s to 3, and the tree answers are the g values, so the six lines read 1, 2, 4, 5, 3, 1 and the trace ends at the printed answer 1.

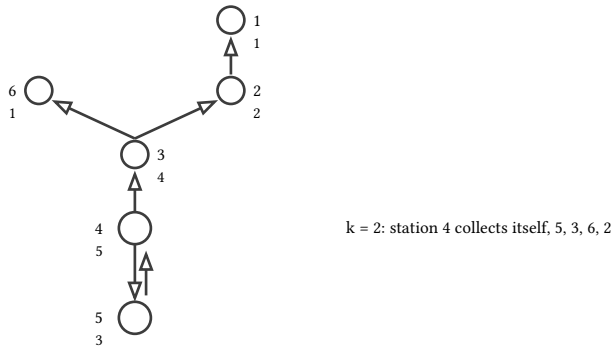


figure 137: sample 1′s ring 4-5 with its hanging tree, every station labeled with its answer, station 4′s five reachers marked

```c
// euler dfs per tree with the ancestor path on a stack
static int estack[500005], path[500005], childit[500005];
memset(visited, 0, (size_t)n + 1);
for (int v = 1; v ≤ n; v++)
  f[v] = 0;
for (int root = 1; root ≤ n; root++) {
  if (visited[root] || onring[root] || !onring[d[root]])
    continue; // only tree roots hanging directly off a ring node
  int esp = 0, plen = 0;
  estack[esp++] = root;
  childit[root] = headc[root];
  visited[root] = 1;
  path[plen++] = root;
  if (plen > K)
    f[path[plen - 1 - K]]++;
  while (esp) {
    int v = estack[esp - 1];
    int c = childit[v];
    while (c && visited[c])
      c = nxt[c];
    if (c) {
      childit[v] = nxt[c];
      estack[esp++] = c;
      childit[c] = headc[c];
      visited[c] = 1;
      path[plen++] = c;
      if (plen > K)
        f[path[plen - 1 - K]]++;
    } else {
      esp--;
      plen--;
    }
  }
}
// g in peel order (children appear before parents)
for (int v = 1; v ≤ n; v++)
  g[v] = 1;
for (int i = 0; i < qt; i++) {
  int v = peel[i];
  long long sum = 1;
  for (int c = headc[v]; c; c = nxt[c])
    sum += g[c] - f[c];
  g[v] = (int)sum;
}
```

listing 377: c: the euler dfs with the ancestor path on a stack, f charged to the k-th ancestor on entry, g folded in peel order (source: icpc/samples-c/src/Ch10/pH.c, lines 54-97)

```
    for i := range ring {
      r := ring[i]
      type frame struct{ v, depth, next int }
      stack := []frame{}
      path := []int{}
      for h := starts[r]; h < starts[r+1]; h++ {
        c := childNode[h]
        if peeled[c] {
          stack = append(stack, frame{c, 1, starts[c]})
        }
      }
      for len(stack) > 0 {
        fr := &stack[len(stack)-1]
        if fr.next == starts[fr.v] {
          // entering v
          path = append(path, fr.v)
          if fr.depth <= k {
            addArc(i, k-fr.depth+1)
          }
          if fr.depth ≥ k && len(path)-1-k ≥ 0 {
            f[path[len(path)-1-k]]++
          }
        }
        if fr.next < starts[fr.v+1] {
          c := childNode[fr.next]
          fr.next++
          if peeled[c] {
            stack = append(stack, frame{c, fr.depth + 1, starts[c]})
          }
        } else {
          // leaving v: compute g now that children are done
          total := 1
          for h := starts[fr.v]; h < starts[fr.v+1]; h++ {
            c := childNode[h]
            if peeled[c] {
              total += g[c] - f[c]
            }
          }
          g[fr.v] = total
          path = path[:len(path)-1]
          stack = stack[:len(stack)-1]
        }
      }
    }
```

listing 378: go: per ring entry, the dfs frames carry depth and cursor, arcs charged on entry, g on the leave step (source: icpc/samples-go/ch10/ph.go, lines 104-147)

```
// euler dfs per tree with the ancestor path on a stack
int[] estack = new int[MAXN], path = new int[MAXN], childit = new int[MAXN];
for (int root = 1; root ≤ n; root++) {
  if (visited[root] || onring[root] || !onring[d[root]])
    continue; // only tree roots hanging directly off a ring node
  int esp = 0, plen = 0;
  estack[esp++] = root;
  childit[root] = headc[root];
  visited[root] = true;
  path[plen++] = root;
  if (plen > K)
    f[path[plen - 1 - K]]++;
  while (esp > 0) {
    int v = estack[esp - 1];
    int c = childit[v];
    while (c ≠ 0 && visited[c])
      c = nxt[c];
    if (c ≠ 0) {
      childit[v] = nxt[c];
      estack[esp++] = c;
      childit[c] = headc[c];
      visited[c] = true;
      path[plen++] = c;
      if (plen > K)
        f[path[plen - 1 - K]]++;
    } else {
      esp--;
      plen--;
    }
  }
}
// g in peel order (children appear before parents)
for (int v = 1; v ≤ n; v++)
  g[v] = 1;
for (int i = 0; i < qt; i++) {
  int v = peel[i];
  long sum = 1;
  for (int c = headc[v]; c ≠ 0; c = nxt[c])
    sum += g[c] - f[c];
  g[v] = (int) sum;
}
```

listing 379: java: the euler dfs with the ancestor path on a stack, f charged to the k-th ancestor on entry, g folded in peel order (source: icpc/samples-java/src/Ch11/PH.java, lines 63-103)

```
        // the tree (the k-th ancestor is the ring entry or beyond)
        var jump = new int[n + 1];
        var path = new int[n + 2];
        var st = new (int Node, int Len)[n];
        for (int root = 1; root ≤ n; root++)
        {
            if (core[root] == 0)
                continue;
            for (int v0 = kidHead[root]; v0 ≥ 0; v0 = kidNext[v0])
            {
                if (core[v0] > 0)
                    continue;
                int top = 0;
                st[top++] = (v0, 1);
                while (top > 0)
                {
                    var (u, pl) = st[--top];
                    path[pl] = u;
                    jump[u] = pl > k ? path[pl - (int)k] : 0;
                    for (int w = kidHead[u]; w ≥ 0; w = kidNext[w])
                        st[top++] = (w, pl + 1);
                }
            }
        }

        // f[u] = subtree nodes at exact distance k = #{w : jump[w] = u}
        var f = new long[n + 1];
        foreach (int w in treeNodes)
            if (jump[w] ≠ 0)
                f[jump[w]]++;
```

listing 380: c#: the k-th-ancestor pointer path, f counted per tree node, g summed deepest first (source: icpc/samples/src/Ch10/PH.cs, lines 76-105)

```
  // each hanging tree: iterative dfs carrying depth and the
  // ancestor path for f
  for (let i = 0; i < L; i++) {
    const r = ring[i];
    const stack = [];
    const path = [];
    for (let h = starts[r]; h < starts[r + 1]; h++) {
      const c = childNode[h];
      if (peeled[c]) stack.push([c, 1, starts[c]]);
    }
    while (stack.length > 0) {
      const fr = stack[stack.length - 1];
      if (fr[2] === starts[fr[0]]) {
        // entering v
        path.push(fr[0]);
        if (fr[1] ≤ k) addArc(i, k - fr[1] + 1);
        if (fr[1] ≥ k && path.length - 1 - k ≥ 0) {
          f[path[path.length - 1 - k]]++;
        }
      }
      if (fr[2] < starts[fr[0] + 1]) {
        const c = childNode[fr[2]];
        fr[2]++;
        if (peeled[c]) stack.push([c, fr[1] + 1, starts[c]]);
      } else {
        // leaving v: compute g now that children are done
        let total = 1;
        for (let h = starts[fr[0]]; h < starts[fr[0] + 1]; h++) {
          const c = childNode[h];
          if (peeled[c]) total += g[c] - f[c];
        }
        g[fr[0]] = total;
        path.pop();
        stack.pop();
      }
    }
  }
```

listing 381: javascript: the same frame stack, path lookback for f, children totals for g (source: icpc/samples-js/src/ch10-ph-hobsonstrains.mjs, lines 70-106)

```
        # each hanging tree: iterative dfs carrying depth and the
        # ancestor path for f
        for i in range(length):
            r = ring[i]
            kids = child[starts[r] : starts[r + 1]]
            stack = [[c, 1, starts[c]] for c in kids if peeled[c]]
            path = []
            while stack:
                fr = stack[-1]
                v, depth = fr[0], fr[1]
                if fr[2] == starts[v]:
                    # entering v
                    path.append(v)
                    if depth ≤ k:
                        add_arc(i, k - depth + 1)
                    if depth ≥ k and len(path) - 1 - k ≥ 0:
                        f[path[len(path) - 1 - k]] += 1
                if fr[2] < starts[v + 1]:
                    c = child[fr[2]]
                    fr[2] += 1
                    if peeled[c]:
                        stack.append([c, depth + 1, starts[c]])
                else:
                    # leaving v: children are done, so g folds together
                    total = 1
                    for c in child[starts[v] : starts[v + 1]]:
                        total += g[c] - f[c]
                    g[v] = total
                    path.pop()
                    stack.pop()
```

listing 382: python: ring arcs for the entry itself, then the hanging trees, enter and leave steps over one stack (source: icpc/samples-py/src/Ch10/ph.py, lines 75-104)

```
    -- each hanging tree: iterative dfs with an ancestor path
    for i2 = 0, L - 1 do
      local r = ring[i2 + 1]
      local sv, sd, sn = {}, {}, {}
      local sp = 0
      for h = starts[r] + 1, starts[r + 1] do
        local c = child[h]
        if peeled[c] then
          sp = sp + 1
          sv[sp], sd[sp], sn[sp] = c, 1, starts[c] + 1
        end
      end
      local path = {}
      while sp > 0 do
        local v3, depth = sv[sp], sd[sp]
        if sn[sp] == starts[v3] + 1 then
          path[#path + 1] = v3
          if depth ≤ k then add_arc(i2, k - depth + 1) end
          if depth ≥ k and #path - 1 - k ≥ 1 then
            local w = path[#path - k]
            f[w] = (f[w] or 0) + 1
          end
        end
        if sn[sp] < starts[v3 + 1] + 1 then
          local c = child[sn[sp]]
          sn[sp] = sn[sp] + 1
          if peeled[c] then
            sp = sp + 1
            sv[sp], sd[sp], sn[sp] = c, depth + 1, starts[c] + 1
          end
        else
          local total = 1
          for h = starts[v3] + 1, starts[v3 + 1] do
            local c = child[h]
            if peeled[c] then
              total = total + (g[c] or 0) - (f[c] or 0)
            end
          end
          g[v3] = total
          path[#path] = nil
          sp = sp - 1
        end
      end
    end
```

listing 383: lua: parallel stacks for vertex, depth, and cursor, the ancestor path charged on entry (source: icpc/samples-lua/ch10_ph.lua, lines 102-145)

Answers stay at most 5e5, int32 and exact in `Number`. The layout splits on the ring accumulation: C and java lay every ring's doubled difference array consecutively in one global buffer and read each node as the sum of its two wrapped prefix readings, the other five keep one local array per ring and handle the wrap inside the add, the same arithmetic in two shapes.

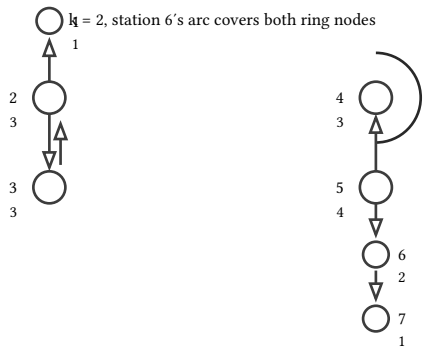


figure 138: the fixture's two rings with their hanging trees, every station labeled with its answer, station 6′s +1 arc drawn as a heavy ring segment

The crafted fixture is seven stations at k = 2 with successor list 2, 3, 2, 5, 4, 5, 6: rings 2-3 and 4-5, tree 1 hanging off 2, tree 7-6 hanging off 5. The pinned answers are 1 3 3 3 4 2 1, one per line, station 5 collecting four because both 6 and 7 reach it within two legs. The C suite pins the fixture, both official samples, and a pure ring in four `CHECK` compares, the C# facts run four over the same family, Go's table takes the fixture and the pure ring, the javascript `it` blocks take those two, python's four checks add a ring with a tail and a k = 1 board, and lua's four run the same family, java's six pinning the fixture, both official samples, the pure ring, a ring with a tail, and a k = 1 board, so the pure ring, where every arc wraps fully and the difference array alone answers, is pinned in every suite.

12.10. problem I, karel the robot

The educational language Karel, named for the writer Karel Capek, controls a robot on a grid of unit squares, some free and some holding a barrier, the robot always on a free square facing north, south, east, or west. A program is a string of five command shapes: m moves one square forward unless a barrier blocks it, l turns left, a capital letter invokes the procedure of that name, i followed by a condition and two parenthesized sub-programs runs the first when the condition holds and the second otherwise, and u followed by a condition and one sub-program does nothing when the condition holds, else runs the sub-program and repeats itself. The one-letter conditions are b, holding exactly when the next square in the current heading holds a barrier, and the four headings n, s, e, w, so ub(m) reads as keep moving until a wall and un(l) as turn to face north. Procedure definitions X=program bind names, forward references are allowed, and every square outside the given grid is a barrier, so the robot can never leave. The task is the interpreter itself: run each given program from its start cell and heading, printing the final cell and heading, or inf when the run never terminates (statement).

The input is four integers r, c, d, e, grid dimensions 1 <= r, c <= 40, at most 26 procedure definitions, and 1 <= e <= 10 programs by the problemset pdf, then r grid lines of c characters each, a dot for free and a hash for barrier, running north to south and west to east, then d definition lines with no name defined twice, then 2e lines in pairs, a start line i j h whose cell is guaranteed free and the program text. Every body and program is 1 to 100 characters, syntactically correct, whitespace-free, and invokes only defined procedures. The output is e lines, the final position in the same two-numbers-and-heading format or the word inf, under the pdf's 10 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `I-karel/sample-1.in` and `sample-1.ans`, the only sample the letter carries, the same pair the problems pdf prints: five procedures, seven programs, and two of them diverge, the wall-follower G cycling from its east start and the self-recursive I, defined as III, from anywhere.

input:

```
4 8 5 7
.......#
..#....#
.###...#
.....###
R=lll
```

```
G=ub(B)
B=ub(m)lib(l)(m)
H=ib()(mmHllmll)
I=III
1 1 w
G
1 1 e
G
2 2 n
G
2 6 w
BR
4 1 s
ib(lib()(mmm))(mmmm)
1 1 e
H
2 2 s
I
```

expected output:

```
1 1 w
inf
1 1 w
2 4 s
4 4 e
1 4 e
inf
```

Recognition: the configuration space is 40 by 40 cells times 4 headings, 6400 states, against at most 26 procedures of 100-character bodies, call it 2600 segment offsets, so the memo table holds about 1.7e7 cells and each is written once, an O(rcs) shape comfortable inside the 10 second limit, and the same table is what proves termination.

The statement's cue is the two-word escape hatch: print inf when the run never terminates, which turns the task from interpreting a program into mapping a transition function over robot configurations and code positions, divergence being exactly a cycle in that product. Problem I is the 2019 face of the dp over an engineered state space family, and the dp over an engineered state space family spans chapter 11 problem J, chapter 13 problem S, chapter 14 problem J, and chapter 15 problem H.

The tempting alternative, running the program under a step counter with a guessed bound, is priced out by honesty rather than time: no counter certifies non-termination, and the 1 by 1 world's us(m) loop already runs forever on a three-character program, so only the three-color memo over packed states answers inf.

The solver compiles every procedure, program, and parenthesized sub-program into its own segment, then evaluates the function that maps a configuration and a segment offset to the configuration at the segment end, memoized with three colors over the packed row, column, heading id, where a revisit of an in-progress state means the evaluation cannot terminate and every state it touched goes divergent, and results are caller-independent so the memo carries across all programs (solutions.pdf pp. 6-7, O(rcs) over the configuration space times the code). The interpreter machine is book 10′s chapter 27, an opcode machine, a memoized program-counter interpreter with configuration-space cycle detection, and this letter is its contest form. The machine runs in two vehicles here: c, java, javascript, and python evaluate over an explicit resume stack, the form javascript needs because its fixed call stack is the shallowest of the seven, while go, C#, and lua let the same machine recurse, their stacks carrying the segment depth, with until loops iterating a seen-set of configurations in place of recursion. The seven machines follow.

The worked run: trace the model on sample 1. Program 1 runs G = ub(B) from 1 1 west: the next square west is off the grid, a barrier, so the until condition holds at once and the answer is the start itself, 1 1 w. Program 2 runs the same G facing east: B's ub(m) walks row 1 to 1 7 before the column-8 wall, one l faces north where the grid edge blocks, the if's l branch turns west, and B ends 1 7 w. The next B runs west to 1 1, turns south, steps to 2 1; the third runs south to 4 1, turns east, steps to 4 2; the fourth east to 4 5, north, steps to 3 5; the fifth north to 1 5, west, steps to 1 4; the sixth west to 1 1, south, steps to 2 1 again, the configuration the until loop already saw, so the seen set closes a cycle and the answer is inf. Program 7's I = III calls itself at its own entry, the memo revisits the in-progress state at once, also inf. The terminating programs resolve by the same table, program 4's B ending 2 4 east before R's three lefts turn it south, and the trace ends at the printed answer `inf`.

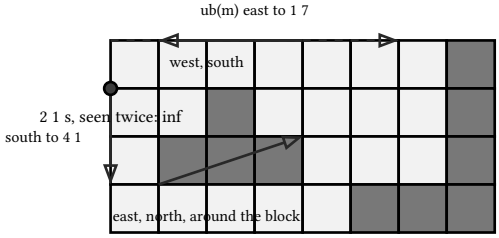


figure 139: sample 1's grid with the wall block shaded and program 2's wall-following lap: east to 1 7, around the block through 2 1, 4 2, 3 5 and 1 4, back to 2 1 south

```
    Op o = ops[pc];
    if (o.kind == OP_SKIP) {
      pc = nxt[pc];
    } else if (o.kind == OP_MOVE) {
      if (!barrier(cfg)) {
        int nr = cfg_row(cfg) + dr[cfg_head(cfg)];
        int nc = cfg_col(cfg) + dc[cfg_head(cfg)];
        cfg = (nr * C + nc) * 4 + cfg_head(cfg);
      }
      pc = nxt[pc];
    } else if (o.kind == OP_LEFT) {
      cfg = (cfg & ~3) | ((cfg_head(cfg) + 3) & 3);
      pc = nxt[pc];
    } else if (o.kind == OP_CALL) {
      resume[nres] = nxt[pc];
      resbase[nres++] = nmarks;
      pc = procseg[o.a];
      fresh = 1;
    } else if (o.kind == OP_IF) {
      resume[nres] = nxt[pc];
      resbase[nres++] = nmarks;
      pc = cond_true(o.cond, cfg) ? o.a : o.b;
      fresh = 1;
    } else { // until: rerun the body, then re-check this same op
      if (cond_true(o.cond, cfg)) {
        pc = nxt[pc];
      } else {
        resume[nres] = pc;
        resbase[nres++] = nmarks;
        pc = o.a;
        fresh = 1;
      }
    }
  }
```

listing 384: c: op dispatch on the resume stack, a finished segment resolving its mark frame, an until re-entry returning to the same op (source: icpc/samples-c/src/Ch10/pI.c, lines 195-228)

```
func (v *karelVM) eval(cfg, seg, off int) (int, bool) {
  if off == len((*v.segs)[seg]) {
    return cfg, true
  }
  key := karelKey{cfg, seg, off}
  switch v.memo[key] {
  case 2:
    return v.result[key], true
  case 3:
    return 0, false
  case 1:
    // cycle through this evaluation: it cannot terminate
    v.memo[key] = 3
    return 0, false
  }
  v.memo[key] = 1
  res, ok := v.step(cfg, seg, off)
  if !ok {
    v.memo[key] = 3
    return 0, false
  }
  v.memo[key] = 2
  v.result[key] = res
  return res, true
}
```

listing 385: go: the three-color memo core, done states replay from the result table, a cycle through the evaluation poisons it (source: icpc/samples-go/ch10/pi.go, lines 154-178)

```
      if (pc < 0) { // a segment finished with cfg: resolve its frame
        if (nres == 0) {
          for (int i = 0; i < nmarks; i++)
            memo[markkey[i]] = cfg + 3;
          return cfg;
        }
        nres--;
        int mb = resbase[nres];
        int rp = resume[nres];
        for (int i = mb; i < nmarks; i++)
          memo[markkey[i]] = cfg + 3;
        nmarks = mb;
        if (rp ≥ 0) { // the caller continues at rp as a fresh state
          pc = rp;
          fresh = true;
        } else { // the caller's segment ends here too: chain the finish
          pc = -1;
        }
        continue;
      }
      Op o = ops[pc];
      if (o.kind == OP_SKIP) {
        pc = nxt[pc];
      } else if (o.kind == OP_MOVE) {
        if (!barrier(cfg)) {
          int nr = cfgRow(cfg) + dr[cfgHead(cfg)];
          int nc = cfgCol(cfg) + dc[cfgHead(cfg)];
          cfg = (nr * C + nc) * 4 + cfgHead(cfg);
        }
        pc = nxt[pc];
      } else if (o.kind == OP_LEFT) {
        cfg = (cfg & ~3) | ((cfgHead(cfg) + 3) & 3);
        pc = nxt[pc];
      } else if (o.kind == OP_CALL) {
        resume[nres] = nxt[pc];
        resbase[nres++] = nmarks;
        pc = procseg[o.a];
        fresh = true;
      } else if (o.kind == OP_IF) {
        resume[nres] = nxt[pc];
        resbase[nres++] = nmarks;
        pc = condTrue(o.cond, cfg) ? o.a : o.b;
        fresh = true;
      } else { // until: rerun the body, then re-check this same op
        if (condTrue(o.cond, cfg)) {
          pc = nxt[pc];
        } else {
          resume[nres] = pc;
          resbase[nres++] = nmarks;
          pc = o.a;
          fresh = true;
        }
      }
    }
```

listing 386: java: op dispatch on the resume stack, a finished segment resolving its mark frame, an until re-entry returning to the same op (source: icpc/samples-java/src/Ch11/PI.java, lines 216-269)

```
                        cur = cur - h * R * C + (h + 3) % 4 * R * C;
                        break;
                    }
                case OpCall:
                    {
                        cur = Eval(cur, ops[idx + 1]);
                        if (cur < 0)
                            return memo[seg][cfg] = Diverged;
                        idx += 1;
                        break;
                    }
                case OpIf:
                    {
                        int chosen = CondVal(cur, ops[idx + 1])
                            ? ops[idx + 2]
                            : ops[idx + 3];
                        cur = Eval(cur, chosen);
                        if (cur < 0)
                            return memo[seg][cfg] = Diverged;
                        idx += 3;
                        break;
                    }
                default:
                    {
                        int cond = ops[idx + 1];
                        int body = ops[idx + 2];
                        var seen = new HashSet<int>();
                        while (!CondVal(cur, cond))
                        {
                            if (!seen.Add(cur))
                                return memo[seg][cfg] = Diverged;
                            cur = Eval(cur, body);
                            if (cur < 0)
                                return memo[seg][cfg] = Diverged;
                        }
                        idx += 2;
                        break;
                    }
            }
        }
        return memo[seg][cfg] = cur;
    }
```

listing 387: c#: the recursive evaluator, call and branch delegating to sub-segments, the until loop cycling on a seen set (source: icpc/samples/src/Ch10/PI.cs, lines 159-200)

```
      const f = frames[frames.length - 1];
      if (f.phase === "u") {
        if (this.condTrue(f.cur, f.op.cond)) {
          if (!advance(this.enter(f, f.cur, f.seg, f.off + 1))) {
            failed = true; break;
          }
          continue;
        }
        const child = {
          keys: [], phase: "run", cfg: f.cur, seg: 0, off: 0,
          op: null, seen: null, cur: 0,
        };
        f.phase = "ubody";
        const cr = this.enter(child, f.cur, f.op.a, 0);
        if (cr === null) { failed = true; break; }
        if (cr !== -2) val = cr;
        else frames.push(child);
        continue;
      }
      // phase "run": dispatch the op at (seg, off)
      const op = this.segs[f.seg][f.off];
      if (op.kind === "u") {
        f.op = op;
        f.cur = f.cfg;
        f.seen = new Set([f.cfg]);
        f.phase = "u";
        continue;
      }
      if (op.kind === "m" || op.kind === "l") {
        const next = op.kind === "m" ? this.stepCfg(f.cfg) : this.turnCfg(f.cfg);
        if (!advance(this.enter(f, next, f.seg, f.off + 1))) {
          failed = true; break;
        }
        continue;
      }
      // call or conditional: evaluate the chosen sub-segment first
      const branch = op.kind === "c"
        ? this.procSeg.get(op.cond)
        : this.condTrue(f.cfg, op.cond) ? op.a : op.b;
      const child = {
        keys: [], phase: "run", cfg: f.cfg, seg: branch, off: 0,
        op: null, seen: null, cur: 0,
      };
      f.phase = "sub";
      const cr = this.enter(child, f.cfg, branch, 0);
      if (cr === null) { failed = true; break; }
      if (cr !== -2) val = cr;
      else frames.push(child);
    }
```

listing 388: javascript: the frame machine, run, sub, and until phases over one explicit stack, settle stamps a whole frame done (source: icpc/samples-js/src/ch10-pi-karel.mjs, lines 126-174)

```
        v = memo[seg_base[seg0] * ncfg + cfg0]
        if v:
            return INF if v == INF else v - 1
        try:
            push(cfg0, seg0)
            while True:
                f = stack[-1]
                cfg, seg, off = f[0], f[1], f[2]
                ops = segs[seg]
                if off == len(ops):
                    finish(cfg)
                    continue
                kind, cond, a, b = ops[off]
                if kind == "m":
                    nxt = move(cfg)
                    to_state(cfg if nxt is None else nxt, off + 1)
                elif kind == "l":
                    to_state(cfg - (cfg & 3) + ((cfg & 3) + 3) % 4, off + 1)
                elif kind == "c":
                    f[3] = 1
                    push(cfg, proc_seg[cond])
                elif kind == "i":
                    f[3] = 1
                    push(cfg, a if cond_true(cfg, cond) else b)
                elif cond_true(cfg, cond):
                    to_state(cfg, off + 1)
                else:
                    f[3] = 2
                    push(cfg, a)
        except _Diverged:
            for fr in stack:
                for key in fr[4]:
                    memo[key] = INF
            return INF
        except _Finished as fin:
            return fin.cfg
```

listing 389: python: the dispatch loop advancing a frame under the three-color protocol, a finished frame unwinding on exceptions (source: icpc/samples-py/src/Ch10/pi.py, lines 158-193)

```lua
eval = function(cfg, seg, off)
  local ops = segs[seg]
  if off > #ops then return cfg end
  local key = cfg * 1000003 * 409 + seg * 409 + off
  local m = memo[key]
  if m == 2 then return res[key] end
  if m == 3 or m == 1 then
    memo[key] = 3
    return nil
  end
  memo[key] = 1
  local op = ops[off]
  local out
  if op[1] == 109 then -- 'm'
    local nxt = blocked(cfg) and cfg or move_cfg(cfg)
    out = eval(nxt, seg, off + 1)
  elseif op[1] == 108 then -- 'l'
    local h = cfg % 4
    out = eval(cfg - h + (h + 3) % 4, seg, off + 1)
  elseif op[1] == 99 then -- 'c'
    local sub = eval(cfg, proc_seg[op[2]], 1)
    if sub then out = eval(sub, seg, off + 1) end
  elseif op[1] == 105 then -- 'i'
    local branch = cond_true(cfg, op[2]) and op[3] or op[4]
    local sub = eval(cfg, branch, 1)
    if sub then out = eval(sub, seg, off + 1) end
  else -- 'u'
    local cur = cfg
    local seen = { [cfg] = true }
    while not cond_true(cur, op[2]) do
      cur = eval(cur, op[3], 1)
      if not cur then return nil end
      if seen[cur] then return nil end
      seen[cur] = true
    end
    out = eval(cur, seg, off + 1)
  end
  if not out then
    memo[key] = 3
    return nil
  end
  memo[key] = 2
  res[key] = out
  return out
end
```

listing 390: lua: the recursive evaluator over a hashed memo key, until loops cycling on seen configurations (source: icpc/samples-lua/ch10_pi.lua, lines 94-138)

Grids are 40 by 40 with at most 26 procedures and 100-character bodies, so configurations pack into one int32 and every language fits its memo densely except lua, whose hash key folds configuration, segment, and offset into one integer. No integer edges are crossed.

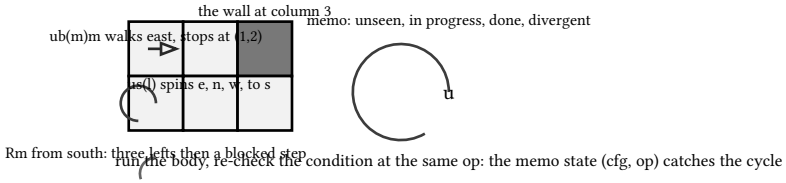


figure 140: the fixture's 2 by 3 grid: the barrier cell, program 1 walking east until the wall, programs 2 and 3 turning in place, the until op drawn as a loop arrow

The crafted fixture is a 2 by 3 grid with a wall at the top right, one procedure R of three lefts, and three programs: ub(m)m from (1, 1) east ends at 1 2 e, Rm from (1, 1) south ends at 1 1 w, and us(l) from (1, 1) east ends at 1 1 s. The pinned output is those three lines. The C suite adds the 1 by 1 world where us(m) never satisfies its condition and the official first sample with all seven programs byte-identical to the judge answer, three `CHECK` compares, the C# facts run three keeping the 1 by 1 world beside a conditional board, Go's table carries the fixture alone, the javascript `it` blocks take the fixture, the full official sample, and the stuck 1 by 1 loop, and python's and lua's three checks keep the 1 by 1 world with one crafted extra each, java's four pinning the fixture, its split variant, the stuck loop, and the official sample, so the divergence witness is pinned in six of the seven suites, only go's table stopping at the fixture.

12.11. problem J, miniature golf

A group of friends has finished a round of miniature golf, and nobody can remember l, the cap the course puts on a hole score: a player who has hit the ball l times without sinking it scores l and the turn ends. They played without any cap and recorded the true hit counts, intending to look up l afterward and adjust, replacing any per-hole score above l with l, with the total the sum over holes and lower better. The rank of a player is the number of players whose adjusted total is less than or equal to theirs, so with adjusted totals 3, 5, 5, 4, 3 the ranks read 2, 5, 5, 3, 2. The task is each player's smallest possible rank over every choice of the positive integer l (statement).

The input is two integers p and h, 2 <= p <= 500 players and 1 <= h <= 50 holes by the problemset pdf, then p lines of h positive integers, player i's hole scores, each at most 1e9. The output is p lines, player i's minimum possible rank in input order, under the pdf's 6 second limit.

Official sample 2, reprinted byte for byte from the judge data pair `J-minigolf/sample-2.in` and `sample-2.ans`, the same pair the problems pdf prints: six players over four holes, and the answers are not a permutation, two players tying at rank 5, because rank counts everyone at or below a total rather than ordering the players.

input:

```
6 4
3 1 2 2
4 3 2 2
6 6 3 2
7 3 4 3
3 4 2 4
2 3 3 5
```

expected output:

```
1
2
5
5
4
3
```

Recognition: p <= 500 players and h <= 50 holes price the pairwise machine: 500 choose 2 = 124750 pairs, each walking at most 2h = 100 merged breakpoints, about 1.2e7 steps plus a 500-position event sweep per player, O(p^2 h) with two orders of magnitude to spare inside the 6 second limit.

The statement's cue is the single scalar cap: the rank is asked over every choice of the positive integer l, and each adjusted total is piecewise linear in l with breakpoints at the scores, so the whole question lives on one axis where each pair's preference is an interval and the best rank is the deepest overlap. Problem J is the 2019 face of the event sweep over a sorted axis family, and the same event sweep over a sorted axis cue drives chapter 11 problem A, chapter 11 problem H, chapter 12 problem D, and chapter 15 problem L.

The tempting alternative, evaluating the rank at every candidate cap, is priced out twice over: the scores run to 1e9, and even the 2e4 distinct score values times 500^2 rank computations is 6e9 steps, five hundred times the pair walk.

Adjusted totals are piecewise linear in l with breakpoints at the scores, so the difference of two players' totals is linear on each segment between their merged breakpoints, the player beats the opponent exactly where it is negative, and a sign flip inside a segment pins the integer interval by exact division, past the last breakpoint by the total-order comparison. Per player, a plus-and-minus event sweep over those intervals takes the best overlap, and the rank is 1 + (p - 1 - best) (solutions.pdf pp. 7-8, O(p^2 h log(ph))). The seven sweeps follow.

The worked run: trace the model on sample 2, the pair the section prints. Capping at l = 2 flattens every row but player 1′s second hole: player 1 totals 7 while the other five total 8 each, five opponents strictly above, rank 1 + (6 - 1 - 5) = 1. Player 2 caps to 10 at l = 3 against 8, 11, 12, 11, 11, four above, rank 2, and never reaches 1 because player 1′s total stays at or below player 2′s at every cap. Player 3 totals 11 at l = 3 with only player 4′s 12 above it, rank 5, and its other caps tie it into the 8s at l = 2 or put its 17 on top at l = 6, both rank 6. Player 4 waits for l = 6, total 16 with only player 3′s 17 above, rank 5. Player 5 at l = 6 totals 13 under the 17 and 16, rank 4. Player 6 at l = 4 totals 12 under 13, 14 and 13, rank 3. The six lines read 1, 2, 5, 5, 4, 3 and the trace ends at the printed answer 3.

player	best cap	own total	opponents above	rank
1	2	7	5	1
2	3	10	4	2
3	3	11	1	5
4	6	16	1	5
5	6	13	2	4
6	4	12	3	3

```
    for (int b = 0; b < u; b++) {
      long long a = bpx[b], e = b + 1 < u ? bpx[b + 1] : bpx[b];
      // half-open piece [a, e): the shared endpoint belongs to the
      // next piece; the last breakpoint is its own closed piece
      long long hi = b + 1 < u ? e - 1 : a;
      long long da = adj_total(score[i], h, a) - adj_total(score[j], h, a);
      long long de = adj_total(score[i], h, e) - adj_total(score[j], h, e);
      // d is linear on [a, e] since no breakpoint lies inside; the
      // slope is the rational (de - da)/(e - a) with |de - da| ≤ h,
      // and a sign flip pins |da| and |de| under h too, so the root
      // arithmetic below stays far inside int64
      if (da < 0 && de < 0) {
        ne = add_win_interval(ne, a, hi);
      } else if (da < 0 && de ≥ 0) {
        long long num = -da * (e - a), den = de - da;
        // d(l) < 0 for l < a + num/den: wins reach ceil(root) - 1
        long long ceilr = a + (num + den - 1) / den; // num, den > 0
        long long last = ceilr - 1 < hi ? ceilr - 1 : hi;
        ne = add_win_interval(ne, a, last);
      } else if (da ≥ 0 && de < 0) {
        long long num = -da * (e - a), den = de - da;
        // d(l) < 0 for l - a > num/den with den < 0; num ≤ 0 makes
        // the quotient nonnegative, so truncating division floors
        ne = add_win_interval(ne, a + num / den + 1, hi);
      }
    }
    if (toti < totj) { // past the last breakpoint, always beating
      ne = add_win_interval(ne, bpx[u - 1] + 1, (long long)4e18);
    }
  }
```

listing 391: c: per segment the endpoints' differences give the slope, the flip emits an interval by floor or ceil division, the tail extends to the sentinel (source: icpc/samples-c/src/Ch10/pJ.c, lines 89-118)

```go
// beatsInterval returns the integer subinterval of [a, b) where
// da + slope*(l-a) < 0, computed by exact integer division at the
// sign flip.
func beatsInterval(a, b, da, slope int64) (int64, int64, bool) {
  switch {
  case slope == 0:
    if da < 0 {
      return a, b, true
    }
    return 0, 0, false
  case slope > 0:
    if da ≥ 0 {
      return 0, 0, false
    }
    hi := a + ceilDiv64(-da, slope)
    if hi > b {
      hi = b
    }
    return a, hi, hi > a
  default:
    lo := a + floorDiv64(da, -slope) + 1
    if lo < a {
      lo = a
    }
    if lo ≥ b {
      return 0, 0, false
    }
    return lo, b, true
  }
}
```

listing 392: go: beatsInterval returns the integer subinterval where the difference is negative, exact at the flip (source: icpc/samples-go/ch10/pj.go, lines 85-114)

```
        for (int b = 0; b < u; b++) {
          long a = bpx[b], e = b + 1 < u ? bpx[b + 1] : bpx[b];
          // half-open piece [a, e): the shared endpoint belongs to the
          // next piece; the last breakpoint is its own closed piece
          long hi = b + 1 < u ? e - 1 : a;
          long da = adjTotal(score[i], a) - adjTotal(score[j], a);
          long de = adjTotal(score[i], e) - adjTotal(score[j], e);
          // d is linear on [a, e] since no breakpoint lies inside; the
          // slope is the rational (de - da)/(e - a) with |de - da| ≤ h,
          // and a sign flip pins |da| and |de| under h too, so the root
          // arithmetic below stays far inside int64
          if (da < 0 && de < 0) {
            ne = addWinInterval(ev, ne, a, hi);
          } else if (da < 0 && de ≥ 0) {
            long num = -da * (e - a), den = de - da;
            // d(l) < 0 for l < a + num/den: wins reach ceil(root) - 1
            long ceilr = a + (num + den - 1) / den; // num, den > 0
            long last = Math.min(ceilr - 1, hi);
            ne = addWinInterval(ev, ne, a, last);
          } else if (da ≥ 0 && de < 0) {
            long num = -da * (e - a), den = de - da;
            // d(l) < 0 for l - a > num/den with den < 0; num ≤ 0 makes
            // the quotient nonnegative, so truncating division floors
            ne = addWinInterval(ev, ne, a + num / den + 1, hi);
          }
        }
        if (toti < totj) { // past the last breakpoint, always beating
          ne = addWinInterval(ev, ne, bpx[u - 1] + 1, 4_000_000_000_000_000_000L);
        }
```

listing 393: java: per segment the endpoints' differences give the slope, the three flip cases emit intervals by ceil or floor division, the tail extends to the sentinel (source: icpc/samples-java/src/Ch11/PJ.java, lines 75-103)

```
        long dPrev = 0;
        for (int t = 0; t < m; t++)
        {
            long x = xs[t];
            long d = Adj(a, x) - Adj(b, x);
            if (t > 0)
            {
                long lo0 = xs[t - 1];
                long slope = (d - dPrev) / (x - lo0);
                if (dPrev < 0 && d ≥ 0)
                {
                    // negative on (lo0, hi]; lo0 covered by the previous emit
                    long hi = lo0 + FloorDiv(-1 - dPrev, slope);
                    if (hi > lo0)
                        yield return (lo0 + 1, hi);
                }
                else if (dPrev ≥ 0 && d < 0)
                {
                    // negative on [lo, x]; lo0 itself was not negative
                    long lo = lo0 + CeilDiv(-1 - dPrev, slope);
                    yield return (lo, x);
                }
                else if (dPrev < 0 && d < 0)
                {
                    // whole segment negative, but lo0 already emitted by the
                    // previous segment: emit (lo0 + 1, x)
                    yield return (lo0 + 1, x);
                }
            }
            else if (d < 0)
            {
                yield return (x, x);
            }
            dPrev = d;
        }
        // beyond the last breakpoint the sign is the total-order
        // comparison: the tail interval extends to infinity
        if (m > 0 && dPrev < 0)
            yield return (xs[m - 1] + 1, long.MaxValue / 4);
    }
```

listing 394: c#: Beats yields closed intervals between consecutive breakpoints, the three flip cases plus the infinity tail (source: icpc/samples/src/Ch10/PJ.cs, lines 74-113)

```
  for (let i = 0; i < p; i++) {
    const events = [];
    for (let j = 0; j < p; j++) {
      if (i === j) continue;
      const bps = [...new Set([...sorted[i], ...sorted[j]])].sort((x, y) => x - y);
      let prev = 0;
      let dprev = 0; // d(0) = 0: min(score, 0) = 0
      for (const b of bps) {
        if (b === prev) continue;
        const db = adjOf(i, b) - adjOf(j, b);
        const slope = (db - dprev) / (b - prev);
        const iv = beatsInterval(prev, b, dprev, slope);
        if (iv) events.push([iv[0], 1], [iv[1], -1]);
        prev = b;
        dprev = db;
      }
      if (dprev < 0) events.push([prev, 1], [INF, -1]);
    }
    events.sort((x, y) => x[0] - y[0]);
    let best = 0;
    let run = 0;
    for (let e = 0; e < events.length;) {
      const at = events[e][0];
      while (e < events.length && events[e][0] === at) run += events[e++][1];
      if (run > best) best = run;
    }
    out += `${1 + (p - 1 - best)}\n`;
  }
  return out;
}
```

listing 395: javascript: adjusted totals from a sorted prefix table, events swept position by position (source: icpc/samples-js/src/ch10-pj-minigolf.mjs, lines 55-84)

```
            a = 0
            ri = rj = 0
            sumi = sumj = 0
            ki = kj = 0
            while ri < ni or rj < nj:
                bi = ui[ri] if ri < ni else BIG
                bj = uj[rj] if rj < nj else BIG
                b = bi if bi < bj else bj
                # d(l) = da + slope*(l-a) on [a, b); slope = kj - ki
                da = (sumi - sumj) + a * (kj - ki)
                slope = kj - ki
                if slope == 0:
                    if da < 0:
                        events.append((a, 1))
                        events.append((b, -1))
                elif slope > 0:
                    if da < 0:
                        hi = a + (-da + slope - 1) // slope
                        events.append((a, 1))
                        events.append((b if hi > b else hi, -1))
                else:
                    lo = a + da // (ki - kj) + 1
                    if lo < a:
                        lo = a
                    if lo < b:
                        events.append((lo, 1))
                        events.append((b, -1))
                a = b
                while ri < ni and ui[ri] ≤ a:
                    sumi += ui[ri] * wi[ri]
                    ki += wi[ri]
                    ri += 1
                while rj < nj and uj[rj] ≤ a:
                    sumj += uj[rj] * wj[rj]
                    kj += wj[rj]
                    rj += 1
            dt = (sumi - sumj) + a * (kj - ki)
            if dt < 0:
                events.append((a, 1))
                events.append((BIG, -1))
```

listing 396: python: the pair walk advances running sums and counts across the merged breakpoints, no adjusted total recomputed (source: icpc/samples-py/src/Ch10/pj.py, lines 42-81)

```
    for j = 1, p do
      if i ~= j then
        local sj = sorted[j]
        local pi, pj = 1, 1
        local sumi, sumj = 0, 0 -- sums of scores ≤ u
        local ai, aj = 0, 0 -- adjusted totals at u
        local u = 0
        while pi ≤ h or pj ≤ h do
          local v = si[pi] or big
          local w = sj[pj] or big
          if w < v then v = w end
          -- slope of adj_i - adj_j over (u, v] is the open-count gap
          local slope = (h - pi + 1) - (h - pj + 1)
          while pi ≤ h and si[pi] ≤ v do
            sumi = sumi + si[pi]
            pi = pi + 1
          end
          while pj ≤ h and sj[pj] ≤ v do
            sumj = sumj + sj[pj]
            pj = pj + 1
          end
          if v > u then
            local nai = sumi + v * (h - pi + 1)
            local naj = sumj + v * (h - pj + 1)
            local lo, hi = beats_interval(u, v, ai - aj, slope)
            if lo then
              ev[#ev + 1] = lo * 4 + 1
              ev[#ev + 1] = hi * 4 + 3
            end
            u = v
            ai, aj = nai, naj
          end
        end
        if ai - aj < 0 then
          ev[#ev + 1] = u * 4 + 1
          ev[#ev + 1] = big * 4 + 3
        end
      end
    end
    table.sort(ev)
```

listing 397: lua: the same incremental walk, events packed as at times four plus their delta so the sweep sorts plain integers (source: icpc/samples-lua/ch10_pj.lua, lines 60-99)

Adjusted totals reach 5e10 and the division arithmetic stays bounded by the slope's magnitude at most h before any product, so int64 carries c, C#, go, java, and lua, python is native, and javascript's `Number` is exact below 2^53 with the same bound. The vehicles split on the pair walk: c, C#, go, and java recompute both adjusted totals at each breakpoint, javascript reads them off a sorted prefix table, and python and lua advance running sums and counts, the incremental shape lua needs because a recomputing walk costs p^2 h^2 interpreter steps. Java packs each event as position times 2 plus its start tag in one long array, so a plain sort keeps the c tie order with the ending minus one ahead of the starting plus one.

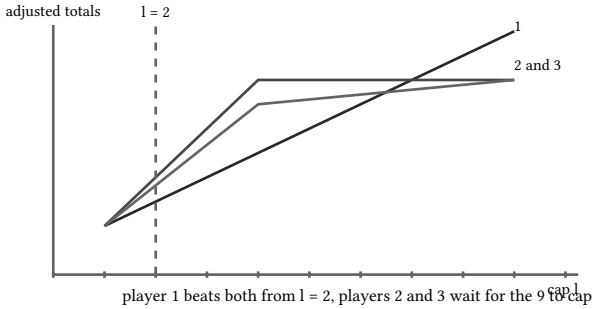


figure 141: the fixture's three adjusted-total lines over the cap axis, the cap 2 dashed vertical where player 1 first stands alone on top, the later crossover past player 1's 9

The crafted fixture is three players over two holes, scores 1 9, 4 4, and 5 3. The adjusted totals cross at cap 2, where player 1's 3 beats the 4s, player 3 passes player 2 at cap 4 where 7 beats 8, and past player 1's 9 the totals settle at 10, 8, 8, so the pinned ranks are 1, 2, and 2, one per line. The C suite pins the fixture, official sample 2, and a uniform crafted board in three `CHECK` compares, the C# facts and python's and lua's three checks stay on crafted boards, the tied 2 by 2 pair and blowout spreads among them, Go's table carries the fixture, and the javascript `it` blocks alone take the fixture and both official samples, answering 1 2 2 and 1 2 5 5 4 3, java's five adding the fixture, sample 2, a fourth player on the same board, and the full-tie and spread families, so sample 2 lives in three suites, sample 1 only in the javascript one, and the fixture in all seven.

12.12. problem K, traffic blights

The Urban Traffic Control department wants a theoretical account of Main Street's lights before anyone risks observing real cars. Lights sit at various points along the street, each cycling red for r seconds then green for g with every light just turned red at time 0. An ideal car mystically appears at the west end of the street at a uniformly random real time in the interval from 0 to 2019 factorial and crawls east at one meter per second until it meets a red light, where it stops forever. For each light the task is the probability that it is the first red light the car hits, followed by the probability the car never stops at all (statement).

The input is one integer n, 1 <= n <= 500 by the problemset pdf, then n lines of three integers x, r, g, the light's position 1 <= x <= 1e5 strictly increasing down the list and its durations with 0 <= r, g and 1 <= r + g <= 100, so a light may be never-red or never-green. The output is n + 1 lines, one probability per light in position order then the never-stop probability, accepted within an absolute error of 1e-6, under the pdf's 2 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `K-trafficblights/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: four lights, and the answer file itself mixes the statement's short decimals with the judge machine's twelve-decimal form, the letter comparing floats within its 1e-6 tolerance.

input:

```
4
1 2 3
6 2 3
10 2 3
16 3 4
```

expected output:

```
0.4
0
0.2
0.171428571429
0.228571428571
```

Recognition: r + g <= 100 and n <= 500 under the 2 second limit collapse the departure phase mod 2520, and the whole computation is 2520 residues times the group walks, each light visited once per phase of its group, about 1e6 steps total, an arithmetic universe small enough to sit in cache.

The statement's cue is the absurd sampling window, a uniform real time from 0 to 2019 factorial: only the departure's phase matters, and 2019 factorial is divisible by 2520, so the universe of departures is 2520 residues, grouped by shared primes and independent across groups by the chinese remainder theorem. Problem K is the 2019 face of the algebraic reduction to a small universe family, and algebraic reduction to a small universe covers chapter 10 problem I and chapter 13 problem Z.

The tempting alternative, multiplying per-light marginals, is priced out by correctness rather than time: lights whose periods share a prime correlate through the same departure residue, sample 1′s three period-5 lights are dependent, and the independence product would answer light 3 with 3/5 times 1/5 = 0.12 against the true 0.2.

Because 2019 factorial is divisible by X = 2520 = 2^3 3^2 5 7, the departure time splits as t0 plus 2520 j, a light of period p sees the phase (t0 + x + 2520 j) mod p, and 2520 j mod p cycles with reduced period p over gcd(p, 2520), which is 1 or a prime power. Lights whose reduced periods share a prime join one group whose modulus is the least common multiple of the members, since their free phases correlate through the shared j residue, distinct groups are independent by the chinese remainder theorem, and the q = 1 lights form one extra group of period one. Book 10′s chapter 16, number theory and modular arithmetic, develops the chinese remainder theorem this independence rides on, and book 10′s chapter 28, advanced number theory, reconstructs each merged residue the garner way. Per residue t0, each group walks its lights in position order over its k phases for first-red counts and survival, another group's lights only matter through the fraction with all of its members before the queried light green, and the fractions multiply across groups (solutions.pdf pp. 7-8). That grouping is one refinement past the sketch text, which groups by reduced period alone: same-prime periods are dependent, and the seven solvers all ship the shared-prime groups. The seven residue walks follow.

The worked run: trace the model on sample 1. The four periods are 5, 5, 5 and 7, so the first three lights share the prime 5 and form one group of modulus 5, light 4 forms its own group of modulus 7, and the two groups are independent by the chinese remainder theorem. Walk the mod-5 group over the departure residue a: light 1 sees phase a + 1, red below 2, so a = 4 and a = 0 stop there, 2 of 5. Light 2 sees phase a + 6 = a + 1, green on every surviving residue, never first. Light 3 sees phase a + 10 = a, red only for a = 1, 1 of 5. Light 4 sees phase b + 16 = b + 2 mod 7 against red 3, red for b = 5, 6 and 0, 3 of 7, and only the 2-in-5 survivors of the first group ever arrive, so light 4 answers 2/5 times 3/7 = 6/35 and survival answers 2/5 times 4/7 = 8/35. The five lines read 0.4, 0, 0.2, 6/35, 8/35, and the trace ends at the printed answer `0.228571428571`.

residue a	light 1, phase a+1	light 2, phase a+1	light 3, phase a	outcome
0	1, red	1, green	-	light 1 first
1	2, green	2, green	1, red	light 3 first
2	3, green	3, green	2, green	survives
3	4, green	4, green	3, green	survives
4	0, red	-	-	light 1 first

```
  for (int t0 = 0; t0 < 2520; t0++) {
    double sv = 1;
    for (int g = 0; g < ngroups; g++) {
      for (int s = 0; s < gq[g]; s++) {
        long long fr = 1 << 30;
        for (int k = 0; k < nmem[g]; k++) {
          int i = members[g][k];
          long long per = RR[i] + GG[i];
          long long q1 = per / gcd_ll(per, 2520);
          long long phase = (t0 + X[i] + 2520 % per * (s % q1)) % per;
          if (phase < RR[i]) {
            fr = X[i];
            break;
          }
        }
        firstred[g][s] = fr;
      }
      for (int k = 0; k ≤ nmem[g]; k++)
        greencnt[g][k] = 0;
      for (int s = 0; s < gq[g]; s++) {
        int k = 0;
        while (k < nmem[g] && X[members[g][k]] < firstred[g][s])
          k++;
        greencnt[g][k]++; // first red sits at member k (k = nmem: none)
      }
      greenfrom[g][nmem[g]] = greencnt[g][nmem[g]];
      for (int k = nmem[g] - 1; k ≥ 0; k--)
        greenfrom[g][k] = greencnt[g][k] + greenfrom[g][k + 1];
      sv *= (double)greenfrom[g][nmem[g]] / gq[g];
    }
    psurv += sv / 2520;
    for (int g = 0; g < ngroups; g++) {
      for (int k = 0; k < nmem[g]; k++) {
        int i = members[g][k];
        int cnt = 0;
        for (int s = 0; s < gq[g]; s++)
          if (firstred[g][s] == X[i])
            cnt++;
        if (!cnt)
          continue;
        double pr = (double)cnt / gq[g];
        for (int g2 = 0; g2 < ngroups && pr > 0; g2++) {
          if (g2 == g)
            continue;
          int k2 = 0;
          while (k2 < nmem[g2] && X[members[g2][k2]] < X[i])
            k2++;
          pr *= (double)greenfrom[g2][k2] / gq[g2];
        }
        pstop[i] += pr / 2520;
      }
    }
```

listing 398: c: per residue, each group's first red per phase, green suffix counts, then the per-light attribution across groups (source: icpc/samples-c/src/Ch10/pK.c, lines 101-152)

```
    for gi := range groups {
      grp := &groups[gi]
      Q := grp.q
      members := grp.member
      stop := make([]float64, len(members))
      var surv int64
      for k := int64(0); k < Q; k++ {
        blocked := false
        for mi, light := range members {
          p := r[light] + g[light]
          phase := (t + x[light] + (bigX%p)*k) % p
          if phase < r[light] {
            stop[mi]++
            blocked = true
            break
          }
        }
        if !blocked {
          surv++
        }
      }
      for mi := range stop {
        stop[mi] /= float64(Q)
      }
      live := float64(surv) / float64(Q)
      // prefix survival: fraction where the first m members are
      // all green = 1 - sum(stop[0..m-1])
      prefix := make([]float64, len(members)+1)
      prefix[0] = 1
      acc := 0.0
      for mi := range members {
        acc += stop[mi]
        prefix[mi+1] = 1 - acc
      }
      results[gi] = gres{stop: stop, survive: live, prefLive: prefix}
```

listing 399: go: the group walk over phase k, prefix survival as one minus the running stop sum (source: icpc/samples-go/ch10/pk.go, lines 72-106)

```
    for (int t0 = 0; t0 < 2520; t0++) {
      double sv = 1;
      for (int g = 0; g < ngroups; g++) {
        for (int s = 0; s < gq[g]; s++) {
          long fr = 1 << 30;
          for (int k = 0; k < nmem[g]; k++) {
            int i = members[g][k];
            long per = RR[i] + GG[i];
            long q1 = per / gcdll(per, 2520);
            long phase = (t0 + X[i] + 2520 % per * (s % q1)) % per;
            if (phase < RR[i]) {
              fr = X[i];
              break;
            }
          }
          firstred[g][s] = fr;
        }
        for (int k = 0; k ≤ nmem[g]; k++)
          greencnt[g][k] = 0;
        for (int s = 0; s < gq[g]; s++) {
          int k = 0;
          while (k < nmem[g] && X[members[g][k]] < firstred[g][s])
            k++;
          greencnt[g][k]++; // first red sits at member k (k = nmem: none)
        }
        greenfrom[g][nmem[g]] = greencnt[g][nmem[g]];
        for (int k = nmem[g] - 1; k ≥ 0; k--)
          greenfrom[g][k] = greencnt[g][k] + greenfrom[g][k + 1];
        sv *= (double) greenfrom[g][nmem[g]] / gq[g];
      }
      psurv += sv / 2520;
      for (int g = 0; g < ngroups; g++) {
        for (int k = 0; k < nmem[g]; k++) {
          int i = members[g][k];
          int cnt = 0;
          for (int s = 0; s < gq[g]; s++)
            if (firstred[g][s] == X[i])
              cnt++;
          if (cnt == 0)
            continue;
          double pr = (double) cnt / gq[g];
          for (int g2 = 0; g2 < ngroups && pr > 0; g2++) {
            if (g2 == g)
              continue;
            int k2 = 0;
            while (k2 < nmem[g2] && X[members[g2][k2]] < X[i])
              k2++;
            pr *= (double) greenfrom[g2][k2] / gq[g2];
          }
          pstop[i] += pr / 2520;
        }
```

listing 400: java: per residue, each group's first red per phase, green suffix counts, then the per-light attribution across groups (source: icpc/samples-java/src/Ch11/PK.java, lines 115-165)

```
        for (long t = 0; t < X; t++)
        {
            for (int gi = 0; gi < gc; gi++)
            {
                long q = gq[gi];
                var mem = members[gi];
                foreach (int light in mem)
                    stop[light] = 0;
                long surv = 0;
                for (long k = 0; k < q; k++)
                {
                    bool blocked = false;
                    foreach (int light in mem)
                    {
                        long phase = (t + x[light]
                            + (X % per[light]) * k) % per[light];
                        if (phase < red[light])
                        {
                            stop[light]++;
                            blocked = true;
                            break;
                        }
                    }
                    if (!blocked)
                        surv++;
                }
```

listing 401: c#: the per-group phase walk stopping at the first red, survival counted, stop fractions divided out (source: icpc/samples/src/Ch10/PK.cs, lines 90-115)

```
  for (let t = 0; t < bigX; t++) {
    // per group: first-red fractions per member, prefix green
    // fractions, and the survival fraction
    const stop = [];
    const prefLive = [];
    const survive = [];
    for (const grp of groups) {
      const Q = grp.q;
      const st = new Array(grp.member.length).fill(0);
      let surv = 0;
      for (let k = 0; k < Q; k++) {
        let blocked = false;
        for (let mi = 0; mi < grp.member.length; mi++) {
          const light = grp.member[mi];
          const p = r[light] + g[light];
          const phase = (t + x[light] + (bigX % p) * k) % p;
          if (phase < r[light]) {
            st[mi]++;
            blocked = true;
            break;
          }
        }
        if (!blocked) surv++;
      }
      for (let mi = 0; mi < st.length; mi++) st[mi] /= Q;
      const pre = [1];
      let acc = 0;
      for (let mi = 0; mi < grp.member.length; mi++) {
        acc += st[mi];
        pre.push(1 - acc);
      }
      stop.push(st);
      prefLive.push(pre);
      survive.push(surv / Q);
    }
```

listing 402: javascript: the same walk, per-group stop fractions and prefix-green arrays, survival per residue (source: icpc/samples-js/src/ch10-pk-trafficblights.mjs, lines 69-103)

```
    for t in range(BIGX):
        results = []
        for modulus, members in groups:
            qmod = modulus
            stops = [0] * len(members)
            seen = 0
            for mi, light in enumerate(members):
                row, rep = masks[light]
                m = row[(t + xs[light]) % (rs[light] + gs[light])] * rep
                stops[mi] = (m & ~seen).bit_count()
                seen |= m
            survive = qmod - seen.bit_count()
            stop_frac = [c / qmod for c in stops]
            prefix = [1.0]
            acc = 0.0
            for s in stop_frac:
                acc += s
                prefix.append(1.0 - acc)
            results.append((stop_frac, prefix, survive / qmod))
        for i in range(n):
            gi = group_of[i]
            prob = results[gi][0][pos_in_group[i]]
            for gj, res in enumerate(results):
                if gj != gi:
                    cut = bisect_left(pos_sorted[gj], xs[i])
                    prob *= res[1][cut]
            stop_prob[i] += prob / BIGX
        sp = 1.0
        for res in results:
            sp *= res[2]
        survive_prob += sp / BIGX
```

listing 403: python: red phases precomputed as bit masks per arrival base, the walk popcount arithmetic (source: icpc/samples-py/src/Ch10/pk.py, lines 69-99)

```
  local stop_prob, survive = {}, 0.0
  for i = 1, n do stop_prob[i] = 0.0 end
  for t = 0, bigx - 1 do
    local stops, preflive, survives = {}, {}, {}
    for gi = 1, #gq do
      local Q = gq[gi]
      local members = gmembers[gi]
      local stop = {}
      local surv = 0
      for k = 0, Q - 1 do
        local blocked = false
        for mi = 1, #members do
          local light = members[mi]
          local p = r[light] + g[light]
          local phase = (t + x[light] + (bigx % p) * k) % p
          if phase < r[light] then
            stop[mi] = (stop[mi] or 0) + 1
            blocked = true
            break
          end
        end
        if not blocked then surv = surv + 1 end
      end
      for mi = 1, #members do stop[mi] = (stop[mi] or 0) / Q end
      local prefix = { 1.0 }
      local acc = 0.0
      for mi = 1, #members do
        acc = acc + stop[mi]
        prefix[mi + 1] = 1.0 - acc
      end
      stops[gi] = stop
      preflive[gi] = prefix
      survives[gi] = surv / Q
    end
    for i = 1, n do
      local gi = group_of[i]
      local prob = stops[gi][member_pos[i]]
      for gj = 1, #gq do
        if gj ~= gi then
          -- other groups must be all green strictly before x[i]
          local cnt = 0
          for _, light in ipairs(gmembers[gj]) do
            if x[light] < x[i] then cnt = cnt + 1 end
          end
          prob = prob * preflive[gj][cnt + 1]
        end
      end
      stop_prob[i] = stop_prob[i] + prob / bigx
    end
```

listing 404: lua: the residue loop over groups, phases walked in position order, fractions in doubles (source: icpc/samples-lua/ch10_pk.lua, lines 55-103)

Probabilities live in 0 to 1 as doubles, printed at twelve decimals, and no integer edge is near. Java renders through `String.format(Locale.ROOT, %.12f)` so the twelve decimals never localize, and its dyadic half fractions round to the same digits the c `printf` prints at this width. The vehicles differ only in the inner walk: python precomputes each light's red phases over its reduced period as a bit mask indexed by the arrival base, so the per-residue walk counts bits, while the other six evaluate the phase per k directly.

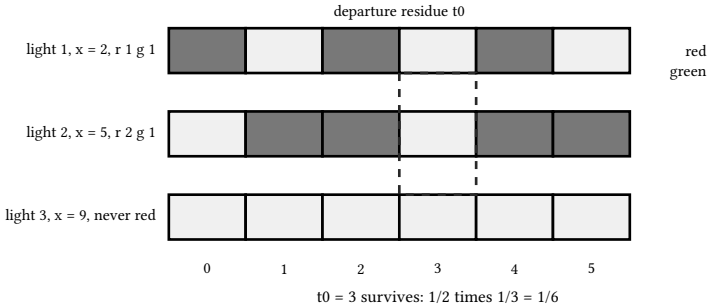


figure 142: the fixture's three lights as timeline strips over the departure residues 0 to 5, red and green bands offset by position, the surviving phase shaded on the last strip

The crafted fixture is three lights, at 2 with r 1 g 1, at 5 with r 2 g 1, and at 9 never red. The first is red on half the residues, the second on two thirds of those where the first is green, the never-red light stops nothing, and the probability the car never stops is the product one half times one third, so the pinned output is 0.500000000000, 0.333333333333, 0.000000000000, and 0.166666666667. The C suite pins the fixture and the crafted single light 3 4 6 answering 0.4 then 0.6 in two `CHECK` compares, the C# facts run the fixture, the always-red 0 3 0 light, and the alternating pair 0 1 1 at 0 with 3 1 1, Go's table carries the fixture, the javascript `it` blocks alone take the fixture and both samples at this book's twelve-decimal rendering, and python's and lua's three checks stay on crafted families, an always-red or never-red single light and a two-light split, java's four staying crafted too, the fixture, an always-red light, an alternating pair, and a same-prime pair that stops together, every suite comparing twelve-decimal strings because the judge answers mix statement and machine formats, java's the one exception, parsing the printed doubles and comparing within 1e-9.

12.13. across the seven languages

The 2019 set put the seven languages under the same eleven problems and the trees answered with the same pinned algorithms in different vehicles, and the differences are the table below. Two problems split the languages outright. Problem F's run treap rebuilds on a fixed 4096-tarp cadence in c, C#, go, java, and javascript and on a depth gate of 96 in python and lua, and problem I's interpreter machine runs over an explicit resume stack in c, java, javascript, and python while go, C#, and lua recurse with a seen-set per until loop. Everywhere else the split is a representation choice, dense or sparse type tables in D, dense or memoized automata in G, recomputing or incremental pair walks in J, and the integer edges never moved past the B surds' float64 compares, the J divisions' int64 bounds, javascript's bigint rank keys on A, and java's single BigInteger site on F's below-at cross products.

problem	c	go	java	c#	javascript	python	lua
oset	bucket stacks	uint64 keys	bucket stacks	SortedSet buckets	bigint keys	composite ranks	int64 keys
A	stack ties	key order	stack ties	set ties	key order	key ranks	key ranks
B	int64, double surds	surd helpers	long, double surds	same	Number under 2^53	native ints	inf 1 << 62
C	piece ids	map clone fork	State clone fork	char boards, P repair	Map fork	dict fork	table fork
D	dense arrays	int32 arrays	dense arrays	dictionaries	typed arrays	lists off max id	hash tables
E	xor adjacency	half-edge lists	half-edge xor	csr adjacency	typed arrays	lists	arrays
F	pending lists, 4096	dag sweep, 4096	RTreap class, 4096	split files, 4096	dag sweep, 4096	block dag, gate 96	dag sweep, gate 96
G	dense goto	fail walk, then fill	dense goto	dense goto	dense goto	dense goto	memo fallback
H	doubled ring buffer	per-ring diff	doubled ring buffer	by-depth lists	per-ring diff	per-ring diff	per-ring diff

problem	c	go	java	c#	javascript	python	lua
I	resume stack	recursion, seen set	resume stack	recursion, seen set	frame stack	resume stack	recursion, seen set
J	recompute totals	recompute totals	recompute, packed events	interval yields	prefix table	running sums	packed events
K	direct phases	direct	direct, Locale.ROOT	direct	direct	bitmask pop-count	direct
suite	37 checks	19 cases	56 checks	42 facts	30 its	42 checks	42 checks

The suite shapes themselves differ by language, and the sources paragraph counts what each tree actually carries: c scatters its checks through per-problem `CHECK` macros, C# keeps one test class per letter, go folds every fixture into a single table-driven test, javascript writes one describe block per letter, python embeds a self-check in each solver file, java embeds an ok-line self-check in each solver beside the judge-mode file read, one dual-mode main per letter, and lua returns named test tables that the runner executes under pcall.

sources: ICPC Foundation, icpc.global, the problemset pdf icpc2019.pdf and the solution sketches finals2019solutions.pdf, both accessed 2026-09-14 and cached under `ref/icpc/2019/` with sha256 digests in `ref/icpc/INDEX.md`. The solutions pdf is the algorithm reference for every method and complexity claim in this chapter, and the contest account in the opening paragraph is its own. The committed suites pin the fixtures above, counted from the frozen trees: 37 `CHECK` invocations across the 11 c solvers, 42 `[Fact]` methods across the 11 c# test files, one table-driven go test carrying 19 cases, 30 `it` blocks across the 11 javascript test files, 42 check calls across the 11 python solvers, and 42 assertions across the 11 lua modules, 212 checks across the six sibling trees, each counted in its own suite's unit, plus the python oset helper's own six-check self-check. The java tree lives under `samples-java/src/Ch11`, the one tree that follows the post-renumber chapter numbers while the six siblings keep their historical `Ch08` through `Ch13` directories: 13 files, the 11 solvers plus the `Oset` and `RTreap` year helpers, 2125 solver SLOC, its 56 ok-line checks run under `tools/run-java-samples.ps1`, and its judge lane answers all 583 cases through the same verify script. Judge data under `ref/icpc/2019/data/` is local only and never committed: 583 input files mapped to answer pairs by the committed `ref/icpc/verify-data.ps1`, and every sample pair reprinted in this chapter is the byte-exact content of that problem's sample-N.in and sample-N.ans files, attributed in place, with A and C accepted on any valid answer since those letters run on checkers and K compared within its 1e-6 float tolerance.

13. icpc world finals 2022

The 46th world finals were played in Luxor in 2023, and the foundation published the set as 11 problems lettered P through Z, the lettering cycle chapter 1 describes. The problemset is icpc2022.pdf, and every algorithm and complexity claim in this chapter is settled against the official solutions pdf rather than reinvented. The solutions pdf also carries the contest statistics: Z was solved by 0 teams and was the judges' pick as the hardest problem, S by exactly 1 team 276 minutes in, X by 8, R by 15, V by 24, Q by 72, T by 79, U by 92, P by 115, W by 116 teams with the first solve at 8 minutes, and Y by 124 with the first solve at 6. This chapter walks all 11 problems in the book's fixed order, one section per letter, seven listings per section, c through lua, every listing sliced from a solver file that ran against the official judge data held locally under `ref/icpc/`.

13.1. the year-local helpers: fenwick counting, planar predicates, io cursors, and the sweep engine

Four helper kits are year-local to this chapter, and everything else comes from the wave 1 toolboxes the year chapters import. Problem Q counts untrucked container values with a 1-based fenwick tree over the value range, point add plus prefix sum, queried as suffix counts. The seven implementations agree on the interface and differ in how the tree starts life: the C header and the js class fill it with n point adds, C#, go, and java allocate an empty tree, java's over a plain `long[]`, and the python and lua versions are born full in linear time, `tree[i] = lowbit(i)`, because a tree of ones has a closed form. The solver then only pays for removals.

```c
static inline void fw_add(long long *tree, long long n, long long i,
                          long long delta) {
  for (; i ≤ n; i += i & -i)
    tree[i] += delta;
}

static inline long long fw_prefix(const long long *tree, long long i) {
  long long s = 0;
  for (; i > 0; i -= i & -i)
    s += tree[i];
  return s;
}
```

listing 405: c, the year-local fenwick: point add and prefix sum over 1..n (source: icpc/samples-c/src/Ch11/ch11_fenwick.h, lines 6-17)

```
type fenwick11 struct {
  tree []int
}

func newFenwick11(n int) *fenwick11 {
  return &fenwick11{tree: make([]int, n+1)}
}

func (f *fenwick11) add(i, delta int) {
  for ; i < len(f.tree); i += i & -i {
    f.tree[i] += delta
  }
}

func (f *fenwick11) prefix(i int) int {
  sum := 0
  for ; i > 0; i -= i & -i {
    sum += f.tree[i]
  }
  return sum
}
```

listing 406: go, fenwick11 as a plain struct with int tree slots (source: icpc/samples-go/ch11/ch11_fenwick.go, lines 6-26)

```
class Fenwick {
  private final long[] tree;
  private final int n;

  Fenwick(int n) {
    this.n = n;
    tree = new long[n + 1];
  }

  void add(int i, long delta) {
    for (; i ≤ n; i += i & -i)
      tree[i] += delta;
  }

  long prefix(int i) {
    long s = 0;
    for (; i > 0; i -= i & -i)
      s += tree[i];
    return s;
  }
}
```

listing 407: java, the class over a long[] tree, add and prefix loops (source: icpc/samples-java/src/Ch12/Fenwick.java, lines 4-24)

```
public sealed class Fenwick
{
    private readonly long[] _tree;

    public Fenwick(int n) => _tree = new long[n + 1];

    public void Add(int i, long delta)
    {
        for (; i < _tree.Length; i += i & -i)
            _tree[i] += delta;
    }

    public long Prefix(int i)
    {
        long s = 0;
        for (; i > 0; i -= i & -i)
            s += _tree[i];
        return s;
    }
}
```

listing 408: c#, the same two loops wrapped in the year-local Fenwick class (source: icpc/samples/src/Ch11/Fenwick.cs, lines 7-26)

```
export class Fenwick {
  constructor(n) {
    this.tree = new Int32Array(n + 1);
    this.n = n;
  }

  // add delta at 1-based value i
  add(i, delta) {
    for (; i ≤ this.n; i += i & -i) this.tree[i] += delta;
  }

  // count of added values in [1, i]
  prefix(i) {
    let sum = 0;
    for (; i > 0; i -= i & -i) sum += this.tree[i];
    return sum;
  }
}
```

listing 409: javascript, the class with an Int32Array tree, add and prefix (source: icpc/samples-js/src/ch11-fenwick.mjs, lines 4-21)

```python
class Fenwick:
    def __init__(self, n):
        self.n = n
        self.tree = [i & -i for i in range(n + 1)]

    def add(self, i, d):
        tree = self.tree
        n = self.n
        while i ≤ n:
            tree[i] += d
            i += i & -i

    def sum(self, i):
        tree = self.tree
        s = 0
        while i > 0:
            s += tree[i]
            i -= i & -i
        return s
```

listing 410: python, born full via tree[i] = i and -i, so the solver only removes (source: icpc/samples-py/src/Ch11/fenwick.py, lines 5-23)

```lua
function fen.new_ones(n)
  local t = {}
  for i = 1, n do t[i] = i & (-i) end
  return { n = n, tree = t }
end

function fen.add(f, i, delta)
  local t, n = f.tree, f.n
  while i ≤ n do
    t[i] = t[i] + delta
    i = i + (i & -i)
  end
end

function fen.prefix(f, i)
  local t, sum = f.tree, 0
  while i > 0 do
    sum = sum + t[i]
    i = i - (i & -i)
  end
  return sum
end

-- untrucked values strictly above m, given `left` values remain overall
function fen.above(f, m, left)
  return left - f:prefix(m)
end
```

listing 411: lua, new_ones builds the full tree in linear time, above wraps the suffix count (source: icpc/samples-lua/ch11_fenwick.lua, lines 6-32)

Problem T needs three planar predicates, the 2d cross product, closed-segment intersection with endpoint tolerance, and strict containment in a CCW convex polygon. C, go, and lua carry them as a year-local module beside the solver; C#, javascript, and python embed the same three predicates directly in the solver file, so their listings land in section T rather than here. Java ports no geo twin at all: only the cross product has a caller, and it lives inline in the T solver.

```c
static inline double cross2d(GPoint o, GPoint a, GPoint b) {
  return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

// closed-segment intersection with a tolerance on the cross products;
// touching endpoints counts
static inline int seg_intersect(GPoint p1, GPoint p2, GPoint p3, GPoint p4,
                                double eps) {
  double d1 = cross2d(p3, p4, p1);
  double d2 = cross2d(p3, p4, p2);
  double d3 = cross2d(p1, p2, p3);
  double d4 = cross2d(p1, p2, p4);
  if (((d1 > eps && d2 < -eps) || (d1 < -eps && d2 > eps)) &&
      ((d3 > eps && d4 < -eps) || (d3 < -eps && d4 > eps)))
    return 1;
  // collinear / touching cases: accept when a midpoint sits on the
  // other closed segment
  GPoint mid = {(p1.x + p2.x) / 2, (p1.y + p2.y) / 2};
  GPoint mid2 = {(p3.x + p4.x) / 2, (p3.y + p4.y) / 2};
  if (-eps ≤ d1 && d1 ≤ eps && -eps ≤ d2 && d2 ≤ eps) {
    double t = cross2d(p3, p4, mid);
    if (-eps ≤ t && t ≤ eps)
      return 1; // p-segment collinear with the p3p4 line
  }
  if (-eps ≤ d3 && d3 ≤ eps && -eps ≤ d4 && d4 ≤ eps) {
    double t = cross2d(p1, p2, mid2);
    if (-eps ≤ t && t ≤ eps)
      return 1;
  }
  return 0;
}

// strict containment in a CCW convex polygon (≥ 3 verts); boundary
// points with tolerance are not inside
static inline int point_in_convex(GPoint p, const GPoint *poly, int vn,
                                  double eps) {
  for (int i = 0; i < vn; i++) {
    double c = cross2d(poly[i], poly[(i + 1) % vn], p);
    if (c ≤ eps)
      return 0;
  }
  return 1;
}
```

listing 412: c, the geo kit: cross, closed-segment intersection with tolerance, strict convex containment (source: icpc/samples-c/src/Ch11/ch11_geo.h, lines 11-53)

```
// geoCross is the z of the cross product (a-o) x (b-o).
func geoCross(o, a, b geoVec) float64 {
  return (a.x-o.x)*(b.y-o.y) - (a.y-o.y)*(b.x-o.x)
}

// geoSegHit reports whether the closed segments p1-p2 and q1-q2 meet;
// on success it returns the meeting point and its parameter along
// p1-p2. Parallel overlaps are rejected: callers treat them as
// degenerate.
func geoSegHit(p1, p2, q1, q2 geoVec) (bool, geoVec, float64) {
  d1 := geoSub(p2, p1)
  d2 := geoSub(q2, q1)
  den := d1.x*d2.y - d1.y*d2.x
  w := geoSub(q1, p1)
  scale := geoLen(d1) * geoLen(d2)
  if scale == 0 || den > -1e-12*scale && den < 1e-12*scale {
    return false, geoVec{}, 0
  }
  t := (w.x*d2.y - w.y*d2.x) / den
  u := (w.x*d1.y - w.y*d1.x) / den
  tol := 1e-9
  if t < -tol || t > 1+tol || u < -tol || u > 1+tol {
    return false, geoVec{}, 0
  }
  if t < 0 {
    t = 0
  }
  if t > 1 {
    t = 1
  }
  return true, geoAdd(p1, geoMul(d1, t)), t
}

// geoInside reports whether p lies strictly inside the CCW convex
// polygon given by its vertices, with a relative tolerance so
// boundary points (segments hugging an edge) never count as inside.
func geoInside(p geoVec, poly []geoVec) bool {
  for i := range poly {
    a, b := poly[i], poly[(i+1)%len(poly)]
    c := geoCross(a, b, p)
    scale := geoLen(geoSub(b, a)) * geoDist(a, p)
    if scale == 0 || c <= 1e-9*scale {
      return false
    }
  }
  return true
}
```

listing 413: go, the same kit with relative tolerances and a parameterized meeting point (source: icpc/samples-go/ch11/ch11_geo.go, lines 22-68)

```lua
function geo.cross2d(ox, oy, ax, ay, bx, by)
  return (ax - ox) * (by - oy) - (ay - oy) * (bx - ox)
end

-- closed-segment intersection; touching endpoints count
function geo.seg_intersect(x1, y1, x2, y2, x3, y3, x4, y4, eps)
  local d1 = geo.cross2d(x3, y3, x4, y4, x1, y1)
  local d2 = geo.cross2d(x3, y3, x4, y4, x2, y2)
  local d3 = geo.cross2d(x1, y1, x2, y2, x3, y3)
  local d4 = geo.cross2d(x1, y1, x2, y2, x4, y4)
  if ((d1 > eps and d2 < -eps) or (d1 < -eps and d2 > eps)) and
     ((d3 > eps and d4 < -eps) or (d3 < -eps and d4 > eps)) then
    return true
  end
  local mx, my = (x1 + x2) / 2, (y1 + y2) / 2
  local nx, ny = (x3 + x4) / 2, (y3 + y4) / 2
  if -eps ≤ d1 and d1 ≤ eps and -eps ≤ d2 and d2 ≤ eps then
    local t = geo.cross2d(x3, y3, x4, y4, mx, my)
    if -eps ≤ t and t ≤ eps then return true end
  end
  if -eps ≤ d3 and d3 ≤ eps and -eps ≤ d4 and d4 ≤ eps then
    local t = geo.cross2d(x1, y1, x2, y2, nx, ny)
    if -eps ≤ t and t ≤ eps then return true end
  end
  return false
end

-- strict containment in a ccw convex polygon (xs/ys 1-based, length n);
-- boundary points within eps are not inside
function geo.point_in_convex(px, py, xs, ys, n, eps)
  for i = 1, n do
    local j = i % n + 1
    local c = geo.cross2d(xs[i], ys[i], xs[j], ys[j], px, py)
    if c ≤ eps then return false end
  end
  return true
end
```

listing 414: lua, flat coordinate parameters instead of tables, same three predicates (source: icpc/samples-lua/ch11_geo.lua, lines 6-42)

The scan kit is the year's io contract, and the only kit the year could not solve without: the 2022 secrets run to 10.7 MB for R, 6.7 MB for Q, 3.5 MB for P, and 1.4 MB for V, so every language reads the file in one piece before parsing. The contract is a whitespace cursor over the whole buffer, skip, a signed int64 reader, and one strtod double. C carries it as `Ch11/ch11_scan.h`, imported by ten of the year's eleven solvers, everything but the one-line Y, and it is the book's one helper that crosses years: chapter 14's solvers and its sphinx header and chapter 15's twelve solvers include it through `../Ch11/ch11_scan.h` paths, twenty-two importers beyond this chapter. Go keeps the same cursor once per package as the `scan11` struct inside the solver files, javascript, python, and C# split the whole-file string inline, and lua scans bytes inside each solver, so no second twin file exists. Java reads the whole file through a `BufferedReader` in 64K char chunks into one string and a static `nextLong` cursor walks it inside each solver, the same subsumption the 2019 chapter records.

```c
// ch11_scan.h: whitespace tokenizer over the io_read_file buffer for
// the multi-megabyte 2022-2025 inputs; a const cursor walks the buffer
#ifndef CH11_SCAN_H
#define CH11_SCAN_H

typedef struct {
  const char *p;
} Ch11Scan;

static inline void ch11_scan_init(Ch11Scan *s, const char *buf) { s->p = buf; }

static inline void ch11_scan_skip(Ch11Scan *s) {
  while (*s->p == ' ' || *s->p == '\n' || *s->p == '\r' || *s->p == '\t')
    s->p++;
}

static inline long long ch11_next_int64(Ch11Scan *s) {
  ch11_scan_skip(s);
  long long sign = 1;
  if (*s->p == '-') {
    sign = -1;
    s->p++;
  }
  long long v = 0;
  while (*s->p >= '0' && *s->p <= '9')
    v = v * 10 + (*s->p++ - '0');
  return sign * v;
}

static inline double ch11_next_double(Ch11Scan *s) {
  ch11_scan_skip(s);
  char *end;
```

listing 415: c, the scan cursor: skip whitespace, signed int64, strtod double, one const pointer walking the whole input buffer (source: icpc/samples-c/src/Ch11/ch11_scan.h, lines 1-32)

The sweep engine is problem S's second lua file, the diagonal (k, g, l) machine the solver requires as `ch11_psweep`, a port of the go judge lane `psfast.go`. States live on anti-diagonals k + l, and the slice below is the expansion core: one live diagonal read through its bitmask of g slots, the finish pace probed against the running best, and every pure-slow fan-out landed in a ring of c - 1 future buffers.

```
    for l = lhi, 0, -1 do
      local b = buf[l]
      if b.m0 ~= 0 or b.m1 ~= 0 then
        local K = s - l
        local rs = touch_row(K)
        local total = n - K - l
        local pace
        if total ≥ 1 then
          local q = 1
          if total ≥ 2 then q = (total + c - 3) // step end
          pace = (q - 1) * t0 + shuttle(K, q)
        end
        local nK = n - K
        local j = c < nK and c or nK
        local nl = l - 1
        local nlz = nl == 0
        local fbOK = nl ≥ 0
        local slow = t[nK - 1]
        local fbSlot = fbuf[(s + j - 1) % nfb]
        for g = 1, D do
          local bit = 1 << ((g - 1) & 63)
          local w = g > 64 and b.m1 or b.m0
          if w & bit ~= 0 then
            local d = b.d[g]
            if pace and d + pace < best then best = d + pace end
            if fbOK then
              local ng = nlz and 1 or g + 1
              if ng ≤ D and K + j + nl ≤ n then
                local v = d + slow + t[g]
                if j == 1 then
                  chm(buf, ng, nl, v)
                else
                  chm(fbSlot, ng, nl, v)
                end
              end
            end
          end
        end
      end
```

listing 416: lua, the sweep engine's diagonal expansion, pure-slow fan-outs landing in a ring of c-1 future buffers (source: icpc/samples-lua/ch11_psweep.lua, lines 250-287)

13.2. p, turning red

Mei's parents spent a year remodeling their house and left it with a complicated lighting system. Each room has one LED light showing red, green, or blue, and buttons around the house are each wired to a set of lights: pressing a button advances every light it controls one step around that cycle, red to green, green to blue, blue to red, and any button may be pressed as often as liked. The wiring predates crossbars, so each light answers to at most two buttons. Mei wants every light red and her parents want the buttons to survive, so the task is the smallest total number of presses that leaves every light red, or a verdict that all-red cannot be reached (statement). Turning Red is one of the five problem names this finals shares with the 2023 set: this set's P is 2023's G retold.

The input, per the problemset pdf, is one line of l and b, with 1 <= l <= 2e5 lights and 0 <= b <= 2l buttons, then a line of l characters over R, G, and B giving each light's starting color, then b button lines: a count k with 1 <= k <= l followed by the k distinct lights that button controls, each light appearing at most twice across all buttons. The output is the minimum press total, or the word impossible, under the pdf's 3 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `P-turningred/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: six buttons sort out eight lights, and the cheapest full redden costs 8 presses.

input:

```
8 6
GBRBRRRG
2 1 4
1 2
4 4 5 6 7
3 5 6 7
1 8
1 8
```

expected output:

```
8
```

Official sample 2, same source: the red lights 1 and 4 pin buttons 1 and 3 to zero presses, the green light 2 then forces two presses on button 2, and the blue light 3 wants two presses on button 3, which light 4 has already pinned, so no assignment exists.

input:

```
4 3
RGBR
2 1 2
2 2 3
2 3 4
```

expected output:

```
impossible
```

Recognition: the bounds make the propagation linear before any algebra starts. With l <= 2e5 lights and b <= 2l = 4e5 buttons under the 3 second limit, one BFS over the light-button graph visits about 6e5 cells, and the three root trials per component triple that to under 2e6 visits, two orders of magnitude inside budget, so the whole solve is a handful of linear passes.

The statement's cue is its arithmetic shape: every light answers to at most two buttons and each press advances a light one fixed step around the color cycle, so each light is one equation over at most two unknowns mod 3, the cue that turns the house wiring into an auxiliary graph where buttons sharing a light chain into components. Problem P is the 2022 face of the one free value per component family, and one free value per component is the trick behind chapter 14 problem G and chapter 15 problem K.

The tempting alternative, Gaussian elimination over all b = 4e5 unknowns, prices out at 4e5 cubed, 6.4e16 operations, and brute force over press vectors is 3^400000, so the per-component root trial is the only shape that closes.

With colors as residues mod 3, each light demands that its color plus its buttons' press counts sum to 0. A light on two buttons is an equation over two unknowns, a light on one button pins its button outright, and buttons sharing lights chain into connected components where fixing one root press count to 0, 1, or 2 determines everything else. Per component the solver tries the consistent root values and keeps the cheapest press total, linear in lights plus buttons, the model the solutions pdf sketches on its page 2, problem at icpc2022.pdf. Book 10′s chapter 34, linear algebra, develops the gaussian elimination that solves exactly this shape of small modular system, of which the component propagation below is the sparse special case. The seven walks below all propagate with a BFS over a

CSR adjacency, and differ in how they retry root values: the C, javascript, and lua versions stamp each trial so no visited cleanup is needed, go resets only the touched cells, C# re-walks a precomputed BFS order and re-validates every edge at the end, java stamps like C over a CSR adjacency, and python never retries at all, carrying each value as affine in the root, `t + s*r` with `s = +-1`, and solving for the allowed roots algebraically.

The worked run: trace the model on sample 1. Colors read as residues, R = 0, G = 1, B = 2, and each light demands that its color plus its buttons' presses sum to 0 mod 3. Light 2 is blue on button 2 alone, so x2 = 1, and light 3 is already red on no button, cost 0. The component of buttons 1, 3 and 4 is chained by lights 4, 5, 6 and 7: light 1, green on button 1, pins x1 = 2, light 4, blue on buttons 1 and 3, forces x3 = 1 - 2 = 2 mod 3, and lights 5, 6 and 7, red on buttons 3 and 4, force x4 = −2 = 1, a component cost of 2 + 2 + 1 = 5 with the root pinned rather than tried. Light 8, green on buttons 5 and 6, wants x5 + x6 = 2, cheapest as 1 + 1 = 2. The total is 5 + 1 + 2 = 8, and the trace ends at the printed answer `8`.

unit	forcing	presses	cost
button 2	light 2 blue on it alone, x2 = 1	1	1
buttons 1, 3, 4	light 1 pins x1 = 2, lights 4 to 7 propagate x3, x4	2, 2, 1	5
buttons 5, 6	light 8 green, x5 + x6 = 2	1, 1	2
light 3	red, wired to no button	none	0

```c
// propagate press counts from root value r; the run carries its own
// stamp so no visited cleanup is needed between root trials. returns
// the press total (queue[0..*touched) lists the component), or -1 when
// an edge sum or a pin is violated.
static long long try_component(int root, int r, int *touched) {
  stamp++;
  int qh = 0, qt = 0;
  long long total = 0;
  xsol[root] = r;
  mark[root] = stamp;
  queue[qt++] = root;
  if (pin[root] >= 0 && pin[root] != r)
    return -1;
  total += r;
  while (qh < qt) {
    int u = queue[qh++];
    for (int e = xadj[u]; e < xadj[u + 1]; e++) {
      int v = eadj_to[e], k = eadj_k[e];
      int want = ((k - xsol[u]) % 3 + 3) % 3;
      if (mark[v] == stamp) {
        if (xsol[v] != want)
          return -1;
        continue;
      }
      if (pin[v] >= 0 && pin[v] != want)
        return -1;
      xsol[v] = want;
      mark[v] = stamp;
      total += want;
      queue[qt++] = v;
    }
  }
  *touched = qt;
  return total;
}
```

listing 417: c, try_component: stamp-carried BFS, press total or −1 on a violated edge or pin (source: icpc/samples-c/src/Ch11/pP.c, lines 32-66)

```
// propagate assigns values from root downward, returning whether
// every constraint holds and the component's press total; the
// caller resets the touched assignments afterwards.
propagate := func(root, rv int) (bool, int64) {
  ok := true
  sum := int64(0)
  queue = append(queue, root)
  val[root] = rv
  touched = append(touched, root)
  for head := 0; head < len(queue) && ok; head++ {
    u := queue[head]
    sum += int64(val[u])
    for _, e := range adj[u] {
      if val[e.to] == -1 {
        nv := (e.rhs - val[u] + 3) % 3
        if pin[e.to] ≥ 0 && pin[e.to] ≠ nv {
          ok = false
          break
        }
        val[e.to] = nv
        queue = append(queue, e.to)
        touched = append(touched, e.to)
      } else if (val[u]+val[e.to])%3 ≠ e.rhs {
        ok = false
        break
      }
    }
  }
  queue = queue[:0]
  for _, u := range touched {
    val[u] = -1
  }
  touched = touched[:0]
  return ok, sum
}
```

listing 418: go, the propagate closure: BFS values from the root, resets only the touched cells (source: icpc/samples-go/ch11/pp.go, lines 75-109)

```
// propagate press counts from root value r; the run carries its own
// stamp so no visited cleanup is needed between root trials. returns
// the press total (queue[0..touched) lists the component), or -1 when
// an edge sum or a pin is violated.
static long tryComponent(int root, int r, int[] touched) {
  stamp++;
  int qh = 0, qt = 0;
  long total = 0;
  xsol[root] = r;
  mark[root] = stamp;
  queue[qt++] = root;
  if (pin[root] ≥ 0 && pin[root] ≠ r)
    return -1;
  total += r;
  while (qh < qt) {
    int u = queue[qh++];
    for (int e = xadj[u]; e < xadj[u + 1]; e++) {
      int v = eadjTo[e], k = eadjK[e];
      int want = ((k - xsol[u]) % 3 + 3) % 3;
      if (mark[v] == stamp) {
        if (xsol[v] ≠ want)
          return -1;
        continue;
      }
      if (pin[v] ≥ 0 && pin[v] ≠ want)
        return -1;
      xsol[v] = want;
      mark[v] = stamp;
      total += want;
      queue[qt++] = v;
    }
  }
  touched[0] = qt;
  return total;
}
```

listing 419: java, tryComponent: stamp-carried BFS over the CSR adjacency, press total or −1 on a violated edge or pin (source: icpc/samples-java/src/Ch12/PP.java, lines 48-82)

```
            long best = long.MaxValue;
            for (int t = 0; t < 3; t++)
            {
                if (pin[r] ≥ 0 && pin[r] ≠ t)
                    continue;
                for (int qi = 0; qi < tail; qi++)
                    val[queue[qi]] = -1;
                val[r] = t;
                bool ok = true;
                long cost = t;
                for (int qi = 1; qi < tail && ok; qi++)
                {
                    int u = queue[qi];
                    val[u] = -1;
                    // value forced by any already-valued neighbor (BFS order
                    // guarantees the tree parent qualifies)
                    for (int e = start[u]; e < start[u + 1] && val[u] < 0; e++)
                        if (val[adjTo[e]] ≥ 0)
                            val[u] = (adjSum[e] - val[adjTo[e]] + 3) % 3;
                    if (val[u] < 0)
                        ok = false;
                    else
                        cost += val[u];
                }
                for (int qi = 0; qi < tail && ok; qi++)
                {
                    int u = queue[qi];
                    if (pin[u] ≥ 0 && pin[u] ≠ val[u])
                        ok = false;
                    for (int e = start[u]; e < start[u + 1] && ok; e++)
                        if ((val[u] + val[adjTo[e]]) % 3 ≠ adjSum[e])
                            ok = false;
                }
                if (ok && cost < best)
                    best = cost;
            }
```

listing 420: c#, per root value t: assign in BFS order from any valued neighbor, then re-check every edge and pin (source: icpc/samples/src/Ch11/PP.cs, lines 102-137)

```
const tryComponent = (root, r) => {
  stamp++;
  let qh = 0, qt = 1, total = r;
  xsol[root] = r;
  mark[root] = stamp;
  queue[0] = root;
  if (pin[root] ≥ 0 && pin[root] =/= r) return -1;
  while (qh < qt) {
    const u = queue[qh++];
    for (let e = xadj[u]; e < xadj[u + 1]; e++) {
      const v = adjTo[e];
      const want = ((adjK[e] - xsol[u]) % 3 + 3) % 3;
      if (mark[v] === stamp) {
        if (xsol[v] =/= want) return -1;
        continue;
      }
      if (pin[v] ≥ 0 && pin[v] =/= want) return -1;
      xsol[v] = want;
      mark[v] = stamp;
      total += want;
      queue[qt++] = v;
    }
  }
  touched = qt;
  return total;
};
```

listing 421: javascript, tryComponent over Int32Arrays, the same stamp trick as C (source: icpc/samples-js/src/ch11-pp-turningred.mjs, lines 82-107)

```
    for root in range(b):
        if seen[root]:
            continue
        seen[root] = 1
        tval[root] = 0
        sval[root] = 1
        comp = [root]
        qi = 0
        while qi < len(comp):
            u = comp[qi]
            qi += 1
            tu = tval[u]
            su = sval[u]
            for e in range(ehead[u], ehead[u + 1]):
                v = enbr[e]
                if not seen[v]:
                    seen[v] = 1
                    tval[v] = (ewt[e] - tu) % 3
                    sval[v] = 3 - su
                    comp.append(v)
        allowed = -1
        for u in comp:
            tu = tval[u]
            su = sval[u]
            for e in range(ehead[u], ehead[u + 1]):
                a = su + sval[enbr[e]]
                need = (ewt[e] - tu - tval[enbr[e]]) % 3
                if a % 3 == 0:
                    if need:
                        return "impossible\n"
                else:
                    r = need * a % 3
                    if allowed < 0:
                        allowed = r
                    elif allowed != r:
                        return "impossible\n"
            p = pin[u]
            if p >= 0:
                r = (p - tu) * su % 3
                if allowed < 0:
                    allowed = r
                elif allowed != r:
                    return "impossible\n"
        cnt = [0] * 9
        for u in comp:
            cnt[tval[u] * 3 + sval[u]] += 1
        rs = (0, 1, 2) if allowed < 0 else (allowed,)
        total += min(
            (sum(cnt[i] * _COST[i][r] for i in range(9) if cnt[i]) for r in rs),
        )
```

listing 422: python, one BFS per component collects t and s = +-1 per button, then non-tree edges and pins pin the root value algebraically (source: icpc/samples-py/src/Ch11/pp.py, lines 85-134)

```
-- propagate press counts from root value r; each trial carries its own
-- stamp so no cleanup is needed. returns the press total and the
-- component size (queue[1..touched]), or -1 on any violation.
local function try_component(root, r)
  stamp = stamp + 1
  xsol[root] = r
  mark[root] = stamp
  queue[1] = root
  local total = r
  local qh, qt = 1, 1
  if pin[root] and pin[root] ~= r then return -1 end
  while qh ≤ qt do
    local u = queue[qh]
    qh = qh + 1
    for e = xadj[u], xadj[u + 1] - 1 do
      local v = at[e]
      local want = (ak[e] - xsol[u]) % 3
      if mark[v] == stamp then
        if xsol[v] ~= want then return -1 end
      else
        if pin[v] and pin[v] ~= want then return -1 end
        xsol[v] = want
        mark[v] = stamp
        total = total + want
        qt = qt + 1
        queue[qt] = v
      end
    end
  end
  touched = qt
  return total
end
```

listing 423: lua, try_component with per-trial stamps, one-based queue (source: icpc/samples-lua/ch11_pp.lua, lines 110-141)

The pinned fixture is 6 lights, 4 buttons, colors RRGBRR: buttons 1 through 3 form a triangle through lights 1, 2, 3 whose only consistent root assignment costs 4, and the blue light 4 sits alone on button 4, adding one press, for a total of 5. Every suite asserts `solve` of that exact input equals 5, and the family around it holds the four official samples, 8, the impossible one, 6, and 3: the C, python, and lua suites count 5 checks each, the C# suite 5 facts, go 5 table rows, js 5 blocks, and java's 8 checks take the fixture, the whole sample family, and three crafted boards, an uncontrolled light, four solo greens, and the green-blue cost split. Press totals stay under 4e5, so no language needs anything beyond its native integers here, and none of the seven takes any special care.

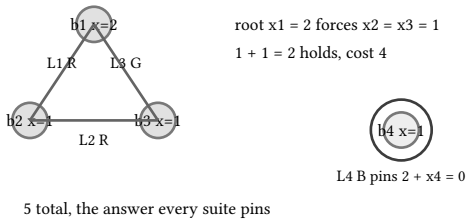


figure 143: the fixture: a button triangle with one consistent assignment, plus the single-button light that pins its own press

13.3. q, doing the container shuffle

A cargo ship unloads containers 1..n in ship order, and each container is dropped onto one of two stacks with probability 1/2, independently, the only randomness in the problem. Trucks then collect the containers in a given pickup order. Loading a requested container costs nothing extra only when it tops its stack. Otherwise the

containers above it shuffle to the other stack one at a time until it tops, and then it moves to the truck. Count every move except the ship's own unloading: each stack-to-stack shuffle and each load onto a truck is one move, and the task is the expected total over the random placements (statement).

The input, per the problemset pdf, is one integer n with 1 <= n <= 1e6, then the pickup order as a permutation of 1..n on the second line. The output is the expected number of moves with an absolute error of at most 1e-3, under the pdf's 2 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `Q-doingthecontainershuffle/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: five containers picked as 4, 2, 5, 3, 1 cost 7.000 moves in expectation, the 5 loads plus an expected 2 shuffles.

input:

```
5
4 2 5 3 1
```

expected output:

```
7.000
```

Official sample 2, same source: even the identity pickup pays, since the random placement interleaves the two stacks and the interval sweeps still cross untrucked containers, 15 of them summed over the six pickups for 7.5 expected shuffles on top of the 6 loads.

input:

```
6
1 2 3 4 5 6
```

expected output:

```
13.500
```

Recognition: n <= 1e6 under the 2 second limit prices one logarithmic pass and nothing more: a fenwick walk over the value range costs about 20 tree hops per container, 2e7 hops in total, while the placement randomness must never be expanded.

The statement's cue is the word expected over every random placement: the 2^1000000 stack assignments stay implicit, so the answer has to come from linearity, each container's shuffles counted separately, the cue that asks for a membership count instead of a simulation. Problem Q is the 2022 face of the counting and expectation without enumeration family, and counting and expectation without enumeration is the shared core of chapter 11 problem D, chapter 11 problem I, chapter 14 problem E, and chapter 13 problem K.

Simulating placements prices out at 2^n, and any dp over joint stack orders inherits the same exponent, so the half-counted interval membership is the only budget that closes.

The joint order, stack 1 bottom to top then stack 2 top to bottom, never changes under shuffles, only loses trucked containers, so the containers moved before loading the next one are exactly the still-present members of the joint-order interval, and a present container v lies in it with probability 1/2 precisely when its ship value exceeds m, the smaller of the two pickups bracketing the load. That gives an expectation of n plus half the sum of untrucked counts above each pair minimum, tallied with the year-local fenwick as a suffix count in one logarithmic-tree pass per container, the derivation the solutions pdf carries on its pages 2 and 3, problem at icpc2022.pdf. The seam before the first pickup is bounded by a1 itself, and the incoming endpoint is loaded rather than shuffled, so it never counts.

The worked run: trace the model on sample 1. The pickup order is 4, 2, 5, 3, 1, and load i pays half the count of still-present ship values above m, the smaller of the two pickups bracketing it, with the first load bracketed by a1 = 4 alone. Load 1: of the present 1 to 5, one value, 5, exceeds 4, count 1. Load 2, bracketed by 4 and 2: of the present 1, 2, 3, 5, two values, 3 and 5, exceed m = 2, count 2. Load 3, bracketed by 2 and 5: of the present 1, 3, 5, two exceed m = 2, but the endpoint 5 itself is loaded, not shuffled, count 1. Loads 4 and 5, bracketed by 5 and 3 then 3 and 1: nothing present exceeds 3, then nothing exceeds 1, counts 0 and 0. The correction sum is 1 + 2 + 1 = 4, the expectation is n plus half of it, 5 + 4/2 = 7, and the trace ends at the printed answer `7.000`.

load	bracket	m	present above m	count
1	seam, a1	4	5	1
2	4, 2	2	3, 5	2
3	2, 5	2	3, the endpoint 5 loads instead	1
4	5, 3	3	none	0
5	3, 1	1	none	0

```c
static char *solve(const char *in) {
  char *buf = dupstr(in);
  Ch11Scan sc;
  ch11_scan_init(&sc, buf);
  long long n = ch11_next_int64(&sc);
  memset(tree, 0, sizeof(long long) * (size_t)(n + 1));
  for (long long i = 1; i ≤ n; i++) {
    pickup[i] = (int)ch11_next_int64(&sc);
    fw_add(tree, n, i, 1);
  }
  long long present = n, extra = 0;
  for (long long i = 1; i ≤ n; i++) {
    long long m = (i == 1)
                    ? pickup[1]
                    : (pickup[i - 1] < pickup[i] ? pickup[i - 1] : pickup[i]);
    // v > m still present, not counting pickup[i] itself
    long long cnt = present - fw_prefix(tree, m);
    if (pickup[i] > m)
      cnt--;
    extra += cnt;
    fw_add(tree, n, pickup[i], -1);
    present--;
  }
  char *out = malloc(32);
  snprintf(out, 32, "%.3f\n", (double)n + extra / 2.0);
  free(buf);
  return out;
}
```

listing 424: c, the whole solver: suffix counts above m, seam bounded by a1, endpoint excluded (source: icpc/samples-c/src/Ch11/pQ.c, lines 28-55)

```
func SolveQ(input string) string {
  sc := &scan11{data: []byte(input)}
  n := sc.nextInt()
  a := make([]int, n)
  for i := range a {
    a[i] = sc.nextInt()
  }
  fen := newFenwick11(n)
  correction := int64(0)
  for i := 0; i < n; i++ {
    m := a[i]
    if i > 0 && a[i-1] < m {
      m = a[i-1]
    }
    // values above m, minus the already-trucked ones above m;
    // the endpoint a[i] itself is loaded, never shuffled
    above := int64(n - m)
    truckedAbove := int64(i - fen.prefix(m))
    correction += above - truckedAbove
    if a[i] > m {
      correction--
    }
    fen.add(a[i], 1)
  }
  e := float64(n) + float64(correction)/2
  return fmt.Sprintf("%.3f\n", e)
}
```

listing 425: go, per pickup: values above m minus the already-trucked above m, minus the endpoint when it exceeds m (source: icpc/samples-go/ch11/pq.go, lines 15-41)

```
  static String solve(String in) {
    pos = 0;
    int n = nextInt(in);
    int[] pickup = new int[n + 1];
    Fenwick tree = new Fenwick(n);
    for (int i = 1; i ≤ n; i++) {
      pickup[i] = nextInt(in);
      tree.add(i, 1);
    }
    long present = n, extra = 0;
    for (int i = 1; i ≤ n; i++) {
      long m = i == 1 ? pickup[1] : Math.min(pickup[i - 1], pickup[i]);
      // v > m still present, not counting pickup[i] itself
      long cnt = present - tree.prefix((int) m);
      if (pickup[i] > m)
        cnt--;
      extra += cnt;
      tree.add(pickup[i], -1);
      present--;
    }
    return String.format(Locale.ROOT, "%.3f", n + extra / 2.0) + "\n";
```

listing 426: java, the whole solver: fenwick suffix counts above m, seam bounded by a1, endpoint excluded (source: icpc/samples-java/src/Ch12/PQ.java, lines 35-55)

```
        // E = n + (1/2) * sum over consecutive pickup pairs of the count of
        // values v > min pair endpoint not yet trucked and not the incoming
        // endpoint a[i+1] itself (it is loaded, not shuffled); the seam pair
        // before a[1] is bounded by a[1] itself
        var fen = new Fenwick(n);
        long corrections = n - a[1];
        fen.Add(a[1], 1);
        for (int i = 1; i < n; i++)
        {
            int m = Math.Min(a[i], a[i + 1]);
            corrections += n - m - (i - fen.Prefix(m)) - (a[i + 1] > m ? 1 : 0);
            fen.Add(a[i + 1], 1);
        }
        double e = n + corrections / 2.0;
        return e.ToString("F3", CultureInfo.InvariantCulture) + "\n";
```

listing 427: c#, the same accounting seeded with the seam correction before the loop (source: icpc/samples/src/Ch11/PQ.cs, lines 15-29)

```
export function solve(input) {
  const n = input.length;
  let p = 0;
  const nextInt = () => {
    while (p < n && (input.charCodeAt(p) < 48 || input.charCodeAt(p) > 57)) p++;
    let v = 0;
    while (p < n && input.charCodeAt(p) ≥ 48 && input.charCodeAt(p) ≤ 57)
      v = v * 10 + (input.charCodeAt(p++) - 48);
    return v;
  };
  const count = nextInt();
  const pickup = new Int32Array(count + 1);
  const fw = new Fenwick(count);
  for (let i = 1; i ≤ count; i++) {
    pickup[i] = nextInt();
    fw.add(i, 1);
  }
  let present = count;
  let extra = 0;
  for (let i = 1; i ≤ count; i++) {
    const m = i === 1 ? pickup[1] : Math.min(pickup[i - 1], pickup[i]);
    // ship values still present above m, not counting pickup[i] itself
    let cnt = present - fw.prefix(m);
    if (pickup[i] > m) cnt--;
    extra += cnt;
    fw.add(pickup[i], -1);
    present--;
  }
  return `${(count + extra / 2).toFixed(3)}\n`;
}
```

listing 428: javascript, present minus prefix with the fenwick shed as trucks load (source: icpc/samples-js/src/ch11-pq-doingthecontainershuffle.mjs, lines 13-42)

```
def solve(text):
    data = text.split()
    n = int(data[0])
    fw = Fenwick(n)
    left = n
    prev = int(data[1])
    fw.add(prev, -1)
    left -= 1
    extra = left - fw.sum(prev)
    for i in range(2, n + 1):
        cur = int(data[i])
        m = prev if prev < cur else cur
        fw.add(cur, -1)
        left -= 1
        extra += left - fw.sum(m)
        prev = cur
    return f"{n + extra / 2:.3f}\n"


if __name__ == "__main__":
    if len(sys.argv) > 1:
        with open(sys.argv[1], encoding="utf-8") as fh:
```

listing 429: python, the born-full fenwick only pays removes, extra counts left - sum (source: icpc/samples-py/src/Ch11/pq.py, lines 12-33)

```
  local n = next_int()
  local p = {}
  for i = 1, n do p[i] = next_int() end
  local f = fen.new_ones(n)
  local add, prefix = fen.add, fen.prefix
  local corr = n - prefix(f, p[1]) -- seam pair: m = a_1, nothing trucked
  add(f, p[1], -1)
  local left = n - 1
  for i = 2, n do
    add(f, p[i], -1)
    left = left - 1
    local pm = p[i - 1]
    local m = pm < p[i] and pm or p[i]
    corr = corr + left - prefix(f, m)
  end
  return string.format("%.3f\n", (2 * n + corr) / 2)
end
```

listing 430: lua, seam first then pairs, printed as (2n + corr) / 2 to stay in integers (source: icpc/samples-lua/ch11_pq.lua, lines 33-49)

The correction sum is an integer, so E always ends in .0 or .5 and prints exactly: `%.3f` in C, go, and lua, through `Locale.ROOT` in java so the digits never localize, `F3` with the invariant culture in C#, and `toFixed(3)` in javascript, which is why the judge tolerances are a backstop rather than a necessity. The fixture is n = 4 with pickup order 3 1 4 2: the seam pair contributes 1/2, the pair (3, 1) contributes 1, the pair (1, 4) contributes 1/2, and the last pair nothing, for E = 4 + 2 = 6. Every suite asserts `6.000` on that input; the family adds the two official samples `7.000` and `13.500` and the crafted single container `1.000`, four checks per language except the C# suite, which folds the two official samples into one fact, java's four parsing the printed double back and comparing within 1e-9.

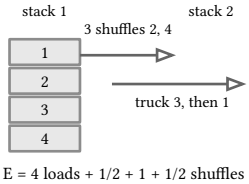


figure 144: one placement of the fixture: all four containers on stack 1, seven moves counted, expectation 6.000

13.4. r, zoo management

A zoo's enclosures all hold animals and are joined by two-way tunnels, and moving anyone is hard because no enclosure may sit empty mid-move. One move picks a set of animals and sends them simultaneously down distinct tunnels, each into a neighboring enclosure that is itself being vacated in the same move, and no tunnel carries two animals in one move, so a move is exactly a rotation around one or more vertex-disjoint cycles of the tunnel graph: two animals cannot simply trade places through one tunnel, they would meet inside. Enclosure i holds an animal of type b_i at the start and should hold an animal of type e_i at the end, the target types a permutation of the start types. Decide whether some sequence of such moves reaches the target arrangement (statement).

The input, per the problemset pdf, is one line of n and m, with 1 <= n <= 4e5 enclosures and 0 <= m <= 4e5 tunnels, then n lines of b_i and e_i, animal types in 1..1e6 whose targets are a permutation of the starts, then m tunnel lines x and y with 1 <= x < y <= n and no pair of enclosures joined twice. The output is possible or impossible under the pdf's 5 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `R-zoomanagement/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: one triangle of enclosures whose targets are exactly the starts rotated one step around it, and a single-cycle component reaches exactly its rotations.

input:

```
3 3
1 4
4 7
7 1
1 2
2 3
1 3
```

expected output:

```
possible
```

Official sample 2, same source: two enclosures joined by their only tunnel, which is a bridge, and no animal ever crosses a bridge, so the wanted swap is out of reach.

input:

```
2 1
1 2
2 1
1 2
```

expected output:

```
impossible
```

Recognition: n and m both <= 4e5 under the 5 second limit price one connectivity pass: a low-link DFS visits 8e5 cells once, and the per-component verdicts are cycle walks of the same order, so the whole decide is linear.

The statement's cue is that a move is a rotation around cycles of the tunnel graph: what any arrangement can reach is decided by how the enclosures decompose, bridges against 2-edge-connected pieces, not by planning moves. Problem R is the 2022 face of the component structure and connectivity family, and the component structure and connectivity family runs through chapter 11 problem B, chapter 12 problem E, chapter 12 problem H, chapter 15 problem E, and chapter 14 problem G.

Search over arrangements prices out immediately, the target alone is one of up to (4e5)! permutations, so dropping bridges and deciding each piece by rotation equality and parity is the only budget that closes.

No animal ever crosses a bridge, so bridges are dropped and each 2-edge-connected component is decided alone: a component that is one simple cycle reaches exactly its cyclic rotations, tested by cyclic string equivalence, and a component with several cycles reaches every permutation when some edge lies on two cycles or some cycle is even, otherwise exactly the even permutations, with the parity of the required relabeling automatic whenever a type repeats. Linear in vertices plus edges, per the solutions pdf pages 3 and 4, problem at icpc2022.pdf. Book 10′s chapter 39, graph connectivity and decomposition, builds the bridge-finding and 2-edge-connected decomposition this drop-bridges step runs on. The seven walks find bridges with one iterative low-link DFS each, then part ways on the cycle bookkeeping.

The worked run: trace the model on sample 1. The three tunnels 1-2, 2-3 and 1-3 form one component of 3 vertices and 3 edges, no edge is a bridge, and one simple cycle, a triangle. Walking it from enclosure 1, the starts read 1, 4, 7 and the targets read 4, 7, 1, and the target string 471 sits inside the doubled start string 147147 at offset 1, so the wanted arrangement is exactly a one-step rotation of the animals. The component verdict is possible, it is the only component, and the trace ends at the printed answer `possible`.

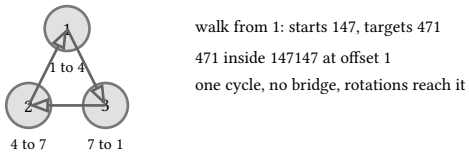


figure 145: sample 1′s triangle with each enclosure's start and target animal, the one-step rotation carrying all three home

```
  } else if (!even_comp[comp[r]]) {
    // several cycles, none even: free only when a tree edge inside
    // the component carries two cycles, else exactly the evens
    int free_all = 0, dup = 0;
    for (int i = 0; i < nm && !free_all; i++) {
      int v = member[i];
      if (par[v] && acc[v] >= 2 && comp[par[v]] == comp[r])
        free_all = 1;
    }
    if (!free_all) {
      for (int i = 1; i < nm && !dup; i++)
        if (cs[i] == cs[i - 1])
          dup = 1;
      if (!dup) {
        for (int i = 0; i < nm; i++)
          posT[t0[member[i]]] = member[i];
        long long cycles = 0;
        for (int i = 0; i < nm; i++) {
          if (vis2[member[i]])
            continue;
          cycles++;
          int u = member[i];
          while (!vis2[u]) {
            vis2[u] = 1;
            u = (int)posT[s0[u]];
          }
        }
        if ((nm - cycles) % 2 != 0)
          possible = 0;
        for (int i = 0; i < nm; i++)
          posT[t0[member[i]]] = 0;
      }
    }
  }
```

listing 431: c, the several-odd-cycles verdict: free only when a tree edge carries two back-edge paths, else even permutations, dup labels absorb parity (source: icpc/samples-c/src/Ch11/pR.c, lines 252-285)

```
      if low[u] < disc[par] {
        continue
      }
      // pop one block: edges down to and including te
      blockStamp++
      verts := 0
      countVert := func(id int) {
        if mark[eu[id]] ≠ blockStamp {
          mark[eu[id]] = blockStamp
          verts++
        }
        if mark[ev[id]] ≠ blockStamp {
          mark[ev[id]] = blockStamp
          verts++
        }
      }
      first := estack[len(estack)-1]
      edges := 0
      for {
        id := estack[len(estack)-1]
        estack = estack[:len(estack)-1]
        countVert(id)
        edges++
        if id == te {
          break
        }
      }
      if edges == 1 {
        isBridge[first] = true
      } else {
        b := block{rep: eu[first]}
        if edges > verts {
          b.multi = true
        } else {
          b.cyclic = true
          b.even = verts%2 == 0
        }
        blocks = append(blocks, b)
      }
```

listing 432: go, carving biconnected blocks off an edge stack: one edge is a bridge, more edges than verts is multi, else cyclic and even or odd (source: icpc/samples-go/ch11/pr.go, lines 95-133)

```
    } else if (evenComp[comp[r]] == 0) {
      // several cycles, none even: free only when a tree edge inside
      // the component carries two cycles, else exactly the evens
      boolean freeAll = false, dup = false;
      for (int i = 0; i < nm && !freeAll; i++) {
        int v = member[i];
        if (par[v] != 0 && acc[v] >= 2 && comp[par[v]] == comp[r])
          freeAll = true;
      }
      if (!freeAll) {
        for (int i = 1; i < nm && !dup; i++)
          if (csS[i] == csS[i - 1])
            dup = true;
        if (!dup) {
          for (int i = 0; i < nm; i++)
            posT[(int) t0[member[i]]] = member[i];
          long cycles = 0;
          for (int i = 0; i < nm; i++) {
            if (vis2[member[i]] != 0)
              continue;
            cycles++;
            int u = member[i];
            while (vis2[u] == 0) {
              vis2[u] = 1;
              u = (int) posT[(int) s0[u]];
            }
          }
          if ((nm - cycles) % 2 != 0)
            possible = false;
          for (int i = 0; i < nm; i++)
            posT[(int) t0[member[i]]] = 0;
        }
      }
    }
```

listing 433: java, the several-odd-cycles verdict: free only when a tree edge carries two cycles, dup labels absorb parity, else the cycle-count parity decides (source: icpc/samples-java/src/Ch12/PR.java, lines 269-302)

```
            if (edges == nv && nv ≥ 2)
            {
                // one simple cycle: target must be a rotation of start
                var s = new System.Text.StringBuilder();
                var tt = new System.Text.StringBuilder();
                int prevE = -1, at = vlist[0];
                for (int i = 0; i < nv; i++)
                {
                    s.Append(sa[at]).Append(',');
                    tt.Append(ta[at]).Append(',');
                    int ne = -1, nx = -1;
                    for (int p = off[at]; p < off[at + 1]; p++)
                        if (adjE[p] ≠ prevE && !bridge[adjE[p]])
                        {
                            ne = adjE[p];
                            nx = adjV[p];
                            break;
                        }
                    prevE = ne;
                    at = nx;
                }
                string ss = s.ToString();
                if (!(ss + ss).Contains(tt.ToString()))
                    ok = false;
            }
```

listing 434: c#, one simple cycle: walk it, then rotation equality as one string contained in the other doubled (source: icpc/samples/src/Ch11/PR.cs, lines 181-205)

```
  for (let r = 1; r ≤ n; r++) {
    if (disc[r]) continue;
    let sp = 0;
    stkV[sp] = r;
    stkE[sp] = -1;
    stkIt[sp] = xadj[r];
    disc[r] = low[r] = ++timer;
    torder[timer] = r;
    sp++;
    while (sp) {
      const v = stkV[sp - 1];
      if (stkIt[sp - 1] =/= xadj[v + 1]) {
        const it = stkIt[sp - 1]++;
        const to = adjTo[it];
        const e = adjSlot[it];
        if (e === stkE[sp - 1]) continue; // the edge we arrived on
        if (!disc[to]) {
          disc[to] = low[to] = ++timer;
          torder[timer] = to;
          par[to] = v;
          depth[to] = depth[v] + 1;
          stkV[sp] = to;
          stkE[sp] = e;
          stkIt[sp] = xadj[to];
          sp++;
        } else if (disc[to] < disc[v]) {
          cover[v]++; // back edge covers the tree path to..v
          cover[to]--;
          if (disc[to] < low[v]) low[v] = disc[to];
          if (((depth[v] - depth[to] + 1) & 1) === 0) evenCycle[e] = 1;
        }
      } else {
        sp--;
        if (sp) {
          const u = stkV[sp - 1];
          if (low[v] < low[u]) low[u] = low[v];
          if (low[v] > disc[u]) isBridge[stkE[sp]] = 1;
        }
      }
    }
  }
```

listing 435: javascript, the low-link DFS with +1/-1 back-edge path marks and even fundamental cycles on Int32Arrays (source: icpc/samples-js/src/ch11-pr-zoomanagement.mjs, lines 57-97)

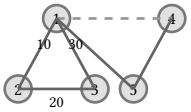
```
        # several cycles: a cactus of odd cycles limits to even perms,
        # tree edges shared by two back-edge cycles break the cactus
        cactus = True
        even_cycle = False
        for r in vs:
            if seen[r]:
                continue
            seen[r] = 1
            stack = [[r, head[r]]]
            while stack:
                fr = stack[-1]
                v = fr[0]
                if fr[1] < head[v + 1]:
                    at = fr[1]
                    fr[1] += 1
                    if bridge[eid[at]]:
                        continue
                    e = eid[at]
                    w = nbr[at]
                    if not seen[w]:
                        seen[w] = 1
                        par[w] = v
                        pare[w] = e
                        depth[w] = depth[v] + 1
                        stack.append([w, head[w]])
                    elif depth[w] < depth[v] and e ≠ pare[v]:
                        u = v
                        while u ≠ w:
                            if owner[pare[u]] ≥ 0:
                                cactus = False
                                break
                            owner[pare[u]] = e
                            u = par[u]
                        if not cactus:
                            break
                        if (depth[v] - depth[w] + 1) % 2 == 0:
                            even_cycle = True
                else:
                    stack.pop()
            if not cactus:
                break
```

listing 436: python, a second component DFS marks each tree edge with the back edge that owns it, sharing two owners breaks the cactus (source: icpc/samples-py/src/Ch11/pr.py, lines 149-189)

```
    elseif ok and edges2 == nm then
      -- one simple cycle: walk it and test cyclic rotation equality
      local walk_s, walk_t = {}, {}
      local v = member[1]
      local prev_e, clen = 0, 0
      repeat
        clen = clen + 1
        walk_s[clen], walk_t[clen] = s0[v], t0[v]
        local next_it = 0
        for it = xadj[v], xadj[v + 1] - 1 do
          if not is_bridge[eslot[it]] and eslot[it] ~= prev_e then
            next_it = it
            break
          end
        end
        prev_e = eslot[next_it]
        v = eadj[next_it]
      until v == member[1]
      -- kmp of the needle over the hay doubled
      local fail = {0}
      local k = 0
      for i = 2, clen do
        while k > 0 and walk_t[i] ~= walk_t[k + 1] do k = fail[k] end
        if walk_t[i] == walk_t[k + 1] then k = k + 1 end
        fail[i] = k
      end
      k = 0
      local found = false
      for i = 1, 2 * clen do
        local c = walk_s[i > clen and i - clen or i]
        while k > 0 and c ~= walk_t[k + 1] do k = fail[k] end
        if c == walk_t[k + 1] then k = k + 1 end
        if k == clen then
          found = true
          break
        end
      end
      ok = found
```

listing 437: lua, the cycle walk collecting both animal strings, then KMP of one inside the other doubled (source: icpc/samples-lua/ch11_pr.lua, lines 177-214)

The fixture is two triangles sharing vertex 1, enclosures 1-2-3 and 1-4-5, an all-odd cactus: the desired relabeling rotates the animals of enclosures 1, 2, 3 as one 3-cycle, which is even, so the answer is possible, and swapping just two animals instead is a single transposition, odd, and impossible, a variant every suite pins beside the official four samples. Counts per language: 6 checks in C and python, 6 facts in the C# suite, 6 go rows and 6 js blocks, and the lua module carries a 7th, a pendant bridge keeping its triangle honest, java's 8 pinning the fixture pair, all four official samples, both even-cycle boards, and the shared-edge pair. Animal types are opaque labels that fit a 32-bit int in every language, so no integer notes here.



start 10 20 30, target 30 10 20: a 3-cycle, even

the shared vertex never moves an animal across

bridge-free rotation, possible

figure 146: the fixture: two odd triangles sharing vertex 1, the relabeling chases three animals around one of them

13.5. s, bridging the gap

A group of n walkers reaches a river at night carrying one torch. The old bridge holds at most c walkers at a time and crossing it needs the torch, every walker has a fixed crossing time, and a group crosses at its slowest member's pace. Some walker must always carry the torch back for the next group, and the task is the shortest total time for everyone to end on the far side (statement). Bridging the Gap is one of the five problem names this finals shares with the 2023 set: this set's S is 2023's J retold.

The input, per the problemset pdf, is one line of n and c, with 2 <= n <= 1e4 walkers and 2 <= c <= 1e4 bridge capacity, then n crossing times t_i with 1 <= t_i <= 1e9. The output is the minimum total crossing time, under the pdf's 4 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `S-bridgingthegap/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints and walks through: the two fastest cross for 2, the fastest returns for 1, the two slowest cross for 10, the second-fastest returns for 2, and the two fastest cross again for 2, totalling 17.

input:

```
4 2
1 2 10 5
```

expected output:

```
17
```

Official sample 2, same source, same walkers at capacity 6: everyone fits in one batch, so the whole group crosses once at the slowest pace.

input:

```
4 6
1 2 10 5
```

expected output:

```
10
```

Recognition: n <= 1e4 walkers and c <= 1e4 capacity under the 4 second limit price an exact state machine, not a smarter formula: the (k, g, l) space holds 5e7 to 3.5e8 live states on the judge secrets, and both shipped engines visit each state once at O(1) amortized, which is what the limit buys.

The statement's cue is the torch ledger: every crossing and every return changes who stands where, and the optimum turns on the contiguous block of banked fast walkers, the cue that the state must be engineered by hand rather than read off the input. Problem S is the 2022 face of the dp over an engineered state space family, and the dp over an engineered state space family spans chapter 14 problem J, chapter 11 problem J, chapter 12 problem I, and chapter 15 problem H.

The tempting alternative, Dijkstra over crossing schedules, pays the same state space again with a log factor on top, and any n-by-l cell grid dies on memory alone, terabytes near c = n/2, so the swept machine with reachability-clamped caps is the shape that fits.

The exact state, with the times sorted ascending so person 0 is the fastest, is (k, g, l): the k slowest are delivered, the fast walkers on the far side are exactly the contiguous block g..g+l-1, the torch is near, and g canonicalizes to 1 whenever l = 0. Four transitions move it. The escorted batch sends j >= 1 of the slowest plus e = c - j escorts with e > g, the e fastest near-side walkers with person 0 among them, at cost t[n-k-1] for the batch and t[0] for the escort's return, landing at (k+j, 1, l+e-1). The pure slow batch sends j of the slowest with j <= min(c, n-k-l) and the fastest banked walker t[g] brings the torch back, which needs l >= 1, at cost t[n-k-1] + t[g], landing at (k+j, g+1, l-1). The pure fast batch sends j > g of the fastest near-side walkers at cost t[l+j-1] + t[0], banking them into (k, 1, l+j-1). And once n-k-l <= c, everyone left crosses at the slowest remaining pace t[n-k-1] and the instance is done. The solutions pdf page 4 sketches a quadratic dp over a machine of this shape, problem at icpc2022.pdf, and this pinned state and transition set was validated against an exhaustive dijkstra over more than 3000 random small instances with zero mismatches, plus both official samples. Two things the sketch does not carry. First, the g dimension is load-bearing: collapsing to (k, l) measurably under- and over-counts against brute force. Second, the editorial's n-squared-over-c bound does not hold for the exact machine on shuttle instances, which bank on the order of n walkers for any c, so the l dimension runs its full range and no cell grid survives the memory rule. The shipped engines are two shapes of the same recurrence, a pull-based row sweep with lazy window heaps in C, C#, and java, and a diagonal sweep s = k + l with sliding-window materialization in go, javascript, python, and lua.

The worked run: trace the model on sample 1. The times sorted ascending are t = 1, 2, 5, 10, person 0 the fastest, and the machine starts at (k, g, l) = (0, 1, 0), torch near, nothing delivered. The pure fast batch sends j = 2 of the fastest, persons 0 and 1, at cost t[l + j - 1] + t[0] = t[1] + t[0] = 2 + 1 = 3, the cross at pace 2 and person 0's return at pace 1, landing at (0, 1, 1) with person 1 banked far side. The pure slow batch then sends j = 2 of the slowest, persons 2 and 3, at t[n - k - 1] = t[3] = 10, with banked walker t[g] = t[1] = 2 bringing the torch back, cost 12, landing at (2, 2, 0). Now n - k - l = 4 - 2 - 0 = 2 <= c, so the finish transition applies: everyone left crosses at t[n - k - 1] = t[1] = 2. The total is 3 + 12 + 2 = 17, and the trace ends at the printed answer 17.

state	transition	who moves	cost	lands
(0, 1, 0)	pure fast, j = 2	persons 0, 1 cross, person 0 returns	2 + 1	(0, 1, 1)
(0, 1, 1)	pure slow, j = 2	persons 2, 3 cross, t[1] returns	10 + 2	(2, 2, 0)
(2, 2, 0)	finish, 2 <= c	persons 0, 1 cross	2	done

```
    for (int k = 0; k ≤ r; k++) {
      int l = r - k;
      int nc = 0;
      if (r == 0 && k == 0)
        cand[nc].d = 0, cand[nc++].g = 1; // seed (0, 1, 0)
      // pure fast: col[k] at cursor l, window pos ≥ l - c
      LHeap *ck = &col[k];
      if (lheap_query(ck, l, l - c))
        cand[nc].d = ck->act[0].val + t[l] + t[0], cand[nc++].g = 1;
      // escorted: escact at cursor k, no recency bound
      if (lheap_query(&escact, k, INT_MIN))
        cand[nc].d = escact.act[0].val, cand[nc++].g = 1;
      // pure slow: diagonal l+1, window pos ≥ k - c, j ≥ 2;
      // buckets with g > n-1-r can never fire (j ≤ mid)
      if (l + 1 ≤ n && k ≥ 2) {
        Diag *dl = &dg[l + 1];
        int gmax = n - 1 - r;
        for (int i = 0; i < dl->nbk; i++) {
          if (dl->bk[i].g > gmax)
            continue;
          if (lheap_query(&dl->bk[i].h, k, k - c))
            cand[nc].d = dl->bk[i].h.act[0].val, cand[nc++].g = dl->bk[i].g + 1;
        }
        for (int i = 0; i < dl->nr;) {
          REnt *e = &dl->rare[i];
          if (e->hi < k || e->k < k - c || e->g > gmax) {
            dl->rare[i] = dl->rare[--dl->nr]; // gmax only shrinks
            continue;
          }
          if (e->k ≤ k - 2)
            cand[nc].d = e->w, cand[nc++].g = e->g + 1;
          i++;
        }
      }
      // in-row: pure slow j=1 from (k-1, l+1)
      for (int i = 0; i < nprev; i++)
        cand[nc].d = prev[i].d + t[n - k] + t[prev[i].g],
        cand[nc++].g = prev[i].g + 1;
      if (nc + 2 > capcand) {
        while (nc + 2 > capcand)
          capcand *= 2;
        cand = realloc(cand, (size_t)capcand * sizeof(Ent));
      }
      if (nc > capfront) {
        while (nc > capfront)
          capfront *= 2;
        prev = realloc(prev, (size_t)capfront * sizeof(Ent));
        cur = realloc(cur, (size_t)capfront * sizeof(Ent));
      }
      int ncur = front_build(cur, cand, nc);
```

listing 438: c, the per-cell pull: seed, pure-fast column heap, escorted row heap, per-g pure-slow buckets, then the pareto front of (d, g) (source: icpc/samples-c/src/Ch11/pS.c, lines 213-262)

```
      for g := 1; g <= D; g++ {
        if b.m[g>>6]&(1<<(g&63)) == 0 {
          continue
        }
        d := b.d[g]
        if total := n - K - l; total ≥ 1 {
          q := 1
          if total ≥ 2 {
            q = (total + c - 3) / step
          }
          if v := d + int64(q-1)*t0 + shuttle(K, q); v < best {
            best = v
          }
        }
        if l ≥ 1 {
          j := c
          if n-K < j {
            j = n - K
          }
          ng := g + 1
          nl := l - 1
          if nl == 0 {
            ng = 1
          }
          if ng <= D && K+j+nl <= n {
            v := d + t[n-K-1] + t[g]
            if j == 1 {
              chmin(K+1, ng, nl, s, v)
            } else {
              td := s + j - 1
              pend[td%R2] = append(pend[td%R2], bp{K + j, ng, nl, v})
            }
          }
        }
      }
```

listing 439: go, the diagonal expansion: shuttle completion priced from stride prefix sums, pure-slow fan-out as one point write (source: icpc/samples-go/ch11/psfast.go, lines 331-365)

```
      for (int k = 0; k ≤ r; k++) {
        int l = r - k;
        int nc = 0;
        if (r == 0 && k == 0) {
          candD[nc] = 0;
          candG[nc++] = 1; // seed (0, 1, 0)
        }
        // pure fast: col[k] at cursor l, window pos ≥ l - c
        LHeap ck = col[k];
        if (ck.query(l, l - c)) {
          candD[nc] = ck.aval[0] + t[l] + t[0];
          candG[nc++] = 1;
        }
        // escorted: escact at cursor k, no recency bound
        if (escact.query(k, Integer.MIN_VALUE)) {
          candD[nc] = escact.aval[0];
          candG[nc++] = 1;
        }
        // pure slow: diagonal l+1, window pos ≥ k - c, j ≥ 2;
        // buckets with g > n-1-r can never fire (j ≤ mid)
        if (l + 1 ≤ n && k ≥ 2) {
          Diag dl = dg[l + 1];
          int gmax = n - 1 - r;
          for (int i = 0; i < dl.nbk; i++) {
            if (dl.bk[i].g > gmax)
              continue;
            if (dl.bk[i].h.query(k, k - c)) {
              candD[nc] = dl.bk[i].h.aval[0];
              candG[nc++] = dl.bk[i].g + 1;
            }
          }
          for (int i = 0; i < dl.nr;) {
            REnt e = dl.rare[i];
            if (e.hi < k || e.k < k - c || e.g > gmax) {
              dl.rare[i] = dl.rare[--dl.nr]; // gmax only shrinks
              continue;
            }
            if (e.k ≤ k - 2) {
              candD[nc] = e.w;
              candG[nc++] = e.g + 1;
            }
            i++;
          }
        }
        // in-row: pure slow j=1 from (k-1, l+1)
        for (int i = 0; i < nprev; i++) {
          candD[nc] = prevD[i] + t[n - k] + t[prevG[i]];
          candG[nc++] = prevG[i] + 1;
        }
```

listing 440: java, the per-cell pull: seed, pure-fast column heap, escorted row heap, per-g pure-slow buckets, the in-row j = 1 offers off the previous front (source: icpc/samples-java/src/Ch12/PS.java, lines 260-308)

```
            for (int k = 0; k ≤ r; k++)
            {
                int l = r - k;
                // candidate bound: seed + col + esc + diag buckets and
                // rare list + the previous cell's front
                int need = 3 + nprev;
                if (l + 1 ≤ n && k ≥ 2)
                {
                    Diag ddx = dg[l + 1];
                    need += ddx.nbk + ddx.nr;
                }
                if (need + 2 > capCand)
                {
                    while (need + 2 > capCand)
                        capCand *= 2;
                    Array.Resize(ref candD, capCand);
                    Array.Resize(ref candG, capCand);
                }
                int nc = 0;
                if (r == 0 && k == 0)
                {
                    candD[0] = 0;
                    candG[0] = 1;
                    nc = 1;
                }
                LHeap ck = col[k];
                if (LQuery(ck, l, l - c))
                {
                    candD[nc] = ck.aval[0] + t[l] + t[0];
                    candG[nc] = 1;
                    nc++;
                }
                if (LQuery(escact, k, int.MinValue))
                {
                    candD[nc] = escact.aval[0];
                    candG[nc] = 1;
                    nc++;
                }
                if (l + 1 ≤ n && k ≥ 2)
```

listing 441: c#, the same pull with positions and spans packed into one int, FIFO rings for the monotone offer streams (source: icpc/samples/src/Ch11/PS.cs, lines 360-398)

```
      const rs = touchRow(K);
      rs.tail.set(vec1, sl * W + 1);
      rs.tailL[sl] = l;
      if (vec[D] < Infinity) {
        rs.anyFin = true;
        rs.stg.push(l);
        rs.stgV.push(vec[D]);
      }
      if (snapLive) {
        sn0.v[l] = t[n - s + l - 1] + vec[D];
        sn0.anyFin = true;
        sn0.tail.set(vec1, sl * W + 1);
        sn0.tailL[sl] = l;
      }
      // shuttle completion is per column: hoist its cost
      const total = n - K - l;
      let shBase = 0;
      if (total ≥ 1) {
        let q = 1;
        if (total ≥ 2) q = Math.floor((total + c - 3) / step);
        shBase = (q - 1) * t0 + shuttle(K, q);
      }
      const tnK = t[n - K - 1];
      let j = c;
      if (n - K < j) j = n - K;
      const tdSlot = (s + j - 1) % R2;
      for (let g = 1; g ≤ D; g++) {
        if (!bufM[base + g]) continue;
        const d = bufD[base + g];
        if (total ≥ 1) {
          const v = d + shBase;
          if (v < best) best = v;
        }
        if (l ≥ 1) {
          let ng = g + 1;
          const nl = l - 1;
          if (nl === 0) ng = 1;
          if (ng ≤ D && K + j + nl ≤ n) {
            const v = d + tnK + t[g];
            if (j === 1) chmin(K + 1, ng, nl, s, v);
            else pend[tdSlot].push(K + j, ng, nl, v);
          }
        }
      }
    }
```

listing 442: javascript, finalize each column into g-prefix minima, then expand every live g with the hoisted shuttle cost, all on flat typed arrays (source: icpc/samples-js/src/ch11-ps-bridgingthegap.mjs, lines 253-297)

```
            m = bm[lb]
            if not m:
                continue
            col = bv[lb]
            total = n - K - lb
            fbase = INF
            if total >= 1:
                q = 1 if total < 2 else (total + c - 3) // step
                j_ = K // step
                pr = pref[K % step]
                maxq = len(pr) - 1 - j_
                if q > maxq:
                    q = maxq
                fbase = (q - 1) * t0 + pr[j_ + q] - pr[j_]
            okps = lb >= 1 and K + (c if c < n - K else n - K) + lb - 1 <= n
            psbase = t[n - K - 1] if okps else 0
            if okps:
                nl = lb - 1
                rem = n - K
                j = c if c < rem else rem
                if j == 1:
                    tgt = bv[nl]
                else:
                    pl2 = pend[(s + j - 1) % R2]
            mm = m
            while mm:
                b = mm & -mm
                g = b.bit_length() - 1
                mm ^= b
                d = col[g]
                x = d + fbase
                if x < best:
                    best = x
                if okps and (nl == 0 or g < D):
                    w = d + psbase + t[g]
                    if j == 1:
                        ng = 1 if nl == 0 else g + 1
                        if not (bm[nl] >> ng) & 1 or w < tgt[ng]:
                            tgt[ng] = w
                            bm[nl] |= 1 << ng
                    else:
                        pl2.append((K + j, g + 1 if nl else 1, nl, w))
```

listing 443: python, the column walk over a bitmask of live g slots, finish and pure-slow emission per bit (source: icpc/samples-py/src/Ch11/ps.py, lines 265-306)

The lua twin of the diagonal engine is a year-local module, `ch11_psweep`, printed with the year's helpers in the section at the chapter top.

The fixture is 4 walkers, capacity 3, times 1 2 5 10: the two fastest cross for 2, the fastest walks back for 1, and the remaining three cross together for 10, totalling 13 against 16 for shipping the three fastest first. Every suite asserts 13 on that input, plus the official samples 17 and 10 and the crafted pair 3 7 crossing at capacity 2 for 7, java's fifth check adding a five-walker escort board answering 104. Totals reach 1e4 times 1e9 = 1e13, which divides the languages: int64 in C, C#, go, and java, lua's 64-bit integer subtype, exact but boxed python ints, and javascript numbers, exact below 2^53 and stored here in Float64Array cells at 1e13 with room to spare.

That same state space is the chapter's hardest measured finding, a ratified and documented wall in two languages. On the 104 judge cases the go engine finishes in roughly 11 s at the worst and javascript in 72.8 s. Lua carries

a faithful port of the go engine, bit-identical on 4670 randoms plus brute force, a restructured future-buffer-ring variant, and an exact lazy-heap port of the C engine, validated against a bitmask dijkstra on 3000 randoms and against C on 3320 cases, and all three share the wall: secret-003 (10000x2) at 142 s, secret-010 (10000x3) at 324 s, secret-013 (10000x8) at 417 s, and secret-042 (9983x18) at 286 s, against the 120 s per-case cap, 1.5 to 3.2 times over. Instrumentation over all 104 cases counts live states from 5e7 at c = 2 to 3.5e8 at c = 18 and 61, with maxL = n - 1, so the l cap never prunes judge data and no design lever exists: the state space is the work, and go walks the same states roughly 11 s worth. Python is the same engine family with a different mechanism: judged 57 of 104 with 47 timeouts and zero wrong answers, every timeout an n = 1e4 secret with c in [3, 500], while the c = 2 classic recurrence and every c >= 1000 shape finish comfortably. The deciding mechanism is CPython integer boxing: totals pass the 30-bit fast-int range, so every sweep operation allocates a boxed multi-digit integer, and the sweep becomes allocation-bound where lua's unboxed int64 and go and javascript native arithmetic are not. The 47 versus 4 timeout asymmetry on the same states is the measured evidence that the wall is the language's arithmetic model, not the algorithm. The C# twin also carries a post-landing allocation fix, 9de5851, dead-bucket reclaim plus ArrayPool rentals with no algorithm change: peak working set 3.6 GB on the heaviest cases against the C twin's 3.2 to 3.4 GB, the engine family's live-data floor, CPU 95 to 129 s loaded against C's 58 to 99 s, the JIT codegen gap, and 104 of 104 judged after the fix. Java carries the pull engine as a third twin and clears the wall outright: 104 of 104 judged, its worst secrets at 15.5 to 57.9 s standalone, the largest runtimes anywhere in this book's sample corpus yet inside the 120 s cap, on plain arrays with no tuning passes. Its candidate buffer grows through the same post-write margin guard the C twin carries, the capacity check landing after the writes it is meant to cover, a latent hazard in both that never fires because the pareto front stays far narrower than the doubling margin. Both walls are terminal, recorded here as evidence rather than todo. Two calibration scars also belong on the record: the hash and dijkstra engine measured over 100 s at n = 1e4 with 9.5M distinct states in the first 24k cells at c = 1000, and any cell grid violates the 4 GB rule near c = n/2, where judge case 10000x5259 would want terabytes. The g chain rule g <= l + 1 bounds the g ladder, and an unclamped one once produced a 49 GB scratch table. Random instances keep g <= 4, but the adversarial judge cases push g to c - 1 with fronts about 48 wide, so every engine carries g exactly. The scar's lesson is narrower: never size a table by an unclamped cap, clamp it to the reachable bounds min(G, L + 1). The pull engines in C and C# do that outright, deriving their window bounds from reachability, while the four diagonal engines in go, javascript, python, and lua cut depth at a measured 120 and cap l at 4n/c + 8, raised to 3c + c/2 + 64 when n >= 4c, clamps validated on the judge shapes rather than guessed.

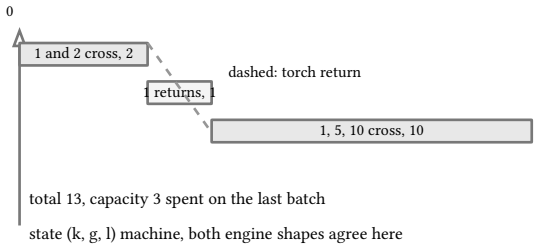


figure 147: the fixture's three crossings on a timeline, torch return walks dashed

13.6. t, carl's vacation

Carl the ant returns for a third finals walk, this time between the apexes of two right square pyramids standing on a common plane, and the desert heat rules out everything but the surfaces: the path may travel over the pyramids' faces and the plane and nothing else. Each pyramid is given by one directed edge of its square base, with the pyramid's body on the left of that direction, plus its height, and the apex sits directly above the base center. The two base squares may touch but their interiors do not overlap. Print the shortest surface distance between the apexes (statement). Carl's Vacation is one of the five problem names this finals shares with the 2023 set: this set's T is 2023′s D retold.

The input, per the problemset pdf, is two lines of five integers each, x1 y1 x2 y2 h: the directed base edge and the height. The coordinates lie within 1e5 in absolute value, the edge's endpoints are distinct, h is between 1 and 1e5,

and the two bases intersect in zero area. The answer carries an absolute or relative error of at most 1e-6, printed to nine decimals by the judge, under the pdf's 1 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `T-carlsvacation/sample-1.in` and `sample-1.ans`, the only sample this problem carries and the pair that forced the completion below: the optimum enters the second pyramid exactly at a base corner, where no unfold-straight is valid.

input:

```
0 0 10 0 4
9 18 34 26 42
```

expected output:

```
60.866649532
```

Recognition: the coordinates stop at 1e5 and the limit at 1 second, and the budget is candidate count, not per-candidate work: 16 edge-pair unfold-straights, 8 one-corner chains and 16 two-corner chains, each leg minimized by a grid sweep plus ternary refinement in a few hundred probes, tens of thousands of predicate calls in total, thousands of times inside budget.

The statement's cue is shortest surface path: on unfoldable surfaces the optimum is a straight line in some unfolding, so the only question is which finitely many hinge and corner configurations attain it, the cue that turns a continuous path search into enumeration over unfold-straights and corner bends. Problem T is the 2022 face of the geometric critical-point enumeration family, and geometric critical-point enumeration is the shared shape of chapter 14 problem D, chapter 10 problem A, chapter 11 problem D, chapter 11 problem E, and chapter 11 problem G.

The tempting alternative, a dense mesh with Dijkstra over it, prices out at 1e6 nodes before any accuracy argument and still resolves the path only to 1e-3 against the 1e-6 tolerance, while the closed candidate set certifies every digit.

For each of the 16 base-edge pairs the solver rotates each apex about its edge's line down onto the plane, and the straight segment between the two landings is a candidate whenever it crosses both hinge edges in order and its open middle misses both square interiors. The solutions pdf pages 4 and 5, problem at icpc2022.pdf, stop at that sketch, and the sketch alone is incomplete: the official sample's optimum enters pyramid 2 exactly at a base corner and no unfold-straight is valid there, so the shipped solvers add corner bends, paths through one or two of the 8 base corners with each end leg minimized independently by a grid sweep plus ternary refinement under segment validity, plus the 16 two-corner chains. The completion was validated against a dense two-parameter sweep reference on 60 random pyramid pairs with a worst difference of 4e-14. Book 10′s chapter 36, numerical methods, builds the ternary-search refinement those corner legs are minimized with. Every predicate is a cross product from the year-local geo kit, endpoints shrink by a relative 1e-9 before the crossing tests to dodge endpoint-rounding artifacts, and the minimum prints at nine decimals.

The worked run: trace the model on sample 1. Pyramid 1 is the square on (0,0) to (10,0) with height 4, apex over (5, 5), and pyramid 2 the square on the directed edge (9,18) to (34,26) with height 42, corners (9,18), (34,26), (26,51), (1,43), apex over the center (17.5, 34.5). Every one of the 16 edge-pair unfold-straights fails validity, each straight that leaves pyramid 1′s near faces runs through pyramid 2′s interior, so the optimum bends at a corner, and the winner is the one-corner chain through (9,18). Leg 1 crosses pyramid 1′s back face: reflecting the apex across the back edge line, offset sqrt(5^2 + 4^2) = sqrt(41) = 6.403, lands it at (5, 10 - sqrt(41)), and the straight from there to (9,18) is sqrt(4^2 + (8 + sqrt(41))^2) = sqrt(223.450) = 14.948244, exiting the back edge inside the segment at (6.778, 10). Leg 2 climbs pyramid 2′s face on the given edge: unfolding apex 2 about that edge, from the edge center (21.5, 22) the slant sqrt(13.125^2 + 42^2) = 44.004 lands at (34.911, −19.909), and the corner-to-unfolded-apex straight is sqrt(25.911^2 + 37.909^2) = sqrt(2108.500) = 45.918406. The plane middle is the corner itself, so the path totals 14.948244 + 45.918406 = 60.866650, and the trace ends at the printed answer `60.866649532`.

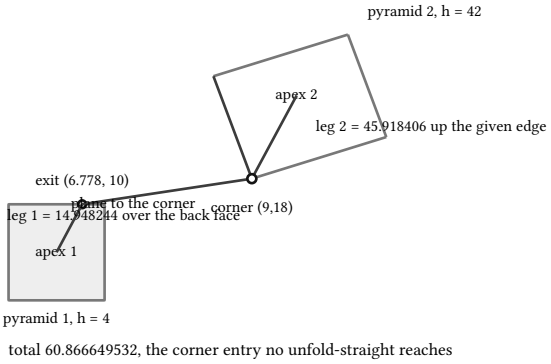


figure 148: sample 1′s one-corner winner: down pyramid 1′s back face, across to the corner (9,18), up pyramid 2′s given-edge face

```c
// minimize slant(s, p(t)) + |p(t) - k| over t in [0,1] along edge ei,
// with the ground segment p(t)→k required valid; the objective is
// convex so grid + ternary suffices
static double best_leg(const Sq *s, int ei, GPoint k) {
  GPoint a = s->v[ei], b = s->v[(ei + 1) % 4];
  int NG = 1024;
  double bestv = -1, bestt = -1;
  for (int i = 0; i <= NG; i++) {
    double t = (double)i / NG;
    GPoint p = add(a, mul(sub(b, a), t));
    if (!ground_ok(p, k))
      continue;
    double f = slant(s, p) + len(sub(p, k));
    if (bestv < 0 || f < bestv) {
      bestv = f;
      bestt = t;
    }
  }
  if (bestv < 0)
    return -1;
  double lo = bestt - 1.0 / NG, hi = bestt + 1.0 / NG;
  if (lo < 0)
    lo = 0;
  if (hi > 1)
    hi = 1;
  for (int it = 0; it < 100; it++) {
    double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
    GPoint p1 = add(a, mul(sub(b, a), m1));
    GPoint p2 = add(a, mul(sub(b, a), m2));
    double f1 = ground_ok(p1, k) ? slant(s, p1) + len(sub(p1, k)) : 1e30;
    double f2 = ground_ok(p2, k) ? slant(s, p2) + len(sub(p2, k)) : 1e30;
    if (f1 <= f2)
      hi = m2;
    else
      lo = m1;
  }
  GPoint p = add(a, mul(sub(b, a), (lo + hi) / 2));
  double f = ground_ok(p, k) ? slant(s, p) + len(sub(p, k)) : bestv;
  return f < bestv ? f : bestv;
}
```

listing 444: c, best_leg: 1024-point grid over the edge, then ternary refinement, validity checked per sample (source: icpc/samples-c/src/Ch11/pT.c, lines 97-136)

```go
// unfoldLen checks the straight candidate between two landings: the
// segment must cross both hinge edges in order and its open middle
// must miss both square interiors.
func unfoldLen(
  p1 *pyr, e1 int, a1 geoVec,
  p2 *pyr, e2 int, a2 geoVec,
) (bool, float64) {
  d := geoSub(a2, a1)
  eps := 1e-9
  s := geoAdd(a1, geoMul(d, eps))
  f := geoSub(a2, geoMul(d, eps))
  h1a, h1b := p1.corners[e1], p1.corners[(e1+1)%4]
  h2a, h2b := p2.corners[e2], p2.corners[(e2+1)%4]
  ok1, c1, t1 := geoSegHit(s, f, h1a, h1b)
  if !ok1 {
    return false, 0
  }
  ok2, c2, t2 := geoSegHit(s, f, h2a, h2b)
  if !ok2 || t2 < t1-1e-12 {
    return false, 0
  }
  // coincident crossings are legal: pyramids sharing a hinge line
  // (one edge a subsegment of the other) are crossed at one shared
  // point, leaving an empty middle
  if t2-t1 < 1e-12 && geoDist2(c1, c2) > 1e-9*geoDist2(s, f) {
    return false, 0
  }
  mid := geoSub(c2, c1)
  m1 := geoAdd(c1, geoMul(mid, 1e-9))
  m2 := geoSub(c2, geoMul(mid, 1e-9))
  if !planeClear(m1, m2, p1.corners[:], p2.corners[:]) {
    return false, 0
  }
  return true, geoDist(a1, a2)
}
```

listing 445: go, unfoldLen: both hinge crossings in order, the shared-hinge case, then the middle cleared (source: icpc/samples-go/ch11/pt.go, lines 115-149)

```java
// minimize slant(s, p(t)) + |p(t) - k| over t in [0,1] along edge ei,
// with the ground segment p(t)→k required valid; the objective is
// convex so grid + ternary suffices
static double bestLeg(Sq s, int ei, GPoint k) {
  GPoint a = s.v[ei], b = s.v[(ei + 1) % 4];
  int ng = 1024;
  double bestv = -1, bestt = -1;
  for (int i = 0; i ≤ ng; i++) {
    double t = (double) i / ng;
    GPoint p = add(a, mul(sub(b, a), t));
    if (!groundOk(p, k))
      continue;
    double f = slant(s, p) + len(sub(p, k));
    if (bestv < 0 || f < bestv) {
      bestv = f;
      bestt = t;
    }
  }
  if (bestv < 0)
    return -1;
  double lo = bestt - 1.0 / ng, hi = bestt + 1.0 / ng;
  if (lo < 0)
    lo = 0;
  if (hi > 1)
    hi = 1;
  for (int it = 0; it < 100; it++) {
    double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
    GPoint p1 = add(a, mul(sub(b, a), m1));
    GPoint p2 = add(a, mul(sub(b, a), m2));
    double f1 = groundOk(p1, k) ? slant(s, p1) + len(sub(p1, k)) : 1e30;
    double f2 = groundOk(p2, k) ? slant(s, p2) + len(sub(p2, k)) : 1e30;
    if (f1 ≤ f2)
      hi = m2;
    else
      lo = m1;
  }
  GPoint p = add(a, mul(sub(b, a), (lo + hi) / 2));
  double f = groundOk(p, k) ? slant(s, p) + len(sub(p, k)) : bestv;
  return Math.min(f, bestv);
```

listing 446: java, bestLeg: 1024-point grid over the edge, then ternary refinement, ground validity checked per sample (source: icpc/samples-java/src/Ch12/PT.java, lines 145-183)

```
    // min over P on edge p-q of sqrt(h^2 + |c-P|^2) + |P-C| with the
    // straight P→C clear of both open squares; convex in the parameter
    private static double MinLeg(Sq home, Sq oth, double[] p, double[] q,
        double cxr, double cyr)
    {
        double hx = home.Cx, hy = home.Cy, h = home.H;
        double ex = q[0] - p[0], ey = q[1] - p[1];
        double best = double.MaxValue;
        const int G = 8192;
        // unconstrained optimum of the convex leg on the full edge:
        // valid whenever it happens to be feasible, independent of run
        // detection
        {
            double lo = 0, hi = 1;
            for (int it = 0; it < 100; it++)
            {
                double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
                if (F(m1) ≤ F(m2))
                    hi = m2;
                else
                    lo = m1;
            }
            double t = Math.Min(Math.Max((lo + hi) / 2, 0), 1);
            if (Feas(t))
                best = Math.Min(best, F(t));
        }
        bool prevFeas = false;
        int runStart = -1;
        double runBest = double.MaxValue;
        double F(double t)
        {
            double x = p[0] + t * ex, y = p[1] + t * ey;
            double dcx = x - hx, dcy = y - hy;
            double dx2 = x - cxr, dy2 = y - cyr;
            return Math.Sqrt(h * h + dcx * dcx + dcy * dcy)
                   + Math.Sqrt(dx2 * dx2 + dy2 * dy2);
        }
        bool Feas(double t)
        {
            double x = p[0] + t * ex, y = p[1] + t * ey;
            return Clear(home, oth, x, y, cxr, cyr);
        }
```

listing 447: c#, MinLeg: the unconstrained convex optimum first, then run detection with bisected feasible edges (source: icpc/samples/src/Ch11/PT.cs, lines 100-141)

```
  // unfold-straights: 4 x 4 edge pairs, both landing sides each
  for (let e1 = 0; e1 < 4; e1++)
    for (let e2 = 0; e2 < 4; e2++)
      for (let s1 = -1; s1 ≤ 1; s1 += 2)
        for (let s2 = -1; s2 ≤ 1; s2 += 2) {
          const a1 = unfoldApex(A, e1, s1);
          const a2 = unfoldApex(B, e2, s2);
          const d = sub(a2, a1);
          const shrink = 1e-9;
          const aa = add(a1, mul(d, shrink));
          const bb = add(a2, mul(d, -shrink));
          // crossing parameters with the two hinge lines
          const tc = [-1, -1], okc = [false, false];
          const hinges = [
            [A.v[e1], A.v[(e1 + 1) % 4]],
            [B.v[e2], B.v[(e2 + 1) % 4]],
          ];
          let both = true;
          for (let hi2 = 0; hi2 < 2; hi2++) {
            const [h1, h2] = hinges[hi2];
            const cr1 = cross2d(h1, h2, aa);
            const cr2 = cross2d(h1, h2, bb);
            const cr3 = cross2d(aa, bb, h1);
            const cr4 = cross2d(aa, bb, h2);
            if (Math.abs(cr2 - cr1) < 1e-18) {
              both = false;
              break;
            }
            const t = cr1 / (cr1 - cr2);
            const u = cr3 / (cr3 - cr4);
            tc[hi2] = t;
            okc[hi2] = t > 1e-12 && t < 1 - 1e-12 && u > -1e-9 && u < 1 + 1e-9;
          }
          if (!both || !okc[0] || !okc[1] || !(tc[0] ≤ tc[1] + 1e-12)) continue;
          const m1 = add(aa, mul(sub(bb, aa), tc[0]));
          const m2 = add(aa, mul(sub(bb, aa), tc[1]));
          if (!groundOk(m1, m2)) continue;
          const cand = len(sub(a2, a1)); // full length; shrink is test-only
          if (cand < best) best = cand;
        }
```

listing 448: javascript, the 16 edge pairs times both landing sides, crossing parameters solved by cross products (source: icpc/samples-js/src/ch11-pt-carlsvacation.mjs, lines 129-168)

```
def _best_leg(s, ei, k, pa, pb):
    a, b = s.v[ei], s.v[(ei + 1) % 4]
    ex, ey = b[0] - a[0], b[1] - a[1]
    bestv = -1.0
    bestt = -1.0
    for i in range(1025):
        t = i / 1024
        p = (a[0] + ex * t, a[1] + ey * t)
        if not _ground_ok(p, k, pa, pb):
            continue
        f = _slant(s, p) + math.hypot(p[0] - k[0], p[1] - k[1])
        if bestv < 0 or f < bestv:
            bestv = f
            bestt = t
    if bestv < 0:
        return -1.0
    lo = max(0.0, bestt - 1 / 1024)
    hi = min(1.0, bestt + 1 / 1024)
    for _ in range(100):
        m1 = lo + (hi - lo) / 3
        m2 = hi - (hi - lo) / 3
        p1 = (a[0] + ex * m1, a[1] + ey * m1)
        p2 = (a[0] + ex * m2, a[1] + ey * m2)
        f1 = (
            _slant(s, p1) + math.hypot(p1[0] - k[0], p1[1] - k[1])
            if _ground_ok(p1, k, pa, pb)
            else 1e30
        )
        f2 = (
            _slant(s, p2) + math.hypot(p2[0] - k[0], p2[1] - k[1])
            if _ground_ok(p2, k, pa, pb)
            else 1e30
        )
        if f1 ≤ f2:
            hi = m2
        else:
            lo = m1
    p = (a[0] + ex * (lo + hi) / 2, a[1] + ey * (lo + hi) / 2)
    f = (
        _slant(s, p) + math.hypot(p[0] - k[0], p[1] - k[1])
        if _ground_ok(p, k, pa, pb)
        else bestv
    )
    return f if f < bestv else bestv
```

listing 449: python, the same guarded grid-plus-ternary leg, infeasible samples priced at 1e30 (source: icpc/samples-py/src/Ch11/pt.py, lines 79-122)

```
-- minimize slant(s, p(t)) + |p(t) - k| over t in [0, 1] along edge ei
-- with the ground leg valid; grid + ternary
local function best_leg(s, ei, kx, ky)
  local j = ei % 4 + 1
  local ax, ay = s.vx[ei], s.vy[ei]
  local ex, ey = s.vx[j] - ax, s.vy[j] - ay
  local NG = 1024
  local bestv, bestt = -1, -1
  for i = 0, NG do
    local t = i / NG
    local px, py = ax + ex * t, ay + ey * t
    if ground_ok(px, py, kx, ky) then
      local dx, dy = px - kx, py - ky
      local f = slant(s, px, py) + sqrt(dx * dx + dy * dy)
      if bestv < 0 or f < bestv then
        bestv, bestt = f, t
      end
    end
  end
  if bestv < 0 then return -1 end
  local lo = bestt - 1 / NG
  local hi = bestt + 1 / NG
  if lo < 0 then lo = 0 end
  if hi > 1 then hi = 1 end
  for _ = 1, 100 do
    local m1 = lo + (hi - lo) / 3
    local m2 = hi - (hi - lo) / 3
    local p1x, p1y = ax + ex * m1, ay + ey * m1
    local p2x, p2y = ax + ex * m2, ay + ey * m2
    local f1 = ground_ok(p1x, p1y, kx, ky) and
               slant(s, p1x, p1y) + sqrt((p1x - kx) ^ 2 + (p1y - ky) ^ 2) or
               1e30
    local f2 = ground_ok(p2x, p2y, kx, ky) and
               slant(s, p2x, p2y) + sqrt((p2x - kx) ^ 2 + (p2y - ky) ^ 2) or
               1e30
    if f1 ≤ f2 then
      hi = m2
    else
      lo = m1
    end
  end
  local t = (lo + hi) / 2
  local px, py = ax + ex * t, ay + ey * t
  local f = ground_ok(px, py, kx, ky) and
            slant(s, px, py) + sqrt((px - kx) ^ 2 + (py - ky) ^ 2) or bestv
  if f < bestv then return f end
  return bestv
end
```

listing 450: lua, best_leg over flat scalars, no table allocation in the sweep (source: icpc/samples-lua/ch11_pt.lua, lines 87-134)

The crafted fixture is two squares twenty units apart, bases [0,10] and [30,40] on the x axis with heights 4: the apexes land at 10 - sqrt(41) and 30 + sqrt(41) after unfolding across the facing edges, and the horizontal candidate is 20 plus twice sqrt(41) = 32.806248475. Every suite asserts that value, plus the official sample 60.866649532, whose optimum is the corner entry that forced the completion, and the crafted close pair at 15.211102551, all tolerance-checked at 1e-6, three checks per language, java printing its nine decimals through `Locale.ROOT` like its q and v

twins. Input coordinates are integers and everything past parsing is double, so the languages diverge nowhere numerically worth noting.

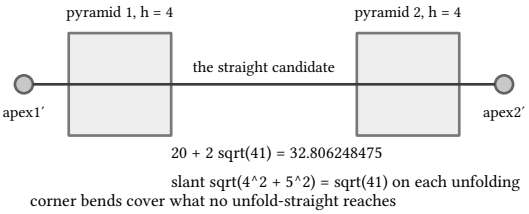


figure 149: the fixture: both squares, the unfolded apexes outside them, one straight candidate crossing both hinges

13.7. u, toy train tracks

The Toy Train Tracks Construction Company sells exactly two piece shapes, straight segments and 90-degree curves, and every piece occupies one cell of a square grid, rotatable in quarter turns. A proper course is a single connected closed loop that never intersects itself. Customers arrive with a bag of s straights and c curves and want the largest course their bag allows: build the longest valid loop and print it (statement).

The input, per the problemset pdf, is one line with s and c, 0 <= s <= 1e5 straight pieces and 4 <= c <= 1e5 curves. The output is one string read off a single traversal of the loop, S for each straight piece, L for a left curve, R for a right curve, and any loop of maximal length is accepted, under the pdf's 1 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `U-toytraintracks/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints beside its picture of the layout: all four straights and all twelve curves spend, sixteen pieces, and the judge's loop is one of several equally long maxima.

input:

```
4 12
```

expected output:

```
LSRLLRLSLSRLLSRL
```

Official sample 2, same source: the odd straight wastes, one curve wastes, and the four remaining curves close the unit ring.

input:

```
1 5
```

expected output:

```
LLLL
```

Recognition: s and c each stop at 1e5 under the 1 second limit, and the output itself is at most s + c = 2e5 characters, so the budget is a formula plus one emission pass, and anything that searches over layouts is already over.

The statement's cue is the largest course their bag allows, build it and print it: a maximum with a printed witness is the classic shape of a closed-form characterization, prove exactly which totals are achievable, then emit one canonical loop. Problem U is the 2022 face of the constructive closed-form characterization family, and the constructive closed-form characterization family holds chapter 13 problem B and chapter 15 problem J.

Search prices out at 4^200000 turn sequences before self-avoidance even enters, and a dp over grid shapes has no budget either, so the achievable-total table plus the canonical strings is the only shape that closes.

Odd budgets waste one piece, s halves to an even count. With no usable straights the achievable totals are exactly 4 and every multiple of 4 from 12 up, built from the unit LLRLLR plus LR repeated, taken twice: totals 6, 8, 10, 14, 18 are impossible, since a closed walk nets +-4 quarter turns and self-avoidance kills every candidate below 12 except 4, verified by exhaustive search through length 18. The constructions follow the solutions pdf pages 5 and 6, problem at icpc2022.pdf. With two or more usable straights, every piece spends: the curve budget drops only to even, c' = c - (c mod 2), and the canonical emission joins two mirrored halves. That corrected budget is a judge-verified fix, not a first draft: the earlier rule that c' must be a multiple of 4 rejects 22 of the 75 official answers, every s >= 2 case with c mod 4 in {2, 3}, while nine of them, secrets 34, 38, 42, 46, 50, 54, 64, 67, and 69, actually made the then-shipped solvers emit short, silently validating suboptimal loops as passes, and the corrected budget accepts 75 of 75, with the obsolete `2 6` to LLSLLS emission as the discriminant. The odd curve count 4q + 6 is the porting trap, found twice in the C# stream: it splits asymmetrically, X S LL swap(X) with the straight split a = 1 then sp/2 - 1 and sp/2 on the two arms, and a naive symmetric split undercounts straights by 2. Every case also runs a grid-walking validator, closed, self-avoiding, within budget, and the go suite sweeps it over 544 budgets.

The worked run: trace the model on sample 1. The bag is s = 4 straights and c = 12 curves, c is already even so c' = 12, and with two or more usable straights every piece spends, a total of 16. The printed witness is one of the equally long maxima: walking LSRLLRLSLSRLLSRL from the origin heading east, turn then step, visits 16 distinct cells and closes at the origin, 8 lefts and 4 rights netting +4 quarter turns over 4 straights, a closed self-avoiding loop that spends the whole bag, and the trace ends at the printed answer `LSRLLRLSLSRLLSRL`.

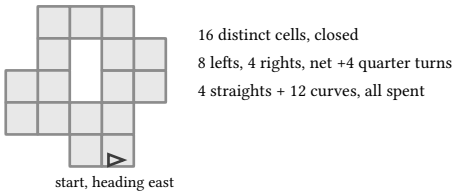


figure 150: sample 1′s sixteen-cell loop walked from the origin heading east, 8 lefts, 4 rights, 4 straights, every piece spent

```
  } else {
    long long cp = c - c % 2; // any even count spends fully
    long long a = s2 / 2 - 1;
    if (a < 0)
      a = 0;
    long long b = s2 / 2 - a;
    if (cp % 4 == 0) {
      long long q = (cp - 4) / 4;
      for (int side = 0; side < 2; side++) {
        out[w++] = 'L';
        for (long long i = 0; i < a; i++)
          out[w++] = 'S';
        out[w++] = 'L';
        for (long long i = 0; i < q; i++) {
          out[w++] = 'R';
          out[w++] = 'L';
        }
        for (long long i = 0; i < b; i++)
          out[w++] = 'S';
      }
    } else {
      // curve count 4q+6: X S LL swap(X) S^a LL S^b
      long long q = (cp - 6) / 4;
      for (long long i = 0; i < q; i++) {
        out[w++] = 'L';
        out[w++] = 'R';
      }
      out[w++] = 'L';
      out[w++] = 'S';
      out[w++] = 'L';
      out[w++] = 'L';
      for (long long i = 0; i < q; i++) {
        out[w++] = 'R';
        out[w++] = 'L';
      }
      out[w++] = 'R';
      for (long long i = 0; i < s2 / 2 - 1; i++)
        out[w++] = 'S';
      out[w++] = 'L';
      out[w++] = 'L';
      for (long long i = 0; i < s2 / 2; i++)
        out[w++] = 'S';
    }
  }
```

listing 451: c, the straight forms: mirrored halves when 4 divides c', the asymmetric 4q+6 split otherwise (source: icpc/samples-c/src/Ch11/pU.c, lines 55-98)

```
// solveUExpected is the achievable piece count the case table pins.
func solveUExpected(s, c int) int {
  sp := 2 * (s / 2)
  if sp == 0 {
    cp := 4 * (c / 4)
    if cp == 8 {
      return 4
    }
    return cp
  }
  return sp + c - c&1
}

// solveUValid walks the loop on the grid: closed, self-avoiding, and
// within the piece budget.
func solveUValid(route string, s, c int) bool {
  if len(route) != solveUExpected(s, c) {
    return false
  }
  x, y, dx, dy := 0, 0, 1, 0
  seen := map[[2]int]bool{{0, 0}: true}
  ns, nc := 0, 0
  for i := 0; i < len(route); i++ {
    switch route[i] {
    case 'L':
      dx, dy = -dy, dx
      nc++
    case 'R':
      dx, dy = dy, -dx
      nc++
    case 'S':
      ns++
    default:
      return false
    }
    x += dx
    y += dy
    if seen[[2]int{x, y}] {
      // only the final step may return to the start cell
      if !(x == 0 && y == 0 && i == len(route)-1) {
        return false
      }
    }
    seen[[2]int{x, y}] = true
  }
  return x == 0 && y == 0 && dx == 1 && dy == 0 && ns <= s && nc <= c
}
```

listing 452: go, the achievable count the table pins and the grid walker that validates every emitted loop (source: icpc/samples-go/ch11/pu.go, lines 60-106)

```
    long s = nextLong(in);
    long c = nextLong(in);
    long s2 = 2 * (s / 2);
    long c4 = c / 4 * 4;
    StringBuilder sb = new StringBuilder((int) (s2 + c + 8));
    if (s2 == 0) {
      if (c4 == 8)
        c4 = 4;
      if (c4 < 12) {
        sb.append("LLLL"); // a 4-curve square
      } else {
        long q = (c4 - 12) / 4;
        for (int side = 0; side < 2; side++) {
          sb.append("LLRLLR");
          for (long i = 0; i < q; i++)
            sb.append("LR");
        }
      }
    } else {
      long cp = c - c % 2; // any even count spends fully
      long a = s2 / 2 - 1;
      if (a < 0)
        a = 0;
      long b = s2 / 2 - a;
      if (cp % 4 == 0) {
        long q = (cp - 4) / 4;
        for (int side = 0; side < 2; side++) {
          sb.append('L');
          for (long i = 0; i < a; i++)
            sb.append('S');
          sb.append('L');
          for (long i = 0; i < q; i++)
            sb.append("RL");
          for (long i = 0; i < b; i++)
            sb.append('S');
        }
      } else {
        // curve count 4q+6: X S LL swap(X) S^a LL S^b
        long q = (cp - 6) / 4;
        for (long i = 0; i < q; i++)
          sb.append("LR");
        sb.append('L').append('S').append("LL");
        for (long i = 0; i < q; i++)
          sb.append("RL");
        sb.append('R');
        for (long i = 0; i < s2 / 2 - 1; i++)
          sb.append('S');
        sb.append("LL");
        for (long i = 0; i < s2 / 2; i++)
          sb.append('S');
      }
    }
    sb.append('\n');
    return sb.toString();
```

listing 453: java, the emission forms: mirrored StringBuilder halves when 4 divides c', the asymmetric 4q+6 split otherwise, the pure-curve ring beside them (source: icpc/samples-java/src/Ch12/PU.java, lines 39-92)

```
        if (se ≥ 2)
        {
            // with at least two usable straights every piece spends:
            // the curve budget only needs to be even
            int cp = c - c % 2;
            int a = Math.Max(0, se / 2 - 1);
            int b = se / 2 - a;
            var sb = new System.Text.StringBuilder();
            if (cp % 4 == 0)
            {
                var half = new System.Text.StringBuilder();
                half.Append('L');
                half.Append('S', a);
                half.Append('L');
                for (int i = 0; i < (cp - 4) / 4; i++)
                    half.Append("RL");
                half.Append('S', b);
                string h = half.ToString();
                sb.Append(h).Append(h);
            }
            else
            {
                // curve count 4q+6: X + S + LL + swap(X) + S^a + LL + S^b
                // with X = (LR)^q L and swap(X) = (RL)^q R
                int q = (cp - 6) / 4;
                for (int i = 0; i < q; i++)
                    sb.Append("LR");
                sb.Append('L');
                sb.Append('S');
                sb.Append("LL");
                for (int i = 0; i < q; i++)
                    sb.Append("RL");
                sb.Append('R');
                sb.Append('S', Math.Max(0, se / 2 - 1));
                sb.Append("LL");
                sb.Append('S', se / 2);
            }
            return sb + "\n";
```

listing 454: c#, the same two forms, StringBuilder halves joined back to back (source: icpc/samples/src/Ch11/PU.cs, lines 24-61)

```javascript
export function solve(input) {
  const parts = input.trim().split(/\s+/);
  const s = parseInt(parts[0], 10);
  const c = parseInt(parts[1], 10);
  const sp = 2 * Math.floor(s / 2);
  if (sp === 0) {
    let cp = 4 * Math.floor(c / 4);
    if (cp === 8) cp = 4;
    if (cp === 4) return "LLLL\n";
    const unit = "LLRLLR" + rep("LR", (cp - 12) / 4);
    return unit + unit + "\n";
  }
  const cp = c - (c & 1); // any even count spends fully with straights
  const a = Math.max(0, sp / 2 - 1);
  const b = sp / 2 - a;
  if (cp % 4 === 0) {
    const half = "L" + rep("S", a) + "L" + rep("RL", (cp - 4) / 4) + rep("S", b);
    return half + half + "\n";
  }
  // curve count 4q+6: X + S + LL + swap(X) + S^a + LL + S^b
  const q = (cp - 6) / 4;
  const x = rep("LR", q) + "L";
  const y = rep("RL", q) + "R";
  return x + "S" + "LL" + y + rep("S", sp / 2 - 1) + "LL" + rep("S", sp / 2) + "\n";
}
```

listing 455: javascript, the whole solve: even c' spends fully, one string per shape (source: icpc/samples-js/src/ch11-pu-toytraintracks.mjs, lines 16-40)

```python
def solve(text):
    s, c = (int(v) for v in text.split())
    sp = 2 * (s // 2)
    if sp == 0:
        cp = 4 * (c // 4)
        if cp < 12:
            return "LLLL\n"
        unit = "LLRLLR" + "LR" * ((cp - 12) // 4)
        return unit + unit + "\n"
    cp = c - c % 2
    a = max(0, sp // 2 - 1)
    b = sp // 2 - a
    if cp % 4 == 0:
        half = "L" + "S" * a + "L" + "RL" * ((cp - 4) // 4) + "S" * b
        return half + half + "\n"
    # curve count 4q+6: X + S + LL + swap(X) + S^(sp/2-1) + LL + S^(sp/2)
    q = (cp - 6) // 4
    x = "LR" * q + "L"
    y = "RL" * q + "R"
    return x + "S" + "LL" + y + "S" * (sp // 2 - 1) + "LL" + "S" * (sp // 2) + "\n"
```

listing 456: python, the same emission in five string expressions (source: icpc/samples-py/src/Ch11/pu.py, lines 12-31)

```lua
-- walk the route on the grid: closed, self-avoiding, within budget
local function valid(route, s, c)
  if #route ~= expected(s, c) then return false end
  local x, y, dx, dy = 0, 0, 1, 0
  local seen = {["0 0"] = true}
  local ns, nc = 0, 0
  for i = 1, #route do
    local ch = route:byte(i)
    if ch == 76 then
      dx, dy = -dy, dx
      nc = nc + 1
    elseif ch == 82 then
      dx, dy = dy, -dx
      nc = nc + 1
    elseif ch == 83 then
      ns = ns + 1
    else
      return false
    end
    x = x + dx
    y = y + dy
    local key = x .. " " .. y
    if seen[key] then
      if not (x == 0 and y == 0 and i == #route) then return false end
    end
    seen[key] = true
  end
  return x == 0 and y == 0 and dx == 1 and dy == 0 and ns ≤ s and nc ≤ c
end
```

listing 457: lua, the validator: walk L as turn-then-step, assert closed, distinct cells, budget (source: icpc/samples-lua/ch11_pu.lua, lines 80-108)

The fixture is 0 straights and 12 curves, the full ring LLRLLRLLRLLR: walking it from the origin heading east, 8 lefts and 4 rights net +4 quarter turns and the twelve cells are distinct. Every suite asserts that string, plus the official `1 5` collapsing to LLLL, the corrected `2 6` spending all six curves as LSLLRLLS for a total of 8, and `4 12` as LSLRLRLSLSLRLRLS. Counts stay under 1e5 pieces, so no language diverges on integers. Java ships the same closed-walk validator go and lua carry, its two extra checks asserting the 4 12 and 0 16 rings close self-avoidingly.

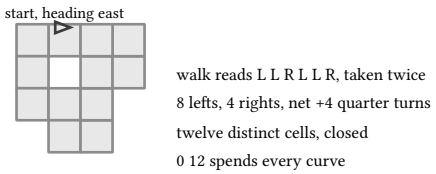


figure 151: the fixture's twelve-cell ring, start and heading marked

13.8. v, three kinds of dice

The Buffett and Gates dice story: pick from three intransitive dice and the second chooser always wins. A die is any collection of one or more faces, each face a positive integer, with one face drawn uniformly per roll. When two dice roll against each other, the higher face earns its die 1 point and equal faces earn each die half a point, and score(D, D') is the expected points D earns per roll, so D has an advantage over D' when score(D, D') > 1/2 and the dice tie at exactly 1/2. Given two dice where one has an advantage over the other, whichever order they are listed in, the advantaged one is D1. Among all dice D3 that have an advantage over or tie with D1, report the lowest score D3 can manage against D2, and among all D3 that D2 has an advantage over or ties with, report the highest score D3 can manage against D1: a first value under 1/2 beside a second at or above it is an intransitive

trio. The two answers may come from different dice (statement). Three Kinds of Dice is one of the five problem names this finals shares with the 2023 set: this set's V is 2023's C retold.

The input, per the problemset pdf, is two lines, one per die in either order, each holding its face count n with 1 <= n <= 1e5 and then the n face values, each between 1 and 1e9, and the pdf promises one of the two really has the advantage. The output is the two scores on one line with an absolute error of at most 1e-6, under the pdf's 1 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `V-threekindsofdice/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: the 6-face die listed first loses the matchup, winning only 8 of 18 pairings, so the advantaged D1 is the 3-face die listed second, the trap that pins the order test.

input:

```
6 1 1 6 6 8 8
3 2 4 9
```

expected output:

```
0.291666667 0.750000000
```

Official sample 2, same source: the 4-face die wins 10 of 12 pairings and is D1, and both queries land at one half.

input:

```
4 9 3 7 5
3 4 2 3
```

expected output:

```
0.500000000 0.500000000
```

Recognition: both dice carry at most 1e5 faces under the 1 second limit. Doubled score coordinates make every achievable point an integer pair, the candidate face values number about 2n1 + 2n2, 4e5 of them, and a sort plus one monotone-chain hull costs about 4e6 comparisons, two orders of magnitude inside budget.

The statement's cue is the pair of extrema over all dice D3 subject to a tie-or-beat constraint: choosing D3 is choosing a distribution over face values, so both queries are constrained optima over one achievable set, the cue that the set is a convex hull and each query a halfplane cut on it. Problem V is the 2022 face of the convex hull of achievable points family, and chapter 14 problem C is the other convex hull of achievable points twin.

Enumerating dice prices out at unbounded multisets over values to 1e9, and a linear program discretized over every value still pays 1e9 weights, so the 4e5-candidate hull is the only budget that closes.

Die D1 beats die D2, and the input may list them in either order, so the first move is detecting the advantage by the exact pairwise score, doubled to 2 per win and 1 per tie, since the official sample 1 lists the winning die second. A D3 face showing v earns D1 an expected S1(v) = wins plus half the ties, likewise S2(v) against D2, and choosing D3 is choosing a distribution over face values, so the achievable average pairs are exactly the convex hull of the points (S1(v), S2(v)). Part 1 maximizes the average against D2 subject to tying or beating D1, the hull's highest y at x <= n1/2, and part 2 minimizes the average against D1 subject to D2 tying or beating D3, the leftmost x at y >= n2/2, both read off the hull with exact edge interpolation, and the score is 1 - avg/n, the model on the solutions pdf page 6, problem at icpc2022.pdf. The candidate face values are a judge-corrected detail: for each distinct union value u the solver generates both u and u + 1, plus one below the minimum when that stays positive and one past the maximum, because S1 and S2 step at each die's own thresholds. All coordinates double, so half-integer expectations stay exact integers on the hull and floats appear only in the final division and print.

The worked run: trace the model on sample 1. The 6-face die wins only 8 of 18 pairings against the 3-face die, 8/18 = 4/9 < 1/2, so D1 is the listed-second die (2, 4, 9) and D2 is (1, 1, 6, 6, 8, 8). A D3 face showing v earns the doubled integers X(v) = 2 wins + tie against D1 and Y(v) against D2, and the candidates run v = 1 to 10, the value 0 below the union minimum dropped because faces are positive. The upper hull of the ten points has three vertices, (0, 0) at v = 10, (2, 8) at v = 5 and (6, 10) at v = 1. Part 1 lets D3 tie D1, X <= n1 = 3, and the hull's best Y sits on the edge from (2, 8) to (6, 10): at X = 3 it reads Y = 8.5, witnessed by the die (5, 5, 5, 1), which ties D1 at exactly 1/2 and holds D2 to 17/24, so score(D3, D2) = 1 - 8.5/12 = 7/24. Part 2 lets D2 tie D3, Y >= n2 = 6, and the hull's lowest X sits on the edge from (0, 0) to (2, 8) at X = 1.5, witnessed by (5, 5, 5, 10), which ties D2 at 1/2 and loses only 1/4 to D1, so score(D3, D1) = 1 - 1.5/6 = 3/4, and the trace ends at the printed answer `0.291666667 0.750000000`.

face v	X(v)	Y(v)	hull place
1	6	10	vertex
2	5	8	below the top edge
3	4	8	below the top edge
4	3	8	below the top edge
5	2	8	vertex
6	2	6	inside
7	2	4	inside
8	2	2	inside
9	1	0	inside
10	0	0	vertex

```c
qsort(pt, (size_t)np, sizeof(Pt), cmp_pt);
// upper hull, x ascending, y concave
Pt *hull = malloc((size_t)np * sizeof(Pt));
int nh = 0;
for (int i = 0; i < np; i++) {
  while (nh ≥ 2 && cross3(hull[nh - 2], hull[nh - 1], pt[i]) ≥ 0)
    nh--;
  hull[nh++] = pt[i];
}
// part 1: max y at x ≤ n1 (doubled bound)
double ymax;
int peak = 0;
for (int i = 1; i < nh; i++)
  if (hull[i].y > hull[peak].y)
    peak = i;
if (hull[peak].x ≤ n1) {
  ymax = (double)hull[peak].y;
} else {
  int i = peak;
  while (i > 0 && hull[i - 1].x > n1)
    i--;
  if (i > 0) {
    Pt a = hull[i - 1], b = hull[i]; // edge straddling x = n1
    ymax = (double)a.y +
           (double)(b.y - a.y) * (double)(n1 - a.x) / (double)(b.x - a.x);
  } else {
    ymax = (double)hull[0].y;
  }
}
// part 2: min x at y ≥ n2 (doubled bound)
double xmin;
int j0 = -1;
for (int i = 0; i < nh; i++)
  if (hull[i].y ≥ n2) {
    j0 = i;
    break;
  }
if (j0 ≤ 0) {
  xmin = (double)hull[0].x; // leftmost already qualifies
} else {
  Pt a = hull[j0 - 1], b = hull[j0];
  xmin = (double)a.x +
         (double)(b.x - a.x) * (double)(n2 - a.y) / (double)(b.y - a.y);
}
double ans1 = 1.0 - ymax / (2.0 * n2);
double ans2 = 1.0 - xmin / (2.0 * n1);
```

listing 458: c, the upper hull over doubled points, then both queries with edge-crossing interpolation (source: icpc/samples-c/src/Ch11/pV.c, lines 141-186)

```
  hull := monoChainHull(pts)

  // part 1: max Y over the hull with X ≤ n1 (doubled: avg S1 ≤
  // n1/2 means avg doubled X ≤ n1); edge crossings interpolate
  // exactly as floats (the magnitudes stay well under 2^53)
  maxY := -1.0
  for i := 0; i < len(hull); i++ {
    p, q := hull[i], hull[(i+1)%len(hull)]
    x1, y1, x2, y2 := p[0], p[1], q[0], q[1]
    if x1 > n1 && x2 > n1 {
      continue
    }
    if x1 <= n1 && float64(y1) > maxY {
      maxY = float64(y1)
    }
    if (x1-n1)*(x2-n1) < 0 {
      yf := float64(y1) + float64(y2-y1)*float64(n1-x1)/float64(x2-x1)
      if yf > maxY {
        maxY = yf
      }
    }
  }
  // part 2: min X over the hull with Y ≥ n2
  minX := -1.0
  for i := 0; i < len(hull); i++ {
    p, q := hull[i], hull[(i+1)%len(hull)]
    x1, y1, x2, y2 := p[0], p[1], q[0], q[1]
    if y1 < n2 && y2 < n2 {
      continue
    }
    if y1 ≥ n2 && (minX < 0 || float64(x1) < minX) {
      minX = float64(x1)
    }
    if (y1-n2)*(y2-n2) < 0 {
      xf := float64(x1) + float64(x2-x1)*float64(n2-y1)/float64(y2-y1)
      if minX < 0 || xf < minX {
        minX = xf
      }
    }
  }
  var out strings.Builder
  fmt.Fprintf(&out, "%.9f %.9f\n", 1-maxY/(2*float64(n2)),
    1-minX/(2*float64(n1)))
  return out.String()
```

listing 459: go, both hull queries as edge walks over the full monotone chain, crossings interpolated as floats (source: icpc/samples-go/ch11/pv.go, lines 99-142)

```
    // upper hull, x ascending, y concave
    Pt[] hull = new Pt[np];
    int nh = 0;
    for (int i = 0; i < np; i++) {
      while (nh ≥ 2 && cross3(hull[nh - 2], hull[nh - 1], sorted[i]) ≥ 0)
        nh--;
      hull[nh++] = sorted[i];
    }
    // part 1: max y at x ≤ n1 (doubled bound)
    double ymax;
    int peak = 0;
    for (int i = 1; i < nh; i++)
      if (hull[i].y > hull[peak].y)
        peak = i;
    if (hull[peak].x ≤ n1) {
      ymax = hull[peak].y;
    } else {
      int i = peak;
      while (i > 0 && hull[i - 1].x > n1)
        i--;
      if (i > 0) {
        Pt a = hull[i - 1], b = hull[i]; // edge straddling x = n1
        ymax = a.y + (double) (b.y - a.y) * (n1 - a.x) / (b.x - a.x);
      } else {
        ymax = hull[0].y;
      }
    }
    // part 2: min x at y ≥ n2 (doubled bound)
    double xmin;
    int j0 = -1;
    for (int i = 0; i < nh; i++)
      if (hull[i].y ≥ n2) {
        j0 = i;
        break;
      }
    if (j0 ≤ 0) {
      xmin = hull[0].x; // leftmost already qualifies
    } else {
      Pt a = hull[j0 - 1], b = hull[j0];
      xmin = a.x + (double) (b.x - a.x) * (n2 - a.y) / (b.y - a.y);
    }
    double ans1 = 1.0 - ymax / (2.0 * n2);
    double ans2 = 1.0 - xmin / (2.0 * n1);
    return String.format(Locale.ROOT, "%.9f %.9f", ans1, ans2) + "\n";
```

listing 460: java, the upper hull over doubled points keeping right turns, then both queries with edge-crossing interpolation (source: icpc/samples-java/src/Ch12/PV.java, lines 145-188)

```
        // candidate face values: each distinct union value contributes
        // its tie point (strictly-greater plus half the ties) and, when
        // an integer fits before the next distinct value (or past the
        // maximum), its strict point (the next integer). doubled coords.
        var vals = new long[n1 + n2];
        int vc = 0;
        for (int i = 0; i < n1; i++)
            vals[vc++] = d1[i];
        for (int i = 0; i < n2; i++)
            vals[vc++] = d2[i];
        Array.Sort(vals, 0, vc);
        var pts = new List<(long, long)>();
        for (int i = 0; i < vc; i++)
        {
            long v = vals[i];
            if (i > 0 && v == vals[i - 1])
                continue;
            long gt1 = n1 - UpperBound(d1, v);
            long eq1 = UpperBound(d1, v) - LowerBound(d1, v);
            long gt2 = n2 - UpperBound(d2, v);
            long eq2 = UpperBound(d2, v) - LowerBound(d2, v);
            pts.Add((2 * gt1 + eq1, 2 * gt2 + eq2));
            int j = i + 1;
            while (j < vc && vals[j] == v)
                j++;
            long next = j < vc ? vals[j] : long.MaxValue;
            if (next - v ≥ 2)
                pts.Add((2 * gt1, 2 * gt2));
        }
        // any legal face below the union minimum gives one more point
        if (vals[0] > 1)
            pts.Add((2 * n1, 2 * n2));
        var ptArr = pts.ToArray();
```

listing 461: c#, candidate generation: each distinct value contributes its tie point and, across a gap, its strict point (source: icpc/samples/src/Ch11/PV.cs, lines 49-81)

```
// D1 is whichever die has the advantage; detect it by the exact
// pairwise score (doubled: 2 per win, 1 per tie per matchup)
let pts = 0;
{
  let j = 0;
  for (let i = 0; i < na; i++) {
    while (j < nb && db[j] < da[i]) j++;
    let eq = 0;
    for (let jj = j; jj < nb && db[jj] === da[i]; jj++) eq++;
    pts += 2 * j + eq;
  }
}
const d1 = pts > na * nb ? da : db;
const d2 = pts > na * nb ? db : da;
const n1 = d1.length, n2 = d2.length;
```

listing 462: javascript, the advantage test: a merge-counted doubled score decides which die is D1 (source: icpc/samples-js/src/ch11-pv-threekindsofdice.mjs, lines 35-49)

```
    # upper hull, x ascending, keep right turns
    hull = []
    for q in pts2:
        while len(hull) ≥ 2 and (
            (hull[-1][0] - hull[-2][0]) * (q[1] - hull[-2][1])
            - (hull[-1][1] - hull[-2][1]) * (q[0] - hull[-2][0])
            ≥ 0
        ):
            hull.pop()
        hull.append(q)
    # part 1: max y at x ≤ n1 (doubled bound)
    peak = 0
    for i in range(1, len(hull)):
        if hull[i][1] > hull[peak][1]:
            peak = i
    if hull[peak][0] ≤ n1:
        ymax = float(hull[peak][1])
    else:
        i = peak
        while i > 0 and hull[i - 1][0] > n1:
            i -= 1
        if i > 0:
            a, b = hull[i - 1], hull[i]
            ymax = a[1] + (b[1] - a[1]) * (n1 - a[0]) / (b[0] - a[0])
        else:
            ymax = float(hull[0][1])
    # part 2: min x at y ≥ n2 (doubled bound)
    j0 = -1
    for i in range(len(hull)):
        if hull[i][1] ≥ n2:
            j0 = i
            break
    if j0 ≤ 0:
        xmin = float(hull[0][0])
    else:
        a, b = hull[j0 - 1], hull[j0]
        xmin = a[0] + (b[0] - a[0]) * (n2 - a[1]) / (b[1] - a[1])
    return f"{1 - ymax / (2 * n2):.9f} {1 - xmin / (2 * n1):.9f}\n"
```

listing 463: python, the upper hull keeping right turns, then the peak and the crossing edge (source: icpc/samples-py/src/Ch11/pv.py, lines 76-113)

```
local idx = {}
for i = 1, np do idx[i] = i end
table.sort(idx, function(a, b)
  if px[a] ~= px[b] then return px[a] < px[b] end
  return py[a] < py[b]
end)
local hx, hy = {}, {}
local nh = 0
for ii = 1, np do
  local i = idx[ii]
  local x, y = px[i], py[i]
  while nh ≥ 2 do
    local ox, oy = hx[nh - 1], hy[nh - 1]
    local ax, ay = hx[nh], hy[nh]
    if (ax - ox) * (y - oy) - (ay - oy) * (x - ox) ≥ 0 then
      nh = nh - 1
    else
      break
    end
  end
  nh = nh + 1
  hx[nh], hy[nh] = x, y
end
-- part 1: max y at x ≤ n1 (all doubled)
local peak = 1
for i = 2, nh do
  if hy[i] > hy[peak] then peak = i end
end
local ymax
if hx[peak] ≤ n1 then
  ymax = hy[peak]
else
  local i = peak
  while i > 1 and hx[i - 1] > n1 do i = i - 1 end
  if i > 1 then
    local ax, ay = hx[i - 1], hy[i - 1]
    local bx, by = hx[i], hy[i]
    ymax = ay + (by - ay) * (n1 - ax) / (bx - ax)
  else
    ymax = hy[1]
  end
end
```

listing 464: lua, the hull over parallel x and y arrays sorted through an index table (source: icpc/samples-lua/ch11_pv.lua, lines 117-158)

The fixture is D1 = (2, 6) against D2 = (1, 1): every candidate point lies under the hull edge from (0, 0) to (2, 1), so the best D3 that ties D1 still scores 0.75 against D2, witnessed by the die (1, 7), and part 2 forces all-ones, which loses every matchup against D1, for 0. Every suite asserts `0.750000000 0.000000000` on that input, tolerance-checked at 1e-6, and the C, C#, java, javascript, python, and lua suites also pin both official samples, `0.291666667 0.750000000` and `0.500000000 0.500000000`; the go tolerance table carries the fixture alone, its three rows for t and one for v covering the family's float side.

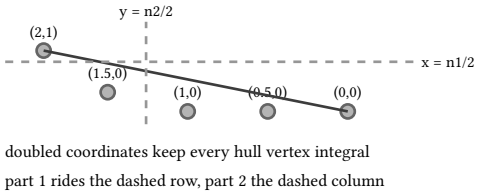


figure 152: the fixture's (S1, S2) points, the hull, and the two query lines

13.9. w, riddle of the sphinx

The sphinx of Thebes asked Oedipus a riddle about legs. This problem hands the riddle to us in reverse. An axex, a basilisk, and a centaur stand before a sphinx, and their leg counts are unknown nonnegative integers we may not look up. The sphinx grants exactly five question rounds, each asking how many legs some number of the creatures have in total, and at most one of her five answers is an outright lie we cannot identify in advance. Name the three leg counts (statement). Riddle of the Sphinx is one of the five problem names this finals shares with the 2023 set: this set's W is 2023's A retold, and the two years' judge files are byte-identical across all 50 cases. The interaction, laid out by the problemset pdf, is line based: each round we write `a b c` with each of a, b, c between 0 and 10, the question a axexes plus b basilisks plus c centaurs, and one integer r between 0 and 1e5 comes back as her answer, and after the fifth reply we print `la lb lc`, all under the pdf's 2 second limit.

The judge data scripts that interaction rather than piping it, so each `.in` file is a four-line driver spec and every `.ans` file is empty, 0 bytes: an interactive runner spawns the solver, answers its five questions, and checks only the final line, which is why W can never byte-diff. Official sample 1, reprinted byte for byte from `W-riddleofthesphinx/sample-1.in`, the layout read in full below:

input:

```
fixed
4 4 4
1 1
4 4 4
```

Line 2 is the true triple, here (4, 4, 4), line 3 lies on the second round with delta +1, so the question (0,1,0) draws 5 instead of 4, and line 4 repeats the truth, so the runner must see `4 4 4` printed. Sample 2 carries legs (0, 42, 2024) with delta −6241, so scripted answers may go negative.

Recognition: five rounds and answers up to 1e5 under the 2 second limit make the budget trivial, six exclusion hypotheses each solved by Cramer's rule on a 3 by 3 system, microseconds of work, so the whole difficulty is the model, not the cost.

The statement's cue is at most one of her five answers is an outright lie we cannot identify in advance: the consistent world is recovered by excluding each suspect row in turn and keeping the one hypothesis whose survivors all agree, the cue of a faulty-oracle consistency problem. Problem W is the 2022 face of the consistency under a faulty oracle family, and consistency under a faulty oracle ties chapter 14 problem A and chapter 15 problem D together.

The tempting alternative, spending rounds hunting the lie before naming legs, breaks the five-question budget the deduction needs, and brute force over leg triples is (1e5)^3 = 1e15 candidates, so the exclusion machine is the only shape that closes.

The sphinx answers five questions of the form (a, b, c) with the dot product of the question and the three leg counts, and at most one answer is an arbitrary lie. The five questions (1,0,0), (0,1,0), (0,0,1), (1,1,1), (1,2,3) have the property that any three are linearly independent, so for each exclusion hypothesis, the lie hit round k or there is no lie, some three surviving rows determine the leg counts by Cramer's rule, and exactly one hypothesis leaves a system that also satisfies every other surviving row, the argument on the solutions pdf page 7, problem at icpc2022.pdf. The pure core is `deduce(answers)`, answers in, legs out, trying each exclusion hypothesis and returning the surviving

triple. The judge data scripts the interaction over four lines: line 2 carries the TRUE leg counts, line 3 the 0-based lying round, 5 meaning never, plus an additive delta, and line 4 alternate leg counts. Every answer is computed truthfully from line 2 except the lying round, which she answers from the line-4 triple plus the delta: the 30 use-alt-sol secrets put a different triple on line 4 with delta 0, the swap being the lie itself, while the other 20 secrets and both samples carry line 4 equal to line 2 with a real delta, sample 2 using −6241 so scripted answers may go negative, and the correct output is always the line-2 triple. This section first landed the reading backwards, truth on line 4 and a decoy on line 2, and that validation was circular: the retired sphinx:4 compare mode echoed whichever line the solver read, so the wrong model confirmed itself. Re-adjudicated against the judge data, with the 2022 W and 2023 A judge files byte-identical across all 50 cases, the fix wave re-landed all six languages, c in 64d294b, python and lua in dbc8658, go and javascript in fc54da4, and c# in 1367ea9, and 2022 W judges 50 of 50 in every one of them post-fix on main.

The worked run: trace the model on sample 1, whose official answer files are empty because the judge scripts the interaction, so the trace targets the prose-stated output, the line-2 triple 4 4 4 the runner must see printed. The five questions are (1,0,0), (0,1,0), (0,0,1), (1,1,1), (1,2,3), and the driver scripts the answers truthfully from (4, 4, 4) except the second round, which draws the delta +1: the answers read 4, 5, 4, 12, 24, the fifth truthful at 4 + 8 + 12 = 24. Excluding each round in turn, rows 1, 3, 4 and 5 solve to la = 4, lc = 4, lb = 12 - 8 = 4, and row 5 checks 4 + 2*4 + 3*4 = 24 exactly, the one consistent hypothesis, while every other exclusion leaves a surviving row off by 1 or 2. The recovered legs are 4 4 4, the line-2 truth.

lie assumed at	solved triple	leftover check
no lie	(4, 5, 4)	row 4 reads 13 against 12, row 5 reads 26 against 24
round 1	(3, 5, 4)	row 5 reads 25 against 24
round 2	(4, 4, 4)	row 5 reads 24, consistent
round 3	(4, 5, 3)	row 5 reads 23 against 24
round 4	(4, 5, 4)	row 5 reads 26 against 24
round 5	(4, 5, 4)	row 4 reads 13 against 12

```
static char *solve(const char *in) {
  char *buf = dupstr(in);
  Ch11Scan sc;
  ch11_scan_init(&sc, buf);
  ch11_scan_skip(&sc);
  while (*sc.p && *sc.p ≠ ' ' && *sc.p ≠ '\n')
    sc.p++;                              // "fixed"
  long long ta = ch11_next_int64(&sc); // true triple on line 2
  long long tb = ch11_next_int64(&sc);
  long long tc = ch11_next_int64(&sc);
  long long rnd = ch11_next_int64(&sc);
  long long delta = ch11_next_int64(&sc);
  long long aa = ch11_next_int64(&sc); // alternate triple on line 4
  long long ab = ch11_next_int64(&sc);
  long long ac = ch11_next_int64(&sc);
  free(buf);
  long long ans[5];
  for (int i = 0; i < 5; i++)
    ans[i] = QX[i] * ta + QY[i] * tb + QZ[i] * tc;
  if (rnd ≥ 0 && rnd < 5)
    ans[rnd] = QX[rnd] * aa + QY[rnd] * ab + QZ[rnd] * ac + delta;
  long long got[3];
  if (!deduce(ans, got))
    return 0;
  char *out = malloc(64);
  snprintf(out, 64, "%lld %lld %lld\n", got[0], got[1], got[2]);
  return out;
}
```

listing 465: c, the file mode: line 2 truth, line 3 round and delta, line 4 alternate, one answer overwritten (source: icpc/samples-c/src/Ch11/pW.c, lines 99-126)

```
func SolveW(input string) string {
  lines := strings.Split(strings.TrimSpace(input), "\n")
  nums := make([]int64, 0, 8)
  for _, tok := range strings.Fields(strings.Join(lines[1:], " ")) {
    if v, err := strconv.ParseInt(tok, 10, 64); err == nil {
      nums = append(nums, v)
    }
  }
  // nums: true legs (3), lie round, lie delta, alternate legs (3)
  rnd, delta := int(nums[3]), nums[4]
  rows := [5][3]int64{{1, 0, 0}, {0, 1, 0}, {0, 0, 1}, {1, 1, 1}, {1, 2, 3}}
  var ans [5]int64
  for i := range ans {
    ans[i] = rows[i][0]*nums[0] + rows[i][1]*nums[1] + rows[i][2]*nums[2]
  }
  if rnd ≥ 0 && rnd < 5 {
    ans[rnd] = rows[rnd][0]*nums[5] + rows[rnd][1]*nums[6] +
      rows[rnd][2]*nums[7] + delta
  }
```

listing 466: go, tokens read straight: true legs, lie spec, alternate legs, the lie applied to one answer (source: icpc/samples-go/ch11/pw.go, lines 20-38)

```
  static String solve(String in) {
    pos = 0;
    while (pos < in.length() && in.charAt(pos) ≠ 'f')
      pos++;
    while (pos < in.length() && in.charAt(pos) ≠ ' ' && in.charAt(pos) ≠ '\n')
      pos++; // "fixed"
    long ta = nextLong(in); // true triple on line 2
    long tb = nextLong(in);
    long tc = nextLong(in);
    long rnd = nextLong(in);
    long delta = nextLong(in);
    long aa = nextLong(in); // alternate triple on line 4
    long ab = nextLong(in);
    long ac = nextLong(in);
    long[] ans = new long[5];
    for (int i = 0; i < 5; i++)
      ans[i] = QX[i] * ta + QY[i] * tb + QZ[i] * tc;
    if (rnd ≥ 0 && rnd < 5)
      ans[(int) rnd] = QX[(int) rnd] * aa + QY[(int) rnd] * ab + QZ[(int) rnd] * ac + delta;
    long[] got = new long[3];
    if (!deduce(ans, got))
      return null;
    return got[0] + " " + got[1] + " " + got[2] + "\n";
```

listing 467: java, the file mode: line 2 truth, line 3 round and delta, line 4 alternate, one answer overwritten, null when no hypothesis fits (source: icpc/samples-java/src/Ch12/PW.java, lines 120-142)

```
    public static string Solve(string input)
    {
        int pos = 0;
        while (pos < input.Length && input[pos] ≠ '\n')
            pos++;
        pos++;
        long ta = Next(input, ref pos);   // true triple on line 2
        long tb = Next(input, ref pos);
        long tc = Next(input, ref pos);
        long rnd = Next(input, ref pos);  // lie round (5 = never)
        long delta = Next(input, ref pos);
        long aa = ta, ab = tb, ac = tc;   // crafted three-line fixtures: no line 4
        if (pos < input.Length)
        {
            long x = Next(input, ref pos);
            if (pos < input.Length)
            {
                aa = x;
                ab = Next(input, ref pos);
                ac = Next(input, ref pos);
            }
        }
        var ans = new long[5];
        for (int i = 0; i < 5; i++)
            ans[i] = Qx[i] * ta + Qy[i] * tb + Qz[i] * tc;
        if (rnd ≥ 0 && rnd < 5)
            ans[rnd] = Qx[rnd] * aa + Qy[rnd] * ab + Qz[rnd] * ac + delta;
        var got = Deduce(ans);
        return got == null ? "" : $"{got[0]} {got[1]} {got[2]}\n";
    }
```

listing 468: c#, the same file mode, crafted three-line fixtures defaulting to the line-2 triple (source: icpc/samples/src/Ch11/PW.cs, lines 26-55)

```javascript
export function solve(input) {
  const tok = input.trim().split(/\s+/);
  // tok[0] = "fixed", tok[1..3] = true triple (line 2), tok[4] =
  // 0-based lie round (5 = never), tok[5] = delta, tok[6..8] =
  // alternate triple (line 4)
  const rnd = parseInt(tok[4], 10);
  const delta = parseInt(tok[5], 10);
  const ta = parseInt(tok[1], 10);
  const tb = parseInt(tok[2], 10);
  const tc = parseInt(tok[3], 10);
  const aa = parseInt(tok[6], 10);
  const ab = parseInt(tok[7], 10);
  const ac = parseInt(tok[8], 10);
  const ans = [];
  for (let i = 0; i < 5; i++)
    ans.push(QX[i] * ta + QY[i] * tb + QZ[i] * tc);
  if (rnd ≥ 0 && rnd < 5)
    ans[rnd] = QX[rnd] * aa + QY[rnd] * ab + QZ[rnd] * ac + delta;
  const got = deduce(ans);
  if (!got) return null;
  return `${got[0]} ${got[1]} ${got[2]}\n`;
}
```

listing 469: javascript, solve: the token comment pins the layout, the lying round answered from line 4 plus the delta (source: icpc/samples-js/src/ch11-pw-riddleofthesphinx.mjs, lines 68-89)

```python
def solve(text):
    lines = text.split("\n")
    ta, tb, tc = (int(v) for v in lines[1].split())
    rnd, delta = (int(v) for v in lines[2].split())
    aa, ab, ac = (int(v) for v in lines[3].split())
    ans = [QX[i] * ta + QY[i] * tb + QZ[i] * tc for i in range(5)]
    if 0 ≤ rnd < 5:
        ans[rnd] = QX[rnd] * aa + QY[rnd] * ab + QZ[rnd] * ac + delta
    legs = deduce(ans)
    if legs is None:
        raise SystemExit("no consistent hypothesis")
    return f"{legs[0]} {legs[1]} {legs[2]}\n"
```

listing 470: python, solve: three split lines, truthful answers from line 2, one overwritten at the lying round (source: icpc/samples-py/src/Ch11/pw.py, lines 70-81)

```lua
while pos ≤ len do -- skip the "fixed" header token
  local b = input:byte(pos)
  if b == 32 or b == 10 or b == 13 or b == 9 then
    pos = pos + 1
  elseif b < 48 or b > 57 and b ~= 45 then
    pos = pos + 1
  else
    break
  end
end
local ta = next_int() -- true triple on line 2
local tb = next_int()
local tc = next_int()
local rnd = next_int()
local delta = next_int()
local aa = next_int() -- alternate triple on line 4
local ab = next_int()
local ac = next_int()
local ans = {}
for i = 0, 4 do
  ans[i + 1] = QX[i + 1] * ta + QY[i + 1] * tb + QZ[i + 1] * tc
end
if rnd ≥ 0 and rnd < 5 then
  ans[rnd + 1] = QX[rnd + 1] * aa + QY[rnd + 1] * ab +
                 QZ[rnd + 1] * ac + delta
end
```

listing 471: lua, the file mode: skip the header, read truth then alternate, apply the single lie (source: icpc/samples-lua/ch11_pw.lua, lines 109-134)

The fixture is the answer sequence 4 4 4 12 18: the truthful fifth answer for legs (4,4,4) is 24, so the lie must sit on round 5, and the remaining rows give la = lb = lc = 4 directly. Every suite drives `deduce` on that sequence and expects `4 4 4`, plus the no-lie answers 12 7 3 22 35, a lie at round 1 with delta +1, and a lie at round 3 with a large negative delta, −1000 in five languages and −40 on the go row, java's variant at −983, all still recovering the legs, and the fix wave's file-mode pair, the alt-sol swap and the equal-triples delta lie, lifts every suite to six checks per language. Java's Cramer rejects non-integer legs through `num % det ≠ 0`, sign-safe under java's truncated remainder, and its solve returns null when no hypothesis fits, judge mode exiting 1 rather than printing. Answers stay within 1e5 and legs within 1e4, so no integer divergence.

question	answer	
(1,0,0)	4	
(0,1,0)	4	
(0,0,1)	4	
(1,1,1)	12	
(1,2,3)	~~18~~	the lie, delta −6

surviving rows solve to la = lb = lc = 4

figure 153: the five question rows, the lying answer struck through, the recovered legs underneath

13.10. x, quartets

Four children play quartets with a 32-card deck of 8 sets of 4 cards, named 1A through 8D, the first digit the set. Every player is dealt 8 cards and player 1 starts. On a turn a player asks another player for one particular card, and the ask is legal only when the asker already holds at least one card of that card's set. If the asked player holds the card they must hand it over and the asker keeps the turn and asks again. Otherwise they say no, the asker's turn ends, and the asked player's turn begins. During a turn a player holding all four cards of a set may lay that quartet aside for a point, removing its four cards from the game. A player out of cards leaves, the turn passing to the next

player in 1-2-3-4 order who still holds cards, nobody may ask for a removed card or ask a player who has left, and asking for a card the asked player cannot possibly hold, one sitting in the asker's own hand, is foolish but legal. Cheating is exactly two claims: asking while holding no card of that set, and denying a held card. Given the log of a game's first n actions, decide whether some initial deal of the 32 cards makes the whole log the play of honest players, and if not, print the first action number after which someone must already have cheated (statement).

The input, per the problemset pdf, is one integer n with 1 <= n <= 1000, then n action lines in one of three shapes: `x A y sk yes`, player x asks player y for card sk and receives it, `x A y sk no`, the ask draws a denial and y's turn begins, or `x Q s`, player x lays quartet s aside, with x and y distinct players in 1..4, sets s in 1..8, and ranks k among A, B, C, D. The log obeys every rule except possibly the two cheat forms. The output is `yes`, or `no` and the first incriminating action number, under the pdf's 2 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `X-quartets/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints and explains: both askers needed a set-3 card in hand, the transfers pin 3A with player 3 and 3D with player 4, and both players 1 and 2 deny holding 3C, so 3B and 3C must have started with players 1 and 2 in some order while neither denial leaves room for 3C anywhere: no deal survives action 4.

input:

```
4
1 A 2 3C no
2 A 3 3A yes
2 A 4 3D yes
2 A 1 3C no
```

expected output:

```
no
4
```

Official sample 2, same source: the fourth denial lands on 3B instead, an honest deal still exists, and player 1 later completes set 5 and lays it down.

input:

```
6
1 A 2 3C no
2 A 3 3A yes
2 A 4 3D yes
2 A 1 3B no
1 A 4 5B yes
1 Q 5
```

expected output:

```
yes
```

Recognition: n <= 1000 actions against a fixed 32-card deck under the 2 second limit. One feasibility test per action at 32 slots through augmenting paths costs at most $32 \cdot 32 = 1024$ augment steps, about 1e6 for the whole log, three orders of magnitude inside budget.

The statement's cue is decide whether some initial deal makes the whole log honest: the deal places 32 cards into start slots under pins and bans, and the question after every action is exactly whether a perfect matching still exists, the cue of a feasibility-as-matching shape. Problem X is the 2022 face of the bipartite assignment feasibility

family, and the bipartite assignment feasibility family collects chapter 10 problem C, chapter 15 problem B, and chapter 15 problem F.

Enumerating deals prices out at 32! over (8!)^4, about 1e17, and tracking one candidate deal goes wrong the first time a denial leaves several homes, so the per-action matching is the only budget that closes.

The solver tracks knowledge about the initial deal: a card's start owner when a transfer reveals it, a card's non-owners on honest-sounding denials, current holders of moved cards, and one pending started-with-some-card-of-set-S constraint per player and set. After every action a perfect matching of the 32 start slots to the 32 cards, hard pins, forbidden edges, and demand slots restricted to their set, decides whether some deal still explains everything, and the first failing action is the answer, with the matching cost 32 cards per action through augmenting paths, per the solutions pdf pages 7 and 8, problem at icpc2022.pdf. Book 10′s chapter 38, network flows ii, builds kuhn's augmenting-path bipartite matching this feasibility test runs once per action. Directly visible contradictions, denying a card visibly held, handing a card held elsewhere, laying a quartet containing someone else's card, answer immediately. The trap that cost one stream a judge case is the demand registration: pending demands must be registered unconditionally at every ask. Registering them only when a justification is still visible lets a provably unjustifiable ask slide, the matching never fails, and the log ends in a false yes, judge witness secret-154-per, old output yes, truth `no` at action
917. The go and js engines compute the justification mask

explicitly and answer `no` on the spot when it comes up empty; C, C#, python, and lua register the demand and let the matching reject it, and both encodings are provably the same model.

The worked run: trace the model on sample 1. Action 1, player 1 asks player 2 for 3C and draws a denial: the ask registers player 1′s demand for a set-3 starter, the denial bans 3C from player 2, and a deal still exists since 3B or 3C can start with player 1. Actions 2 and 3 transfer 3A from player 3 and 3D from player 4 to player 2, pinning both cards' starts, and the demand side stays satisfiable through 3B or 3C. Action 4, player 2 asks player 1 for 3C and is denied: 3C is now banned from players 1 and 2, and with 3A pinned to player 3 and 3D to player 4, the only set-3 card either demand slot can still take is 3B, one card for the two demands of players 1 and 2, so the matching fails. The first incriminating action is 4, and the trace ends at the printed answer `4`.

set-3 card	start pinned	banned from	after action 4
3A	player 3, action 2		one home left
3B			the only card both demands can take
3C		player 2, action 1; player 1, action 4	players 3, 4 only
3D	player 4, action 3		one home left

```c
static int allow(int p, int slotset, int c) {
  if (forb[p][c])
    return 0;
  if (slotset)
    // a demand justification must be a never-moved card: any pin to
    // the asker predating the ask means the card already left
    return !pin[c] && c / 4 == slotset - 1;
  return !pin[c] || pin[c] == p;
}

static int matchr[32], seen[32];

static int aug(int slot, const unsigned char *slotset) {
  int p = slot / 8 + 1;
  for (int c = 0; c < 32; c++) {
    if (seen[c] || !allow(p, slotset[slot], c))
      continue;
    seen[c] = 1;
    if (matchr[c] < 0 || aug(matchr[c], slotset)) {
      matchr[c] = slot;
      return 1;
    }
  }
  return 0;
}

static int deal_exists(void) {
  // dedicate the first slots of each player to their pending demands
  unsigned char slotset[32];
  memset(slotset, 0, sizeof slotset);
  for (int p = 1; p ≤ 4; p++) {
    int j = 0;
    for (int s = 1; s ≤ 8; s++)
      if (dem[p][s] ≥ 0)
        slotset[(p - 1) * 8 + j++] = (unsigned char)s;
  }
  memset(matchr, -1, sizeof matchr);
  for (int slot = 0; slot < 32; slot++) {
    memset(seen, 0, sizeof seen);
    if (!aug(slot, slotset))
      return 0;
  }
  return 1;
}
```

listing 472: c, the matching engine: allow, augmenting path, deal_exists with one dedicated slot per demand (source: icpc/samples-c/src/Ch11/pX.c, lines 55-98)

```
      y8, _ := strconv.Atoi(lines[idx+2])
      y := int8(y8 - 1)
      tok := lines[idx+3]
      answ := lines[idx+4]
      idx += 5
      c := (int(tok[0]-'0')-1)*4 + int(tok[1]-'A')
      if askKnown(x, c/4) == 1 {
        return "no\n" + strconv.Itoa(a) + "\n"
      }
      if askKnown(x, c/4) == 0 {
        set := c / 4
        var m uint8
        for cc := set * 4; cc < set*4+4; cc++ {
          if cur[cc] < 0 && notStart[cc]&(1<<uint(x)) == 0 {
            m |= 1 << uint(cc%4)
          }
        }
        if m == 0 {
          // every same-set card is held elsewhere or was
          // denied by the asker: no deal can justify the ask
          return "no\n" + strconv.Itoa(a) + "\n"
        }
        demand[x][set] = m
      }
```

listing 473: go, the ask branch: askKnown triage, the computed mask, and the immediate no when no card can justify the ask (source: icpc/samples-go/ch11/px.go, lines 164-187)

```
  static boolean allow(int p, int slotset, int c) {
    if (forb[p][c] ≠ 0)
      return false;
    if (slotset ≠ 0)
      // a demand justification must be a never-moved card: any pin to
      // the asker predating the ask means the card already left
      return pin[c] == 0 && c / 4 == slotset - 1;
    return pin[c] == 0 || pin[c] == p;
  }

  static final int[] matchr = new int[32], seen = new int[32];
  static final int[] slotset = new int[32];

  static boolean aug(int slot) {
    int p = slot / 8 + 1;
    for (int c = 0; c < 32; c++) {
      if (seen[c] ≠ 0 || !allow(p, slotset[slot], c))
        continue;
      seen[c] = 1;
      if (matchr[c] < 0 || aug(matchr[c])) {
        matchr[c] = slot;
        return true;
      }
    }
    return false;
  }

  static boolean dealExists() {
    // dedicate the first slots of each player to their pending demands
    java.util.Arrays.fill(slotset, 0);
    for (int p = 1; p ≤ 4; p++) {
      int j = 0;
      for (int s = 1; s ≤ 8; s++)
        if (dem[p][s] ≥ 0)
          slotset[(p - 1) * 8 + j++] = s;
    }
    java.util.Arrays.fill(matchr, -1);
    for (int slot = 0; slot < 32; slot++) {
      java.util.Arrays.fill(seen, 0);
      if (!aug(slot))
        return false;
    }
    return true;
```

listing 474: java, the matching engine: allow, the augmenting path, dealExists with one dedicated slot per demand (source: icpc/samples-java/src/Ch12/PX.java, lines 55-97)

```
    // perfect matching of the 32 cards into per-player slots; each
    // pending demand dedicates one of the player's slots to its set,
    // fillable only by a card that has never moved (a moved card's
    // start is hard-known and cannot witness a pending ask)
    private static bool Feasible(int[] start, int[] holder, int[] notStart,
        bool[,] pending)
    {
        // slot per player: 8 free-ish, demand slots restricted to a set
        var slotSet = new int[32];
        var slotPl = new int[32];
        int sc = 0;
        for (int p = 0; p < 4; p++)
        {
            for (int s = 0; s < 8; s++)
                if (pending[p, s])
                {
                    slotSet[sc] = s;
                    slotPl[sc++] = p;
                }
            int freeSlots = 8;
            for (int s = 0; s < 8; s++)
                freeSlots -= pending[p, s] ? 1 : 0;
            for (int i = 0; i < freeSlots; i++)
            {
                slotSet[sc] = -1;
                slotPl[sc++] = p;
            }
        }

        var owner = new int[32];
        Array.Fill(owner, -1);
        for (int c = 0; c < 32; c++)
            if (!Augment(c, start, holder, notStart, slotSet, slotPl,
                    owner, new bool[32]))
                return false;
        return true;
    }
```

listing 475: c#, Feasible: demand slots first, free slots behind them, then the 32-card matching (source: icpc/samples/src/Ch11/PX.cs, lines 106-142)

```
      const y = parseInt(lines[idx + 2], 10) - 1;
      const tok = lines[idx + 3];
      const answ = lines[idx + 4];
      idx += 5;
      const c = (parseInt(tok[0], 10) - 1) * 4 + (tok.charCodeAt(1) - 65);
      if (askKnown(x, (c / 4) | 0) === 1) return `no\n${a}\n`;
      if (askKnown(x, (c / 4) | 0) === 0) {
        const set = (c / 4) | 0;
        let m = 0;
        for (let cc = set * 4; cc < set * 4 + 4; cc++)
          if (cur[cc] < 0 && (notStart[cc] & (1 << x)) === 0) m |= 1 << (cc % 4);
        if (m === 0)
          // every same-set card is held elsewhere or was denied by the
          // asker: no deal can justify the ask
          return `no\n${a}\n`;
        demand[x][set] = m;
      }
      if (answ === "no") {
        if (cur[c] ≥ 0) {
          if (cur[c] === y) return `no\n${a}\n`;
        } else {
          notStart[c] |= 1 << y;
        }
```

listing 476: javascript, the same triage and mask-0 rejection, denials recorded as notStart bits (source: icpc/samples-js/src/ch11-px-quartets.mjs, lines 111-133)

```
    def allow(p, slotset, c):
        if forb[p][c]:
            return False
        if slotset:
            # a demand justification is a never-moved card of the set
            return not pin[c] and c // 4 == slotset - 1
        return not pin[c] or pin[c] == p

    def aug(slot, slotset, matchr, seen):
        p = slot // 8 + 1
        for c in range(32):
            if seen[c] or not allow(p, slotset[slot], c):
                continue
            seen[c] = 1
            if matchr[c] < 0 or aug(matchr[c], slotset, matchr, seen):
                matchr[c] = slot
                return True
        return False

    def deal_exists():
        slotset = [0] * 32
        for p in range(1, 5):
            j = 0
            for s in range(1, 9):
                if dem[p][s] ≥ 0:
                    slotset[(p - 1) * 8 + j] = s
                    j += 1
        matchr = [-1] * 32
        for slot in range(32):
            seen = [False] * 32
            if not aug(slot, slotset, matchr, seen):
                return False
        return True
```

listing 477: python, the nested allow, aug, and deal_exists the action loop calls after every step (source: icpc/samples-py/src/Ch11/px.py, lines 27-59)

```
while i ≤ n and not out do
  local x = tonumber(next_tok())
  local t = next_tok()
  if t == "Q" then
    local s = tonumber(next_tok())
    local bad = false
    for c = (s - 1) * 4 + 1, s * 4 do
      if not bad then
        if cur[c] ~= 0 and cur[c] ~= x then
          bad = true
        elseif pin[c] == 0 then
          do_pin(c, x) -- never moved and in x's hand
        end
      end
    end
    if bad or not deal_exists() then
      out = "no\n" .. i .. "\n"
    end
  else
    local y = tonumber(next_tok())
    local cardtok = next_tok()
    local card = (tonumber(cardtok:sub(1, 1)) - 1) * 4 +
                 (cardtok:byte(2) - 64)
    local s = tonumber(cardtok:sub(1, 1))
    local answ = next_tok()
    local yes = answ:sub(1, 1) == "y"
    local bad = false
    if not holds_set(x, s) then
      dem[x][s] = i -- justification must be a start card, pinned later
    end
    if yes then
      if cur[card] ~= 0 and cur[card] ~= y then
        bad = true
      else
        if cur[card] == 0 then
          do_pin(card, y) -- first transfer reveals the starter
        end
        cur[card] = x
      end
    else
      if cur[card] == y then
        bad = true
      elseif cur[card] == 0 and pin[card] == 0 then
        forb[y][card] = true
      end
    end
    if bad or not deal_exists() then
      out = "no\n" .. i .. "\n"
    end
  end
```

listing 478: lua, the action loop: quartets pin never-moved cards, asks register demands, denials forbid (source: icpc/samples-lua/ch11_px.lua, lines 117-166)

The fixture is four actions, each player denying 7B in turn: 7B never moves, so each denial excludes the denying player as its starter, the fourth action closes the last escape, and the answer is `no` at action 4. Every suite asserts that plus both official samples. The C, C#, python, and lua suites add the crafted yes case whose laid quartet start-matches three cards to player 1, four checks each, java's four taking the fixture, both samples, and a two-action

crafted yes. Go swaps that crafted yes for the pinned regression of the registration trap, `2 A 1 3B no / 3 A 1 3C no / 4 A 1 3D no /
2 A 3 3A yes / 1 A 4 3C yes`, which must answer `no` at action 5, and javascript carries that regression beside the crafted yes, five blocks to the others' four. Cards are 32 two-character tokens and players fit in a bitmask everywhere, so no integer notes.

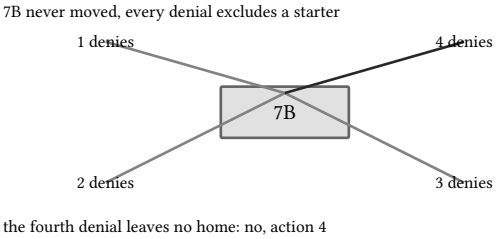


figure 154: four denials arranged around card 7B, the fourth closing the circle

13.11. y, compression

The Infinite Compression Plan Consortium's DRY scheme exploits repetition: whenever the string contains two consecutive copies of the same substring, it deletes one of them, and later deletions may use opportunities that earlier ones uncover. When several doubled substrings are on offer the choice matters, and DRY must pick the removal order that drives the final length as low as possible. The task works this on binary strings: given the string, print a shortest result any sequence of legal deletions can reach (statement).

The input, per the problemset pdf, is one nonempty line of zeroes and ones of length at most 1e5, and any shortest reachable result is an accepted answer, under the pdf's 1 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `Y-compression/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: a uniform string collapses to its one value.

input:

```
1111
```

expected output:

```
1
```

Official sample 3, same source: the endpoints 1 and 0 both survive, and 10 is already square-free.

input:

```
10110
```

expected output:

```
10
```

Sample 2, also official, is the fixed point: 101 admits no deletion and prints itself.

Recognition: the string runs to 1e5 characters under the 1 second limit, so the budget is one scan, and even an O(n^2) repetition scan at 1e10 pair checks is already an order of magnitude over.

The statement's cue is any sequence of legal deletions: the reachable set sounds like a search, but the first and last characters never change, no value dies, and the end state admits no further deletion, meaning square-free, and binary square-free strings stop at length 3, so one invariant collapses the task to three cases with no search

at all. Problem Y is the 2022 face of the invariant collapse family, and chapter 14 problem I is the other invariant collapse twin.

Greedy deletion orders price out at exponentially many choice points, and a dp over positions and lengths has no statement bound to hang on, so the invariant is the only budget that closes.

The first and last characters can never change, no character value vanishes entirely, and the final string admits no further deletion, meaning it is square-free, and binary square-free strings have length at most 3. Book 10's chapter 33, string algorithms ii, finds the square repetitions a square-free string must avoid. So the answer is the single character when the input is uniform, the two endpoints when they differ, and the endpoints around the other value otherwise, reachable and unique, per the solutions pdf pages 8 and 9, problem at icpc2022.pdf. The solvers are three branches and a print, and the interesting listing is which scan each language uses.

The worked run: trace the model on sample 1. The input is 1111, uniform, exactly one value present. The first and last characters are both 1, the value set cannot shrink to empty, and the final string must be square-free: deleting one copy of every doubled block drives 1111 to 11 to 1, and the single character 1 is square-free at length 1, so the trace ends at the printed answer 1.

input shape	law	answer
uniform, sample 1	one value present, collapse to it	the character
endpoints differ, sample 3	both survive, length 2 is square-free	both endpoints
endpoints equal, mixed middle	endpoints keep one middle value	length 3

```c
static char *solve(const char *in) {
  char *buf = dupstr(in);
  char *s = buf;
  while (*s == ' ' || *s == '\n' || *s == '\r' || *s == '\t')
    s++;
  size_t len = 0;
  while (s[len] && s[len] != '\n' && s[len] != '\r')
    len++;
  char *out = malloc(8);
  if (s[0] == s[len - 1]) {
    // uniform input collapses to the single character; otherwise the
    // other value must survive and only the middle seat avoids a
    // doubled block
    int other = 0;
    for (size_t i = 0; i < len; i++)
      if (s[i] != s[0])
        other = 1;
    if (!other)
      out[0] = s[0], out[1] = '\n', out[2] = 0;
    else
      out[0] = s[0], out[1] = s[0] ^ 1, out[2] = s[0], out[3] = '\n',
      out[4] = 0;
  } else {
    out[0] = s[0], out[1] = s[len - 1], out[2] = '\n', out[3] = 0;
  }
  free(buf);
  return out;
}
```

listing 479: c, solve: trim, then the three-case answer over the endpoints (source: icpc/samples-c/src/Ch11/pY.c, lines 21-48)

```
func SolveY(input string) string {
  s := input
  for len(s) > 0 && (s[len(s)-1] == '\n' || s[len(s)-1] == '\r') {
    s = s[:len(s)-1]
  }
  first, last := s[0], s[len(s)-1]
  var out []byte
  switch {
  case first == last:
    uniform := true
    for i := 0; i < len(s); i++ {
      if s[i] ≠ first {
        uniform = false
        break
      }
    }
    if uniform {
      out = []byte{first}
    } else {
      other := byte('0')
      if first == '0' {
        other = '1'
      }
      out = []byte{first, other, first}
    }
  default:
    out = []byte{first, last}
  }
  return string(out) + "\n"
}
```

listing 480: go, the byte-level twin of the C branch (source: icpc/samples-go/ch11/py.go, lines 11-40)

```
  static String solve(String in) {
    int p = 0, n = in.length();
    while (p < n && (in.charAt(p) == ' ' || in.charAt(p) == '\n'
        || in.charAt(p) == '\r' || in.charAt(p) == '\t'))
      p++;
    int len = 0;
    while (p + len < n && in.charAt(p + len) ≠ '\n' && in.charAt(p + len) ≠ '\r')
      len++;
    if (in.charAt(p) == in.charAt(p + len - 1)) {
      // uniform input collapses to the single character; otherwise the
      // other value must survive and only the middle seat avoids a
      // doubled block
      boolean other = false;
      for (int i = 0; i < len; i++)
        if (in.charAt(p + i) ≠ in.charAt(p))
          other = true;
      if (!other)
        return in.charAt(p) + "\n";
      return "" + in.charAt(p) + (char) (in.charAt(p) ^ 1) + in.charAt(p) + "\n";
    }
    return "" + in.charAt(p) + in.charAt(p + len - 1) + "\n";
```

listing 481: java, solve: trim, then the three-case answer over the endpoints (source: icpc/samples-java/src/Ch12/PY.java, lines 23-43)

```
    public static string Solve(string input)
    {
        int a = 0, b = 0;
        char f = '0', l = '0';
        bool first = true;
        foreach (char ch in input)
        {
            if (ch ≠ '0' && ch ≠ '1')
                continue;
            if (ch == '0')
                a++;
            else
                b++;
            if (first)
            {
                f = ch;
                first = false;
            }
            l = ch;
        }
        if (a == 0 || b == 0)
            return f + "\n";
        if (f ≠ l)
            return $"{f}{l}\n";
        return $"{f}{(f == '0' ? '1' : '0')}{l}\n";
    }
```

listing 482: c#, one pass counting both values while catching first and last (source: icpc/samples/src/Ch11/PY.cs, lines 14-39)

```
export function solve(input) {
  const s = input.trim();
  const a = s[0], b = s[s.length - 1];
  if (a === b) {
    const other = a === "0" ? "1" : "0";
    return s.includes(other) ? a + other + a + "\n" : a + "\n";
  }
  return a + b + "\n";
}
```

listing 483: javascript, the whole solve: includes decides uniform (source: icpc/samples-js/src/ch11-py-compression.mjs, lines 11-19)

```
def solve(text):
    s = text.strip()
    if s[0] == s[-1]:
        if len(set(s)) == 1:
            return f"{s[0]}\n"
        mid = "1" if s[0] == "0" else "0"
        return f"{s[0]}{mid}{s[0]}\n"
    return f"{s[0]}{s[-1]}\n"
```

listing 484: python, set(s) of length 1 is the uniform test (source: icpc/samples-py/src/Ch11/py.py, lines 10-17)

```
local function solve(input)
  local s = input:match("^%s*(%S+)")
  local f, l = s:sub(1, 1), s:sub(-1)
  if f == l then
    if not s:find("[^" .. f .. "]") then
      return f .. "\n"
    end
    local o = f == "0" and "1" or "0"
    return f .. o .. f .. "\n"
  end
  return f .. l .. "\n"
end
```

listing 485: lua, a pattern negation tests uniformity in one find (source: icpc/samples-lua/ch11_py.lua, lines 9-20)

The fixture is 0110: keep first 0, last 0, and one 1, and length 2 cannot hold all three because 00 is itself a doubled block, so 010, reached by deleting one of the doubled 1s. Every suite asserts `010`, plus the official samples 1111 to 1, 101 staying 101, and 10110 to 10, plus the crafted 00000 to 0: five checks per language in C, C#, java, python, and lua, the same five asserts in javascript folded into four blocks with 00000 riding the uniform one, and four go rows that stop at the official samples. One binary line of at most 1e5 characters, no integers at all.

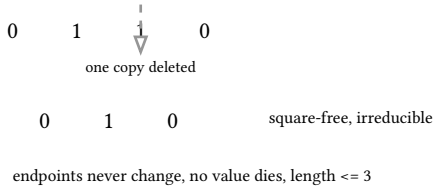


figure 155: the fixture: one of the doubled 1s removed, the irreducible result underneath

13.12. z, archaeological recovery

Professor Z Mummer's newly found tomb wall holds a row of k pyramid-shaped stone slabs, each showing one of three hieroglyphs face up: an ankh, an eye of Horus, or an ibis. Beside the wall stand n levers, and each lever rotates some of the pyramids one step, clockwise or counter-clockwise, leaving the others untouched. Flipping a lever back reverses exactly its own rotation, and the levers act independently, so the wall state under any setting of the levers is the sum of the chosen rotations. A student enumerated all 2^n lever settings and tallied the resulting wall configurations, multiplicities included, but an ink disaster destroyed the only record of the individual levers. From the multiset of reachable configurations alone, print any lever list that produces exactly that multiset (statement).

The input, per the problemset pdf, is one line of n, k, and t, with 1 <= n <= 40 levers, 1 <= k <= 5 pyramids, and 1 <= t <= 3^k distinct configurations, then t lines: a string of k characters over A, E, and I naming the hieroglyph face up on each pyramid, and a count f with 1 <= f <= 2^n, the number of lever settings reaching it. The configurations are pairwise distinct, the counts sum to exactly 2^n, and at least one lever list is guaranteed to exist. The output is n lever strings over +, -, and 0, clockwise, counter-clockwise, and no rotation per pyramid, any valid list accepted, under the pdf's 1 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `Z-archaeologicalrecovery/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints beside its figure: the first lever turns pyramid 1 clockwise and pyramid 2 counter-clockwise, the second turns all three clockwise, and each of the four states is reached exactly once.

input:

```
2 3 4
EEE 1
EIA 1
```

```
IAE 1
AAA 1
```

expected output:

```
+-0
+++
```

Official sample 2, same source: two pyramids, three levers, two configurations reached 4 times each, one real lever and two that move nothing.

input:

```
3 2 2
IA 4
AA 4
```

expected output:

```
-0
00
00
```

Recognition: n <= 40 levers but k <= 5 pyramids and t <= 3^k = 243 configurations under the 1 second limit. The counts live in the 3^5 = 243-cell universe, the identity and the convolution cost n·3^k, about 1e4 cell updates, and the shift search 3^k, thousands of operations in total, and the tight n is what forces the algebra, since the counts themselves reach 2^40.

The statement's cue is from the multiset alone, print any lever list: the multiset of 2^n sums is pinned by residues mod 3 over a 3^k universe, the cue that the whole configuration space collapses into Z_3^k and the levers are read off its projections. Problem Z is the 2022 face of the algebraic reduction to a small universe family, and algebraic reduction to a small universe covers chapter 11 problem I and chapter 12 problem K.

Enumerating lever lists prices out at 3^(n·k) = 3^200, and fitting the multiset by simulating 2^40 settings per candidate is equally dead, so the valuation identity is the only budget that closes.

States encode as digits A/E/I for 0/1/2, levers print over 0, +, and -. Projecting the state multiset onto any x gives three residue counts whose minimum 2-adic valuation is the number of levers orthogonal to x: appending a zero lever doubles all three counts, a nonzero lever permutes them as a 3-cycle and keeps one odd. A counting identity then hands out the levers, the number in the subspace G is 3 times the f-sum over G-perp, divided by the perp size, minus n, all over 2, evaluated at G = {0} for the zero levers and at each antipodal line {0, x, -x} for the rest. The solver emits zeros and one + representative per line member, convolves the 2^n subset sums over arrays of size 3^k carrying reachability witnesses, finds the shift v whose shifted multiset equals the input, and flips the signs of any lever subset summing to v, which shifts the whole multiset uniformly onto the input, with the work dominated by 3^k times n plus 3^k, per the solutions pdf pages 9 and 10, problem at icpc2022.pdf. Book 10′s chapter 34, linear algebra, carries the linear-system machinery over Z_p^k this reconstruction runs on. Zero teams solved it in contest.

The worked run: trace the model on sample 1. Encoding A, E, I as 0, 1, 2, the four states are AAA = (0, 0, 0), EEE = (1, 1, 1), EIA = (1, 2, 0) and IAE = (2, 0, 1), each reached exactly once, and the counts sum to 4 = 2^2, so n = 2 levers. All four counts are odd, so no lever is the zero lever, a zero lever would double every count to 2. The emission reads the levers off the small universe as `+-0` = (1, 2, 0) and `+++` = (1, 1, 1), and the closing check, simulating every subset sum, verifies them: neither lever on is (0, 0, 0) = AAA, `+-0` alone is EIA, `+++` alone is EEE, and both on sum

to (1, 2, 0) + (1, 1, 1) = (2, 0, 1) mod 3, exactly IAE. Every input state is hit once, and the trace ends at the printed answer +++.

levers on	sum mod 3	state	input count
none	(0, 0, 0)	AAA	1
+-0	(1, 2, 0)	EIA	1
+++	(1, 1, 1)	EEE	1
both	(2, 0, 1)	IAE	1

```
// f(x): common 2-adic valuation of the three projected counts
static int fcnt[MAXS];
for (int x = 0; x < tsz; x++) {
  long long c[3] = {0, 0, 0};
  for (int a = 0; a < tsz; a++)
    c[dotp(a, x)] += freq[a];
  int z = v2ll(c[0]);
  for (int r = 1; r < 3; r++)
    if (v2ll(c[r]) < z)
      z = v2ll(c[r]);
  fcnt[x] = z;
}
// zero levers: G = {0}, G-perp = the whole space
long long sumAll = 0;
for (int y = 0; y < tsz; y++)
  sumAll += fcnt[y];
int zeros = (int)(3 * sumAll / tsz - n) / 2;
// per-line lever counts at canonical reps (x < -x); line levers
// are emitted before the zero levers, in descending rep order
int lev[48][5], ln = 0;
for (int x = tsz - 1; x >= 1; x--) {
  int neg = 0;
  for (int i = 0; i < k; i++)
    neg += (3 - x / pw3(i) % 3) % 3 * pw3(i);
  if (neg < x)
    continue;
  long long sum = 0;
  for (int y = 0; y < tsz; y++)
    if (dotp(x, y) == 0)
      sum += fcnt[y];
  int cnt = (int)(3 * sum / (tsz / 3) - n) / 2 - zeros;
  for (int i = 0; i < cnt; i++) {
    for (int d = 0; d < k; d++)
      lev[ln][d] = x / pw3(d) % 3;
    ln++;
  }
}
for (int i = 0; i < zeros; i++) {
  memset(lev[ln], 0, sizeof lev[ln]);
  ln++;
}
```

listing 486: c, f(x) as the min 2-adic valuation of the projected counts, then zeros and per-line lever counts from the identity (source: icpc/samples-c/src/Ch11/pZ.c, lines 85-125)

```
  space := func(perp int, perpSize int) int {
    sum := 0
    for y := 0; y < size; y++ {
      if dot(y, perp) == 0 {
        sum += f[y]
      }
    }
    return (3*sum/perpSize - n) / 2
  }
  zeros := space(0, size)

  // one +representative per antipodal pair class, with counts
  var levers []int
  for i := 0; i < zeros; i++ {
    levers = append(levers, 0)
  }
  for x := 1; x < size; x++ {
    if !canonicalZ(x, k) {
      continue
    }
    m := space(x, size/3) - zeros
    for i := 0; i < m; i++ {
      levers = append(levers, x)
    }
  }

  // convolve subset sums, remembering one witness subset per state
  counts := make([]int64, size)
  wit := make([]uint64, size)
  counts[0] = 1
  for j, u := range levers {
    nc := make([]int64, size)
    copy(nc, counts)
    nw := make([]uint64, size)
    copy(nw, wit)
    for s := 0; s < size; s++ {
      if counts[s] == 0 {
        continue
      }
      s2 := add(s, u)
      nc[s2] += counts[s]
      if nw[s2] == 0 {
        nw[s2] = wit[s] | 1<<uint(j)
      }
    }
    counts, wit = nc, nw
  }
```

listing 487: go, the identity as one space closure, then the subset-sum convolution with witness bitmasks (source: icpc/samples-go/ch11/pz.go, lines 99-145)

```
    // f(x): common 2-adic valuation of the three projected counts
    int[] fcnt = new int[tsz];
    for (int x = 0; x < tsz; x++) {
      long[] c = new long[3];
      for (int a = 0; a < tsz; a++)
        c[dotp(a, x)] += freq[a];
      int z = v2ll(c[0]);
      for (int r = 1; r < 3; r++)
        if (v2ll(c[r]) < z)
          z = v2ll(c[r]);
      fcnt[x] = z;
    }
    // zero levers: G = {0}, G-perp = the whole space
    long sumAll = 0;
    for (int y = 0; y < tsz; y++)
      sumAll += fcnt[y];
    int zeros = (int) (3 * sumAll / tsz - n) / 2;
    // per-line lever counts at canonical reps (x < -x); line levers
    // are emitted before the zero levers, in descending rep order
    int[][] lev = new int[48][5];
    int ln = 0;
    for (int x = tsz - 1; x ≥ 1; x--) {
      int neg = 0;
      for (int i = 0; i < k; i++)
        neg += (3 - x / pw3(i) % 3) % 3 * pw3(i);
      if (neg < x)
        continue;
      long sum = 0;
      for (int y = 0; y < tsz; y++)
        if (dotp(x, y) == 0)
          sum += fcnt[y];
      int cnt = (int) (3 * sum / (tsz / 3) - n) / 2 - zeros;
      for (int i = 0; i < cnt; i++) {
        for (int d = 0; d < k; d++)
          lev[ln][d] = x / pw3(d) % 3;
        ln++;
      }
    }
```

listing 488: java, f(x) as the min 2-adic valuation of the projected counts, then zeros and per-line lever counts from the identity (source: icpc/samples-java/src/Ch12/PZ.java, lines 92-129)

```
        // m[x] = number of levers u with u.x = 0, from projected counts
        var m = new int[size];
        var proj = new long[3];
        for (int x = 1; x < size; x++)
        {
            proj[0] = proj[1] = proj[2] = 0;
            for (int s = 0; s < size; s++)
            {
                if (f[s] == 0)
                    continue;
                proj[Dot(x, s, k, size)] += f[s];
            }
            long g = Gcd(Gcd(proj[0], proj[1]), proj[2]);
            int v2 = 0;
            while (g != 0 && g % 2 == 0)
            {
                g /= 2;
                v2++;
            }
            m[x] = v2;
        }
        m[0] = n;

        // zero levers via the counting identity over the whole space
        long sumAll = 0;
        for (int x = 0; x < size; x++)
            sumAll += m[x];
        int zeros = (int)((3 * sumAll / size - n) / 2);
```

listing 489: c#, the same projection through one gcd then one valuation loop, m of 0 set to n (source: icpc/samples/src/Ch11/PZ.cs, lines 42-69)

```javascript
// convolution of subset sums + reachability witnesses
const cnt2 = new Float64Array(tsz);
const reach = new Uint8Array(tsz);
const prevst = new Int32Array(tsz);
const prevj = new Int32Array(tsz).fill(-1);
cnt2[0] = 1;
reach[0] = 1;
for (let j = 0; j < lev.length; j++) {
  const u = lev[j];
  const nxt = Float64Array.from(cnt2);
  for (let a = 0; a < tsz; a++) if (cnt2[a] =/= 0) nxt[add3(a, u)] += cnt2[a];
  cnt2.set(nxt);
  // snapshot: sources must predate this lever, or a witness path
  // could use the same lever twice
  const snap = Uint8Array.from(reach);
  for (let a = 0; a < tsz; a++)
    if (snap[a]) {
      const b = add3(a, u);
      if (!reach[b]) {
        reach[b] = 1;
        prevst[b] = a;
        prevj[b] = j;
      }
    }
}
```

listing 490: javascript, the convolution and the snapshot rule: witness sources must predate the lever (source: icpc/samples-js/src/ch11-pz-archaeologicalrecovery.mjs, lines 83-107)

```python
  fcnt = [0] * tsz
  for x in range(tsz):
      c = [0, 0, 0]
      for a in range(tsz):
          c[dotp(a, x)] += freq[a]
      fcnt[x] = min(v2(c[0]), v2(c[1]), v2(c[2]))
  zeros = (3 * sum(fcnt) // tsz - n) // 2
  levs = []
  for x in range(tsz - 1, 0, -1):
      neg = 0
      for i in range(k):
          neg += (3 - x // 3**i % 3) % 3 * 3**i
      if neg < x:
          continue
      total = sum(fcnt[y] for y in range(tsz) if dotp(x, y) == 0)
      cnt = (3 * total // (tsz // 3) - n) // 2 - zeros
      for _ in range(cnt):
          levs.append([x // 3**d % 3 for d in range(k)])
  for _ in range(zeros):
      levs.append([0] * k)
```

listing 491: python, fcnt, the zeros identity, and the per-line counts (source: icpc/samples-py/src/Ch11/pz.py, lines 50-69)

```
-- convolution of subset sums + reachability witnesses
local cnt2, nxt = {}, {}
local reach, prevst, prevj = {}, {}, {}
for i = 0, tsz - 1 do
  cnt2[i], reach[i], prevj[i] = 0, false, -1
end
cnt2[0], reach[0] = 1, true
for j = 1, ln do
  local u = lev[j]
  for i = 0, tsz - 1 do nxt[i] = cnt2[i] end
  for a = 0, tsz - 1 do
    local ca = cnt2[a]
    if ca ~= 0 then
      local b = add3(a, u)
      nxt[b] = nxt[b] + ca
    end
  end
  for i = 0, tsz - 1 do cnt2[i] = nxt[i] end
  -- snapshot: sources must predate this lever, or a witness path
  -- could use the same lever twice
  local snap = {}
  for i = 0, tsz - 1 do snap[i] = reach[i] end
  for a = 0, tsz - 1 do
    if snap[a] then
      local b = add3(a, u)
      if not reach[b] then
        reach[b] = true
        prevst[b] = a
        prevj[b] = j
      end
    end
  end
end
-- find reachable v with {s - v} equal to the input multiset
local vshift = -1
for v = 0, tsz - 1 do
  if reach[v] then
    local ok = true
    for b = 0, tsz - 1 do
      if cnt2[add3(b, v)] ~= freq[b] then
        ok = false
        break
      end
    end
    if ok then
      vshift = v
      break
    end
```

listing 492: lua, convolution, snapshot witnesses, and the shift search (source: icpc/samples-lua/ch11_pz.lua, lines 127-174)

The fixture is k = 1, n = 2, counts A 1, E 1, I 2: the projections are the counts themselves, valuation 0 means no zero lever, the antipodal line holds both, both start at +, their subset sums give counts 1, 2, 1, the input is that multiset shifted by 2, and flipping both levers lands on it, so the canonical answer is two minus signs. Every suite asserts - and -, plus the crafted 2 1 2 answering + and 0, plus both official samples, and every case re-verifies by simulating all 2^n sums, four checks per language, java's solver returning null when no reachable shift matches, judge mode exiting 1 rather than printing. Frequencies reach 2^40, which pins the containers: int64 in C, C#, go,

and java, lua's 64-bit integers, python's exact ints, and javascript numbers exact below 2^53 with the counts held in a Float64Array. The counting identity divides by |G-perp| and halved, and both divisions land on integers first.

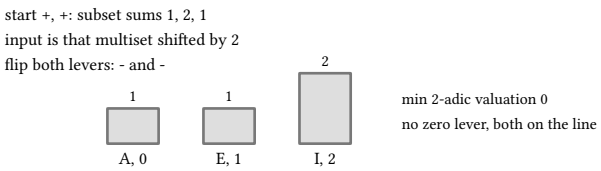


figure 156: the fixture's projected counts as three towers, the valuation trick and the double flip

13.13. across the seven languages

problem	kernel	go	java	c#	javascript	python	lua
p	stamp-retry BFS propagation over button components	same engine on slices	stamps over CSR	revalidates BFS order	same engine on typed arrays	affine root algebra, no retries	stamps like c
q	long long suffix counts over the joint order	struct tree	long[] class tree	F3 culture	class tree	born-full tree	born-full tree
r	cover marks dropped, rotations and parity per component	carves blocks	cover marks, cycle walk	KMP rotation equality	cover marks	tree-edge owners	KMP over the walk
s	pull sweep, lazy heaps (k,	diagonal sweep	pull sweep, lazy heaps	same pull, pool fix	diagonal sweep	same sweep, boxed ints	future ring

problem	kernel	go	java	c#	javascript	python	lua
	g, l) machine, caps clamped to reachability						
t	1024-grid folding straights plus corner bends	unfoldLen crossings	1024-grid ternary legs	run detection, bisected edges	16 edge pairs, cross products	grid ternary	grid ternary, flat scalars
u	mirrored halves, curve budget, canonical strings	emission plus walk validator	mirrored halves, walk validator	same, StringBuilder	one string per shape	one-expression emissions	one-expression emissions
v	upper hull, edge interpolated score points	monotone chain, edge queries	hull, merge-counted advantage	tie and strict candidates	merge-counted advantage	hull keeping right turns	parallel arrays, index table
w	reduce, integer checks hypotheses over 3-subsets, Cramer	file-mode tokens	Cramer, num % det != 0	same	file mode, token layout	nested comprehensions	truth-line file mode

problem	kernel	go	java	c#	javascript	python	lua
x	dedicated demand slots perfect matching of 32 slots	explicit mask, immediate no	dedicated demand slots	Feasible, slots first	mask, not-Start bits	demand flags	matching rejects
y	pointer free, three branches	byte-level twin	trim, three branches	one pass, first and last	includes decides uniform	set(s) test	pattern negation
z	int64 counts, index digits	witness bitmasks	long counts, null exit	one gcd, one valuation loop	Float64Array counts	exact ints	snapshot witnesses

The pattern this set adds to the book's ledger: one problem, S, where the exact state machine outruns two interpreters on honest code and the chapter records the measured wall rather than a workaround, java's pull twin walking the same states inside the cap, one constructive problem, U, where the judge corrected the budget mathematics after the fact, and one knowledge problem, X, where the difference between registering a constraint unconditionally and registering it conditionally was one full judge case. Everything else is the book's standing lesson again, that the same derivation ports across seven languages and the ports agree because the suites pin the same fixtures everywhere.

sources: ICPC Foundation and icpc.global, the problemset at icpc2022.pdf and the solutions at finals2022solutions.pdf, both accessed 2026-09-14 and cached under `ref/icpc/2022/` with sha256 digests in `ref/icpc/INDEX.md`. The solutions pdf is the algorithm and complexity reference for every section above, and its page 1 statistics are the contest counts in the introduction. The judge data, 652 inputs across the 11 letters, is local-only under `ref/icpc/2022/data/`, its secret files are never quoted, and every sample pair reprinted above is the byte-exact content of that problem's `sample-N.in` and `sample-N.ans` files, attributed in place, with W's pair carried as its driver spec because that problem is adjudicated by an interactive runner against empty answer files. The chapter's suites are the frozen solver files: C runs 48 self-checks across the 11 programs through its CHECK, CHECKD, CHECKS, and CHECKF macros, the C# suite is 47 [Fact] methods carrying 49 assertions, go pins 41 table rows, 33 exact, 4 tolerance, and 4 convolution re-verifications, plus the 544-budget walk validator for u, javascript runs 49 it blocks with 60 assertions, python 48 check self-checks, and lua reports 49 named checks through run.lua, 48

of the shared family plus the pendant-bridge variant on r. The java tree lives under `samples-java/src/Ch12`, the one tree that follows the post-renumber chapter numbers while the six siblings keep their historical `Ch08` through `Ch13` directories: 12 files, the 11 solvers plus the `Fenwick` year helper, 2048 solver SLOC, its 56 ok-line checks run under `tools/run-java-samples.ps1`, and its judge lane answers 652 of 652 cases through the same verify script.

14. icpc world finals 2023

The 47th world finals were played in Luxor concurrently with the 46th, which is why this set of 11 problems lettered A through K shares five problem names with the 2022 set of chapter 13 under different letters, A doubling 2022′s W, C doubling V, D doubling T, G doubling P, and J doubling S. The problemset is icpc2023.pdf, and every algorithm and complexity claim in this chapter is settled against the official solutions pdf rather than reinvented. The solutions pdf also carries the contest statistics: E was solved by 0 teams and was the judges' pick as the hardest problem, J by exactly 1 team at minute 292, B by 7, K by 11, C by 30, F by 26, D by 86, H by 96, I by 110, G by 117, and A by 120. NRU Higher School of Economics took the title, and the field wrote 98.5 percent of its submissions in C++ with about 1 percent python and no java at all, a flat opposite of this book's seven-way spread. This chapter walks all 11 problems in letter order, one section per letter, seven listings per section, c through lua, every listing sliced from a solver file that ran against the official judge data held locally under `ref/icpc/`.

14.1. the year-local helpers: the sphinx oracle, ordered subsets, score hulls, pyramid vectors, and the failure functions

Five helper kits are year-local to this chapter, one per problem that wanted one, plus the go package's shared scan cursor, and everything else the year's solvers import comes from the wave 1 toolbox chapters 2 through 9. Each kit states its contract and prints its twin files below in the book's fixed language order, and java joins all five families as package-private classes compiled beside the solvers, Sphinx through Kmp. The C solvers parse through chapter 13′s `ch11_scan.h` rather than a year-local twin, so that cursor prints in chapter 13′s helper section and only go adds a new file for it here. The go judge lane `pjfast.go`, the twin of chapter 13′s `psfast.go`, stays deliberately unlisted: it is problem J's measured engine, not a helper.

The sphinx kit backs problem A and is the interesting one, because it encodes the judge contract for the set's only interactive problem: the file names the true leg counts, the lying round, and the additive lie, and `ask` simulates her five answers so the deduction core stays a pure function of the answer vector. C carries the struct and its two calls in `Ch12/ch12_sphinx.h` on top of chapter 13′s scan header, javascript exports a factory returning the ask closure, python and java classes whose constructors parse the fixed file, and lua a module whose init walks the file bytes itself. C# and go carry no twin file: both read the judge file directly inside their problem A solvers, the C# `Solve` answering from line 4 and go's `SolveA` building the five answers inline.

```c
static inline void sphinx_init(Sphinx *s, const char *buf) {
  Ch11Scan sc;
  ch11_scan_init(&sc, buf);
  ch11_scan_skip(&sc);
  while (*sc.p && *sc.p != ' ' && *sc.p != '\n')
    sc.p++; // "fixed"
  s->la = ch11_next_int64(&sc);
  s->lb = ch11_next_int64(&sc);
  s->lc = ch11_next_int64(&sc);
  s->lie = ch11_next_int64(&sc);
  s->delta = ch11_next_int64(&sc);
  s->asked = 0;
}

static inline long long sphinx_ask(Sphinx *s, long long a, long long b,
                                   long long c) {
  long long v = a * s->la + b * s->lb + c * s->lc;
  if (s->asked == s->lie)
    v += s->delta;
  s->asked++;
  return v;
}
```

listing 493: c, sphinx_init parses the fixed judge file, sphinx_ask answers truth plus the one lie on its round (source: icpc/samples-c/src/Ch12/ch12_sphinx.h, lines 15-36)

```java
  Sphinx(String text) {
    int pos = 0;
    while (pos < text.length() && text.charAt(pos) != ' ' && text.charAt(pos) != '\n')
      pos++; // "fixed"
    long[] vals = new long[5];
    for (int k = 0; k < 5; k++) {
      while (pos < text.length() && (text.charAt(pos) < '0' || text.charAt(pos) > '9'))
        pos++;
      long v = 0;
      while (pos < text.length() && text.charAt(pos) >= '0' && text.charAt(pos) <= '9')
        v = v * 10 + (text.charAt(pos++) - '0');
      vals[k] = v;
    }
    la = vals[0];
    lb = vals[1];
    lc = vals[2];
    lie = vals[3];
    delta = vals[4];
    asked = 0;
  }

  long ask(long a, long b, long c) {
    long v = a * la + b * lb + c * lc;
    if (asked == lie)
      v += delta;
    asked++;
    return v;
  }
```

listing 494: java, the Sphinx class, its constructor parsing the fixed judge file, ask adding the delta exactly on the lying round (source: icpc/samples-java/src/Ch13/Sphinx.java, lines 9-36)

```javascript
// ch12-sphinx parses the fixed 2023 A judge format (line 1 "fixed",
// line 2 the true leg counts, line 3 the 0-based lie question and its
// additive delta) and simulates the sphinx: ask(a, b, c) returns
// a*la + b*lb + c*lc plus the one lie on its round.
export function makeSphinx(input) {
  const tok = input.trim().split(/\s+/);
  const la = parseInt(tok[1], 10);
  const lb = parseInt(tok[2], 10);
  const lc = parseInt(tok[3], 10);
  const lieQ = parseInt(tok[4], 10);
  const delta = parseInt(tok[5], 10);
  let asked = 0;
  return {
    ask(a, b, c) {
      return a * la + b * lb + c * lc + (asked++ === lieQ ? delta : 0);
    },
```

listing 495: javascript, the same contract as a factory returning the ask closure (source: icpc/samples-js/src/ch12-sphinx.mjs, lines 1-16)

```python
class Sphinx:
    def __init__(self, text):
        lines = text.split("\n")
        self.la, self.lb, self.lc = (int(v) for v in lines[1].split())
        self.lie_q, self.delta = (int(v) for v in lines[2].split())
        self.round = 0

    def ask(self, a, b, c):
        r = a * self.la + b * self.lb + c * self.lc
        if self.delta and self.round == self.lie_q:
            r += self.delta
        self.round += 1
        return r
```

listing 496: python, the Sphinx class, ask adding the delta exactly on the lying round (source: icpc/samples-py/src/Ch12/ch12_sphinx.py, lines 5-17)

```lua
  local s = {}
  s.la = next_int()
  s.lb = next_int()
  s.lc = next_int()
  s.lie = next_int()
  s.delta = next_int()
  s.asked = 0
  return s
end

function sphinx.ask(s, a, b, c)
  local v = a * s.la + b * s.lb + c * s.lc
  if s.asked == s.lie then
    v = v + s.delta
  end
  s.asked = s.asked + 1
  return v
end
```

listing 497: lua, sphinx.init reads the five integers past the header word, sphinx.ask bakes in the one lie (source: icpc/samples-lua/ch12_sphinx.lua, lines 43-60)

The combinatorics kit backs problem B with a multiplicative binomial and a lexicographic k-subset enumerator that stops after m emissions, ascending position tuples, which is descending lexicographic order on the indicator strings. C's `ch12_comb.h` drives a callback emitter, javascript's `firstSubsets` collects arrays, python's `first_subsets` the same tuples, java's `Comb.firstSubsets` the same as int arrays, and lua's `comb.emit` visits without allocating. C# and go carry no twin file: the C# solver enumerates all 2^(k-1) masks and sorts them descending, and go's `gen` recursion tries the 1 branch first so it emits descending lexicographic order naturally.

```c
static inline long long ch12_binom(int n, int k) {
  if (k < 0 || k > n)
    return 0;
  long long r = 1;
  for (int i = 0; i < k; i++)
    r = r * (n - i) / (i + 1);
  return r;
}

// visit(pos, k) is called per emitted subset; returns the count.
typedef void (*Ch12Visit)(const int *pos, int k, void *ctx);

static inline int ch12_emit(int n, int k, int m, Ch12Visit visit, void *ctx) {
  if (k < 0 || k > n || m <= 0)
    return 0;
  int idx[64], emitted = 0;
  for (int i = 0; i < k; i++)
    idx[i] = i;
  for (;;) {
    visit(idx, k, ctx);
    if (++emitted == m)
      break;
    int i = k - 1;
    while (i >= 0 && idx[i] == n - k + i)
      i--;
    if (i < 0)
      break;
    idx[i]++;
    for (int j = i + 1; j < k; j++)
      idx[j] = idx[j - 1] + 1;
  }
  return emitted;
}
```

listing 498: c, ch12_binom plus ch12_emit, ascending position tuples, which is descending lexicographic on the indicator strings (source: icpc/samples-c/src/Ch12/ch12_comb.h, lines 9-41)

```java
static long binom(int n, int k) {
  if (k < 0 || k > n)
    return 0;
  long r = 1;
  for (int i = 0; i < k; i++)
    r = r * (n - i) / (i + 1);
  return r;
}

// the first m k-subsets of 0..n-1 in ascending lex order
static int[][] firstSubsets(int n, int k, int m) {
  int[][] out = new int[Math.max(m, 0)][];
  int count = 0;
  if (k < 0 || k > n || m ≤ 0)
    return out;
  int[] cur = new int[k];
  for (int i = 0; i < k; i++)
    cur[i] = i;
  while (count < m) {
    out[count++] = cur.clone();
    int i = k - 1;
    while (i ≥ 0 && cur[i] == n - k + i)
      i--;
    if (i < 0)
      break;
    cur[i]++;
    for (int j = i + 1; j < k; j++)
      cur[j] = cur[j - 1] + 1;
  }
  return java.util.Arrays.copyOf(out, count);
}
```

listing 499: java, binom plus firstSubsets, the m-capped recursion emitting ascending position tuples (source: icpc/samples-java/src/Ch13/Comb.java, lines 7-37)

```
export function binom(n, r) {
  if (r < 0 || r > n) return 0;
  let v = 1;
  for (let i = 0; i < r; i++) v = (v * (n - i)) / (i + 1);
  return v;
}

// positions are 0-based; ascending lexicographic subset order, which
// is descending order over the induced fixed-weight bit strings
export function firstSubsets(len, weight, m) {
  const out = [];
  const cur = [];
  const rec = (start, left) => {
    if (out.length ≥ m) return;
    if (left === 0) {
      out.push(cur.slice());
      return;
    }
    for (let i = start; i ≤ len - left; i++) {
      cur.push(i);
      rec(i + 1, left - 1);
      cur.pop();
      if (out.length ≥ m) return;
    }
  };
  rec(0, weight);
  return out;
}
```

listing 500: javascript, binom plus firstSubsets, the m-capped recursion in ascending subset order (source: icpc/samples-js/src/ch12-comb.mjs, lines 4-31)

```python
def binom(n, k):
    if k < 0 or k > n:
        return 0
    k = min(k, n - k)
    out = 1
    for i in range(k):
        out = out * (n - i) // (i + 1)
    return out


def first_subsets(n, k, m):
    # the first m k-subsets of 0..n-1 in ascending lex order
    out = []
    cur = list(range(k))
    if k > n:
        return out
    for _ in range(m):
        out.append(tuple(cur))
        i = k - 1
        while i ≥ 0 and cur[i] == n - k + i:
            i -= 1
        if i < 0:
            break
        cur[i] += 1
        for j in range(i + 1, k):
            cur[j] = cur[j - 1] + 1
    return out
```

listing 501: python, binom with the min(k, n-k) fold, first_subsets advancing one position tuple at a time (source: icpc/samples-py/src/Ch12/ch12_comb.py, lines 5-31)

```lua
function comb.binom(n, k)
  if k < 0 or k > n then return 0 end
  local r = 1
  for i = 0, k - 1 do
    r = r * (n - i) // (i + 1)
  end
  return r
end

-- visit(pos) is called per emitted subset; returns the count
function comb.emit(n, k, m, visit)
  if k < 0 or k > n or m ≤ 0 then return 0 end
  local idx = {}
  for i = 0, k - 1 do idx[i] = i end
  local emitted = 0
  while true do
    visit(idx, k)
    emitted = emitted + 1
    if emitted == m then break end
    local i = k - 1
    while i ≥ 0 and idx[i] == n - k + i do
      i = i - 1
    end
    if i < 0 then break end
    idx[i] = idx[i] + 1
    for j = i + 1, k - 1 do
      idx[j] = idx[j - 1] + 1
    end
  end
  return emitted
end
```

listing 502: lua, comb.binom plus comb.emit, the visitor handed each ascending tuple without allocation (source: icpc/samples-lua/ch12_comb.lua, lines 8-38)

The hull kit backs problem C: a monotone upper chain over sorted doubled points plus the two halfplane queries the dice need, the best y at x <= X and the best x at y >= Y, edges interpolating. C, python, and java carry chain and queries together in `Ch12/ch12_hull.h`, `ch12_hull.py`, and the `Hull` class over record points, while the javascript and lua twins carry only the chain, the full counterclockwise monotone hull in `ch12-hull.mjs` and the upper chain over parallel arrays in `ch12_hull.lua`, the two queries living inside their problem C solvers. C# and go inline both pieces in the solver files.

```c
static inline int h12_upper(H12Pt *hull, const H12Pt *pt, int np) {
  int nh = 0;
  for (int i = 0; i < np; i++) {
    while (nh ≥ 2 && h12cross(hull[nh - 2], hull[nh - 1], pt[i]) ≥ 0)
      nh--;
    hull[nh++] = pt[i];
  }
  return nh;
}

// max hull y at x ≤ X (doubled coordinates)
static inline double h12_maxy_at(const H12Pt *hull, int nh, long long X) {
  int peak = 0;
  for (int i = 1; i < nh; i++)
    if (hull[i].y > hull[peak].y)
      peak = i;
  if (hull[peak].x ≤ X)
    return (double)hull[peak].y;
  int i = peak;
  while (i > 0 && hull[i - 1].x > X)
    i--;
  if (i > 0) {
    H12Pt a = hull[i - 1], b = hull[i]; // edge straddling x = X
    return (double)a.y +
           (double)(b.y - a.y) * (double)(X - a.x) / (double)(b.x - a.x);
  }
  return (double)hull[0].y;
}

// min hull x at y ≥ Y (doubled coordinates)
static inline double h12_minx_at(const H12Pt *hull, int nh, long long Y) {
  int j0 = -1;
  for (int i = 0; i < nh; i++)
    if (hull[i].y ≥ Y) {
      j0 = i;
      break;
    }
  if (j0 ≤ 0)
    return (double)hull[0].x; // leftmost already qualifies
  H12Pt a = hull[j0 - 1], b = hull[j0];
  return (double)a.x +
         (double)(b.x - a.x) * (double)(Y - a.y) / (double)(b.y - a.y);
}
```

listing 503: c, the upper hull over sorted doubled points, then max y at x <= X and min x at y >= Y with edge interpolation (source: icpc/samples-c/src/Ch12/ch12_hull.h, lines 18-60)

```java
record Pt(long x, long y) {}

static long cross(Pt o, Pt a, Pt b) {
  return (a.x() - o.x()) * (b.y() - o.y()) - (a.y() - o.y()) * (b.x() - o.x());
}

// builds the upper hull (right turns only) of points pre-sorted by
// (x, y)
static Pt[] upper(Pt[] pt) {
  Pt[] hull = new Pt[pt.length];
  int nh = 0;
  for (Pt cur : pt) {
    while (nh ≥ 2 && cross(hull[nh - 2], hull[nh - 1], cur) ≥ 0)
      nh--;
    hull[nh++] = cur;
  }
  return Arrays.copyOf(hull, nh);
}

// max hull y at x ≤ X (doubled coordinates)
static double maxYat(Pt[] hull, long capX) {
  int peak = 0;
  for (int i = 1; i < hull.length; i++)
    if (hull[i].y() > hull[peak].y())
      peak = i;
  if (hull[peak].x() ≤ capX)
    return hull[peak].y();
  int i = peak;
  while (i > 0 && hull[i - 1].x() > capX)
    i--;
  if (i > 0) {
    Pt a = hull[i - 1], b = hull[i]; // edge straddling x = capX
    return a.y() + (double) (b.y() - a.y()) * (capX - a.x()) / (b.x() - a.x());
  }
  return hull[0].y();
}

// min hull x at y ≥ Y (doubled coordinates)
static double minXat(Pt[] hull, long floorY) {
  int j0 = -1;
  for (int i = 0; i < hull.length; i++)
    if (hull[i].y() ≥ floorY) {
      j0 = i;
      break;
    }
  if (j0 ≤ 0)
    return hull[0].x(); // leftmost already qualifies
  Pt a = hull[j0 - 1], b = hull[j0];
  return a.x() + (double) (b.x() - a.x()) * (floorY - a.y()) / (b.y() - a.y());
}
```

listing 504: java, the Hull kit whole: the upper chain over record points, then max y at x <= X and min x at y >= Y with edge interpolation (source: icpc/samples-java/src/Ch13/Hull.java, lines 8-57)

```
export function monotoneHull(points) {
  const pts = points.slice().sort((p, q) => (p.x !== q.x ? p.x - q.x : p.y - q.y));
  const uniq = [];
  for (const p of pts)
    if (!uniq.length || p.x !== uniq[uniq.length - 1].x || p.y !== uniq[uniq.length - 1].y)
      uniq.push(p);
  if (uniq.length < 3) return uniq;
  const cross = (o, a, b) => (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
  const lower = [];
  for (const p of uniq) {
    while (lower.length >= 2 && cross(lower[lower.length - 2], lower[lower.length - 1], p) <= 0)
      lower.pop();
    lower.push(p);
  }
  const upper = [];
  for (let i = uniq.length - 1; i >= 0; i--) {
    const p = uniq[i];
    while (upper.length >= 2 && cross(upper[upper.length - 2], upper[upper.length - 1], p) <= 0)
      upper.pop();
    upper.push(p);
  }
  lower.pop();
  upper.pop();
  return lower.concat(upper);
```

listing 505: javascript, monotoneHull, the deduped sort and the two popped chains closing the loop (source: icpc/samples-js/src/ch12-hull.mjs, lines 4-27)

```python
def upper_hull(pts):
    # pts sorted ascending; keep right turns so y stays concave
    hull = []
    for q in pts:
        while len(hull) ≥ 2 and (
            (hull[-1][0] - hull[-2][0]) * (q[1] - hull[-2][1])
            - (hull[-1][1] - hull[-2][1]) * (q[0] - hull[-2][0])
            ≥ 0
        ):
            hull.pop()
        hull.append(q)
    return hull


def max_y_under(hull, cap):
    # highest hull y at x ≤ cap, linear along the straddling edge
    peak = 0
    for i in range(1, len(hull)):
        if hull[i][1] > hull[peak][1]:
            peak = i
    if hull[peak][0] ≤ cap:
        return float(hull[peak][1])
    i = peak
    while i > 0 and hull[i - 1][0] > cap:
        i -= 1
    if i > 0:
        a, b = hull[i - 1], hull[i]
        return a[1] + (b[1] - a[1]) * (cap - a[0]) / (b[0] - a[0])
    return float(hull[0][1])


def min_x_above(hull, floor):
    # smallest hull x at y ≥ floor, linear along the straddling edge
    j0 = -1
    for i in range(len(hull)):
        if hull[i][1] ≥ floor:
            j0 = i
            break
    if j0 ≤ 0:
        return float(hull[0][0])
    a, b = hull[j0 - 1], hull[j0]
    return a[0] + (b[0] - a[0]) * (floor - a[1]) / (b[1] - a[1])
```

listing 506: python, upper_hull plus max_y_under and min_x_above, both queries interpolating the straddling edge (source: icpc/samples-py/src/Ch12/ch12_hull.py, lines 4-45)

```
function hull.upper(px, py, np)
  local hx, hy = {}, {}
  local nh = 0
  for i = 1, np do
    local x, y = px[i], py[i]
    while nh ≥ 2 do
      local ox, oy = hx[nh - 1], hy[nh - 1]
      local ax, ay = hx[nh], hy[nh]
      if (ax - ox) * (y - oy) - (ay - oy) * (x - ox) ≥ 0 then
        nh = nh - 1
      else
        break
      end
    end
    nh = nh + 1
    hx[nh], hy[nh] = x, y
  end
  return hx, hy, nh
end
```

listing 507: lua, hull.upper over parallel coordinate arrays, popping non-strict clockwise turns (source: icpc/samples-lua/ch12_hull.lua, lines 7-25)

The geometry kit backs problem D with the pyramid primitives: vector algebra, the cross product, closed-segment intersection with endpoint tolerance, and strict containment in a convex polygon. C's `Ch12/ch12_geom.h` also folds in the reflection across a line and the line-crossing parameter, go keeps one small `pd_geo.go` with the same predicates in relative tolerances, java's `Geom` the same over record points, javascript the same as arrow functions, while the python and lua twins carry the flat point algebra, the quarter turn and the cross, with the intersection and containment inlined in their solvers. C# embeds its copies in the `MinLeg` machinery of its problem D solver.

```c
} G12;

static inline G12 g12sub(G12 a, G12 b) { return (G12){a.x - b.x, a.y - b.y}; }
static inline G12 g12add(G12 a, G12 b) { return (G12){a.x + b.x, a.y + b.y}; }
static inline G12 g12mul(G12 a, double s) { return (G12){a.x * s, a.y * s}; }
static inline double g12dot(G12 a, G12 b) { return a.x * b.x + a.y * b.y; }
static inline double g12len(G12 a) { return sqrt(g12dot(a, a)); }
static inline G12 g12l90(G12 a) { return (G12){-a.y, a.x}; }
static inline double g12cross(G12 o, G12 a, G12 b) {
  return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

// mirror p across the line through a and b
static inline G12 g12reflect(G12 p, G12 a, G12 b) {
  G12 d = g12sub(b, a);
  G12 ap = g12sub(p, a);
  double t = g12dot(ap, d) / g12dot(d, d);
  G12 foot = g12add(a, g12mul(d, t));
  return g12add(foot, g12sub(foot, p));
}

// parameter t with a + t*(b-a) on the line through c and d, or -1 if
// parallel
static inline double g12lineparam(G12 a, G12 b, G12 c, G12 d) {
  double den = g12cross(c, d, b) - g12cross(c, d, a);
  if (fabs(den) < 1e-18)
    return -1;
  return g12cross(c, d, a) / -den;
}

// strict containment in a CCW convex polygon (≥ 3 verts); boundary
// points within eps are not inside
static inline int g12inside(G12 p, const G12 *poly, int vn, double eps) {
  for (int i = 0; i < vn; i++)
    if (g12cross(poly[i], poly[(i + 1) % vn], p) ≤ eps)
      return 0;
  return 1;
}
```

listing 508: c, the 2d kit: vector ops, reflection, line-crossing parameter, strict containment (source: icpc/samples-c/src/Ch12/ch12_geom.h, lines 12-49)

```go
// geoCrossG is the z of the cross product (a-o) x (b-o).
func geoCrossG(o, a, b gVec) float64 {
  return (a.x-o.x)*(b.y-o.y) - (a.y-o.y)*(b.x-o.x)
}

// geoSegHitG reports whether the closed segments p1-p2 and q1-q2 meet;
// on success it returns the meeting point and its parameter along
// p1-p2. Parallel overlaps are rejected: callers treat them as
// degenerate.
func geoSegHitG(p1, p2, q1, q2 gVec) (bool, gVec, float64) {
  d1 := geoSubG(p2, p1)
  d2 := geoSubG(q2, q1)
  den := d1.x*d2.y - d1.y*d2.x
  w := geoSubG(q1, p1)
  scale := geoLenG(d1) * geoLenG(d2)
  if scale == 0 || den > -1e-12*scale && den < 1e-12*scale {
    return false, gVec{}, 0
  }
  t := (w.x*d2.y - w.y*d2.x) / den
  u := (w.x*d1.y - w.y*d1.x) / den
  tol := 1e-9
  if t < -tol || t > 1+tol || u < -tol || u > 1+tol {
    return false, gVec{}, 0
  }
  if t < 0 {
    t = 0
  }
  if t > 1 {
    t = 1
  }
  return true, geoAddG(p1, geoMulG(d1, t)), t
}

// geoInsideG reports whether p lies strictly inside the CCW convex
// polygon given by its vertices, with a relative tolerance so
// boundary points (segments hugging an edge) never count as inside.
func geoInsideG(p gVec, poly []gVec) bool {
  for i := range poly {
    a, b := poly[i], poly[(i+1)%len(poly)]
    c := geoCrossG(a, b, p)
    scale := geoLenG(geoSubG(b, a)) * geoDistG(a, p)
    if scale == 0 || c <= 1e-9*scale {
      return false
    }
  }
  return true
}
```

listing 509: go, geoSegHitG with relative tolerances and the clamped meeting parameter, geoInsideG strict against a CCW polygon (source: icpc/samples-go/ch12/pd_geo.go, lines 22-68)

```
  static P sub(P a, P b) { return new P(a.x() - b.x(), a.y() - b.y()); }

  static P add(P a, P b) { return new P(a.x() + b.x(), a.y() + b.y()); }

  static P mul(P a, double s) { return new P(a.x() * s, a.y() * s); }

  static double dot(P a, P b) { return a.x() * b.x() + a.y() * b.y(); }

  static double len(P a) { return Math.sqrt(dot(a, a)); }

  static P l90(P a) { return new P(-a.y(), a.x()); }

  static double cross(P o, P a, P b) {
    return (a.x() - o.x()) * (b.y() - o.y()) - (a.y() - o.y()) * (b.x() - o.x());
  }

  // mirror p across the line through a and b
  static P reflect(P p, P a, P b) {
    P d = sub(b, a);
    P ap = sub(p, a);
    double t = dot(ap, d) / dot(d, d);
    P foot = add(a, mul(d, t));
    return add(foot, sub(foot, p));
  }

  // parameter t with a + t*(b-a) on the line through c and d, or -1
  // if parallel
  static double lineParam(P a, P b, P c, P d) {
    double den = cross(c, d, b) - cross(c, d, a);
    if (Math.abs(den) < 1e-18)
      return -1;
    return cross(c, d, a) / -den;
  }

  // strict containment in a CCW convex polygon (≥ 3 verts); boundary
  // points within eps are not inside
  static boolean inside(P p, P[] poly, double eps) {
    for (int i = 0; i < poly.length; i++)
      if (cross(poly[i], poly[(i + 1) % poly.length], p) ≤ eps)
        return false;
    return true;
  }
```

listing 510: java, the 2d kit over record points: vector ops, reflection, line-crossing parameter, strict containment (source: icpc/samples-java/src/Ch13/Geom.java, lines 8-49)

```
export const gCross = (o, a, b) => (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);

// do the closed segments p1-p2 and q1-q2 meet? on success return the
// meeting point and its parameter along p1-p2. parallel overlaps are
// rejected: callers treat them as degenerate
export function segHit(p1, p2, q1, q2) {
  const d1 = gSub(p2, p1);
  const d2 = gSub(q2, q1);
  const den = d1.x * d2.y - d1.y * d2.x;
  const w = gSub(q1, p1);
  const scale = gLen(d1) * gLen(d2);
  if (scale === 0 || (den > -1e-12 * scale && den < 1e-12 * scale)) return null;
  const t = (w.x * d2.y - w.y * d2.x) / den;
  const u = (w.x * d1.y - w.y * d1.x) / den;
  const tol = 1e-9;
  if (t < -tol || t > 1 + tol || u < -tol || u > 1 + tol) return null;
  const tc = Math.min(1, Math.max(0, t));
  return { pt: gAdd(p1, gMul(d1, tc)), t: tc };
}

// is p strictly inside the CCW convex polygon, with a relative
// tolerance so boundary points never count as inside
export function insideStrict(p, poly) {
  for (let i = 0; i < poly.length; i++) {
    const a = poly[i], b = poly[(i + 1) % poly.length];
    const c = gCross(a, b, p);
    const scale = gLen(gSub(b, a)) * gDist(a, p);
    if (scale === 0 || c ≤ 1e-9 * scale) return false;
  }
  return true;
}
```

listing 511: javascript, gCross, segHit rejecting parallel overlaps, insideStrict with the relative bound (source: icpc/samples-js/src/ch12-geom.mjs, lines 16-46)

```python
def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])


def add(a, b):
    return (a[0] + b[0], a[1] + b[1])


def mul(a, s):
    return (a[0] * s, a[1] * s)


def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]


def length(a):
    return math.sqrt(dot(a, a))


def l90(a):
    return (-a[1], a[0])


def cross(o, a, b):
    return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0])
```

listing 512: python, the tuple algebra: sub through l90 and cross, the pyramid walk's atoms (source: icpc/samples-py/src/Ch12/ch12_geom.py, lines 7-32)

```lua
function geom.l90(x, y)
  return -y, x
end

function geom.sub(ax, ay, bx, by)
  return ax - bx, ay - by
end

function geom.add(ax, ay, bx, by)
  return ax + bx, ay + by
end

function geom.mul(x, y, s)
  return x * s, y * s
end

function geom.dot(ax, ay, bx, by)
  return ax * bx + ay * by
end

function geom.cross2d(ox, oy, ax, ay, bx, by)
  return (ax - ox) * (by - oy) - (ay - oy) * (bx - ox)
end

function geom.len(x, y)
  return math.sqrt(x * x + y * y)
end
```

listing 513: lua, the flat scalar pairs, l90 through len, cross2d reading ccw positive (source: icpc/samples-lua/ch12_geom.lua, lines 5-31)

The string kit backs problem F: the failure function, an all-occurrence search, and the minimal period F runs on doubled cycle strings. The four twins agree on the interface, C's `Ch12/ch12_kmp.h` reporting hits through a callback, javascript and python returning positions, java the same hits as int arrays, lua 1-based with the failure table reused between calls. C# and go carry no twin file: the C# `CycMatch` walks its own failure function and go's `cycMatch` closure builds its table inline.

```c
static inline void ch12_fail(const char *s, int n, int *f) {
  f[0] = 0;
  for (int i = 1, k = 0; i < n; i++) {
    while (k && s[i] != s[k])
      k = f[k - 1];
    if (s[i] == s[k])
      k++;
    f[i] = k;
  }
}

// calls hit(at) for every occurrence of pat (m) in txt (n);
// f must hold pat's failure function
static inline void ch12_find_all(const char *txt, int n, const char *pat, int m,
                                 const int *f, void (*hit)(int, void *),
                                 void *ctx) {
  for (int i = 0, k = 0; i < n; i++) {
    while (k && txt[i] != pat[k])
      k = f[k - 1];
    if (txt[i] == pat[k])
      k++;
    if (k == m) {
      hit(i - m + 1, ctx);
      k = f[k - 1];
    }
  }
}

static inline int ch12_min_period(const char *s, int n, int *f) {
  ch12_fail(s, n, f);
  int p = n - f[n - 1];
  return n % p == 0 ? p : n;
}
```

listing 514: c, failure function, all-occurrence search, and the minimal period F uses on doubled cycles (source: icpc/samples-c/src/Ch12/ch12_kmp.h, lines 9-41)

```java
static int[] fail(char[] s) {
  int n = s.length;
  int[] f = new int[n];
  for (int i = 1, k = 0; i < n; i++) {
    while (k > 0 && s[i] != s[k])
      k = f[k - 1];
    if (s[i] == s[k])
      k++;
    f[i] = k;
  }
  return f;
}

// every start index of pat (m) inside txt (n); f must hold pat's
// failure function
static int[] findAll(char[] txt, char[] pat, int[] f) {
  int n = txt.length, m = pat.length;
  List<Integer> hits = new ArrayList<>();
  for (int i = 0, k = 0; i < n; i++) {
    while (k > 0 && txt[i] != pat[k])
      k = f[k - 1];
    if (txt[i] == pat[k])
      k++;
    if (k == m) {
      hits.add(i - m + 1);
      k = f[k - 1];
    }
  }
  int[] out = new int[hits.size()];
  for (int i = 0; i < out.length; i++)
    out[i] = hits.get(i);
  return out;
}

static int minPeriod(char[] s) {
  int n = s.length;
  int[] f = fail(s);
  int p = n - f[n - 1];
  return n % p == 0 ? p : n;
}
```

listing 515: java, fail, findAll returning every overlapping hit, minPeriod off the last border (source: icpc/samples-java/src/Ch13/Kmp.java, lines 8-47)

```
export function kmpFail(s, n, f) {
  f[0] = 0;
  for (let i = 1, k = 0; i < n; i++) {
    while (k && s[i] !== s[k]) k = f[k - 1];
    if (s[i] === s[k]) k++;
    f[i] = k;
  }
}

// calls hit(at) for every occurrence of pat (m) in txt (n); f must
// hold pat's failure function
export function kmpFindAll(txt, n, pat, m, f, hit) {
  for (let i = 0, k = 0; i < n; i++) {
    while (k && txt[i] !== pat[k]) k = f[k - 1];
    if (txt[i] === pat[k]) k++;
    if (k === m) {
      hit(i - m + 1);
      k = f[k - 1];
    }
  }
}

export function kmpMinPeriod(s, n, f) {
  kmpFail(s, n, f);
  const p = n - f[n - 1];
  return n % p === 0 ? p : n;
}
```

listing 516: javascript, kmpFail, kmpFindAll through the hit callback, kmpMinPeriod off the last border (source: icpc/samples-js/src/ch12-kmp.mjs, lines 5-31)

```python
def fail(s):
    n = len(s)
    f = [0] * n
    k = 0
    for i in range(1, n):
        while k and s[i] != s[k]:
            k = f[k - 1]
        if s[i] == s[k]:
            k += 1
        f[i] = k
    return f


def find_all(txt, pat, f=None):
    if f is None:
        f = fail(pat)
    m = len(pat)
    k = 0
    for i, ch in enumerate(txt):
        while k and ch != pat[k]:
            k = f[k - 1]
        if ch == pat[k]:
            k += 1
        if k == m:
            yield i - m + 1
            k = f[k - 1]


def min_period(s):
    f = fail(s)
    p = len(s) - f[-1]
    return p if len(s) % p == 0 else len(s)
```

listing 517: python, fail, find_all yielding every overlapping hit, min_period from the last border (source: icpc/samples-py/src/Ch12/ch12_kmp.py, lines 3-34)

```lua
function kmp.fail(s, n)
  local f = {0}
  local k = 0
  for i = 2, n do
    while k > 0 and s[i] ~= s[k + 1] do
      k = f[k]
    end
    if s[i] == s[k + 1] then
      k = k + 1
    end
    f[i] = k
  end
  return f
end

-- calls hit(at) for every occurrence of pat (m) in txt (n); f must
-- hold pat's failure function
function kmp.find_all(txt, n, pat, m, f, hit)
  local k = 0
  for i = 1, n do
    while k > 0 and txt[i] ~= pat[k + 1] do
      k = f[k]
    end
    if txt[i] == pat[k + 1] then
      k = k + 1
    end
    if k == m then
      hit(i - m + 1)
      k = f[k]
    end
  end
end

function kmp.min_period(s, n)
  local f = kmp.fail(s, n)
  local p = n - f[n]
  if n % p == 0 then return p end
  return n
end
```

listing 518: lua, kmp.fail 1-based, kmp.find_all calling hit per occurrence, kmp.min_period (source: icpc/samples-lua/ch12_kmp.lua, lines 7-45)

14.2. the go scan cursor

The scan kit is the go package's io contract, one file the whole year shares: `scan12` walks the input buffer with a whitespace `token`, a signed `nextInt`, and `line` for the fixed-line judge files, the same cursor design chapter 12 named and chapter 13 printed for C. Ten of the eleven go solvers parse through it, everything but the sphinx's four-line fixed file, which `SolveA` splits directly.

```
// scan12 is the year-12 whitespace tokenizer over the whole input.
type scan12 struct {
  data []byte
  pos  int
}

func blank12(c byte) bool {
  return c == ' ' || c == '\n' || c == '\r' || c == '\t'
}

func (s *scan12) token() []byte {
  for s.pos < len(s.data) && blank12(s.data[s.pos]) {
    s.pos++
  }
  start := s.pos
  for s.pos < len(s.data) && !blank12(s.data[s.pos]) {
    s.pos++
  }
  return s.data[start:s.pos]
}

func (s *scan12) nextInt() int {
  tok := s.token()
  neg := false
  i := 0
  if i < len(tok) && (tok[i] == '-' || tok[i] == '+') {
    neg = tok[i] == '-'
    i++
  }
  v := 0
  for ; i < len(tok); i++ {
    v = v*10 + int(tok[i]-'0')
  }
  if neg {
    return -v
  }
  return v
}

func (s *scan12) line() string {
  for s.pos < len(s.data) && (s.data[s.pos] == '\r' || s.data[s.pos] == '\n') {
    s.pos++
  }
  start := s.pos
  for s.pos < len(s.data) && s.data[s.pos] != '\n' {
    s.pos++
  }
  return strings.TrimSuffix(string(s.data[start:s.pos]), "\r")
}
```

listing 519: go, scan12: token skips whitespace in place, nextInt folds a signed value, line reads through the next newline (source: icpc/samples-go/ch12/ch12.go, lines 5-53)

Java ships no shared cursor: every solver reads the whole input through a BufferedReader into one string and walks it with a private index scanner cut to the year's formats, sign-aware only in I, whose crafted fixtures feed a negative reading, and every solver writes its answer through one System.out.print.

14.3. a, riddle of the sphinx

This is the 2022 W task back under a new letter, and the judge files for the two years are byte-identical. The statement reverses the classic riddle: name the number of legs of an axex, a basilisk, and a centaur, each an unknown nonnegative integer, and instead of guessing you may ask the sphinx five questions. A question is a triple (a, b, c), how many legs do a axexes, b basilisks, and c centaurs have in total, and she answers each with a single integer. The catch is that sphinxes are tricky: at most one of her five answers may be an outright lie, and you do not know which one, so the task is to choose five questions that pin the three leg counts regardless.

This is the set's only interactive problem. There are exactly five rounds: each round the program writes one line of three integers a, b, and c with 0 <= a, b, c <= 10 by the problems pdf, then reads her answer, one integer r with 0 <= r <= 1e5. After the fifth round the program prints one line of three space-separated nonnegative integers, the leg counts of the axex, the basilisk, and the centaur, under the pdf's 2 second limit. The local judge harness replaces the live interaction with the fixed four-line file described below the listings, and because the official .ans files are empty transcripts the harness compares that single output line against the file's truth through its sphinx mode.

Official sample 1, reprinted byte for byte from the judge data pair `A-riddleofthesphinx/sample-1.in` and `sample-1.ans`, the same interaction the problems pdf prints as its first transcript: the truth is 4 4 4 and the lying round is the second of five with additive delta 1, so asked (1,1,1), (1,1,1), (5,0,1), (1,0,0), and (1,1,0) she answers 12, 13, 24, 4, and 8. The .ans side of the pair is empty by design, so the expected block below prints the line the sphinx mode demands, the truth triple from line 2:

input:

```
fixed
4 4 4
1 1
4 4 4
```

expected output:

```
4 4 4
```

Recognition: five rounds, coefficients at most 10 and answers at most 1e5 under the 2 second limit make the budget trivial: six exclusion hypotheses, no lie or the lie on round k, each resolved by Cramer's rule on a 3 by 3 system and cross-checked against the survivors, about sixty tiny solves in all, so the whole difficulty is the model, not the cost.

The statement's cue is at most one of her five answers may be an outright lie and you do not know which one: the consistent world is recovered by excluding each suspect round in turn and keeping the one hypothesis whose survivors all agree. Problem A is the 2023 face of the consistency under a faulty oracle family. Consistency under a faulty oracle ties chapter 13 problem W and chapter 15 problem D together.

Searching the triple space is the tempting tool and the numbers kill it: every answer is a sum of at most thirty coefficient legs bounded by 1e5, so the triples run past 1e15 candidates against five answers, and any adaptive questioning that trusts a single answer is steered wrong by the one lie, while five rows with every triple independent decide outright.

The book asks the five questions (1,0,0), (0,1,0), (0,0,1), (1,1,1), (1,2,3), whose every triple of rows is linearly independent, the ten triple determinants running −2, −2, −1, 1, 1, 1, 1, 1, 1, and 3, the (q1,q2,q5) triple at 3, a multiset pinned by a reference computation after an earlier gloss of plain +-1 and +-2 proved wrong for that one triple. Each exclusion hypothesis, the lie hit round k or there is no lie, leaves a system where some three surviving rows determine the legs by Cramer's rule and the other survivors cross-check them, and exactly one hypothesis survives, the argument on the solutions pdf page 2, problem at icpc2023.pdf. Book 10′s chapter 34, linear algebra, builds the determinant screen and the small-system solve this lie isolation rests on. The last row is not negotiable:

replacing (1,2,3) with (1,1,2) or (0,1,2) creates dependent triples and breaks the guarantee. The deduction is O(1) after five answers.

The worked run: trace the model on sample 1. The judge file names truth 4 4 4 and the lie on 0-based round 1 with delta 1, so the book's five questions (1,0,0), (0,1,0), (0,0,1), (1,1,1), (1,2,3) draw the answers 4, 5, 4, 12, 24, the second carrying 4 + 1 = 5. The no-lie hypothesis solves rows one through three for x = 4, y = 5, z = 4, and row four demands x + y + z = 13 against the drawn 12, dead. The lie-on-round-two hypothesis survives: rows one and three give x = 4 and z = 4, row four gives y = 12 - 4 - 4 = 4, and row five cross-checks `4 + 2*4 + 3*4 = 24` = 24. Every other placement dies on the same arithmetic, the table below, the legs are 4 4 4, and the trace ends at the printed answer `4 4 4`.

hypothesis	legs solved	cross-check	verdict
no lie	4 5 4	row four 13 vs 12	dead
lie on row 1	3 5 4	row five 25 vs 24	dead
lie on row 2	4 4 4	row five 24 = 24	the survivor
lie on row 3	4 5 3	row five 23 vs 24	dead
lie on row 4	4 5 4	row five 26 vs 24	dead
lie on row 5	4 5 4	row four 13 vs 12	dead

```c
static int deduce(const long long *ans, long long *legs) {
  for (int k = 0; k ≤ 5; k++) { // k = 5: no lie
    int rest[5], nr = 0;
    for (int i = 0; i < 5; i++)
      if (i ≠ k)
        rest[nr++] = i;
    for (int a = 0; a < nr; a++)
      for (int b = a + 1; b < nr; b++)
        for (int c = b + 1; c < nr; c++) {
          int sel[3] = {rest[a], rest[b], rest[c]};
          long long lg[3];
          if (!cram3(sel, ans, lg))
            continue;
          if (lg[0] < 0 || lg[1] < 0 || lg[2] < 0)
            continue;
          int ok = 1;
          for (int i = 0; i < 5 && ok; i++)
            if (i ≠ k &&
                QX[i] * lg[0] + QY[i] * lg[1] + QZ[i] * lg[2] ≠ ans[i])
              ok = 0;
          if (ok) {
            legs[0] = lg[0];
            legs[1] = lg[1];
            legs[2] = lg[2];
            return 1;
          }
        }
  }
  return 0;
}
```

listing 520: c, deduce: every 3-subset of every hypothesis through Cramer, integer and nonnegative, then verify all rows (source: icpc/samples-c/src/Ch12/pA.c, lines 48-77)

```
  }
  rows := [5][3]int64{{1, 0, 0}, {0, 1, 0}, {0, 0, 1}, {1, 1, 1}, {1, 2, 3}}
  for lie := 0; lie <= 5; lie++ {
    var pick []int
    for i := 0; i < 5 && len(pick) < 3; i++ {
      if i != lie-1 {
        pick = append(pick, i)
      }
    }
    a := [3][3]int64{}
    for r := 0; r < 3; r++ {
      a[r] = rows[pick[r]]
    }
    det := det3(a)
    if det == 0 {
      continue
    }
    ok := true
    var legs [3]int64
    for col := 0; col < 3 && ok; col++ {
      m := a
      for r := 0; r < 3; r++ {
        m[r][col] = ans[pick[r]]
      }
      num := det3(m)
      if num%det != 0 {
        ok = false
        break
      }
      legs[col] = num / det
    }
    if !ok {
      continue
    }
    for i := 0; i < 5; i++ {
      if i == lie-1 {
        continue
      }
      if rows[i][0]*legs[0]+rows[i][1]*legs[1]+rows[i][2]*legs[2] != ans[i] {
        ok = false
        break
      }
    }
    if ok {
      return strconv.FormatInt(legs[0], 10) + " " +
        strconv.FormatInt(legs[1], 10) + " " +
        strconv.FormatInt(legs[2], 10) + "\n"
    }
  }
  return "0 0 0\n"
}
```

listing 521: go, one det3 helper, the first three surviving rows per hypothesis, then the full cross-check (source: icpc/samples-go/ch12/pa.go, lines 30-81)

```java
static long[] deduce(long[] ans) {
  for (int k = 0; k ≤ 5; k++) { // k = 5: no lie
    int[] rest = new int[5];
    int nr = 0;
    for (int i = 0; i < 5; i++)
      if (i ≠ k)
        rest[nr++] = i;
    for (int a = 0; a < nr; a++)
      for (int b = a + 1; b < nr; b++)
        for (int c = b + 1; c < nr; c++) {
          long[] lg = cram3(new int[] {rest[a], rest[b], rest[c]}, ans);
          if (lg == null || lg[0] < 0 || lg[1] < 0 || lg[2] < 0)
            continue;
          boolean good = true;
          for (int i = 0; i < 5 && good; i++)
            if (i ≠ k && QX[i] * lg[0] + QY[i] * lg[1] + QZ[i] * lg[2] ≠ ans[i])
              good = false;
          if (good)
            return lg;
        }
  }
  return null;
}
```

listing 522: java, deduce over every hypothesis and 3-subset through cram3, exact quotients and nonnegative legs, then the full cross-check (source: icpc/samples-java/src/Ch13/PA.java, lines 58-80)

```
    public static string Solve(string input)
    {
        int pos = 0;
        while (pos < input.Length && input[pos] ≠ '\n')
            pos++;
        pos++;
        long ta = Next(input, ref pos);   // true triple on line 2
        long tb = Next(input, ref pos);
        long tc = Next(input, ref pos);
        long rnd = Next(input, ref pos);  // lie round (5 = never)
        long delta = Next(input, ref pos);
        long aa = ta, ab = tb, ac = tc;   // crafted three-line fixtures: no line 4
        if (pos < input.Length)
        {
            long x = Next(input, ref pos);
            if (pos < input.Length)
            {
                aa = x;
                ab = Next(input, ref pos);
                ac = Next(input, ref pos);
            }
        }
        var ans = new long[5];
        for (int i = 0; i < 5; i++)
            ans[i] = Qx[i] * ta + Qy[i] * tb + Qz[i] * tc;
        if (rnd ≥ 0 && rnd < 5)
            ans[rnd] = Qx[rnd] * aa + Qy[rnd] * ab + Qz[rnd] * ac + delta;
        var got = Deduce(ans);
        return got == null ? "" : $"{got[0]} {got[1]} {got[2]}\n";
    }
```

listing 523: c#, Solve reads the four-line judge file: truth on line 2, the lying round answered from line 4 plus the delta (source: icpc/samples/src/Ch12/PA.cs, lines 26-55)

```
export function deduce(ans) {
  for (let lie = 0; lie ≤ 5; lie++) {
    // lie = 5: no lie
    const pick = [];
    for (let i = 0; i < 5 && pick.length < 3; i++)
      if (i =/= lie) pick.push(i);
    const a = pick.map((r) => [QX[r], QY[r], QZ[r]]);
    const det = det3(a);
    if (det === 0) continue;
    const legs = [0, 0, 0];
    let ok = true;
    for (let col = 0; col < 3 && ok; col++) {
      const m = a.map((row) => row.slice());
      for (let r = 0; r < 3; r++) m[r][col] = ans[pick[r]];
      const num = det3(m);
      if (num % det =/= 0) {
        ok = false;
        break;
      }
      legs[col] = num / det;
    }
    if (!ok) continue;
    for (let i = 0; i < 5; i++) {
      if (i === lie) continue;
      if (QX[i] * legs[0] + QY[i] * legs[1] + QZ[i] * legs[2] =/= ans[i]) {
        ok = false;
        break;
      }
    }
    if (ok) return legs;
  }
  return null;
}
```

listing 524: javascript, the exported pure core, null on a singular or non-integer system (source: icpc/samples-js/src/ch12-pa-riddleofthesphinx.mjs, lines 21-54)

```
def deduce(ans):
    for k in range(6):  # k = 5: no lie
        rest = [i for i in range(5) if i != k]
        for a in range(len(rest)):
            for b in range(a + 1, len(rest)):
                for c in range(b + 1, len(rest)):
                    legs = _cram3((rest[a], rest[b], rest[c]), ans)
                    if legs is None or min(legs) < 0:
                        continue
                    la, lb, lc = legs
                    if all(
                        QX[i] * la + QY[i] * lb + QZ[i] * lc == ans[i]
                        for i in range(5)
                        if i != k
                    ):
                        return legs
    return None


def solve(text):
    spx = Sphinx(text)
    ans = [spx.ask(QX[i], QY[i], QZ[i]) for i in range(5)]
    legs = deduce(ans)
    if legs is None:
        raise SystemExit("no consistent hypothesis")
    return f"{legs[0]} {legs[1]} {legs[2]}\n"


if __name__ == "__main__":
    if len(sys.argv) > 1:
        with open(sys.argv[1], encoding="utf-8") as fh:
            print(solve(fh.read()), end="")
    else:
        n = 0
```

listing 525: python, deduce as nested comprehensions over hypotheses and triples, solve simulates through the helper (source: icpc/samples-py/src/Ch12/pa.py, lines 46-79)

```
local function deduce(ans)
  for k = 0, 5 do -- k = 5: no lie
    local rest, nr = {}, 0
    for i = 0, 4 do
      if i ~= k then
        nr = nr + 1
        rest[nr] = i
      end
    end
    for a = 1, nr do
      for b = a + 1, nr do
        for c = b + 1, nr do
          local legs = cram3({rest[a], rest[b], rest[c]}, ans)
          if legs and legs[1] ≥ 0 and legs[2] ≥ 0 and legs[3] ≥ 0 then
            local ok = true
            for i = 0, 4 do
              if i ~= k and
                 QX[i + 1] * legs[1] + QY[i + 1] * legs[2] +
                 QZ[i + 1] * legs[3] ~= ans[i + 1] then
                ok = false
                break
              end
            end
            if ok then return legs end
          end
        end
      end
    end
  end
  return nil
end
```

listing 526: lua, deduce over a flat nine-scalar determinant, legs rejected unless the quotient is exact (source: icpc/samples-lua/ch12_pa.lua, lines 47-77)

The judge file format cost this wave a retraction, recorded here because the wrong reading looked green for a year. The four lines are the literal word `fixed`, the TRUE leg counts, the 0-based lying round where 5 means never plus an additive delta, and ALTERNATE leg counts. Every answer is truthful from line 2 except the lying round, answered from the line-4 triple plus the delta. The 30 use-alt-sol secrets put a different triple on line 4 with delta 0, the swap itself is the lie, and the other 20 cases and both samples carry line 4 equal to line 2 with a real delta, sample 2 runs delta −6241 so answers may go negative. The correct output is always the line-2 triple. The original 2022-era reading treated line 2 as a decoy and emulated from line 4, which is circular validation, the emulator agreeing with itself rather than with the sphinx, and it was retracted when commit 64d294b fixed the harness compare for 2022/W from a four-line parse to the plain sphinx mode. The C# port self-played line 4 and failed exactly the 30 use-alt-sol secrets before commit 1367ea9 moved it to truth from line 2 with the lying round from line 4, and its suite now pins both traps, the swap fixture `fixed / 1 2 3 / 1 0 / 1750 2795 941` answering `1 2 3` and the delta lie with an identical line 4. The other five languages answer from line 2 with the delta applied on the lying round, which provably prints the same triple, since with all-truthful answers every consistent hypothesis recovers the true legs. All seven languages now judge 50 of 50 on both years.

The crafted fixture is `fixed / 2 5 7 / 3 1`: rounds one through three read the legs off directly, round four answers 15 instead of 14, and round five cross-checks at 33, so the lie sits on round four and the legs are 2 5 7. Every suite asserts that pair plus the never-lying `fixed / 9 8 7 / 2 0` answering 9 8 7: c runs 2 CHECKs, the C# suite 4 facts including the two judge traps, go pins 2 table rows, java 7 ok checks, javascript 2 it blocks carrying 3 assertions, python 2 checks, lua 2 named checks. Products stay under 10 times the leg sums, well inside int32, so no language needs wider arithmetic than its default int, and go stores the answers in int64 only because it prints them there.

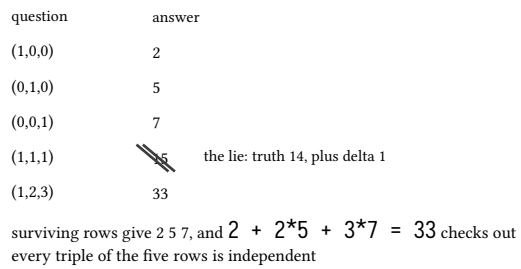


figure 157: the five question rows, the lie bolted onto round four, the surviving rows solving the fixture

14.4. b, schedule

The institute for creative product combinations fields n two-person teams, members (i, 1) and (i, 2), and keeps its pandemic rule that each week exactly one member of every team is in the office. Teammates collaborate fine apart, but colleagues from different teams must meet, and for two of them with shared office weeks w1 < w2 < ... < wk their isolation is the maximum of w1, w2 - w1, ..., wk - wk-1, and w + 1 - wk, the head before their first shared week, the gaps between shared weeks, and the tail after the last, or infinity if they never meet at all. The company's isolation is the worst over all colleague pairs from different teams, and the task schedules the w weeks so that worst is as small as it can be.

The input is a single line of two integers, the team count n with 2 <= n <= 1e4 and the week count w with 1 <= w <= 52, by the problems pdf. The output is one line holding either the minimum isolation as an integer or the word infinity, followed, when it is finite, by w schedule lines: line j is a string of length n over the symbols 1 and 2, the ith symbol naming which member of team i comes in during week j. Any schedule achieving the optimum is accepted, the pdf says so plainly, which is why this book's harness scores B on the first line alone, under the pdf's 2 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `B-schedule/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: two teams over six weeks meet with worst gap 4, and the judges' block is one of many valid schedules, the book's canonical rows below answer the same 4 with different lines.

input:

```
2 6
```

expected output:

```
4
11
12
21
22
11
12
```

Official sample 2, same source: with one week and two teams, each team seats one member and two of the four colleague pairs never meet, so the answer is infinity.

input:

```
2 1
```

expected output:

```
infinity
```

Recognition: n <= 1e4 teams and w <= 52 weeks under the 2 second limit price a formula, not a search: the viable block lengths k run 1 through 17, since C(16, 8) = 12870 already seats 1e4 teams, and the output itself is w lines of n symbols, at most 5.2e5 characters, so the solve is a binomial scan plus one emission pass.

The statement's cue is schedules the w weeks so that worst is as small as it can be, plus any schedule achieving the optimum is accepted: a minimum with a printed witness is the classic shape of a closed-form characterization, prove exactly which isolations are achievable at all, then emit one canonical block. Problem B is the 2023 face of the constructive closed-form characterization family. The constructive closed-form characterization family holds chapter 12 problem U and chapter 15 problem J.

Searching week by week is priced out by counting alone: each week admits 2^n member choices with n up to 1e4, a number with 3000 digits, chained over 52 weeks, while the block characterization answers in 17 binomial lookups.

An isolation-k schedule is periodic, the solutions pdf opens, so the task reduces to the smallest k in which every pair can meet, printing infinity when that k exceeds w. Code each team as a binary string of length k, one bit per week naming which member is in: two colleagues (i, a) and (j, b) share exactly the weeks whose bit pair for teams i and j reads (a-1, b-1), so two strings are compatible, all four bit pairs occurring across the k weeks, precisely when every colleague pair meets somewhere in the block, and repeating a compatible block cyclically keeps every gap inside one period. Strings starting with 0 and carrying ceil(k/2) ones among the remaining k-1 bits are pairwise compatible, an antichain in the subset lattice whose equal suffix weights also force a shared 1 by pigeonhole, and Sperner's theorem, with a complement argument for odd k, makes C(k-1, ceil(k/2)) the exact maximum team count, so k stays at most 17 for n up to 1e4. Emit the n largest such strings in descending lexicographic order, team 1 taking the largest, and repeat the block cyclically to w weeks, `O(n*k)` after a tiny binomial search, the construction on the solutions pdf pages 2 and 3, problem at icpc2023.pdf. Book 10′s chapter 35, combinatorics, teaches the binomial search over ordered selections that pins this k. Descending lex is the canonical output every suite pins.

The worked run: trace the model on sample 1. The k search seats two teams first at k = 4: k = 3 admits C(2, 2) = 1 compatible string, k = 4 admits C(3, 2) = 3. The judges' printed witness read column-wise over its first four weeks, 11, 12, 21, 22, hands team 1 the string 0011 and team 2 the string 0101, both cap members with two ones among the last three bits, and all four colleague pairs meet inside the block: members (1,1) in week 1, (1,2) in week 2, (2,1) in week 3, (2,2) in week 4. Weeks 5 and 6 repeat the block's weeks 1 and 2, so every pair meets again within one period and the isolation of each pair is at most k, the table below, and no k = 3 schedule seats two teams, so 4 is optimal. Week 6 repeats week 2′s row, and the trace ends at the printed answer 12.

colleague pair	shared weeks	isolation
(1,1)	1 and 5	max(1, 5 - 1, 7 - 5) = 4
(1,2)	2 and 6	max(2, 6 - 2, 7 - 6) = 4
(2,1)	3	max(3, 7 - 3) = 4
(2,2)	4	max(4, 7 - 4) = 4
company	all four pairs	4 = k

```c
static char *solve(const char *in) {
  char *buf = dupstr(in);
  Ch11Scan sc;
  ch11_scan_init(&sc, buf);
  long long n = ch11_next_int64(&sc);
  long long w = ch11_next_int64(&sc);
  free(buf);
  int k = 1;
  while (ch12_binom(k - 1, (k + 1) / 2) < n)
    k++;
  if (k > w) {
    char *out = malloc(24);
    snprintf(out, 24, "infinity\n");
    return out;
  }
  int c = (k + 1) / 2; // ceil(k/2) ones among the k-1 suffix bits
  char *strs = malloc((size_t)n * k);
  Ctx ctx = {strs, k};
  ch12_emit(k - 1, c, (int)n, take_str, &ctx);
  char *out = malloc(16 + (size_t)w * (n + 1));
  char *o = out;
  o += snprintf(o, 16, "%d\n", k);
  for (long long t = 0; t < w; t++) {
    for (long long i = 0; i < n; i++)
      *o++ = strs[i * k + t % k] == '0' ? '1' : '2';
    *o++ = '\n';
  }
  *o = 0;
  free(strs);
  return out;
}
```

listing 527: c, the whole solve: k search, callback emission of the n largest strings, cyclic repeat to w (source: icpc/samples-c/src/Ch12/pB.c, lines 38-68)

```
  if k > w {
    return "infinity\n"
  }
  // strings: bit 1 = 0, exactly ceil(k/2) ones among bits 2..k,
  // descending lex order; take the first n
  ones := (k + 1) / 2
  var strs []string
  var gen func(pos int, onesLeft int, cur []byte)
  gen = func(pos int, onesLeft int, cur []byte) {
    if len(strs) ≥ n {
      return
    }
    if pos == k {
      strs = append(strs, string(cur))
      return
    }
    // descending lex: try 1 first
    if onesLeft > 0 {
      cur[pos] = '1'
      gen(pos+1, onesLeft-1, cur)
    }
    rem := k - pos - 1
    if rem ≥ onesLeft {
      cur[pos] = '0'
      gen(pos+1, onesLeft, cur)
    }
  }
  cur := make([]byte, k)
  cur[0] = '0'
  gen(1, ones, cur)

  var b strings.Builder
  b.WriteString(strconv.Itoa(k))
  b.WriteByte('\n')
  for week := 0; week < w; week++ {
```

listing 528: go, gen tries the 1 branch first so the recursion itself emits descending lex (source: icpc/samples-go/ch12/pb.go, lines 29-63)

```
static String solve(String in) {
  src = in;
  sp = 0;
  long n = nextLong();
  long w = nextLong();
  int k = 1;
  while (Comb.binom(k - 1, (k + 1) / 2) < n)
    k++;
  if (k > w)
    return "infinity\n";
  int c = (k + 1) / 2; // ceil(k/2) ones among the k-1 suffix bits
  int[][] subsets = Comb.firstSubsets(k - 1, c, (int) n);
  char[][] strs = new char[(int) n][k];
  for (int i = 0; i < n; i++) {
    java.util.Arrays.fill(strs[i], '0');
    for (int pos : subsets[i])
      strs[i][pos + 1] = '1'; // suffix position -> string index
  }
  StringBuilder out = new StringBuilder((int) (16 + w * (n + 1)));
  out.append(k).append('\n');
  for (long t = 0; t < w; t++) {
    for (int i = 0; i < n; i++)
      out.append(strs[i][(int) (t % k)] == '0' ? '1' : '2');
    out.append('\n');
  }
  return out.toString();
}
```

listing 529: java, the whole solve: k search through Comb.binom, the n largest strings from Comb.firstSubsets, cyclic repeat to w (source: icpc/samples-java/src/Ch13/PB.java, lines 38-64)

```
    public static string Solve(string input)
    {
        int pos = 0;
        int n = (int)Next(input, ref pos);
        int w = (int)Next(input, ref pos);
        int k = 2;
        while (Binom(k - 1, (k + 1) / 2) < n)
            k++;
        if (k > w)
            return "infinity\n";
        int r = (k + 1) / 2;

        // all length-(k-1) patterns with r ones, descending lex order
        var pats = new List<int>();
        int rem = k - 1;
        for (int mask = 0; mask < 1 << rem; mask++)
        {
            int bits = 0;
            for (int i = 0; i < rem; i++)
                bits += mask >> i & 1;
            if (bits == r)
                pats.Add(mask);
        }
        pats.Sort((a, b) => b.CompareTo(a));
        var str = new int[n];
        for (int i = 0; i < n; i++)
            str[i] = pats[i];                  // leading 0 is implicit

        var sb = new System.Text.StringBuilder();
        sb.Append(k).Append('\n');
        var row = new char[n];
        for (int t = 0; t < w; t++)
        {
            int b = t % k;
            for (int i = 0; i < n; i++)
                row[i] = (char)('1' + (str[i] >> (k - 1 - b) & 1));
            sb.Append(row).Append('\n');
        }
        return sb.ToString();
    }
```

listing 530: c#, all 2^(k-1) masks with r bits enumerated, sorted descending, rows read off the bits (source: icpc/samples/src/Ch12/PB.cs, lines 17-57)

```javascript
export function solve(input) {
  const tok = input.trim().split(/\s+/);
  const n = parseInt(tok[0], 10);
  const w = parseInt(tok[1], 10);
  let k = 2;
  for (; k ≤ 18; k++) if (binom(k - 1, (k + 1) >> 1) ≥ n) break;
  if (k > w) return "infinity\n";
  // bit 1 = 0, exactly ceil(k/2) ones among bits 2..k, descending lex
  const ones = (k + 1) >> 1;
  const strs = firstSubsets(k - 1, ones, n).map((sub) => {
    const bits = new Array(k).fill("0");
    for (const pos of sub) bits[pos + 1] = "1";
    return bits.join("");
  });
  const out = [`${k}`];
  for (let week = 0; week < w; week++) {
    let line = "";
    for (let i = 0; i < n; i++) line += strs[i][week % k] === "0" ? "1" : "2";
    out.push(line);
  }
  return out.join("\n") + "\n";
}
```

listing 531: javascript, firstSubsets from the year-local comb module, bits mapped to member digits (source: icpc/samples-js/src/ch12-pb-schedule.mjs, lines 17-39)

```python
def solve(text):
    n, w = (int(v) for v in text.split())
    k = 2
    while binom(k - 1, (k + 1) // 2) < n:
        k += 1
    if k > w:
        return "infinity\n"
    p = (k + 1) // 2
    suffixes = first_subsets(k - 1, p, n)
    out = [str(k)]
    for wk in range(w):
        j = wk % k
        if j == 0:
            out.append("1" * n)
        else:
            out.append("".join("2" if j - 1 in s else "1" for s in suffixes))
    return "\n".join(out) + "\n"
```

listing 532: python, first_subsets over the one-positions, week 1 is the all-1 column (source: icpc/samples-py/src/Ch12/pb.py, lines 16-33)

```lua
local function solve(input)
  local n = tonumber(input:match("^%s*(%d+)"))
  local w = tonumber(input:match("^%s*%d+%s+(%d+)"))
  local k = 1
  while comb.binom(k - 1, (k + 1) // 2) < n do
    k = k + 1
  end
  if k > w then
    return "infinity\n"
  end
  local c = (k + 1) // 2 -- ceil(k/2) ones among the k-1 suffix bits
  local strs = {}
  comb.emit(k - 1, c, n, function(pos, kk)
    local bits = {"0"}
    for j = 1, k - 1 do bits[j + 1] = "0" end
    for j = 0, kk - 1 do bits[pos[j] + 2] = "1" end
    strs[#strs + 1] = table.concat(bits)
  end)
  local out = {k .. "\n"}
  for t = 0, w - 1 do
    local col = t % k + 1
    local row = {}
    for i = 1, n do
      row[i] = strs[i]:byte(col) == 48 and "1" or "2"
    end
    out[#out + 1] = table.concat(row) .. "\n"
  end
  return table.concat(out)
end
```

listing 533: lua, comb.emit callback builds each string, byte 48 flips bits to office digits (source: icpc/samples-lua/ch12_pb.lua, lines 14-43)

The fixture is 2 teams over 6 weeks: k = 4 is the first count that fits two teams, the strings are 0110 and 0101, and the cyclic repeat prints 11, 22, 21, 12, 11, 22. The colleague pair of both first members meets in weeks 1 and 5, its gap run is 4, and k = 3 admits only one string so it cannot cover two teams, making 4 optimal. Every suite asserts that output plus the crafted 3 4 printing 4 over 111, 221, 212, 122, and 2 1 printing infinity: c 3 CHECKs, the C# suite 3 facts, go 3 table rows, java 7 ok checks, javascript 3 blocks, python 3 checks, lua 3 named checks. The binomials top out at C(16,8) = 12870, so every language stays in plain integers.

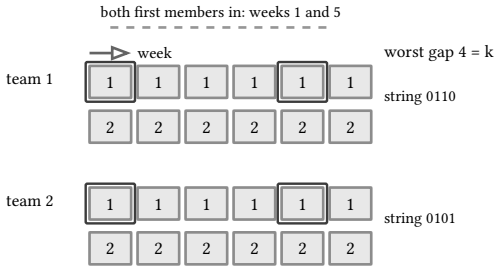


figure 158: the fixture's two team strings over six weeks, the shared weeks of the first-member pair ringed, the worst gap measured

14.5. c, three kinds of dice

The 2022 V task returns with fresh numbers, the Warren Buffett and Bill Gates story of intransitive dice behind it. A die is any collection of at least one face, each face a positive integer, one face chosen uniformly per roll. When two dice roll, the higher face earns its die one point and equal faces earn each die half a point, score(D, D') is D's expected points per roll, D has an advantage over D' when the score tops 1/2, and the two dice tie at exactly 1/2.

Given D1 with an advantage over D2, listed on either input line, the task prints two numbers over all third dice D3, and D3 may be any die at all, any number of faces of any positive integers: the lowest score(D3, D2) among the D3 that tie with or beat D1, and the highest score(D3, D1) among the D3 that D2 ties with or beats. The two scores need not share one D3, and a first score under 1/2 is exactly an intransitive trio.

The input is two lines, one die per line: a face count n with 1 <= n <= 1e5 followed by n face values, each 1 <= f <= 1e9, by the problems pdf, and whichever of the two lines carries the advantage plays D1. The output is one line with the two scores and an absolute error of at most 1e-6, under the pdf's 1 second limit. The face bound binds the input dice only, D3′s faces range over all positive integers, which is the trap the unclamped candidates below return to.

Official sample 1, reprinted byte for byte from the judge data pair `C-threekindsofdice/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints and works its 4/9 example on: the three-face die 2 4 9 beats the six-face die 1 1 6 6 8 8 by 5/9 to 4/9, so here D1 is the second line, and the two answers say a tying D3 can be held to 0.291666667 by D2 while a D2-tied D3 still earns 0.750000000 against D1.

input:

```
6 1 1 6 6 8 8
3 2 4 9
```

expected output:

```
0.291666667 0.750000000
```

Recognition: both dice carry at most 1e5 faces under the 1 second limit. Doubling the score coordinates makes every achievable point an integer pair, the candidate values number at most 2n1 + 2n2 plus two, about 4e5, and the sort, the monotone chain and the two edge walks cost a few million comparisons, two orders of magnitude inside budget.

The statement's cue is a pair of extrema over all third dice D3 under a tie-or-beat constraint: choosing D3 is choosing a distribution over face values, so both queries are constrained optima over one shared achievable set, the cue that the set is a convex hull and each query a halfplane cut on it. Problem C is the 2023 face of the convex hull of achievable points family. Chapter 12 problem V is the other convex hull of achievable points twin.

Enumerating D3 as multisets of the input face values is the tempting tool and it is wrong before it is slow: the winning mixes may need a face one below the union minimum or one past its maximum, judge secret-55 answers through 999999998 and 1000000001, faces no bounded enumeration ever emits.

One of two given dice beats the other and the input may list the winner first or second, so the solver detects the advantage by the exact pairwise score before anything else. A third die D3 is a distribution over face values, and a D3 face showing v earns D1 exactly S1(v), faces of D1 above v plus half the ties, likewise S2(v) against D2, so the achievable average pairs are exactly the convex hull of the doubled points (2 S1, 2 S2) over the candidate values: each distinct merged face value u and u+1, one past the maximum, and one below the minimum when that stays positive, because D3 faces are positive integers unbounded above and the candidate set must never clamp to the input range. A die splitting its faces between two candidate values realizes any point of the segment joining their points, the face ratio is the mixing weight, so hull edges are achievable dice and not just bounds. Part 1 maximizes avgS2 at avgS1 <= n1/2, part 2 minimizes avgS1 at avgS2 >= n2/2, both read off the upper hull with edge interpolation, and the scores print as 1 - avg/n, the model on the solutions pdf pages 3 and 4, O(n log n) for the sort and O(n) for the hull walk, problem at icpc2023.pdf.

The worked run: trace the model on sample 1. The listed-first die wins only 8 of the 18 pairings, 8/18 = 4/9 < 1/2, so D1 is the second line (2, 4, 9) and D2 the first (1, 1, 6, 6, 8, 8). A D3 face showing v earns the doubled integers X(v) against D1 and Y(v) against D2, candidates v = 1 through 10, the value 0 below the union minimum dropped because faces are positive, and the upper hull of the ten points has three vertices, (0, 0) at v = 10, (2, 8) at v = 5, (6,

10) at v = 1. Part 1 walks the hull edge from (2, 8) to (6, 10) out to X = 3 = n1, where Y = 8 + (10 - 8)/(6 - 2) = 8.5, the mix (1, 5, 5, 5) tying D1 at exactly 1/2 while D2 earns 8.5/12, so score(D3, D2) = 1 - 8.5/12 = 7/24. Part 2 walks the edge from (0, 0) to (2, 8) up to Y = 6 = n2, where `X = 2*6/8` = 1.5, the mix (5, 5, 5, 10) tying D2 at 1/2 while D1 earns 1.5/6, so score(D3, D1) = 1 - 1.5/6 = 3/4, and the trace ends at the printed answer `0.291666667 0.750000000`.

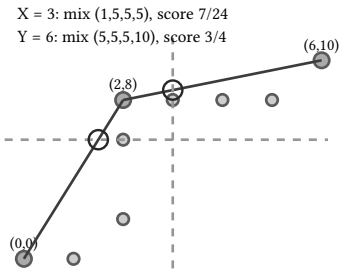


figure 159: sample 1′s doubled score points, the three-vertex upper hull, the two query cuts and their winning mixes

```c
  if (mn >= 2)
    vals[nv++] = mn - 1;
  for (int i = 0; i < na; i++)
    if (i == 0 || da[i] != da[i - 1]) {
      vals[nv++] = da[i];
      vals[nv++] = da[i] + 1;
    }
  for (int i = 0; i < nb; i++)
    if (i == 0 || db[i] != db[i - 1]) {
      vals[nv++] = db[i];
      vals[nv++] = db[i] + 1;
    }
  vals[nv++] = mx + 1;
  qsort(vals, (size_t)nv, sizeof(long long), cmpll);
  H12Pt *pt = malloc((size_t)nv * sizeof(H12Pt));
  int np = 0, i1 = 0, i2 = 0;
  for (int i = 0; i < nv; i++) {
    long long v = vals[i];
    if (i > 0 && v == vals[i - 1])
      continue;
    while (i1 < n1 && d1[i1] < v)
      i1++;
    while (i2 < n2 && d2[i2] < v)
      i2++;
    long long e1 = 0, e2 = 0;
    while (i1 + e1 < n1 && d1[i1 + e1] == v)
      e1++;
    while (i2 + e2 < n2 && d2[i2 + e2] == v)
      e2++;
    pt[np].x = 2 * (n1 - i1 - e1) + e1;
    pt[np].y = 2 * (n2 - i2 - e2) + e2;
    np++;
  }
  qsort(pt, (size_t)np, sizeof(H12Pt), cmppt);
  H12Pt *hull = malloc((size_t)np * sizeof(H12Pt));
  int nh = h12_upper(hull, pt, np);
  double ymax = h12_maxy_at(hull, nh, n1);
  double xmin = h12_minx_at(hull, nh, n2);
  double ans1 = 1.0 - ymax / (2.0 * n2);
  double ans2 = 1.0 - xmin / (2.0 * n1);
```

listing 534: c, candidate values, doubled points through a two-pointer merge, then the two hull queries and scores (source: icpc/samples-c/src/Ch12/pC.c, lines 92-131)

```
sort.Ints(a)
sort.Ints(b)
// D1 is whichever die has the advantage: the doubled pair score
// of a against b from one sorted merge, over distinct values of a
// so duplicate runs are counted, not rescanned
adv := 0
j := 0
for i := 0; i < na; {
  v := a[i]
  run := 1
  for i+run < na && a[i+run] == v {
    run++
  }
  for j < nb && b[j] < v {
    j++
  }
  eq := 0
  for j+eq < nb && b[j+eq] == v {
    eq++
  }
  adv += run * (2*j + eq)
  i += run
}
d1, d2 := a, b
n1, n2 := na, nb
if adv <= 2*na*nb-adv {
  d1, d2 = b, a
  n1, n2 = nb, na
}
```

listing 535: go, the advantage from one sorted merge, duplicated advantage faces counted by run length instead of rescanned, D1 kept only on strictly more than half the pairs (source: icpc/samples-go/ch12/pc.go, lines 32-60)

```
    long[] vals = new long[2 * (na + nb) + 2];
    int nv = 0;
    if (mn ≥ 2)
      vals[nv++] = mn - 1;
    for (int i = 0; i < na; i++)
      if (i == 0 || da[i] ≠ da[i - 1]) {
        vals[nv++] = da[i];
        vals[nv++] = da[i] + 1;
      }
    for (int i = 0; i < nb; i++)
      if (i == 0 || db[i] ≠ db[i - 1]) {
        vals[nv++] = db[i];
        vals[nv++] = db[i] + 1;
      }
    vals[nv++] = mx + 1;
    Arrays.sort(vals, 0, nv);
    Hull.Pt[] pt = new Hull.Pt[nv];
    int np = 0, i1 = 0, i2 = 0;
    for (int i = 0; i < nv; i++) {
      long v = vals[i];
      if (i > 0 && v == vals[i - 1])
        continue;
      while (i1 < n1 && d1[i1] < v)
        i1++;
      while (i2 < n2 && d2[i2] < v)
        i2++;
      long e1 = 0, e2 = 0;
      while (i1 + e1 < n1 && d1[i1 + (int) e1] == v)
        e1++;
      while (i2 + e2 < n2 && d2[i2 + (int) e2] == v)
        e2++;
      pt[np++] = new Hull.Pt(2 * (n1 - i1 - e1) + e1, 2 * (n2 - i2 - e2) + e2);
    }
    pt = Arrays.copyOf(pt, np);
    Arrays.sort(pt, (a, b) -> a.x() ≠ b.x() ? Long.compare(a.x(), b.x())
        : Long.compare(a.y(), b.y()));
    Hull.Pt[] hull = Hull.upper(pt);
    double ymax = Hull.maxYat(hull, n1);
    double xmin = Hull.minXat(hull, n2);
    double ans1 = 1.0 - ymax / (2.0 * n2);
    double ans2 = 1.0 - xmin / (2.0 * n1);
    return String.format(Locale.ROOT, "%.9f %.9f\n", ans1, ans2);
```

listing 536: java, candidate values with the below-minimum guard, doubled points through the two-pointer merge, then the two hull queries and scores (source: icpc/samples-java/src/Ch13/PC.java, lines 96-137)

```
        // candidate face values: each distinct union value contributes
        // its tie point (strictly-greater plus half the ties) and, when
        // an integer fits before the next distinct value (or past the
        // maximum), its strict point (the next integer). doubled coords.
        var vals = new long[n1 + n2];
        int vc = 0;
        for (int i = 0; i < n1; i++)
            vals[vc++] = d1[i];
        for (int i = 0; i < n2; i++)
            vals[vc++] = d2[i];
        Array.Sort(vals, 0, vc);
        var pts = new List<(long, long)>();
        for (int i = 0; i < vc; i++)
        {
            long v = vals[i];
            if (i > 0 && v == vals[i - 1])
                continue;
            long gt1 = n1 - UpperBound(d1, v);
            long eq1 = UpperBound(d1, v) - LowerBound(d1, v);
            long gt2 = n2 - UpperBound(d2, v);
            long eq2 = UpperBound(d2, v) - LowerBound(d2, v);
            pts.Add((2 * gt1 + eq1, 2 * gt2 + eq2));
            int j = i + 1;
            while (j < vc && vals[j] == v)
                j++;
            long next = j < vc ? vals[j] : long.MaxValue;
            if (next - v ≥ 2)
                pts.Add((2 * gt1, 2 * gt2));
        }
        // any legal face below the union minimum gives one more point
        if (vals[0] > 1)
            pts.Add((2 * n1, 2 * n2));
        var ptArr = pts.ToArray();

        var hull = ConvexHull(ptArr);
        // part 1: max y at x ≤ n1; part 2: min x at y ≥ n2. crossing
        // values are exact rationals; doubles hold the products exactly
        double bestY = double.MinValue;
        double bestX = double.MaxValue;
        for (int i = 0; i < hull.Count; i++)
        {
            var (ax, ay) = hull[i];
            var (bx, by) = hull[(i + 1) % hull.Count];
            if (ax ≤ n1)
                bestY = Math.Max(bestY, ay);
            if (ay ≥ n2)
                bestX = Math.Min(bestX, ax);
```

listing 537: c#, tie and strict points per distinct value, gap detection, then the hull edge walk with bounds (source: icpc/samples/src/Ch12/PC.cs, lines 49-95)

```
// D1 is whichever die has the advantage, by doubled pairwise points
let adv = 0;
{
  let j = 0;
  for (let i = 0; i < da.length; i++) {
    while (j < db.length && db[j] < da[i]) j++;
    let eq = 0;
    for (let jj = j; jj < db.length && db[jj] === da[i]; jj++) eq++;
    adv += 2 * j + eq;
  }
}
const d1 = adv ≥ 2 * da.length * db.length - adv ? da : db;
const d2 = adv ≥ 2 * da.length * db.length - adv ? db : da;
const n1 = d1.length, n2 = d2.length;
// candidate face values: each input face, one past it, and one
// below the minimum when positive
const vals = [];
const minf = Math.min(d1[0], d2[0]);
if (minf > 1) vals.push(minf - 1);
for (const f of d1) vals.push(f, f + 1);
for (const f of d2) vals.push(f, f + 1);
vals.sort((a, b) => a - b);
const pts = [];
let i1 = 0, i2 = 0;
for (let i = 0; i < vals.length; i++) {
  const v = vals[i];
  if (i > 0 && v === vals[i - 1]) continue;
  while (i1 < n1 && d1[i1] < v) i1++;
  while (i2 < n2 && d2[i2] < v) i2++;
  let e1 = 0, e2 = 0;
  while (i1 + e1 < n1 && d1[i1 + e1] === v) e1++;
  while (i2 + e2 < n2 && d2[i2 + e2] === v) e2++;
  pts.push({ x: 2 * (n1 - i1 - e1) + e1, y: 2 * (n2 - i2 - e2) + e2 });
}
```

listing 538: javascript, the advantage by merge counting, then the candidate values and two-pointer points (source: icpc/samples-js/src/ch12-pc-threekindsofdice.mjs, lines 36-69)

```
def solve(text):
    data = [int(v) for v in text.split()]
    n1 = data[0]
    a = data[1 : 1 + n1]
    n2 = data[1 + n1]
    b = data[2 + n1 :]
    if _wins(a, b):
        d1, d2, m1, m2 = a, b, n1, n2
    else:
        d1, d2, m1, m2 = b, a, n2, n1
    s1 = sorted(d1)
    s2 = sorted(d2)
    vals = sorted(set(s1) | set(s2))
    pts = []
    if vals[0] > 1:
        pts.append((2 * m1, 2 * m2))
    for i, u in enumerate(vals):
        h1 = bisect_right(s1, u)
        h2 = bisect_right(s2, u)
        g1 = m1 - h1
        g2 = m2 - h2
        p1 = 2 * g1 + h1 - bisect_left(s1, u)
        p2 = 2 * g2 + h2 - bisect_left(s2, u)
        pts.append((p1, p2))
        if i + 1 < len(vals) and vals[i + 1] > u + 1:
            pts.append((2 * g1, 2 * g2))
    pts.append((0, 0))  # a D3 face may ride past the input face cap
    hull = upper_hull(sorted(pts))
    y = max_y_under(hull, m1)
```

listing 539: python, union values with gap points and a past-the-cap (0,0), bisect for wins and ties (source: icpc/samples-py/src/Ch12/pc.py, lines 26-54)

```lua
local da = {}
for i = 1, na do da[i] = next_int() end
local nb = next_int()
local db = {}
for i = 1, nb do db[i] = next_int() end
table.sort(da)
table.sort(db)
-- doubled points of A against B: 2 per win, 1 per tie;
-- run-length compressed so all-equal inputs stay linear
local pts = 0
local i, j = 1, 1
while i ≤ na do
  local v = da[i]
  local run = 0
  while i ≤ na and da[i] == v do
    run = run + 1
    i = i + 1
  end
  while j ≤ nb and db[j] < v do j = j + 1 end
  local eq = 0
  while j + eq ≤ nb and db[j + eq] == v do eq = eq + 1 end
  pts = pts + run * (2 * (j - 1) + eq)
end
local d1, d2, n1, n2
if pts > na * nb then
  d1, n1, d2, n2 = da, na, db, nb
else
  d1, n1, d2, n2 = db, nb, da, na
end
local mn, mx = da[1], da[1]
for i2 = 1, na do
  if da[i2] < mn then mn = da[i2] end
  if da[i2] > mx then mx = da[i2] end
end
for i2 = 1, nb do
  if db[i2] < mn then mn = db[i2] end
  if db[i2] > mx then mx = db[i2] end
end
local vals = {}
local nv = 0
```

listing 540: lua, run-length compressed advantage so all-equal dice stay linear, then the candidate build (source: icpc/samples-lua/ch12_pc.lua, lines 38-77)

Two judge corrections live in this section. First, hull-edge mixes beat pure breakpoints: the original derivation priced only dice whose faces all sit on breakpoints and printed 2/3 and 1/4 on the fixture, but its own witness D3 = [2] earns 2/6 against D2, contradicting the 2/3, and the corrected answers ride the hull edge from (0,1) to (4,4), the mix [1,3] ties D1 at exactly 1/2 while scoring 7/12 against D2, and the six-face mix [1,1,1,3,3] holds the D2 tie at avgS1 = 4/3 for a part-2 score of 1/3, so the fixture answers 0.583333333 0.333333333, confirmed 78 of 78 on the judge. Second, the unclamped candidates: judge secret-55 feeds `1 999999999` against `1 1000000000` and expects `0.5 0.5`, reachable only through the faces 999999998 and 1000000001, one below the minimum and one past the maximum. The fixture is D1 = (2,2) against D2 = (1,1,3), and the swapped input must return the same pair. Counts: c 2 float CHECKs, the C# suite 2 facts, go 3 tolerance rows including official sample 1 at 0.291666667 0.750000000, javascript 2 blocks, python 5 checks spanning both input orders, the all-ones pair, and the unclamped 999999999 against 1000000000 pair, lua 2 tolerance checks, java 7 ok checks, the hull queries plus the corrected fixture in both input orders. Go's first port scored the advantage with a naive double loop plus linear rescans, judge-green at 78

of 78 but 14.3 seconds on secret-41-gen, and the sorted-merge run-length pass in the listing above is the rewrite, green again with secret-41-gen at 0.031 seconds and the whole 78-case judge run at 1.06 seconds, the defect being complexity against the five sibling languages rather than wrong answers. Coordinates double to keep every hull vertex integral, and floats appear only in the final interpolation and print.

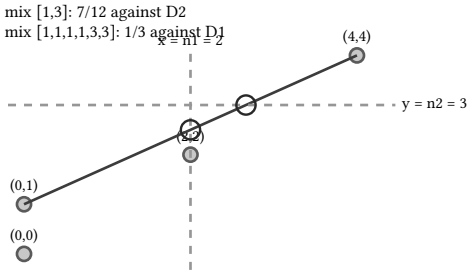


figure 160: the fixture's doubled points, the hull edge from (0,1) to (4,4), the two query lines and their winning points

14.6. d, carl's vacation

The 2022 T task returns. Carl the ant, veteran of the 2004 and 2009 finals, is on vacation in Egypt and wants to walk from the apex of one right square pyramid to the apex of another, touching only the two pyramid surfaces and the plane they stand on, and the task prints the shortest such walk. Each pyramid is given by one directed base edge, the body lying to the left of the direction of travel, plus its height, the apex sitting straight above the base center, and the two square bases may share edges or corners but never area.

The input is two lines of five integers, x1 y1 x2 y2 and then h for the first pyramid, the same shape again for the second, by the problems pdf: the coordinates run to 1e5 in absolute value, the two edge endpoints are distinct, the height satisfies 1 <= h <= 1e5, and the base intersection is promised to have area 0. The output is the minimum travel distance with an absolute or relative error of at most 1e-6 under the pdf's 1 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `D-carlsvacation/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints with its path figure: the first pyramid rises 4 over the edge from (0,0) to (10,0), and the optimum crosses into the second pyramid exactly at a base corner, the case that breaks the solutions' own sketch.

input:

```
0 0 10 0 4
9 18 34 26 42
```

expected output:

```
60.866649532
```

Recognition: coordinates stop at 1e5 and the limit at 1 second, and the budget is candidate count, not per-candidate work: 16 edge-pair unfold-straights, 8 one-corner chains, 16 two-corner chains and the four-corner chain, each leg a convex one-dimensional minimization priced in a few hundred probes, tens of thousands of predicate calls in all, thousands of times inside budget.

The statement's cue is shortest such walk over surfaces that unfold: on unfoldable surfaces an optimum is a straight line in some unfolding, so the only question is which finitely many hinge and corner configurations attain it, the cue that turns a continuous path search into critical-point enumeration. Problem D is the 2023 face of the geometric critical-point enumeration family. Geometric critical-point enumeration is the shared shape of chapter 13 problem T, chapter 10 problem A, chapter 10 problem D, chapter 11 problem E, and chapter 11 problem G.

Numerically sweeping entry and exit points on a fine grid is the priced-out alternative: 1e-6 resolution over 1e5-sized coordinates asks 1e11 samples for a single face pair, sixteen of them, against the 1 second limit, and any smooth sweep undershoots the corner entries where this year's optimum lives.

The solutions pdf page 4 sketch, 16 face pairs with both apexes unfolded onto the ground, is incomplete: implemented literally it produces no valid candidate on the official sample, whose optimum enters the second pyramid exactly at a base corner. Unfolding is reflection arithmetic: for the face hinged at a base edge e of length s, the apex lands on the ground at the foot of the base center on the edge line, pushed outward along the edge's outward normal by the slant sqrt(h^2 + (s/2)^2), and a surface leg from the apex to a point of e is then a straight segment from that copy. A shortest path bends only where its medium changes, at the exit edge of pyramid 1, the entry edge of pyramid 2, or at a base corner, so the shipped solvers price nine candidate categories per face pair, exit by unfold, mirror, or corner crossed with the same three entries: the unfold-straights, the apex rotated about each of the four base edges onto either side of the plane, accepted when the segment crosses the two hinge edges in order, the equality t1 = t2 covering pyramids that share a base edge, with its open middle missing both square interiors, then the mirror-to-corner hybrids each way, and the corner bends, paths bending at one or more of the 8 base corners with each end leg minimized over its exit edge. The completion was derived for this book and validated against all 45 judge pairs to 1e-6, constant work per face pair, problem at icpc2023.pdf.

The worked run: trace the model on sample 1. Pyramid 1 is the square on (0, 0) to (10, 0) with height 4, apex above (5, 5), and pyramid 2 the square on the directed edge (9, 18) to (34, 26) with height 42, corners (9, 18), (34, 26), (26, 51), (1, 43), apex above the center (17.5, 34.5). Every one of the 16 unfold-straights fails validity, each straight leaving pyramid 1's near faces crosses into pyramid 2's interior, so the optimum bends at a corner, and the winner is the unfold-exit, corner-entry hybrid through (9, 18). Leg 1 unfolds the apex across the back edge line to (5, 10 - sqrt(41)), the slant sqrt(5^2 + 4^2) = sqrt(41), and the straight from there to the corner is sqrt(4^2 + (8 + sqrt(41))^2) = sqrt(223.450) = 14.948244, crossing the back edge at (6.778, 10) inside the segment. Leg 2 unfolds apex 2 about the given edge, landing at (34.911, −19.909) past the edge center (21.5, 22) by the slant sqrt(42^2 + 13.124^2) = 44.003, and the corner-to-unfolded-apex straight is sqrt(25.911^2 + 37.909^2) = sqrt(2108.500) = 45.918406. The plane middle is the corner itself, and 14.948244 + 45.918406 = 60.866650, and the trace ends at the printed answer `60.866649532`.

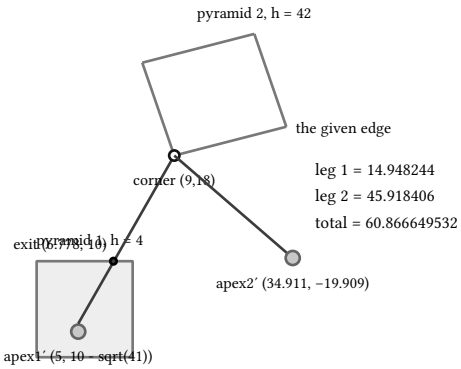


figure 161: sample 1′s winning chain: the back-face unfold to apex1′, the bend at the corner (9,18), the given-edge unfold to apex2′

```
// corner chains: the geodesic on the ground bends only at obstacle
// corners, so enumerate corner sequences of length 1..4 over the 8
// corners with valid consecutive segments; the two legs minimize
// independently per end corner
G12 corner[8];
double leg1[8], leg2[8];
for (int ci = 0; ci < 8; ci++) {
  corner[ci] = ci < 4 ? A.v[ci] : B.v[ci - 4];
  leg1[ci] = -1;
  leg2[ci] = -1;
  for (int ei = 0; ei < 4; ei++) {
    double f = best_leg(&A, ei, corner[ci]);
    if (f ≥ 0 && (leg1[ci] < 0 || f < leg1[ci]))
      leg1[ci] = f;
    f = best_leg(&B, ei, corner[ci]);
    if (f ≥ 0 && (leg2[ci] < 0 || f < leg2[ci]))
      leg2[ci] = f;
  }
}
int seq[4];
for (int d0 = 0; d0 < 8; d0++) {
  if (leg1[d0] < 0)
    continue;
  seq[0] = d0;
  for (int d1 = 0; d1 < 8; d1++) {
    if (leg2[d1] < 0 || !ground_ok(corner[d0], corner[d1]))
      continue;
    double mid = g12len(g12sub(corner[d0], corner[d1]));
    double cand = leg1[d0] + mid + leg2[d1];
    if (best < 0 || cand < best)
      best = cand;
    // extend chains to 4 corners
    for (int d2 = 0; d2 < 8; d2++) {
      if (!ground_ok(corner[d1], corner[d2]))
        continue;
      double mid2 = mid + g12len(g12sub(corner[d1], corner[d2]));
      for (int d3 = 0; d3 < 8; d3++) {
        if (leg2[d3] < 0 || !ground_ok(corner[d2], corner[d3]))
          continue;
        double cand3 = leg1[d0] + mid2 +
                       g12len(g12sub(corner[d2], corner[d3])) + leg2[d3];
        if (best < 0 || cand3 < best)
          best = cand3;
      }
    }
  }
}
```

listing 541: c, corner chains: per-corner best legs over the four edges, then sequences of one, two, and four corners with valid ground (source: icpc/samples-c/src/Ch12/pD.c, lines 199-245)

```go
func bestLeg(p *pyr, k gVec, sq1, sq2 []gVec) (float64, bool) {
  best := math.Inf(1)
  eval := func(e int, u float64) float64 {
    x, y := p.corners[e], p.corners[(e+1)%4]
    q := geoAddG(x, geoMulG(geoSubG(y, x), u))
    if !planeClear(q, k, sq1, sq2) {
      return math.Inf(1)
    }
    return math.Sqrt(p.h*p.h+geoDist2G(p.center, q)) + geoDistG(q, k)
  }
  const M = 512
  for e := 0; e < 4; e++ {
    run := -1 // start index of the current valid run
    for i := 0; i <= M+1; i++ {
      valid := i <= M && eval(e, float64(i)/M) < math.Inf(1)
      if valid && run < 0 {
        run = i
      }
      if !valid && run ≥ 0 {
        lo, hi := float64(run)/M, float64(i-1)/M
        if v := eval(e, lo); v < best {
          best = v
        }
        if v := eval(e, hi); v < best {
          best = v
        }
        for it := 0; it < 100; it++ {
          m1 := lo + (hi-lo)/3
          m2 := hi - (hi-lo)/3
          v1, v2 := eval(e, m1), eval(e, m2)
          if v1 <= v2 {
            hi = m2
            if v1 < best {
              best = v1
            }
          } else {
            lo = m1
            if v2 < best {
              best = v2
            }
          }
        }
        run = -1
      }
    }
  }
  return best, best < math.Inf(1)
}
```

listing 542: go, bestLeg as one eval closure, convex in the edge parameter, infeasible samples priced at infinity (source: icpc/samples-go/ch12/pd.go, lines 201-248)

```
// minimize slant(s, p(t)) + |p(t) - k| over t in [0,1] along edge
// ei, with the ground segment p(t)→k required valid; the objective
// is convex so grid + ternary suffices
static double bestLeg(Sq s, int ei, Geom.P k) {
  Geom.P a = s.v[ei], b = s.v[(ei + 1) % 4];
  int grid = 1024;
  double bestv = -1, bestt = -1;
  for (int i = 0; i ≤ grid; i++) {
    double t = (double) i / grid;
    Geom.P p = Geom.add(a, Geom.mul(Geom.sub(b, a), t));
    if (!groundOk(p, k))
      continue;
    double f = slant(s, p) + Geom.len(Geom.sub(p, k));
    if (bestv < 0 || f < bestv) {
      bestv = f;
      bestt = t;
    }
  }
  if (bestv < 0)
    return -1;
  double lo = Math.max(0, bestt - 1.0 / grid), hi = Math.min(1, bestt + 1.0 / grid);
  for (int it = 0; it < 100; it++) {
    double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
    Geom.P p1 = Geom.add(a, Geom.mul(Geom.sub(b, a), m1));
    Geom.P p2 = Geom.add(a, Geom.mul(Geom.sub(b, a), m2));
    double f1 = groundOk(p1, k) ? slant(s, p1) + Geom.len(Geom.sub(p1, k)) : 1e30;
    double f2 = groundOk(p2, k) ? slant(s, p2) + Geom.len(Geom.sub(p2, k)) : 1e30;
    if (f1 ≤ f2)
      hi = m2;
    else
      lo = m1;
  }
  Geom.P p = Geom.add(a, Geom.mul(Geom.sub(b, a), (lo + hi) / 2));
  double f = groundOk(p, k) ? slant(s, p) + Geom.len(Geom.sub(p, k)) : bestv;
  return Math.min(f, bestv);
}
```

listing 543: java, bestLeg: the guarded 1024-grid scan plus ternary refinement, infeasible probes skipped and priced at 1e30 (source: icpc/samples-java/src/Ch13/PD.java, lines 113-148)

```
    private static double MinLeg(Sq home, Sq oth, double[] p, double[] q,
        double cxr, double cyr)
    {
        double hx = home.Cx, hy = home.Cy, h = home.H;
        double ex = q[0] - p[0], ey = q[1] - p[1];
        double best = double.MaxValue;
        const int G = 8192;
        // unconstrained optimum of the convex leg on the full edge:
        // valid whenever it happens to be feasible, independent of run
        // detection
        {
            double lo = 0, hi = 1;
            for (int it = 0; it < 100; it++)
            {
                double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
                if (F(m1) ≤ F(m2))
                    hi = m2;
                else
                    lo = m1;
            }
            double t = Math.Min(Math.Max((lo + hi) / 2, 0), 1);
            if (Feas(t))
                best = Math.Min(best, F(t));
        }
        bool prevFeas = false;
        int runStart = -1;
        double runBest = double.MaxValue;
        double F(double t)
        {
            double x = p[0] + t * ex, y = p[1] + t * ey;
            double dcx = x - hx, dcy = y - hy;
            double dx2 = x - cxr, dy2 = y - cyr;
            return Math.Sqrt(h * h + dcx * dcx + dcy * dcy)
                   + Math.Sqrt(dx2 * dx2 + dy2 * dy2);
        }
        bool Feas(double t)
        {
            double x = p[0] + t * ex, y = p[1] + t * ey;
            return Clear(home, oth, x, y, cxr, cyr);
        }
```

listing 544: c#, MinLeg: the unconstrained convex optimum first, then run detection with bisected feasible edges (source: icpc/samples/src/Ch12/PD.cs, lines 102-141)

```
function unfoldLen(p1, e1, a1, p2, e2, a2) {
  const d = gSub(a2, a1);
  const eps = 1e-9;
  const s = gAdd(a1, gMul(d, eps));
  const f = gSub(a2, gMul(d, eps));
  const h1 = segHit(s, f, p1.corners[e1], p1.corners[(e1 + 1) % 4]);
  if (!h1) return null;
  const h2 = segHit(s, f, p2.corners[e2], p2.corners[(e2 + 1) % 4]);
  if (!h2 || h2.t < h1.t - 1e-12) return null;
  // coincident crossings are legal: pyramids sharing a hinge line are
  // crossed at one shared point, leaving an empty middle
  const tiny = h2.t - h1.t < 1e-12;
  if (tiny && gDist2(h1.pt, h2.pt) > 1e-9 * gDist2(s, f)) return null;
  const mid = gSub(h2.pt, h1.pt);
  const m1 = gAdd(h1.pt, gMul(mid, 1e-9));
  const m2 = gSub(h2.pt, gMul(mid, 1e-9));
  if (!planeClear(m1, m2, p1.corners, p2.corners)) return null;
  return gDist(a1, a2);
}

// minimize sqrt(h^2 + |center-P|^2) + |P-k| over base-edge points P
// with the plane walk P-k clear; convex per edge, so every maximal
// valid subinterval gets a grid sweep plus ternary refinement
```

listing 545: javascript, unfoldLen: both hinge crossings solved by cross products, order and middle clearance checked (source: icpc/samples-js/src/ch12-pd-carlsvacation.mjs, lines 75-97)

```python
def best_leg(s, ei, k):
    # minimize slant(s, p(t)) + |p(t) - k| over t in [0, 1] along edge
    # ei, with the ground segment p(t) -> k required valid; the
    # objective is convex so grid plus ternary suffices
    a = s.v[ei]
    edge = sub(s.v[(ei + 1) % 4], a)
    bestv = bestt = None
    for i in range(1025):
        t = i / 1024
        p = add(a, mul(edge, t))
        if not ground_ok(p, k):
            continue
        f = slant(s, p) + length(sub(p, k))
        if bestv is None or f < bestv:
            bestv = f
            bestt = t
    if bestv is None:
        return None
    lo = max(bestt - 1 / 1024, 0.0)
    hi = min(bestt + 1 / 1024, 1.0)
    for _ in range(100):
        m1 = lo + (hi - lo) / 3
        m2 = hi - (hi - lo) / 3
        p1 = add(a, mul(edge, m1))
        p2 = add(a, mul(edge, m2))
        f1 = slant(s, p1) + length(sub(p1, k)) if ground_ok(p1, k) else 1e30
        f2 = slant(s, p2) + length(sub(p2, k)) if ground_ok(p2, k) else 1e30
        if f1 <= f2:
            hi = m2
        else:
            lo = m1
    p = add(a, mul(edge, (lo + hi) / 2))
    f = slant(s, p) + length(sub(p, k)) if ground_ok(p, k) else bestv
    return min(f, bestv)
```

listing 546: python, the same guarded grid-plus-ternary leg, a port of the judge-green c engine (source: icpc/samples-py/src/Ch12/pd.py, lines 76-110)

```
local function best_leg(s, ei, kx, ky)
  local j = ei % 4 + 1
  local ax, ay = s.vx[ei], s.vy[ei]
  local ex, ey = s.vx[j] - ax, s.vy[j] - ay
  local NG = 1024
  local bestv, bestt = -1, -1
  for i = 0, NG do
    local t = i / NG
    local px, py = ax + ex * t, ay + ey * t
    if ground_ok(px, py, kx, ky) then
      local dx, dy = px - kx, py - ky
      local f = slant(s, px, py) + sqrt(dx * dx + dy * dy)
      if bestv < 0 or f < bestv then
        bestv, bestt = f, t
      end
    end
  end
  if bestv < 0 then return -1 end
  local lo = bestt - 1 / NG
  local hi = bestt + 1 / NG
  if lo < 0 then lo = 0 end
  if hi > 1 then hi = 1 end
  for _ = 1, 100 do
    local m1 = lo + (hi - lo) / 3
    local m2 = hi - (hi - lo) / 3
    local p1x, p1y = ax + ex * m1, ay + ey * m1
    local p2x, p2y = ax + ex * m2, ay + ey * m2
    local f1 = ground_ok(p1x, p1y, kx, ky) and
               slant(s, p1x, p1y) + sqrt((p1x - kx) ^ 2 + (p1y - ky) ^ 2) or
               1e30
    local f2 = ground_ok(p2x, p2y, kx, ky) and
               slant(s, p2x, p2y) + sqrt((p2x - kx) ^ 2 + (p2y - ky) ^ 2) or
               1e30
    if f1 ≤ f2 then
      hi = m2
    else
      lo = m1
    end
  end
  local t = (lo + hi) / 2
  local px, py = ax + ex * t, ay + ey * t
  local f = ground_ok(px, py, kx, ky) and
            slant(s, px, py) + sqrt((px - kx) ^ 2 + (py - ky) ^ 2) or bestv
  if f < bestv then return f end
  return bestv
end
```

listing 547: lua, best_leg over flat scalars, no table allocation inside the sweep (source: icpc/samples-lua/ch12_pd.lua, lines 89-135)

The fixture is two squares with a two-unit gap, bases [0,2] and [4,6] with heights 2: the inner unfold lands each apex inside the other square and the plain segment crosses out of order, so both apexes mirror across their facing edges, the mirror segment threads the hinge midpoints (2,1) and (4,1), and its length is `2 + 2*sqrt(5) = 6.472135955`. The corner-kissing extra, squares [0,6] and [6,12] on a diagonal touching at (6,6), admits no smooth mirror crossing and walks the corner chain at `2*sqrt(22) = 9.380831520`. Every suite asserts both, and go's tolerance table adds the official sample 60.866649532 whose corner entry forced the completion: c 2 float CHECKs, the C# suite 2 facts, go 3 tolerance rows, java 9 ok checks over the

Geom kit and both spans, javascript 2 blocks, python 2 checks, lua 2 tolerance checks, all compared at 1e-6. Input coordinates are integers and everything past parsing is double, so the languages diverge nowhere worth noting.

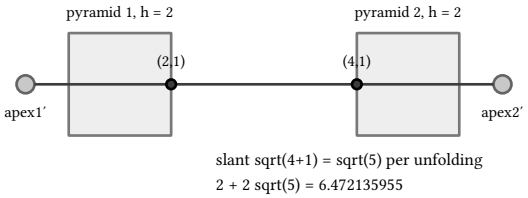


figure 162: the fixture: both squares, both apexes mirrored across the facing edges, the straight candidate threading both hinge midpoints

14.7. e, a recurring problem

You wrote down every positive linear recurrence relation on index cards, one per card, and now they need ordering. A plrr picks an order k, then k positive coefficients c1 through ck, then k positive starting values a1 through ak, and generates the rest of its sequence forever through a(i+k) = c1 a(i) + c2 a(i+1) + ... + ck a(i+k-1), Fibonacci being the famous case. The cards are indexed from 1 by lexicographic order of the generated tail, everything after the k starting values, with ties broken by lexicographic order of the coefficient list: the pdf's own example, k = 1 with coefficient 2 starting from 2, precedes k = 2 with coefficients 2 and 1 starting from 1 and 2, even though both generate the same numbers. Given a card's index n, the task prints the card.

The input is a single line with the integer n, 1 <= n <= 1e9 by the problems pdf. The output is four lines: the order k, the k coefficients, the k starting values, and the first ten generated values, all space separated, under the pdf's 20 second limit, the widest in the set and headroom the judges grant for a counting dp they call complicated.

Official sample 1, reprinted byte for byte from the judge data pair `E-arecurringproblem/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: card 3 is Fibonacci, order 2, coefficients 1 1, starting values 1 1.

input:

```
3
```

expected output:

```
2
1 1
1 1
2 3 5 8 13 21 34 55 89 144
```

Official sample 2, same source: card 1235 is an order-4 card whose tenth generated value already reaches 22377, the growth that forces the saturated comparisons below.

input:

```
1235
```

expected output:

```
4
1 1 3 1
3 2 1 1
9 15 44 99 255 611 1519 3706 9129 22377
```

Recognition: n <= 1e9 under the widest limit in the set, 20 seconds, and the judges themselves cite no tighter shape than a complicated dp: the memo counts prefixes of generated tails, the walk consumes the index one value at a time, and the measured state counts peak near 26 million, so the budget is memory and constant factors, not asymptotics.

The statement's cue is cards indexed from 1 by lexicographic order of the generated tail, ties broken by the coefficient list: an index into a lexicographic enumeration is unranking, and unranking a family far too large to list is counting without enumeration. Problem E is the 2023 face of the counting and expectation without enumeration family. Counting and expectation without enumeration is the shared core of chapter 11 problem D, chapter 11 problem I, chapter 13 problem Q, and chapter 14 problem K.

Listing the cards in order until index n is the priced-out alternative: sample 2's card 1235 already generates 22377 at its tenth value, the tails saturate int64 within forty terms, and n runs to 1e9 cards, so generation costs 1e9 cards times ten saturated terms against the one dp the judges grant 20 seconds.

The solutions pdf pages 4 and 5 count with a dp: f(x, a) counts the plrrs whose tail starts with vector x given the trailing context a, via f = 1 on the all-zero vector, 0 on any negative entry, and otherwise the sum over each a0 and c of f on the vector x minus c times the window (a0, a1, and on), with (a0, and on) as the new context, memoized and pruned with the forward condition that dropping the last entry must leave a positive count, since dead ends never revive. The walk consumes the index n one generated value at a time, or binary-searches the next entry with a <= y variant of f, then enumerates directly once fewer than about half a million cards share the prefix. Zero teams solved it in contest and the judges cite no tighter bound than a complicated dp, the 20-second limit is the headroom, problem at icpc2023.pdf.

The worked run: trace the model on sample 1. The index is n = 3. The first generated value is a sum of k positive terms, at least k, so only the k = 1 card c = 1 from a = 1 opens its tail at 1, card 1, the all-ones tail. Cards opening at 2 number three: the k = 1 cards c = 1 from a = 2 with tail 2 2 2 2 and c = 2 from a = 1 with tail 2 4 8 16, plus the k = 2 card c = 1 1 from a = 1 1, whose recurrence a(i+2) = a(i) + a(i+1) opens 2 3 5 8. Lexicographic tail order ranks 2 2 2 2 before 2 3 5 8 before 2 4 8 16, the table below, so card 2 is c = 1 from a = 2 and card 3 is the k = 2 card: a3 = 1 + 1 = 2, a4 = 1 + 2 = 3, a5 = 2 + 3 = 5, a6 = 3 + 5 = 8, on to a11 = 34 + 55 = 89 and a12 = 55 + 89 = 144, and the trace ends at the printed answer `2 3 5 8 13 21 34 55 89 144`.

card	k	coefficients	start	tail opens
1	1	1	1	1 1 1 1
2	1	1	2	2 2 2 2
3	2	1 1	1 1	2 3 5 8
4	1	2	1	2 4 8 16

```
  for (int i = 0; i < lx; i++) {
    if (x[i] < 0)
      neg = 1;
    if (x[i] ≠ 0)
      zero = 0;
    if (x[i] < mn)
      mn = x[i];
  }
  if (neg)
    return 0;
  if (zero)
    return 1;
  if (lx ≥ 2) { // forward prune: dropping the last entry must live
    int x2[MAXL], a2[MAXL];
    memcpy(x2, x, sizeof(int) * (size_t)(lx - 1));
    memcpy(a2, a, sizeof(int) * (size_t)(lx - 2));
    if (f_memo(x2, a2, lx - 1) == 0)
      return 0;
  }
  long long total = 0;
  // this step subtracts c*(a0, a1, ..); c must fit under every entry
  int cmax = x[0];
  for (int i = 1; i < lx; i++) {
    int lim = x[i] / a[i - 1];
    if (lim < cmax)
      cmax = lim;
  }
  for (int c = 1; c ≤ cmax; c++) {
    for (int a0 = 1; a0 ≤ x[0] / c; a0++) {
      int nx[MAXL], na[MAXL];
      nx[0] = x[0] - c * a0;
      if (nx[0] < 0)
        break; // larger a0 only drives nx[0] further down
      int ok = 1;
      for (int i = 1; i < lx && ok; i++) {
        nx[i] = x[i] - c * a[i - 1];
        if (nx[i] < 0)
          ok = 0;
      }
      if (!ok)
        break;
      na[0] = a0;
      for (int i = 1; i < lx - 1; i++)
        na[i] = a[i - 1];
      total += f_memo(nx, na, lx);
      if (total > BIG)
        return BIG;
    }
  }
  return total;
}
```

listing 548: c, f_rec: negative and zero base cases, the forward prune, the capped a0 and c loops, saturation at 4e18 (source: icpc/samples-c/src/Ch12/pE.c, lines 70-121)

```
      var walk func(x, a []int64, depth int)
      walk = func(x, a []int64, depth int) {
        dead := true
        for _, q := range x {
          if q > 0 {
            dead = false
          }
        }
        if dead {
          k := depth
          c := make([]int64, k)
          av := make([]int64, k)
          for i := 0; i < k; i++ {
            c[i] = cs[k-1-i]
            av[i] = as[k-1-i]
          }
          cards = append(cards, card{c, av, genN(c, av, 48)})
          return
        }
        var nx, na [maxK]int64
        for a0 := int64(1); a0 <= x[0]; a0++ {
          if len(a) > 0 {
            na[0] = a0
            copy(na[1:], a[:len(a)-1])
          }
          for cc := int64(1); cc*a0 <= x[0]; cc++ {
            ok := true
            for i := range x {
              w := a0
              if i > 0 {
                w = a[i-1]
              }
              v := x[i] - cc*w
              if v < 0 {
                ok = false
                break
              }
              nx[i] = v
            }
            if !ok {
              break
            }
            if f(nx[:len(x)], na[:len(a)]) == 0 {
              continue
            }
            cs[depth], as[depth] = cc, a0
            walk(nx[:len(x)], na[:len(a)], depth+1)
          }
        }
      }
      x := append([]int64{}, prefix...)
```

listing 549: go, the enumeration walk mirrors f's branching, (c, a0) pairs kept in depth-indexed scratch (source: icpc/samples-go/ch12/pe.go, lines 111-161)

```java
static long fRec(int[] x, int[] a, int lx) {
  boolean neg = false, zero = true;
  for (int i = 0; i < lx; i++) {
    if (x[i] < 0)
      neg = true;
    if (x[i] ≠ 0)
      zero = false;
  }
  if (neg)
    return 0;
  if (zero)
    return 1;
  if (lx ≥ 2) { // forward prune: dropping the last entry must live
    int[] x2 = Arrays.copyOf(x, lx - 1), a2 = Arrays.copyOf(a, lx - 2);
    if (fMemo(x2, a2, lx - 1) == 0)
      return 0;
  }
  long total = 0;
  // this step subtracts c*(a0, a1, ..); c must fit under every entry
  int cmax = x[0];
  for (int i = 1; i < lx; i++) {
    int lim = x[i] / a[i - 1];
    if (lim < cmax)
      cmax = lim;
  }
  int[] nx = new int[MAXL], na = new int[MAXL];
  for (int c = 1; c ≤ cmax; c++) {
    for (int a0 = 1; a0 ≤ x[0] / c; a0++) {
      nx[0] = x[0] - c * a0;
      if (nx[0] < 0)
        break; // larger a0 only drives nx[0] further down
      boolean good = true;
      for (int i = 1; i < lx && good; i++) {
        nx[i] = x[i] - c * a[i - 1];
        if (nx[i] < 0)
          good = false;
      }
      if (!good)
        break;
      na[0] = a0;
      for (int i = 1; i < lx - 1; i++)
        na[i] = a[i - 1];
      total += fMemo(nx, na, lx);
      if (total > BIG)
        return BIG;
    }
  }
  return total;
}
```

listing 550: java, fRec: negative and zero base cases, the forward prune, the capped a0 and c loops, saturation at 4e18 (source: icpc/samples-java/src/Ch13/PE.java, lines 68-116)

```
    private static long F(List<int> x, List<int> a)
    {
        int m = x.Count;
        bool zero = true;
        for (int i = 0; i < m; i++)
        {
            if (x[i] < 0)
                return 0;
            if (x[i] != 0)
                zero = false;
        }
        if (zero)
            return 1;
        var key = new System.Text.StringBuilder();
        foreach (int v in x)
            key.Append(v).Append(',');
        key.Append('|');
        foreach (int v in a)
            key.Append(v).Append(',');
        if (_memo.TryGetValue(key.ToString(), out long cached))
            return cached;
        // pruning: dropping the last window entry must stay feasible
        if (m >= 2 && F(x.GetRange(0, m - 1), a.GetRange(0, m - 2)) == 0)
        {
            _memo[key.ToString()] = 0;
            return 0;
        }
        long total = 0;
        int x1 = x[0];
        for (int c = 1; c <= x1; c++)
        {
            for (int a0 = 1; a0 * c <= x1; a0++)
            {
                var nx = new List<int>(m);
                bool ok = true;
                for (int i = 0; i < m && ok; i++)
                {
                    int sub = c * (i == 0 ? a0 : a[i - 1]);
                    int val = x[i] - sub;
                    if (val < 0)
                        ok = false;
                    else
                        nx.Add(val);
                }
                if (!ok)
                    break;
                var na = new List<int>(m - 1) { a0 };
                for (int i = 0; i < m - 2; i++)
                    na.Add(a[i]);
                total += F(nx, na);
```

listing 551: c#, the same recurrence over List windows with a string-keyed memo and a 4e9 cap (source: icpc/samples/src/Ch12/PE.cs, lines 59-108)

```
function packKey(buf, x, a, lx) {
  buf[0] = lx;
  for (let j = 0; j < 2 * lx - 1; j++) {
    const v = j < lx ? x[j] : a[j - lx];
    if (v > 65535) throw new Error("key overflow");
    buf[(j >> 1) + 1] = j & 1 ? (buf[(j >> 1) + 1] & 0xffff0000) | v : v << 16;
  }
  return lx + 1;
}

function fMemo(x, a, lx) {
  const kbuf = kbufs[dep] || (kbufs[dep] = new Int32Array(KEYW));
  const kw = packKey(kbuf, x, a, lx);
  let h = 2166136261;
  for (let w = 0; w < kw; w++) {
    h ^= kbuf[w];
    h = Math.imul(h, 16777619);
  }
  h &= HSZ - 1;
  for (let e = hslot[h] - 1; e ≥ 0; e = anext[e] - 1) {
    const o = e * KEYW;
    let hit = true;
    for (let w = 0; w < kw; w++)
      if (akey[o + w] =/= kbuf[w]) {
        hit = false;
        break;
      }
    if (hit) return aval[e];
  }
  const v = fRaw(x, a, lx);
  if (nent === acap) memoGrow();
  const e = nent++;
  const o = e * KEYW;
  for (let w = 0; w < kw; w++) akey[o + w] = kbuf[w];
  aval[e] = v;
  anext[e] = hslot[h];
  hslot[h] = e + 1;
  return v;
}
```

listing 552: javascript, the typed-array chained hash: 13-word keys packing two 16-bit fields per int, per-depth buffers (source: icpc/samples-js/src/ch12-pe-arecurringproblem.mjs, lines 60-99)

```python
def _f(x, a, mg=memo.get):
    # x and a carry exact lengths lx and lx-1; the step subtracts
    # c*(a0, a1, ..) so for fixed c the child tail entries are fixed
    # and only the head varies with a0. entries never go negative:
    # cmax caps c by every x[i]//a[i-1] and a0 by x[0]//c.
    lx = len(x)
    if not any(x):
        return 1
    if lx == 1:
        return f1(x[0])
    key = x[:-1] + a[:-1]
    v = mg(key)
    if v is None:
        v = _f(x[:-1], a[:-1])
        memo[key] = v
    if v == 0:
        return 0  # forward prune: dead prefixes never revive
    total = 0
    x0 = x[0]
    pairs = list(zip(x[1:], a))
    ashort = a[: lx - 2]
    cmax = x0
    for xi, ai in pairs:
        lim = xi // ai
        if lim < cmax:
            cmax = lim
    for c in range(1, cmax + 1):
        tail = tuple(xi - c * ai for xi, ai in pairs)
        for a0 in range(1, x0 // c + 1):
            key = (x0 - c * a0, *tail, a0, *ashort)
            v = mg(key)
            if v is None:
                v = _f((x0 - c * a0, *tail), (a0, *ashort))
                memo[key] = v
            total += v
            if total > BIG:
                return BIG
    return total
```

listing 553: python, f with hoisted memo.get, fixed child tails per c, saturation at 4e18 (source: icpc/samples-py/src/Ch12/pe.py, lines 57-96)

```
    -- prune: dead shorter prefixes stay dead
    if lx > 1 then
      local d = depth + 1
      local px, pa = nxs[d], nas[d]
      for i = 1, lx - 1 do px[i] = x[i] end
      for i = lx, #px do px[i] = nil end
      for i = 1, la - 1 do pa[i] = a[i] end
      for i = la, #pa do pa[i] = nil end
      if f(px, pa, d) == 0 then
        memo[key] = 0
        return 0
      end
    end
    local total = 0
    local nx, na = nxs[depth + 1], nas[depth + 1]
    for a0 = 1, x[1] do
      if la > 0 then
        na[1] = a0
        for i = 1, la - 1 do
          na[i + 1] = a[i]
        end
        for i = la + 1, #na do
          na[i] = nil
        end
      else
        for i = 1, #na do
          na[i] = nil
        end
      end
      for c = 1, x[1] // a0 do
        local ok = true
        for i = 1, lx do
          local w = i == 1 and a0 or a[i - 1]
          local v = x[i] - c * w
          if v < 0 then
            ok = false
            break
          end
          nx[i] = v
        end
        if not ok then
          break
        end
        for i = lx + 1, #nx do
          nx[i] = nil
        end
        total = total + f(nx, na, depth + 1)
      end
    end
    memo[key] = total
    return total
  end
```

listing 554: lua, depth-indexed scratch vectors shared across the prune and the child loop, tails cleared past the live length (source: icpc/samples-lua/ch12_pe.lua, lines 53-104)

Two measured stories separate the languages here. The dp itself is heavy: the judge set peaks around 26 million memo states, past the v8 Map's 2^24 entry ceiling, so javascript packs keys into 16-bit fields of a typed-array

chained hash and runs tail evolution on BigInt windows saturating at 4e18, and python raises its enumeration cap to 1.2 million, which stops the walk one prefix entry sooner and roughly halves both the dp state count and the dict memory. Java rides the same states through a HashMap keyed by an int-array value type, and its cardNext is the year's one BigInteger site: one multiply per coefficient against BigInteger.valueOf, saturating through compareTo at the same 4e18 cap where c sums in __int128. The walk itself hides the constant-tail trap: the k=1, c=(1) family repeats its value instead of strictly increasing, it is counted by the dp at deeper windows, and a walk that resumes the next candidate at prefix-last + 1 skips it. The constant cards sit at n = 99029, 243835, 600347, and 1478167 for z = 14 through 17, and the z = 17 block lies past the half-million descent threshold, so a threshold-skipping walk silently misses it. The C# stream hit this live, fixed it in 0c01f23 by resuming non-constant tails at prefix-last + 1 and constant tails at prefix-last itself, and re-judged 52 of 52 with a scatter-diff against the C engine showing zero mismatches through n = 999999999. Lua's first port timed out at 276 seconds worst on secrets 37 and 38, 50 of 52, because the go engine's stack-local child vectors have no lua equivalent and per-child table allocation became the wall. Depth-indexed scratch vectors shared across the prune call and the child loop, with explicit tail clearing past the live length since slots are reused across branches of different lengths, dropped the worst case to 107 seconds, a re-judge of 52 of 52 with the thinnest margin 13 seconds under the cap. All seven suites pin the fixture family: card 1 is k=1, c=(1), a=(1) with tail all ones, card 2 is the same recurrence from a=(2), card 4 is c=(2) from a=(1) printing powers of two, card 5 is c=(1) from a=(3), so 2 answers `1 / 1 / 2 / 2 2 2 2 2 2 2 2 2 2`. Counts: c 4 CHECKs, the C# suite 7 facts including both official samples and the constant-tail card pinned at 99029, go 4 rows plus a dedicated count-curve test, java 4 ok checks, javascript 4 blocks, python 4 checks, lua 4 named checks. The dp counts in int64 with saturation, and the generated values grow past int64 quickly, so every engine compares tails on saturated values and prints only the ten small terms of the final card.

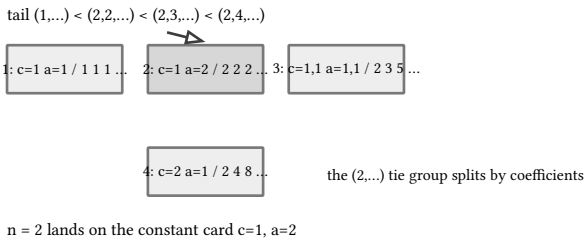


figure 163: cards ordered by generated tail, the tie at (2,...) split by coefficient lists, the pointer on card 2

14.8. f, tilting tiles

A puzzle found in the attic is an h by w grid in which some cells carry colored tiles, colors lowercase letters, and tiles of one color cannot be told apart. Tilting the grid in one of the four cardinal directions slides every tile that way until it is blocked by the boundary or by another tile, and given a starting and an ending arrangement the question is whether some sequence of tilts turns the one into the other.

The input gives the height and width h and w, each from 1 to 500 by the problems pdf, then h lines of length w describing the starting arrangement from top to bottom, a dot for an empty cell and a lowercase letter for a tile, then one empty line, then h more lines in the same format for the ending arrangement. The output is yes or no under the pdf's 3 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `F-tiltingtiles/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints and solves in its figure with four tilts: left, towards you, right, away from you.

input:

```
4 4
.r..
rgyb
.b..
.yr.
```

```
yrbr
..yr
...g
...b
```

expected output:

```
yes
```

Recognition: h and w at most 500 give at most 250000 tiles under the 3 second limit. After one horizontal and one vertical tilt the outline is frozen, each corner cycle permutes a fixed cell set, and the work is one permutation build plus a rotation match per cycle, O(hw) with alphabet-size factors, thousands of times inside budget.

The statement's cue is whether some sequence of tilts turns the one arrangement into the other, against tiles that slide flush every move: after compaction the only freedom left is cyclic rotation within each corner's cell cycle, the cue that the reachability question is string rotation. Problem F is the 2023 face of the string borders, rotations, and automata family. String borders, rotations, and automata connect chapter 10 problem K and chapter 12 problem G; a secondary thread, pairwise congruence consistency, covers the cycle-length compatibility the rotation match must satisfy.

Simulating tilts breadth-first from the start is the priced-out alternative: the reachable anchored states number one per power of the corner permutation, and the modulus those powers live on is the lcm this section elsewhere measured overflowing at 1.9e18, against 250000 cells.

After one horizontal and one vertical tilt the tiles sit flush in a corner and the outline is frozen, each corner's return cycle, right-down-left-up and its rotations, induces a fixed permutation of the tile cells, and from then on the only moves are the four corner re-anchors. The reachable end states are the handful reachable in zero or one tilts plus the corner-anchored states along the permutation's cycles, so per cycle the solutions pdf page 6 reduces the question to whether some power of the permutation maps the start string to the end string, which is the end string searched inside the doubled start cycle, every match at offset a yielding the congruence n = a (mod m) for m the cycle length, problem at icpc2023.pdf. Book 10′s chapter 33, string algorithms ii, builds the rotation match on the doubled cycle, z-function and all, that this coset search runs on. Two contracts the sketch leaves open, and every porting language hits both. First, the congruences, per cycle the valid powers form residues best modulo g with g the gcd of the cycle length and every match difference, are never merged incrementally by lcm, the accumulator was observed overflowing 64-bit at 1.9e18 and then by a factor of 185. Keep one canonical residue per distinct modulus and check every pair with the gcd of the two moduli dividing the difference of the two residues, which is a chinese-remainder consistency check in plain text, no constructed solution. Book 10′s chapter 16, number theory and modular arithmetic, develops the chinese-remainder residue merge over shared prime groups that this pairwise check stands in for. Moduli divide their cycle lengths, so distinct moduli sum to at most the tile count, 250000 tiles admit under 707 distinct moduli, and same-modulus congruences from cycles of different lengths compare residues modulo the modulus, never raw rotation offsets. Second, from an entry corner every word reduces to sigma^r followed by one monotone path, an away-then-back pair is the identity once both axes are compact and any full loop conjugates back to the entry corner's sigma, and the monotone paths are exactly seven per corner: the empty word, both single away moves, both two-move paths to the opposite corner, and both three-move paths to an adjacent corner, which the earlier one-and-two-move word set misses. With clockwise at index 0 everywhere the words are uniform, and each step resolves against the current corner's away table, not the entry's.

The worked run: trace the model on sample 1. The four printed tilts, left, towards you, right, away from you, walk the boards directly, each one a compaction with order preserved. Left slides every row flush west: r… / rgyb / b… / yr.. Then towards you slides every column flush south: r… / r… / bg.. / yryb, all eight tiles in the bottom-left staircase, column heights 4, 2, 1, 1, the frozen outline. Right slides the rows flush east: …r / …r / ..bg / yryb, and away from you slides the columns flush north into the top-right corner with the same heights 4, 2, 1, 1 reading

from the right: yrbr / ..yr / ...g / ...b, exactly the printed target arrangement. The word is two compacting tilts plus one return pair, the shape the model's corner cycles generate, so the target sits on the cycle and the model answers yes, and the trace ends at the printed answer yes.

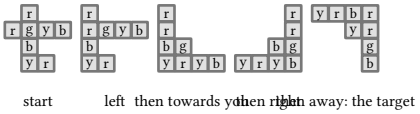


figure 164: sample 1's four tilts in order: the start, left, plus towards you, plus right, and the final away landing the printed target

```c
static int crt_add(long long best, long long g) {
  if (g == 1)
    return 1;
  for (int q = 0; q < crtN; q++)
    if (crtmod[q] == g)
      return crtres[q] % g == best % g;
  if (crtN ≥ 1024)
    return 0; // unreachable given the modulus-sum bound
  crtmod[crtN] = g;
  crtres[crtN] = best;
  crtN++;
  return 1;
}

// rotation match of one cycle: entry letters s (len L) vs target
// letters t; the valid k form the coset best + gZ where g is the gcd
// of L and every match difference. returns 0 on mismatch
static int cyc_match(const char *s, const char *t, int L) {
  static char dbl[2 * MAXD * MAXD];
  memcpy(dbl, s, (size_t)L);
  memcpy(dbl + L, s, (size_t)L);
  static int f[MAXD * MAXD];
  ch12_fail(t, L, f);
  long long best = -1, g = L;
  int k = 0;
  for (int q = 0; q < 2 * L; q++) {
    while (k && dbl[q] ≠ t[k])
      k = f[k - 1];
    if (dbl[q] == t[k])
      k++;
    if (k == L) {
      long long kk = ((-(long long)(q - L + 1)) % L + L) % L;
      if (best < 0)
        best = kk;
      else {
        long long d = (kk - best) % L;
        if (d < 0)
          d += L;
        while (d) { // g = gcd(g, d)
          long long tmp = g % d;
          g = d;
          d = tmp;
        }
      }
      k = f[k - 1];
    }
  }
  if (best < 0)
    return 0;
  return crt_add(best, g);
}
```

listing 555: c, crt_add keeping one residue per distinct modulus, cyc_match finding the coset best + gZ on the doubled cycle (source: icpc/samples-c/src/Ch12/pF.c, lines 107-157)

```
cycMatch := func(s, t []byte, crt *[1024][2]int64, crtN *int) bool {
  L := len(s)
  dbl := make([]byte, 2*L)
  copy(dbl, s)
  copy(dbl[L:], s)
  f := make([]int, L)
  k := 0
  for q := 1; q < L; q++ {
    for k > 0 && t[q] != t[k] {
      k = f[k-1]
    }
    if t[q] == t[k] {
      k++
    }
    f[q] = k
  }
  var best int64 = -1
  g := int64(L)
  k = 0
  for q := 0; q < 2*L; q++ {
    for k > 0 && dbl[q] != t[k] {
      k = f[k-1]
    }
    if dbl[q] == t[k] {
      k++
    }
    if k == L {
      kk := (int64(-(q-L+1))%int64(L) + int64(L)) % int64(L)
      if best < 0 {
        best = kk
      } else {
        d := (kk - best) % int64(L)
        if d < 0 {
          d += int64(L)
        }
        for d != 0 {
          g, d = d, g%d
        }
      }
      k = f[k-1]
    }
  }
  if best < 0 {
    return false
  }
  if g == 1 {
    return true
  }
  for q := 0; q < *crtN; q++ {
    if crt[q][0] == g {
      return crt[q][1]%g == best%g
```

listing 556: go, the same cycMatch closure with the crt table carried as a pointer (source: icpc/samples-go/ch12/pf.go, lines 110-160)

```java
static long[] crtMod = new long[1024], crtRes = new long[1024];
static int crtN;

static boolean crtAdd(long best, long g) {
  if (g == 1)
    return true;
  for (int q = 0; q < crtN; q++)
    if (crtMod[q] == g)
      return crtRes[q] % g == best % g;
  if (crtN ≥ 1024)
    return false; // unreachable given the modulus-sum bound
  crtMod[crtN] = g;
  crtRes[crtN] = best;
  crtN++;
  return true;
}

// rotation match of one cycle: entry letters s (len L) vs target
// letters t; the valid k form the coset best + gZ where g is the
// gcd of L and every match difference
static boolean cycMatch(char[] s, char[] t, int L) {
  char[] dbl = new char[2 * L];
  System.arraycopy(s, 0, dbl, 0, L);
  System.arraycopy(s, 0, dbl, L, L);
  int[] f = Kmp.fail(t);
  long best = -1, g = L;
  int k = 0;
  for (int q = 0; q < 2 * L; q++) {
    while (k > 0 && dbl[q] ≠ t[k])
      k = f[k - 1];
    if (dbl[q] == t[k])
      k++;
    if (k == L) {
      long kk = ((-(long) (q - L + 1)) % L + L) % L;
      if (best < 0)
        best = kk;
      else {
        long d = (kk - best) % L;
        if (d < 0)
          d += L;
        while (d ≠ 0) { // g = gcd(g, d)
          long tmp = g % d;
          g = d;
          d = tmp;
        }
      }
      k = f[k - 1];
    }
  }
  if (best < 0)
    return false;
  return crtAdd(best, g);
}
```

listing 557: java, crtAdd keeping one residue per distinct modulus, cycMatch off the Kmp twin's failure table over the doubled cycle (source: icpc/samples-java/src/Ch13/PF.java, lines 141-193)

```
    private static bool CycMatch(char[] s, char[] t, int L)
    {
        Array.Copy(s, Dbl, L);
        Array.Copy(s, 0, Dbl, L, L);
        Fail[0] = 0;
        for (int i = 1, k = 0; i < L; i++)
        {
            while (k > 0 && t[i] != t[k])
                k = Fail[k - 1];
            if (t[i] == t[k])
                k++;
            Fail[i] = k;
        }
        long best = -1, g = L;
        int kk = 0;
        for (int q = 0; q < 2 * L; q++)
        {
            while (kk > 0 && Dbl[q] != t[kk])
                kk = Fail[kk - 1];
            if (Dbl[q] == t[kk])
                kk++;
            if (kk == L)
            {
                long rot = (-(long)(q - L + 1) % L + L) % L;
                if (best < 0)
                    best = rot;
                else
                {
                    long d = (rot - best) % L;
                    if (d < 0)
                        d += L;
                    while (d != 0)
                    {
                        long tmp = g % d;
                        g = d;
                        d = tmp;
                    }
                }
                kk = Fail[kk - 1];
            }
        }
        if (best < 0)
            return false;
        return CrtAdd(best, g);
    }

    // entry anchor grid ag, return-cycle permutation sigma, re-tilt
    // map fm: does B equal fm(sigma^k(ag)) for some k?
```

listing 558: c#, CycMatch through the failure function, matches folded into the gcd, then crt_add (source: icpc/samples/src/Ch12/PF.cs, lines 145-192)

```
function crtAdd(best, g) {
  if (g === 1) return true;
  for (let q = 0; q < crtN; q++)
    if (crtmod[q] === g) return crtres[q] % g === best % g;
  crtmod[crtN] = g;
  crtres[crtN] = best;
  crtN++;
  return true;
}

// rotation match of one cycle: entry letters s (len L) vs target
// letters t; the valid k form the coset best + gZ where g is the gcd
// of L and every match difference
function cycMatch(s, t, L, f, dbl) {
  for (let q = 0; q < L; q++) {
    dbl[q] = s[q];
    dbl[q + L] = s[q];
  }
  kmpFail(t, L, f);
  let best = -1, g = L, k = 0;
  for (let q = 0; q < 2 * L; q++) {
    while (k && dbl[q] !== t[k]) k = f[k - 1];
    if (dbl[q] === t[k]) k++;
    if (k === L) {
      const kk = ((-(q - L + 1)) % L + L) % L;
      if (best < 0) best = kk;
      else {
        let d = (kk - best) % L;
        if (d < 0) d += L;
        while (d) {
          const tmp = g % d;
          g = d;
          d = tmp;
        }
      }
      k = f[k - 1];
    }
  }
  if (best < 0) return false;
  return crtAdd(best, g);
}

// entry anchor grid ag, return-cycle permutation sigma (cell -> cell),
// re-tilt map fm: does B equal fm(sigma^k(ag)) for some k?
```

listing 559: javascript, crtAdd plus cycMatch over the KMP failure function from the year-local kit (source: icpc/samples-js/src/ch12-pf-tiltingtiles.mjs, lines 83-126)

```
    for s, cyc in cycs:
        L = len(s)
        t = []
        for q in cyc:
            src = fm[q]
            if src < 0:
                return False
            ch = fb[src]
            if ch == ".":
                return False
            t.append(ch)
        f = fail(t)
        dbl = s + s
        best = -1
        g = L
        k = 0
        for q in range(2 * L):
            while k and dbl[q] != t[k]:
                k = f[k - 1]
            if dbl[q] == t[k]:
                k += 1
            if k == L:
                kk = (-(q - L + 1)) % L
                if best < 0:
                    best = kk
                else:
                    d = (kk - best) % L
                    while d:
                        g, d = d, g % d
                k = f[k - 1]
        if best < 0:
            return False
        if g == 1:
            continue
        for q, m in enumerate(mods):
            if m == g:
                if res[q] % g != best % g:
                    return False
                break
        else:
            mods.append(g)
            res.append(best)
    for q in range(len(mods)):
        for u in range(q + 1, len(mods)):
            a, b = mods[q], mods[u]
            while b:
                a, b = b, a % b
            if (res[q] - res[u]) % a:
                return False
    return True
```

listing 560: python, orbit_hit: per cycle the coset and gcd, residues collected per modulus, pairwise check at the end (source: icpc/samples-py/src/Ch12/pf.py, lines 110-160)

```
local function crt_add(best, g)
  if g == 1 then return true end
  for q = 1, crtN do
    if crtmod[q] == g then
      return crtres[q] % g == best % g
    end
  end
  crtN = crtN + 1
  crtmod[crtN] = g
  crtres[crtN] = best
  return true
end

local function gcd(a, b)
  while b ~= 0 do
    a, b = b, a % b
  end
  return a
end

-- rotation match of one cycle: entry letters s (len L) vs target
-- letters t; the valid k form the coset best + gZ, g the gcd of L and
-- every match difference. false on mismatch
local function cyc_match(s, t, L)
  local dbl = {}
  for q = 1, L do dbl[q] = s[q] end
  for q = 1, L do dbl[L + q] = s[q] end
  local f = kmp.fail(t, L)
  local best = -1
  local g = L
  kmp.find_all(dbl, 2 * L, t, L, f, function(at)
    local kk = ((-(at - 1)) % L + L) % L
    if best < 0 then
      best = kk
    else
      local d = (kk - best) % L
      if d < 0 then d = d + L end
      if d > 0 then
        g = gcd(g, d)
      end
    end
  end)
  if best < 0 then return false end
  return crt_add(best, g)
end

-- entry anchor grid ag, return-cycle permutation sigma (id -> id),
-- re-tilt map fm: does B equal fm(sigma^k(ag)) for some k?
```

listing 561: lua, crt_add and cyc_match, the failure table from the year-local kmp module (source: icpc/samples-lua/ch12_pf.lua, lines 100-147)

The C engine's first cut shipped a satisfiability flag that marked 6 of the 55 judge secrets wrong, all large possible boards, fixed in 9586ce6 by the seven-word enumeration and judged 55 of 55, with no false positives on the 49 previously passing cases and 14 thousand random boards cross-checked against an exhaustive tilt BFS. Porting hygiene: the C `run_moves` reused one sigma array for both the entry corner's cycle and the landing corner's, out-of-bounds reads that stayed judge-green because they were never load-bearing, and the oversight crashed the C#

port with an index error on the `1 3` mini, so the landing cycle now lives in its own array everywhere. The fixture is the 2x2 board with `ab` on top sliding down to `ab` on the bottom, answered yes. Every suite also pins the one-row `ab.` against `.ba`, no, order is preserved along a row so only rotations of the occupied block are reachable, and the trivial `1 1` board that needs zero tilts, yes: c 3 CHECKs, the C# suite 5 facts carrying the crash mini, go 3 rows, java 8 ok checks over the Kmp kit and the three boards, javascript 3 blocks, python 3 checks, lua 3 named checks. No integers beyond cell indices anywhere.

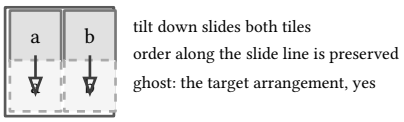


figure 165: the fixture mid-tilt: both tiles sliding down, the ghost target underneath

14.9. g, turning red

The 2022 P task returns. Mei's remodeled house gives every room an LED light that is red, green, or blue, and its pre-crossbar wiring scatters buttons, each connected to one or more lights, with every light answering to at most two buttons. Pressing a button steps each connected light one place around the red, green, blue cycle, buttons work any number of times, and the fewest total presses turning every light red is wanted, or impossible.

The input opens with the light count l, 1 <= l <= 2e5, and the button count b, 0 <= b <= 2l, by the problems pdf, the formal bound admitting the b = 0 corner that the phrase two positive integers papers over. A line of l characters R, G, or B gives the initial colors, and each of the b button lines opens with a count k, 1 <= k <= l, and lists k distinct lights, with each light appearing at most twice across all buttons. The output is the minimum press count or the word impossible under the pdf's 3 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `G-turningred/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: eight lights over six buttons, lights 5, 6, and 7 shared by two buttons and light 8 by two more, and the cheapest consistent press count per component sums to 8.

input:

```
8 6
GBRBRRRG
2 1 4
1 2
4 4 5 6 7
3 5 6 7
1 8
1 8
```

expected output:

```
8
```

Recognition: l <= 2e5 lights and b <= 2l = 4e5 buttons under the 3 second limit price linear propagation: one adjacency build plus a BFS per component, three root trials on the unpinned ones, under 2e6 cell visits in all, two orders of magnitude inside budget.

The statement's cue is arithmetic: every light answers to at most two buttons and each press advances a light one fixed step around the red, green, blue cycle, so each light is one equation over at most two unknowns mod 3, the cue that turns the house wiring into per-component propagation with one free value. Problem G is the 2023 face of the one free value per component family. One free value per component is the trick behind chapter 13 problem P and chapter 15 problem K.

Searching press vectors directly is priced out at 3^b = 3^400000 worlds, and integer shortest paths or flows cannot express the mod-3 sums at all; the three root trials per component are the entire search space.

With colors as residues mod 3, a light on two buttons is an edge demanding that the two press counts sum to minus its color, a light on one button pins its button, a light listed twice on the same button is a self-loop pinning 2x = -c, that is x = c, and a light on no button must already be red. Buttons sharing lights chain into connected components, a node per button and an edge per two-button light, and per component the solver fixes one root press count to each of 0, 1, 2, propagates, discards inconsistent roots, and keeps the cheapest, linear in lights plus buttons, the solutions pdf pages 6 and 7, problem at icpc2023.pdf. The formal bounds admit b = 0, which the prose's "two positive integers" hides, and the solvers answer 0 or impossible straight from the initial colors in that case.

The worked run: trace the model on sample 1. Colors as residues, R = 0, G = 1, B = 2, the printed string GBRBRRRG reads 1, 2, 0, 2, 0, 0, 0, 1. Lights 1 and 2 sit alone on buttons 1 and 2 and pin them at x = -c mod 3: x1 = 2, x2 = 1, while light 3 is red and wired to no button, already right at cost 0. Light 4, blue, joins buttons 1 and 3 with x1 + x3 = 1 mod 3, forcing x3 = 2, and lights 5, 6 and 7, all red, join buttons 3 and 4 with x3 + x4 = 0 mod 3, forcing x4 = 1, so the 1-3-4 component, pinned through light 1 rather than root-tried, costs 2 + 2 + 1 = 5. Light 8, green, joins buttons 5 and 6 with x5 + x6 = 2 mod 3, an unpinned pair: the three root trials read (0, 2), (1, 1), (2, 0) and all press total 2. The whole is 5 + 1 + 2 + 0 = 8, the table below, and the trace ends at the printed answer 8.

component	pins and demands	press total
buttons 1, 3, 4	light 1 G pins x1 = 2, light 4 B forces x3 = 2, lights 5-7 R force x4 = 1	2 + 2 + 1 = 5
button 2	light 2 B pins x2 = 1	1
buttons 5 and 6	light 8 G demands x5 + x6 = 2	2
light 3	red, wired to no button	0
total	three components, light 3 unwired	8

```c
static long long try_component(int root, int r, int *touched) {
  stamp++;
  int qh = 0, qt = 0;
  long long total = 0;
  xsol[root] = r;
  mark[root] = stamp;
  queue[qt++] = root;
  if (pin[root] ≥ 0 && pin[root] ≠ r)
    return -1;
  total += r;
  while (qh < qt) {
    int u = queue[qh++];
    for (int e = xadj[u]; e < xadj[u + 1]; e++) {
      int v = eadj_to[e], k = eadj_k[e];
      int want = ((k - xsol[u]) % 3 + 3) % 3;
      if (mark[v] == stamp) {
        if (xsol[v] ≠ want)
          return -1;
        continue;
      }
      if (pin[v] ≥ 0 && pin[v] ≠ want)
        return -1;
      xsol[v] = want;
      mark[v] = stamp;
      total += want;
      queue[qt++] = v;
    }
  }
  *touched = qt;
  return total;
}
```

listing 562: c, try_component: stamp-carried BFS over the CSR adjacency, press total or −1 on a violated edge or pin (source: icpc/samples-c/src/Ch12/pG.c, lines 37-67)

```
  var touched []int
  // propagate assigns values from root downward, returning whether
  // every constraint holds and the component's press total; the
  // caller resets the touched assignments afterwards.
  propagate := func(root, rv int) (bool, int64) {
    ok := true
    sum := int64(0)
    queue = append(queue, root)
    val[root] = rv
    touched = append(touched, root)
    for head := 0; head < len(queue) && ok; head++ {
      u := queue[head]
      sum += int64(val[u])
      for _, e := range adj[u] {
        if val[e.to] == -1 {
          nv := (e.rhs - val[u] + 3) % 3
          if pin[e.to] ≥ 0 && pin[e.to] ≠ nv {
            ok = false
            break
          }
          val[e.to] = nv
          queue = append(queue, e.to)
          touched = append(touched, e.to)
        } else if (val[u]+val[e.to])%3 ≠ e.rhs {
          ok = false
          break
        }
      }
    }
    queue = queue[:0]
    for _, u := range touched {
```

listing 563: go, the propagate closure resetting only the touched cells between root trials (source: icpc/samples-go/ch12/pg.go, lines 71-101)

```java
// propagate press counts from root value r; the run carries its own
// stamp so no visited cleanup is needed between root trials.
// returns the press total (queue[0..touched) lists the component),
// or -1 when an edge sum or a pin is violated
static long tryComponent(int root, int r, int[] touchedOut) {
  stamp++;
  int qh = 0, qt = 0;
  long total = 0;
  xsol[root] = r;
  mark[root] = stamp;
  queue[qt++] = root;
  if (pin[root] ≥ 0 && pin[root] ≠ r)
    return -1;
  total += r;
  while (qh < qt) {
    int u = queue[qh++];
    for (int e = xadj[u]; e < xadj[u + 1]; e++) {
      int v = eadjTo[e], k = eadjK[e];
      int want = ((k - xsol[u]) % 3 + 3) % 3;
      if (mark[v] == stamp) {
        if (xsol[v] ≠ want)
          return -1;
        continue;
      }
      if (pin[v] ≥ 0 && pin[v] ≠ want)
        return -1;
      xsol[v] = want;
      mark[v] = stamp;
      total += want;
      queue[qt++] = v;
    }
  }
  touchedOut[0] = qt;
  return total;
}
```

listing 564: java, tryComponent: stamp-carried BFS over the CSR adjacency, the press total or −1 on a violated edge or pin (source: icpc/samples-java/src/Ch13/PG.java, lines 52-86)

```
            }
            long best = long.MaxValue;
            for (int t = 0; t < 3; t++)
            {
                if (pin[r] ≥ 0 && pin[r] ≠ t)
                    continue;
                for (int qi = 0; qi < tail; qi++)
                    val[queue[qi]] = -1;
                val[r] = t;
                bool ok = true;
                long cost = t;
                for (int qi = 1; qi < tail && ok; qi++)
                {
                    int u = queue[qi];
                    val[u] = -1;
                    // value forced by any already-valued neighbor (BFS
                    // order guarantees the tree parent qualifies)
                    for (int e = start[u]; e < start[u + 1] && val[u] < 0; e++)
                        if (val[adjTo[e]] ≥ 0)
                            val[u] = (adjSum[e] - val[adjTo[e]] + 3) % 3;
                    if (val[u] < 0)
                        ok = false;
                    else
                        cost += val[u];
                }
                for (int qi = 0; qi < tail && ok; qi++)
                {
                    int u = queue[qi];
                    if (pin[u] ≥ 0 && pin[u] ≠ val[u])
                        ok = false;
                    for (int e = start[u]; e < start[u + 1] && ok; e++)
                        if ((val[u] + val[adjTo[e]]) % 3 ≠ adjSum[e])
                            ok = false;
                }
                if (ok && cost < best)
                    best = cost;
            }
            if (best == long.MaxValue)
```

listing 565: c#, per root t: values assigned in BFS order from any valued neighbor, then every edge and pin re-validated (source: icpc/samples/src/Ch12/PG.cs, lines 118-155)

```
function propagate(root, rv) {
  let ok = true, sum = 0, qh = 0, qt = 0, nt = 0;
  queue[qt++] = root;
  val[root] = rv;
  touched[nt++] = root;
  while (qh < qt && ok) {
    const u = queue[qh++];
    sum += val[u];
    for (let e = head[u]; e ≥ 0; e = link[e]) {
      const to = adjTo[e], rhs = adjRhs[e];
      if (val[to] === -1) {
        const nv = (rhs - val[u] + 3) % 3;
        if (pin[to] ≥ 0 && pin[to] =/= nv) {
          ok = false;
          break;
        }
        val[to] = nv;
        queue[qt++] = to;
        touched[nt++] = to;
      } else if ((val[u] + val[to]) % 3 =/= rhs) {
        ok = false;
        break;
      }
    }
  }
  for (let t = 0; t < nt; t++) val[touched[t]] = -1;
  return ok ? sum : -1;
}
```

listing 566: javascript, the same closure over chained edge arrays, touched cells reset afterwards (source: icpc/samples-js/src/ch12-pg-turningred.mjs, lines 69-96)

```
    for r in range(b):
        if seen[r]:
            continue
        comp = []
        q = deque([r])
        seen[r] = True
        while q:
            u = q.popleft()
            comp.append(u)
            for v, _ in adj[u]:
                if not seen[v]:
                    seen[v] = True
                    q.append(v)
        pin = next((u for u in comp if u in fixed), None)
        # anchor the propagation AT the pin (a component root that is
        # itself unpinned must not inherit the pin's value)
        roots = [(r, x0) for x0 in range(3)]
        if pin is not None:
            roots = [(pin, fixed[pin])]
        best = None
        for anchor, x0 in roots:
            x = {anchor: x0}
            ok = True
            dq = deque([anchor])
            while dq and ok:
                u = dq.popleft()
                for v, rhs in adj[u]:
                    xv = (rhs - x[u]) % 3
                    if v in x:
                        if x[v] != xv:
                            ok = False
                            break
                    else:
                        x[v] = xv
                        dq.append(v)
                if not ok:
                    break
            if not ok:
                continue
            if any(x[u] != fixed[u] for u in comp if u in fixed):
                continue
            s = sum(x.values())
            if best is None or s < best:
                best = s
        if best is None:
            return "impossible\n"
        total += best
```

listing 567: python, anchored at the pin when one exists, three roots only for unpinned components (source: icpc/samples-py/src/Ch12/pg.py, lines 47-93)

```
local stamp = 0
local queue = {}
local touched = 0
-- propagate press counts from root value r; each trial carries its own
-- stamp so no cleanup is needed. returns the press total and the
-- component size (queue[1..touched]), or -1 on any violation.
local function try_component(root, r)
  stamp = stamp + 1
  xsol[root] = r
  mark[root] = stamp
  queue[1] = root
  local total = r
  local qh, qt = 1, 1
  if pin[root] and pin[root] ~= r then return -1 end
  while qh ≤ qt do
    local u = queue[qh]
    qh = qh + 1
    for e = xadj[u], xadj[u + 1] - 1 do
      local v = at[e]
      local want = (ak[e] - xsol[u]) % 3
      if mark[v] == stamp then
        if xsol[v] ~= want then return -1 end
      else
        if pin[v] and pin[v] ~= want then return -1 end
        xsol[v] = want
        mark[v] = stamp
        total = total + want
        qt = qt + 1
        queue[qt] = v
      end
    end
  end
  touched = qt
  return total
end
local claimed = {}
```

listing 568: lua, try_component with per-trial stamps, one-based queue like c (source: icpc/samples-lua/ch12_pg.lua, lines 107-142)

The fixture is 4 lights over 2 buttons, colors RBBB: light 1 is red and alone on button 1, pinning x1 = 0, light 3 is blue and alone on button 2, pinning x2 = 1, and lights 2 and 4 sit on both buttons with x1 + x2 = 1 mod 3, consistent. One press of button 2 turns lights 2, 3, and 4 red, and the answer is 1. The crafted pair carries the contradiction, both lights green on both buttons demanding x1 + x2 equal 2 and 1 at once, impossible, and the single button with both lights green, 2. Counts: c 3 CHECKs, the C# suite 5 facts, go 3 rows, java 6 ok checks, javascript 3 blocks, python 6 checks, lua 3 named checks. Everything lives mod 3 and press totals stay under 4e5, so no integer notes.

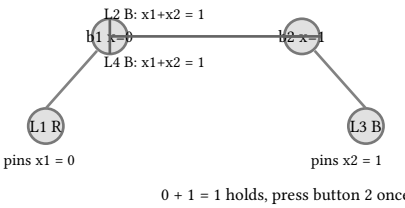


figure 166: two button nodes joined by two light edges, the single-button leaves pinning x1 and x2

14.10. h, jet lag

The world finals pack speeches, events, and the contest itself into the schedule, and at minute 0 you arrive so exhausted that you must sleep immediately. A sleep of k whole minutes, k any positive integer, is followed by k rested minutes in which falling asleep is impossible, then k functional minutes in which you may sleep again or stay awake, and past that comes an involuntary endless sleep: a sleep ending at t must have its successor start inside [t+k, t+2k], and the final sleep's functional stretch must carry you past the end of the last activity. Activities occupy fixed intervals you must attend entirely, no sleep may overlap one, and none may run past the start of the last, so the task prints any valid schedule or impossible.

The input is the activity count n, 1 <= n <= 2e5 by the problems pdf, then n lines of begin and end minutes bi and ei with bi < ei, ei <= b(i+1) so touching activities are allowed, 0 <= b1, and en <= 1e10. A valid schedule prints the period count p, the pdf guaranteeing one with at most 1e6 periods exists whenever any does, then p lines of start and end times si and ti obeying 0 = s1 < t1 < s2 < ... < tp <= bn: you may fall asleep the minute an activity ends, si = ej, and wake the minute one begins, ti = bj, and any valid schedule is accepted under the pdf's 2 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `H-jetlag/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: three activities, and sleeping the full first gap then the full last gap covers every window, exactly the greedy's canonical output. The judges' own sample 3 answer spends five sleeps where four suffice, both valid, the any-schedule scoring at work.

input:

```
3
30 45
60 90
120 180
```

expected output:

```
2
0 30
90 120
```

Recognition: n <= 2e5 activities with ends to 1e10 under the 2 second limit, and any valid schedule accepted: the greedy's passes visit each gap once, about 6e5 steps, five orders of magnitude inside budget, and the pdf itself guarantees a schedule with at most 1e6 periods exists whenever any does.

The statement's cue is the window law, a sleep ending at t must have its successor start inside [t+k, t+2k]: the constraint is a local feasibility window per gap, the cue that a provably safe local rule, take the full-gap sleep whenever it strictly extends the awake horizon, then repair, orders the whole schedule. Problem H is the 2023 face of the greedy with a safety proof family. Greedy with a safety proof is the family of chapter 10 problem H, chapter 11 problem K, and chapter 12 problem A.

A dp over the minute timeline is priced out by the coordinates alone, 1e10 minutes, and a dp over gap subsets costs (2e5)^2 = 4e10 pairs against the linear pass.

The sleep model is the statement's own, pinned against the official samples and carried on the solutions pdf page 7, problem at icpc2023.pdf. The greedy walks the gaps between activities, the first gap starting at 0, takes a full-gap sleep spanning each gap ahead whenever it strictly extends the awake horizon b + 2(b - e), then repairs consecutive sleeps: whenever the next start would come before twice the previous end minus its start, the previous end trims down to half the sum of the two starts, which only shrinks k, so the crash bounds survive. Linear time, and the book's suites pin this exact canonical construction byte for byte, which is why the judge harness needs a validator rather than a diff.

The worked run: trace the model on sample 1. The gaps are [0, 30] before the first activity, [45, 60] between the first two, and [90, 120] before the last. The greedy takes the full-gap sleep [0, 30]: k = 30, and the awake horizon becomes b + 2(b - e) = 30 + 2(30 - 0) = 90. The candidate [45, 60] would set the horizon to 60 + 2(60 - 45) = 90, not a strict extension, so it is skipped, the safety proof's own exchange: the skipped sleep buys no awake time. The greedy takes [90, 120]: the successor window of [0, 30] is [t + k, t + 2k] = [60, 90], the start 90 sits at its right end, the horizon becomes 120 + 2(120 - 90) = 180, and the final sleep's functional stretch reaches t + 2k = 180 >= 180, the last activity's end, so no repair pass fires. Two sleeps, printed 0 30 and 90 120, and the trace ends at the printed answer `90 120`.

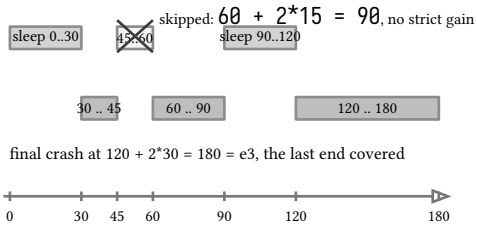


figure 167: sample 1′s timeline: the two taken sleeps, the skipped middle gap crossed out, the crash bound landing exactly at 180

```c
// greedy: full-gap sleeps that extend the awake horizon; a sleep
// [s,t] with k = t-s keeps you going until t + 2k (crash), and the
// next sleep must start inside [t+k, t+2k]
int cap = n + 8;
long long *S = malloc((size_t)cap * sizeof(long long));
long long *T = malloc((size_t)cap * sizeof(long long));
int ns = 0;
long long horizon = 0; // crash-free until here
int impossible = 0;
for (int i = 0; i ≤ n && !impossible; i++) {
  if (e[i] > horizon) {
    impossible = 1; // a gap or activity begins after the crash
    break;
  }
  if (i == n)
    break;
  long long k = b[i + 1] - e[i]; // the full gap ahead
  long long nh = b[i + 1] + 2 * k;
  if (k > 0 && nh > horizon) {
    S[ns] = e[i];
    T[ns] = b[i + 1];
    ns++;
    horizon = nh;
  }
}
// the last sleep's window must cover the last activity's end
if (!impossible && ns > 0) {
  long long k = T[ns - 1] - S[ns - 1];
  if (T[ns - 1] + 2 * k < e[n])
    impossible = 1;
}
// repair: s_{i+1} ≥ 2 t_i - s_i, trimming t_i keeps k positive
for (int i = 0; i + 1 < ns; i++) {
  if (2 * T[i] - S[i] > S[i + 1]) {
    long long t = (S[i] + S[i + 1]) / 2;
    if (t ≤ S[i])
      impossible = 1; // cannot rest long enough before the next
    else
      T[i] = t;
  }
}
```

listing 569: c, the greedy pass, the last-window cover check, then the rested-window repair with the trim floor (source: icpc/samples-c/src/Ch12/pH.c, lines 38-78)

```
  var crash int64
  prevE := int64(0)
  for i := 0; i < n; i++ {
    if prevE > crash {
      return "impossible\n"
    }
    k := b[i] - prevE
    if k ≥ 1 && b[i]+2*k > crash {
      s = append(s, prevE)
      t = append(t, b[i])
      crash = b[i] + 2*k
    }
    prevE = e[i]
  }
  if crash < e[n-1] {
    return "impossible\n"
  }
  // repair: the next sleep must start at or after t_i + k_i
  for i := 0; i+1 < len(s); i++ {
    if s[i+1] < 2*t[i]-s[i] {
      t[i] = (s[i] + s[i+1]) / 2
    }
  }
  var out strings.Builder
  out.WriteString(strconv.Itoa(len(s)) + "\n")
```

listing 570: go, the compact core: gap sleeps extending crash, then the one-line trim loop (source: icpc/samples-go/ch12/ph.go, lines 26-50)

```
    // greedy: full-gap sleeps that extend the awake horizon; a sleep
    // [s,t] with k = t-s keeps you going until t + 2k (crash), and
    // the next sleep must start inside [t+k, t+2k]
    long[] S = new long[n + 8], T = new long[n + 8];
    int ns = 0;
    long horizon = 0; // crash-free until here
    boolean impossible = false;
    for (int i = 0; i ≤ n && !impossible; i++) {
      if (e[i] > horizon) {
        impossible = true; // a gap or activity begins after the crash
        break;
      }
      if (i == n)
        break;
      long k = b[i + 1] - e[i]; // the full gap ahead
      long nh = b[i + 1] + 2 * k;
      if (k > 0 && nh > horizon) {
        S[ns] = e[i];
        T[ns] = b[i + 1];
        ns++;
        horizon = nh;
      }
    }
    // the last sleep's window must cover the last activity's end
    if (!impossible && ns > 0) {
      long k = T[ns - 1] - S[ns - 1];
      if (T[ns - 1] + 2 * k < e[n])
        impossible = true;
    }
    // repair: s_{i+1} ≥ 2 t_i - s_i, trimming t_i keeps k positive
    for (int i = 0; i + 1 < ns; i++) {
      if (2 * T[i] - S[i] > S[i + 1]) {
        long t = (S[i] + S[i + 1]) / 2;
        if (t ≤ S[i])
          impossible = true; // cannot rest long enough before the next
        else
          T[i] = t;
      }
    }
```

listing 571: java, the greedy pass, the last-window cover check, then the rested-window repair with the floor trim (source: icpc/samples-java/src/Ch13/PH.java, lines 48-86)

```
        var s = new long[n + 8];
        var t = new long[n + 8];
        int ns = 0;
        long horizon = 0; // crash-free until here
        bool impossible = false;
        for (int i = 0; i ≤ n && !impossible; i++)
        {
            if (e[i] > horizon)
            {
                impossible = true; // a gap or activity begins after crash
                break;
            }
            if (i == n)
                break;
            long k = b[i + 1] - e[i]; // the full gap ahead
            long nh = b[i + 1] + 2 * k;
            if (k > 0 && nh > horizon)
            {
                s[ns] = e[i];
                t[ns] = b[i + 1];
                ns++;
                horizon = nh;
            }
        }
        // the last sleep's window must cover the last activity's end
        if (!impossible && ns > 0)
        {
            long k = t[ns - 1] - s[ns - 1];
            if (t[ns - 1] + 2 * k < e[n])
                impossible = true;
        }
        // repair: s_{i+1} ≥ 2 t_i - s_i; trimming keeps k positive
        for (int i = 0; i + 1 < ns; i++)
        {
            if (2 * t[i] - s[i] > s[i + 1])
            {
                long tt = (s[i] + s[i + 1]) / 2;
                if (tt ≤ s[i])
                    impossible = true; // cannot rest long enough before next
                else
                    t[i] = tt;
            }
        }
        if (impossible || ns == 0)
```

listing 572: c#, the same three passes over long arrays, horizon, cover, repair (source: icpc/samples/src/Ch12/PH.cs, lines 30-73)

```
  let crash = 0;
  let prevE = 0;
  for (let i = 0; i < n; i++) {
    if (prevE > crash) return "impossible\n";
    const k = b[i] - prevE;
    if (k ≥ 1 && b[i] + 2 * k > crash) {
      s.push(prevE);
      t.push(b[i]);
      crash = b[i] + 2 * k;
    }
    prevE = e[i];
  }
  if (crash < e[n - 1]) return "impossible\n";
  // repair: the next sleep must start at or after t_i + k_i
  for (let i = 0; i + 1 < s.length; i++)
    if (s[i + 1] < 2 * t[i] - s[i]) t[i] = Math.floor((s[i] + s[i + 1]) / 2);
  let out = `${s.length}\n`;
  for (let i = 0; i < s.length; i++) out += `${s[i]} ${t[i]}\n`;
  return out;
}
```

listing 573: javascript, the same core over Float64Array times, floor on the trim (source: icpc/samples-js/src/ch12-ph-jetlag.mjs, lines 23-42)

```
    impossible = False
    for b, e in acts:
        if e_prev > cur:  # a gap or activity begins after the crash
            impossible = True
            break
        k = b - e_prev
        if k > 0 and b + 2 * k > cur:
            sleeps.append([e_prev, b])
            cur = b + 2 * k
        e_prev = e
    if not impossible and e_prev > cur:
        impossible = True
    if not impossible:
        for i in range(len(sleeps) - 1):
            s1, t1 = sleeps[i]
            s2 = sleeps[i + 1][0]
            if s2 < 2 * t1 - s1:
                t = (s1 + s2) // 2
                if t ≤ s1:
                    impossible = True  # cannot rest long enough
                    break
                sleeps[i][1] = t
    if impossible or not sleeps:
        return "impossible\n"
    out = [str(len(sleeps))]
    out.extend(f"{s} {t}" for s, t in sleeps)
    return "\n".join(out) + "\n"
```

listing 574: python, sleeps as [s, t] pairs, the trim rejecting a rest that cannot fit (source: icpc/samples-py/src/Ch12/ph.py, lines 21-47)

```lua
local bb, ee = {}, {}
bb[0], ee[0] = -1, 0 -- gap 0 spans [0, b1]
for i = 1, n do
  bb[i] = next_int()
  ee[i] = next_int()
end
local S, T = {}, {}
local ns = 0
local horizon = 0 -- crash-free until here
local impossible = false
for i = 0, n do
  if ee[i] > horizon then
    impossible = true -- a gap or activity begins after the crash
    break
  end
  if i == n then break end
  local k = bb[i + 1] - ee[i] -- the full gap ahead
  local nh = bb[i + 1] + 2 * k
  if k > 0 and nh > horizon then
    ns = ns + 1
    S[ns] = ee[i]
    T[ns] = bb[i + 1]
    horizon = nh
  end
end
-- the last sleep's window must cover the last activity's end
if not impossible and ns > 0 then
  local k = T[ns] - S[ns]
  if T[ns] + 2 * k < ee[n] then
    impossible = true
  end
end
-- repair: s_{i+1} ≥ 2 t_i - s_i, trimming t_i keeps k positive
for i = 1, ns - 1 do
  if 2 * T[i] - S[i] > S[i + 1] then
    local t = (S[i] + S[i + 1]) // 2
    if t ≤ S[i] then
      impossible = true -- cannot rest long enough before the next
    else
      T[i] = t
    end
```

listing 575: lua, gap 0 spans [0, b1] through a sentinel pair, repair trims with floor division (source: icpc/samples-lua/ch12_ph.lua, lines 32-72)

The fixture is one activity 40 to 50: the only gap is [0,40], the full-gap sleep keeps the window alive to 120, and the answer is `1 / 0 40`. The repair case, activities 12 to 20 and 30 to 40, takes both gaps and then trims the first sleep to `0 10`, and the activity starting at 0 makes any first sleep overlap it, impossible. Every suite asserts those three exact outputs: c 3 CHECKs, the C# suite 3 facts, go 3 rows, java 4 ok checks including the official sample, javascript 3 blocks, python 4 checks, lua 3 named checks. Times run to 1e10 and window arithmetic to 3e10, so int64 in c, C#, go, and lua, java longs, and javascript numbers, exact below 2^53 with room to spare. The validator itself is part of this set's measured record: the harness's first overlap check was O(p*n) in pure shell and would never finish on the big cases, about 4e10 iterations, and the fix is a monotone base pointer over the sleeps plus `[System.Array]::BinarySearch` for the first activity at or after each sleep end, O(p log n), roughly 9 seconds per big case and 18 minutes for all 126. Never hand-roll a midpoint binary search in powershell, its `[int]` cast rounds to even, so `hi = mid` can spin forever on an odd sum, and go's first patch hung for 90 minutes exactly that way before the BinarySearch rewrite.

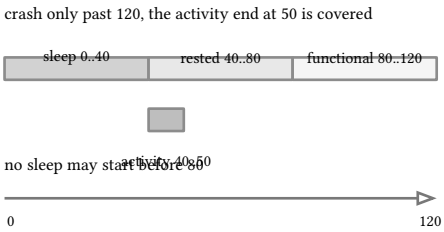


figure 168: the fixture: one full-gap sleep, its rested and functional stretches, the activity inside the window

14.11. i, waterworld

A faraway rotating planet is measured by a technology the statement dresses up as relativistic quantum-polarized spectroscopy, and the model under the dressing is clean: the planet is a sphere turning about a vertical axis, the telescope resolves only the strip through that axis, and the read proceeds in m steps of d = 360/m degrees, each step reporting the water percentage of n equal-height bands from pole to pole for the slice it just saw, the md = 360 steps together covering the whole surface. The task is the planet's total water percentage.

The input is n and m, each from 2 to 1000 by the problems pdf, then n lines of m integers a(i,j), each 0 to 100, row i holding band i's readings across the m rotation steps in order. The output is one percentage with an absolute error of at most 1e-6 under the pdf's 3 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `I-waterworld/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: 21 readings over 3 bands and 7 steps, their sum 1088, and the average is 51.809523810.

input:

```
3 7
63 61 55 54 77 87 89
73 60 38 5 16 56 91
75 43 11 3 16 20 95
```

expected output:

```
51.809523810
```

Recognition: n and m at most 1000 give at most 1e6 readings under the 3 second limit, one summation pass, so the cost is trivial and the only trap is the invariant: the tempting per-band spherical quadrature, 1000 bands of trig-weighted areas, about 1e6 extra transcendentals, computes a weighting the equal-area law pins as constant.

The statement's cue is the planet's total water percentage read in equal-height bands over a full rotation: spherical zone area is proportional to height, so every band covers the same surface fraction at every step, the cue that one proportionality collapses the whole task to a plain average. Problem I is the 2023 face of the invariant collapse family. Chapter 12 problem Y is the other invariant collapse member.

Spherical zone areas are proportional to height, the cylindrical equal-area projection, so every band covers the same surface fraction during a full rotation and hence during every step, each of the `n*m` readings weighs the same, and the planet's water percentage is the plain average of all of them, the one-line derivation on the solutions pdf pages 7 and 8, problem at icpc2023.pdf. This is the set's breather, one sum and one division.

The worked run: trace the model on sample 1. The three printed band rows sum to 486, 339 and 263, the table below, and 486 + 339 + 263 = 1088 readings-total over the grid's 3 times 7 = 21 readings. Equal-height bands carry equal zone area and every rotation step covers the same fraction of each zone, so each of the 21 readings weighs the same and the planet's water percentage is the plain average 1088/21 = 51.809523810, and the trace ends at the printed answer `51.809523810`.

band	row sum	weight
1	63 + 61 + 55 + 54 + 77 + 87 + 89 = 486	1/21
2	73 + 60 + 38 + 5 + 16 + 56 + 91 = 339	1/21
3	75 + 43 + 11 + 3 + 16 + 20 + 95 = 263	1/21
planet	1088	1088/21 = 51.809523810

```c
static char *solve(const char *in) {
  char *buf = dupstr(in);
  Ch11Scan sc;
  ch11_scan_init(&sc, buf);
  int n = (int)ch11_next_int64(&sc);
  int m = (int)ch11_next_int64(&sc);
  long long sum = 0;
  for (int i = 0; i < n * m; i++)
    sum += ch11_next_int64(&sc);
  free(buf);
  char *out = malloc(32);
  snprintf(out, 32, "%.9f\n", (double)sum / ((double)n * m));
  return out;
}
```

listing 576: c, the whole solver: sum the n times m readings, print the average at nine decimals (source: icpc/samples-c/src/Ch12/pI.c, lines 20-33)

```go
func SolveI(input string) string {
  sc := &scan12{data: []byte(input)}
  n := sc.nextInt()
  m := sc.nextInt()
  var sum int64
  for i := 0; i < n*m; i++ {
    sum += int64(sc.nextInt())
  }
  return fmt.Sprintf("%.9f\n", float64(sum)/float64(int64(n)*int64(m)))
}
```

listing 577: go, int64 sum, one Sprintf (source: icpc/samples-go/ch12/pi.go, lines 10-19)

```
  static long nextLong() {
    while (sp < src.length() && src.charAt(sp) ≠ '-' && (src.charAt(sp) < '0' || src.charAt(sp) > '9'))
      sp++;
    long sign = 1;
    if (sp < src.length() && src.charAt(sp) == '-') {
      sign = -1;
      sp++;
    }
    long v = 0;
    while (sp < src.length() && src.charAt(sp) ≥ '0' && src.charAt(sp) ≤ '9')
      v = v * 10 + (src.charAt(sp++) - '0');
    return sign * v;
  }

  static String solve(String in) {
    src = in;
    sp = 0;
    int n = (int) nextLong();
    int m = (int) nextLong();
    long sum = 0;
    for (int i = 0; i < n * m; i++)
      sum += nextLong();
    return String.format(Locale.ROOT, "%.9f\n", (double) sum / ((double) n * m));
  }
```

listing 578: java, the whole solver over the sign-aware index scanner: sum the readings, one format at nine decimals (source: icpc/samples-java/src/Ch13/PI.java, lines 24-47)

```
    {
        int pos = 0;
        int n = (int)Next(input, ref pos);
        int m = (int)Next(input, ref pos);
        long sum = 0;
        for (int i = 0; i < n * m; i++)
            sum += Next(input, ref pos);
        return (sum / ((double)n * m)).ToString("F9",
            System.Globalization.CultureInfo.InvariantCulture) + "\n";
    }
```

listing 579: c#, the same sum over the parsed longs, F9 with the invariant culture (source: icpc/samples/src/Ch12/PI.cs, lines 13-23)

```
export function solve(input) {
  const tok = input.split(/\s+/).filter((t) => t.length > 0);
  const n = parseInt(tok[0], 10);
  const m = parseInt(tok[1], 10);
  let sum = 0;
  for (let i = 0; i < n * m; i++) sum += parseFloat(tok[2 + i]);
  return `${(sum / (n * m)).toFixed(9)}\n`;
}
```

listing 580: javascript, parseFloat accumulation, toFixed(9) (source: icpc/samples-js/src/ch12-pi-waterworld.mjs, lines 10-18)

```python
def solve(text):
    vals = list(map(int, text.split()))
    n, m = vals[0], vals[1]
    total = sum(vals[2 : 2 + n * m])
    return f"{total / (n * m):.9f}\n"
```

listing 581: python, one f-string over the int sum (source: icpc/samples-py/src/Ch12/pi.py, lines 8-12)

```lua
  local n = next_int()
  local m = next_int()
  local sum = 0
  for _ = 1, n do
    for _ = 1, m do
      sum = sum + next_int()
    end
  end
  return string.format("%.9f\n", sum / (n * m))
end
```

listing 582: lua, the double loop over rows and steps, string.format (source: icpc/samples-lua/ch12_pi.lua, lines 27-37)

The fixture is the 2x2 scan 0 100 / 100 0, whose equal-area average is exactly 50.000000000, and the crafted all-seven 2x3 grid answers 7.000000000. Counts: c 2 float CHECKs, the C# suite 3 facts, go 2 rows, java 3 ok checks including a negative reading, javascript 2 blocks, python 3 checks, lua 2 named checks. The sum tops out at 1e6 readings of 100, one hundred million, an int32 ceiling with margin, so the int accumulators are safe everywhere and only the final division is floating point.

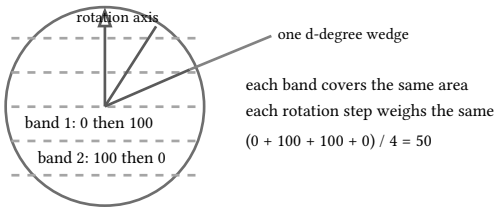


figure 169: the sphere in equal-height bands, one rotation wedge lifted out, the fixture percentages per band

14.12. j, bridging the gap

The 2022 S task returns with the same input and output shapes. A group of n walkers reaches a river at night carrying one torch, the old bridge holds at most c walkers at a time, the torch must travel with every crossing and come back for the next, a group crosses at its slowest member's pace, and the task is the minimum total crossing time. The statement works its own sample by hand, four walkers of 1, 2, 5, and 10 minutes at capacity 2 finishing in 17: the two fastest over, the fastest back, the two slowest over, the second-fastest back, the two fastest over again.

The input is the walker count n, 2 <= n <= 1e4, and the capacity c, 2 <= c <= 1e4, by the problems pdf, then one line of n crossing times t1 through tn, each from 1 to 1e9 minutes, arriving in any order. The output is the single minimum total time under the pdf's 4 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `J-bridgingthegap/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints and works by hand: the four times arrive unsorted, 1 2 10 5, and the answer is the classic 17.

input:

```
4 2
1 2 10 5
```

expected output:

```
17
```

Recognition: n <= 1e4 and c <= 1e4 under the 4 second limit price an exact state machine, not a smarter formula: the (k, g, l) space holds 5e7 to 3.5e8 live states on the judge secrets, and the shipped engines visit each state once at O(1) amortized, which is what the limit buys, the same ratification chapter 13 measured.

The statement's cue is the torch ledger: every crossing and every return changes who stands where, and the optimum turns on the contiguous block of banked fast walkers, the cue that the state must be engineered by hand rather than read off the input. Problem J is the 2023 face of the dp over an engineered state space family. The dp over an engineered state space family spans chapter 12 problem S, chapter 11 problem J, chapter 12 problem I, and chapter 15 problem H.

The classic c = 2 greedy shuffle is the tempting rule and the year's own judge data prices it out: secrets 054 through 102 are tagged wrong-greedy torture cases, and the shuffle misprices every one of them, while the exact sweep fits the same limit.

Every language here ships the engine chapter 13 ratified for the 2022 original, and the exact state is (k, g, l), the k slowest delivered, the far side banking exactly walkers g through g+l-1, torch near. The structure the solutions pdf page 8 proves keeps the machine small: a back crossing is always a single walker, and a forward crossing is the escorted batch, j slow walkers plus c-j escorts who will cross back, the pure slow batch returned by the banked walker g, or the pure fast batch, with the person-0 shuttle completion pricing the endgame. The useful banked count l never passes about n/c, so the states run O(n^2/c) with O(c) transitions each and the real cost is O(n^2) after pruning unreachable states, and the judges warn the implementation is tricky, problem at icpc2023.pdf. The c solver parses through chapter 13's `ch11_scan.h` and go through the package's own `ch12.go` twin, and chapter 13's helper section prints the scan cursor this solver includes. The 2023 secrets 054 through 102 are tagged wrong-greedy torture cases, the classic c=2 shuffle is not enough. Two engine shapes cover the seven languages, the pull-based row sweep with lazy window heaps in c and java and the diagonal sweep with sliding-window materialization in go, javascript, python, and lua, and the C# and javascript entries literally delegate to their chapter 13 engines. Java's port keeps c's struct discipline in its own coin: the C engine's heap entries are structs grown by realloc, java's are pooled Ent objects copied by value so the candidate slots stay reusable, and every growth point is an Arrays.copyOf doubling.

The worked run: trace the model on sample 1. The times sorted ascending are t = 1, 2, 5, 10, capacity 2, and the machine opens at (k, g, l) = (0, 1, 0), torch near, nothing delivered. The pure fast batch sends persons 0 and 1 over at t[l + j - 1] = t[1] = 2 with person 0 shuttling back at t[0] = 1, cost 2 + 1 = 3, banking the block g = 1, l = 1 at (0, 1, 1). The pure slow batch sends the two slowest, persons 2 and 3, over at t[n - k - 1] = t[3] = 10 with the banked walker t[g] = t[1] = 2 returning the torch, cost 10 + 2 = 12, landing at (2, 2, 0). Now n - k - l = 4 - 2 - 0 = 2 <= c, so the finish transition crosses everyone left at the near side's slowest pace t[n - k - 1] = t[1] = 2. The ledger reads 3 + 12 + 2 = 17, the table below, and the trace ends at the printed answer 17.

transition	batch	cost	landing
pure fast, j = 2	persons 0 and 1 over at 2, person 0 back at 1	3	(0, 1, 1)
pure slow, j = 2	persons 2 and 3 over at 10, banked t[1] = 2 back	12	(2, 2, 0)
finish	everyone left over at t[1] = 2	2	done
total	three machine moves	17	

```
    for (int k = 0; k ≤ r; k++) {
      int l = r - k;
      int nc = 0;
      if (r == 0 && k == 0)
        cand[nc].d = 0, cand[nc++].g = 1; // seed (0, 1, 0)
      // pure fast: col[k] at cursor l, window pos ≥ l - c
      LHeap *ck = &col[k];
      if (lheap_query(ck, l, l - c))
        cand[nc].d = ck->act[0].val + t[l] + t[0], cand[nc++].g = 1;
      // escorted: escact at cursor k, no recency bound
      if (lheap_query(&escact, k, INT_MIN))
        cand[nc].d = escact.act[0].val, cand[nc++].g = 1;
      // pure slow: diagonal l+1, window pos ≥ k - c, j ≥ 2;
      // buckets with g > n-1-r can never fire (j ≤ mid)
      if (l + 1 ≤ n && k ≥ 2) {
        Diag *dl = &dg[l + 1];
        int gmax = n - 1 - r;
        for (int i = 0; i < dl->nbk; i++) {
          if (dl->bk[i].g > gmax)
            continue;
          if (lheap_query(&dl->bk[i].h, k, k - c))
            cand[nc].d = dl->bk[i].h.act[0].val, cand[nc++].g = dl->bk[i].g + 1;
        }
        for (int i = 0; i < dl->nr;) {
          REnt *e = &dl->rare[i];
          if (e->hi < k || e->k < k - c || e->g > gmax) {
            dl->rare[i] = dl->rare[--dl->nr]; // gmax only shrinks
            continue;
          }
          if (e->k ≤ k - 2)
            cand[nc].d = e->w, cand[nc++].g = e->g + 1;
          i++;
        }
      }
      // in-row: pure slow j=1 from (k-1, l+1)
      for (int i = 0; i < nprev; i++)
        cand[nc].d = prev[i].d + t[n - k] + t[prev[i].g],
        cand[nc++].g = prev[i].g + 1;
      if (nc + 2 > capcand) {
        while (nc + 2 > capcand)
          capcand *= 2;
        cand = realloc(cand, (size_t)capcand * sizeof(Ent));
      }
      if (nc > capfront) {
        while (nc > capfront)
          capfront *= 2;
        prev = realloc(prev, (size_t)capfront * sizeof(Ent));
        cur = realloc(cur, (size_t)capfront * sizeof(Ent));
      }
      int ncur = front_build(cur, cand, nc);
```

listing 583: c, the per-cell pull: seed, pure-fast column heap, escorted row heap, per-g pure-slow buckets, in-row handoff, finish (source: icpc/samples-c/src/Ch12/pJ.c, lines 203-252)

```
// stride prefix sums: pref[r] covers t[r], t[r+c-1], ... so the
// shuttle completion ss(k, q) = pref[r][j+1] - pref[r][j+1-q] in
// O(1), with r = (n-k-1) mod (c-1) and j = (n-k-1-r)/(c-1)
step := c - 1
pref := make([][]int64, step)
for r := 0; r < step; r++ {
  var arr []int
  for i := n - 1 - r; i ≥ 0; i -= step {
    arr = append(arr, i)
  }
  p := make([]int64, len(arr)+1)
  for i, idx := range arr {
    p[i+1] = p[i] + t[idx]
  }
  pref[r] = p
}
shuttle := func(k, q int) int64 {
  m := n - k - 1
  r := (n - 1 - m) % step
  j := (n - 1 - r - m) / step
  if max := len(pref[r]) - 1 - j; q > max {
    q = max
  }
  return pref[r][j+q] - pref[r][j]
}

// the diagonal cell buffer: buf[l] holds g slots 1..D
type dcol struct {
  d []int64
  m [2]uint64
}
buf := make([]dcol, L0+1)
for i := range buf {
  buf[i].d = make([]int64, D+1)
}
```

listing 584: go, stride prefix sums price the shuttle in O(1), cells live in one reusable diagonal buffer (source: icpc/samples-go/ch12/pjfast.go, lines 62-96)

```
      for (int k = 0; k ≤ r; k++) {
        int l = r - k;
        int nc = 0;
        if (r == 0 && k == 0) {
          cand[nc].d = 0;
          cand[nc++].g = 1; // seed (0, 1, 0)
        }
        // pure fast: col[k] at cursor l, window pos ≥ l - c
        LHeap ck = col[k];
        if (ck.query(l, l - c)) {
          cand[nc].d = ck.aval[0] + t[l] + t[0];
          cand[nc++].g = 1;
        }
        // escorted: escact at cursor k, no recency bound
        if (escact.query(k, Integer.MIN_VALUE)) {
          cand[nc].d = escact.aval[0];
          cand[nc++].g = 1;
        }
        // pure slow: diagonal l+1, window pos ≥ k - c, j ≥ 2;
        // buckets with g > n-1-r can never fire (j ≤ mid)
        if (l + 1 ≤ n && k ≥ 2) {
          Diag dl = dg[l + 1];
          int gmax = n - 1 - r;
          for (int i = 0; i < dl.nbk; i++) {
            if (dl.bkG[i] > gmax)
              continue;
            if (dl.bkH[i].query(k, k - c)) {
              cand[nc].d = dl.bkH[i].aval[0];
              cand[nc++].g = dl.bkG[i] + 1;
            }
          }
          for (int i = 0; i < dl.nr;) {
            REnt e = dl.rare[i];
            if (e.hi < k || e.k < k - c || e.g > gmax) {
              dl.rare[i] = dl.rare[--dl.nr]; // gmax only shrinks
              continue;
            }
            if (e.k ≤ k - 2) {
              cand[nc].d = e.w;
              cand[nc++].g = e.g + 1;
            }
            i++;
          }
        }
        // in-row: pure slow j=1 from (k-1, l+1)
        for (int i = 0; i < nprev; i++) {
          cand[nc].d = prev[i].d + t[n - k] + t[prev[i].g];
          cand[nc++].g = prev[i].g + 1;
        }
```

listing 585: java, the per-cell pull: seed, pure-fast column heap, escorted row heap, per-g pure-slow buckets and rare list, in-row handoff (source: icpc/samples-java/src/Ch13/PJ.java, lines 280-328)

```
namespace IcpcBook.Ch12;

/// <summary>
/// ICPC WF 2023 J: bridging the gap, the concurrent 2022 set's S with
/// identical input and output shapes (n, c, then n crossing times;
/// print the minimal total crossing time). Delegates to the Ch11 S
/// engine.
/// </summary>
public static class PJ
{
    public static string Solve(string input) => Ch11.PS.Solve(input);
}
```

listing 586: c#, the whole file: Solve forwards to the Ch11 S engine, the problems are byte-compatible (source: icpc/samples/src/Ch12/PJ.cs, lines 1-12)

```
// icpc 2023 j bridging the gap: n walkers share one torch over a
// bridge of capacity c, a batch moving at its slowest member's pace.
// the state (k, g, l) records the k slowest delivered, the banked
// walkers exactly g..g+l-1, torch near (solutions.pdf p.8). the
// recurrence is the one validated for 2022 s: the diagonal-swept
// engine in ch11-ps-bridgingthegap.mjs solves the identical problem,
// so this entry point delegates to it
import { readFileSync } from "node:fs";
import { pathToFileURL } from "node:url";
import assert from "node:assert/strict";
import { solve as solveBridge } from "./ch11-ps-bridgingthegap.mjs";

export const solve = solveBridge;

function selfcheck() {
  assert.equal(solve("3 2\n1 2 3\n"), "6\n");
  assert.equal(solve("2 2\n5 7\n"), "7\n");
  assert.equal(solve("3 5\n4 9 2\n"), "9\n");
}

if (process.argv[1] && import.meta.url === pathToFileURL(process.argv[1]).href) {
  if (process.argv[2]) {
    process.stdout.write(solve(readFileSync(process.argv[2], "utf8")));
```

listing 587: javascript, the entry point imports ch11-ps outright and re-runs its self-check (source: icpc/samples-js/src/ch12-pj-bridgingthegap.mjs, lines 1-23)

```
def solve(text):
    toks = text.split()
    n = int(toks[0])
    c = int(toks[1])
    t = sorted(int(v) for v in toks[2 : 2 + n])
    if n ≤ c:
        return f"{t[n - 1]}\n"
    if c == 2:
        return f"{solve_c2(t)}\n"
    depth = n // c + 4
    if c < depth:
        depth = c
    if depth > n - 1:
        depth = n - 1
    if depth > 120:
        depth = 120
    D = depth
    lcap = 4 * n // c + 8
    if n ≥ 4 * c:
        alt = 3 * c + c // 2 + 64
        if alt > lcap:
            lcap = alt
    if lcap > n:
        lcap = n
    L0 = lcap
    ring2 = D + 2
    t0 = t[0]
    best = INF

    step = c - 1
    # pref[r] walks t[n-1-r], t[n-1-r-step], ... so shuttle(k, q) is
    # pref[k % step][k//step + q] - pref[k % step][k//step]
    pref = []
    for r in range(step):
        p = [0]
        acc = 0
        i = n - 1 - r
        while i ≥ 0:
            acc += t[i]
            p.append(acc)
            i -= step
        pref.append(p)

    # diagonal cell buffer: bm[l] is a bitmask over g slots, bv[l][g]
    # the value (stale where the bit is clear)
    bm = [0] * (L0 + 1)
    bv = [[INF] * (D + 1) for _ in range(L0 + 1)]
```

listing 588: python, the c=2 classic shortcut, judged caps, stride prefix sums, and the bitmask diagonal buffer (source: icpc/samples-py/src/Ch12/pj.py, lines 46-92)

```
local sweep = require("ch11_psweep")

local function solve(input)
  local pos, len = 1, #input
  local function next_int()
    while pos ≤ len do
      local b = input:byte(pos)
      if b == 32 or b == 10 or b == 13 or b == 9 then
        pos = pos + 1
      else
        break
      end
    end
    local v = 0
    while pos ≤ len do
      local b = input:byte(pos)
      if b < 48 or b > 57 then break end
      v = v * 10 + (b - 48)
      pos = pos + 1
    end
    return v
  end
  local n = next_int()
  local c = next_int()
  local tt = {}
  for i = 1, n do tt[i] = next_int() end
  table.sort(tt)
  return sweep.solve(n, c, tt) .. "\n"
```

listing 589: lua, requires ch11_psweep verbatim, parse and sort only, the wall documented in the header (source: icpc/samples-lua/ch12_pj.lua, lines 26-53)

The fixture is 3 walkers at capacity 2 with times 1 2 3: the pair 1 and 2 crosses for 2, walker 1 returns for 1, and 1 and 3 cross for 3, total 6 against 7 for shipping the slow pair first. Every suite asserts 6, plus the pair crossing together at 7 and the capacity-covers-all 9: c 3 CHECKs, the C# suite 3 facts, go 3 rows, javascript 3 blocks, python 3 checks, lua 4 named checks, java 4 ok checks, its escorted pair 4 walkers at capacity 2 with times 10 1 10 1 answering 14 proven by a C-oracle reference run. Totals reach about 2e13, int64 in c, C#, go, and lua, java longs, exact below 2^53 in javascript doubles, native in python. The judged record splits the families, and this chapter records the walls rather than pretending: go finishes the 104 cases in roughly 11 seconds worst, javascript in 72.8, c in 58 to 99 seconds of CPU, and C# in 95 to 129 at JIT parity with c. Python judges 57 of 104 with exactly 47 timeouts and zero wrong answers, every timeout an n = 1e4 secret with c in [3, 500], and the mechanism is CPython's 30-bit fast-int boxing: once the dp totals pass 1e13 every sweep operation allocates a boxed multi-digit integer, so the sweep becomes allocation-bound where the compiled languages are not. Lua's verdict is load-dependent, and the chapter records both measurements with their context: on a quiet machine the same `ch11_psweep` engine judges 88 of 104 with 16 timeout-only failures, under four to five concurrent book agents the count was 70 of 104 with 34, and both runs score zero wrong answers, which closes the year at 635 of 651 lua cases with ten of eleven problems fully clean. The 2022 ratification measured this engine at 142 to 417 seconds on the heaviest shapes, and letting three of the wrong-greedy cases run to 192 to 198 seconds produced answers matching the official ones, so every failure is interpreter speed, not correctness. Java judges all 104 with no wall at all: the three hot wrong-greedy cases measure 25.3, 38.9, and 62.4 seconds against the same engine's 103.1-second C run under like load, inside the harness cap with margin. Both walls are terminal in the ratified sense, the sweep operation is itself the work and no restructure exists, the 47 versus 4 timeout asymmetry between python and lua on the same state space being the measured evidence.

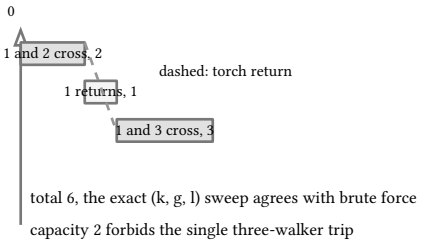


figure 170: the fixture's three crossings on a timeline, the torch return dashed

14.13. k, alea iacta est

The game plays with d fair six-sided dice, each face carrying one symbol, and a dictionary of valid words, every word exactly d symbols long. You roll all the dice, and after each roll you may lock any dice and reroll the rest, stopping as soon as the shown symbols can be arranged into a dictionary word, and the task prints the expected number of rolls under optimal play, or impossible when no play ever spells one. The statement walks its own sample: five dice first show P, X, R, E, and S, you keep P, E, and S, the two rerolled dice yield E and A, then you keep four symbols and reroll the last die, one with three C faces, a 50 percent shot at PEACE.

The input is d and w, the die count 1 <= d <= 6 and the word count 1 <= w <= 2e5 by the problems pdf, then d lines of 6 symbols each, the faces of each die, then w distinct words of exactly d symbols over uppercase A through Z and digits 0 through 9. The output is the expected roll count with an absolute or relative error of at most 1e-6, or the word impossible, under the pdf's 10 second limit.

Official sample 1, reprinted byte for byte from the judge data pair `K-aleaiactaest/sample-1.in` and `sample-1.ans`, the same pair the problems pdf prints: the five dice and eight P-words of the story, and optimal play averages 9.677887141 rolls.

input:

```
5 8
ABCDEP
AEHOXU
AISOLR
ABCDEF
ABCSCC
PARSE
PAUSE
PHASE
POISE
PROSE
PULSE
PURSE
PEACE
```

expected output:

```
9.677887141
```

Recognition: d <= 6 and w <= 2e5 under the 10 second limit price the state machine: 7^d lock vectors, 7^6 = 117649 at worst, with 2^d - 1 = 63 starring choices per fixed state, about 8e6 transitions materialized once and swept in place, while the w words hash in O(w d), linear in everything that matters.

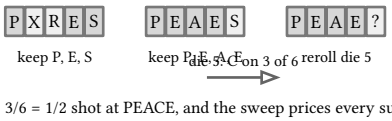
The statement's cue is the expected number of rolls under optimal play, or impossible: an expectation over an enormous implicit outcome tree, priced by a fixed point over the lock states rather than by listing plays. Problem K is the 2023 face of the counting and expectation without enumeration family. Counting and expectation without

enumeration is the shared core of chapter 11 problem D, chapter 11 problem I, chapter 13 problem Q, and chapter 13 problem E.

Enumerating play sequences is the priced-out alternative: each round branches 63 starring choices into up to 6^6 = 46656 roll outcomes, and the expectation needs the whole tree, while the fixed point is 117649 cells swept to convergence.

Each die shows at most six distinct symbols, so per die the symbols map to local ids with face multiplicities and digit 6 marks rerolling, and states are base-7 vectors, 7^6 = 117649 at worst. From a starred state one roll costs 1 and lands on the multiplicity-weighted average of its outcome refinements, from a fixed non-goal state any nonempty subset may be starred at cost 0, and the goal states are the fixed states whose shown symbols sort to a dictionary word's sorted multiset, each word canonicalized by sorting into one hash set of w entries. The solutions pdf pages 8 and 9 run dijkstra backwards with incremental sums, per undetermined state a count of processed refinements and their running dist sum relaxing dist at (6^s + sum) / count, about O(7^d **d + w** d), but the book's engines sweep in-place value iteration to a 1e-13 fixed point, because the backward dijkstra shortcut can freeze early when refinements with larger distances pop late, and the pdf itself notes every contest team used value iteration, problem at icpc2023.pdf. Book 10′s chapter 32, dynamic programming ii, develops this style of value iteration over the word-multiset states.

The worked run: trace the model on sample 1. The five printed dice map to per-symbol face counts and the eight printed words all open with P, a symbol only die 1 carries, on one face of six. The lock space is 7^5 = 16807 vectors, the goal states the fixed vectors whose sorted symbols match one of the eight sorted words. The statement's own walk plays the model's moves: from P, X, R, E, S no word matches, and the move keeps P, E, S, starring X and R. The reroll yields E and A, giving P, E, A, E, S, still no match, and the move keeps four symbols, rerolling only die 5, whose printed faces A, B, C, S, C, C carry C on three of six: the shot at PEACE is 3/6 = 1/2. The sweep prices every such decision together and settles at the expectation the judge prints, and the trace ends at the printed answer `9.677887141`.



3/6 = 1/2 shot at PEACE, and the sweep prices every such line together

figure 171: sample 1′s statement walk: the kept symbols shaded at each state, the final shot at PEACE priced off die 5

```
    double delta = 0;
    for (int s = 0; s < nst; s++) {
      if (isgoal[s])
        continue;
      decode(s);
      double nd;
      if (starcnt[s]) { // starred: expectation over outcomes
        int ns = 0;
        for (int i = 0; i < d; i++)
          if (dig[i] == STAR)
            starpos[ns++] = i;
          else if (fc[i][dig[i]] == 0) {
            nd = dist[s]; // unreachable refinement: keep
            goto next;
          }
        for (int k = 0; k < ns; k++) {
          combo[k] = 0;
          while (combo[k] < 6 && fc[starpos[k]][combo[k]] == 0)
            combo[k]++;
        }
        double sum = 0;
        long long tot = 0;
        for (;;) {
          int y = 0;
          long long mult = 1;
          for (int i = 0; i < d; i++) {
            int v = dig[i];
            if (v == STAR) {
              int k = 0;
              for (int j = 0; j < i; j++)
                k += dig[j] == STAR;
              v = combo[k];
              mult *= fc[i][v];
            }
            y = y * 7 + v;
          }
          sum += mult * dist[y];
          tot += mult;
          int k = ns - 1;
          while (k >= 0) {
            do {
              combo[k]++;
            } while (combo[k] < 6 && fc[starpos[k]][combo[k]] == 0);
            if (combo[k] < 6)
              break;
            combo[k] = 0;
            while (combo[k] < 6 && fc[starpos[k]][combo[k]] == 0)
              combo[k]++;
            k--;
```

listing 590: c, the Gauss-Seidel sweep: outcome enumeration by combo odometer over the starred dice, starring masks for fixed states (source: icpc/samples-c/src/Ch12/pK.c, lines 157-205)

```
  var sp, combos [6]int
  for sweep := 0; sweep < 200000; sweep++ {
    delta := 0.0
    for s := 0; s < nst; s++ {
      if isgoal[s] {
        continue
      }
      decode(s)
      var nd float64
      if starcnt[s] > 0 {
        phantom := false
        for i := 0; i < d; i++ {
          if dig[i] ≠ 6 && fc[i][dig[i]] == 0 {
            phantom = true
            break
          }
        }
        if phantom {
          continue
        }
        ns := 0
        for i := 0; i < d; i++ {
          if dig[i] == 6 {
            sp[ns] = i
            combos[ns] = 0
            for combos[ns] < 6 && fc[i][combos[ns]] == 0 {
              combos[ns]++
            }
            ns++
          }
        }
        // enumerate outcome symbols over the starred dice
        var sum float64
        var tot int64
        for {
          y := 0
          var mult int64 = 1
          k := 0
          for i := 0; i < d; i++ {
            v := dig[i]
            if v == 6 {
              v = combos[k]
              mult *= fc[i][v]
              k++
            }
            y = y*7 + v
          }
          sum += float64(mult) * dist[y]
          tot += mult
          j := ns - 1
          for ; j ≥ 0; j-- {
```

listing 591: go, the same sweep in decreasing state order, the odometer over per-die live symbols (source: icpc/samples-go/ch12/pk.go, lines 93-143)

```
        if (starcnt[s] > 0) { // starred: expectation over outcomes
          int ns = 0;
          boolean unreachable = false;
          for (int i = 0; i < d; i++) {
            if (dig[i] == STAR)
              starpos[ns++] = i;
            else if (fc[i][dig[i]] == 0)
              unreachable = true; // dead refinement: keep the value
          }
          if (unreachable) {
            newVal = dist[s];
          } else {
            for (int k = 0; k < ns; k++) {
              combo[k] = 0;
              while (combo[k] < 6 && fc[starpos[k]][combo[k]] == 0)
                combo[k]++;
            }
            double sum = 0;
            long tot = 0;
            for (;;) {
              int y = 0;
              long mult = 1;
              for (int i = 0; i < d; i++) {
                int v = dig[i];
                if (v == STAR) {
                  int k = 0;
                  for (int j = 0; j < i; j++)
                    if (dig[j] == STAR)
                      k++;
                  v = combo[k];
                  mult *= fc[i][v];
                }
                y = y * 7 + v;
              }
              sum += mult * dist[y];
              tot += mult;
              int k = ns - 1;
              while (k ≥ 0) {
                do {
                  combo[k]++;
                } while (combo[k] < 6 && fc[starpos[k]][combo[k]] == 0);
                if (combo[k] < 6)
                  break;
                combo[k] = 0;
                while (combo[k] < 6 && fc[starpos[k]][combo[k]] == 0)
                  combo[k]++;
                k--;
              }
              if (k < 0)
                break;
            }
```

listing 592: java, the starred half of the sweep: the combo odometer over per-die live symbols, multiplicity-weighted dist sums folded in place (source: icpc/samples-java/src/Ch13/PK.java, lines 156-206)

```
            if (isgoal[s] || dead[s])
                continue;
            int t = s;
            for (int i = d - 1; i ≥ 0; i--)
            {
                dig[i] = t % 7;
                t /= 7;
            }
            if (!starred[s])
            {
                for (int mask = 1; mask < (1 << d); mask++)
                {
                    int y = 0;
                    for (int i = 0; i < d; i++)
                        y = y * 7 + ((mask >> i & 1) ≠ 0 ? Star : dig[i]);
                    outY.Add(y);
                    outM.Add(0); // index alignment; unused for masks
                }
                continue;
            }
            int ns = 0;
            for (int i = 0; i < d; i++)
                if (dig[i] == Star)
                    starpos[ns++] = i;
            for (int k = 0; k < ns; k++)
            {
                combo[k] = 0;
                while (combo[k] < 6 && fc[starpos[k], combo[k]] == 0)
                    combo[k]++;
            }
            long tot = 0;
            for (; ; )
            {
                int y = 0;
                long mult = 1;
                for (int i = 0; i < d; i++)
                {
                    int v = dig[i];
                    if (v == Star)
                    {
                        int sk = 0;
                        for (int j = 0; j < i; j++)
                            if (dig[j] == Star)
                                sk++;
                        v = combo[sk];
                        mult *= fc[i, v];
                    }
```

listing 593: c#, transitions materialized once into flat outY and outM arrays so every sweep is a linear scan (source: icpc/samples/src/Ch12/PK.cs, lines 130-176)

```
        // enumerate outcome symbols over the starred dice
        let sum = 0, tot = 0;
        for (;;) {
          let y = 0, mult = 1, k = 0;
          for (let i = 0; i < d; i++) {
            let v = dig[i];
            if (v === 6) {
              v = combos[k];
              mult *= fci[i][v];
              k++;
            }
            y = y * 7 + v;
          }
          sum += mult * dist[y];
          tot += mult;
          let j = ns - 1;
          for (; j ≥ 0; j--) {
            combos[j]++;
            while (combos[j] < 6 && fci[sp[j]][combos[j]] === 0) combos[j]++;
            if (combos[j] < 6) break;
            combos[j] = 0;
            while (combos[j] < 6 && fci[sp[j]][combos[j]] === 0) combos[j]++;
          }
          if (j < 0) break;
        }
        nd = 1 + sum / tot;
      } else {
        let reachable = true;
        for (let i = 0; i < d; i++)
          if (fci[i][dig[i]] === 0) {
            reachable = false;
            break;
          }
        if (!reachable) continue;
        let best = INFK;
        for (let mask = 1; mask < 1 << d; mask++) {
          let y = 0;
          for (let i = 0; i < d; i++) {
            let v = dig[i];
            if ((mask >> i) & 1) v = 6;
            y = y * 7 + v;
          }
          if (dist[y] < best) best = dist[y];
        }
```

listing 594: javascript, the outcome odometer and the 2^d-1 starring masks over typed arrays (source: icpc/samples-js/src/ch12-pk-aleaiactaest.mjs, lines 98-141)

```
    # contraction dag first: T[S] lives over comp(S) digits in
    # product order; each child contracts the top axis i of S out of
    # its parent, rows[S][j] being the parent cells child cell j
    # sums (the parent's layout strides), and css[S] the T[S] cell
    # strides the star map shares
    subs = sorted(range(1, 1 << d), key=lambda m: (bin(m).count("1"), m))
    rows = {}
    wv = {}
    css = {}
    for S in subs:
        i = S.bit_length() - 1
        comp = [a for a in range(d) if not S >> a & 1]
        pax = sorted(comp + [i])
        ps = {}
        acc = 1
        for a in reversed(pax):
            ps[a] = acc
            acc *= K[a]
        rl = []
        for digs in product(*(range(K[a]) for a in comp)):
            b = 0
            for a, v in zip(comp, digs):
                b += v * ps[a]
            rl.append(range(b, b + K[i] * ps[i], ps[i]))
        rows[S] = rl
        wv[S] = fcs[i]
        cs = {}
        acc = 1
        for a in reversed(comp):
            cs[a] = acc
            acc *= K[a]
        css[S] = cs
```

listing 595: python, the contraction dag: rows map each subset's cells onto its parent's layout, work shared down the hierarchy (source: icpc/samples-py/src/Ch12/pk.py, lines 63-94)

```
  -- precompute transitions: starred states get (y, mult) outcome
  -- pairs in flat arrays; fixed states get their 2^d-1 starring
  -- targets
  local oy, om = {}, {}
  local npair = 0
  local poff, pend, ptot = {}, {}, {}
  local ftgt = {}
  local nmask = (1 << d) - 1
  for s = 0, nst - 1 do
    local q = s + 1
    decode(s)
    local ns = 0
    local unreachable = false
    for i = 1, d do
      if dig[i] == STAR then
        ns = ns + 1
        starpos[ns] = i
      elseif not fc[i][dig[i]] then
        unreachable = true
      end
    end
    if ns > 0 and not unreachable then
      poff[q] = npair + 1
      local combo = {}
      for k = 1, ns do combo[k] = live[starpos[k]][1] end
      local tot = 0
      while true do
        local y = 0
        local mult = 1
        local k = 0
        for i = 1, d do
          local v = dig[i]
          if v == STAR then
            k = k + 1
            v = combo[k]
            mult = mult * fc[i][v]
          end
          y = y * 7 + v
        end
        npair = npair + 1
        oy[npair] = y + 1
        om[npair] = mult
```

listing 596: lua, outcome pairs and starring targets precomputed into flat arrays, poff slicing per state (source: icpc/samples-lua/ch12_pk.lua, lines 108-149)

The fixture is one die ABBBBB against the word A: each roll shows A with probability 1/6, the roll count is geometric, and the answer is 6.000000000 with no choices to optimize. The crafted pair adds the free die, AAAAAA beside ABCDEF spelling AB still averages 6 because only the second die matters, and the word B on an all-A die is impossible. Counts: c 3 float CHECKs, the C# suite 3 facts, go 3 rows, javascript 3 blocks, python 4 checks including the deterministic AAAAAA BBBBBB pair at 1.000000000, lua 3 named checks, java 4 ok checks. Doubles throughout, 7^6 states, and the 1e18 seed separates impossible cleanly. C scrambles each sorted word through an unsigned golden-ratio hash; java holds the sorted multiset as a plain long key below 128^6 = 2^42, so the year's java tree opens no unsigned road at all. Performance is where the languages split, and K is the set's constructive counterpoint to J: the wall was table construction, not the arithmetic, and it restructured away. Python's first value iteration capped at about 700 Gauss-Seidel sweeps and ran the four d=6 secrets at 190 to 230 seconds against the 10-second limit. Porting lua's precomputed transition arrays brought flat sweeps with zero per-state construction,

lua's plain sweeps needing about 400 passes on the d=6 cases before the precompute, and got python to 98 to 132 seconds. Replacing the starred half of each python sweep with a contraction hierarchy over the 63 subset sums, refinements of reroll-S share their work down a dag, cut each sweep from roughly 205 to 85 milliseconds and closed the d=6 secrets at 61 to 81 seconds, 33 to 50 percent under the cap, judged 27 of 27 with correctness held against an 80-instance independent reference within 4e-9. The contrast is one sentence: K's wall was build cost and moved, J's sweep arithmetic is the work itself and does not move.

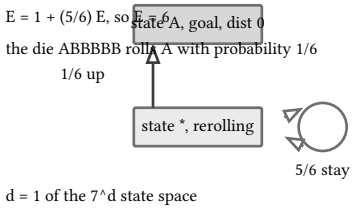


figure 172: the one-die state ladder: A on top, the star rung below, the 1/6 chance up, the 5/6 self-loop

14.14. across the seven languages

problem	kernel	c	go and java	c# and js	python and lua
a	exclusion hypotheses over triples of the five rows	all ten triples per hypothesis	go first three surviving rows, one det3, java cram3 with exact quotients	c# reads line 4 at the lie, js exports the pure core	comprehension walk, nine-scalar det
b	sperner cap, n largest strings, cyclic repeat	callback enumerator	go gen tries the 1 branch first, java Comb tuples repeated cyclically	c# sorted masks, js firstSubsets from the comb kit	`first_subsets` helpers over one-positions
c	doubled score points, two hull halfplane queries	`ch12_hull` kit	go monotone chain, per-edge interpolation, java Hull kit with chain and both queries	c# edge walk with bounds, js monotoneHull twin, queries in the solver	bisect counts, hull modules
d	unfold-straights plus corner chains	1024-grid ternary legs	go bestLeg closure, infeasible at infinity, java the same grid plus ternary through the Geom kit	c# run detection with bisected edges, js cross-product crossings, order checked	grid ternary ports over flat scalars
e	counting dp with forward prune, direct enumeration	saturating int128 sums	go enumeration walk mirrors f's branching, java cardNext in BigInteger, compareTo saturation at 4e18	c# string-keyed memo, js typed-array chained hash past the Map cap	depth-indexed scratch, py raised cap
f	corner sigma cycles, rotation congruences pairwise	`crt_add` per distinct modulus, kmp kit	go cycMatch closure, crt table by pointer, java crtAdd plus cy-	c# CycMatch failure function, js crtAdd over the year-local kit	orbit walk, residues per modulus

problem	kernel	c	go and java	c# and js	python and lua
			cMatch off the Kmp twin		
g	mod-3 propagation over button components	stamp-retry BFS	go touched-cell reset closure, java stamps with epoch claims between components	c# revalidates in BFS order, js chained edge arrays with resets	anchored at pins, lua stamps like c
h	full-gap greedy plus rested-window repair	three passes over long arrays	go compact core, one-line trim, java sentinel gap 0, floor-div trim	c# horizon, cover, repair passes, js Float64Array times, floor trims	pair lists, floor division trims
i	equal-area bands, plain average	int sums	go one Sprintf, java long sum, Locale.ROOT at nine decimals	c# F9 culture, js toFixed(9)	one f-string, string.format
j	exact (k, g, l) machine, no safe caps	pull sweep with lazy heaps	go diagonal sweep, java pull sweep with pooled Ent fronts	c# delegates to the Ch11 engine, js imports `ch11-ps`	bitmask sweep, lua requires `ch11_psweep`
k	value iteration to a 1e-13 fixed point	combo odometer	go decreasing-order sweep, java combo odometer over live symbols	c# flat transition arrays, js typed arrays	py contraction dag, lua precomputed pairs

The pattern this set adds to the book's ledger: one interactive problem whose judge-file reading was retracted as circular validation and re-derived from the file format itself, A, one hull problem where the judge corrected the book's own first derivation and the candidate range, C, one problem zero teams solved whose constant-tail family hides past the enumeration threshold, E, one constructive fix where the official sketch admits no candidate at all on its own sample, D, and the two interpreter walls of J carried over from 2022 beside K's restructured wall that moved. Everything else is the standing lesson again, the same derivation ports across seven languages and the ports agree because the suites pin the same fixtures everywhere.

sources: ICPC Foundation and icpc.global, the problemset at icpc2023.pdf and the solutions at finals2023solutions.pdf, both accessed 2026-09-14 and cached under `ref/icpc/2023/` with sha256 digests in `ref/icpc/INDEX.md`. The solutions pdf is the algorithm and complexity reference for every section above, and its page 1 statistics are the contest counts in the introduction. The judge data, 651 inputs across the 11 letters, sits local-only under `ref/icpc/2023/data/`, and the official sample pairs reprinted in the sections above are byte-for-byte its sample-N.in and sample-N.ans files, attributed per section. The chapter's suites are the frozen solver files: C runs 30 CHECK-family self-checks across the 11 Ch12 programs, the C# suite is 40 [Fact] tests including the two post-fix sphinx traps, go pins the fixtures in 10 table-driven test functions across its two ch12 test files, javascript runs 30 it blocks carrying 31 assertions, python's 16 Ch12 files carry 81 `check(` occurrences, and lua reports 31 named checks through run.lua, 2 to 4 per file. The java tree joins them under `samples-java/src/Ch13`: 16 files, the 11 solvers plus the five year-helper classes, reporting 63 ok checks through `tools/run-java-samples.ps1` and judging 651 of 651 official cases against the same local data. The java tree alone follows the post-renumber chapter numbers, so its 2023 solvers sit in Ch13 beside the siblings' historical Ch12 directories.

15. icpc world finals 2025

The 49th ICPC World Championship ran in Baku in 2025 with twelve problems, A through L, published as problemset.pdf. St Petersburg State University took the title. The set has a judge killer in C, which no team solved, and an implementation marathon in G, which fell to two teams at 270 minutes, while L opened the scoreboard at 6 minutes, D and L each going to 138 teams. Every walkthrough in this chapter takes its algorithm and complexity from the official solutions sketch, cited per problem, and the fixtures are crafted minis with hand-derived answers, a few retracing configurations from the public samples. Two structural exceptions shape the set for this book: B is output-validating with no judge answers at all, judged through a validity checker over the matching criterion, and I is interactive with no answers to diff, judged through a driver that speaks the protocol against each language's solver core, both beside the self-check simulators the walkthroughs name where they occur.

15.1. the year-local helpers: kuhn matchings, the rollback sweep engine, and slot wheels

Four helper kits are year-local to this chapter, the match, moat, and wheels engines plus the go package's shared scan cursor, and everything else the year's solvers import comes from the wave 1 toolbox chapters 2 through 9. The C solvers parse through the c tree's `Ch11/ch11_scan.h` beside the chapter 2 io header rather than a year-local twin, so both print in their own chapters and only go adds a file for parsing here. Each kit states its contract and prints its twin files below in the book's fixed language order, and C# inlines the match and wheels kits inside its problem B and I solvers, so its one twin file is the moat engine's Moat.cs. Java's twins are the match class and the rollback dsu, its moat engine inline in its G solver and its wheels simulator nested in its I file.

The match kit backs problem B with Kuhn's augmenting-path bipartite matching over a caller-built edge list, the parity-of-omega graphs never exceeding 200 plus 200 nodes below the Bertrand cutover. C keeps 1-based edge arrays inside `Ch13/ch13_match.h`, go's `match13` carries the same arrays as slices, java's `Match` grows the same 1-based edge arrays by doubling, javascript's `Match` class stamps `seen` over Int32Arrays, python's `Match` class walks plain lists under a raised recursion limit, and lua's `match.kuhn` takes adjacency lists with 0 the unmatched sentinel. C# inlines the whole augment inside its problem B solver:

```c
static int match_try(Match *m, int l) {
  for (int e = m->head[l]; e; e = m->nxt[e]) {
    int r = m->dst[e];
    if (m->seen[r] == m->stamp)
      continue;
    m->seen[r] = m->stamp;
    if (m->matchr[r] < 0 || match_try(m, m->matchr[r])) {
      m->matchr[r] = l;
      return 1;
    }
  }
  return 0;
}

// returns the matching size
static int match_run(Match *m) {
  int size = 0;
  for (int r = 0; r <= m->nr; r++)
    m->matchr[r] = -1;
  for (int l = 1; l <= m->nl; l++) {
    m->stamp++;
    size += match_try(m, l);
  }
  return size;
}
```

listing 597: c, match_try augments over the 1-based edge list, match_run stamping seen once per left vertex and returning the size (source: icpc/samples-c/src/Ch13/ch13_match.h, lines 41-65)

```go
func (m *match13) try(l int) bool {
  for e := m.head[l]; e != 0; e = m.nxt[e-1] {
    r := m.dst[e-1]
    if m.seen[r] == m.stamp {
      continue
    }
    m.seen[r] = m.stamp
    if m.matchr[r] == 0 || m.try(m.matchr[r]) {
      m.matchr[r] = l
      return true
    }
  }
  return false
}

// run returns the maximum matching size.
func (m *match13) run() int {
  size := 0
  for l := 1; l <= m.nl; l++ {
    m.stamp++
    if m.try(l) {
      size++
    }
  }
  return size
}
```

listing 598: go, the same try recursion over slice edge lists, run stamping seen per left vertex, 0 sentinel unmatched (source: icpc/samples-go/ch13/match.go, lines 31-56)

```
  private boolean try_(int l) {
    for (int e = head[l]; e != 0; e = nxt[e]) {
      int r = dst[e];
      if (seen[r] == stamp)
        continue;
      seen[r] = stamp;
      if (matchr[r] < 0 || try_(matchr[r])) {
        matchr[r] = l;
        return true;
      }
    }
    return false;
  }

  // returns the matching size
  int run() {
    java.util.Arrays.fill(matchr, -1);
    int size = 0;
    for (int l = 1; l ≤ nl; l++) {
      stamp++;
      if (try_(l))
        size++;
    }
    return size;
  }
```

listing 599: java, the Match class, try_ over stamp-seen edge walks, run stamping once per left vertex and returning the size (source: icpc/samples-java/src/Ch14/Match.java, lines 34-58)

```
  try(l) {
    for (let e = this.head[l]; e; e = this.nxt[e]) {
      const r = this.dst[e];
      if (this.seen[r] === this.stamp) continue;
      this.seen[r] = this.stamp;
      if (this.matchr[r] < 0 || this.try(this.matchr[r])) {
        this.matchr[r] = l;
        return 1;
      }
    }
    return 0;
  }

  // returns the matching size
  run() {
    let size = 0;
    this.matchr.fill(-1);
    for (let l = 1; l ≤ this.nl; l++) {
      this.stamp++;
      size += this.try(l);
    }
    return size;
  }
```

listing 600: javascript, the Match class, try over Int32Array seen stamps, run refilling matchr and driving every left vertex (source: icpc/samples-js/src/ch13-match.mjs, lines 23-45)

```
    def _try(self, lu):
        head, nxt, dst = self.head, self.nxt, self.dst
        e = head[lu]
        while e:
            r = dst[e - 1]
            if self.seen[r] != self.stamp:
                self.seen[r] = self.stamp
                if not self.matchr[r] or self._try(self.matchr[r]):
                    self.matchr[r] = lu
                    return True
            e = nxt[e - 1]
        return False

    def run(self):
        size = 0
        for lu in range(1, self.nl + 1):
            self.stamp += 1
            if self._try(lu):
                size += 1
        return size
```

listing 601: python, the Match class over plain lists, _try walking nxt with 1-based edge indices, run returning the size (source: icpc/samples-py/src/Ch13/ch13_match.py, lines 23-42)

```
-- ch13 year-local bipartite matching helper: kuhn's augmenting paths
-- over a caller-built adjacency list (adj[u] lists right nodes)
local match = {}

-- returns the matching size
function match.kuhn(left, right, adj)
  local match_r = {}
  for v = 1, right do match_r[v] = 0 end
  local seen = {}
  local function augment(u)
    for _, v in ipairs(adj[u]) do
      if not seen[v] then
        seen[v] = true
        if match_r[v] == 0 or augment(match_r[v]) then
          match_r[v] = u
          return true
        end
      end
    end
    return false
  end
  local size = 0
  for u = 1, left do
    for v = 1, right do seen[v] = false end
    if augment(u) then size = size + 1 end
  end
  return size
end

return match
```

listing 602: lua, match.kuhn whole, the nested augment closure resetting seen per left vertex (source: icpc/samples-lua/ch13_match.lua, lines 1-30)

The moat kit backs problem G and is the year's largest contract, the offline dynamic connectivity sweep: per-triangle cut segments with linear lengths, lifetimes hung on a segment tree over level intervals, and a rollback dsu whose components carry the summed length a + b*h plus west and east border flags, an intrusive list exposing both-border components, every payload change logged for undo. C splits the contract in two, the engine's static arrays in `Ch13/ch13_moat.h` over the inline dsu of `Ch13/ch13_rollback.h`, go rolls both into the `moat13` struct's slices, java inverts the c split, the dsu alone a twin in `Rollback.java` with the sweep engine inline in its G solver, javascript's module-static typed arrays inline the dsu inside `ch13-moat.mjs`, python's `ch13_rollback.py` holds the whole `Moat` class despite the name, lua mirrors the c split with `ch13_moat.lua` requiring `ch13_rollback.lua`, and C#'s static `Moat.cs` is the language's one twin file:

```c
static void ch13_add_seg(int id) {
  ch13_rb_unite(par, dsz, rblog, &rbtop, SU[id], SV[id]);
  int c = rblog[rbtop - 2], p = rblog[rbtop - 1];
  if (c < 0)
    return;
  ppay[paytop] = p;
  payA[paytop] = cA[p];
  payB[paytop] = cB[p];
  payF[paytop] = cfl[p];
  paytop++;
  if ((cfl[c] & 3) == 3)
    lop_remove(c);
  int wasp = (cfl[p] & 3) == 3;
  cA[p] += cA[c] + SA[id];
  cB[p] += cB[c] + SB[id];
  cfl[p] = (unsigned char)(cfl[p] | cfl[c]);
  if (!wasp && (cfl[p] & 3) == 3)
    lop_insert(p);
}

static void ch13_undo_unions(int count) {
  for (int k = 0; k < count; k++) {
    if (rblog[rbtop - 2] >= 0) {
      paytop--;
      cA[ppay[paytop]] = payA[paytop];
      cB[ppay[paytop]] = payB[paytop];
      cfl[ppay[paytop]] = payF[paytop];
    }
    ch13_rb_undo(par, dsz, rblog, &rbtop);
  }
}

// fresh singletons with the caller's per-node border flags
static void ch13_reset_dsu(int n) {
  for (int i = 0; i < n; i++) {
    par[i] = i;
    dsz[i] = 1;
    cA[i] = 0;
    cB[i] = 0;
    lnx[i] = lpr[i] = -1;
  }
  lhead = -1;
  rbtop = paytop = ltop = 0;
}
```

listing 603: c, ch13_add_seg merging child payload, cut length, and border flags into the parent with the list moves logged, ch13_undo_unions unwinding both, ch13_reset_dsu seeding singletons (source: icpc/samples-c/src/Ch13/ch13_moat.h, lines 106-149)

```c
static inline int ch13_rb_find(const int *par, int x) {
  while (par[x] != x)
    x = par[x];
  return x;
}

static inline int ch13_rb_unite(int *par, int *sz, int *log, int *top, int x,
                                int y) {
  int rx = ch13_rb_find(par, x), ry = ch13_rb_find(par, y), t;
  if (rx == ry) {
    log[(*top)++] = -1;
    log[(*top)++] = -1;
    return rx;
  }
  if (sz[rx] > sz[ry]) {
    t = rx;
    rx = ry;
    ry = t;
  }
  par[rx] = ry;
  sz[ry] += sz[rx];
  log[(*top)++] = rx;
  log[(*top)++] = ry;
  return ry;
}

static inline void ch13_rb_undo(int *par, int *sz, const int *log, int *top) {
  *top -= 2;
  int p = log[*top + 1], c = log[*top];
  if (c >= 0) {
    par[c] = c;
    sz[p] -= sz[c];
  }
}
```

listing 604: c, the dsu half, size-indexed unite logging each child, parent pair with −1 sentinels, undo popping exactly one pair (source: icpc/samples-c/src/Ch13/ch13_rollback.h, lines 9-42)

```go
func (m *moat13) addSeg(id int) {
  c, p := m.unite(m.su[id], m.sv[id])
  if c < 0 {
    return
  }
  m.ppay = append(m.ppay, p)
  m.paya = append(m.paya, m.ca[p])
  m.payb = append(m.payb, m.cb[p])
  m.payf = append(m.payf, m.cfl[p])
  if m.cfl[c]&3 == 3 {
    m.lopRemove(c)
  }
  wasp := m.cfl[p]&3 == 3
  m.ca[p] += m.ca[c] + m.sa[id]
  m.cb[p] += m.cb[c] + m.sb[id]
  m.cfl[p] |= m.cfl[c]
  if !wasp && m.cfl[p]&3 == 3 {
    m.lopInsert(p)
  }
}

func (m *moat13) undoUnions(count int) {
  for k := 0; k < count; k++ {
    if m.rblog[len(m.rblog)-2] ≥ 0 {
      top := len(m.ppay) - 1
      p := m.ppay[top]
      m.ca[p] = m.paya[top]
      m.cb[p] = m.payb[top]
      m.cfl[p] = m.payf[top]
      m.ppay = m.ppay[:top]
      m.paya = m.paya[:top]
      m.payb = m.payb[:top]
      m.payf = m.payf[:top]
    }
    m.undoUnion()
  }
}
```

listing 605: go, addSeg saving the parent payload before the merge, undoUnions restoring it beside each logged union (source: icpc/samples-go/ch13/moat.go, lines 155-191)

```
  int find(int x) {
    while (par[x] ≠ x)
      x = par[x];
    return x;
  }

  // returns the surviving root
  int unite(int x, int y) {
    if (top + 2 > log.length)
      log = java.util.Arrays.copyOf(log, log.length * 2);
    int rx = find(x), ry = find(y);
    if (rx == ry) {
      log[top++] = -1;
      log[top++] = -1;
      return rx;
    }
    if (sz[rx] > sz[ry]) {
      int t = rx;
      rx = ry;
      ry = t;
    }
    par[rx] = ry;
    sz[ry] += sz[rx];
    log[top++] = rx;
    log[top++] = ry;
    return ry;
  }

  void undo() {
    top -= 2;
    int p = log[top + 1], c = log[top];
    if (c ≥ 0) {
      par[c] = c;
      sz[p] -= sz[c];
    }
  }
```

listing 606: java, the dsu half as the year's one java twin beside the engine, unite logging each child, parent pair with −1 sentinels, undo popping exactly one pair (source: icpc/samples-java/src/Ch14/Rollback.java, lines 25-60)

```
    internal static void AddSeg(int id)
    {
        int x = Su[id], y = Sv[id];
        int rx = Find(x), ry = Find(y);
        int c = -1, p = -1;
        if (rx != ry)
        {
            if (Dsz[rx] > Dsz[ry])
                (rx, ry) = (ry, rx);
            Par[rx] = ry;
            Dsz[ry] += Dsz[rx];
            Rblog[_rbtop++] = rx;
            Rblog[_rbtop++] = ry;
            c = rx;
            p = ry;
        }
        else
        {
            Rblog[_rbtop++] = -1;
            Rblog[_rbtop++] = -1;
        }
        if (c < 0)
            return;
        Ppay[_paytop] = p;
        PayA[_paytop] = CA[p];
        PayB[_paytop] = CB[p];
        PayF[_paytop] = Cfl[p];
        _paytop++;
        if ((Cfl[c] & 3) == 3)
            LopRemove(c);
        bool wasp = (Cfl[p] & 3) == 3;
        CA[p] += CA[c] + Sa[id];
        CB[p] += CB[c] + Sb[id];
        Cfl[p] |= Cfl[c];
        if (!wasp && (Cfl[p] & 3) == 3)
            LopInsert(p);
    }
```

listing 607: c#, AddSeg with the unite, payload stash, and both-border list splice inlined, lifo against Rblog and Ppay (source: icpc/samples/src/Ch13/Moat.cs, lines 143-180)

```
export function addSeg(id) {
  // unite SU[id], SV[id] with rollback logging
  let rx = rbFind(SU[id]), ry = rbFind(SV[id]);
  if (rx === ry) {
    rblog[rbtop++] = -1;
    rblog[rbtop++] = -1;
    return;
  }
  if (dsz[rx] > dsz[ry]) [rx, ry] = [ry, rx];
  par[rx] = ry;
  dsz[ry] += dsz[rx];
  rblog[rbtop++] = rx;
  rblog[rbtop++] = ry;
  const c = rx, p = ry;
  ppay[paytop] = p;
  payA[paytop] = cA[p];
  payB[paytop] = cB[p];
  payF[paytop] = cfl[p];
  paytop++;
  if ((cfl[c] & 3) === 3) lopRemove(c);
  const wasp = (cfl[p] & 3) === 3;
  cA[p] += cA[c] + SA[id];
  cB[p] += cB[c] + SB[id];
  cfl[p] = cfl[p] | cfl[c];
  if (!wasp && (cfl[p] & 3) === 3) lopInsert(p);
}

export function undoUnions(count) {
  for (let k = 0; k < count; k++) {
    if (rblog[rbtop - 2] ≥ 0) {
      paytop--;
      cA[ppay[paytop]] = payA[paytop];
      cB[ppay[paytop]] = payB[paytop];
      cfl[ppay[paytop]] = payF[paytop];
    }
    rbtop -= 2; // each unite pushed one (child, parent) pair
    const p = rblog[rbtop + 1], c = rblog[rbtop];
    if (c ≥ 0) {
      par[c] = c;
      dsz[p] -= dsz[c];
    }
  }
}
```

listing 608: javascript, addSeg over module-static typed arrays with the unite inlined, undoUnions unwinding payloads and roots in lockstep (source: icpc/samples-js/src/ch13-moat.mjs, lines 100-142)

```
    def add_seg(self, sid):
        c, p = self.unite(self.su[sid], self.sv[sid])
        if c < 0:
            return
        self.ppay.append(p)
        self.paya.append(self.ca[p])
        self.payb.append(self.cb[p])
        self.payf.append(self.cfl[p])
        if self.cfl[c] & 3 == 3:
            self.lop_remove(c)
        wasp = self.cfl[p] & 3 == 3
        self.ca[p] += self.ca[c] + self.sa[sid]
        self.cb[p] += self.cb[c] + self.sb[sid]
        self.cfl[p] |= self.cfl[c]
        if not wasp and self.cfl[p] & 3 == 3:
            self.lop_insert(p)

    def undo_unions(self, count):
        for _ in range(count):
            if self.rblog[-2] ≥ 0:
                top = len(self.ppay) - 1
                p = self.ppay[top]
                self.ca[p] = self.paya[top]
                self.cb[p] = self.payb[top]
                self.cfl[p] = self.payf[top]
                del self.ppay[top:]
                del self.paya[top:]
                del self.payb[top:]
                del self.payf[top:]
            self.undo_union()
```

listing 609: python, the Moat class, add_seg stashing the parent payload, undo_unions restoring it beside each logged union (source: icpc/samples-py/src/Ch13/ch13_rollback.py, lines 112-141)

```lua
function moat.add_seg(m, id)
  local p = rb.unite(m.par, m.dsz, m.rblog, m.SU[id], m.SV[id])
  local c = m.rblog[#m.rblog - 1]
  if c < 0 then
    return
  end
  local pt = #m.payp + 1
  m.payp[pt] = p
  m.payA[pt] = m.cA[p]
  m.payB[pt] = m.cB[p]
  m.payF[pt] = m.cfl[p]
  if (m.cfl[c] & 3) == 3 then
    lop_remove(m, c)
  end
  local wasp = (m.cfl[p] & 3) == 3
  m.cA[p] = m.cA[p] + m.cA[c] + m.SA[id]
  m.cB[p] = m.cB[p] + m.cB[c] + m.SB[id]
  m.cfl[p] = m.cfl[p] | m.cfl[c]
  if not wasp and (m.cfl[p] & 3) == 3 then
    lop_insert(m, p)
  end
end

function moat.undo_unions(m, count)
  for _ = 1, count do
    if m.rblog[#m.rblog - 1] ≥ 0 then
      local pt = #m.payp
      m.cA[m.payp[pt]] = m.payA[pt]
      m.cB[m.payp[pt]] = m.payB[pt]
      m.cfl[m.payp[pt]] = m.payF[pt]
      m.payp[pt] = nil
      m.payA[pt] = nil
      m.payB[pt] = nil
      m.payF[pt] = nil
    end
    rb.undo(m.par, m.dsz, m.rblog)
  end
end
```

listing 610: lua, moat.add_seg through the rollback module's unite, moat.undo_unions restoring payloads and roots in lifo order (source: icpc/samples-lua/ch13_moat.lua, lines 114-151)

```lua
-- ch13 year-local size-indexed union-find with lifo rollback, used
-- by 2025 G for the offline dynamic connectivity level sweep. every
-- unite pushes exactly one (child, parent) pair onto the log (-1
-- sentinels when the nodes already share a root) so unwinding stays
-- paired with the caller's own payload restoration.
local rb = {}

function rb.find(par, x)
  while par[x] ~= x do
    x = par[x]
  end
  return x
end

-- returns the surviving root; pushes child, parent (or -1, -1)
function rb.unite(par, dsz, log, x, y)
  local rx = rb.find(par, x)
  local ry = rb.find(par, y)
  if rx == ry then
    log[#log + 1] = -1
    log[#log + 1] = -1
    return rx
  end
  if dsz[rx] > dsz[ry] then
    rx, ry = ry, rx
  end
  par[rx] = ry
  dsz[ry] = dsz[ry] + dsz[rx]
  log[#log + 1] = rx
  log[#log + 1] = ry
  return ry
end

function rb.undo(par, dsz, log)
  local p = log[#log]
  log[#log] = nil
  local c = log[#log]
  log[#log] = nil
  if c ≥ 0 then
    par[c] = c
    dsz[p] = dsz[p] - dsz[c]
  end
end

return rb
```

listing 611: lua, rb.find, rb.unite pushing child and parent onto the log, rb.undo popping one pair (source: icpc/samples-lua/ch13_rollback.lua, lines 1-45)

The wheels kit backs problem I with the slot-machine friend: n wheels over one shared cyclic symbol order, `init` normalizing positions mod n, `rotate` adding an offset mod n and counting queries, and `k` answering the count of distinct visible symbols, the judge's only feedback. C's `Ch13/ch13_wheels.h`, go's `wheels13`, javascript's `Wheels` class, python's `Wheels` class, and lua's `wheels` module all print whole, and C# inlines the simulator as the sealed class its problem I section already shows. Java ships no twin at all: the simulator lives nested inside PI.java as a static Wheels class, so the interactor shim compiles against the same file in the harness's single javac pass:

```c
typedef struct {
  int n;
  int pos[52];
  long long queries;
} Ch13Wheels;

static inline void ch13_wheels_init(Ch13Wheels *w, int n, const int *pos) {
  w->n = n;
  w->queries = 0;
  for (int i = 0; i < n; i++)
    w->pos[i] = ((pos[i] - 1) % n + n) % n + 1;
}

static inline int ch13_wheels_k(const Ch13Wheels *w) {
  int seen[52] = {0}, k = 0;
  for (int i = 0; i < w->n; i++) {
    int s = (w->pos[i] - 1) % w->n;
    if (!seen[s]) {
      seen[s] = 1;
      k++;
    }
  }
  return k;
}

static inline int ch13_wheels_rotate(Ch13Wheels *w, int i, int j) {
  int n = w->n;
  w->pos[i - 1] = ((w->pos[i - 1] - 1 + j) % n + n) % n + 1;
  w->queries++;
  return ch13_wheels_k(w);
}
```

listing 612: c, the Ch13Wheels friend, init normalizing positions mod n, k counting distinct symbols, rotate answering through k (source: icpc/samples-c/src/Ch13/ch13_wheels.h, lines 8-38)

```go
func newWheels13(n int, pos []int) *wheels13 {
  w := &wheels13{n: n, pos: make([]int, n)}
  for i := 0; i < n; i++ {
    w.pos[i] = ((pos[i]-1)%n+n)%n + 1
  }
  return w
}

func (w *wheels13) k() int {
  seen := make([]bool, w.n+1)
  k := 0
  for i := 0; i < w.n; i++ {
    s := (w.pos[i] - 1) % w.n
    if !seen[s] {
      seen[s] = true
      k++
    }
  }
  return k
}

func (w *wheels13) rotate(i, j int) int {
  n := w.n
  w.pos[i-1] = ((w.pos[i-1]-1+j)%n+n)%n + 1
  w.queries++
  return w.k()
}
```

listing 613: go, wheels13, newWheels13 normalizing, k over a fresh seen slice, rotate answering through k (source: icpc/samples-go/ch13/wheels.go, lines 13-39)

```javascript
export class Wheels {
  constructor(n, pos) {
    this.n = n;
    this.pos = pos.map((p) => (((p - 1) % n) + n) % n + 1);
    this.queries = 0;
  }

  k() {
    const seen = new Uint8Array(this.n + 1);
    let k = 0;
    for (let i = 0; i < this.n; i++) {
      const s = (this.pos[i] - 1) % this.n;
      if (!seen[s]) {
        seen[s] = 1;
        k++;
      }
    }
    return k;
  }

  rotate(i, j) {
    const n = this.n;
    this.pos[i - 1] = (((this.pos[i - 1] - 1 + j) % n) + n) % n + 1;
    this.queries++;
    return this.k();
  }
}
```

listing 614: javascript, the Wheels class, k over a Uint8Array of seen symbols, rotate adding j mod n (source: icpc/samples-js/src/ch13-wheels.mjs, lines 5-31)

```python
class Wheels:
    def __init__(self, n, pos):
        self.n = n
        self.pos = [((p - 1) % n + n) % n + 1 for p in pos]
        self.queries = 0

    def k(self):
        seen = set()
        for p in self.pos:
            seen.add((p - 1) % self.n)
        return len(seen)

    def rotate(self, i, j):
        n = self.n
        self.pos[i - 1] = ((self.pos[i - 1] - 1 + j) % n + n) % n + 1
        self.queries += 1
        return self.k()
```

listing 615: python, the Wheels class, k as a set of normalized symbols, rotate returning the new count (source: icpc/samples-py/src/Ch13/ch13_wheels.py, lines 7-23)

```lua
function wheels.init(n, pos)
  local w = {n = n, pos = {}, queries = 0}
  for i = 1, n do
    w.pos[i] = (pos[i] - 1) % n + 1
  end
  return w
end

function wheels.k(w)
  local seen = {}
  local k = 0
  for i = 1, w.n do
    local s = (w.pos[i] - 1) % w.n
    if not seen[s] then
      seen[s] = true
      k = k + 1
    end
  end
  return k
end

function wheels.rotate(w, i, j)
  local n = w.n
  w.pos[i] = (w.pos[i] - 1 + j) % n + 1
  w.queries = w.queries + 1
  return wheels.k(w)
end
```

listing 616: lua, wheels.init, wheels.k, wheels.rotate, 1-based positions held mod n (source: icpc/samples-lua/ch13_wheels.lua, lines 7-33)

15.2. the go scan cursor

The scan kit is the go package's io contract, one file the whole year shares: `scan13` walks the input buffer with a whitespace `token`, a signed `nextInt`, its 64-bit twin `nextInt64`, and `line` for the fixed-line judge files, the same cursor design chapter 14 printed as scan12. All twelve go solvers parse through it:

```go
// scan13 is the year-13 whitespace tokenizer over the whole input.
type scan13 struct {
  data []byte
  pos  int
}

func blank13(c byte) bool {
  return c == ' ' || c == '\n' || c == '\r' || c == '\t'
}

func (s *scan13) token() []byte {
  for s.pos < len(s.data) && blank13(s.data[s.pos]) {
    s.pos++
  }
  start := s.pos
  for s.pos < len(s.data) && !blank13(s.data[s.pos]) {
    s.pos++
  }
  return s.data[start:s.pos]
}

func (s *scan13) nextInt() int {
  tok := s.token()
  neg := false
  i := 0
  if i < len(tok) && (tok[i] == '-' || tok[i] == '+') {
    neg = tok[i] == '-'
    i++
  }
  v := 0
  for ; i < len(tok); i++ {
    v = v*10 + int(tok[i]-'0')
  }
  if neg {
    return -v
  }
  return v
}
```

listing 617: go, scan13: token skips whitespace over the shared buffer, nextInt folds a signed value, nextInt64 the same in 64 bits (source: icpc/samples-go/ch13/ch13.go, lines 5-42)

Java again ships no shared cursor: each solver reads the whole input through a BufferedReader into one string and walks a private index scanner, sign-aware exactly in G, K, and L where coordinates and depths run negative, one System.out.print at the end.

15.3. problem a, a-skew-ed reasoning

A skew heap is a mergeable heap that grows by insertion, and this task runs its construction backwards. The heap starts empty and a permutation of 1..n is fed in one value at a time. Inserting x into a heap rooted at y swaps the root's children and descends left when y < x, and makes x the new root with the old heap as its left child when y > x. The input then hands over a binary tree on the values 1..n, described as children lists rather than as a permutation, and the task is to name the lexicographically minimal and the lexicographically maximal insertion permutations that produce exactly that tree, or declare it impossible.

The input is one integer n, then n lines of two integers each, the left and right child of node i, 0 meaning no child, with every child index either 0 or a node numbered above i so the lists always describe a binary tree. The

problemset pdf bounds n at 2e5 with the values 1..n used exactly once, under a 2 second limit, and the judge data carries 52 cases for the letter, 3 samples and 49 secrets. The output is two lines of n space-separated integers, minimal first then maximal, or the single word impossible.

Official sample 1, reprinted byte for byte from the judge data pair `A-askewedreasoning/sample-1.in` and `sample-1.ans`: the tree gives node 1 the children 2 and 3 and gives both of them two leaf children, a full binary tree of depth two, and the two extremes split on where the root value 1 sits, first position for the minimum and second for the maximum:

input:

```
7
2 3
4 5
6 7
0 0
0 0
0 0
0 0
```

expected output:

```
1 3 2 7 5 6 4
7 1 5 3 2 6 4
```

Official sample 2, same source: a two-node tree whose root owns only a right child, and since a fresh root always takes the old heap on its left, no permutation builds it.

input:

```
2
0 2
0 0
```

expected output:

```
impossible
```

Recognition: n at 2e5 under the 2 second limit buys about 1e8 cheap operations, and the reconstruction needs only n log n of them, each node's interleave touching nothing but the shorter side, about 3.6e6 element moves for the whole build.

The statement prints the growth rule in full, inserting x swaps the root's children and descends left when y < x and re-roots when y > x, then hands over the finished tree and asks for the lexicographically minimal and the maximal permutation building exactly it: a deterministic forward process identified by its final state, the shape to run backwards. Problem A is the 2025 face of the inverting a deterministic process family. Chapter 9 problem G is the other half of the inverting a deterministic process pair.

Searching forward is the priced-out alternative: the candidate permutations run to (2e5)!, and even a backward splice that walks the long side at every node costs (2e5)^2 = 4e10 moves against the limit, four orders past it.

The reconstruction recurses from the root. The root value 1 lands at some input position q, the prefix before it builds one side's base heap, and because every later insertion swaps the root's children before descending left, the post-1 suffix alternates between the sides at the rate of one element each, the parity of its length fixing which child ends up left and the pre-1 prefix having built the even-offset side's base. The two subtree sizes pin q to at most two candidates, q = size - 2*ls + 1 when the odd-offset side is the left child of size ls, and q = size - 2*rs otherwise, two

candidates only when they land on the first two positions of the node's span, and min takes the smaller feasible q at every node while max takes the larger (statement, solutions.pdf p. 1). Combining a node with its two built sides is an interleave of the shorter sequence into an equal-length suffix of the longer one, the mergesort merge, so the whole build runs in O(n log n), the small side's cost charged each time an element lands there.

The worked run: trace the model on sample 1. The input's tree gives node 1 the children 2 and 3 and both of them two leaf children, so the span is 7 over two subtrees of size 3, and the root candidates are q = 7 - 2 × 3 + 1 = 2 and q = 7 - 2 × 3 = 1, the span's first two positions, min taking 1 and max taking 2. Min, the line 1 3 2 7 5 6 4: the empty prefix leaves both bases empty, the suffix 3, 2, 7, 5, 6, 4 alternates one element per side, dealing 3, 7, and 6 to node 3 and 2, 5, and 4 to node 2, and the leaf recursion below each repeats the split, node 3′s triple 3, 7, 6 taking q = 1 so 7 then 6 alternate onto its leaves as 6 left and 7 right, node 2′s triple 2, 5, 4 settling 4 left and 5 right, both exactly the input's children lists. Max, the line 7 1 5 3 2 6 4: q = 2, the one-element prefix 7 pre-building node 3′s base, the suffix 5, 3, 2, 6, 4 dealing 3 and 6 to node 3 beside that base and 5, 2, and 4 to node 2, and the same leaf recursions end at node 3 with children 6 and 7, the base 7 nested right, and node 2 with children 4 and 5, the input's tree again, and the trace ends at the printed answer `7 1 5 3 2 6 4`.

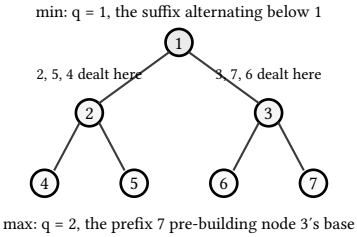


figure 173: the sample-1 tree, node 1 owning 2 and 3 over four leaves, with the min line's root-first split and the max line's one-element prefix annotated

```c
// build the min (mx=0) or max (mx=1) insertion sequence into out
static int build(int *out, int mx) {
  int cnt = 0, top = 0;
  stk_u[top] = 1;
  stk_o[top++] = 0;
  while (top) {
    int u = stk_u[--top], off = stk_o[top];
    int q = pick_q(u, mx);
    if (q < 0)
      return 0;
    order_[cnt] = u;
    base_[u] = off;
    cnt++;
    int s = sz[u] - 1 - (q - 1);
    int evenn = (q - 1) + s / 2;
    int oddT = s % 2 ? lc[u] : rc[u]; // x2 side ends left iff s odd
    int evenT = s % 2 ? rc[u] : lc[u];
    if (oddT) {
      stk_u[top] = oddT;
      stk_o[top++] = off + evenn;
    }
    if (evenT) {
      stk_u[top] = evenT;
      stk_o[top++] = off;
    }
  }
  for (int i = cnt - 1; i ≥ 0; i--) {
    int u = order_[i], off = base_[u];
    int q = pick_q(u, mx);
    int p = q - 1, s = sz[u] - 1 - p;
    int oddn = (s + 1) / 2, half = s / 2, evenn = p + half;
    int *o = out + off;
    // o: [E[0..p-1] | E-tail | O[0..oddn-1]]. shuffle to
    // E[0..p-1], u, O[0], E[p], O[1], E[p+1], ...
    memcpy(temp, o + p, (size_t)half * sizeof(int));
    memcpy(temp + half, o + evenn, (size_t)oddn * sizeof(int));
    for (int k = 0; k < s; k++)
      o[p + 1 + k] = k % 2 ? temp[k / 2] : temp[half + k / 2];
    o[p] = u;
  }
  return 1;
}
```

listing 618: c, build in two passes, parity forcing the odd-offset side, then one in-place shuffle per node through a shared temp (source: icpc/samples-c/src/Ch13/pA.c, lines 74-115)

The c family writes each node's even side at the head of its output buffer and its odd side right after it, so the alternation is one memcpy pair plus a parity-indexed shuffle, and the reverse preorder makes children settle before the parent splices itself in.

The go, javascript, and python engines splice linked lists instead. Each node walks its base child's list to position q-1 from the nearer end, so the walk costs at most the fresh side's length, and that from-the-nearer-end rule is the small-into-large bound made visible:

```
    for i := len(order) - 1; i ≥ 0; i-- {
      u := order[i]
      l, r := lch[u], rch[u]
      if l == 0 && r == 0 {
        head[u], tail[u] = u, u
        continue
      }
      c1 := l ≠ 0 && size[r]+1 ≥ size[l]
      q1 := size[r] + 2 - size[l]
      c2 := size[l] ≥ size[r]
      q2 := size[l] + 1 - size[r]
      pick1 := false
      switch {
      case c1 && c2:
        pick1 = (q1 < q2) ≠ maxMode
      case c1:
        pick1 = true
      case c2:
      default:
        return nil
      }
      var fresh, base, q int32
      if pick1 {
        fresh, base, q = l, r, q1
      } else {
        fresh, base, q = r, l, q2
      }
      plen := q - 1
      // p is the base node at 1-based position plen (0 = before
      // the head), bs the next base node; walk from the nearer
      // end so the walk costs at most the fresh length
      var p, bs int32
      if 2*plen <= size[base] {
        bs = head[base]
        for j := int32(0); j < plen; j++ {
          p = bs
          bs = nxt[bs]
        }
      } else {
        p = tail[base]
        for j := size[base]; j > plen; j-- {
          p = prv[p]
        }
        bs = nxt[p]
      }
```

listing 619: go, both q candidates scored, min or max picked, then the base walk started from whichever end is closer (source: icpc/samples-go/ch13/pa.go, lines 72-116)

Java rejoins the c family, buffer and shuffle, both passes carried by explicit stacks because judge data grows 2e5-deep combs:

```
// build the min or max insertion sequence into out; false when a
// node admits no feasible q
static boolean build(int[] out, int n, boolean max) {
  int cnt = 0, top = 0;
  stackU[top] = 1;
  stackO[top++] = 0;
  while (top > 0) {
    int u = stackU[--top], off = stackO[top];
    int q = pickQ(u, max);
    if (q < 0)
      return false;
    order[cnt] = u;
    base[u] = off;
    cnt++;
    int s = sz[u] - 1 - (q - 1);
    int evenn = (q - 1) + s / 2;
    int oddT = s % 2 == 1 ? lc[u] : rc[u]; // x2 side ends left iff s odd
    int evenT = s % 2 == 1 ? rc[u] : lc[u];
    if (oddT != 0) {
      stackU[top] = oddT;
      stackO[top++] = off + evenn;
    }
    if (evenT != 0) {
      stackU[top] = evenT;
      stackO[top++] = off;
    }
  }
  for (int i = cnt - 1; i >= 0; i--) {
    int u = order[i], off = base[u];
    int q = pickQ(u, max);
    int p = q - 1, s = sz[u] - 1 - p;
    int oddn = (s + 1) / 2, half = s / 2, evenn = p + half;
    // o: [E[0..p-1] | E-tail | O[0..oddn-1]]. shuffle to
    // E[0..p-1], u, O[0], E[p], O[1], E[p+1], ...
    System.arraycopy(out, off + p, temp, 0, half);
    System.arraycopy(out, off + evenn, temp, half, oddn);
    for (int k = 0; k < s; k++)
      out[off + p + 1 + k] = k % 2 == 1 ? temp[k / 2] : temp[half + k / 2];
    out[off + p] = u;
  }
  return true;
}
```

listing 620: java, the same build, two explicit-stack passes over one out array, the reverse-preorder shuffle through a shared temp (source: icpc/samples-java/src/Ch14/PA.java, lines 82-123)

The c# port is the same algorithm over Array.Copy:

```
    // build the min (mx=false) or max (mx=true) insertion sequence
    private static bool Build(int[] o, bool mx)
    {
        int cnt = 0, top = 0;
        _stkU[top] = 1;
        _stkO[top++] = 0;
        while (top > 0)
        {
            int u = _stkU[--top], off = _stkO[top];
            int q = PickQ(u, mx);
            if (q < 0)
                return false;
            _order[cnt] = u;
            _base[u] = off;
            cnt++;
            int s = _sz[u] - 1 - (q - 1);
            int evenn = (q - 1) + s / 2;
            // the x2 side ends left iff s is odd
            int oddT = s % 2 != 0 ? _lc[u] : _rc[u];
            int evenT = s % 2 != 0 ? _rc[u] : _lc[u];
            if (oddT != 0)
            {
                _stkU[top] = oddT;
                _stkO[top++] = off + evenn;
            }
            if (evenT != 0)
            {
                _stkU[top] = evenT;
                _stkO[top++] = off;
            }
        }
        for (int i = cnt - 1; i >= 0; i--)
        {
            int u = _order[i], off = _base[u];
            int q = PickQ(u, mx);
            int p = q - 1, s = _sz[u] - 1 - p;
            int oddn = (s + 1) / 2, half = s / 2, evenn = p + half;
            // o: [E[0..p-1] | E-tail | O[0..oddn-1]]. shuffle to
            // E[0..p-1], u, O[0], E[p], O[1], E[p+1], ...
            Array.Copy(o, off + p, _temp, 0, half);
            Array.Copy(o, off + evenn, _temp, half, oddn);
            for (int k = 0; k < s; k++)
                o[off + p + 1 + k] = k % 2 != 0 ? _temp[k / 2]
                    : _temp[half + k / 2];
            o[off + p] = u;
        }
        return true;
    }
```

listing 621: c#, the same build, Array.Copy standing in for memcpy, min and max one bool apart (source: icpc/samples/src/Ch13/PA.cs, lines 97-144)

The javascript and python entries splice the same lists:

```
      const plen = q - 1;
      // p is the base node at 1-based position plen (0 = before the
      // head), bs the next base node; walk from the nearer end so the
      // walk costs at most the fresh length
      let p = 0, bs = 0;
      if (2 * plen ≤ size[base]) {
        bs = head[base];
        for (let j = 0; j < plen; j++) {
          p = bs;
          bs = nxt[bs];
        }
      } else {
        p = tail[base];
        for (let j = size[base]; j > plen; j--) p = prv[p];
        bs = nxt[p];
      }
      // splice root then alternate fresh nodes with the base suffix
      // nodes already in place
      let prev = p;
      const ap = (x) => {
        if (prev) nxt[prev] = x;
        prv[x] = prev;
        prev = x;
      };
      ap(u);
      let f = head[fresh];
      while (f || bs) {
        if (f) {
          const fn = nxt[f];
          ap(f);
          f = fn;
        }
        if (bs) {
          const bn = nxt[bs];
          ap(bs);
          bs = bn;
        }
      }
      nxt[prev] = 0;
      head[u] = p ? head[base] : u;
      tail[u] = prev;
    }
    const out = [];
    for (let v = head[1]; v; v = nxt[v]) out.push(v);
    return out;
  };
```

listing 622: javascript, the splice itself, root appended after the base prefix, fresh and base-suffix nodes alternating onto one prev pointer (source: icpc/samples-js/src/ch13-pa-askewedreasoning.mjs, lines 80-125)

```
            if pick1:
                fresh, base, q = le, r, q1
            else:
                fresh, base, q = r, le, q2
            plen = q - 1
            # pv is the base node at 1-based position plen (0 = before
            # the head), bs the next base node; walk from the nearer
            # end so the walk costs at most the fresh length
            if 2 * plen ≤ size[base]:
                bs = head[base]
                pv = 0
                for _ in range(plen):
                    pv = bs
                    bs = nxt[bs]
            else:
                pv = tail[base]
                for _ in range(size[base] - plen):
                    pv = prv[pv]
                bs = nxt[pv]
            # splice root then alternate fresh nodes with the base
            # suffix nodes already in place, and rewire in one pass
            out = [u]
            f = head[fresh]
            while f or bs:
                if f:
                    out.append(f)
                    f = nxt[f]
                if bs:
                    out.append(bs)
                    bs = nxt[bs]
            if pv:
                nxt[pv] = out[0]
                prv[out[0]] = pv
                head[u] = head[base]
            else:
                head[u] = out[0]
            for a, b in zip(out, out[1:]):
                nxt[a] = b
                prv[b] = a
            nxt[out[-1]] = 0
            tail[u] = out[-1]
```

listing 623: python, the same splice with the alternation collected into one out list, then rewired in a single zip pass (source: icpc/samples-py/src/Ch13/pa.py, lines 76-116)

Lua rejoins the c family, buffer and shuffle, with floor division where c shifts:

```lua
  local function pick_q(u, mx)
    local ls, rs, total = sz[lc[u]], sz[rc[u]], sz[u]
    local qOdd = ls ≥ 1 and 2 * ls ≤ total and total - 2 * ls + 1 or -1
    local qEven = 2 * rs ≤ total - 1 and total - 2 * rs or -1
    if qOdd < 0 and qEven < 0 then
      return -1
    elseif qOdd < 0 then
      return qEven
    elseif qEven < 0 then
      return qOdd
    elseif mx then
      return qOdd > qEven and qOdd or qEven
    end
    return qOdd < qEven and qOdd or qEven
  end
  -- build the min (mx false) or max (mx true) insertion sequence
  local function build(out, mx)
    local cnt = 0
    local top = 1
    local stk = {1}
    stk[n + 2] = 0 -- offset slot paired with the root at slot 1
    local base = {}
    while top > 0 do
      local u = stk[top]
      local off = stk[top + n + 1]
      top = top - 1
      local q = pick_q(u, mx)
      if q < 0 then return false end
      cnt = cnt + 1
      order[cnt] = u
      base[u] = off
      local s = sz[u] - 1 - (q - 1)
      local evenn = (q - 1) + s // 2
      local oddT = s % 2 == 1 and lc[u] or rc[u]
      local evenT = s % 2 == 1 and rc[u] or lc[u]
      if oddT ~= 0 then
        top = top + 1
        stk[top] = oddT
        stk[top + n + 1] = off + evenn
      end
      if evenT ~= 0 then
        top = top + 1
        stk[top] = evenT
        stk[top + n + 1] = off
      end
    end
```

listing 624: lua, pick_q over the two candidates, then the placement pass stamping each node's offset before the reverse shuffle (source: icpc/samples-lua/ch13_pa.lua, lines 67-112)

The crafted fixture is the four-node tree 1(L:3, R:2(L:4)). The side sizes force q = 3, the prefix {2,4} builds 2(L:4) in either order, and the single suffix element 3 swings into the left child after the root swap, so the answer is "2 4 1 3" then "4 2 1 3". Every suite pins that pair: c, java, javascript, python, and lua carry the two-fixture plan with the single-node tree min = max = 1 as the extra, while c# adds the impossible lone right child, whose fresh root would always take the old heap on the left, and go adds a three-node left comb plus an exhaustive sweep that brute-forces every permutation for n up to 6 and checks the solver against the min and max it produces. Per language

this section pins 2 checks in c, 3 facts in c#, 3 table cases plus the sweep in go, 2 ok checks in java, 2 assertions in javascript, 2 checks in python, and 2 named checks in lua.

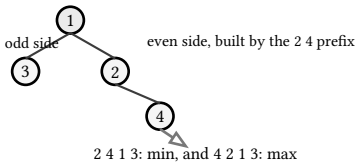


figure 174: the crafted fixture: the prefix 2, 4 builds the right chain, the root 1 lands third, and the suffix 3 swings into the left child after the child swap

Integers stay small here, indices up to 2e5 in int32 with plain prints, and the output is two lines of n numbers, so no language needs anything beyond its default integer.

15.4. problem b, blackboard game

Numbers 1 through n are written on a blackboard and two players share one circle. The first player circles any even number, and every later move must jump the circle from its current number to a never-circled number that is the current number times a prime or divided by a prime. The player unable to move loses. Per n the output names the winner: “second” outright, or “first” together with any even opening move that belongs to a winning strategy.

The input is one integer t, then t lines each holding one n. The problemset pdf bounds t at 40, n between 2 and 1e7, and the sum of n across the tests at 1e7, under a 1 second limit. The judge data carries 118 cases for the letter, 2 samples and 116 secrets. The output is one line per test, “second”, or “first k” with k even, and every valid winning k scores alike.

The sample pair cannot be reprinted the usual way, because this is the set’s first structural exception: the judge directory holds the two sample inputs, n = 5 alone and n = 12 beside n = 17, and not one .ans file across the letter’s 118 cases, so the chapter asserts the emptiness rather than quoting a pair. Any valid winning opening scores under the official judge, so the letter judges through `checker:2025-b` instead of a byte diff: below a boundary of 2′000 the checker recomputes the criterion exactly, one hopcroft-karp maximum matching plus alternating reachability out of the exposed vertices, and above it the pdf’s certified opening decides, first 2p with p a prime above n/4 and at most n/3. Only the exact tier accepts an arbitrary winning opening, the certified tier validating family openings with every accepted opening genuinely winning, and the six solvers emit family openings above their own cutoff, so the lanes judge their own outputs.

Recognition: t at 40 with the sum of n capped at 1e7 against the 1 second limit leaves no room for per-n work above the engine’s cutoff, and below it the retry machine stays small: n = 200 builds about 370 prime-multiple edges, one kuhn pass costs on the order of 200 × 370 ≈ 7e4 augment steps, and the at most 100 even deletions keep a whole verdict under 1e7 steps.

The cue sits in the win condition, any even opening move that belongs to a winning strategy: an opening wins exactly when every opponent reply can be met with a matched partner forever, a matching-avoidance question on the prime-multiple graph. Problem B is the 2025 face of the bipartite assignment feasibility family. The bipartite assignment feasibility family collects chapter 10 problem C, chapter 13 problem X, and chapter 15 problem F. A secondary thread, semiprime certificate for large n, covers the primality test the opening move reduces to.

Game-tree search on the move graph is the priced-out alternative, 2^(1e7) positions at the cap, and even memoized retrograde analysis must hold the whole graph in memory where one maximum matching per deletion answers outright.

Build the undirected graph on 1..n joining x to x*p for primes p. An even k is a winning opening exactly when some maximum matching of that graph avoids k, because the first player then answers every opponent move with the matched partner forever and the opponent can never reach an unmatched vertex without exhibiting an augmenting path, while if every maximum matching uses all even numbers the second player mirrors. Small

n runs kuhn over the parity-of-omega bipartition trying each even deletion, the graph being bipartite because every edge changes the count of prime factors by exactly one. Past the cutoff the answer is always “first 2p”: the solutions pdf proves first wins for every n above 176, generalized Bertrand supplies distinct primes p, q, and r above n/4 and at most n/3, opening 2p leaves the six semiprimes 2p, 3p, 2q, 3q, 2r, and 3r matching perfectly onto 2, 3, p, q, and r, and the book cuts over at n = 200, the pdf noting any cutoff between about 200 and 1e5 serves (statement, solutions.pdf pp. 2-3). Book 10′s chapter 38, network flows ii, builds kuhn’s augmenting-path matching that the small-n engine runs here. Book 10′s chapter 40, game theory and scheduling, collects the games-on-graphs strategy arguments the matching criterion instantiates. Book 10′s chapter 28, advanced number theory, carries the primality testing behind the Bertrand prime hunt, and the c engine opens the walkthrough.

The worked run: trace the model on sample 1, whose judge directory prints no answers at all because any valid winning opening scores alike, so the trace targets the verdict the matching criterion pins. Sample 1 is n = 5: the graph joins 1 to 2, 3, and 5, and 2 to 4, the parity-of-omega sides {1, 4} against {2, 3, 5}, and the full graph matches at size 2, 1-3 beside 4-2. Deleting k = 2 strands 4, whose only edge dies with it, size 1, and deleting k = 4 leaves the three edges of 1 alone, size 1 again, so both even numbers sit inside every maximum matching, the second player mirrors forever, and the verdict is second.

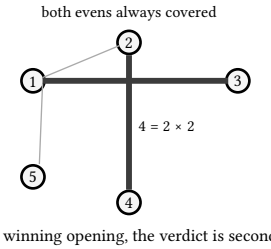


figure 175: the n = 5 prime-multiple graph, the size-2 matching bolded, both even numbers 2 and 4 covered by every maximum matching

```c
static int match_skip(int skip) {
  Match m;
  match_init(&m, gnl, gnr);
  for (int x = 1; x ≤ gn; x++) {
    if (x == skip)
      continue;
    for (int p = 2; (long long)x * p ≤ gn; p++) {
      if (!is_prime(p))
        continue;
      int y = x * p;
      if (y == skip)
        continue;
      if (gop[x]) // L side has odd omega parity
        match_edge(&m, gid[x], gid[y]);
      else
        match_edge(&m, gid[y], gid[x]);
    }
  }
  int size = match_run(&m);
  match_free(&m);
  return size;
}

// one n ≤ CUTOFF: 0 if second wins, else the smallest winning even
static int small_answer(int n) {
  gn = n;
  gnl = gnr = 0;
  for (int x = 1; x ≤ n; x++) {
    int y = x, w = 0;
    for (int d = 2; d * d ≤ y; d++)
      while (y % d == 0) {
        y /= d;
        w ^= 1;
      }
    if (y > 1)
      w ^= 1;
    gop[x] = w;
    if (w)
      gid[x] = ++gnl;
    else
      gid[x] = ++gnr;
  }
  int full = match_skip(0);
  for (int k = 2; k ≤ n; k += 2)
    if (match_skip(k) == full)
      return k;
  return 0;
}
```

listing 625: c, match_skip rebuilding the graph without one vertex, small_answer trying each even deletion against the full matching (source: icpc/samples-c/src/Ch13/pB.c, lines 39-86)

C links the year-local ch13_match header, go its match13 twin, java the Match class:

```
  for x := 1; x <= n; x++ {
    y, w := x, 0
    for d := 2; d*d <= y; d++ {
      for y%d == 0 {
        y /= d
        w ^= 1
      }
    }
    if y > 1 {
      w ^= 1
    }
    gop[x] = w
    if w == 1 {
      nl++
      gid[x] = nl
    } else {
      nr++
      gid[x] = nr
    }
  }
  skip := func(k int) int {
    m := newMatch13(nl, nr)
    for x := 1; x <= n; x++ {
      if x == k {
        continue
      }
      for p := 2; x*p <= n; p++ {
        if !isPrime13(p) {
          continue
        }
        y := x * p
        if y == k {
          continue
        }
        if gop[x] == 1 {
          m.edge(gid[x], gid[y])
        } else {
          m.edge(gid[y], gid[x])
        }
      }
    }
    return m.run()
  }
  full := skip(0)
  for k := 2; k <= n; k += 2 {
    if skip(k) == full {
      return k
    }
  }
  return 0
}
```

listing 626: go, omega parity by trial division, the skip closure over the shared matcher, the retry loop (source: icpc/samples-go/ch13/pb.go, lines 38-88)

```
  static int matchSkip(int skip) {
    Match m = new Match(gnl, gnr);
    for (int x = 1; x ≤ gn; x++) {
      if (x == skip)
        continue;
      for (int p = 2; (long) x * p ≤ gn; p++) {
        if (!isPrime(p))
          continue;
        int y = x * p;
        if (y == skip)
          continue;
        if (gop[x] == 1) // L side has odd omega parity
          m.edge(gid[x], gid[y]);
        else
          m.edge(gid[y], gid[x]);
      }
    }
    return m.run();
  }

  // one n ≤ CUTOFF: 0 if second wins, else the smallest winning even
  static int smallAnswer(int n) {
    gn = n;
    gnl = gnr = 0;
    for (int x = 1; x ≤ n; x++) {
      int y = x, w = 0;
      for (int div = 2; (long) div * div ≤ y; div++)
        while (y % div == 0) {
          y /= div;
          w ^= 1;
        }
      if (y > 1)
        w ^= 1;
      gop[x] = w;
      if (w == 1)
        gid[x] = ++gnl;
      else
        gid[x] = ++gnr;
    }
    int full = matchSkip(0);
    for (int k = 2; k ≤ n; k += 2)
      if (matchSkip(k) == full)
        return k;
    return 0;
  }
```

listing 627: java, matchSkip rebuilding the graph without one vertex over the shared Match class, smallAnswer trying each even deletion (source: icpc/samples-java/src/Ch14/PB.java, lines 52-96)

C# inlines the whole thing, an adjacency array with a stamp-seen augmenting recursion, no helper at all:

```
    private static bool Try(int l)
    {
        for (int e = _head[l]; e ≠ 0; e = _nxt[e])
        {
            int r = _dst[e];
            if (_seen[r] == _stamp)
                continue;
            _seen[r] = _stamp;
            if (_matchr[r] < 0 || Try(_matchr[r]))
            {
                _matchr[r] = l;
                return true;
            }
        }
        return false;
    }

    private static int MatchSkip(int[] gop, int[] gid, int skip)
    {
        int capEdges = 1024;
        _head = new int[_gnl + 1];
        _nxt = new int[capEdges];
        _dst = new int[capEdges];
        _ec = 1;
        _matchr = new int[_gnr + 1];
        _seen = new int[_gnr + 1];
        _stamp = 0;
        for (int x = 1; x ≤ _gn; x++)
        {
            if (x == skip)
                continue;
            for (int p = 2; (long)x * p ≤ _gn; p++)
            {
                if (!IsPrime(p))
                    continue;
                int y = x * p;
                if (y == skip)
                    continue;
                if (gop[x] ≠ 0)
                    Edge(gid[x], gid[y]);
                else
                    Edge(gid[y], gid[x]);
```

listing 628: c#, the augmenting path and the edge build inlined, seen stamps instead of a visited reset (source: icpc/samples/src/Ch13/PB.cs, lines 75-116)

```
function smallAnswer(n) {
  const gop = new Uint8Array(n + 1);
  const gid = new Int32Array(n + 1);
  let gnl = 0, gnr = 0;
  for (let x = 1; x ≤ n; x++) {
    let y = x, w = 0;
    for (let d = 2; d * d ≤ y; d++)
      while (y % d === 0) {
        y /= d;
        w ^= 1;
      }
    if (y > 1) w ^= 1;
    gop[x] = w;
    gid[x] = w ? ++gnl : ++gnr;
  }
  const matchSkip = (skip) ⇒ {
    const m = new Match(gnl, gnr);
    for (let x = 1; x ≤ n; x++) {
      if (x === skip) continue;
      for (let p = 2; x * p ≤ n; p++) {
        if (!isPrime(p)) continue;
        const y = x * p;
        if (y === skip) continue;
        if (gop[x]) m.edge(gid[x], gid[y]); // L side: odd omega parity
        else m.edge(gid[y], gid[x]);
      }
    }
    return m.run();
  };
  const full = matchSkip(0);
  for (let k = 2; k ≤ n; k += 2)
    if (matchSkip(k) === full) return k;
  return 0;
}
```

listing 629: javascript, smallAnswer with the matchSkip closure, one matcher rebuild per candidate (source: icpc/samples-js/src/ch13-pb-blackboardgame.mjs, lines 26-59)

```
def small_answer(n):
    # 0 when second wins, else the smallest even k whose deletion
    # keeps the maximum matching size
    gop = [0] * (n + 1)
    gid = [0] * (n + 1)
    nl = nr = 0
    primes = [p for p in range(2, n + 1) if is_prime(p)]
    for x in range(1, n + 1):
        y, w = x, 0
        for p in primes:
            if p * p > y:
                break
            while y % p == 0:
                y //= p
                w ^= 1
        if y > 1:
            w ^= 1
        gop[x] = w
        if w:
            nl += 1
            gid[x] = nl
        else:
            nr += 1
            gid[x] = nr

    def skip(k):
        m = Match(nl, nr)
        edge = m.edge
        for x in range(1, n + 1):
            if x == k:
                continue
            for p in primes:
                y = x * p
                if y > n:
                    break
                if y == k:
                    continue
                if gop[x]:
                    edge(gid[x], gid[y])
                else:
                    edge(gid[y], gid[x])
        return m.run()

    full = skip(0)
    for k in range(2, n + 1, 2):
        if skip(k) == full:
            return k
    return 0
```

listing 630: python, the same engine, primes precomputed once and reused by every skip (source: icpc/samples-py/src/Ch13/pb.py, lines 31-78)

```lua
-- one n ≤ CUTOFF: 0 if second wins, else the smallest winning even
local function small_answer(n)
  gn = n
  gnl, gnr = 0, 0
  gop, gid = {}, {}
  for x = 1, n do
    local y, w = x, 0
    local d = 2
    while d * d ≤ y do
      while y % d == 0 do
        y = y // d
        w = w ~ 1
      end
      d = d + 1
    end
    if y > 1 then w = w ~ 1 end
    gop[x] = w
    if w == 1 then
      gnl = gnl + 1
      gid[x] = gnl
    else
      gnr = gnr + 1
      gid[x] = gnr
    end
  end
  local full = match_skip(0)
  for k = 2, n, 2 do
    if match_skip(k) == full then
      return k
    end
  end
  return 0
end
```

listing 631: lua, small_answer over the shared match module, the parity and the retry loop (source: icpc/samples-lua/ch13_pb.lua, lines 53-85)

The crafted pair pins the criterion. For n = 6 every even number is in every maximum matching, so “second”, and for n = 8 the matching {1-7, 4-8, 3-6} keeps size 3 while avoiding 2, so “first 2”. All seven suites assert exactly that pair, most add the n = 2 lone-even case, “second” again, java’s third check pinning n = 4 as second too, proven by a C-oracle reference run, and the c# and javascript suites reach past the cutoff: c# checks n = 1000003 opens “first 666662” and javascript validates any large-n opening structurally, k even with k/2 above n/4 and at most n/3, which is the Bertrand claim asserted as it is used. The book’s canonical output is the smallest even k whose deletion keeps the maximum matching size, pinned everywhere.

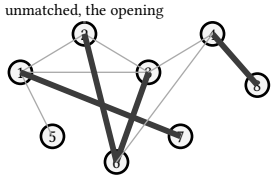


figure 176: the n = 8 prime-multiple graph, the matching 1-7, 4-8, 3-6 bolded, vertex 2 left unmatched as the winning opening

Everything fits int32, no product beyond n is ever formed, and the primality checks are trial division, so no sieve and no big integers anywhere in the seven ports.

15.5. problem c, bride of pipe stream

Flubber flows from station 1, the factory, down through ducts. A station splits its inflow among its ducts in any proportion it likes and may simply discard what it does not forward, a duct forwards fixed percentages of its inflow to lower stations or reservoirs and may lose the rest, and the objective is the largest possible minimum percentage of factory flow reaching any reservoir, the factory's own inflow counting as one unit.

The input opens with three integers s, r, and d, then describes each duct on one line: its station i, its output count k, then k pairs of a target o and a percentage. The problemset pdf bounds s at 10000, r at 3, d between s and 20000, each duct at 1 to 10 outputs with distinct targets o above i and percentages 1 to 100 summing to at most 100 per duct, every station owning at least one duct, and stations numbered by decreasing altitude so the duct graph is a dag. The limit is 12 seconds with answers accepted inside an absolute error of 1e-6, the output is one line holding the optimal percentage, and the judge data carries 82 cases for the letter, 2 samples and 80 secrets.

Official sample 1, reprinted byte for byte from the judge data pair `C-brideofpipestream/sample-1.in` and `sample-1.ans`: two stations and three reservoirs, and the optimum splits the factory's unit flow 0.3 and 0.7 between the two station-1 ducts so reservoirs 3 and 4 land level at 0.24 each while reservoir 5 takes 0.28 through station 2:

input:

```
2 3 3
1 2 3 80 4 10
1 2 2 40 4 30
2 1 5 100
```

expected output:

```
24.0
```

Official sample 2, same source: one station, two reservoirs, three ducts, the two 50 percent singles and the 40-60 mixer, and the optimum sends nothing down the second single, splitting 2/7 and 5/7 so both tanks land at 3/7, the repeating decimal behind the letter's float compare:

input:

```
1 2 3
1 1 2 50
1 1 3 50
1 2 2 40 3 60
```

expected output:

```
42.8571428571
```

Recognition: s at 1e4 with d at 2e4 and at most 10 outputs per duct bounds one dual propagation at the duct count plus the output count, about 2e5 operations along the altitude order the numbering hands over for free, and the nested simplex searches at about 70 outer and 50 inner probes budget 3500 × 2e5 ≈ 7e8 operations inside the 12 second limit.

The task sentence's shape, the largest possible minimum percentage, names a single scalar objective whose level the propagation prices directly, so the answer is one point on a searched axis rather than a combinatorial object. Problem C is the 2025 face of the one-dimensional answer-space search family. The one-dimensional answer-space search family also collects chapter 10 problem E and chapter 11 problem F. A secondary thread, linear programming duality, covers the equalized-tank target the split converges to.

Modeling the ducts as a max-flow network is the tempting wrong tool and the statement breaks it itself: a duct may forward as little as 1 percent and keep the rest, so conservation fails at every station, and a generic lp over the 2e4 duct variables carries no budget the nested searches do not.

The primal says it in flows: give every duct a flow and every reservoir receipt a common lower bound t, write conservation at each station as inflow at least outflow, and maximize t. The dual of that max-min lp settles it: non-negative weights w with the reservoir weights summing to exactly 1, each duct forcing w upstream to dominate the percentage-scaled sum of its downstream weights, minimizing w(station 1), and strong duality makes the dual optimum the primal answer fraction. For fixed reservoir weights the station weights are a max over ducts propagated bottom-up along the altitude order, which the decreasing station numbering gives for free, so r = 1 is one pass, r = 2 is a one-dimensional search over the single free weight, and r = 3 nests one more level on the simplex, about 200 iterations per level holding the error near 1e-9 (statement, solutions.pdf p. 3). Book 10′s chapter 36, numerical methods, develops the lp duality and dual weights this reduction rests on together with the ternary and golden-section searches that scan the reservoir simplex.

The worked run: trace the model on sample 1. Reservoir weights drive the dual: with w3, w4, and w5 summing to 1, the bottom-up pass reads w2 = w5 through duct three's lone 100 and w1 as the max of duct one's 0.8 w3 + 0.1 w4 and duct two's 0.4 w5 + 0.3 w4. Trying w5 = 0 first, the two arms balance where 0.8 w3 + 0.1 w4 = 0.3 w4, that is w4 = 4 w3, and w3 + w4 = 1 hands back w3 = 0.2 with w4 = 0.8, both arms at 0.8 × 0.2 + 0.1 × 0.8 = 0.24. Raising w5 only lifts the balance, the table's three probe rows, so the search parks the simplex at w5 = 0 and the dual value 0.24. The primal confirms the fraction: send x and 1 - x down the two station-1 ducts, reservoir 3 takes 0.8x and reservoir 4 takes 0.3 - 0.2x, the level pair meets at x = 0.3 with both at 0.24, reservoir 5 takes 0.4 × 0.7 = 0.28 above the bound, and the trace ends at the printed answer `24.0`.

w5	w3	w4	duct arms	w1
0.5	0.3	0.2	both 0.26	0.26
0.25	0.25	0.5	both 0.25	0.25
0	0.2	0.8	both 0.24	0.24

```c
// one bottom-up pass with reservoir weights wr[0..r-1]
static double evalw(const double *wr) {
  for (int j = 0; j < gr; j++)
    W[gs + 1 + j] = wr[j];
  for (int i = gs; i ≥ 1; i--) {
    double best = 0;
    for (int e = dstart[i]; e < dstart[i + 1]; e++) {
      double sum = 0;
      int b = tbase[e];
      for (int k = 0; k < ocnt[e]; k++)
        sum += pgrp[b + k] * W[tgrp[b + k]];
      if (sum > best)
        best = sum;
    }
    W[i] = best;
  }
  return W[1];
}

static double solve2(int n, double a, double b) {
  // ternary minimization of evalw over the r=2 simplex
  double lo = a, hi = b;
  for (int it = 0; it < n; it++) {
    double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
    double x1[2] = {m1, 1 - m1}, x2[2] = {m2, 1 - m2};
    if (evalw(x1) ≤ evalw(x2))
      hi = m2;
    else
      lo = m1;
  }
  double x[2] = {lo, 1 - lo};
  return evalw(x);
}
```

listing 632: c, evalw propagating the max over ducts bottom-up, then the r = 2 ternary on the simplex (source: icpc/samples-c/src/Ch13/pC.c, lines 34-66)

```go
// cTernary2 minimizes evalw over the r=2 simplex (wr[1] = 1 - wr[0]).
func (n *cDuctNet) cTernary2(iters int) float64 {
  lo, hi := 0.0, 1.0
  for it := 0; it < iters; it++ {
    m1 := lo + (hi-lo)/3
    m2 := hi - (hi-lo)/3
    x1 := [2]float64{m1, 1 - m1}
    x2 := [2]float64{m2, 1 - m2}
    if n.evalw(x1[:]) <= n.evalw(x2[:]) {
      hi = m2
    } else {
      lo = m1
    }
  }
  x := [2]float64{lo, 1 - lo}
  return n.evalw(x[:])
}

// cInner3 minimizes evalw over y for fixed x (z = 1 - x - y).
func (n *cDuctNet) cInner3(x float64, iters int) float64 {
  ylo, yhi := 0.0, 1-x
  for j := 0; j < iters; j++ {
    y1 := ylo + (yhi-ylo)/3
    y2 := yhi - (yhi-ylo)/3
    w1 := [3]float64{x, y1, 1 - x - y1}
    w2 := [3]float64{x, y2, 1 - x - y2}
    if n.evalw(w1[:]) <= n.evalw(w2[:]) {
      yhi = y2
    } else {
      ylo = y1
    }
  }
  w := [3]float64{x, ylo, 1 - x - ylo}
  return n.evalw(w[:])
}

// cTernary3 nests the ternary search over the r=3 simplex.
func (n *cDuctNet) cTernary3(iters int) float64 {
  lo, hi := 0.0, 1.0
  for it := 0; it < iters; it++ {
    m1 := lo + (hi-lo)/3
    m2 := hi - (hi-lo)/3
    if n.cInner3(m1, iters) <= n.cInner3(m2, iters) {
      hi = m2
    } else {
      lo = m1
    }
  }
  return n.cInner3(lo, iters)
}
```

listing 633: go, the three search tiers, ternary over x, inner ternary over y, z = 1 - x - y (source: icpc/samples-go/ch13/pc.go, lines 45-94)

```java
static double evalw(double[] wr) {
  for (int j = 0; j < gr; j++)
    W[gs + 1 + j] = wr[j];
  for (int i = gs; i ≥ 1; i--) {
    double best = 0;
    for (int e = dstart[i]; e < dstart[i + 1]; e++) {
      double sum = 0;
      int b = tbase[e];
      for (int k = 0; k < ocnt[e]; k++)
        sum += pgrp[b + k] * W[tgrp[b + k]];
      if (sum > best)
        best = sum;
    }
    W[i] = best;
  }
  return W[1];
}

static double solve2(int iters, double lo, double hi) {
  // ternary minimization of evalw over the r=2 simplex
  for (int it = 0; it < iters; it++) {
    double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
    double[] x1 = {m1, 1 - m1}, x2 = {m2, 1 - m2};
    if (evalw(x1) ≤ evalw(x2))
      hi = m2;
    else
      lo = m1;
  }
  double[] x = {lo, 1 - lo};
  return evalw(x);
}
```

listing 634: java, evalw propagating the max over ducts bottom-up, then the r = 2 ternary on the simplex (source: icpc/samples-java/src/Ch14/PC.java, lines 46-76)

```
    private static double EvalW(double[] wr)
    {
        for (int j = 0; j < _gr; j++)
            _w[_gs + 1 + j] = wr[j];
        for (int i = _gs; i ≥ 1; i--)
        {
            double best = 0;
            for (int e = _dstart[i]; e < _dstart[i + 1]; e++)
            {
                double sum = 0;
                int b = _tbase[e];
                for (int k = 0; k < _ocnt[e]; k++)
                    sum += _pgrp[b + k] * _w[_tgrp[b + k]];
                if (sum > best)
                    best = sum;
            }
            _w[i] = best;
        }
        return _w[1];
    }

    private static double Solve2(int n, double a, double b)
    {
        // ternary minimization of EvalW over the r=2 simplex
        double lo = a, hi = b;
        for (int it = 0; it < n; it++)
        {
            double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
            var x1 = new double[2];
            var x2 = new double[2];
            x1[0] = m1;
            x1[1] = 1 - m1;
            x2[0] = m2;
            x2[1] = 1 - m2;
            if (EvalW(x1) ≤ EvalW(x2))
                hi = m2;
            else
                lo = m1;
        }
        var x = new double[2];
        x[0] = lo;
        x[1] = 1 - lo;
        return EvalW(x);
    }
```

listing 635: c#, the same two functions, EvalW then Solve2 (source: icpc/samples/src/Ch13/PC.cs, lines 85-128)

```
  const evalw = (wr) => {
    for (let j = 0; j < r; j++) w[s + 1 + j] = wr[j];
    for (let i = s; i >= 1; i--) {
      let best = 0;
      for (let e = dstart[i]; e < dstart[i + 1]; e++) {
        let sum = 0;
        const b = tbase[e];
        for (let k = 0; k < ocnt[e]; k++)
          sum += pgrp[b + k] * w[tgrp[b + k]];
        if (sum > best) best = sum;
      }
      w[i] = best;
    }
    return w[1];
  };
  // minimize evalw over the r=2 simplex (wr[1] = 1 - wr[0])
  const ternary2 = (iters) => {
    let lo = 0, hi = 1;
    for (let it = 0; it < iters; it++) {
      const m1 = lo + (hi - lo) / 3;
      const m2 = hi - (hi - lo) / 3;
      if (evalw([m1, 1 - m1]) <= evalw([m2, 1 - m2])) hi = m2;
      else lo = m1;
    }
    return evalw([lo, 1 - lo]);
  };
  // minimize evalw over y for fixed x (z = 1 - x - y)
  const inner3 = (x, iters) => {
    let ylo = 0, yhi = 1 - x;
    for (let j = 0; j < iters; j++) {
      const y1 = ylo + (yhi - ylo) / 3;
      const y2 = yhi - (yhi - ylo) / 3;
      if (evalw([x, y1, 1 - x - y1]) <= evalw([x, y2, 1 - x - y2])) yhi = y2;
      else ylo = y1;
    }
    return evalw([x, ylo, 1 - x - ylo]);
  };
  const ternary3 = (iters) => {
    let lo = 0, hi = 1;
    for (let it = 0; it < iters; it++) {
      const m1 = lo + (hi - lo) / 3;
      const m2 = hi - (hi - lo) / 3;
      if (inner3(m1, iters) <= inner3(m2, iters)) hi = m2;
      else lo = m1;
    }
    return inner3(lo, iters);
  };
  let ans;
  if (r === 1) ans = evalw([1]);
  else if (r === 2) ans = ternary2(70);
  else ans = ternary3(50);
  return `${(ans * 100).toFixed(10)}\n`;
```

listing 636: javascript, evalw plus both tiers in one slice, 70 outer and 50 inner iterations (source: icpc/samples-js/src/ch13-pc-brideofpipestream.mjs, lines 51-102)

Python rides golden-section search instead of ternary, one fresh evaluation per iteration rather than two, and the header measures about five times fewer full propagations on the large secrets, the same jointly convex dual for a fraction of the passes:

```
    def evalw(wr):
        for j in range(r):
            w[res_base + j] = wr[j]
        wg = w.__getitem__
        for i, ds in stations:
            best = 0.0
            for ts, ps in ds:
                sm = sum(map(mul, ps, map(wg, ts)))
                if sm > best:
                    best = sm
            w[i] = best
        return w[1]

    def golden(f, lo, hi, iters):
        # f convex on [lo, hi]; one new evaluation per iteration
        span = hi - lo
        x1 = hi - INV_PHI * span
        x2 = lo + INV_PHI * span
        f1 = f(x1)
        f2 = f(x2)
        for _ in range(iters):
            if f1 <= f2:
                hi, x2, f2 = x2, x1, f1
                span = hi - lo
                x1 = hi - INV_PHI * span
                f1 = f(x1)
            else:
                lo, x1, f1 = x1, x2, f2
                span = hi - lo
                x2 = lo + INV_PHI * span
                f2 = f(x2)
        return min(f1, f2)

    if r == 1:
        ans = evalw([1.0])
    elif r == 2:
        ans = golden(lambda x: evalw([x, 1.0 - x]), 0.0, 1.0, 60)
    else:

        def gx(x):
            return golden(lambda y: evalw([x, y, 1.0 - x - y]), 0.0, 1.0 - x, 44)

        ans = golden(gx, 0.0, 1.0, 40)
```

listing 637: python, evalw unchanged, then golden section reusing one endpoint value per step, nested once for r = 3 (source: icpc/samples-py/src/Ch13/pc.py, lines 40-82)

```
  W = {}
  local ans
  if r == 1 then
    ans = evalw({1.0})
  elseif r == 2 then
    local lo, hi = 0.0, 1.0
    for _ = 1, 70 do
      local m1 = lo + (hi - lo) / 3
      local m2 = hi - (hi - lo) / 3
      if evalw({m1, 1 - m1}) ≤ evalw({m2, 1 - m2}) then
        hi = m2
      else
        lo = m1
      end
    end
    ans = evalw({lo, 1 - lo})
  else
    local function inner(x)
      local ylo, yhi = 0.0, 1 - x
      for _ = 1, 40 do
        local y1 = ylo + (yhi - ylo) / 3
        local y2 = yhi - (yhi - ylo) / 3
        if evalw({x, y1, 1 - x - y1}) ≤ evalw({x, y2, 1 - x - y2}) then
          yhi = y2
        else
          ylo = y1
        end
      end
      return evalw({x, ylo, 1 - x - ylo})
    end
    local lo, hi = 0.0, 1.0
    for _ = 1, 40 do
      local m1 = lo + (hi - lo) / 3
      local m2 = hi - (hi - lo) / 3
      if inner(m1) ≤ inner(m2) then
        hi = m2
      else
        lo = m1
      end
    end
    ans = inner(lo)
  end
  return string.format("%.10f\n", ans * 100.0)
end
```

listing 638: lua, the r dispatch with the ternary inline and the inner closure over y (source: icpc/samples-lua/ch13_pc.lua, lines 100-143)

The crafted fixture is one station, two reservoirs, two ducts: duct 1 sends 50 percent to reservoir 2, duct 2 sends 30 percent there and 80 percent to reservoir 3. Splitting a + c = 1 equalizes both tanks at 0.5a + 0.3c = 0.8c, which forces a = c and both at 0.4, and the dual check lands on w3 = 0.2 with value 0.4, so the answer prints “40.0000000000”. Every suite asserts that, most add the r = 1 case where the 100 percent duct takes everything for “100.0000000000”, and the c and lua suites add a lone 50 percent duct for “50.0000000000”. The judge comparison runs in last-line-float mode because the official answers print ten decimals and the dual search carries its own tolerance, and the c# suite adds a third two-reservoir case with a repeating-decimal optimum.

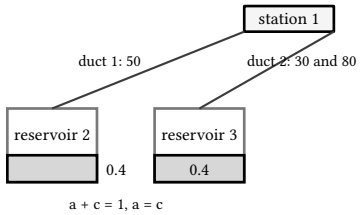


figure 177: station 1 splitting its flow between the two ducts, the percentage arrows, and both reservoir tanks filling to the equalized 40 percent line

The percentages are integers and every real quantity is a double, so there is no integer divergence to report, only the print format, %.10f of a percentage, identical in all seven.

15.6. problem d, buggy rover

A rover sits on an r by c grid of flat and rocky cells, one cell marked S, and carries a preference order over the four directions N, E, S, W. Each move steps in the first direction of the current order that stays on the grid and off rocks, so the order only ever steers among the feasible directions. Cosmic rays may hit between moves and replace the order wholesale, and given the grid, the start, and the observed log of moves, the task is the fewest ray hits that explain the log, some sequence of orders changing at most that many times.

The input is two integers r and c, then the r grid rows over ‘.’, ‘#’, and exactly one ‘S’, then the move string. The problemset pdf bounds r and c at 200, the log at 1 to 10000 letters with every logged move landing on a flat cell, and the limit is 2 seconds. The output is one integer, and the judge data carries 26 cases for the letter, 3 samples and 23 secrets.

Official sample 1, reprinted byte for byte from the judge data pair `D-buggyrover/sample-1.in` and `sample-1.ans`: a 5 by 3 grid with one rock at the top left, the rover starting mid-bottom, and the N, N, E, N log. N twice from the fully open interior forces N to rank first, the E then costs a ray since the interior leaves every direction feasible, and the final N survives under an E-first order because the right edge makes E infeasible there, so the answer is 1:

input:

```
5 3
#..
...
...
...
.S.
NNEN
```

expected output:

```
1
```

Recognition: r and c at 200 bound the walk’s footprint, the log at 1e4 letters, and only 24 orders exist, so the survivor filter costs 24 × 1e4 = 2.4e5 first-feasible checks against the 2 second limit, microseconds of work, and the whole difficulty is the reset rule.

The cue is the word fewest: the log is an observation stream that a handful of faults must explain, and the consistent world is recovered by intersecting survivor sets and paying one fault whenever the intersection empties. Problem D is the 2025 face of the consistency under a faulty oracle family. Consistency under a faulty oracle ties chapter 13 problem W and chapter 14 problem A together.

Guessing the actual order schedule is the priced-out alternative: 24^10000 candidate sequences dwarf the log, and a dynamic program over order subsets pays 2^24 states times the log where one monotone intersection pass decides.

Only 24 orders exist and the position after each move depends only on the move index, so per move the solver computes which orders have the observed direction as their first feasible one, intersects with the surviving set, and an empty intersection costs one ray and resets to the current move's set. Keeping the maximal surviving set is optimal because consistency is monotone under supersets, and the whole pass is O(24 n) (statement, solutions.pdf pp. 3-4). The seven ports differ only in how they generate the 24 permutations, a hand-rolled next-permutation everywhere except python, which imports them.

The worked run: trace the model on sample 1. The rover starts mid-bottom at row 4, column 1, and the position after each move depends only on the move index. Move 1, N: the bottom row leaves N, E, and W feasible, so N must outrank E and W, and 8 of the 24 orders survive. Move 2, N: the interior at row 3 leaves all four feasible, so N must rank first outright, 6 orders inside the 8, no ray. Move 3, E: the interior at row 2 again leaves all four feasible, E first outright contradicts every survivor, one ray, and the set resets to the 6 E-first orders. Move 4, N: the right edge at column 2 makes E infeasible, so N only needs to outrank S and W, and 2 of the E-first orders comply, no second ray. One ray in total, and the trace ends at the printed answer 1.

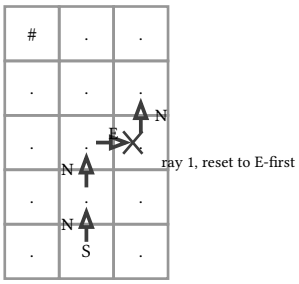


figure 178: the sample 5 by 3 grid, the rock top left, the N-N-E-N walk from the mid-bottom S, and the single ray where the E reset happens

```
int cr = sr, cc = sco, rays = 0;
unsigned cur = 0xFFFFFFu;
for (const char *m = moves; *m == 'N' || *m == 'E' || *m == 'S' || *m == 'W';
     m++) {
  char d = *m;
  unsigned mask = 0;
  for (int pi = 0; pi < 24; pi++) {
    for (int k = 0; k < 4; k++) {
      int dir = perms[pi][k];
      int nr = cr + dr[dir], nc = cc + dc[dir];
      if (nr < 0 || nr ≥ r || nc < 0 || nc ≥ c || grid[nr][nc] == '#')
        continue;
      if (names[dir] == d)
        mask |= 1u << pi;
      break;
    }
  }
  cur &= mask;
  if (!cur) {
    rays++;
    cur = mask;
  }
  int di = d == 'N' ? 0 : d == 'E' ? 1 : d == 'S' ? 2 : 3;
  cr += dr[di];
  cc += dc[di];
}
```

listing 639: c, the survivor filter, mask of consistent orders, intersect, empty means a ray and a reset (source: icpc/samples-c/src/Ch13/pD.c, lines 76-101)

```
cr, cc, rays := sr, sco, 0
cur := uint(0xFFFFFF)
for i := 0; i < len(moves); i++ {
  d := moves[i]
  var mask uint
  for pi := 0; pi < 24; pi++ {
    for _, dir := range perms[pi] {
      nr, nc := cr+dr[dir], cc+dc[dir]
      if nr < 0 || nr ≥ r || nc < 0 || nc ≥ c || grid[nr][nc] == '#' {
        continue
      }
      if names[dir] == d {
        mask |= 1 << pi
      }
      break
    }
  }
  cur &= mask
  if cur == 0 {
    rays++
    cur = mask
  }
  di := 0
  switch d {
  case 'E':
    di = 1
  case 'S':
    di = 2
  case 'W':
    di = 3
  }
  cr += dr[di]
  cc += dc[di]
}
```

listing 640: go, the filter with the direction decode as a switch (source: icpc/samples-go/ch13/pd.go, lines 53-86)

```
    int cr = sr, cc = sco, rays = 0;
    int cur = 0xFFFFFF;
    for (int mi = 0; mi < moves.length()
        && "NESW".indexOf(moves.charAt(mi)) ≥ 0; mi++) {
      char d = moves.charAt(mi);
      int mask = 0;
      for (int pi = 0; pi < 24; pi++) {
        for (int k = 0; k < 4; k++) {
          int dir = perms[pi][k];
          int nr = cr + dr[dir], nc = cc + dc[dir];
          if (nr < 0 || nr ≥ r || nc < 0 || nc ≥ c || grid[nr][nc] == '#')
            continue;
          if (names[dir] == d)
            mask |= 1 << pi;
          break;
        }
      }
      cur &= mask;
      if (cur == 0) {
        rays++;
        cur = mask;
      }
      int di = d == 'N' ? 0 : d == 'E' ? 1 : d == 'S' ? 2 : 3;
      cr += dr[di];
      cc += dc[di];
    }
    return rays + "\n";
```

listing 641: java, the survivor filter over the 24 hand-permuted orders, mask intersect, empty means a ray and a reset (source: icpc/samples-java/src/Ch14/PD.java, lines 95-121)

```
        int cr = sr, cc = sc, rays = 0;
        uint cur = 0xFFFFFF;
        for (int m = 0; m < mlen; m++)
        {
            char d = input[mstart + m];
            uint mask = 0;
            for (int pi = 0; pi < 24; pi++)
            {
                for (int k = 0; k < 4; k++)
                {
                    int dir = perms[pi][k];
                    int nr = cr + dr[dir], nc = cc + dc[dir];
                    if (nr < 0 || nr ≥ r || nc < 0 || nc ≥ c
                        || grid[nr, nc] == '#')
                        continue;
                    if (names[dir] == d)
                        mask |= 1u << pi;
                    break;
                }
            }
            cur &= mask;
            if (cur == 0)
            {
                rays++;
                cur = mask;
            }
            int di = d == 'N' ? 0 : d == 'E' ? 1 : d == 'S' ? 2 : 3;
            cr += dr[di];
            cc += dc[di];
        }
        return rays + "\n";
```

listing 642: c#, the same filter over a uint mask, the first-feasible-direction scan per order (source: icpc/samples/src/Ch13/PD.cs, lines 68-98)

```
  let cr = sr, cc = sc, rays = 0;
  let cur = 0xffffff;
  for (let i = 0; i < moves.length; i++) {
    const d = moves[i];
    let mask = 0;
    for (let pi = 0; pi < 24; pi++) {
      for (const dir of perms[pi]) {
        const nr = cr + dr[dir], nc = cc + dc[dir];
        if (nr < 0 || nr ≥ r || nc < 0 || nc ≥ c || grid[nr][nc] === "#")
          continue;
        if (names[dir] === d) mask |= 1 << pi;
        break;
      }
    }
    cur &= mask;
    if (cur === 0) {
      rays++;
      cur = mask;
    }
    let di = 0;
    if (d === "E") di = 1;
    else if (d === "S") di = 2;
    else if (d === "W") di = 3;
    cr += dr[di];
    cc += dc[di];
  }
  return `${rays}\n`;
```

listing 643: javascript, the filter, mask bits over the 24 hand-permuted orders (source: icpc/samples-js/src/ch13-pd-buggyrover.mjs, lines 51-77)

```
    cr, cc, rays = sr, sco, 0
    cur = (1 << 24) - 1
    for d in moves:
        mask = 0
        for pi, perm in enumerate(PERMS):
            for dir in perm:
                nr = cr + DR[dir]
                nc = cc + DC[dir]
                if nr < 0 or nr ≥ r or nc < 0 or nc ≥ c or grid[nr][nc] == "#":
                    continue
                if NAMES[dir] == d:
                    mask |= 1 << pi
                break
        cur &= mask
        if not cur:
            rays += 1
            cur = mask
        di = NAMES.index(d)
        cr += DR[di]
        cc += DC[di]
    return f"{rays}\n"
```

listing 644: python, itertools.permutations instead of a next-permutation loop, the filter eight lines long (source: icpc/samples-py/src/Ch13/pd.py, lines 31-51)

```
  local cr, cc = sr, sco
  local rays = 0
  local cur = 0xFFFFFF
  for mi = 1, #moves do
    local ch = moves:byte(mi)
    if ch == 78 or ch == 69 or ch == 83 or ch == 87 then
      local d = ch
      local mask = 0
      for pi = 1, 24 do
        local perm = perms[pi]
        for k = 1, 4 do
          local dir = perm[k] + 1
          local nr = cr + dr[dir]
          local nc = cc + dc[dir]
          if nr ≥ 1 and nr ≤ r and nc ≥ 1 and nc ≤ c and
             grid[nr]:byte(nc) ~= 35 then
            if names:byte(dir) == d then
              mask = mask | (1 << (pi - 1))
            end
            break
          end
        end
      end
      cur = cur & mask
      if cur == 0 then
        rays = rays + 1
        cur = mask
      end
      local di = d == 78 and 1 or d == 69 and 2 or d == 83 and 3 or 4
      cr = cr + dr[di]
      cc = cc + dc[di]
    end
```

listing 645: lua, the filter over byte compares, 1-based grid bounds, lua 5.4 bitwise and (source: icpc/samples-lua/ch13_pd.lua, lines 68-99)

The crafted walk is a 3 by 3 grid starting at the bottom left with moves N, E, N. The first move leaves 12 orders, those ranking N before E at a cell where both are feasible. The second move needs E ranked first at a fully open cell, contradicting every survivor, one ray, and the third move needs N first, contradicting the E-first reset, a second ray, and no single cut does better, so the answer is 2. Every suite pins that plus the single-move corridor “0”, and c# adds a third case where the contradiction lands on move two for “1”. Counts per language: 2, 3, 2, 2, 2, 2, java 2.

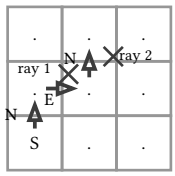


figure 179: the crafted 3 by 3 grid, the N-E-N walk from the bottom-left S, and the two lightning bolts where the preference order had to change

Plain int32 grid walking, no counts beyond the move length, so the seven languages agree everywhere including the integers.

15.7. problem e, delivery service

A delivery service hires couriers one at a time, and every courier shuttles daily between a home city and a destination city: home overnight and at 9:00, at the destination from 12:00 to 14:00, home again at 17:00. Packages

travel only with couriers and pass from one courier to another exactly when the two share a city at the same time. After each hire the output is the number of city pairs that can exchange packages in both directions, meaning some chain of rides and same-city handoffs moves a package either way between them.

The input is two integers n and m, then m hire lines of two integers a and b, the home and destination of the hired courier. The problemset pdf bounds n at 2e5 cities and m at 4e5 couriers, every hire naming two distinct cities with no repeated pair. The limit is 12 seconds, the output is one integer per hire, and the judge data carries 25 cases for the letter, a single sample and 24 secrets.

Official sample 1, reprinted byte for byte from the judge data pair `E-delivery/sample-1.in` and `sample-1.ans`: four hires over four cities. The second leaves cities 1 and 3 apart even though both share city 2, because noon 2 and morning 2 are different vertices, the third adds the pair set of the component holding cities 2, 3, and 4, and the fourth welds everything into all six pairs:

input:

```
4 4
1 2
2 3
4 3
4 2
```

expected output:

```
1
2
4
6
```

Recognition: n at 2e5 cities doubled to 4e5 vertices and m at 4e5 hires against the 12 second limit rule out any per-hire traversal, 4e5 hires × 4e5 vertices = 1.6e11 visits, while the union-find with small-into-large merging plus the signature table runs O(m log^2 n), about 4e5 × 17^2 ≈ 1.2e8 table operations.

The asked-for count, city pairs that can exchange packages in both directions, is a connectivity closure quantity: it only moves when components weld, and the morning-noon doubling turns every hire into one edge, the cue that the answer lives on the components. Problem E is the 2025 face of the component structure and connectivity family. The component structure and connectivity family runs through chapter 11 problem B, chapter 12 problem E, chapter 12 problem H, chapter 13 problem R, and chapter 15 problem G.

Rebuilding the component map per hire is the tempting alternative and the arithmetic kills it, the 1.6e11 visits above, and recomputing the pair count from scratch per hire adds another factor of n the incrementally maintained score already carries.

The model gives each city a morning vertex and a noon vertex, because two couriers meet exactly when they share a home or a destination, so courier (a, b) is the undirected edge {morning(a), noon(b)}, and two cities are connected when their non-isolated vertices share a component. One subtlety cost this wave a mid-week correction: a component that touches both of its own cities, like {morning 2, noon 4, morning 3} after hiring (2, 4) onto the (3, 4) component, connects those cities too, and the original walkthrough narrative missed it until two ports converged on the official sample's arithmetic (statement, solutions.pdf p. 4). Counting runs over the 2n vertices with union-find under small-into-large merging: the score sum of c(C)^2 over components, c(C) the distinct cities a component touches, counts each connected ordered pair once per shared component, and subtracting n and X, the sum of f(sig)^2 over the unordered two-component signatures held in an incrementally updated table, then halving, yields exactly the unordered pairs, O(m log^2 n) with a balanced tree behind the table.

The worked run: trace the model on sample 1, four hires over four cities. Hire (1, 2) welds morning 1 onto noon 2, a component touching cities 1 and 2, one pair, printed 1. Hire (2, 3) starts a second component touching cities 2

and 3, city 2's two vertices still apart, so the pairs stay (1, 2) and (2, 3), printed 2. Hire (4, 3) welds morning 4 onto it, a component touching 2, 3, and 4, and the pair set gains (2, 3), (2, 4), and (3, 4) for four in total, printed 4. Hire (4, 2) welds everything into one component touching all four cities, and the score formula agrees, one component with c(C) = 4 contributing 4^2 = 16, so (16 - 4 - 0) / 2 = 6, printed 6, and the trace ends at the printed answer 6.

hire	components touching cities	pairs
(1, 2)	{1, 2}	1
(2, 3)	{1, 2} and {2, 3}	2
(4, 3)	{1, 2} and {2, 3, 4}	4
(4, 2)	{1, 2, 3, 4}	6

```c
static void unite(int x, int y) {
  int rx = find(x), ry = find(y);
  if (rx == ry)
    return;
  if (sz[rx] > sz[ry]) {
    int t = rx;
    rx = ry;
    ry = t;
  }
  int p = rx, q = ry;
  score -= cc[p] * cc[p] + cc[q] * cc[q];
  curstamp++;
  for (int w = hd[p]; w; w = nxt[w]) {
    int u = (w + 1) >> 1;
    if (stamp[u] == curstamp)
      continue;
    stamp[u] = curstamp;
    int v1 = 2 * u - 1, v2 = 2 * u;
    int r1 = find(v1), r2 = find(v2);
    sig_delta(r1, r2, -1);
    sig_delta(r1 == p ? q : r1, r2 == p ? q : r2, +1);
  }
  long long overlap = 0;
  for (int w = hd[p]; w; w = nxt[w]) {
    int o = (w & 1) ? w + 1 : w - 1;
    if (find(o) == q)
      overlap++;
  }
  par[p] = q;
  sz[q] += sz[p];
  cc[q] += cc[p] - overlap;
  score += cc[q] * cc[q];
  nxt[tl[q]] = hd[p];
  tl[q] = tl[p];
}
```

listing 646: c, unite under small-into-large, every moved city's signature retired and re-added, city counts and score updated in place (source: icpc/samples-c/src/Ch13/pE.c, lines 111-145)

The signature table is where the languages spread out. C open-addresses a 64-bit packed key with tombstones so only live signatures occupy memory, go, java, c#, javascript, and python use their native maps, java's a HashMap keyed by the (min, max) root pair packed into one long, and the c# slice shows the counting rule itself:

```
unite := func(x, y int) {
  p, q := find(x), find(y)
  if p == q {
    return
  }
  if sz[p] > sz[q] {
    p, q = q, p
  }
  score -= cc[p]*cc[p] + cc[q]*cc[q]
  curstamp++
  for w := hd[p]; w != 0; w = nxt[w] {
    u := (w + 1) >> 1
    if stamp[u] == curstamp {
      continue
    }
    stamp[u] = curstamp
    r1, r2 := find(2*u-1), find(2*u)
    sigDelta(r1, r2, -1)
    if r1 == p {
      r1 = q
    }
    if r2 == p {
      r2 = q
    }
    sigDelta(r1, r2, 1)
  }
  var overlap int64
  for w := hd[p]; w != 0; w = nxt[w] {
    o := w + 1
    if w&1 == 0 {
      o = w - 1
    }
    if find(o) == q {
      overlap++
    }
  }
  par[p] = q
  sz[q] += sz[p]
  cc[q] += cc[p] - overlap
  score += cc[q] * cc[q]
  nxt[tl[q]] = hd[p]
  tl[q] = tl[p]
}
```

listing 647: go, the unite closure, map-based signatures, the overlap fix when both endpoints already sit together (source: icpc/samples-go/ch13/pe.go, lines 69-111)

```
  static void unite(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx == ry)
      return;
    if (sz[rx] > sz[ry]) {
      int t = rx;
      rx = ry;
      ry = t;
    }
    int p = rx, q = ry;
    score -= cc[p] * cc[p] + cc[q] * cc[q];
    curstamp++;
    for (int w = hd[p]; w != 0; w = nxt[w]) {
      int u = (w + 1) >> 1;
      if (stampArr[u] == curstamp)
        continue;
      stampArr[u] = curstamp;
      int v1 = 2 * u - 1, v2 = 2 * u;
      int r1 = find(v1), r2 = find(v2);
      sigDelta(r1, r2, false);
      sigDelta(r1 == p ? q : r1, r2 == p ? q : r2, true);
    }
    long overlap = 0;
    for (int w = hd[p]; w != 0; w = nxt[w]) {
      int o = (w & 1) != 0 ? w + 1 : w - 1;
      if (find(o) == q)
        overlap++;
    }
    par[p] = q;
    sz[q] += sz[p];
    cc[q] += cc[p] - overlap;
    score += cc[q] * cc[q];
    nxt[tl[q]] = hd[p];
    tl[q] = tl[p];
  }
```

listing 648: java, unite under small-into-large, moved cities' signatures retired and re-added through the HashMap, the overlap discounted, lists spliced (source: icpc/samples-java/src/Ch14/PE.java, lines 87-121)

```
    private static void SigDelta(int r1, int r2, bool add)
    {
        if (r1 == r2)
            return;
        long key = (long)Math.Min(r1, r2) << 20 | (uint)Math.Max(r1, r2);
        if (add)
        {
            _sig.TryGetValue(key, out int f);
            _x += 2 * (long)f + 1;
            _sig[key] = f + 1;
        }
        else
        {
            int f = _sig[key];
            if (f == 1)
                _sig.Remove(key);
            else
                _sig[key] = f - 1;
            f--;
            _x -= 2 * (long)f + 1;
        }
    }
```

listing 649: c#, SigDelta over a Dictionary of packed roots, X moving by 2f+1 per add or remove (source: icpc/samples/src/Ch13/PE.cs, lines 38-60)

```
  const unite = (vx, vy) => {
    let p = find(vx), q = find(vy);
    if (p === q) return;
    if (sz[p] > sz[q]) [p, q] = [q, p];
    score -= cc[p] * cc[p] + cc[q] * cc[q];
    curstamp++;
    for (let w = hd[p]; w; w = nxt[w]) {
      const u = (w + 1) >> 1;
      if (stamp[u] === curstamp) continue;
      stamp[u] = curstamp;
      let r1 = find(2 * u - 1), r2 = find(2 * u);
      sigDelta(r1, r2, -1);
      if (r1 === p) r1 = q;
      if (r2 === p) r2 = q;
      sigDelta(r1, r2, 1);
    }
    let overlap = 0;
    for (let w = hd[p]; w; w = nxt[w]) {
      const o = w & 1 ? w + 1 : w - 1;
      if (find(o) === q) overlap++;
    }
    par[p] = q;
    sz[q] += sz[p];
    cc[q] += cc[p] - overlap;
    score += cc[q] * cc[q];
    nxt[tl[q]] = hd[p];
    tl[q] = tl[p];
  };
  for (let u = 1; u ≤ n; u++) sigDelta(2 * u - 1, 2 * u, 1);
  const out = [];
  for (let k = 0; k < m; k++) {
    const a = parseInt(tok[2 + 2 * k], 10);
    const bb = parseInt(tok[3 + 2 * k], 10);
    unite(2 * a - 1, 2 * bb);
    out.push(`${(score - n - x) / 2}`);
  }
  return out.join("\n") + "\n";
```

listing 650: javascript, unite plus the hire loop printing (score - n - x) / 2 per line (source: icpc/samples-js/src/ch13-pe-delivery.mjs, lines 62-98)

```
    curstamp = 0

    def unite(vx, vy):
        nonlocal score, curstamp
        p, q = find(vx), find(vy)
        if p == q:
            return
        if sz[p] > sz[q]:
            p, q = q, p
        score -= cc[p] * cc[p] + cc[q] * cc[q]
        curstamp += 1
        w = hd[p]
        while w:
            u = (w + 1) >> 1
            if stamp[u] != curstamp:
                stamp[u] = curstamp
                r1, r2 = find(2 * u - 1), find(2 * u)
                sig_delta(r1, r2, -1)
                if r1 == p:
                    r1 = q
                if r2 == p:
                    r2 = q
                sig_delta(r1, r2, 1)
            w = nxt[w]
        overlap = 0
        w = hd[p]
        while w:
            o = w + 1 if w & 1 else w - 1
            if find(o) == q:
                overlap += 1
            w = nxt[w]
        par[p] = q
        sz[q] += sz[p]
        cc[q] += cc[p] - overlap
        score += cc[q] * cc[q]
        nxt[tl[q]] = hd[p]
        tl[q] = tl[p]

    for u in range(1, n + 1):
        sig_delta(2 * u - 1, 2 * u, 1)
    out = []
    p = 2
    for _ in range(m):
        a = vals[p]
        b = vals[p + 1]
        p += 2
        unite(2 * a - 1, 2 * b)
        out.append(str((score - n - x) // 2))
```

listing 651: python, the same engine, dict signatures, the formula in the hire loop (source: icpc/samples-py/src/Ch13/pe.py, lines 57-104)

```
local function unite(x, y)
  local rx, ry = find(x), find(y)
  if rx == ry then return end
  if sz[rx] > sz[ry] then
    rx, ry = ry, rx
  end
  local p, q = rx, ry
  score = score - cc[p] * cc[p] - cc[q] * cc[q]
  curstamp = curstamp + 1
  local w = hd[p]
  while w do
    local u = (w + 1) // 2
    if stamp[u] ~= curstamp then
      stamp[u] = curstamp
      local r1 = find(2 * u - 1)
      local r2 = find(2 * u)
      sig_delta(r1, r2, -1)
      sig_delta(r1 == p and q or r1, r2 == p and q or r2, 1)
    end
    w = nxt[w]
  end
  local overlap = 0
  w = hd[p]
  while w do
    local o = w % 2 == 1 and w + 1 or w - 1
    if find(o) == q then
      overlap = overlap + 1
    end
    w = nxt[w]
  end
  par[p] = q
  sz[q] = sz[q] + sz[p]
  cc[q] = cc[q] + cc[p] - overlap
  score = score + cc[q] * cc[q]
  nxt[tl[q]] = hd[p]
  tl[q] = tl[p]
end
```

listing 652: lua, unite with table signatures, the same small-into-large walk (source: icpc/samples-lua/ch13_pe.lua, lines 55-92)

The crafted instance hires (1, 2), then (3, 4), then (2, 4). The first two hires give two separate components and pairs (1, 2) and (3, 4), so the first two lines are 1 and 2. The third hire welds morning 2 onto the component holding noon 4 and morning 3, city 2 straddles both components, and the corrected model counts (1, 2), (2, 3), (2, 4), (3, 4), so the third line is 4. All seven suites assert 1, 2, 4 and the straddling extra “3 2 / 1 2 / 2 3” asserting 1 then 2, which is the correction lesson in miniature, and c# adds a third stepwise case asserting the same two answers. Counts per language: 2, 3, 2, 2, 2, 2, java 3.

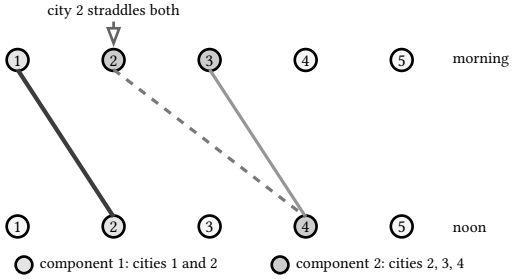


figure 180: morning vertices above, noon vertices below, the three courier edges, and after the third hire city 2 straddling both shaded components

Pair counts reach n(n-1)/2 near 2e10 and score squares 4e10, so c, c#, go, and lua carry int64, java longs, python is native, and javascript's plain Number is exact below 2^53, which covers 4e10 with room, so no BigInt anywhere. C scrambles its packed signature key through an unsigned golden-ratio hash; java's HashMap keys pack the (min, max) root pair into one long below 2^42, so no unsigned road opens anywhere in the year's java tree.

15.8. problem f, herding cats

m distinct catnip varieties go one per pot into a row of m pots. Each cat walks the row from pot 1, sniffing every pot in turn, and stops at the first pot holding a variety it likes. Given each cat's liked set and the pot it is supposed to stop at, the answer is whether some placement of varieties into pots makes every cat stop exactly at its target.

The input is a test count t, then per test the cat count n and pot count m, then n cat blocks: a target pot and a like count k, then the k distinct liked varieties. The problemset pdf bounds t at 10000, n and m at 2e5 per test with the sums of n and m over all tests capped at 2e5 each, and the total like count at 5e5. The limit is 2 seconds, the output is one line per test, yes or no, and the judge data carries 48 cases for the letter, a single sample and 47 secrets.

Official sample 1, reprinted byte for byte from the judge data pair `F-herdingcats/sample-1.in` and `sample-1.ans`: two tests over three cats and five pots, the second identical except the last cat's target moves from pot 4 to pot 5, which lifts varieties 3 and 4 to lower bound 5 where only one pot remains and flips the verdict:

input:

```
2
3 5
2 2 1 5
2 3 1 4 5
4 2 3 4
3 5
2 2 1 5
2 3 1 4 5
5 2 3 4
```

expected output:

```
yes
no
```

Recognition: the sums of n and m over all tests cap at 2e5 each and the total like count at 5e5 against the 2 second limit, and the two conditions cost one pass over the cats then one sorted pass over the lower bounds, about 5e5 log 5e5 ≈ 1e7 steps, so the budget is trivial next to the model.

The cue is the placement question, whether some placement of varieties into pots makes every cat stop exactly at its target: feasibility under per-pot intersection quotas and per-prefix counts, Hall's condition in prefix form. Problem F is the 2025 face of the bipartite assignment feasibility family. The bipartite assignment feasibility family collects chapter 10 problem C, chapter 13 problem X, and chapter 15 problem B.

Running an explicit variety-to-pot matching is the priced-out alternative: 5e5 likes through a matching engine cost hundreds of millions of augment steps where two linear scans decide, and a greedy pot-by-pot fill without the prefix test can reject placements that exist.

A cat targeting p means every variety it likes sits at a pot at or beyond p, so variety z carries the lower bound L(z), the maximum target over the cats liking it, 1 when unliked. The placement exists exactly when both of two conditions hold: every pot hosting cats offers a variety in their intersection with L at most the pot, and for every prefix q at least q varieties have L at most q, Hall's condition in prefix form. The two conditions are also sufficient outright, because filling pots left to right always leaves a legal variety for the next pot. Both are checkable in one sorted pass, O(n + m + K log K) with K the total like count (statement, solutions.pdf pp. 4-5). Book 10′s chapter 38,

network flows ii, states hall's theorem in exactly this prefix form and proves it, the matching argument the two conditions encode.

The worked run: trace the model on sample 1, two tests over the same three cats. Test 1: the liked sets give the lower bounds L(1) = 2 and L(5) = 2 from the two pot-2 cats, L(4) = 4 from the pot-4 cat, L(3) = 4, and L(2) = 1 unliked. Both hosted pots pass the intersection test, pot 2 offering 1 or 5 with L at most 2 and pot 4 offering 4 with L exactly 4, and the prefix counts hold at every q, the table, so the verdict is yes. Test 2 moves the last cat's target to pot 5, lifting L(3) and L(4) to 5, and the prefix count at q = 4 drops to the three varieties 1, 2, and 5 against four pots, Hall fails, and the trace ends at the printed answer no.

q	test 1, varieties with L at most q	test 2	verdict
1	1: {2}	1: {2}	both pass
2	3: {1, 2, 5}	3: {1, 2, 5}	both pass
3	3: {1, 2, 5}	3: {1, 2, 5}	both pass
4	5: {1, 2, 3, 4, 5}	3: {1, 2, 5}	test 2 fails

```
  int ok = 1;
  for (int q = 1; q ≤ m && ok; q++) {
    int ncats = 0;
    for (int i = potcat[q]; i ≥ 0; i = catnext[i])
      ncats++;
    if (ncats == 0)
      continue;
    stamp++;
    int nt = 0;
    for (int i = potcat[q]; i ≥ 0; i = catnext[i])
      for (int j = 0; j < catk[i]; j++) {
        int z = likes[catoff[i] + j];
        if (vstamp[z] ≠ stamp) {
          vstamp[z] = stamp;
          vcnt[z] = 1;
          touched[nt++] = z;
        } else
          vcnt[z]++;
      }
    int hit = 0;
    for (int j = 0; j < nt && !hit; j++)
      hit = vcnt[touched[j]] == ncats && L[touched[j]] ≤ q;
    ok = hit;
  }
  for (int q = 0; q ≤ m; q++)
    freq[q] = 0;
  for (int z = 1; z ≤ m; z++)
    freq[L[z]]++;
  int running = 0;
  for (int q = 1; q ≤ m && ok; q++) {
    running += freq[q];
    if (running < q)
      ok = 0;
  }
```

listing 653: c, per-pot intersection by stamp, the hit test against L, then the prefix count (source: icpc/samples-c/src/Ch13/pF.c, lines 67-100)

```
    vstamp := make([]int, m+1)
    vcnt := make([]int, m+1)
    ok := true
    for q := 1; q <= m && ok; q++ {
      ncats := 0
      for i := potcat[q]; i ≥ 0; i = catnext[i] {
        ncats++
      }
      if ncats == 0 {
        continue
      }
      var touched []int
      for i := potcat[q]; i ≥ 0; i = catnext[i] {
        for j := 0; j < catk[i]; j++ {
          z := likes[catoff[i]+j]
          if vstamp[z] ≠ q {
            vstamp[z] = q
            vcnt[z] = 1
            touched = append(touched, z)
          } else {
            vcnt[z]++
          }
        }
      }
      hit := false
      for _, z := range touched {
        if vcnt[z] == ncats && low[z] <= q {
          hit = true
          break
        }
      }
      ok = hit
    }
    freq := make([]int, m+1)
    for z := 1; z <= m; z++ {
      freq[low[z]]++
    }
    running := 0
    for q := 1; q <= m && ok; q++ {
      running += freq[q]
      if running < q {
        ok = false
      }
    }
```

listing 654: go, cat lists threaded through pot buckets, touched varieties collected per pot (source: icpc/samples-go/ch13/pf.go, lines 46-89)

```
    for (int q = 1; q ≤ m && feasible; q++) {
      int ncats = 0;
      for (int i = potcat[q]; i ≥ 0; i = catnext[i])
        ncats++;
      if (ncats == 0)
        continue;
      stamp++;
      int nt = 0;
      for (int i = potcat[q]; i ≥ 0; i = catnext[i])
        for (int j = 0; j < catk[i]; j++) {
          int z = likes[catoff[i] + j];
          if (vstamp[z] ≠ stamp) {
            vstamp[z] = stamp;
            vcnt[z] = 1;
            touched[nt++] = z;
          } else
            vcnt[z]++;
        }
      boolean hit = false;
      for (int j = 0; j < nt && !hit; j++)
        hit = vcnt[touched[j]] == ncats && L[touched[j]] ≤ q;
      feasible = hit;
    }
    int[] freq = new int[m + 2];
    for (int z = 1; z ≤ m; z++)
      freq[L[z]]++;
    int running = 0;
    for (int q = 1; q ≤ m && feasible; q++) {
      running += freq[q];
      if (running < q)
        feasible = false;
    }
    out.append(feasible ? "yes" : "no").append('\n');
```

listing 655: java, per-pot intersection by stamp over the bucket-threaded cat lists, the hit test against L, then the prefix count (source: icpc/samples-java/src/Ch14/PF.java, lines 74-106)

```
            bool ok = true;
            for (int q = 1; q ≤ m && ok; q++)
            {
                int ncats = 0;
                for (int i = potcat[q]; i ≥ 0; i = catnext[i])
                    ncats++;
                if (ncats == 0)
                    continue;
                stamp++;
                int nt = 0;
                for (int i = potcat[q]; i ≥ 0; i = catnext[i])
                    for (int j = 0; j < catk[i]; j++)
                    {
                        int z = likes[catoff[i] + j];
                        if (vstamp[z] ≠ stamp)
                        {
                            vstamp[z] = stamp;
                            vcnt[z] = 1;
                            touched[nt++] = z;
                        }
                        else
                            vcnt[z]++;
                    }
                bool hit = false;
                for (int j = 0; j < nt && !hit; j++)
                    hit = vcnt[touched[j]] == ncats && l[touched[j]] ≤ q;
                ok = hit;
            }
            for (int q = 0; q ≤ m; q++)
                freq[q] = 0;
            for (int z = 1; z ≤ m; z++)
                freq[l[z]]++;
            int running = 0;
            for (int q = 1; q ≤ m && ok; q++)
            {
                running += freq[q];
                if (running < q)
                    ok = false;
            }
```

listing 656: c#, the same two conditions, arrays instead of stamps (source: icpc/samples/src/Ch13/PF.cs, lines 59-97)

```
  const vstamp = new Int32Array(m + 1);
  const vcnt = new Int32Array(m + 1);
  let ok = true;
  for (let q = 1; q ≤ m && ok; q++) {
    let ncats = 0;
    for (let i = potcat[q]; i ≥ 0; i = catnext[i]) ncats++;
    if (ncats === 0) continue;
    const touched = [];
    for (let i = potcat[q]; i ≥ 0; i = catnext[i])
      for (let j = 0; j < catk[i]; j++) {
        const z = likes[catoff[i] + j];
        if (vstamp[z] =/= q) {
          vstamp[z] = q;
          vcnt[z] = 1;
          touched.push(z);
        } else vcnt[z]++;
      }
    let hit = false;
    for (const z of touched)
      if (vcnt[z] === ncats && low[z] ≤ q) {
        hit = true;
        break;
      }
    ok = hit;
  }
  const freq = new Int32Array(m + 1);
  for (let z = 1; z ≤ m; z++) freq[low[z]]++;
  let running = 0;
  for (let q = 1; q ≤ m && ok; q++) {
    running += freq[q];
    if (running < q) ok = false;
  }
```

listing 657: javascript, the intersection scan and the hit test, the prefix count below (source: icpc/samples-js/src/ch13-pf-herdingcats.mjs, lines 41-72)

```
        vcnt = {}
        ok = True
        for q in range(1, m + 1):
            bs = buckets[q]
            if not bs:
                continue
            ncats = len(bs)
            vcnt.clear()
            for lk in bs:
                for z in lk:
                    vcnt[z] = vcnt.get(z, 0) + 1
            if not any(c == ncats and low[z] ≤ q for z, c in vcnt.items()):
                ok = False
                break
        if ok:
            freq = [0] * (m + 2)
            for z in range(1, m + 1):
                freq[low[z]] += 1
            running = 0
            for q in range(1, m + 1):
                running += freq[q]
                if running < q:
                    ok = False
                    break
        out.append("yes" if ok else "no")
    return "\n".join(out) + "\n"
```

listing 658: python, bucket lists and a dict intersection count, any() for the hit test (source: icpc/samples-py/src/Ch13/pf.py, lines 36-61)

```
    local stamp = 0
    local q = 1
    while q ≤ m and ok do
      local i = potcat[q]
      if i then
        local ncats = 0
        local ii = i
        while ii do
          ncats = ncats + 1
          ii = catnext[ii]
        end
        stamp = stamp + 1
        local touched = {}
        local nt = 0
        ii = i
        while ii do
          local off = catoff[ii]
          for j = 1, catk[ii] do
            local z = likes[off + j]
            if vstamp[z] ~= stamp then
              vstamp[z] = stamp
              vcnt[z] = 1
              nt = nt + 1
              touched[nt] = z
            else
              vcnt[z] = vcnt[z] + 1
            end
          end
          ii = catnext[ii]
        end
        local hit = false
        for j = 1, nt do
          local z = touched[j]
          if vcnt[z] == ncats and L[z] ≤ q then
            hit = true
            break
          end
        end
        ok = hit
      end
      q = q + 1
    end
    if ok then
      local freq = {}
      for zq = 0, m do freq[zq] = 0 end
      for z = 1, m do
        freq[L[z]] = freq[L[z]] + 1
      end
      local running = 0
      for zq = 1, m do
        running = running + freq[zq]
        if running < zq then
```

listing 659: lua, stamped intersection and the two conditions, the verdict assembled per test (source: icpc/samples-lua/ch13_pf.lua, lines 62-113)

The crafted pair of tests pins both directions. In the first, two cats target pot 2 over four pots, L(1) = L(2) = 2 with two unliked varieties free at pot 1, so a placement like 3, 1, 2, 4 exists and the verdict is yes. In the second, three

cats target pots 2 and 3 and like all three varieties between them, so no variety can sit at pot 1 without stopping a cat early, the prefix count fails at q = 1, and the verdict is no. Every suite asserts that yes/no pair, most add the omnivore cat whose liked set stops it at pot 1 while it targets pot 2, and c# adds a single-cat first-pot case. Counts per language: 2, 3, 2, 2, 2, 2, java 2.

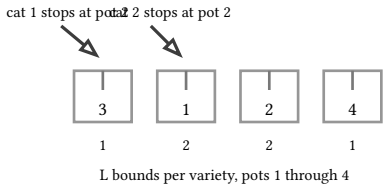


figure 181: the pot row with varieties planted, two cats walking from the left, and the L bounds bracketed beneath their pots

Counts fit int32 everywhere, likes total 5e5, and no language needs more than its default integer.

15.9. problem g, lava moat

A triangulated terrain stretches w by l, every vertex carrying a distinct integer elevation and every triangle interpolating linearly between its corners. A lava moat must cross the terrain from the west border to the east border at one constant elevation, following the level curve, and the task is the shortest feasible moat length, or impossible when no level connects the two borders at all.

The input is a test count t, then per test the extents w and l, the vertex count n, and the triangle count m, then n vertex lines of x, y, and z and m triangle lines of three vertex indices. The problemset pdf bounds t at 10000, w and l at 1e6, n between 4 and 50000, m between n-2 and 2n-6, coordinates at 0 to 1e6 with only the corner vertices on the west and east borders, all positions and all elevations distinct with z at most 1e6, a full triangulation, and the sum of n over tests at 50000. The limit is 4 seconds with a 1e-6 absolute or relative tolerance, the output is one line per test, the length or impossible, and the judge data carries 101 cases for the letter, a single sample and 100 secrets.

Official sample 1, reprinted byte for byte from the judge data pair `G-lavamoat/sample-1.in` and `sample-1.ans`: three terrains in one file, the first admitting no west-east level at all, the second answering 6.708203932, the third 15.849260054, the mixed impossible-plus-float lines that force the letter's all-lines float compare:

input:

```
3
6 6 4 2
0 0 1
6 0 4
6 6 3
0 6 2
1 2 3
1 3 4
6 6 4 2
0 0 1
6 0 2
6 6 4
0 6 3
1 2 3
1 3 4
10 6 7 7
6 1 8
10 0 10
10 6 4
2 6 6
0 6 0
```

```
4 3 11
0 0 7
2 1 7
2 3 1
3 6 1
3 4 6
6 4 5
5 7 6
7 1 6
```

expected output:

```
impossible
6.708203932
15.849260054
```

Recognition: n summed at 50000 with m near 2n, about 1e5 triangle cuts, against the 4 second limit: the offline sweep hangs each cut's lifetime on a segment tree over at most 5e4 level intervals and unwinds every union, O(n log^2 n), about 5e4 × 17^2 ≈ 1.4e7 logged unions.

The cue is the constant elevation: the moat is one level curve crossing west to east, so the whole task lives on the components the cuts form while the level moves, connectivity under edge lifetimes. Problem G is the 2025 face of the component structure and connectivity family. The component structure and connectivity family runs through chapter 11 problem B, chapter 12 problem E, chapter 12 problem H, chapter 13 problem R, and chapter 15 problem E.

Rebuilding the level-curve graph per candidate elevation is the priced-out alternative: 5e4 anchor levels × 1e5 cuts is 5e9 segment insertions, and even sorting events per level pays n^2 log n where the segment tree touches each cut log n times in total.

Shifting the level changes every cut length linearly, so an optimal moat sits at a vertex elevation. At a fixed level each triangle cut contributes a segment joining its two edges that span the level, level curves are chains over edge-crossing nodes, and sweeping the levels bottom-up with offline dynamic connectivity, edge lifetimes hung on a segment tree over level intervals and a union-find that unwinds, tracks each component's total length as a linear function a + b*h plus west and east border flags. The answer for an anchor vertex is the cheapest component touching both borders, O(n log^2 n) (statement, solutions.pdf p. 5). Book 10′s chapter 31, sqrt decomposition and offline queries, builds this offline dynamic connectivity from scratch, the segment tree over time carrying the edge lifetimes and the rollback union-find holding the growing chains. Two solved teams at 270 minutes, and this book agrees with them: the sweep engine is the largest of the year, so every language but java splits it out of the solver file to stay under the 400-line cap, ch13_moat.h plus ch13_rollback.h in c, moat.go beside pg.go, ch13-moat.mjs beside the solver in javascript, ch13_moat.lua plus ch13_rollback.lua, ch13_rollback.py in python, and Moat.cs in c#; java keeps the whole sweep inline in PG.java, the tree's largest file at 511 lines, with only the base dsu escaping to Rollback.java. The sweep scores each leaf twice, once per closed endpoint, crediting border chains through the corner vertices that own the level.

The worked run: trace the model on sample 1, three terrains in one file. Terrain 1: the west border vertices carry elevations 1 and 2, the east border 4 and 3, the open bands [1,2] and [3,4] never share a level, and the first line prints impossible. Terrain 2: every level between 2 and 3 draws the same west-east chain, from the west crossing (0, 3(h-1)) through the diagonal point (2(h-1), 2(h-1)) to the east crossing (6, 3(h-2)), the two segments scaling as (h-1) sqrt(5) and (4-h) sqrt(5), a constant 3 sqrt(5), so the sweep's first both-border candidate, scored at the closed endpoint h = 2, prints 6.708203932. Terrain 3: the borders span [0,7] west and [4,10] east, and in the interval just above 6 the sweep scores its both-border chain at the closed endpoint h = 6, from the west crossing (0, 6/7) over (24/11, 48/11) and through the vertex (2,6) whose elevation is exactly 6, then over (58/7, 36/7) and (8, 7/2) to the

east crossing (10, 4), five cuts of length 4.129870130, 1.646433661, 6.343886660, 1.667516790, and 2.061552813, and no other level's chain beats it, and the trace ends at the printed answer `15.849260054`.

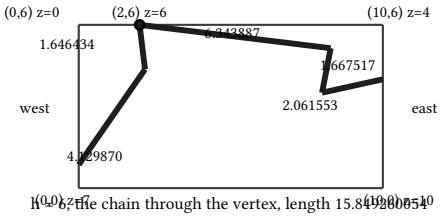


figure 182: terrain 3 in plan view, the corners and the north vertex labeled with elevations, and the level-6 west-east chain through the vertex (2,6) bolded

```
static void credit_and_eval(int s, int side) {
  long double h;
  int gval;
  if (side == 1) {
    h = (long double)zs[s];
    gval = zs[s];
  } else {
    h = (long double)zs[s + 1];
    gval = zs[s + 1];
  }
  int xtn = 0;
  int glo = lowb(gval), ghi = uppb(gval);
  for (int r = glo; r < ghi; r++) {
    int v = zord[r];
    unsigned char cf = 0;
    if (VX[v] == 0)
      cf = 1;
    else if (VX[v] == WD)
      cf = 2;
    if (!cf)
      continue;
    for (int i = vhead[v]; i ≥ 0; i = anxt[i]) {
      int o = EA[aedg[i]] == v ? EB[aedg[i]] : EA[aedg[i]];
      if ((side == 1 && VZ[o] > gval) || (side == 2 && VZ[o] < gval)) {
        int root = ch13_rb_find(par, aedg[i]);
        if (!(xfl[root] & 4)) {
          xfl[root] = 4;
          xt[xtn++] = root;
        }
        xfl[root] |= cf;
      }
    }
  }
  for (int r = lhead; r ≥ 0; r = lnx[r]) {
    long double cand = cA[r] + cB[r] * h;
    if (cand < best)
      best = cand;
  }
  for (int k = 0; k < xtn; k++) {
    int r = xt[k];
    if (((cfl[r] | xfl[r]) & 3) == 3) {
      long double cand = cA[r] + cB[r] * h;
      if (cand < best)
        best = cand;
    }
    xfl[r] = 0;
  }
}
```

listing 660: c, credit_and_eval at one endpoint, border credit stamped per root, both-border components scored as a + b*h (source: icpc/samples-c/src/Ch13/pG.c, lines 87-134)

```
func (g *gState) dfs(node, lo, hi int) {
  markl := len(g.m.lops)
  done := 0
  for i := g.m.th[node]; i ≥ 0; i = g.m.tn[i] {
    g.m.addSeg(g.m.sid[i])
    done++
  }
  if lo == hi {
    if g.armPass {
      g.anchorArmEval(lo)
    } else {
      g.creditAndEval(lo, 1)
      g.creditAndEval(lo, 2)
    }
  } else {
    mid := (lo + hi) / 2
    g.dfs(2*node, lo, mid)
    g.dfs(2*node+1, mid+1, hi)
  }
  for len(g.m.lops) > markl {
    g.m.undoLop()
  }
  g.m.undoUnions(done)
}
```

listing 661: go, the segment-tree descent, add every hanging segment, recurse, undo every union and list op (source: icpc/samples-go/ch13/pg.go, lines 165-188)

```
  static void creditAndEval(int s, int side) {
    double h;
    int gval;
    if (side == 1) {
      h = zs[s];
      gval = zs[s];
    } else {
      h = zs[s + 1];
      gval = zs[s + 1];
    }
    int xtn = 0;
    int glo = lowb(gval), ghi = uppb(gval);
    for (int r = glo; r < ghi; r++) {
      int v = zord[r];
      int cf = 0;
      if (VX[v] == 0)
        cf = 1;
      else if (VX[v] == WD)
        cf = 2;
      if (cf == 0)
        continue;
      for (int i = vhead[v]; i ≥ 0; i = anxt[i]) {
        int o = EA[aedg[i]] == v ? EB[aedg[i]] : EA[aedg[i]];
        if ((side == 1 && VZ[o] > gval) || (side == 2 && VZ[o] < gval)) {
          int root = rb.find(aedg[i]);
          if ((xfl[root] & 4) == 0) {
            xfl[root] = 4;
            xt[xtn++] = root;
          }
          xfl[root] |= cf;
        }
      }
    }
    for (int r = lhead; r ≥ 0; r = lnx[r]) {
      double cand = cA[r] + cB[r] * h;
      if (cand < best)
        best = cand;
    }
    for (int k = 0; k < xtn; k++) {
      int r = xt[k];
      if (((cfl[r] | xfl[r]) & 3) == 3) {
        double cand = cA[r] + cB[r] * h;
        if (cand < best)
          best = cand;
      }
      xfl[r] = 0;
    }
  }
```

listing 662: java, creditAndEval at one endpoint, border credit stamped per root, both-border components scored as a + b*h (source: icpc/samples-java/src/Ch14/PG.java, lines 257-304)

```
        int xtn = 0;
        int glo = LowB(gval), ghi = UppB(gval);
        for (int r = glo; r < ghi; r++)
        {
            int v = _zord[r];
            byte cf = 0;
            if (_vx[v] == 0)
                cf = 1;
            else if (_vx[v] == _wd)
                cf = 2;
            if (cf == 0)
                continue;
            for (int i = _vhead[v]; i ≥ 0; i = _anxt[i])
            {
                int o = _ea[_aedg[i]] == v ? _eb[_aedg[i]] : _ea[_aedg[i]];
                if ((side == 1 && _vz[o] > gval)
                    || (side == 2 && _vz[o] < gval))
                {
                    int root = Moat.Find(_aedg[i]);
                    if ((Moat.Xfl[root] & 4) == 0)
                    {
                        Moat.Xfl[root] = 4;
                        Moat.Xt[xtn++] = root;
                    }
                    Moat.Xfl[root] |= cf;
                }
            }
        }
        for (int r = Moat.Lhead; r ≥ 0; r = Moat.Lnx[r])
        {
            double cand = Moat.CA[r] + Moat.CB[r] * h;
            if (cand < _best)
                _best = cand;
        }
        for (int k = 0; k < xtn; k++)
        {
            int r = Moat.Xt[k];
            if (((Moat.Cfl[r] | Moat.Xfl[r]) & 3) == 3)
            {
                double cand = Moat.CA[r] + Moat.CB[r] * h;
                if (cand < _best)
                    _best = cand;
            }
            Moat.Xfl[r] = 0;
        }
    }

    // through-anchor candidates at level zs[s] in the arm state: every
```

listing 663: c#, the same endpoint scoring over the static Moat arrays (source: icpc/samples/src/Ch13/PG.cs, lines 94-141)

A second pass drops each cut's first leaf so the anchor's legs dangle, and through a vertex the shortest west-east path is the shortest west leg plus the shortest east leg, each leg read off its v-middle triangle's crossing component:

```
function anchorArmEval(s) {
  const gval = zs[s];
  const glo = lowb(gval), ghi = uppb(gval);
  const h = gval;
  for (let r = glo; r < ghi; r++) {
    const v = zord[r];
    let mw = VX[v] === 0 ? 0 : Infinity;
    let me = VX[v] === WD ? 0 : Infinity;
    for (let i = vsh[v]; i ≥ 0; i = vsn[i]) {
      const root = rbFind(vsop[i]);
      const len = SA[vseg[i]] + SB[vseg[i]] * h + cA[root] + cB[root] * h;
      if (cfl[root] & 1 && len < mw) mw = len;
      if (cfl[root] & 2 && len < me) me = len;
    }
    if (mw < Infinity && me < Infinity && mw + me < best) best = mw + me;
  }
}

function dfs() {
  moatDfs(1, 0, nleaf, (leaf) => {
    if (armPass) anchorArmEval(leaf);
    else {
      creditAndEval(leaf, 1);
      creditAndEval(leaf, 2);
    }
  });
}

export function solve(input) {
```

listing 664: javascript, anchorArmEval scoring through-anchor candidates, then the same undo-wrapped dfs (source: icpc/samples-js/src/ch13-pg-lavamoat.mjs, lines 117-145)

```
    def credit_and_eval(self, s, side):
        # score the leaf interval (zs[s], zs[s+1]) at one closed
        # endpoint: side 1 scores h = zs[s] crediting chains that end
        # at a border-corner vertex whose elevation is zs[s], side 2
        # symmetric on zs[s+1] with lower neighbors
        if side == 1:
            h = gval = self.zs[s]
        else:
            h = gval = self.zs[s + 1]
        m = self.m
        xtn = 0
        for r in range(self.lowb(gval), self.uppb(gval)):
            v = self.zord[r]
            cf = 0
            if self.vx[v] == 0:
                cf = 1
            elif self.vx[v] == self.wd:
                cf = 2
            if not cf:
                continue
            i = self.vhead[v]
            while i ≥ 0:
                e = self.aedg[i]
                o = self.eb[e] if self.ea[e] == v else self.ea[e]
                if (side == 1 and self.vz[o] > gval) or (
                    side == 2 and self.vz[o] < gval
                ):
                    root = m.find(e)
                    if not m.xfl[root] & 4:
                        m.xfl[root] = 4
                        m.xt[xtn] = root
                        xtn += 1
                    m.xfl[root] |= cf
                i = self.anxt[i]
        r = m.lhead
        while r ≥ 0:
            cand = m.ca[r] + m.cb[r] * h
            if cand < self.best:
                self.best = cand
            r = m.lnx[r]
        for k in range(xtn):
            r = m.xt[k]
            if (m.cfl[r] | m.xfl[r]) & 3 == 3:
                cand = m.ca[r] + m.cb[r] * h
                if cand < self.best:
                    self.best = cand
            m.xfl[r] = 0
```

listing 665: python, the endpoint scoring again, bisect for the level bounds, arrays from the rollback module (source: icpc/samples-py/src/Ch13/pg.py, lines 36-82)

```
local function anchor_arm_eval(s)
  local gval = zs[s]
  local glo = lowb(gval)
  local ghi = uppb(gval) - 1
  local h = gval
  for r = glo, ghi do
    local v = zord[r]
    local mw = VX[v] == 0 and 0 or HUGE
    local me = VX[v] == WD and 0 or HUGE
    local i = vsh[v]
    while i ≥ 0 do
      local root = rb.find(m.par, vsop[i])
      local len = m.SA[vseg[i]] + m.SB[vseg[i]] * h + m.cA[root] +
                  m.cB[root] * h
      if (m.cfl[root] & 1) ~= 0 and len < mw then
        mw = len
      end
      if (m.cfl[root] & 2) ~= 0 and len < me then
        me = len
      end
      i = vsn[i]
    end
    if mw < HUGE and me < HUGE and mw + me < best then
      best = mw + me
    end
  end
end
```

listing 666: lua, the arm pass, per-anchor west and east legs summed through the crossing components (source: icpc/samples-lua/ch13_pg.lua, lines 130-157)

The crafted terrain is a unit square with corners z = 0, 2, 4, 2 and the 1-3 diagonal. The west border spans elevations 0 to 2 and the east border 2 to 4, so no open level interval touches both, and exactly at level 2 the curve runs (2, 0) to (1, 1) to (0, 2), both segments of length sqrt(2) with the diagonal midpoint interpolating to exactly 2, so the answer is 2.828427125. Every suite pins that plus the impossible variant z = 0, 2, 3, 1 whose borders never coincide, two fixtures per language everywhere, and the judge comparison runs in the all-lines float mode, numeric lines within 1e-4, because the output mixes “impossible” lines with floats printed to 9 or more significant digits.

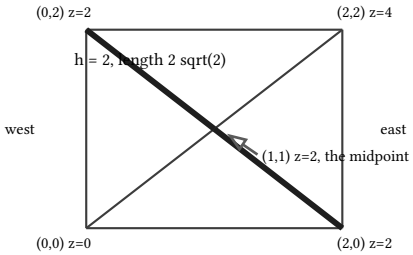


figure 183: the two triangles in plan view, elevations at the corners, and the level-2 polyline through (2,0), (1,1), (0,2) bolded

Squared cut lengths reach 8e12, int64 in c, c#, go and lua integers, exact in javascript’s Number, and c alone carries its lengths in long double for the judge’s 1e-6 tolerance while printing %.9f like the rest. Java carries them in plain double and holds the floats 1e-6 compare, the official sample printing byte-exact at nine decimals, and the c engine’s fixed TREECAP segment-tree stacks became growable arrays there, the lists starting at 1≪22 and doubling on push, so multi-test files cannot overflow the cap.

15.10. problem h, score values

A contact-bridge scoreboard starts at 0, every move adds one of n fixed point values, and any move that would push past the maximum m leaves the score sitting at m. The club must buy digit signs, and the fewest of them able to display every score reachable at any moment, where a 6 sign doubles as a 9 by flipping, so a 9 in a number spends a 6 sign. The report is the per-digit purchase count for the digits 0 through 8.

The input is two integers m and n, then the n point values. The problemset pdf bounds m at 1e18 and n at 10, the values distinct and each between 1 and 1000. The limit is 2 seconds, the output is one “d c” line per digit 0 through 8 in ascending order with zero counts omitted, the digit 9 never appearing because 6 covers it, and the judge data carries 108 cases for the letter, 2 samples and 106 secrets.

Official sample 1, reprinted byte for byte from the judge data pair `H-scorevalues/sample-1.in` and `sample-1.ans`: m = 1000 with the values 60, 100, 222, and 650. The cap itself supplies three 0 signs, 222 with its double 444 and its triple 666 supplies three 2s, 4s, and 6s, 770 = 650 + 120 supplies two 7s, 888 = 4 times 222 three 8s, and no reachable score ever repeats a 1, a 3, or a 5:

input:

```
1000 4
60
100
222
650
```

expected output:

```
0 3
1 1
2 3
3 1
4 3
5 1
6 3
7 2
8 3
```

Recognition: m at 1e18 with only 10 generators of size at most 1000 kills every value-indexed table: the split sieves below about 1e6 or 1e7 and runs a digit dp over 19 positions by g by 10 states, at most about 2e6 sieve marks plus 2e5 dp cells against the 2 second limit.

The cue is the purchase order, the fewest signs able to display every score reachable at any moment: the reachable set is a capped semigroup, and the per-digit maxima come from a hand-built state space of digit position, residue mod g, and tightness against m’s digits. Problem H is the 2025 face of the dp over an engineered state space family. The dp over an engineered state space family spans chapter 11 problem J, chapter 12 problem I, chapter 13 problem S, and chapter 14 problem J.

Enumerating the reachable scores is the priced-out alternative: past the Frobenius region every multiple of g is reachable, so the count approaches 5e17 values below 1e18, and the sieve tier only exists because the digit dp cannot start below the bound.

The reachable scores are 0, the semigroup values capped at m, and m itself, always reachable by repeating the largest value. With a single point value the dp tracks value mod p directly, and with several the large scores are exactly the multiples of g = gcd of the values, so the normalized generators drive an integer knapsack sieve below the split and the digit dp above it. Past the Frobenius region, bounded by the square of the largest generator, at most 1e6 here, every multiple of g is reachable, so the solver sieves the semigroup below a cap and runs a digit dp

above it, value mod g per position tracking the best count of the target digit, with m's own length staying tight along m's digits, a sieve to about 2e6 plus 9 digits by 19 lengths by g by 10 in the dp (statement, solutions.pdf pp. 5-6). Book 10′s chapter 35, combinatorics, carries the frobenius numerical-semigroup bound this split rides on.

The worked run: trace the model on sample 1, m = 1000 with the values 60, 100, 222, and 650. The generators share gcd 2, and m sits inside the sieve tier, so the reachable scores are enumerated outright and the nine counts read off their witnesses, the table. The cap itself is reachable, 1000 = 10 × 100, and supplies three 0 signs, 222 with its multiples 444 and 666 supplies three 2s, 4s, and 6s, 770 = 650 + 2 × 60 supplies two 7s, 888 = 4 × 222 supplies three 8s, and no reachable score repeats a 1, a 3, or a 5, the repeated-1 candidates 110 through 118 all missing from the semigroup and 1100 sitting past the cap, so those digits peak at one sign, and the trace ends at the printed answer `8 3`.

digit	witness score	signs
0	1000 = 10 × 100	3
1	100 = 1 × 100	1
2	222 = 1 × 222	3
3	300 = 5 × 60	1
4	444 = 2 × 222	3
5	500 = 5 × 100	1
6	666 = 3 × 222	3
7	770 = 650 + 2 × 60	2
8	888 = 4 × 222	3

```
    } else {
      // m's length: stay tight along m's own digits
      int tres = 0, tcnt = 0;
      for (int r = 0; r < g; r++)
        dp[r] = -1;
      for (int pos = 0; pos < lm; pos++) {
        for (int r = 0; r < g; r++)
          ndp[r] = -1;
        for (int r = 0; r < g; r++) {
          if (dp[r] < 0)
            continue;
          for (int dig = 0; dig ≤ 9; dig++) {
            int r2 = (int)((r * 10LL + dig) % g);
            int v = dp[r] + hit(d, dig);
            if (v > ndp[r2])
              ndp[r2] = v;
          }
        }
        int lodig = pos == 0 ? 1 : 0;
        for (int dig = lodig; dig < md[pos]; dig++) {
          int r2 = (int)((tres * 10LL + dig) % g);
          int v = tcnt + hit(d, dig);
          if (v > ndp[r2])
            ndp[r2] = v;
        }
        memcpy(dp, ndp, sizeof(int) * (size_t)g);
        tcnt += hit(d, md[pos]);
        tres = (int)((tres * 10LL + md[pos]) % g);
      }
      int best = dp[0];
      if (tres == 0 && tcnt > best)
        best = tcnt;
      if (best > cnt[d])
        cnt[d] = best;
```

listing 667: c, the tight pass at m's length, free digits below the prefix, m itself appended at the end (source: icpc/samples-c/src/Ch13/pH.c, lines 130-163)

```
      for l := 8; l <= lm; l++ {
        if l < lm {
          for r := range dp {
            dp[r] = -1
          }
          for dig := 1; dig <= 9; dig++ {
            if v := hHit(d, dig); v > dp[dig%int(g)] {
              dp[dig%int(g)] = v
            }
          }
          for pos := 1; pos < l; pos++ {
            for r := range ndp {
              ndp[r] = -1
            }
            for r := 0; r < int(g); r++ {
              if dp[r] < 0 {
                continue
              }
              for dig := 0; dig <= 9; dig++ {
                r2 := int((int64(r)*10 + int64(dig)) % g)
                if v := dp[r] + hHit(d, dig); v > ndp[r2] {
                  ndp[r2] = v
                }
              }
            }
            copy(dp, ndp)
          }
          if dp[0] > cnt[d] {
            cnt[d] = dp[0]
          }
        } else {
```

listing 668: go, the same free lengths, residues over g, hit() folding 9 into 6 (source: icpc/samples-go/ch13/ph.go, lines 118-148)

```
    long S = Math.min(m, 10000000L);
    java.util.Arrays.fill(reach, 0, (int) S + 1, (byte) 0);
    reach[0] = 1;
    for (int x = 0; x ≤ S; x++)
      if (reach[x] ≠ 0)
        for (int i = 0; i < n; i++)
          if (x + p[i] ≤ S)
            reach[x + p[i]] = 1;
    java.util.Arrays.fill(cnt, 0);
    // scan reachable scores with incrementally maintained digit counts
    int[] c = {1, 0, 0, 0, 0, 0, 0, 0, 0}; // decimal(0)
    int[] pd = new int[10];
    int ndig = 1;
    for (int x = 0; x ≤ S; x++) {
      if (reach[x] ≠ 0)
        for (int d = 0; d < 9; d++)
          if (c[d] > cnt[d])
            cnt[d] = c[d];
      int pos = 0;
      while (pos < ndig && pd[pos] == 9) {
        pd[pos] = 0;
        c[6]--; // 9 and 0 both merge or map to 6 and 0
        c[0]++;
        pos++;
      }
      if (pos == ndig) {
        pd[pos] = 1;
        ndig++;
        c[1]++;
      } else {
        c[mrg(pd[pos])]--;
        pd[pos]++;
        c[mrg(pd[pos])]++;
      }
    }
```

listing 669: java, the 1e7 cap of the first group, the sieve then walked once with per-digit counts carried incrementally, 9s merging into the 6 tally (source: icpc/samples-java/src/Ch14/PH.java, lines 61-95)

```
            for (int l = 8; l ≤ lm; l++)
            {
                if (l < lm)
                {
                    Array.Fill(dp, -1);
                    for (int dig = 1; dig ≤ 9; dig++)
                    {
                        int r = dig % gi;
                        if (Hit(d, dig) > dp[r])
                            dp[r] = Hit(d, dig);
                    }
                    for (int q = 1; q < l; q++)
                    {
                        Array.Fill(ndp, -1);
                        for (int r = 0; r < gi; r++)
                        {
                            if (dp[r] < 0)
                                continue;
                            for (int dig = 0; dig ≤ 9; dig++)
                            {
                                int r2 = (int)((r * 10L + dig) % g);
                                int v = dp[r] + Hit(d, dig);
                                if (v > ndp[r2])
                                    ndp[r2] = v;
                            }
                        }
                        Array.Copy(ndp, dp, gi);
                    }
                    if (dp[0] > cnt[d])
                        cnt[d] = dp[0];
                }
                else
                {
```

listing 670: c#, the free-length dp, every residue carrying the best digit count (source: icpc/samples/src/Ch13/PH.cs, lines 123-155)

```
    for (let L = 8; L ≤ lm; L++) {
      if (L < lm) {
        dp.fill(-1, 0, g);
        for (let dig = 1; dig ≤ 9; dig++)
          if (hit(d, dig) > dp[dig % g]) dp[dig % g] = hit(d, dig);
        for (let pos = 1; pos < L; pos++) {
          ndp.fill(-1, 0, g);
          for (let r = 0; r < g; r++) {
            if (dp[r] < 0) continue;
            for (let dig = 0; dig ≤ 9; dig++) {
              const r2 = (r * 10 + dig) % g;
              const v = dp[r] + hit(d, dig);
              if (v > ndp[r2]) ndp[r2] = v;
            }
          }
          dp.set(ndp.subarray(0, g));
        }
        if (dp[0] > cnt[d]) cnt[d] = dp[0];
      } else {
```

listing 671: javascript, the free tier compact, m parsed as BigInt because 1e18 exceeds 2^53 (source: icpc/samples-js/src/ch13-ph-scorevalues.mjs, lines 86-104)

The caps diverge by design. C, c#, go, and javascript sieve to 1e7 and start the residue dp at eight digits, python sieves to 1e6 + 1000 and lua to 1e6, both starting the dp at seven digits, which the Frobenius bound of at most 1e6 licenses, and lua walks the sieve once maintaining the per-digit decimal counts incrementally instead of formatting each reachable value. Java joins the 1e7 cap of the first group but borrows lua's walk, carrying the per-digit decimal counts incrementally through the sieve sweep:

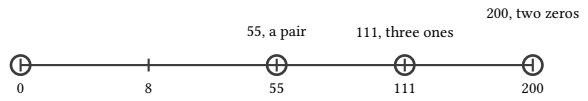
```
        md = [int(ch) for ch in str(m)]
        lm = len(md)
        dp = [0] * g
        ndp = [0] * g
        for d in range(9):
            for ln in range(7, lm + 1):
                if ln < lm:
                    # every l-digit multiple of g is reachable and
                    # under the cap: free dp over residues
                    for r in range(g):
                        dp[r] = -1
                    for dig in range(1, 10):
                        r2 = dig % g
                        if hit(d, dig) > dp[r2]:
                            dp[r2] = hit(d, dig)
                    for _ in range(ln - 1):
                        for r in range(g):
                            ndp[r] = -1
                        for r in range(g):
                            v = dp[r]
                            if v < 0:
                                continue
                            for dig in range(10):
                                r2 = (r * 10 + dig) % g
                                nv = v + hit(d, dig)
                                if nv > ndp[r2]:
                                    ndp[r2] = nv
                        dp, ndp = ndp, dp
                    if dp[0] > cnt[d]:
                        cnt[d] = dp[0]
```

listing 672: python, the free-length residue dp from seven digits on, one row swap per position (source: icpc/samples-py/src/Ch13/ph.py, lines 66-95)

```
local cnt = {}
local c = {}
for d = 0, 8 do
  cnt[d] = 0
  c[d] = 0
end
c[0] = 1 -- decimal(0)
local pd = {0}
local ndig = 1
for x = 0, S do
  if reach[x] then
    for d = 0, 8 do
      if c[d] > cnt[d] then
        cnt[d] = c[d]
      end
    end
  end
  local ppos = 1
  while ppos ≤ ndig and pd[ppos] == 9 do
    pd[ppos] = 0
    c[6] = c[6] - 1 -- 9 and 0 both merge or map to 6 and 0
    c[0] = c[0] + 1
    ppos = ppos + 1
  end
  if ppos > ndig then
    pd[ppos] = 1
    ndig = ppos
    c[1] = c[1] + 1
  else
    c[mrg(pd[ppos])] = c[mrg(pd[ppos])] - 1
    pd[ppos] = pd[ppos] + 1
    c[mrg(pd[ppos])] = c[mrg(pd[ppos])] + 1
  end
end
```

listing 673: lua, the sieve sweep with digit counts carried incrementally, 9s merging into the 6 tally (source: icpc/samples-lua/ch13_ph.lua, lines 68-101)

The crafted case is m = 200 with values 3 and 5. The semigroup is 3, 5, 6, and everything from 8 up, digit 0 peaks in 100 and 200 for 2, digit 1 peaks in 111, which is 3 times 37, for 3, every other digit peaks twice, and 6 never triples because 666 exceeds the cap and 9s count as 6s, so the answer is the nine lines 0 2 through 8 2. Every suite pins that and the unit case m = 1, value 1, answering 0 1 and 1 1, two per language.



8 and up all reachable, so each digit's maximum is a formatting question

figure 184: a strip of reachable scores up to 200, with 100, 111 and 200 highlighted and one repeated-digit pair marked per digit

m reaches 1e18, so c, c#, go, and lua use 64-bit integers, java longs, python is native, and javascript parses m and walks its digits as BigInt while the residues stay small.

15.11. problem i, slot machine

A slot machine carries n wheels, each showing one of n symbols at a time, and the symbols sit in the same cyclic order on every wheel, the wheels merely pointing at different positions on that shared cycle. One action rotates any single wheel any number of steps, and after each action the machine reports k, the count of distinct symbols

currently visible across the wheels. The task is to reach the jackpot, every wheel showing the same symbol so k reads 1, within the round budget.

The interaction opens by reading n, then one line per round holding k, the count of distinct symbols the friend currently sees: the shown symbols never cross the wire, the initial configuration being the judge's private state, fixed in advance and carried by the case file. The problemset pdf bounds n between 3 and 50 and guarantees the initial configuration is not already solved. Each round prints one line holding i and j, with 1 <= i <= n and j any integer, which rotates wheel i by j steps, then reads the new k, and the program exits once k reads 1, all inside 10000 rotations under a 2 second limit. The judge data carries 64 cases for the letter, 2 samples and 62 secrets.

No sample pair can be reprinted the usual way, because this is the set's second structural exception: the problem is interactive, the judge directory holds the two fixed wheel configurations, a five-wheel one and a three-wheel one, plus .interaction transcripts that mark every judge line with '<' and every contestant line with '>', and not one .ans file, so there is nothing to byte-diff and the chapter asserts the emptiness rather than quoting a transcript.

Recognition: n at most 50 wheels under 10000 rotations and 2 seconds makes the budget countable in rounds: the two-phase strategy spends around 3n^2 rotations, the measured worst 8241 at n = 50 sitting near 3.3n^2, inside the cap with room, and every k recount costs O(n).

The cue is the protocol itself, print a rotation, read k, reach the jackpot: the algorithm is the query strategy, feedback steering hidden state toward a goal, not a function of a fixed input. Problem I is the 2025 face of the interactive strategy by feedback family, the only member of its family in this book.

Blind alignment is the priced-out alternative: without the distinctness phase the offsets stay entangled, n^(n-1) alignments at n = 50, and even pairwise probing without undoing every probe costs n^3 = 1.25e5 rounds against the 10000 budget.

Phase one makes all symbols distinct, sweeping each wheel through its positions and keeping the argmax, which works because the other wheels occupy at most n - 1 symbols so a fresh symbol always wins. Phase two recovers the relative offsets, nudging one wheel forward and another back, the count staying at n exactly when the second sits one symbol further along, every probe undone literally so all relations refer to one frozen state, the successor cycle's last edge forced, and n - 1 final rotations align everything, about 3n^2 rounds at n = 50 (statement, solutions.pdf p. 6). Every language exposes the pure driver, solve over a query callback, and the self-check runs crafted configurations through a bundled simulator, the year's ch13_wheels helper, asserting the jackpot, the round budget, and protocol well-formedness. The judge lane drives the same cores through `interactive:2025-i`: the interactor implements the judge side of the protocol, spawning each language's solver core as a child through harness shims under `ref/icpc/driver/`, lua replayed from its judge-mode transcript instead, answering every rotation from its own machine and passing iff the jackpot lands inside 10000 well-formed actions. Java's shim `ref/icpc/driver/2025-i-java/JDriver.java` hosts the package-private solveMachine core over the pdf protocol with an explicit flush after every write and the k = 1 exit at the protocol layer, mirroring the cs and c shims, and reaches the core by reflection because the harness compiles JDriver beside each year's PI in one javac, 2023′s waterworld PI compiling along and never running. The pdf's exit rule, stop forever at the first k = 1, lives in those shims: the strategy cores keep probing when a probe leg lands on the jackpot mid-sweep, the already-distinct sample 2 being the official witness, so the adapters carry the exit the cores omit.

The worked run: trace the crafted fixture, the three wheels at positions 1, 2, 1, because the interactive judge ships transcripts rather than answers, so the trace targets the jackpot the simulator must report. The wheels show symbols 1, 2, and 1, so k reads 2. Phase one sweeps wheel 1 through its three rotations: at symbol 2 the row reads 2, 2, 1 with k = 2, and at symbol 3 it reads 3, 2, 1 with k = 3, the argmax kept, the table, and wheels 2 and 3 sweep and keep their symbols too, either move dropping k back to 2, so after 9 rotations the row is distinct at 3, 2, 1. Phase two probes pairs, rotating one wheel forward and another back, the count staying at 3 exactly when the second sat one symbol further along, every probe undone, and the final n - 1 = 2 rotations align wheels 2 and 3 onto wheel 1′s symbol. The simulator reports k = 1 well inside the 3n^2 + 10 = 37-round fixture budget.

wheel 1 shows	row	k	kept
1	1 2 1	2	start
2	2 2 1	2	no
3	3 2 1	3	yes, the argmax

```
  for (int i = 1; i ≤ n; i++) {
    int best = curk, bestj = 0;
    for (int j = 1; j < n; j++) {
      curk = Q(i, 1);
      if (curk > best) {
        best = curk;
        bestj = j;
      }
    }
    int back = (bestj - (n - 1) + n) % n;
    if (back)
      curk = Q(i, back);
  }
  for (int i = 1; i ≤ n; i++)
    succ[i] = pred[i] = 0;
  int edges = 0;
  for (int d = 1; d < n && edges < n - 1; d++)
    for (int a = 1; a + d ≤ n && edges < n - 1; a++) {
      int b = a + d;
      if (!succ[a]) {
        int k = Q(a, 1); // answer arrives after both legs
        (void)k;
        k = Q(b, -1);
        if (k == n) {
          succ[a] = b;
          pred[b] = a;
          edges++;
        }
        Q(a, -1); // literal undo keeps the state frozen
        Q(b, 1);
      }
      if (!succ[b] && edges < n - 1) {
        int k = Q(b, 1);
        (void)k;
        k = Q(a, -1);
        if (k == n) {
          succ[b] = a;
          pred[a] = b;
          edges++;
        }
        Q(b, -1);
        Q(a, 1);
      }
    }
```

listing 674: c, the distinctness sweep then the pair probes through one function-pointer query, every probe literally undone (source: icpc/samples-c/src/Ch13/pI.c, lines 32-75)

```
  for i := 1; i <= n; i++ {
    best, bestj := curk, 0
    for j := 1; j < n; j++ {
      curk = qry(i, 1)
      if curk > best {
        best, bestj = curk, j
      }
    }
    back := (bestj - (n - 1) + n) % n
    if back != 0 {
      curk = qry(i, back)
    }
  }
  succ := make([]int, n+1)
  pred := make([]int, n+1)
  edges := 0
  for d := 1; d < n && edges < n-1; d++ {
    for a := 1; a+d <= n && edges < n-1; a++ {
      b := a + d
      if succ[a] == 0 {
        qry(a, 1) // the answer arrives after both legs
        if qry(b, -1) == n {
          succ[a], pred[b] = b, a
          edges++
        }
        qry(a, -1) // literal undo keeps the state frozen
        qry(b, 1)
      }
      if succ[b] == 0 && edges < n-1 {
        qry(b, 1)
        if qry(a, -1) == n {
          succ[b], pred[a] = a, b
          edges++
        }
        qry(b, -1)
        qry(a, 1)
      }
    }
  }
```

listing 675: go, phase one and the probe loop over the wheels13 engine (source: icpc/samples-go/ch13/pi.go, lines 42-80)

```
static long solveMachine(int n, int k0, IntBinaryOperator q) {
  long used = 0;
  int curk = k0;
  for (int i = 1; i ≤ n; i++) {
    int best = curk, bestj = 0;
    for (int j = 1; j < n; j++) {
      used++;
      curk = q.applyAsInt(i, 1);
      if (curk > best) {
        best = curk;
        bestj = j;
      }
    }
    int back = ((bestj - (n - 1)) % n + n) % n;
    if (back ≠ 0) {
      used++;
      curk = q.applyAsInt(i, back);
    }
  }
  int[] succ = new int[n + 1], pred = new int[n + 1];
  int edges = 0;
  for (int d = 1; d < n && edges < n - 1; d++)
    for (int a = 1; a + d ≤ n && edges < n - 1; a++) {
      int b = a + d;
      if (succ[a] == 0) {
        used++;
        q.applyAsInt(a, 1); // the answer arrives after both legs
        used++;
        if (q.applyAsInt(b, -1) == n) {
          succ[a] = b;
          pred[b] = a;
          edges++;
        }
        used++;
        q.applyAsInt(a, -1); // literal undo keeps the state frozen
        used++;
        q.applyAsInt(b, 1);
      }
      if (succ[b] == 0 && edges < n - 1) {
        used++;
        q.applyAsInt(b, 1);
        used++;
        if (q.applyAsInt(a, -1) == n) {
          succ[b] = a;
          pred[a] = b;
          edges++;
        }
        used++;
        q.applyAsInt(b, -1);
        used++;
        q.applyAsInt(a, 1);
      }
    }
```

listing 676: java, solveMachine over the IntBinaryOperator callback: the distinctness sweep, then the pair probes, every probe literally undone (source: icpc/samples-java/src/Ch14/PI.java, lines 71-123)

C# is the one language that inlines the simulator as a sealed class instead of shipping a helper file:

```
    // the slot-machine friend: wheel i shows symbol ((pos-1) mod n)+1,
    // a rotation adds its offset mod n, and the friend answers with
    // the count of distinct visible symbols
    private sealed class Wheels
    {
        private readonly int _n;
        private readonly int[] _pos;

        internal Wheels(int n, int[] pos)
        {
            _n = n;
            _pos = new int[n];
            for (int i = 0; i < n; i++)
                _pos[i] = ((pos[i] - 1) % n + n) % n + 1;
        }

        internal int K()
        {
            var seen = new bool[_n + 1];
            int k = 0;
            for (int i = 0; i < _n; i++)
            {
                int s = (_pos[i] - 1) % _n;
                if (!seen[s])
                {
                    seen[s] = true;
                    k++;
                }
            }
            return k;
        }

        internal int Rotate(int i, int j)
        {
            _pos[i - 1] = ((_pos[i - 1] - 1 + j) % _n + _n) % _n + 1;
            return K();
        }
    }
```

listing 677: c#, the Wheels simulator inlined, position p showing symbol ((p-1) mod n) + 1, rotate adding j mod n (source: icpc/samples/src/Ch13/PI.cs, lines 20-57)

```
  for (let i = 1; i ≤ n; i++) {
    let best = curk, bestj = 0;
    for (let j = 1; j < n; j++) {
      curk = qry(i, 1);
      if (curk > best) {
        best = curk;
        bestj = j;
      }
    }
    const back = (((bestj - (n - 1)) % n) + n) % n;
    if (back =/= 0) curk = qry(i, back);
  }
  const succ = new Int32Array(n + 1);
  const pred = new Int32Array(n + 1);
  let edges = 0;
  for (let d = 1; d < n && edges < n - 1; d++)
    for (let a = 1; a + d ≤ n && edges < n - 1; a++) {
      const b = a + d;
      if (succ[a] === 0) {
        qry(a, 1); // the answer arrives after both legs
        if (qry(b, -1) === n) {
          succ[a] = b;
          pred[b] = a;
          edges++;
        }
        qry(a, -1); // literal undo keeps the state frozen
        qry(b, 1);
      }
      if (succ[b] === 0 && edges < n - 1) {
        qry(b, 1);
        if (qry(a, -1) === n) {
          succ[b] = a;
          pred[a] = b;
          edges++;
        }
        qry(b, -1);
        qry(a, 1);
      }
    }
```

listing 678: javascript, the same two phases, the probe pair and its literal undo (source: icpc/samples-js/src/ch13-pi-slotmachine.mjs, lines 29-67)

```
    succ = [0] * (n + 1)
    pred = [0] * (n + 1)
    edges = 0
    for d in range(1, n):
        if edges ≥ n - 1:
            break
        for a in range(1, n - d + 1):
            if edges ≥ n - 1:
                break
            b = a + d
            if not succ[a]:
                qry(a, 1)  # the answer arrives after both legs
                if qry(b, -1) == n:
                    succ[a], pred[b] = b, a
                    edges += 1
                qry(a, -1)  # literal undo keeps the state frozen
                qry(b, 1)
            if not succ[b] and edges < n - 1:
                qry(b, 1)
                if qry(a, -1) == n:
                    succ[b], pred[a] = a, b
                    edges += 1
                qry(b, -1)
                qry(a, 1)
    if edges == n - 1:  # the last edge is forced
        u = v = 0
        for i in range(1, n + 1):
            if not succ[i]:
                u = i
            if not pred[i]:
                v = i
        succ[u] = v
    chain = [0] * (n + 1)
    b, t = succ[1], 0
    while b ≠ 1:
        chain[b] = t + 1
        t += 1
        b = succ[b]
    for i in range(2, n + 1):
        qry(i, n - chain[i])
```

listing 679: python, the probes, the forced last edge, and the alignment chain built from succ (source: icpc/samples-py/src/Ch13/pi.py, lines 38-77)

```
      if not succ[b] and edges < n - 1 then
        Q(b, 1)
        local k = Q(a, -1)
        if k == n then
          succ[b] = a
          pred[a] = b
          edges = edges + 1
        end
        Q(b, -1)
        Q(a, 1)
      end
      a = a + 1
    end
    d = d + 1
  end
  if edges == n - 1 then -- the last edge is forced
    local u, v
    for i = 1, n do
      if not succ[i] then
        u = i
      end
      if not pred[i] then
        v = i
      end
    end
    succ[u] = v
  end
  local chain = {}
  chain[1] = 0
  local b = succ[1]
  local t = 0
  while b ~= 1 do
    t = t + 1
    chain[b] = t
    b = succ[b]
  end
  for i = 2, n do
    Q(i, n - chain[i])
  end
```

listing 680: lua, the reverse probe, the forced edge, and the final alignment rotations (source: icpc/samples-lua/ch13_pi.lua, lines 62-100)

The crafted configurations are n = 3 at positions 1, 2, 1, n = 3 already distinct, and n = 4 alternating 2, 4, 2, 4, and every suite asserts the simulator reports k = 1 within 3n^2 + 10 rounds on each. The c# suite sticks to the three, go and python match it, the c suite carries four checks, java four configurations with the five-wheel official sample among them, javascript ten assertions including protocol bounds and the pinned worst-round sweep, and lua adds a fourth, five-wheel configuration. One cross-language measurement came out of this problem, re-measured under the interactive driver: sweeping the same 383 deterministic configurations up to n = 50 through javascript and c, the per-config round counts agree exactly, zero mismatches over the whole sweep, the worst 8241 at n = 50 in both, and the official inputs measure worst 8045 in every language, so the originally quoted 8037-versus-8045 spread was sampling noise from sweeping different generated config sets, not callback discipline: the strategy is step-identical, and the javascript suite pins the deterministic sweep's worst count.

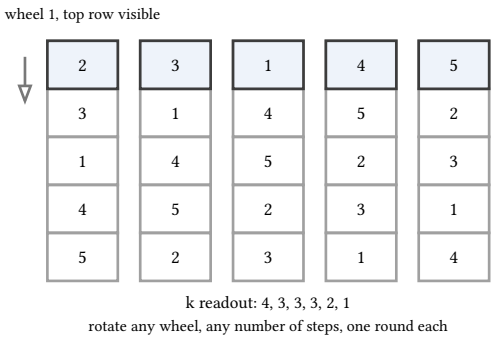


figure 185: five wheels as symbol columns with the visible row highlighted, one rotation arrow, and a distinct-count readout that shrinks toward 1 as rounds pass

n is at most 50 and rotation offsets stay small, so plain integers carry everything and the only divergence is the callback shape, a function pointer in c, a delegate-shaped closure elsewhere, an IntBinaryOperator method reference in java.

15.12. problem j, stacking cups

Cup i is 2i-1 centimeters tall with a 1 centimeter base, diameters grow with i so cup i nests inside cup j exactly when i is below j, and the smallest cup is all base, a single centimeter of sentiment. After washing, the n cups are stacked into one tower: each cup drops centered and upright onto the common axis, a falling cup passes straight through the interiors of wider cups already placed, and the tower's height is the vertical distance from the lowest point of any cup to the highest. The task names a placement order whose final tower spans exactly the favorite number h, using all n cups, or answers impossible.

The input is a single line of two integers, n and h. The problemset pdf bounds n at 2e5 and h at 4e10, under a 2 second limit, and the judge data carries 136 cases for the letter, 2 samples and 134 secrets. The output is the n cup heights in placement order, and the pdf accepts any one valid ordering, which is why the letter judges through validity rather than bytes.

Official sample 1, reprinted byte for byte from the judge data pair `J-stackingcups/sample-1.in` and `sample-1.ans`: four cups and favorite height 9. The judges' order drops the 7 cm cup on the ground, the 3 cm cup inside it onto its floor, the 5 cm cup onto the smaller one's rim spanning 4 to 9, and the 1 cm cup hidden inside on the 5 cm cup's floor at 5, the top rim closing the tower at 9:

input:

```
4 9
```

expected output:

```
7 3 5 1
```

Official sample 2, same source: four cups wanting height 100, past the n^2 = 16 ceiling, so impossible.

input:

```
4 100
```

expected output:

```
impossible
```

Recognition: n at 2e5 cups and h at 4e10 under the 2 second limit rule out both search over orders and any height-indexed table: the achievable set is proved in closed form, 2n-1 through n^2 except the single hole n^2-2, the construction emits the order in one pass, and n at most 5 brute-forces the remainder.

The cue is the exact favorite height, a placement order whose final tower spans exactly h: a characterization question, which heights are achievable at all, answered by an interval-with-one-hole theorem plus a canonical witness. Problem J is the 2025 face of the constructive closed-form characterization family. The constructive closed-form characterization family holds chapter 13 problem U and chapter 14 problem B.

Trying orders is the priced-out alternative, (2e5)! permutations, and even a dynamic program over the values of h needs 4e10 states where the interval theorem answers in one comparison.

The resting law came out of this wave's judge witnesses rather than the statement's gloss: all cups share one axis, and a dropped cup stops with its base bottom at the highest obstacle below it, the maximum over the rim of every narrower placed cup and the interior floor, base bottom plus 1, of every wider placed cup it descends into, or the ground. The original narrowest-floor-only reading, derived from the printed sample story, agreed on every achievable span yet misplaced cups from n = 4 on, and sixteen official witnesses reject it while all 136 pass under the corrected law, so the solvers simulate the corrected one. Achievable heights are exactly 2n-1 through n^2 except n^2-2, because every cup contributes 0, 1, or its full height and the only 2 centimeter reduction from the n^2 stack is tucking the smallest cup inside. The construction recurses, the largest cup goes on top when h is high enough, adding 2k-1, or to the bottom when it is not, lifting the rest by exactly 1, the two ranges overlap from n = 6, and n at most 5 brute-forces permutations in lexicographic order (statement, solutions.pdf pp. 6-7). The judge runs through a validity checker, checker:2025-j, which accepts any permutation of the odd heights whose simulated tower equals h, validated over the 136 official witnesses plus 183 mutated rejections, with "impossible" still exact.

The worked run: trace the model on sample 1, four cups wanting height 9, the odd heights 1, 3, 5, and 7. The judges' order 7 3 5 1 drops cup 7 on the ground first, base bottom 0 and rim at 7. Cup 3 descends through 7′s interior and stops on its floor, the obstacle at 0 + 1 = 1, spanning 1 to 4. Cup 5 meets two obstacles, cup 3′s rim at 4 and cup 7′s floor at 1, and stops on the higher, spanning 4 to 9. Cup 1 descends into both wider cups and meets cup 3′s rim at 4, cup 5′s floor at 4 + 1 = 5, and cup 7′s floor at 1, stops on the highest at 5, hidden inside cup 5 and spanning 5 to 6. The tower runs from the ground 0 to cup 5′s rim 9, exactly h, and the trace ends at the printed answer 7 3 5 1.

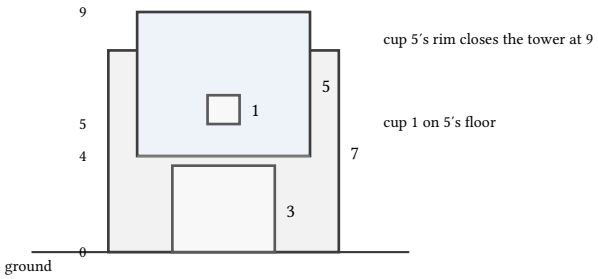


figure 186: cross-section of the sample tower, cup 7 on the ground, cup 3 on its floor, cup 5 on cup 3′s rim, cup 1 hidden inside cup 5 on its floor, the spans labeled up to 9

```c
static void seg_set(long long *tr, int i, long long v, int n) {
  for (i += n; i ≥ 1; i /= 2)
    if (v > tr[i])
      tr[i] = v;
    else
      break; // ancestors already hold the larger value
}

static long long seg_max(long long *tr, int l, int r, int n) { // [l, r]
  long long res = 0;
  for (l += n, r += n + 1; l < r; l /= 2, r /= 2) {
    if (l & 1) {
      if (tr[l] > res)
        res = tr[l];
      l++;
    }
    if (r & 1) {
      r--;
      if (tr[r] > res)
        res = tr[r];
    }
  }
  return res;
}

// drop the cups in the given order and return the tower span
static long long simulate(int n, const int *ord) {
  long long top = 0;
  // leaf for width n sits at index 2n: the floor query [c+1, n]
  // reads it, so the zeroing must cover it
  memset(rimtree, 0, sizeof(long long) * (size_t)(2 * n + 1));
  memset(flrtree, 0, sizeof(long long) * (size_t)(2 * n + 1));
  for (int t = 0; t < n; t++) {
    int c = ord[t];
    long long stop = c > 1 ? seg_max(rimtree, 1, c - 1, n) : 0;
    long long flr = c < n ? seg_max(flrtree, c + 1, n, n) : 0;
    if (flr > stop)
      stop = flr;
    if (stop + 2 * c - 1 > top)
      top = stop + 2 * c - 1;
    seg_set(rimtree, c, stop + 2 * c - 1, n);
    seg_set(flrtree, c, stop + 1, n);
  }
  return top;
}
```

listing 681: c, two max segment trees, the drop as max of narrower rims and wider floors, the width-n leaf zeroed through index 2n (source: icpc/samples-c/src/Ch13/pJ.c, lines 30-74)

The c comment pins a real porting trap: the floor query reads width n's leaf at tree index 2n, so the per-simulation zeroing must cover it, and the c self-check catches the stale leaf instantly. Python is the one port that swaps the segment trees for two max fenwicks, a prefix rim tree and a reversed suffix floor tree:

```go
func (s *jSim) simulate(ord []int) int64 {
  n := s.n
  var top int64
  for i := 0; i <= 2*n; i++ {
    s.rim[i] = 0
    s.flr[i] = 0
  }
  for _, c := range ord {
    var stop int64
    if c > 1 {
      stop = s.segMax(s.rim, 1, c-1)
    }
    if c < n {
      if flr := s.segMax(s.flr, c+1, n); flr > stop {
        stop = flr
      }
    }
    if stop+int64(2*c-1) > top {
      top = stop + int64(2*c-1)
    }
    s.segSet(s.rim, c, stop+int64(2*c-1))
    s.segSet(s.flr, c, stop+1)
  }
  return top
}

// jBrute finds the lexicographically first permutation of 1..n whose
// span equals h; nil when none exists.
```

listing 682: go, the jSim drop loop over its own segment trees (source: icpc/samples-go/ch13/pj.go, lines 57-84)

```java
  // drop the cups in the given order and return the tower span
  static long simulate(int n, int[] ord) {
    long top = 0;
    // leaf for width n sits at index 2n: the floor query [c+1, n]
    // reads it, so the zeroing must cover it
    java.util.Arrays.fill(rimtree, 0, 2 * n + 1, 0L);
    java.util.Arrays.fill(flrtree, 0, 2 * n + 1, 0L);
    for (int t = 0; t < n; t++) {
      int c = ord[t];
      long stop = c > 1 ? segMax(rimtree, 1, c - 1, n) : 0;
      long flr = c < n ? segMax(flrtree, c + 1, n, n) : 0;
      if (flr > stop)
        stop = flr;
      if (stop + 2L * c - 1 > top)
        top = stop + 2L * c - 1;
      segSet(rimtree, c, stop + 2L * c - 1, n);
      segSet(flrtree, c, stop + 1, n);
    }
    return top;
  }
```

listing 683: java, simulate over two iterative max segment trees, fill covering index 2n so the floor query cannot read a stale leaf (source: icpc/samples-java/src/Ch14/PJ.java, lines 67-86)

```
    internal static long Simulate(int n, int[] ord)
    {
        long top = 0;
        // leaf for width n sits at index 2n: the floor query [c+1, n]
        // reads it, so the zeroing must cover it
        Array.Fill(Rim, 0L, 0, 2 * n + 1);
        Array.Fill(Flr, 0L, 0, 2 * n + 1);
        for (int t = 0; t < n; t++)
        {
            int c = ord[t];
            long stop = c > 1 ? SegMax(Rim, 1, c - 1, n) : 0;
            long flr = c < n ? SegMax(Flr, c + 1, n, n) : 0;
            if (flr > stop)
                stop = flr;
            if (stop + 2 * c - 1 > top)
                top = stop + 2 * c - 1;
            SegSet(Rim, c, stop + 2 * c - 1, n);
            SegSet(Flr, c, stop + 1, n);
        }
        return top;
    }

    private static readonly int[] OrdBuf = new int[6];
    private static readonly int[] Ord2 = new int[MaxN];

    // lexicographically first permutation of 1..n whose span equals h,
    // true on success
```

listing 684: c#, Simulate over the same two trees, stop, top, and the two set operations (source: icpc/samples/src/Ch13/PJ.cs, lines 57-83)

```
  // drop the cups in the given order and return the tower span: the
  // leaf for width n sits at index 2n, so the zeroing must cover it.
  simulate(ord) {
    const n = this.n;
    let top = 0;
    this.rim.fill(0, 0, 2 * n + 1);
    this.flr.fill(0, 0, 2 * n + 1);
    for (const c of ord) {
      let stop = 0;
      if (c > 1) stop = this.segMax(this.rim, 1, c - 1);
      if (c < n) {
        const flr = this.segMax(this.flr, c + 1, n);
        if (flr > stop) stop = flr;
      }
      if (stop + (2 * c - 1) > top) top = stop + (2 * c - 1);
      this.segSet(this.rim, c, stop + (2 * c - 1));
      this.segSet(this.flr, c, stop + 1);
    }
    return top;
  }
```

listing 685: javascript, the simulate method, fill(0, 0, 2n + 1) covering the same stale-leaf trap (source: icpc/samples-js/src/ch13-pj-stackingcups.mjs, lines 46-65)

```
    def simulate(self, ord_):
        n = self.n
        rf = self.rfen
        ff = self.ffen
        for i in range(n + 1):
            rf[i] = 0
            ff[i] = 0
        top = 0
        for c in ord_:
            stop = 0
            if c > 1:  # prefix max of rim over widths 1..c-1
                i = c - 1
                while i:
                    if rf[i] > stop:
                        stop = rf[i]
                    i &= i - 1
            if c < n:  # suffix max of flr over widths c+1..n
                j = n - c
                while j:
                    if ff[j] > stop:
                        stop = ff[j]
                    j &= j - 1
            t = stop + 2 * c - 1
            if t > top:
                top = t
            i = c
            while i ≤ n:
                if t > rf[i]:
                    rf[i] = t
                i += i & -i
            j = n + 1 - c
            v = stop + 1
            while j ≤ n:
                if v > ff[j]:
                    ff[j] = v
                j += j & -j
        return top
```

listing 686: python, two fenwick max trees instead, prefix rims against suffix floors, i and i - 1 walking the masks (source: icpc/samples-py/src/Ch13/pj.py, lines 25-61)

```
local function simulate(n, ord)
  local top = 0
  -- leaf for width n sits at index 2n: the floor query [c+1, n]
  -- reads it, so the zeroing must cover it
  rimtree, flrtree = {}, {}
  for i = 1, 2 * n do
    rimtree[i] = 0
    flrtree[i] = 0
  end
  for t = 1, n do
    local c = ord[t]
    local stop = c > 1 and seg_max(rimtree, 1, c - 1, n) or 0
    local flr = c < n and seg_max(flrtree, c + 1, n, n) or 0
    if flr > stop then
      stop = flr
    end
    local rt = stop + 2 * c - 1
    if rt > top then
      top = rt
    end
    seg_set(rimtree, c, rt, n)
    seg_set(flrtree, c, stop + 1, n)
  end
  return top
end

local ordbuf

-- lexicographically first permutation of 1..n whose span equals h
```

listing 687: lua, the drop loop over the year's second segment-tree idiom (source: icpc/samples-lua/ch13_pj.lua, lines 52-80)

The crafted case is n = 3, h = 6. Brute force in lexicographic order finds (1, 3, 2): cup 1 sits on the ground, cup 3 rests on its rim spanning 1 to 6, and cup 2 stops on cup 1's rim hidden inside 3, so the output is "1 5 3". All seven suites pin the same four fixtures, that one, n = 3 h = 7 hitting the n^2-2 hole for "impossible", the single cup "1", and n = 2 h = 3 nesting the small cup for "3 1". The emitted order is re-simulated before printing in every port, and lua's own assembly of the final order moved to a membership set during the wave, its worst judge case dropping to half a second.

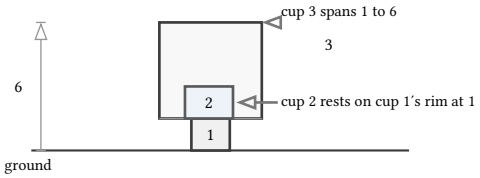


figure 187: cross-section of the crafted tower, cup 1 on the ground, cup 3 resting on its rim, cup 2 hidden inside 3, the spans labeled up to 6

h and n^2 reach 4e10, int64 in c, c#, go, and lua, java longs, python native, and javascript's Number is exact below 2^53, so no BigInt is needed.

15.13. problem k, treasure map

Captain Blackbeard's hypsometric map covers a rectangular stretch of ocean floor, an n by m array of depth marks over an (n-1) by (m-1) grid of unit squares, and time has left only k of the marks legible. There is no single natural interpolation across a unit square, so the mapmaker's rule matters: whichever of the two diagonal triangulations of a square you interpolate over, the result must come out the same, and the known region carries no islets, every

depth non-negative. The task decides whether the legible depths extend to a full map obeying the rule, and if so reports the smallest depth the treasure cell can carry.

The input is five integers, n, m, k, and the treasure cell's two coordinates, then k lines of a cell's coordinates and depth. The problemset pdf bounds n and m at 3e5, k at 3e5 with no duplicated cells and depths from 0 to 1e9. The limit is 4 seconds, the output is one integer or impossible, and the judge data carries 55 cases for the letter, 5 samples and 50 secrets.

Official sample 1, reprinted byte for byte from the judge data pair `K-treasuremap/sample-1.in` and `sample-1.ans`: a 3 by 3 map with five legible depths, all tied into one component through row 2 and column 3, so the treasure at (1,1) is the forced same-component sum, r1 + c1 equal to 3, immune to shifting:

input:

```
3 3 5 1 1
1 3 1
3 3 2
2 3 3
2 2 4
2 1 5
```

expected output:

```
3
```

Official sample 4, same source: four legible depths closing a cycle through rows 1 and 2 and columns 1 and 3, opposite corners summing 8 against 6, and no r_i + c_j map can say both:

input:

```
3 3 4 3 2
2 1 2
2 3 3
1 3 4
1 1 5
```

expected output:

```
impossible
```

Recognition: n and m at 3e5 with k at 3e5 against the 4 second limit: the row-column graph holds at most n + m = 6e5 nodes and k = 3e5 edges, and one breadth-first sweep per component decides feasibility, treasure depth, and shifting all at once, linear work.

The cue is the interpolation rule, whichever diagonal triangulation you interpolate over the result must come out the same: that forces every depth to the form r_i + c_j, pairwise additive constraints with exactly one free value per component, fixed by any known cell and propagated. Problem K is the 2025 face of the one free value per component family. One free value per component is the trick behind chapter 13 problem P and chapter 14 problem G.

Filling the grid square by square is the priced-out alternative: (n-1) × (m-1) runs to 9e10 cells before any consistency check, and a linear program over the same cells pays a solver where six-int arithmetic per edge settles it.

The two interpolations agree on a square exactly when opposite corners sum equally, which over the whole grid forces every depth to the form r_i + c_j. Build the bipartite row-column graph with known cells as edges, one value per component is free and the rest are forced by breadth-first search, contradictions answering impossible.

Per component the invariant a + b, the minimum row value plus the minimum column value, survives shifting, the map is feasible exactly when every component has a + b at least 0, and the treasure is the fixed sum when its row and column share a component, the shifted bridge sum when they do not, one side's shift when the other is unconstrained, else 0, all O(n + m + k) (statement, solutions.pdf p. 7).

The worked run: trace the model on sample 1, a 3 by 3 map with five legible depths and the treasure at (1,1). The known cells tie everything into one component: column 3 carries (1,3), (3,3), and (2,3), and row 2 reaches (2,2) and (2,1). Seeding the free value r2 = 0 forces c3 = 3 - 0 = 3, c2 = 4 - 0 = 4, and c1 = 5 - 0 = 5, then r1 = 1 - 3 = −2 and r3 = 2 - 3 = −1. The treasure's row and column share the component, so its depth is the fixed sum r1 + c1 = −2 + 5 = 3, immune to any shift, and the trace ends at the printed answer 3.

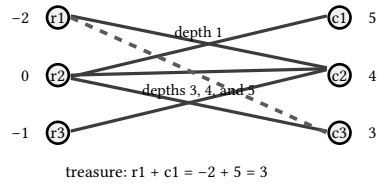


figure 188: the sample's row-column graph, the five known cells as edges, the seeded r2 = 0 with the forced values beside their nodes, and the treasure query dashed between r1 and c1

```c
int nc = 0, feasible = 1;
for (int s = 1; s ≤ nn && feasible; s++) {
  if (ehead[s] < 0 || comp[s])
    continue;
  nc++;
  int qh = 0, qt = 0;
  comp[s] = nc;
  if (s ≤ n) {
    rval[s] = 0;
    minr[nc] = 0;
    minc[nc] = (long long)4e18;
  } else {
    cval[s] = 0;
    minc[nc] = 0;
    minr[nc] = (long long)4e18;
  }
  queue_[qt++] = s;
  while (qh < qt && feasible) {
    int u = queue_[qh++];
    long long ubase = u ≤ n ? rval[u] : cval[u];
    for (int e = ehead[u]; e ≥ 0; e = enxt[e]) {
      int v = eto[e];
      long long want = eval_[e] - ubase;
      if (comp[v]) {
        long long have = v ≤ n ? rval[v] : cval[v];
        if (have ≠ want)
          feasible = 0;
        continue;
      }
      comp[v] = nc;
      if (v ≤ n) {
        rval[v] = want;
        if (want < minr[nc])
          minr[nc] = want;
      } else {
        cval[v] = want;
        if (want < minc[nc])
          minc[nc] = want;
      }
      queue_[qt++] = v;
    }
  }
  if (minr[nc] + minc[nc] < 0)
    feasible = 0;
}
```

listing 688: c, the component bfs forcing values edge by edge, min row and min column carried per component (source: icpc/samples-c/src/Ch13/pK.c, lines 65-109)

```
    if s <= n {
      rval[s] = 0
      minr[nc] = 0
      minc[nc] = 4e18
    } else {
      cval[s] = 0
      minc[nc] = 0
      minr[nc] = 4e18
    }
    queue[qt] = s
    qt++
    for qh < qt && feasible {
      u := queue[qh]
      qh++
      ubase := cval[u]
      if u <= n {
        ubase = rval[u]
      }
      for e := ehead[u]; e ≥ 0; e = enxt[e] {
        v := eto[e]
        want := eval[e] - ubase
        if comp[v] ≠ 0 {
          have := cval[v]
          if v <= n {
            have = rval[v]
          }
          if have ≠ want {
            feasible = false
          }
          continue
        }
        comp[v] = nc
        if v <= n {
          rval[v] = want
          if want < minr[nc] {
            minr[nc] = want
          }
        } else {
          cval[v] = want
          if want < minc[nc] {
            minc[nc] = want
          }
        }
        queue[qt] = v
        qt++
      }
    }
    if minr[nc]+minc[nc] < 0 {
```

listing 689: go, the seeded bfs with the want = depth - base forcing rule (source: icpc/samples-go/ch13/pk.go, lines 60-107)

```
    int nc = 0;
    boolean feasible = true;
    for (int s = 1; s ≤ nn && feasible; s++) {
      if (ehead[s] < 0 || comp[s] ≠ 0)
        continue;
      nc++;
      int qh = 0, qt = 0;
      comp[s] = nc;
      if (s ≤ n) {
        rval[s] = 0;
        minr[nc] = 0;
        minc[nc] = (long) 4e18;
      } else {
        cval[s] = 0;
        minc[nc] = 0;
        minr[nc] = (long) 4e18;
      }
      queue[qt++] = s;
      while (qh < qt && feasible) {
        int u = queue[qh++];
        long ubase = u ≤ n ? rval[u] : cval[u];
        for (int e = ehead[u]; e ≥ 0; e = elink[e]) {
          int v = edest[e];
          long want = eval[e] - ubase;
          if (comp[v] ≠ 0) {
            long have = v ≤ n ? rval[v] : cval[v];
            if (have ≠ want)
              feasible = false;
            continue;
          }
          comp[v] = nc;
          if (v ≤ n) {
            rval[v] = want;
            if (want < minr[nc])
              minr[nc] = want;
          } else {
            cval[v] = want;
            if (want < minc[nc])
              minc[nc] = want;
          }
          queue[qt++] = v;
        }
      }
      if (minr[nc] + minc[nc] < 0)
        feasible = false;
    }
    if (!feasible)
```

listing 690: java, the component bfs forcing values edge by edge, min row and min column carried per component (source: icpc/samples-java/src/Ch14/PK.java, lines 76-122)

```
        long ans;
        if (comp[tx] != 0 && comp[n + ty] != 0)
        {
            if (comp[tx] == comp[n + ty])
                ans = rval[tx] + cval[n + ty];
            else
                ans = rval[tx] - minr[comp[tx]] + cval[n + ty]
                    - minc[comp[n + ty]];
        }
        else if (comp[tx] != 0)
            ans = rval[tx] - minr[comp[tx]];
        else if (comp[n + ty] != 0)
            ans = cval[n + ty] - minc[comp[n + ty]];
        else
            ans = 0;
        return ans.ToString() + "\n";
    }
```

listing 691: c#, the four treasure cases, same component, bridge, one-sided shift, free (source: icpc/samples/src/Ch13/PK.cs, lines 116-132)

```
  for (let s = 1; s ≤ nn && feasible; s++) {
    if (ehead[s] < 0 || comp[s] =/= 0) continue;
    nc++;
    let qh = 0, qt = 0;
    comp[s] = nc;
    if (s ≤ n) {
      rval[s] = 0;
      minr[nc] = 0;
      minc[nc] = 4e18;
    } else {
      cval[s] = 0;
      minc[nc] = 0;
      minr[nc] = 4e18;
    }
    queue[qt++] = s;
    while (qh < qt && feasible) {
      const u = queue[qh++];
      const ubase = u ≤ n ? rval[u] : cval[u];
      for (let e = ehead[u]; e ≥ 0; e = enxt[e]) {
        const v = eto[e];
        const want = eval_[e] - ubase;
        if (comp[v] =/= 0) {
          const have = v ≤ n ? rval[v] : cval[v];
          if (have =/= want) feasible = false;
          continue;
        }
        comp[v] = nc;
        if (v ≤ n) {
          rval[v] = want;
          if (want < minr[nc]) minr[nc] = want;
        } else {
          cval[v] = want;
          if (want < minc[nc]) minc[nc] = want;
        }
        queue[qt++] = v;
      }
    }
    if (minr[nc] + minc[nc] < 0) feasible = false;
  }
  if (!feasible) return "impossible\n";
  let ans = 0;
  const cx = comp[tx], cy = comp[n + ty];
  if (cx =/= 0 && cy =/= 0) {
    if (cx === cy) ans = rval[tx] + cval[n + ty];
    else ans = rval[tx] - minr[cx] + (cval[n + ty] - minc[cy]);
  } else if (cx =/= 0) ans = rval[tx] - minr[cx];
  else if (cy =/= 0) ans = cval[n + ty] - minc[cy];
  return `${ans}\n`;
}
```

listing 692: javascript, the bfs and the treasure dispatch, values in Float64Array (source: icpc/samples-js/src/ch13-pk-treasuremap.mjs, lines 49-97)

```
    for st in range(1, n + m + 1):
        if not adj[st] or comp[st]:
            continue
        nc += 1
        comp[st] = nc
        if st ≤ n:
            rval[st] = 0
            minr.append(0)
            minc.append(4 * 10**18)
        else:
            cval[st] = 0
            minc.append(0)
            minr.append(4 * 10**18)
        queue = [st]
        qh = 0
        while qh < len(queue):
            u = queue[qh]
            qh += 1
            ubase = rval[u] if u ≤ n else cval[u]
            for v, h in adj[u]:
                want = h - ubase
                if comp[v]:
                    have = rval[v] if v ≤ n else cval[v]
                    if have ≠ want:
                        feasible = False
                    continue
                comp[v] = nc
                if v ≤ n:
                    rval[v] = want
                    if want < minr[nc]:
                        minr[nc] = want
                else:
                    cval[v] = want
                    if want < minc[nc]:
                        minc[nc] = want
                queue.append(v)
        if minr[nc] + minc[nc] < 0:
            feasible = False
        if not feasible:
            break
    if not feasible:
        return "impossible\n"
```

listing 693: python, the bfs with minr and minc per component, the feasibility verdict at the end (source: icpc/samples-py/src/Ch13/pk.py, lines 34-75)

```
  while s ≤ nn and feasible do
    if ehead[s] ≥ 0 and comp[s] == 0 then
      nc = nc + 1
      local qh, qt = 1, 1
      comp[s] = nc
      if s ≤ n then
        rval[s] = 0
        minr[nc] = 0
        minc[nc] = 4e18
      else
        cval[s] = 0
        minc[nc] = 0
        minr[nc] = 4e18
      end
      queue_[qt] = s
      qt = qt + 1
      while qh < qt and feasible do
        local u = queue_[qh]
        qh = qh + 1
        local ubase = u ≤ n and rval[u] or cval[u]
        local e = ehead[u]
        while e ≥ 0 do
          local v = eto[e]
          local want = eval_[e] - ubase
          if comp[v] ~= 0 then
            local have = v ≤ n and rval[v] or cval[v]
            if have ~= want then
              feasible = false
            end
          else
            comp[v] = nc
            if v ≤ n then
              rval[v] = want
              if want < minr[nc] then
                minr[nc] = want
              end
            else
              cval[v] = want
              if want < minc[nc] then
                minc[nc] = want
              end
            end
            queue_[qt] = v
            qt = qt + 1
          end
          e = enxt[e]
```

listing 694: lua, the while-loop bfs, 4e18 sentinels on the unused side (source: icpc/samples-lua/ch13_pk.lua, lines 69-114)

The crafted map is 3 rows by 4 columns with four known depths. They split into two components, {row 1, column 1} and {rows 2 and 3, columns 2 and 4}, with a + b equal to 3 and 4, both non-negative, so the map is feasible. The treasure sits at row 2, column 1, one side in each component, and the bridge gives r2 - a2 plus c1 - b1 = 2 + 0 = 2, with an explicit witness shifting component 2 to rows 2 and 0. Every suite pins answer 2, and every suite adds the unit square whose cycle contradicts, 1 + 2 against 3 + 4, for “impossible”, with c also carrying a same-component treasure case as its second fixture. Counts per language: 3, 2, 2, 2, 2, 3, java 3.

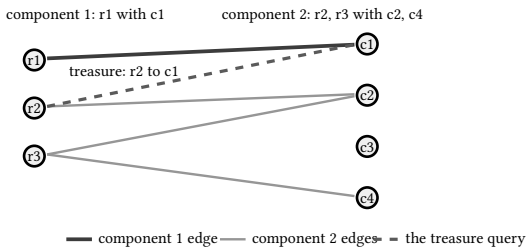


figure 189: rows and columns as the two banks of a bipartite graph, the two components by line weight, the treasure cell a dashed bridge between them

Forced values drift up to k times 1e9, about 3e14, so c, c#, and go carry int64, java longs read through its sign-aware scanner, lua's integers are 64-bit, python is native, and javascript keeps the forced values in a Float64Array, exact because 3e14 sits far below 2^53.

15.14. problem l, walking on sunshine

The walk runs from the contest point to the awards point across a plane dotted with n shaded rectangles, axis-aligned and pairwise disjoint, neither touching nor crossing. Any movement with a southward component hurts, adding its length to the hurting distance, unless it happens inside shade, while horizontal movement and northward movement are always free, and the task is the smallest total hurting distance.

The input is n and the four walk coordinates, contest then awards, then n rectangle lines of x1, y1, x2, y2. The problemset pdf bounds n at 1e5, every coordinate and both walk endpoints within +-1e6, and the rectangles non-touching and non-intersecting. The limit is 2 seconds with a 1e-7 absolute or relative tolerance, the output is one line, and the judge data carries 59 cases for the letter, 6 samples and 53 secrets, the sample answers printing one decimal.

Official sample 1, reprinted byte for byte from the judge data pair `L-walkingonsunshine/sample-1.in` and `sample-1.ans`: contest at (1,7), awards at (5,1), a descent of 6 past two rectangles whose y-bands contribute the low sliver of [6,9] clipped to 6 through 7 and the whole of [3,5], 3 of the descent in shade and 3 hurting:

input:

```
2 1 7 5 1
3 6 5 9
2 3 6 5
```

expected output:

```
3.0
```

Recognition: n at 1e5 rectangles under the 2 second limit: sorting the y-projections and merging costs n log n ≈ 1.7e6 comparisons in one pass, and the arithmetic on the walk's own span is constant, so the budget is trivial.

The cue is the asymmetry sentence, horizontal movement and northward movement are always free: x never matters, the walk collapses onto the y-axis, and the answer is one descent minus one union of sorted intervals, a single sweep over a sorted axis. Problem L is the 2025 face of the event sweep over a sorted axis family. The same event sweep over a sorted axis cue drives chapter 11 problem A, chapter 11 problem H, chapter 12 problem D, and chapter 12 problem J.

Building the rectangle geometry is the priced-out alternative: pairwise checks over the rectangles cost n^2 = 1e10 crossings, and any shortest-path model over the free space pays far more than the one merge pass the collapse licenses.

It is a trick question: x never matters because horizontal movement is free, so the walk collapses onto the y-axis, total southward travel is exactly max(0, yc - ya), and shade at height y is any rectangle whose y-interval covers y,

entered by walking horizontally into its x-range first. The answer is that descent minus the length of the union of the rectangle y-projections clipped to the walk's y range, a sort by lower bound and one merge pass, O(n log n) (statement, solutions.pdf p. 8). Every port is the same eight statements, so the listings show the whole algorithm each time and the comparison is one of spelling.

The worked run: trace the model on sample 1. The walk runs from (1,7) to (5,1), so the total southward travel is 7 - 1 = 6 and the relevant y-range is [1,7]. The first rectangle's y-band [6,9] clips to [6,7], length 1, the second's [3,5] lies wholly inside, length 2, and the two never touch, so the sorted merge holds them apart and the union length is 3. The hurting distance is 6 - 3 = 3, and the trace ends at the printed answer `3.0`.

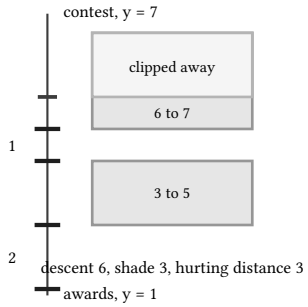


figure 190: the y-axis track, contest at 7 and awards at 1, the band 6 to 9 clipped at the contest's level, the band 3 to 5 whole, and the two unshaded gaps summing to 3

```
long long drop = yc > ya ? yc - ya : 0;
long long cover = 0;
if (drop > 0 && m > 0) {
  qsort(iv, (size_t)m, sizeof(Iv), cmp_iv);
  long long lo = iv[0].y1, hi = iv[0].y2;
  for (long long i = 1; i < m; i++) {
    if (iv[i].y1 > hi) {
      long long a = lo > ya ? lo : ya, b = hi < yc ? hi : yc;
      if (b > a)
        cover += b - a;
      lo = iv[i].y1;
      hi = iv[i].y2;
    } else if (iv[i].y2 > hi)
      hi = iv[i].y2;
  }
  long long a = lo > ya ? lo : ya, b = hi < yc ? hi : yc;
  if (b > a)
    cover += b - a;
}
long long ans = drop - cover;
if (ans < 0)
  ans = 0;
char *out = malloc(16);
snprintf(out, 16, "%.1f\n", (double)ans);
```

listing 695: c, drop, merge, clip, subtract, print %.1f (source: icpc/samples-c/src/Ch13/pL.c, lines 59-82)

```
  drop := 0
  if yc > ya {
    drop = yc - ya
  }
  cover := 0
  if drop > 0 && len(iv) > 0 {
    sort.Slice(iv, func(a, b int) bool {
      if iv[a][0] != iv[b][0] {
        return iv[a][0] < iv[b][0]
      }
      return iv[a][1] < iv[b][1]
    })
    clip := func(lo, hi int) int {
      a, b := lo, hi
      if a < ya {
        a = ya
      }
      if b > yc {
        b = yc
      }
      if b > a {
        return b - a
      }
      return 0
    }
    lo, hi := iv[0][0], iv[0][1]
    for i := 1; i < len(iv); i++ {
      if iv[i][0] > hi {
        cover += clip(lo, hi)
        lo, hi = iv[i][0], iv[i][1]
      } else if iv[i][1] > hi {
        hi = iv[i][1]
      }
    }
    cover += clip(lo, hi)
  }
  ans := drop - cover
  if ans < 0 {
    ans = 0
```

listing 696: go, the sort.Slice comparator and the clip closure (source: icpc/samples-go/ch13/pl.go, lines 33-71)

```java
static String solve(String in) {
  src = in;
  sp = 0;
  long n = nextLong();
  nextLong(); // xc: x never matters
  long yc = nextLong();
  nextLong(); // xa
  long ya = nextLong();
  long[][] iv = new long[(int) Math.max(n, 1)][2];
  int m = 0;
  for (int i = 0; i < n; i++) {
    nextLong();
    long y1 = nextLong();
    nextLong();
    long y2 = nextLong();
    if (y1 > y2) {
      long t = y1;
      y1 = y2;
      y2 = t;
    }
    iv[m][0] = y1;
    iv[m][1] = y2;
    m++;
  }
  long drop = Math.max(0, yc - ya);
  long cover = 0;
  if (drop > 0 && m > 0) {
    java.util.Arrays.sort(iv, 0, m, (a, b) -> a[0] != b[0]
        ? Long.compare(a[0], b[0]) : Long.compare(a[1], b[1]));
    long lo = iv[0][0], hi = iv[0][1];
    for (int i = 1; i < m; i++) {
      if (iv[i][0] > hi) {
        long a = Math.max(lo, ya), b = Math.min(hi, yc);
        if (b > a)
          cover += b - a;
        lo = iv[i][0];
        hi = iv[i][1];
      } else if (iv[i][1] > hi) {
        hi = iv[i][1];
      }
    }
    long a = Math.max(lo, ya), b = Math.min(hi, yc);
    if (b > a)
      cover += b - a;
  }
  long ans = Math.max(0, drop - cover);
  return String.format(Locale.ROOT, "%.1f\n", (double) ans);
}
```

listing 697: java, drop, merge, clip, subtract, one String.format at Locale.ROOT with .1f (source: icpc/samples-java/src/Ch14/PL.java, lines 42-89)

```
        long drop = yc > ya ? yc - ya : 0;
        long cover = 0;
        if (drop > 0 && n > 0)
        {
            var idx = new int[n];
            for (int i = 0; i < n; i++)
                idx[i] = i;
            Array.Sort(idx, (a, b) => lo0[a] != lo0[b]
                ? lo0[a].CompareTo(lo0[b]) : hi0[a].CompareTo(hi0[b]));
            long lo = lo0[idx[0]], hi = hi0[idx[0]];
            for (int i = 1; i < n; i++)
            {
                if (lo0[idx[i]] > hi)
                {
                    cover += Clip(lo, hi, ya, yc);
                    lo = lo0[idx[i]];
                    hi = hi0[idx[i]];
                }
                else if (hi0[idx[i]] > hi)
                    hi = hi0[idx[i]];
            }
            cover += Clip(lo, hi, ya, yc);
        }
        long ans = drop - cover;
        if (ans < 0)
            ans = 0;
        return ans.ToString("F1",
            System.Globalization.CultureInfo.InvariantCulture) + "\n";
    }

    private static long Clip(long lo, long hi, long ya, long yc)
    {
        long a = lo < ya ? ya : lo;
        long b = hi > yc ? yc : hi;
        return b > a ? b - a : 0;
    }
```

listing 698: c#, the merge over an index sort, Clip as a two-line helper (source: icpc/samples/src/Ch13/PL.cs, lines 35-70)

```
  const drop = yc > ya ? yc - ya : 0;
  let cover = 0;
  const clip = (lo, hi) => {
    const a = Math.max(lo, ya);
    const b = Math.min(hi, yc);
    return b > a ? b - a : 0;
  };
  if (drop > 0 && iv.length > 0) {
    iv.sort((a, b) => (a[0] !== b[0] ? a[0] - b[0] : a[1] - b[1]));
    let lo = iv[0][0], hi = iv[0][1];
    for (let i = 1; i < iv.length; i++) {
      if (iv[i][0] > hi) {
        cover += clip(lo, hi);
        lo = iv[i][0];
        hi = iv[i][1];
      } else if (iv[i][1] > hi) hi = iv[i][1];
    }
    cover += clip(lo, hi);
  }
  const ans = Math.max(0, drop - cover);
  return `${ans.toFixed(1)}\n`;
```

listing 699: javascript, clip as an arrow function, toFixed(1) at the end (source: icpc/samples-js/src/ch13-pl-walkingonsunshine.mjs, lines 23-43)

```
    drop = yc - ya if yc > ya else 0
    cover = 0
    if drop > 0 and iv:
        iv.sort()
        lo, hi = iv[0]
        for y1, y2 in iv[1:]:
            if y1 > hi:
                a = lo if lo > ya else ya
                b = hi if hi < yc else yc
                if b > a:
                    cover += b - a
                lo, hi = y1, y2
            elif y2 > hi:
                hi = y2
        a = lo if lo > ya else ya
        b = hi if hi < yc else yc
        if b > a:
            cover += b - a
    ans = drop - cover
    if ans < 0:
        ans = 0
    return f"{ans:.1f}\n"
```

listing 700: python, tuple sort, the merge loop, the f-string with .1f (source: icpc/samples-py/src/Ch13/pl.py, lines 25-46)

```
local drop = yc > ya and yc - ya or 0
local cover = 0
if drop > 0 and m > 0 then
  table.sort(iv, function(a, b)
    if a[1] ~= b[1] then return a[1] < b[1] end
    return a[2] < b[2]
  end)
  local lo, hi = iv[1][1], iv[1][2]
  for i = 2, m do
    if iv[i][1] > hi then
      local a = lo > ya and lo or ya
      local b = hi < yc and hi or yc
      if b > a then
        cover = cover + b - a
      end
      lo, hi = iv[i][1], iv[i][2]
    elseif iv[i][2] > hi then
      hi = iv[i][2]
    end
  end
  local a = lo > ya and lo or ya
  local b = hi < yc and hi or yc
  if b > a then
    cover = cover + b - a
  end
end
local ans = drop - cover
if ans < 0 then
  ans = 0
end
return string.format("%.1f\n", ans)
```

listing 701: lua, table.sort with a comparator, string.format("%.1f") (source: icpc/samples-lua/ch13_pl.lua, lines 54-84)

The crafted walk descends from y = 8 to y = 2 past two rectangles whose y-intervals clip to the ranges 6 through 8 and 3 through 5, union length 4, so the hurting distance is 6 - 4 = 2 and the answer prints "2.0". Every suite pins that plus the northbound no-shade case "0.0". The solvers keep %.1f everywhere while the judge comparison runs in float mode: the official sample answers print one decimal but the secret answers print bare integers, so an exact byte diff would fail more than half the cases, and compare.json carries 2025/L as float, exact on integral values either way.

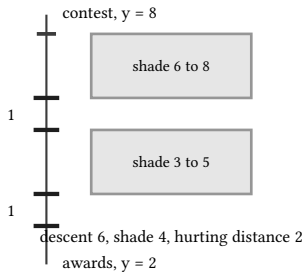


figure 191: the y-axis as a vertical track, contest and awards marked, the two shaded bands crossing it, and the unshaded gaps summing to 2

Every coordinate is an integer and the answer is an integer printed with one decimal, so the seven languages differ only in print spelling.

15.15. across the seven languages

The 2025 set pushes the seven ports in different directions, and the closing table lines them up. Every language ships all twelve solvers. Judge verdicts: all twelve problems ran 874 official cases each in c, go, java, c#, javascript, python, and lua, all green, B judging through its validity checker and I through the interactive driver, both closing the phase 4 ledger, and J's judge runs through the validity checker described in its section.

language	2025 solver shape	self-checks	notes
c	12 solvers in samples-c/src/Ch13 plus ch13_match.h, ch13_wheels.h, ch13_moat.h, ch13_rollback.h	30 CHECK assertions	long double lengths in G, function-pointer queries in I
go	12 solvers in samples-go/ch13 plus match.go, moat.go, wheels.go, cmd/ch13 dispatch	28 table cases and one exhaustive sweep	the only brute-force equivalence test, problem A over every n up to 6
java	12 solvers in samples-java/src/Ch14 plus Match.java and Rollback.java, 1960 solver lines over 103 helper lines	33 ok checks	double lengths and growable tree stacks in G, solveMachine hosted by the ref/icpc/driver/2025-i-java shim in I
c#	12 solvers in samples/src/Ch13 plus Moat.cs, tests in samples/tests/Ch13	34 facts	inlines the B matcher and the I simulator instead of helpers
javascript	12 solvers in samples-js/src plus ch13-match.mjs, ch13-moat.mjs, ch13-wheels.mjs	36 assertions	BigInt only where m reaches 1e18, in H
python	12 solvers in samples-py/src/Ch13 plus ch13_match.py, ch13_rollback.py, ch13_wheels.py	27 checks	golden section in C, fenwicks in J, sieve cut to 1e6 in H
lua	12 solvers in samples-lua plus ch13_match.lua, ch13_moat.lua, ch13_rollback.lua, ch13_wheels.lua	30 named checks	engine split keeps every file under 400 lines, incremental digit counts in H

sources: ICPC Foundation, icpc.global and worldfinals.icpc.global, the 2025 problemset at worldfinals.icpc.global/problems/2025/finals/problems/problemset.pdf and the official solutions at icpc.global/worldfinals/problems/2025-ICPC-World-Finals/2025-ICPC-Solutions.pdf, both accessed 2026-09-14 and cached under `ref/icpc/` with sha256 digests in `ref/icpc/INDEX.md`. The solutions pdf is the algorithm and complexity reference for every walkthrough above. The 2025 set postdates training data, so every statement fact, sample derivation, and judge behavior in this chapter comes from the cached pdfs and the local judge runs, never from memory. Judge data

under `ref/icpc/2025/data/` is local-only with its secret cases never quoted, and the official sample pairs reprinted in the sections above are byte-for-byte its sample-N.in and sample-N.ans files, attributed per section. The data verifies the solvers through `ref/icpc/verify-data.ps1`, 874 official inputs, 874 judged verdicts per language over all twelve problems in c, go, java, c#, javascript, python, and lua, with B carrying no answers at all and judging through its checker, I interactive through the driver, and J checked by validity. The committed self-check suites measured for this chapter are 30 CHECK assertions in the c files, 34 facts in the c# tests, 28 table-driven go cases plus the exhaustive A sweep, 33 ok checks in the java tree's 14 Ch14 files, 12 solvers plus Match.java and Rollback.java, through `tools/run-java-samples.ps1`, 36 assertions in the javascript tests, 27 python checks, and 30 named lua checks, 218 checks in total, each counted in its own suite's unit. The java tree alone follows the post-renumber chapter numbers, its 2025 solvers in Ch14 beside the siblings' historical Ch13 dirs, and its interactive leg runs through the harness shim `ref/icpc/driver/2025-i-java`.

16. fast big arithmetic

Six finals, 68 problems, and not one accepted submission needed more than schoolbook multiply or small-divisor division: no C or Lua year solver loads the toolbox bigint, and the native-stack languages stay in the one-to-three limb regime where a 64-bit word already carries the answer. This chapter builds the missing ladder anyway, because the education is the point: a dynamic bigint core, karatsuba, exact ntt convolution, complex fft in a plain and a packed-split path, and full long division by Knuth's algorithm D, from scratch in all seven languages, one algorithm per section in the fixed order c, go, java, c#, javascript, python, lua. Every suite chains all four multipliers to the same decimal string on shared random operands, and the measured section reports this machine's crossovers, dated 2026-09-19 and 2026-10-06.

16.1. the dynamic bigint core

Sign-magnitude over base one billion, dynamic in every language. A limb holds nine decimal digits, 0 through 999999999, limbs sit least significant first, and the sign lives in its own cell as +1 or −1. Zero canonicalizes to sign +1 with zero limbs, and every operation normalizes on the way out: strip high zero limbs first, then fix the sign, so no result ever carries a phantom leading limb. Comparison reads sign, then limb count, then limb values from the top, and decimal io accepts a leading '-' and nothing else, no '+', no leading zeros, "0" for zero. Parsing cuts the decimal string into nine-digit chunks from the right, so the chapter's 23-digit fixture 12345678901234567890123 parses to exactly three limbs, 12345, 678901234, 567890123 from most to least significant, and printing is the reverse, top limb bare, every limb below it padded to nine digits.

The C toolbox of book 11, ch 2 (the icpc world finals, the c toolbox) already carries a bigint, but its twelve fixed slots, 108 digits of headroom, cannot host a karatsuba recursion that needs dynamically sized halves down the tree, and its zero convention, sign +1 with one zero limb, disagrees with the Lua toolbox's sign-0 zero of book 11, ch 9 (the icpc world finals, the lua toolbox), so chapter 16 grows one new canonical core rather than importing either sibling. The language mappings keep contest discipline: C packs `struct {int sign; int n; unsigned limb[4096];}` with `BIG_MAX 4096` limbs, 36864 decimal digits, static arrays, assert guards, no malloc. C# builds a sealed class over `uint[]`, go a struct of sign plus `[]uint32`, java a nested Big class over an explicit-capacity `int[]`, javascript keeps limbs as plain Number, each under 2^30 and exact in f64, with every exact limb product routed through BigInt because 999999999 squared passes 2^53, python keeps an int list, lua integer table entries.

The dry run: the C suite's round-trip and carry facts walk the 23-digit fixture and the 20-nine addend, `ch14_bigint_test.c` asserting the limb layout, both printed strings, and `r.n = 3` on either result, the same family every suite pins.

1. Parse cuts `12345678901234567890123` nine digits at a time from the right: `567890123`, then `678901234`, top limb `12345`, three limbs under sign +1.
2. Print reverses the walk, top limb bare, both limbs below padded to nine digits, and the same 23 come back.
3. The carry opener parses `999999999999999999` to limbs 999999999, 999999999, 99, then adds 1.
4. Limb 0: 999999999 + 1 = 1000000000, writes 0, carries 1. Limb 1 repeats the rollover, writes 0, carries 1.
5. Limb 2: 99 + 1 = 100, the new top, so the result limbs are 0, 0, 100.
6. Printing pads the two low limbs to nine zeros each, and the string lands `100000000000000000000`, a 1 with 20 zeros.
7. The zero side canonicalizes the other way, `a - a` leaving sign +1 with zero limbs, while "+5" and "007" die at parse.

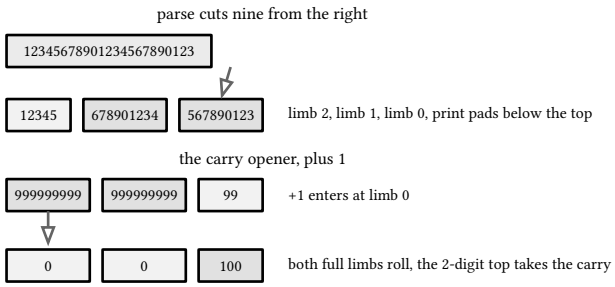


figure 192: the run as a sequence, the string cut from the right into three limbs then printed back top-bare, the carry opener rolling over beneath

The three-limb round trip and the 100-topped rollover are what the checks read, and the listings below build the core in seven languages.

```c
#define BIG_MAX 4096
#define BIG_BASE 1000000000u
#define BIG_BASE4 10000u
#define BIG_STR_MAX (9 * BIG_MAX + 2)
#define BIG4_MAX (9 * BIG_MAX / 4 + 2)

enum {
  BIG_OK = 0,
  BIG_ERR_PARSE = 1,
  BIG_ERR_RANGE = 2,
  BIG_ERR_DIV0 = 3,
};

typedef struct {
  int sign; // +1 or -1, zero always carries +1
  int n;    // limb count, 0..BIG_MAX, no high zero limbs
  unsigned limb[BIG_MAX];
} Big;

static inline void big_norm(Big *r) {
  while (r->n > 0 && r->limb[r->n - 1] == 0)
    r->n--;
  if (r->n == 0)
    r->sign = 1;
}

static inline void big_zero(Big *r) {
  r->sign = 1;
  r->n = 0;
}

static inline int big_is_zero(const Big *a) { return a->n == 0; }

static inline void big_copy(Big *r, const Big *a) {
  r->sign = a->sign;
  r->n = a->n;
  memcpy(r->limb, a->limb, (size_t)a->n * sizeof(unsigned));
}
```

listing 702: c, the base constants, the sign-magnitude struct over 4096 static limbs, and the normalize that strips high zeros then fixes the sign (source: icpc/samples-c/src/Ch14/ch14_bigint.h, lines 15-52)

```go
const base = 1000000000 // 9 decimal digits per limb

const baseDigits = 9

// Big is a sign-magnitude integer over base-1e9 little-endian limbs.
// Zero canonicalizes to sign +1 with no limbs. Every operation normalizes.
type Big struct {
  sign int
  limb []uint32
}

// normalize strips high zero limbs and fixes the sign of zero.
func (b Big) normalize() Big {
  i := len(b.limb)
  for i > 0 && b.limb[i-1] == 0 {
    i--
  }
  b.limb = b.limb[:i]
  if len(b.limb) == 0 {
    b.sign = 1
  }
  return b
}
```

listing 703: go, the Big struct and normalize, zero collapsing to sign +1 with no limbs (source: icpc/samples-go/ch14/bigint.go, lines 9-31)

```java
// the shared magnitude holder: sign +1/-1, little-endian base-1e9
// limbs in an explicit-capacity array (java's answer to py's dynamic
// list), len the live limb count, zero = len 0 with sign +1. the
// (sign, limb) constructor requires an exact array with no stale
// cells past the magnitude: randBig and fromString guarantee it
public static final class Big {
  int sign;
  int len;
  int[] limb;

  public Big(int sign, int[] limb) {
    this.sign = limb.length == 0 ? 1 : sign;
    this.limb = limb;
    this.len = limb.length;
    while (this.len > 0 && limb[this.len - 1] == 0)
      this.len--;
    if (this.len == 0)
      this.sign = 1;
  }

  public Big(int cap) {
    this.sign = 1;
    this.limb = new int[cap];
    this.len = 0;
  }

  Big() {
    this.sign = 1;
    this.limb = new int[0];
    this.len = 0;
  }
}
```

listing 704: java, the nested Big core, sign plus a live limb count over an explicit-capacity int array, the constructor itself the normalization contract (source: icpc/samples-java/src/Ch15/Bigint.java, lines 377-408)

```
/// <summary>
/// Chapter 14 dynamic big integer: sign-magnitude, base B = 1e9 limbs stored
/// little-endian (limb values 0..999999999), zero canonicalized to sign +1
/// with zero limbs. The constructor normalizes every operation's result.
/// Split across Karatsuba.cs, NttMul.cs, FftMul.cs and DivMod.cs.
/// </summary>
public sealed partial class Big
{
    private const uint Base = 1000000000;

    public static Big Zero { get; } = new(1, Array.Empty<uint>());

    public int Sign { get; }

    public IReadOnlyList<uint> Limbs => _limb;

    public int Length => _limb.Length;

    private readonly uint[] _limb;

    private Big(int sign, uint[] limb)
    {
        int n = limb.Length;
        while (n > 0 && limb[n - 1] == 0)
            n--;
        Sign = n == 0 ? 1 : sign < 0 ? -1 : 1;
        _limb = n == limb.Length ? limb : limb[..n];
    }
```

listing 705: c#, the sealed Big class whose constructor is the normalization contract, every factory routed through it (source: icpc/samples/src/Ch14/FastArith.cs, lines 6-33)

```
// chapter 14 core: the dynamic sign-magnitude bigint every other ch14
// module builds on. base 1e9 limbs, little-endian, limb values 0..999999999
// held as plain Number (under 2^30, exact in f64), but every exact limb
// product routes through BigInt because 999999999^2 passes 2^53 and Number
// would silently round it; Number() is called only after the product is
// already exact again (reduced below the base)

export const BASE = 1000000000n;
const BASE_DIGITS = 9;

// strip high zeros, then fix sign: zero is sign +1 with 0 limbs
export function make(sign, limb) {
  let n = limb.length;
  while (n > 0 && limb[n - 1] === 0) n--;
  limb.length = n;
  return n === 0 ? { sign: 1, limb } : { sign, limb };
}
```

listing 706: javascript, Number limbs under 2^30, BigInt at every exact product, make as the normalize gate (source: icpc/samples-js/src/ch14-bigint.mjs, lines 1-17)

```python
# the chapter 14 bigint core: dynamic sign-magnitude over base 1e9 int
# lists, little-endian, zero canonicalized to sign +1 with no limbs,
# plus the shared lcg ladder generator and the base 1e4 regroup helpers
# the transforms consume
B = 1000000000
B4 = 10000
MASK64 = (1 << 64) - 1
LCG_MULT = 6364136223846793005
LCG_INC = 1442695040888963407
LCG_SEED = 0x9E3779B97F4A7C15
LADDER = (1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233)


class Big:
    # sign-magnitude: sign is +1 or -1, limb holds base 1e9 digits
    # little-endian with high zeros stripped; zero is sign +1, no limbs
    __slots__ = ("sign", "limb")

    def __init__(self, sign, limb):
        limb = list(limb)
        while limb and limb[-1] == 0:
            limb.pop()
        self.limb = limb
        self.sign = 1 if not limb else sign
```

listing 707: python, base, the shared lcg constants, the size ladder, and the normalizing Big constructor (source: icpc/samples-py/src/Ch14/ch14_bigint.py, lines 1-24)

```lua
-- icpc ch14: the dynamic bigint core for fast big arithmetic.
-- sign-magnitude over base-1e9 limbs, little-endian, limb values
-- 0..999999999. zero canonicalizes to sign +1 with zero limbs and every
-- operation normalizes. ch02's fixed 12 slots and ch07's sign-0 zero stay
-- untouched: a karatsuba recursion needs dynamically sized limbs.
local T = require("lib")

local M = {}
M.BASE = 1000000000
M.SEED = 0x9E3779B97F4A7C15
M.LADDER = {1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233}

local function norm(b)
  local n = #b.limbs
  while n > 0 and b.limbs[n] == 0 do
    b.limbs[n] = nil
    n = n - 1
  end
  if n == 0 then b.sign = 1 end
  return b
end
```

listing 708: lua, base-1e9 limbs in a plain table, norm trimming high limbs and pinning zero to sign +1 (source: icpc/samples-lua/ch14_bigint.lua, lines 1-21)

The fixture families pin the contract in every suite: the 23-digit round trip through those exact three limbs, the carry opener `9999999999999999999 + 1 = 1000000000000000000` whose result limbs are 0, 0, 100, zero canonicalized from `a - a`, "−0" and "007" and "+5" refused at parse, signed subtraction across signs, and the compare ladder sign first then length then top limb. Random agreement runs on the shared 64-bit lcg, `x' = (6364136223846793005 * x + 1442695040888963407) mod 2^64` seeded `0x9E3779B97F4A7C15`, one draw per limb reduced mod 1e9, top limb forced

nonzero, over the size ladder 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233 limbs, 4 pairs per class, against native integers where the language has them. Counts: C 237 checks, javascript 12 tests, python 293 checks, lua 7 named checks, beside the anchor `123456789 * 987654321 = 121932631112635269` that chapter 2 already pins on the schoolbook.

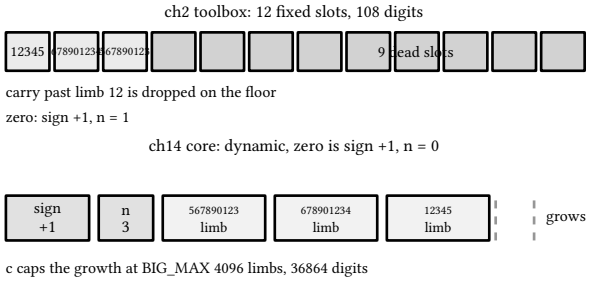


figure 193: memory layout: the chapter 16 core as one sign cell plus exactly the limbs it needs, beside the ch2 toolbox's twelve fixed slots

16.2. karatsuba multiplication

Split both operands at m, half the larger limb count rounded down: `a = a1 * B^m + a0` and `b = b1 * B^m + b0`. The product expands to `a0*b0 + (a0*b1 + a1*b0) * B^m + a1*b1 * B^2m`, four half-size multiplies in the schoolbook grid. Karatsuba's move computes the middle term without ever forming its two pieces: `z0 = a0*b0`, `z2 = a1*b1`, and `z1 = (a0+a1)*(b0+b1) - z0 - z2`, three multiplies where the grid crosses four, and the recurrence `T(n) = 3T(n/2) + O(n)` lands at n^1.585 against n^2. The operand sums cost one limb of headroom per level, the recursion runs on magnitudes only with the sign applied once at the end, and the base case falls back to schoolbook below KARA_CUT limbs. The cut is a measured constant, not a constant of nature, and section 6 retunes it per language: go and lua moved it, the other four held the contract's 24. The C slice here is that base case, the schoolbook every recursion bottoms out in.

The dry run: the C suite's carry stressor pins `(10^30 - 1) x (10^30 + 1) = 10^60 - 1` through `big_mul_kara`, sixty nines asserted by string compare, one recursion level hand-derived here because the four-limb pair rides the schoolbook base case under `KARA_CUT` 24, the recursion itself pinned by the CUT-1, CUT, CUT+1 and ladder agreements.

1. The operands parse to 4 limbs each: a = 10^30 - 1 carries 999999999 three times with top 999, b = 10^30 + 1 carries 1, 0, 0 with top 1000.
2. m = 4 / 2 = 2. The halves: a0 = 999999999 + 999999999 x 10^9 = 10^18 - 1, a1 = 999999999 + 999 x 10^9 = 10^12 - 1, b0 = 1 + 0 = 1, b1 = 0 + 1000 x 10^9 = 10^12.
3. z2 = (10^12 - 1) x 10^12 = 10^24 - 10^12, and z0 = (10^18 - 1) x 1 = 10^18 - 1.
4. The sums cost their headroom: a0 + a1 = 10^18 + 10^12 - 2 and b0 + b1 = 10^12 + 1, so (a0 + a1) x (b0 + b1) = 10^30 + 10^24 + 10^18 - 10^12 - 2.
5. z1 = 10^30 + 10^24 + 10^18 - 10^12 - 2 - (10^18 - 1) - (10^24 - 10^12) = 10^30 - 1, one multiply covering the grid's two crossed products.
6. The recombination at offsets 2m, m, 0 telescopes: (10^24 - 10^12) x 10^36 + (10^30 - 1) x 10^18 + (10^18 - 1) = 10^60 - 1.
7. The suite's compare reads sixty nines against the memset-built expectation, digit for digit.

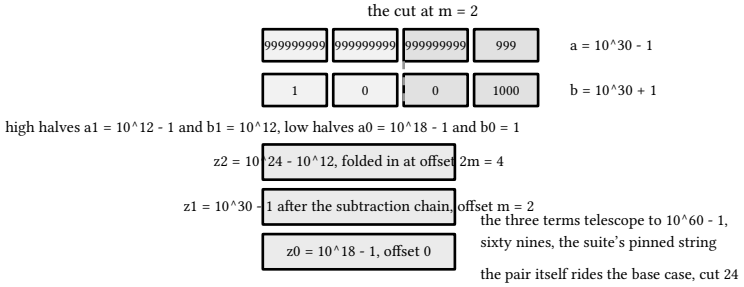


figure 194: one level on the carry stressor, both operands cut at m = 2, three multiplies, the recombination telescoping to sixty nines

The pinned sixty nines close the run, and the listings below build the split and the recombination in seven languages.

```c
// schoolbook on raw limb arrays, out[0..la+lb) fully written
static inline void big_school_raw(const unsigned *a, int la, const unsigned *b,
                                  int lb, unsigned *out) {
  memset(out, 0, (size_t)(la + lb) * sizeof(unsigned));
  for (int i = 0; i < la; i++) {
    u64 carry = 0;
    for (int j = 0; j < lb; j++) {
      u64 cur = out[i + j] + (u64)a[i] * b[j] + carry;
      out[i + j] = (unsigned)(cur % BIG_BASE);
      carry = cur / BIG_BASE;
    }
    out[i + lb] = (unsigned)carry;
  }
}

static inline void big_mul_school(Big *r, const Big *a, const Big *b) {
  assert(a->n + b->n <= BIG_MAX);
  if (a->n == 0 || b->n == 0) {
    big_zero(r);
    return;
  }
  unsigned prod[2 * BIG_MAX];
  big_school_raw(a->limb, a->n, b->limb, b->n, prod);
  memcpy(r->limb, prod, (size_t)(a->n + b->n) * sizeof(unsigned));
  r->n = a->n + b->n;
  r->sign = a->sign * b->sign;
  big_norm(r);
}
```

listing 709: c, the schoolbook base case on raw limb arrays, column accumulate in u64, carry written one limb past each row (source: icpc/samples-c/src/Ch14/ch14_bigint.h, lines 202-229)

```
#define KARA_CUT 24
#define KARA_DEPTH 12
#define KARA_HALF (BIG_MAX / 2 + 4)

static unsigned kara_sa[KARA_DEPTH][KARA_HALF];
static unsigned kara_sb[KARA_DEPTH][KARA_HALF];
static unsigned kara_z1[KARA_DEPTH][2 * KARA_HALF];

// a[0..la) * b[0..lb) into out[0..la+lb), fully written, la ≥ 1, lb ≥ 1
static void kara_rec(const unsigned *a, int la, const unsigned *b, int lb,
                     unsigned *out, int depth) {
  assert(la ≥ 1 && lb ≥ 1 && depth ≥ 0 && depth < KARA_DEPTH);
  int mx = la > lb ? la : lb;
  if (la < KARA_CUT || lb < KARA_CUT || 2 * la ≤ lb || 2 * lb ≤ la) {
    big_school_raw(a, la, b, lb, out);
    return;
  }
  int m = mx / 2;
  int tot = la + lb;
  kara_rec(a, m, b, m, out, depth);                          // z0
  kara_rec(a + m, la - m, b + m, lb - m, out + 2 * m, depth); // z2

  // a0 + a1 and b0 + b1, one limb of headroom each
  unsigned *sa = kara_sa[depth], *sb = kara_sb[depth];
  int lsa = 0, lsb = 0;
  u64 c = 0;
  for (int i = 0; i < (la - m > m ? la - m : m); i++) {
    u64 s = c;
    if (i < m)
      s += a[i];
    if (i < la - m)
      s += a[m + i];
    sa[lsa++] = (unsigned)(s % BIG_BASE);
    c = s / BIG_BASE;
  }
  if (c)
    sa[lsa++] = (unsigned)c;
  c = 0;
  for (int i = 0; i < (lb - m > m ? lb - m : m); i++) {
    u64 s = c;
    if (i < m)
      s += b[i];
    if (i < lb - m)
      s += b[m + i];
    sb[lsb++] = (unsigned)(s % BIG_BASE);
    c = s / BIG_BASE;
  }
  if (c)
    sb[lsb++] = (unsigned)c;
```

listing 710: c, the karatsuba core, cut 24, depth-indexed static scratch, no malloc, the split with the z0 and z2 recursions, both operand sums costing a limb of headroom, the z1 chain and the offset add past the slice (source: icpc/samples-c/src/Ch14/ch14_karatsuba.h, lines 12-60)

```
// KaraCut is the karatsuba base case: below this many limbs the recursion
// falls back to schoolbook. The contract's initial 24 was retuned to 48
// after measuring schoolbook vs karatsuba at 8/16/24/32/48/64 limbs (first
// karatsuba win at 64 limbs) and sweeping cut 24/32/48/64 at the ladder
// tail: 48 is fastest at 128 limbs and within noise of the best at 233.
const KaraCut = 48

// MulKara returns a*b by karatsuba on magnitudes, sign applied at the end.
// Split at m = max(la,lb)/2: z0 = a0*b0, z2 = a1*b1,
// z1 = (a0+a1)*(b0+b1) - z0 - z2, result = z2*B^2m + z1*B^m + z0.
func MulKara(a, b Big) Big {
  a, b = a.normalize(), b.normalize()
  return Big{sign: a.sign * b.sign, limb: mulKaraMag(a.limb, b.limb)}.normalize()
}

func mulKaraMag(u, v []uint32) []uint32 {
  if len(u) < KaraCut || len(v) < KaraCut {
    return mulSchoolMag(u, v)
  }
  m := len(u)
  if len(v) > m {
    m = len(v)
  }
  m /= 2
  a0, a1 := u[:min(m, len(u))], u[min(m, len(u)):]
  b0, b1 := v[:min(m, len(v))], v[min(m, len(v)):]
  z0 := mulKaraMag(a0, b0)
  z2 := mulKaraMag(a1, b1)
  z1 := magSub(magSub(mulKaraMag(magAdd(a0, a1), magAdd(b0, b1)), z0), z2)
  out := make([]uint32, len(u)+len(v))
  magAddInto(out, magTrim(z0), 0)
  magAddInto(out, magTrim(z1), m)
  magAddInto(out, magTrim(z2), 2*m)
  return out
}
```

listing 711: go, the measured cut comment, the recursion, and magAddInto folding z2, z1, z0 back at offsets 2m, m, 0 (source: icpc/samples-go/ch14/karatsuba.go, lines 3-37)

```
// karatsuba on magnitudes: z0 and z2 land at arena offsets 0 and 2m
// of the product, z1 = (a0+a1)(b0+b1) - z0 - z2 at offset m, then
// the three pieces addAt into the output at strides 0, m, 2m
static void kmul(int[] a, int ao, int la, int[] b, int bo, int lb,
    int[] out, int oo) {
  for (int i = 0; i < la + lb; i++)
    out[oo + i] = 0;
  if (la == 0 || lb == 0)
    return;
  if (la ≤ CUT && lb ≤ CUT) {
    school(a, ao, la, b, bo, lb, out, oo);
    return;
  }
  int m = Math.max(la, lb) / 2;
  int a0 = Math.min(la, m), a1 = la - a0;
  int b0 = Math.min(lb, m), b1 = lb - b0;
  int base = ap;
  int z0p = ap;
  ap += a0 + b0;
  int z2p = ap;
  ap += a1 + b1;
  int sa = Math.max(a0, a1) + 1, sb = Math.max(b0, b1) + 1;
  int asp = ap;
  ap += sa;
  int bsp = ap;
  ap += sb;
  int z1p = ap;
  ap += sa + sb;
  kmul(a, ao, a0, b, bo, b0, arena, z0p);
  kmul(a, ao + a0, a1, b, bo + b0, b1, arena, z2p);
  addArr(a, ao, a0, ao + a0, a1, arena, asp, sa);
  addArr(b, bo, b0, bo + b0, b1, arena, bsp, sb);
  kmul(arena, asp, sa, arena, bsp, sb, arena, z1p);
  int z0l = normLen(arena, z0p, a0 + b0);
  int z2l = normLen(arena, z2p, a1 + b1);
  int[] zlen = {sa + sb};
  subArr(arena, z1p, zlen, arena, z0p, z0l);
  subArr(arena, z1p, zlen, arena, z2p, z2l);
  int z1l = zlen[0];
  addAt(out, oo, la + lb, arena, z0p, z0l, 0);
  addAt(out, oo, la + lb, arena, z1p, z1l, m);
  addAt(out, oo, la + lb, arena, z2p, z2l, 2 * m);
  ap = base;
}
```

listing 712: java, cut 24 held, the recursion over arena windows travelling as array, offset, length triples, one shared scratch with offsets pushed and popped, zero allocation, the pieces folding back at offsets 2m, m, 0 (source: icpc/samples-java/src/Ch15/Karatsuba.java, lines 94-137)

```
    private static uint[] MulMagKara(uint[] a, uint[] b)
    {
        if (a.Length < KARA_CUT || b.Length < KARA_CUT)
            return MulMagSchool(a, b);
        int m = (a.Length > b.Length ? a.Length : b.Length) / 2;
        uint[] a0 = LowMag(a, m), a1 = HighMag(a, m);
        uint[] b0 = LowMag(b, m), b1 = HighMag(b, m);
        uint[] z0 = MulMagKara(a0, b0);
        uint[] z2 = MulMagKara(a1, b1);
```

listing 713: c#, the split and the two half products, the z1 subtraction chain and the shift recombination follow in the same routine (source: icpc/samples/src/Ch14/Karatsuba.cs, lines 10-18)

```
// chapter 14 karatsuba: magnitudes only, sign applied once at the end.
// split at m = max(la, lb) >> 1, then the contract identity
// z0 = a0*b0, z2 = a1*b1, z1 = (a0+a1)*(b0+b1) - z0 - z2, reassembled
// as z2*B^2m + z1*B^m + z0; recombination is pure limb addition, exact
// in Number, all products stay inside the schoolbook base case
import { addMag, subMag, mulSchoolMag, make } from "./ch14-bigint.mjs";

export const KARA_CUT = 24;

// out[off..] += t, in base 1e9; partial sums are bounded by the final
// product so writes never leave out
function accum(out, off, t) {
  let carry = 0;
  for (let i = 0; i < t.length; i++) {
    const s = out[off + i] + t[i] + carry;
    out[off + i] = s >= 1000000000 ? s - 1000000000 : s;
    carry = s >= 1000000000 ? 1 : 0;
  }
  for (let k = off + t.length; carry; k++) {
    const s = out[k] + carry;
    out[k] = s >= 1000000000 ? s - 1000000000 : s;
    carry = s >= 1000000000 ? 1 : 0;
  }
}

function karaMag(u, v) {
  const la = u.length, lb = v.length;
  if (la < KARA_CUT || lb < KARA_CUT) return mulSchoolMag(u, v);
  const m = Math.max(la, lb) >> 1;
  const a0 = u.slice(0, m), a1 = u.slice(m);
  const b0 = v.slice(0, m), b1 = v.slice(m);
  const z0 = karaMag(a0, b0);
  const z2 = karaMag(a1, b1);
  const z1 = subMag(subMag(karaMag(addMag(a0, a1), addMag(b0, b1)), z0), z2);
  const out = new Array(la + lb).fill(0);
  accum(out, 0, z0);
  accum(out, m, z1);
  accum(out, 2 * m, z2);
  let n = out.length;
  while (n > 0 && out[n - 1] === 0) n--;
  out.length = n;
  return out;
}
```

listing 714: javascript, the full recursion, recombination by pure limb addition through accum, exact in Number (source: icpc/samples-js/src/ch14-karatsuba.mjs, lines 1-43)

```
KARA_CUT = 24
# pinned 40-digit operands, drawn from the shared lcg stream with a
# 4-digit top limb so each is exactly 40 digits
PIN_A = "591604825218555767633847759239992953804 8"
PIN_B = "792710321168861980681378955213493098699 5"


def _shift(mag, k):
    return [0] * k + mag


def _kara_mag(u, v):
    if len(u) < len(v):
        u, v = v, u
    if len(v) < KARA_CUT:
        return mul_school_mag(u, v)
    m = len(u) // 2
    a0 = u[:m]
    a1 = u[m:]
    b0 = v[:m]
    b1 = v[m:]
    z0 = _kara_mag(a0, b0)
    z2 = _kara_mag(a1, b1)
    z1 = sub_mag(
        sub_mag(_kara_mag(add_mag(a0, a1), add_mag(b0, b1)), z0),
        z2,
    )
    return add_mag(add_mag(_shift(z2, 2 * m), _shift(z1, m)), z0)


def mul_kara(a, b):
    return Big(a.sign * b.sign, _kara_mag(a.limb, b.limb))
```

listing 715: python, the pinned 40-digit operands beside the cut, the recursion with list-slice halves (source: icpc/samples-py/src/Ch14/ch14_karatsuba.py, lines 17-48)

```
local function kara(x, y)
  local nx, ny = #x, #y
  if nx < KARA_CUT or ny < KARA_CUT then return school(x, y) end
  local m = math.max(nx, ny) // 2
  local x0, x1, y0, y1 = {}, {}, {}, {}
  for i = 1, m do x0[i] = x[i] or 0 end
  for i = m + 1, nx do x1[i - m] = x[i] end
  for i = 1, m do y0[i] = y[i] or 0 end
  for i = m + 1, ny do y1[i - m] = y[i] end
  local z0 = kara(x0, y0)
  local z2 = kara(x1, y1)
  local z1 = kara(BI.add(arr(x0), arr(x1)).limbs, BI.add(arr(y0), arr(y1)).limbs)
  z1 = BI.sub(arr(z1), arr(z0)).limbs
  z1 = BI.sub(arr(z1), arr(z2)).limbs
  -- prefilled so no nil holes survive the three offset adds
  local out = {}
  for i = 1, nx + ny do out[i] = 0 end
  add_at(out, 0, z0)
  add_at(out, m, z1)
  add_at(out, 2 * m, z2)
  -- canonicalize: BI.add/sub compare lengths first, so a phantom top
  -- zero limb here would flip a downstream subtraction's operand order
  local n = #out
  while n > 0 and out[n] == 0 do
    out[n] = nil
    n = n - 1
  end
  return out
end

function M.mul_kara(a, b)
  if #a.limbs == 0 or #b.limbs == 0 then return {sign = 1, limbs = {}} end
  return BI.norm({sign = a.sign * b.sign, limbs = kara(a.limbs, b.limbs)})
end
```

listing 716: lua, the recursion at cut 64, prefilled output so no nil hole survives the three offset adds, canonicalized against phantom top zeros (source: icpc/samples-lua/ch14_karatsuba.lua, lines 42-75)

The suites pin one 40-digit by 40-digit product against schoolbook and the native oracle, python carrying the drawn operands 5916048252185557676338477592399929538048 and 7927103211688619806813789552134930986995, lua carrying the algebraic twin `(10^40 - 1) * (10^40 + 1) = 10^80 - 1`, then all four sign combinations, zero operands, the cut boundary at CUT-1, CUT, CUT+1 limbs, asymmetric pairs like 144 by 21, and the full ladder agreement with schoolbook, lua extending its cross-check past its 89-limb quadratic cap at 90, 92, 144, and 233 limbs. Two stream bugs became warnings: javascript's `subMag` left phantom high zeros in karatsuba returns, caught by an unbalanced-length sweep because the phantom limb flipped a later comparison's operand order, and lua returned unnormalized magnitudes from the same trap's other side, both fixed by trimming before the value leaves the routine. Counts: C 79 checks, javascript 6 tests, python 71 checks, lua 4 named checks.

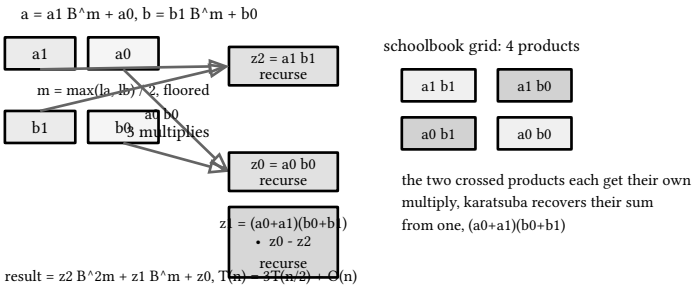


figure 195: one karatsuba level: three half-size multiplies against the schoolbook grid's four crossed products

16.3. exact ntt multiplication

The transforms want small limbs, so they regroup first: a base-1e4 limb is four decimal digits, every convolution column stays below `n * 9999^2`, and the fft gets 14-bit inputs. Regrouping always hops through the decimal string, `to_string` then four digits per limb packed from the right, one rule in all seven languages: the hop is O(n) against the transforms' O(n log n) and cannot drift a digit. The transform length is the next power of two at or above la+lb, capped at `2^16` base-1e4 limbs, 262144 decimal digits for the pair, refused loudly past it.

Exactness needs primes whose multiplicative group orders swallow that length. `P1 = 998244353` factors as `P1 - 1 = 2^23 \* 119` with primitive root 3, and `P2 = 754974721` factors as `P2 - 1 = 2^24 \* 45` with root 11, so every power of two up to the cap divides both orders. The transform itself is the iterative Cooley-Tukey shape, identical control flow in all seven languages: bit-reversal permutation, then doubling stages, the stage root by fast exponentiation and a running power within the stage, the inverse the same routine on inverted roots plus a scale by `pow(n, P-2, P)`. Each prime convolves the pair independently, then the chinese remainder theorem folds them: `x = x1 + P1 * t` with `t = ((x2 - x1) mod P2) * INV mod P2`, and INV, the inverse of P1 mod P2, is pinned as `416537774`, the same digits in all seven sources so no language drifts a constant, lua and java asserting `(243269632 * INV) mod P2 = 1` outright. The combined modulus `P1*P2 = 753649251896000513` stays under `2^63` while the true column value tops out at `n * 9999^2 = 6.55e12`, five orders of headroom, and that headroom is what makes the strange paths exact: butterfly products are a residue times a residue, `(P-1)^2 < 9.97e17`, inside signed 64-bit for c, c#, go, java, and lua's int64, trivially fine for python, and beyond f64 precision in javascript, whose butterflies run on BigInt, the reconstructed columns always under 6.6e12 and exact back on plain arithmetic.

The dry run: the C suite pins the hand convolution 9999, 5926, 5358 by 3238, 4626 through `ntt_conv4`, asserting the five output limbs 6762, 6999, 9424, 584, 2479 with `lc = 5`.

1. Both operands are base-1e4 limbs already, la + lb = 5, so the transform runs at the next power of two, 8, one convolution per prime folded by the CRT with the pinned INV = 416537774.
2. The folded columns are the polynomial products: column 0 is 9999 x 3238 = 32376762, column 1 is 9999 x 4626 + 5926 x 3238 = 65443762, column 2 is 5926 x 4626 + 5358 x 3238 = 44762880, and column 3 is 5358 x 4626 = 24786108.
3. The carry pass walks little-endian: 32376762 = 3237 x 10^4 + 6762 writes limb 6762, carrying 3237.
4. 65443762 + 3237 = 65446999 = 6544 x 10^4 + 6999 writes 6999, carrying 6544.
5. 44762880 + 6544 = 44769424 = 4476 x 10^4 + 9424 writes 9424, carrying 4476.
6. 24786108 + 4476 = 24790584 = 2479 x 10^4 + 584 writes 584, and the last carry stands alone as limb 2479, so `lc = 5`.

j	column + carry in	split k x 10^4 + limb	limb	carry out
0	32376762	3237 x 10^4 + 6762	6762	3237
1	65446999	6544 x 10^4 + 6999	6999	6544
2	44769424	4476 x 10^4 + 9424	9424	4476
3	24790584	2479 x 10^4 + 584	584	2479

j	column + carry in	split k x 10^4 + limb	limb	carry out
4	2479	the final carry alone	2479	

The five limbs are exactly what the CHECK lines read, and the listings below run both primes and the fold in seven languages.

```c
static void ntt_transform(unsigned *a, int n, int invert, u64 mod, u64 g) {
  for (int i = 1, j = 0; i < n; i++) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1)
      j ^= bit;
    j ^= bit;
    if (i < j) {
      unsigned t = a[i];
      a[i] = a[j];
      a[j] = t;
    }
  }
  for (int len = 2; len <= n; len <<= 1) {
    u64 wlen = ntt_powmod(g, (mod - 1) / (u64)len, mod);
    if (invert)
      wlen = ntt_powmod(wlen, mod - 2, mod);
    for (int i = 0; i < n; i += len) {
      u64 w = 1;
      for (int j = 0; j < len / 2; j++) {
        u64 u = a[i + j];
        u64 v = (u64)a[i + j + len / 2] * w % mod;
        a[i + j] = (unsigned)((u + v) % mod);
        a[i + j + len / 2] = (unsigned)((u - v + mod) % mod);
        w = w * wlen % mod;
      }
    }
  }
  if (invert) {
    u64 ninv = ntt_powmod((u64)n, mod - 2, mod);
    for (int i = 0; i < n; i++)
      a[i] = (unsigned)((u64)a[i] * ninv % mod);
  }
}
```

listing 717: c, the iterative transform, bit reversal, doubling stages, stage root by powmod, running power, inverted roots plus the n^-1 scale (source: icpc/samples-c/src/Ch14/ch14_ntt.h, lines 41-73)

```
func ntt(a []uint32, p, g uint64, invert bool) {
  n := len(a)
  for i, j := 1, 0; i < n; i++ {
    bit := n >> 1
    for ; j&bit != 0; bit >>= 1 {
      j ^= bit
    }
    j |= bit
    if i < j {
      a[i], a[j] = a[j], a[i]
    }
  }
  for length := 2; length <= n; length <<= 1 {
    w := powMod(g, (p-1)/uint64(length), p)
    if invert {
      w = powMod(w, p-2, p)
    }
    half := length / 2
    for start := 0; start < n; start += length {
      wp := uint64(1)
      for k := 0; k < half; k++ {
        u := uint64(a[start+k])
        v := uint64(a[start+k+half]) * wp % p
        a[start+k] = uint32((u + v) % p)
        a[start+k+half] = uint32((u + p - v) % p)
        wp = wp * w % p
      }
    }
  }
  if invert {
    inv := powMod(uint64(n), p-2, p)
    for i := range a {
      a[i] = uint32(uint64(a[i]) * inv % p)
    }
  }
}
```

listing 718: go, the same transform over uint64 butterflies, inverted roots for the inverse pass (source: icpc/samples-go/ch14/ntt.go, lines 63-98)

```java
// iterative cooley-tukey: bit-reversal permutation, then butterflies
// stage by stage, one root per stage powered up by repeated
// multiplication; invert flips the root and scales by 1/n
static void ntt(long[] a, int len, boolean invert, long p, long g) {
  for (int i = 1, j = 0; i < len; i++) {
    int bit = len >> 1;
    for (; (j & bit) != 0; bit >>= 1)
      j ^= bit;
    j ^= bit;
    if (i < j) {
      long t = a[i];
      a[i] = a[j];
      a[j] = t;
    }
  }
  for (int half = 2; half ≤ len; half <<= 1) {
    long w = modpow(g, (p - 1) / half, p);
    if (invert)
      w = modpow(w, p - 2, p);
    for (int i = 0; i < len; i += half) {
      long cur = 1;
      for (int j = 0; j < half / 2; j++) {
        long u = a[i + j];
        long v = a[i + j + half / 2] * cur % p;
        a[i + j] = (u + v) % p;
        a[i + j + half / 2] = (u + p - v) % p;
        cur = cur * w % p;
      }
    }
  }
  if (invert) {
    long ninv = modpow(len % p, p - 2, p);
    for (int i = 0; i < len; i++)
      a[i] = a[i] * ninv % p;
  }
}
```

listing 719: java, the iterative transform over long butterflies, bit reversal, doubling stages, one root per stage by modpow, inverted roots plus the len^-1 scale (source: icpc/samples-java/src/Ch15/Ntt.java, lines 52-87)

```csharp
  private static void Convolve(uint[] x, uint[] y, long[] c, long p, long g)
  {
      int n = c.Length;
      for (int i = 0; i < x.Length; i++)
          c[i] = x[i];
      var b = new long[n];
      for (int i = 0; i < y.Length; i++)
          b[i] = y[i];
      Ntt(c, p, g, false);
      Ntt(b, p, g, false);
      for (int k = 0; k < n; k++)
          c[k] = c[k] * b[k] % p;
      Ntt(c, p, g, true);
  }
```

listing 720: c#, one convolution per prime, the identical routine over P1 then P2, pointwise between the two transforms (source: icpc/samples/src/Ch14/NttMul.cs, lines 47-60)

```
function ntt(f, n, p, g, invert) {
  for (let i = 1, j = 0; i < n; i++) {
    let bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j |= bit;
    if (i < j) [f[i], f[j]] = [f[j], f[i]];
  }
  for (let len = 2; len ≤ n; len <<= 1) {
    const root = modpow(g, (p - 1n) / BigInt(len), p);
    const wlen = invert ? modpow(root, p - 2n, p) : root;
    const half = len >> 1;
    for (let i = 0; i < n; i += len) {
      let cur = 1n;
      for (let j = 0; j < half; j++) {
        const u = f[i + j];
        const v = (f[i + j + half] * cur) % p;
        f[i + j] = u + v < p ? u + v : u + v - p;
        f[i + j + half] = u ≥ v ? u - v : u - v + p;
        cur = (cur * wlen) % p;
      }
    }
  }
  if (invert) {
    const ninv = modpow(BigInt(n), p - 2n, p);
    for (let i = 0; i < n; i++) f[i] = (f[i] * ninv) % p;
  }
}
```

listing 721: javascript, the transform on BigInt, stated openly, exactness outranks speed here (source: icpc/samples-js/src/ch14-ntt.mjs, lines 30-56)

```
def mul_ntt(a, b):
    sign = a.sign * b.sign
    fa = pack4(a)
    fb = pack4(b)
    if len(fa) + len(fb) > NTT_CAP:
        raise ValueError("ntt product exceeds the 2^16 limb cap")
    size = 1
    while size < len(fa) + len(fb):
        size <<= 1
    pa = fa + [0] * (size - len(fa))
    pb = fb + [0] * (size - len(fb))
    x1 = _convolve_prime(pa, pb, P1, G1)
    x2 = _convolve_prime(pa, pb, P2, G2)
    out = []
    carry = 0
    for k in range(size):
        # crt: x = x1 + P1 * t, t = ((x2 - x1) mod P2) * INV mod P2
        t = (x2[k] - x1[k]) % P2 * INV % P2
        cur = x1[k] + P1 * t + carry
        carry, digit = divmod(cur, B4)
        out.append(digit)
    while carry:
        carry, digit = divmod(carry, B4)
        out.append(digit)
    return unpack4(out, sign)
```

listing 722: python, the wrapper, both primes, the CRT fold with the pinned INV, carry back to base 1e4 (source: icpc/samples-py/src/Ch14/ch14_ntt.py, lines 70-94)

```
function M.mul_ntt(a, b)
  if #a.limbs == 0 or #b.limbs == 0 then return {sign = 1, limbs = {}} end
  local x = BI.pack4(a)
  local y = BI.pack4(b)
  local la, lb = #x, #y
  if la + lb > CAP then error("ntt over the 2^16 limb cap") end
  local need = la + lb - 1
  local n = 1
  while n < need do n = n * 2 end
  local c1 = conv_prime(x, y, n, P1, G1)
  local c2 = conv_prime(x, y, n, P2, G2)
  local out = {}
  for i = 1, need do
    -- x = x1 + P1*t with t = ((x2-x1) mod P2) * INV mod P2, fits 2^63
    local t = ((c2[i] - c1[i]) % P2) * INV % P2
    out[i] = c1[i] + P1 * t
  end
  carry4(out)
  local r = BI.unpack4(out)
  return BI.norm({sign = a.sign * b.sign, limbs = r.limbs})
end
```

listing 723: lua, the CRT reconstruction per column, x = x1 + P1*t inside int64, then the carry pass (source: icpc/samples-lua/ch14_ntt.lua, lines 95-115)

The pinned stressor is `(10^60 - 1) * (10^60 + 1) = 10^120 - 1`, every digit a 9, with all four sign combinations, zero operands, the hand convolutions `9999 * 9999 = 99980001` and a seam-crossing product, and the ladder agreeing with schoolbook everywhere. The cap is tested at the cap, not just past it: C# squares 131072-digit operands in the exact-at-cap fact, go runs n = 65536 exactly against `math/big`, and java squares 80000 nines into 160000 digits at

transform length 2^16 exactly. Two stream bugs shaped the listings above. Go's CRT wrapped: P1 exceeds P2, so a residue above P2 made the subtraction `x2 - x1` go negative in uint64, hidden below 89 limbs until a reproducer replayed the lcg operands, fixed as `x2 + 2*P2 - x1`. C shipped its first transform without the inverse pass before CRT, caught by the stressor. Counts: C 83 checks, javascript 5 tests, python 59 checks, lua 5 named checks.

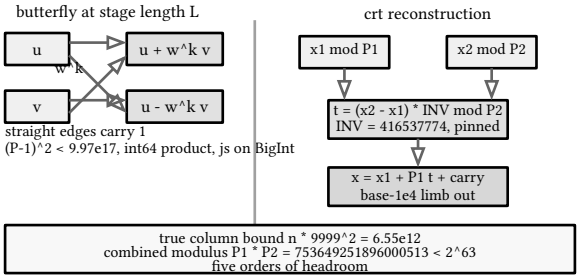


figure 196: one butterfly over w^k, then the CRT fold: two prime residues rebuilt into one column under the 2^63 ceiling

16.4. fft multiplication, plain and split

Complex fft trades exactness for speed and then earns the exactness back with an error budget. Pack base-1e4 limbs into doubles, transform, multiply pointwise, transform back, round. With unit roundoff `u = 2^-53` and columns up to `(10^4 - 1)^2`, the plain path's drift is bounded by `n * (10^4-1)^2 * (3*log2(n) + 4) * u`, which evaluates to 0.0378 at n = 2^16. The split path is the main one: pack `P[j] = a[j] + i*b[j]`, two 14-bit limbs riding one complex number inside a 15-bit envelope, one forward transform, recover both spectra by conjugate reversal, `Ahat[k] = (Phat[k] + conj(Phat[n-k])) / 2` and `Bhat[k] = -i * (Phat[k] - conj(Phat[n-k])) / 2`, pointwise, one inverse. Half the transforms, roughly half the error.

That budget assumed exact twiddles, and the pinned twiddle policy does not buy them: one `cos(2*pi/L)` and one `sin(2*pi/L)` per stage form the principal root, powers advance by repeated multiplication, the inverse flips the imaginary sign, nothing else touches libm, fixed so seven libms agree. Measured on full-range random limbs, max fractional distance at transform 2^14, 2^15, 2^16: javascript 0.049, 0.250, 0.334 on its draw, python 0.049, 0.261, 0.807 on a fresh seed, c 0.049, 0.262, 0.500, go 0.049, 0.270, 0.500, java 0.0477, 0.2617, 0.500 under its pinned jdk 27, c# 0.048, 0.261, 0.500 split with plain within 0.001, and lua, probed on worst-case all-9999 limbs, 0.191, 0.500, 0.500. Go ran the control experiment: with twiddles computed directly per power, drift stays under 0.0004 at the same sizes, which isolates the recurrence, not the transform, as the cause.

VERIFY: the cap followed the measurement, not the plan

The plan's 0.0378 at 2^16 was an 18x underestimate of what the pinned recurrence does, about 0.5 across languages at 2^16 where a half-integer distance means rounding can flip a limb. The ruling, identical in every language: both fft paths cap at transform 2^14, about 5x inside the 0.25 guard, rounding stays `floor(x + 0.5)` everywhere, never rint, and the guard stays the tripwire, a loud failure per language, c and java return a status, go error, c# InvalidOperationException, javascript and python throw, lua `error()`. The loud 2^16 trip is pinned suite behavior, python asserting the throw on its fresh-seed probe, java the guard trip on its uncapped 2^16 probe, lua the cap refusal one doubling past the boundary product that passes exactly at the cap.

The dry run: the python suite pins the stressor `(10^100 - 1)^2` through both paths against the constructed string of 99 nines, one 8, 99 zeros, one 1, and unit-tests the guard `_roundc` on 3.0, 2.76, 2.24, and the refusals, the C suite carrying the same stressor.

1. Regroup 100 nines into base-1e4 limbs: 100 / 4 = 25 limbs, every one 9999, and la + lb = 50 puts the transform at the next power of two, 64, far under the 2^14 cap.
2. The split path packs `P[j]` as 9999 + 9999i, and a square makes the two recovered spectra equal, so the pointwise stage squares each entry.
3. One inverse, then rounding under the guard: 2.76 rounds up to 3 at distance 0.24 and 2.24 down to 2 at the same distance, both inside the 0.25 band.

4. The refusals sit on the band’s edge and past it: 2.25 and 2.75 at exactly 0.25, 0.5 at half a unit, all three loud.
5. The identity behind the fixture: (10^100 - 1)^2 = 10^200 - 2 x 10^100 + 1.
6. The carry pass lands it as digits, 99 nines, 8, 99 zeros, one 1, 200 digits, both paths printing the same string.

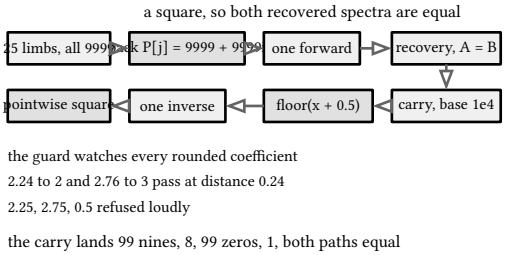


figure 197: the split run on the stressor as a sequence, pack, one forward, recovery, square, one inverse, round under the guard, carry

The 99 nines, one 8, 99 zeros, one 1 string is the suite’s constructed expectation, and the listings below carry both paths in seven languages.

```c
static void fft(cpx *a, int n, int invert) {
  for (int i = 1, j = 0; i < n; i++) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1)
      j ^= bit;
    j ^= bit;
    if (i < j) {
      cpx t = a[i];
      a[i] = a[j];
      a[j] = t;
    }
  }
  for (int len = 2; len ≤ n; len <<= 1) {
    double wl_re = cos(2.0 * FFT_PI / (double)len);
    double wl_im = invert ? -sin(2.0 * FFT_PI / (double)len)
                          : sin(2.0 * FFT_PI / (double)len);
    for (int i = 0; i < n; i += len) {
      double w_re = 1.0, w_im = 0.0;
      for (int j = 0; j < len / 2; j++) {
        cpx u = a[i + j], v = a[i + j + len / 2];
        double vr = v.re * w_re - v.im * w_im;
        double vi = v.re * w_im + v.im * w_re;
        a[i + j].re = u.re + vr;
        a[i + j].im = u.im + vi;
        a[i + j + len / 2].re = u.re - vr;
        a[i + j + len / 2].im = u.im - vi;
        double nw = w_re * wl_re - w_im * wl_im;
        w_im = w_re * wl_im + w_im * wl_re;
        w_re = nw;
      }
    }
  }
  if (invert) {
    for (int i = 0; i < n; i++) {
      a[i].re /= (double)n;
      a[i].im /= (double)n;
    }
  }
}
```

listing 724: c, the hand-rolled cpx transform, one cos and one sin per stage, powers by repeated multiplication, the drift source in plain sight (source: icpc/samples-c/src/Ch14/ch14_fft.h, lines 33-71)

```
func MulFftSplit(a, b Big) (Big, error) {
  a, b = a.normalize(), b.normalize()
  if len(a.limb) == 0 || len(b.limb) == 0 {
    return Big{sign: 1}, nil
  }
  ua := toLimbs4(Big{sign: 1, limb: a.limb})
  ub := toLimbs4(Big{sign: 1, limb: b.limb})
  n, err := fftLength(len(ua), len(ub), fftMaxLen)
  if err != nil {
    return Big{}, err
  }
  p := make([]complex128, n)
  for i := range p {
    var re, im float64
    if i < len(ua) {
      re = float64(ua[i])
    }
    if i < len(ub) {
      im = float64(ub[i])
    }
    p[i] = complex(re, im)
  }
  fft(p, false)
  re0, im0 := real(p[0]), imag(p[0])
  p[0] = complex(re0*im0, 0)
  for k := 1; k <= n/2; k++ {
    j := n - k
    pk := p[k]
    pj := complex(real(p[j]), -imag(p[j])) // conj
    ak := (pk + pj) / 2
    diff := pk - pj
    bk := complex(imag(diff), -real(diff)) / 2 // -i*(diff)/2
    c := ak * bk
    p[k] = c
    if j != k {
      p[j] = complex(real(c), -imag(c))
    }
  }
  fft(p, true)
  coef, err := roundCoeffs(p)
  if err != nil {
    return Big{}, err
  }
  out := fromLimbs4(carryBase4(coef))
  out.sign = a.sign * b.sign
  return out.normalize(), nil
}
```

listing 725: go, the split path, the packed spectrum recovered in one loop, the measured-cap comment at the head of the file (source: icpc/samples-go/ch14/fft.go, lines 53-99)

```
static int fftConv(int[] a, int la, int[] b, int lb, double[] out, boolean split) {
  if (la == 0 || lb == 0)
    return -3;
  int len = 1;
  while (len < la + lb)
    len <<= 1;
  if (len > FFT_CAP)
    return -1;
  if (split) {
    for (int i = 0; i < len; i++) {
      ar[i] = i < la ? a[i] : 0.0;
      ai[i] = i < lb ? b[i] : 0.0;
    }
    fft(ar, ai, len, false);
    // recover both spectra from the packed transform before
    // overwriting anything, staged through temp
    for (int k = 0; k < len; k++) {
      int k2 = (len - k) & (len - 1);
      double dr = ar[k] - ar[k2], di = ai[k] + ai[k2];
      br[k] = di / 2;
      bi[k] = -dr / 2;
      tr[k] = (ar[k] + ar[k2]) / 2;
      ti[k] = (ai[k] - ai[k2]) / 2;
    }
    System.arraycopy(tr, 0, ar, 0, len);
    System.arraycopy(ti, 0, ai, 0, len);
    for (int k = 0; k < len; k++) {
      double cr = ar[k] * br[k] - ai[k] * bi[k];
      double ci = ar[k] * bi[k] + ai[k] * br[k];
      ar[k] = cr;
      ai[k] = ci;
    }
    fft(ar, ai, len, true);
    for (int i = 0; i < la + lb; i++)
      out[i] = ar[i];
```

listing 726: java, the split path over plain double arrays, the cap refusal at the entry, pack, one forward, conjugate-reversal recovery staged through scratch, pointwise, one inverse (source: icpc/samples-java/src/Ch15/Fft.java, lines 89-123)

```
    /// <summary>
    /// Main path: pack P[j] = a[j] + i*b[j], one forward, recover both spectra
    /// via conjugate reversal, pointwise, one inverse. Half the transforms.
    /// </summary>
    private static uint[] MulMagFftSplit(uint[] a, uint[] b)
    {
        if (a.Length == 0 || b.Length == 0)
            return Array.Empty<uint>();
        uint[] x = ToBase1e4(a), y = ToBase1e4(b);
        int n = TransformLength(x.Length + y.Length, FftCap);
        var p = new Complex[n];
        for (int i = 0; i < n; i++)
            p[i] = new Complex(i < x.Length ? x[i] : 0, i < y.Length ? y[i] : 0);
        Fft(p, false);
        Complex negI = new(0, -1);
        var prod = new Complex[n];
        for (int k = 0; k < n; k++)
        {
            int mk = (n - k) & (n - 1);
            Complex aHat = (p[k] + Complex.Conjugate(p[mk])) / 2;
            Complex bHat = negI * (p[k] - Complex.Conjugate(p[mk])) / 2;
            prod[k] = aHat * bHat;
        }
        Fft(prod, true);
        return RoundCarryRegroup(prod, x.Length + y.Length);
    }
```

listing 727: c#, the split path over native Complex, pack, one forward, conjugate-reversal recovery, pointwise, one inverse (source: icpc/samples/src/Ch14/FftMul.cs, lines 42-67)

```
// split path, the chapter's main fft: pack P[j] = a[j] + i*b[j], one
// forward, recover both spectra by conjugate reversal, pointwise, one
// inverse. half the transforms, roughly half the error
export function mulFftSplit(a, b) {
  return fftSplit(a, b, CAP);
}

// raw uncapped split entry for the test pins only: it documents where
// the pinned twiddle recurrence loses the error budget (measured drift
// 0.334-0.807 at 2^16, past 0.5 the guard distance wraps and would
// pass a wrong limb), which is exactly why the public cap is 2^14
export function mulFftSplitRaw(a, b) {
  return fftSplit(a, b, Number.MAX_SAFE_INTEGER);
}

function fftSplit(a, b, cap) {
  const A = toLimbs4(a);
  const Bl = toLimbs4(b);
  if (A.length === 0 || Bl.length === 0) return make(1, []);
  const n = transformLength(A.length + Bl.length, cap);
  const pr = new Float64Array(n), pi = new Float64Array(n);
  for (let i = 0; i < A.length; i++) pr[i] = A[i];
  for (let i = 0; i < Bl.length; i++) pi[i] = Bl[i];
  transform(pr, pi, n, false);
  const ar = new Float64Array(n), ai = new Float64Array(n);
  const br = new Float64Array(n), bi = new Float64Array(n);
  for (let k = 0; k < n; k++) {
    const k2 = (n - k) % n;
    ar[k] = (pr[k] + pr[k2]) / 2; // Ahat[k] = (Phat[k] + conj(Phat[n-k]))/2
    ai[k] = (pi[k] - pi[k2]) / 2;
    br[k] = (pi[k] + pi[k2]) / 2; // Bhat[k] = -i*(Phat[k] - conj(Phat[n-k]))/2
    bi[k] = (pr[k2] - pr[k]) / 2;
  }
  for (let i = 0; i < n; i++) {
    const r = ar[i] * br[i] - ai[i] * bi[i];
    ai[i] = ar[i] * bi[i] + ai[i] * br[i];
    ar[i] = r;
  }
  transform(ar, ai, n, true);
  return fromLimbs4(carryLimbs(ar, n), a.sign * b.sign);
}
```

listing 728: javascript, the capped split entry beside the uncapped raw hook the test pins use, then fftSplit on paired Float64Arrays with the recovery formulas at the seam (source: icpc/samples-js/src/ch14-fft.mjs, lines 121-161)

```
def mul_fft_plain(a, b):
    sign = a.sign * b.sign
    fa = pack4(a)
    fb = pack4(b)
    if len(fa) + len(fb) > FFT_PLAIN_CAP:
        raise ValueError("plain fft product exceeds the 2^14 limb cap")
    size = _pow2_at_least(len(fa) + len(fb))
    ca = [complex(t, 0.0) for t in fa] + [0j] * (size - len(fa))
    cb = [complex(t, 0.0) for t in fb] + [0j] * (size - len(fb))
    _fft(ca, False)
    _fft(cb, False)
    for k in range(size):
        ca[k] = ca[k] * cb[k]
    _fft(ca, True)
    return _carry_out(ca, sign)


def _split_spectrum(fa, fb):
    # unrounded inverse spectrum of the packed product, shared by the
    # capped multiply and the drift guard-trip check
    size = _pow2_at_least(len(fa) + len(fb))
    cp = []
    for k in range(size):
        re = fa[k] if k < len(fa) else 0
        im = fb[k] if k < len(fb) else 0
        cp.append(complex(re, im))
    _fft(cp, False)
    ch = [0j] * size
    for k in range(size):
        pk = cp[k]
        pkc = cp[size - k if k else 0].conjugate()
        # A = (P[k] + conj(P[n-k])) / 2, B = -i (P[k] - conj(P[n-k])) / 2
        ch[k] = (pk + pkc) * 0.5 * ((pk - pkc) * -0.5j)
    _fft(ch, True)
    return ch
```

listing 729: python, both paths, the plain two-transform multiply and the shared split spectrum with its conjugate reversal (source: icpc/samples-py/src/Ch14/ch14_fft.py, lines 96-130)

```lua
local function convolve(x, y, n, split)
  local re, im = {}, {}
  if split then
    for i = 1, n do
      re[i] = (x[i] or 0) + 0.0
      im[i] = (y[i] or 0) + 0.0
    end
    fft(re, im, n, false)
    local ar, ai, br, bi = {}, {}, {}, {}
    for k = 0, n - 1 do
      local k1 = k + 1
      local m1 = (n - k) % n + 1
      -- A = (P[k] + conj(P[n-k]))/2, B = -i*(P[k] - conj(P[n-k]))/2
      local sr = re[k1] + re[m1]
      local si = im[k1] - im[m1]
      local dr = re[k1] - re[m1]
      local di = im[k1] + im[m1]
      ar[k1] = sr / 2
      ai[k1] = si / 2
      br[k1] = di / 2
      bi[k1] = -dr / 2
    end
    for k = 1, n do
      local cr = ar[k] * br[k] - ai[k] * bi[k]
      local ci = ar[k] * bi[k] + ai[k] * br[k]
      re[k], im[k] = cr, ci
    end
```

listing 730: lua, the split branch of convolve, packed forward, conjugate-reversal recovery into four tables, coefficients returned with the max fractional distance for the caller's guard (source: icpc/samples-lua/ch14_fft.lua, lines 68-94)

The stressor is `(10^100 - 1)^2 = 10^200 - 2*10^100 + 1`, both paths, all sign combinations, zero operands, the rounding guard unit-tested on exact integers, on values 0.24 below and 0.76 above the half, and on the refusals at 0.5, 2.25, 2.75, then the ladder with both paths agreeing with schoolbook, the four-way chain `schoolbook == karatsuba == ntt == fft_split` on every ladder pair, and python's exactness probe at 300 limbs against native integers. C's first fft padded zero limbs into the packing and go's first version skipped the base-1e4 carry pass after rounding, both caught by the chain. Counts: C 461 checks, javascript 7 tests, python 165 checks, lua 4 named checks.

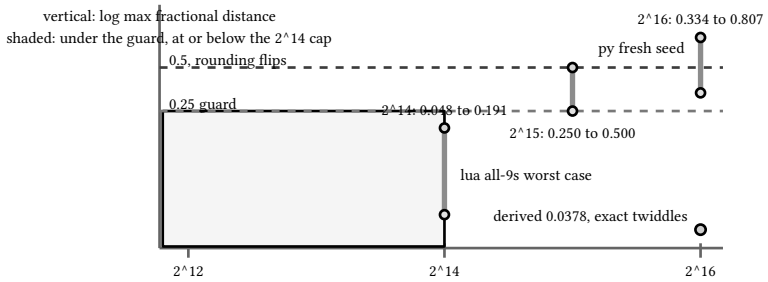


figure 198: measured fractional drift against the 0.25 guard and the 0.5 rounding cliff, the safe range shaded under the 2^14 cap

16.5. long division, knuth algorithm d

One-limb divisors take the fast path, per-limb 64 over 32 division with the remainder carried down, and everything else runs Knuth's algorithm D over magnitudes. Normalize first, multiplying dividend and divisor by `s = B / (v[n-1] + 1)` floored so the divisor's top limb reaches at least B/2, the dividend buffer gaining one high sentinel limb. Estimate each quotient digit two limbs over one: `qhat = (u[j+n]*B + u[j+n-1]) / v[n-1]`, a numerator under 1e18 and inside 2^63 in plain 64/64 division, no 128-bit anywhere, clamp `qhat` $\geq$ `B` down to `B-1`, then refine while `qhat * v[n-2] > rhat*B + u[j+n-2]`, at most two decrements once the top limb

is normalized. Multiply and subtract `qhat * v` from the window with a 64-bit borrow chain, and if the final borrow goes negative, qhat was one too large: decrement it and add the divisor back once, Knuth's theorem bounding the correction to a single add. The remainder is the window's low limbs divided by s again. Every product in the loop stays under 1e18, under `2^60`, which is why C bans `__uint128_t` outright, grepping every commit for it, javascript routes qhat and the mulsub products through BigInt, and java holds every product in plain longs. The signed wrapper truncates q toward zero and gives r the dividend's sign, python computing its oracle by hand as `abs(a) // abs(b)` with the sign pasted on, because native divmod floors. Zero divisors are loud per language: C and java return a status, C# throws ArgumentOutOfRangeException, go returns an error, javascript RangeError, python ZeroDivisionError, lua `error()`.

The dry run: the C suite's anchor fact walks `10^21 / 7`, asserting the quotient string `142857142857142857142` and remainder 6.

1. 10^21 parses to 3 limbs, 0, 0, 1000 top, and 7 is one limb, so the fast path runs, 64 over 32 per limb, top down.
2. Top limb: cur = 0 x 10^9 + 1000 = 1000, quotient limb 1000 / 7 = 142, remainder 1000 - 7 x 142 = 6.
3. Middle limb: cur = 6 x 10^9 + 0 = 6000000000, quotient limb 6000000000 / 7 = 857142857, remainder 6000000000 - 7 x 857142857 =
   1.
4. Bottom limb: cur = 1 x 10^9 = 1000000000, quotient limb 1000000000 / 7 = 142857142, remainder 1000000000 - 7 x 142857142 =
   6.
5. Printing pads below the bare top: 142, 857142857, 142857142 concatenate to the 21-digit quotient, the remainder standing at 6.
6. The multi-limb side is quoted where the suite pins it, the add-back pair 750000000000000000000000000 over 500000000000000000999999999 landing quotient 14 and remainder 499999999999999986000000014.

step	limb	cur = rem x 10^9 + limb	out	rem
1	1000	0 x 10^9 + 1000 = 1000	142	6
2	0	6 x 10^9 + 0 = 6000000000	857142857	1
3	0	1 x 10^9 + 0 = 1000000000	142857142	6
4	print	142, 857142857, 142857142	142857142857142857142	6

The table's last row is what the two asserts read, and the listings below run algorithm D in seven languages.

```
  for (int j = m; j ≥ 0; j--) {
    u64 num = (u64)dv_u[j + n] * BIG_BASE + dv_u[j + n - 1];
    u64 qhat = num / dv_v[n - 1];
    u64 rhat = num % dv_v[n - 1];
    if (qhat ≥ BIG_BASE) { // clamp before the refinement
      qhat = BIG_BASE - 1;
      rhat = num - qhat * dv_v[n - 1];
    }
    int refs = 0; // the estimate is at most two too large
    while (qhat * dv_v[n - 2] > rhat * BIG_BASE + dv_u[j + n - 2]) {
      qhat--;
      rhat += dv_v[n - 1];
      refs++;
      if (rhat ≥ BIG_BASE || refs == 2)
        break;
    }
    u64 borrow = 0, car = 0; // multiply and subtract
    for (int i = 0; i < n; i++) {
      u64 t = qhat * dv_v[i] + car;
      car = t / BIG_BASE;
      long long sub = (long long)dv_u[j + i] - (long long)borrow -
                      (long long)(t % BIG_BASE);
      if (sub < 0) {
        sub += BIG_BASE;
        borrow = 1;
      } else {
        borrow = 0;
      }
      dv_u[j + i] = (unsigned)sub;
    }
    long long top = (long long)dv_u[j + n] - (long long)borrow - (long long)car;
    if (top < 0) { // rare add-back: qhat was one too large
      qhat--;
      u64 c2 = 0;
      for (int i = 0; i < n; i++) {
        u64 t = (u64)dv_u[j + i] + dv_v[i] + c2;
        dv_u[j + i] = (unsigned)(t % BIG_BASE);
        c2 = t / BIG_BASE;
      }
      top += (long long)c2;
    }
    assert(top ≥ 0);
    dv_u[j + n] = (unsigned)top;
    q_out[j] = (unsigned)qhat;
  }
```

listing 731: c, the full quotient loop, qhat estimate, clamp and refine, mulsub borrow, the single add-back, remainder descale (source: icpc/samples-c/src/Ch14/ch14_divmod.h, lines 52-96)

```
  for j := m; j ≥ 0; j-- {
    num := uint64(un[j+n])*base + uint64(un[j+n-1])
    qhat := num / uint64(vn[n-1])
    rhat := num % uint64(vn[n-1])
    if qhat ≥ base {
      qhat = base - 1
      rhat = num - qhat*uint64(vn[n-1])
    }
    // D3: refine, at most two decrements once v[n-1] ≥ B/2
    for qhat*uint64(vn[n-2]) > rhat*base+uint64(un[j+n-2]) {
      qhat--
      rhat += uint64(vn[n-1])
      if rhat ≥ base {
        break
      }
    }
    // D4: u -= qhat * v at limb j
    var borrow, carry int64
    for i := 0; i < n; i++ {
      cur := int64(qhat)*int64(vn[i]) + carry
      t := int64(un[j+i]) - (cur % base) + borrow
      if t < 0 {
        t += base
        borrow = -1
      } else {
        borrow = 0
      }
      un[j+i] = uint32(t)
      carry = cur / base
    }
    t := int64(un[j+n]) - carry + borrow
    if t < 0 {
      // D6: add-back, at most once by Knuth's theorem
      qhat--
      var c uint64
      for i := 0; i < n; i++ {
        sum := uint64(un[j+i]) + uint64(vn[i]) + c
        un[j+i] = uint32(sum % base)
        c = sum / base
      }
      // t + base + c reaches base exactly when t = -1 and c = 1,
      // so normalize: the carry out of this dead slot drops with it.
      un[j+n] = uint32((t + base + int64(c)) % base)
    } else {
      un[j+n] = uint32(t)
    }
    q[j] = uint32(qhat)
  }
  return q, divSmall(un[:n], s) // D8: remainder scaled back
```

listing 732: go, D1 through D8 annotated in place, int64 borrows, the add-back on a negative top with the dead slot normalized (source: icpc/samples-go/ch14/divmod.go, lines 47-95)

```
    for (int j = m; j ≥ 0; j--) {
      long num = un[j + n] * (long) Bigint.B + un[j + n - 1];
      long qhat = num / vn[n - 1];
      long rhat = num % vn[n - 1];
      if (qhat > Bigint.B - 1) {
        qhat = Bigint.B - 1;
        rhat = num - qhat * vn[n - 1];
      }
      while (qhat * vn[n - 2] > rhat * Bigint.B + un[j + n - 2]) {
        qhat--;
        rhat += vn[n - 1];
      }
      // mulsub: window -= qhat * vn
      long carry = 0, borrow = 0;
      for (int i = 0; i < n; i++) {
        long prod = qhat * vn[i] + carry;
        carry = prod / Bigint.B;
        long sub = prod % Bigint.B + borrow;
        long t = un[j + i] - sub;
        if (t < 0) {
          t += Bigint.B;
          borrow = 1;
        } else
          borrow = 0;
        un[j + i] = (int) t;
      }
      long top = un[j + n] - carry - borrow;
      if (top < 0) { // the single add-back
        addbacks++;
        qhat--;
        long c3 = 0;
        for (int i = 0; i < n; i++) {
          long cur = un[j + i] + vn[i] + c3;
          c3 = cur / Bigint.B;
          un[j + i] = (int) (cur % Bigint.B);
        }
        top += c3 + Bigint.B;
      }
      un[j + n] = (int) top;
      q.limb[j] = (int) qhat;
    }
```

listing 733: java, the full quotient loop, qhat two-over-one in plain long division, clamp then refine, mulsub borrow chain, the counted add-back, remainder descale (source: icpc/samples-java/src/Ch15/Divmod.java, lines 60-100)

```
        for (int j = m - 1; j ≥ 0; j--)
        {
            // 2-over-1 estimate in plain 64/64 arithmetic, numerator < 1e18
            ulong num = (ulong)ub[j + n] * Base + ub[j + n - 1];
            ulong qhat = num / v[n - 1];
            ulong rhat = num % v[n - 1];
            while (qhat ≥ Base || qhat * v[n - 2] > rhat * Base + ub[j + n - 2])
            {
                qhat--; // clamp plus refine, at most 2 useful iterations
                rhat += v[n - 1];
                if (rhat ≥ Base)
                    break;
            }

            // multiply and subtract with a 64-bit borrow chain
            ulong mcarry = 0, borrow = 0;
            for (int i = 0; i < n; i++)
            {
                ulong p = qhat * v[i] + mcarry;
                mcarry = p / Base;
                long t = (long)ub[i + j] - (long)(p % Base) - (long)borrow;
                if (t < 0)
                {
                    t += (long)Base;
                    borrow = 1;
                }
                else
                    borrow = 0;
                ub[i + j] = (uint)t;
            }
            long top = (long)ub[j + n] - (long)mcarry - (long)borrow;
            if (top < 0)
            {
                // rare add-back: qhat survived refinement one too large
                ub[j + n] = (uint)(top + (long)Base);
                qhat--;
                ulong carryIn = 0;
                for (int i = 0; i < n; i++)
                {
                    ulong sum = ub[i + j] + v[i] + carryIn;
                    ub[i + j] = (uint)(sum % Base);
                    carryIn = sum / Base;
                }
                ub[j + n] = (uint)((ub[j + n] + carryIn) % Base);
            }
            else
                ub[j + n] = (uint)top;
            q[j] = (uint)qhat;
        }
```

listing 734: c#, the same loop, clamp folded into the refinement condition, add-back restoring the window (source: icpc/samples/src/Ch14/DivMod.cs, lines 43-91)

```
  for (let j = m; j ≥ 0; j--) {
    // qhat 2-over-1: numerator < 1e18 < 2^63
    const num = BigInt(us[j + n]) * B + BigInt(us[j + n - 1]);
    let qhat = num / top;
    let rhat = num % top;
    if (qhat ≥ B) {
      qhat = B - 1n;
      rhat = num - qhat * top;
    }
    // at most 2 refinement iterations
    while (qhat * next > rhat * B + BigInt(us[j + n - 2])) {
      qhat -= 1n;
      rhat += top;
      if (rhat ≥ B) break;
    }
    // multiply and subtract with a BigInt borrow
    let borrow = 0n;
    for (let i = 0; i < n; i++) {
      const t = BigInt(us[i + j]) - qhat * BigInt(vs[i]) + borrow;
      const r = ((t % B) + B) % B;
      us[i + j] = Number(r);
      borrow = (t - r) / B;
    }
    const t = BigInt(us[j + n]) + borrow;
    const r = ((t % B) + B) % B;
    us[j + n] = Number(r);
    if (t < 0n) {
      // rare add-back: qhat was one too large
      qhat -= 1n;
      let carry = 0n;
      for (let i = 0; i < n; i++) {
        const c = BigInt(us[i + j]) + BigInt(vs[i]) + carry;
        us[i + j] = Number(c % B);
        carry = c / B;
      }
      us[j + n] = (us[j + n] + Number(carry)) % BN;
    }
    q[j] = Number(qhat);
  }
```

listing 735: javascript, qhat and mulsub on BigInt, floored-mod limb recovery, the add-back branch (source: icpc/samples-js/src/ch14-divmod.mjs, lines 60-98)

```
def _knuth_d(u, v):
    # magnitudes only, len(v) ≥ 2, |u| ≥ |v|
    n = len(v)
    m = len(u) - n
    s = B // (v[n - 1] + 1)
    vn = _mul_limb(v, s)
    un = _mul_limb(u, s)
    un += [0] * (m + n + 1 - len(un))
    q = [0] * (m + 1)
    for j in range(m, -1, -1):
        num = un[j + n] * B + un[j + n - 1]
        qhat, rhat = divmod(num, vn[n - 1])
        if qhat ≥ B:
            qhat = B - 1
            rhat = num - qhat * vn[n - 1]
        while qhat * vn[n - 2] > rhat * B + un[j + n - 2]:
            qhat -= 1
            rhat += vn[n - 1]
        borrow = 0
        carry = 0
        for i in range(n):
            carry, d = divmod(qhat * vn[i] + carry, B)
            t = un[j + i] - d - borrow
            if t < 0:
                t += B
                borrow = 1
            else:
                borrow = 0
            un[j + i] = t
        t = un[j + n] - carry - borrow
        if t < 0:
            t += B
            borrow = 1
        else:
            borrow = 0
        un[j + n] = t
        if borrow:
            ADDBACK[0] += 1
            qhat -= 1
            carry = 0
            for i in range(n):
                carry, un[j + i] = divmod(un[j + i] + vn[i] + carry, B)
            un[j + n] = (un[j + n] + carry) % B
        q[j] = qhat
    while q and q[-1] == 0:
        q.pop()
    r_mag, _exact = _div_one(un[:n], s)
    return q, r_mag
```

listing 736: python, the algorithm with the ADDBACK counter beside the branch, remainder descaled by dividing by s (source: icpc/samples-py/src/Ch14/ch14_divmod.py, lines 48-95)

```
  local q = {}
  for j = m, 0, -1 do
    local num = uu[j + n + 1] * BASE + uu[j + n]
    local qhat = num // vv[n]
    local rhat = num % vv[n]
    while qhat ≥ BASE or qhat * vv[n - 1] > rhat * BASE + uu[j + n - 1] do
      qhat = qhat - 1
      rhat = rhat + vv[n]
      if rhat ≥ BASE then break end
    end
    -- multiply and subtract over the window: t can dip below -BASE, so
    -- the limb comes from floored mod and the borrow from HD's
    -- p//B - t//B form (floor division), not a single conditional wrap
    local borrow = 0
    for i = 1, n do
      local p = qhat * vv[i] -- < 1e18, fits int64
      local t = uu[j + i] - borrow - (p % BASE)
      uu[j + i] = t % BASE
      borrow = p // BASE - t // BASE
    end
    local t = uu[j + n + 1] - borrow
    uu[j + n + 1] = t
    if t < 0 then
      -- qhat was one too large: add back once
      qhat = qhat - 1
      local carry2 = 0
      for i = 1, n do
        local w = uu[j + i] + vv[i] + carry2
        uu[j + i] = w % BASE
        carry2 = w // BASE
      end
      uu[j + n + 1] = (uu[j + n + 1] + carry2) % BASE
    end
    q[j + 1] = qhat
  end
```

listing 737: lua, the quotient loop with the Hacker's Delight floored-division borrow, add-back once on a negative top (source: icpc/samples-lua/ch14_divmod.lua, lines 66-100)

The anchors: `10^21 / 7` leaves quotient 142857142857142857142 and remainder 6, the sign matrix over `(±123456789012345678901234, ±98765)` against the truncating oracle, zero dividend, `|b| > |a|` giving q = 0 and r = a, one-limb divisors, power-of-base carries, and the invariants `q*b + r == a` and `0 ≤ |r| < |b|` asserted on every random ladder pair. The add-back branch is the part random testing never reaches, so every language pins a constructed pair that forces it: c uses `750000000000000000000000000 / 500000000000000000999999999` `= 14 r 499999999999999986000000014`, built analytically and cross-verified over 250000 random divisions against a python algorithm D simulation. C# scoured for one, 3.2M random and 209k structured divisions finding nothing, then found the family by exhaustive base-10 scan, `2*B^2 / (B^2+1)` and `3*B^2 / (B^2+1)`, the base-10 analog 200/101, and cobertura confirms the branch ran exactly twice. Go pins `(10^27 + 2*10^17) / (5*10^26 + 10^17 + 123456789)` `= 1 r` `500000000099999999876543211`, firing exactly once. Javascript lifts 1110/101 one digit per limb. Python's counter watches `150000000000000000000000000 / 50000000000000000001 = 2999999 r` `49999999999997000001` walk the branch, and java counts its branch on the same pair, the counter moving by exactly one. Lua derives its pair, `99999999899999999900000000001000000000 / (27 nines) = 999999998 r` `999999999000000001999999998`, from the D = 0 escape of its two-limb qhat test. Lua also owns the borrow lesson: a single conditional wrap in mulsub is wrong when the difference dips below -BASE, fixed with the Hacker's

Delight floored-division borrow, limb from `t % BASE`, borrow from `p//B - t//B`. Counts: C 180 checks, javascript 9 tests, python 233 checks, lua 8 named checks.

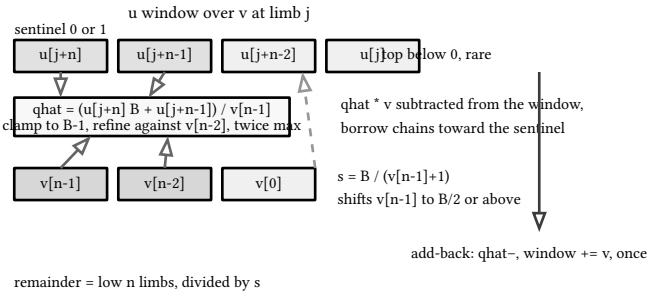


figure 199: one quotient digit: normalize, estimate two-over-one, refine, multiply-subtract down the window, add back on the rare negative top

16.6. measured: four multipliers, seven languages

Measured 2026-09-19 on this machine, one bench per language, medians over repeated runs on shared lcg operands, never inside `make verify`, java's bench joining 2026-10-06. Karatsuba crosses schoolbook at 16 limbs in C, 16 to 48 in python with the clear win from 48 and 2.2x at 233, 28 in javascript, about 48 in java with the clear win from 48, 48 to 64 in C# with the strict win at 64, 64 in go, first clear win 1.15x, 1.6x at 128 limbs, 1.9x at 233, and 89 to 144 in lua. The cut moved where the crossover said to: go swept 24/32/48/64 and retuned KARA_CUT from 24 to 48, fastest at 128 limbs and within noise of the best at 233, and lua swept 24/32/48/64/96 and retuned 24 to 64, PUC lua's table overhead punishing the recursion, cut 64 dominant at 613 against 781 microseconds over schoolbook at 144 limbs and 1375 against 1953 at
233. C held 24, its first karatsuba win at 16 limbs by a 10 percent

margin inside timer granularity, C# held 24 on its 48-to-64 crossover, java held 24 on its clear-from-48 crossover, javascript held 24, parity at 24 with the clear win from 28 at 0.183 against 0.247 milliseconds, a cold phantom at 24 that never reproduced warm, python held 24. The transforms split by runtime: fft split overtakes schoolbook at 13 limbs in C, about 55 in javascript, about 233 in python at 8.48 against 8.81 milliseconds, and about 444 limbs, 4k digits, in lua, while C#, go, and java keep the transforms behind schoolbook through the 233-limb ladder, java at 6 to 46x behind, narrowest at 233 limbs and widest at 55 on the quiet box, regroup and allocation overhead eating the asymptotics, so in go and java karatsuba is the multiplier at ICPC scales. Lua caps its ladders at 89 limbs for the quadratic checks and 233 for the transforms, and its digit-cap bench runs 1k, 4k, 16k digits:

digits	schoolbook	karatsuba	ntt	fft split
1000	0.43 ms	0.41 ms	1.22 ms	1.06 ms
4000	7.56 ms	3.94 ms	5.13 ms	3.50 ms
16000	113 ms	37 ms	23 ms	21.5 ms

language	KARA_CUT	crossover, limbs	why the cut landed there
c	24, held	16	first win 10 percent inside timer granularity, the pinned constant stays
go	24 to 48	64	cut swept, 48 fastest at 128 limbs, within noise at 233
java	24, held	about 48	clear win from 48 limbs, the c#/go class, transforms 6 to 46x behind schoolbook through the ladder, cut unchanged
c#	24, held	48 to 64	noise boundary 24 to 48, strict win at 64, cut unchanged
javascript	24, held	28	parity at 24, clear win from 28, cold phantom not reproducible warm
python	24, held	16 to 48	clear win from 48, 2.2x at 233, cut unchanged

language	KARA_CUT	crossover, limbs	why the cut landed there
lua	24 to 64	89 to 144	table overhead punishes recursion, 64 dominant across the scan

Each bench is a named invocation, none wired into verify: C runs the ladder behind the `CH14_BENCH` environment variable, printing `ok 1 bench gated` without it, C# gates one fact the same way, go exposes four `Benchmark` functions under `-bench`, javascript runs `node src/ch14-bench.mjs` from outside the test glob, python's `ch14_bench.py` unlocks the crossover scan and the ladder on `--bench`, java's `Bench.java` keeps the runner-invoked run bare at 7 checks with the crossover matrix and the four-multiplier ladder behind `--bench`, and lua's `ch14_bench.lua` is manual and unlisted, the `ch10_pf_runs` precedent.

```
public static void main(String[] args) {
  Bigint.Big u = Bigint.fromString(B34_U), v = Bigint.fromString(B34_V);
  Bigint.Big c = new Bigint.Big(Bigint.CAP);
  Bigint.mulSchool(u, v, c);
  ok(Bigint.str(c).equals(B34_P), "bench sanity schoolbook");
  Karatsuba.mulKara(u, v, c);
  ok(Bigint.str(c).equals(B34_P), "bench sanity karatsuba");
  ok(Ntt.mulNtt(u, v, c) == 0 && Bigint.str(c).equals(B34_P), "bench sanity ntt");
  ok(Fft.mulFft(u, v, c, true) == 0 && Bigint.str(c).equals(B34_P),
      "bench sanity fft split");
  for (int size = Karatsuba.CUT - 1; size ≤ Karatsuba.CUT + 1; size++) {
    Bigint.reseed();
    Bigint.Big a = Bigint.randBig(size), b = Bigint.randBig(size);
    Karatsuba.mulKara(a, b, c);
    Bigint.Big d = new Bigint.Big(Bigint.CAP);
    Bigint.mulSchool(a, b, d);
    ok(Bigint.cmp(c, d) == 0, "cut boundary " + size);
  }
  if (args.length > 0 && args[0].equals("--bench")) {
    crossoverMatrix();
    ladderMatrix();
  }
}
```

listing 738: java, the bare run a sanity chain over the pinned size-34 pair and the cut boundary, the crossover matrix and the four-multiplier ladder printing only behind the –bench flag (source: icpc/samples-java/src/Ch15/Bench.java, lines 116-138)

The dry run: no suite carries the bench, the gates keep it out of every verify run by design, so this walk reads the medians the lua digit-cap bench printed on 2026-09-19, every ratio below derived in one step.

1. At 1k digits, about 112 limbs, karatsuba already leads schoolbook 0.41 against 0.43 ms, ratio 0.43 / 0.41 = 1.05, inside the 89-to-144 limb crossover window quoted above.
2. The lead compounds: 7.56 / 3.94 = 1.92 at 4k digits and 113 / 37 = 3.05 at 16k.
3. Schoolbook's own climb from 1k to 16k, 16 times the digits, costs 113 / 0.43 = 263 against the quadratic's 16^2 = 256, 3 percent over prediction.
4. Karatsuba climbs 37 / 0.41 = 90 over the same span against the recurrence's 16^1.585 = 81, 11 percent over.
5. The transforms pass everything at 16k, ntt at 23 and fft split at 21.5 ms, factors 23 / 1.22 = 18.9 and 21.5 / 1.06 = 20.3 over their 1k selves, between linear 16 and the n log n envelope.

multiplier	1k	16k	factor	prediction
schoolbook	0.43 ms	113 ms	263	16^2 = 256, quadratic
karatsuba	0.41 ms	37 ms	90	16^1.585 = 81, the recurrence
ntt	1.22 ms	23 ms	18.9	n log n, above linear 16
fft split	1.06 ms	21.5 ms	20.3	n log n, above linear 16

Every factor lands within 11 percent of its asymptote, and the figure below charts the medians these ratios read.

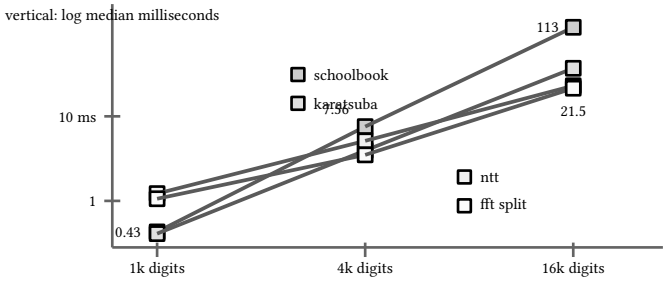


figure 200: lua medians, log milliseconds: karatsuba passes schoolbook between 1k and 4k digits, the transforms take over by 16k

16.7. across the seven languages

Build size counts the implementation files each stream reported:

language	build SLOC	dependency	boundary note
c	842	libc and libm only	static arrays, no malloc, `__uint128_t` banned and grepped every commit, all products under 2^60
go	798	stdlib, math/big oracle in tests	errors carry the refusals, transforms stay behind schoolbook through 233 limbs, karatsuba owns ICPC scales
java	1530	jdk 27 stdlib	nested Big core over int limbs, arena windows as array, offset, length triples, BigInteger never appears, the dsa chapter 29 law
c#	574	bcl, System.Numerics	sealed partial class split over five files, BigInteger oracle, one FftCap shared by both fft paths
javascript	604	node stdlib, BigInt	limbs as Number under 2^30, every exact limb product and every ntt butterfly on BigInt
python	958	stdlib only	native ints, the 4300-digit int-to-str guard lifted for the 300-limb oracle, truncating-sign oracle by hand
lua	911	lib.lua harness	integer tables, products under 2^63, worst-case all-9s probe validated the 2^14 cap with margin

sources: Knuth, The Art of Computer Programming volume 2, sections 4.3.1 multiple-precision arithmetic and 4.3.3, algorithm D and the fast multiplication survey, Karatsuba and Ofman, 1962, Cooley and Tukey, 1965, and CLRS 3rd edition chapter 30 for the polynomial multiplication framing. Native-library documentation: learn.microsoft.com for `System.Numerics.BigInteger` and `System.Numerics.Complex`, go.dev/pkg/math/big for the go oracle, developer.mozilla.org for BigInt arithmetic, and docs.python.org for the int type and three-argument `pow`, all accessed 2026-09-14. Benches run 2026-09-19, java's on 2026-10-06, medians quoted from the seven streams' reports. Suite counts as measured: C 1041 checks plus a 247-CHECK chain, 7 anchors and 240 ladder rows, C# 29 facts with the suite 278 of 278 green, go 1079 checks plus a 148-row chain, java 889 checks across the six files of `samples-java/src/Ch15`, Bigint 288, Karatsuba 77, Ntt 115, Divmod 232, Fft 170, and Bench 7 bare under `tools/run-java-samples.ps1`, javascript 39 tests carrying 5941 instrumented assertions, python 825 checks plus a 52-check chain, 4 anchors and 48 ladder equalities, lua 28 named checks across five modules plus a 23-assertion chain, the run suite 303 green from a 275 baseline at 0.42 seconds median. The java tree numbers its chapter dir post-renumber, Ch15 matching the book's chapter 16 while the six sibling trees keep their historical Ch14 dirs, and its file headers cross-cite the dsa chapter 29 bigint family, the teaches-the-mechanism law that keeps BigInteger out of every listing above.